基础教育系列

21世纪高校计算机应用技术系列规划教材

谭浩强　主编

微型计算机原理与接口技术

（第二版）

杨立 邓振杰 荆淑霞 等编著

37

中国铁道出版社
CHINA RAILWAY PUBLISHING HOUSE

基础教育系列

21世纪高校计算机应用技术系列规划教材
谭浩强 主编

微型计算机原理与接口技术（第二版）

杨立　邓振杰　荆淑霞　等编著

中国铁道出版社
CHINA RAILWAY PUBLISHING HOUSE

内 容 简 介

本书以目前流行的微型计算机为对象，将微型计算机的系统结构、典型微处理器、指令系统、汇编语言、总线技术、存储器系统、输入/输出接口技术、中断技术、通用可编程接口芯片、人机交互设备及接口、D/A 及 A/D 转换器等知识融为一体，内容的组合体现出结构化和模块化，合理地形成完整的课程教学体系，突出计算机应用的新知识和新技术。每章均给出导读和小结以及思考与练习题，为读者的学习提供帮助。

本书融入作者多年的教学和实践经验，内容由浅入深、循序渐进、重点突出、应用性强。从教学规律和人们的学习习惯出发，合理地编排教学内容，全面阐述微型计算机原理与接口技术中必须掌握的基本知识和基本技能，为今后的实际应用奠定坚实的基础。

本书可作为高等院校本科应用型专业以及高职高专相关专业学生的教材，也可作为成人教育、在职人员培训、高等教育自学人员和从事微型计算机硬件和软件开发的工程技术人员学习和应用的参考书。

图书在版编目（CIP）数据

微型计算机原理与接口技术/杨立等编著. —2 版. —北京：中国铁道出版社，2006.6
（21 世纪高校计算机应用技术系列规划教材）
ISBN 7-113-07051-5

Ⅰ.微… Ⅱ.杨… Ⅲ.①微型计算机－理论－高等学校－教材②微型计算机－接口－高等学校－教材
Ⅳ.TP36

中国版本图书馆 CIP 数据核字（2006）第 065913 号

书　　名：微型计算机原理与接口技术（第二版）
作　　者：杨　立　邓振杰　荆淑霞　等
出版发行：中国铁道出版社（100054，北京市宣武区右安门西街 8 号）
策划编辑：严晓舟　秦绪好
责任编辑：苏　茜　崔晓静　熊严飞
封面设计：薛　为
封面制作：白　雪
责任校对：刘　洁
印　　刷：北京新魏印刷厂
开　　本：787×1092　1/16　**印张**：22.25　**字数**：524 千
版　　本：2006 年 8 月第 2 版　2006 年 8 月第 1 次印刷
印　　数：1～5 000 册
书　　号：ISBN 7-113-07051-5/TP・1803
定　　价：29.00 元

版权所有　侵权必究

本书封面贴有中国铁道出版社激光防伪标签，无标签者不得销售

凡购买铁道版的图书，如有缺页、倒页、脱页者，请与本社计算机图书批销部调换。

更多资源请关注网易博客
http://shujihui.blog.163.com/

21世纪高校计算机应用技术系列规划教材

编委会

主　任：谭浩强

副主任：陈维兴　　严晓舟

委　员：（按姓氏字母先后为序）

安淑芝	安志远	陈志泊	韩　劼	侯冬梅
李　宁	李雁翎	林成春	刘宇君	秦建中
秦绪好	曲建民	尚晓航	邵丽萍	宋　红
宋金珂	王兴玲	魏善沛	熊伟建	薛淑斌
张　玲	赵乃真	訾秀玲		

更多资源请关注网易博客
http://shujihui.blog.163.com/

序

PREFACE

21 世纪是信息技术高度发展且得到广泛应用的时代，信息技术从多方面改变着人类的生活、工作和思维方式。每一个人都应当学习信息技术、应用信息技术。人们平常所说的计算机教育其内涵实际上已经发展为信息技术教育，内容主要包括计算机和网络的基本知识及应用。

对多数人来说，学习计算机的目的是为了利用这个现代化工具工作或处理面临的各种问题，使自己能够跟上时代前进的步伐，同时在学习的过程中努力培养自己的信息素养，使自己具有信息时代所要求的科学素质，站在信息技术发展和应用的前列，推动我国信息技术的发展。

学习计算机课程有两种不同的方法：一是从理论入手；一是从实际应用入手。不同的人有不同的学习内容和学习方法。大学生中的多数人将来是各行各业中的计算机应用人才。对他们来说，不仅需要解决知道什么，更重要的是会做什么。因此，在学习过程中要以应用为目的，注重培养应用能力，大力加强实践环节，激励创新意识。

根据实际教学的需要，我们组织编写了这套“21 世纪高校计算机应用技术系列规划教材”。顾名思义，这套教材的特点是突出应用技术，面向实际应用。在选材上，根据实际应用的需要决定内容的取舍，坚决舍弃那些现在用不到、将来也用不到的内容。在叙述方法上，采取 “提出问题——介绍解决问题的方法——归纳一般规律和概念”的三部曲，这种从实际到理论、从具体到抽象、从个别到一般的方法，符合人们的认知规律，且在实践过程中已取得了很好的效果。

本套教材采取模块化的结构，根据需要确定一批书目，提供了一个课程菜单供各校选用，以后可根据信息技术的发展和教学的需要，不断地补充和调整。我们的指导思想是面向实际、面向应用、面向对象。只有这样，才能比较灵活地满足不同学校、不同专业的需要。在此，希望各校的老师把你们的要求反映给我们，我们将会尽最大努力满足大家的要求。

本套教材可以作为大学计算机应用技术课程的教材以及高职高专、成人高校和面向社会的培训班的教材，也可作为学习计算机的自学教材。

本套教材自 2003 年出版以来，已出版了 60 多种，受到了许多高校师生的欢迎。

由于全国各地区、各高等院校的情况不同，因此需要有不同特点的教材以满足不同学校、不同专业教学的需要，尤其是高职高专教育发展迅速，不能照搬普通高校的教材和教学方法，必须要针对它们的特点组织教材和教学，因此我们在原有基础上，对这套教材做了进一步的规划。本套教材包括以下两个系列：第一系列是面对应用型高校的教材，对象是普通高校的应用性专业；第二系列是面向高职高专的教材，对象是两年制或三年制的高职高专院校的学生，突出实用技术和应用技能，不涉及过多的理论和概念，强调实践环节，学以致用。

本套教材由中国铁道出版社与浩强创作室共同策划，由全国一些普通高等学校和高职高专院校的老师编写，对于他们的智慧、奉献和劳动表示深切的谢意。中国铁道出版社以很高

更多资源请关注网易博客
http://shujihui.blog.163.com/

的热情和效率组织了这套教材的出版工作，在组织编写出版的过程中，得到全国高等院校计算机基础教育研究会和各高等院校老师的热情鼓励和支持，对此谨表衷心的感谢。

本套教材如有不足之处，请各位专家、老师和广大读者不吝指正。希望通过本套教材的不断完善和出版，为我国计算机教育事业的发展和人才培养做出更大贡献。

全国高等院校计算机基础教育研究会会长
“21 世纪高校计算机应用技术系列规划教材”丛书主编

谭浩强

更多资源请关注网易博客
http://shujihui.blog.163.com/

第一版前言

FOREWORD

我国高等教育正在快速发展，教材建设也必须与之相适应，尤其是教育部关于“高等教育面向21世纪内容与课程改革”计划的实施，对教材建设提出了新的要求。微型计算机技术随着其软、硬件的不断升级换代，教学内容也在不断更新，要求不断推出适应课程教学特点和满足不同层次学生学习的新型教材。本书的编写目的就是为了适应高等教育的快速发展，满足教学改革和课程建设的需求，体现应用型技术和高职高专教育的特点。

《微型计算机原理与接口技术》是学习微型计算机基本知识和应用技能的重要课程。本课程帮助学生掌握微型计算机的硬件组成及使用；学会运用指令系统和汇编语言进行程序设计；熟悉各种类型的接口及其应用，树立起微型计算机体系结构的基本概念，为后继计算机课程的学习及应用打好基础。对于应用型技术和高职高专教育来讲，要打破以学科为特征的传统教学方法，注重面向应用型人才的专业技能和实用技术的培养。基于这种指导思想，本书在编写过程中力争做到：相关概念、理论及应用均以基本要求为主，突出实用的特点；在表达上以实例引出概念、提出问题，然后通过阐述与分析，进行归纳总结，做到层次清晰，脉络分明；在内容编排上，以8086微型计算机为对象，多讲实例，多介绍和现代微型计算机密切相关的技术，力求循序渐进，举一反三，突出重点，通俗易懂。

本教材的教学参考学时为80～90学时（含实训），并可按照实际情况进行调整。

全书共计10章：

第1章概述微型计算机；

第2章介绍典型微处理器；

第3章介绍指令系统与汇编语言；

第4章介绍微型计算机的总线技术；

第5章介绍存储器系统；

第6章介绍输入/输出接口技术；

第7章介绍中断技术；

第8章介绍通用可编程接口芯片；

第9章介绍人机交互设备及接口；

第10章介绍D/A及A/D转换器。

本书由杨立担任主编，邓振杰、荆淑霞任副主编。各章编写的分工如下：第1章由赵丑民编写；第2～4章及附录由杨立编写；第5章和第7章由邓振杰编写；第8章和第9章由荆淑霞编写；第6章由金永涛编写；第10章由王喜斌编写；宋存米、邹澎涛、邵温、王振夺也参加了部分内容的编写和绘图工作，全书由杨立负责统稿，陈兰芳、崔仙翠、程瑞芬等参与了本书的编排工作。

由于时间仓促，水平有限，书中不足之处在所难免，敬请读者批评指正，我们也会在适当时间进行修订和补充，并发布在天勤网站：http://www.tqbooks.net“图书修订”栏目中。

编 者

2003年8月

更多资源请关注网易博客
http://shujihui.blog.163.com/

第二版前言

FOREWORD

《微型计算机原理与接口技术（第一版）》自2003年9月出版以来，得到了广大读者的欢迎和好评。该教材体现出注重对应用型人才的专业技能和实用技术的培养，融合了微型计算机的硬件和软件知识，树立了微型计算机体系结构的基本概念，通过各种类型的接口及其应用的学习，培养了学生运用微型计算机进行相关开发和设计的能力，为后继计算机课程的学习及应用打下了坚实的基础。教材内容层次清晰、脉络分明；阐述问题由浅入深、循序渐进；各章知识重点突出、通俗易懂，为高等院校计算机应用技术类专业的学生学习该课程切实起到了积极的推动作用并取得较好的效果。

随着计算机技术的不断发展，新知识也在不断涌现。微型计算机原理与接口技术课程根据系统软、硬件的不断升级换代，教学内容也在不断更新，要求不断推出适应课程教学特点和满足不同层次学生学习的新型教材。本书再版就是为了达到教育部对计算机基础教学的基本要求，反映出教学内容和课程体系的改革成果，适应课程建设的需求，体现出应用技术类专业的教学特点。

本版教材保留了第一版的组织结构，在此基础上修改和扩充了部分章节，删去了一些比较浅显和累赘的内容，补充了部分应用实例，同时引入了一些实用知识。例如，增加了Pentium微处理器的内部结构描述、采用的新技术、指令系统概述等；将原书的第3章分解为“寻址方式与指令系统”、“汇编语言”两章，并扩充了“汇编语言”的内容；将原书的第6章分解为“输入/输出接口技术”、“可编程DMA控制器8237A”两章；将原书的第8章分解为“可编程并行接口芯片8255A”、“可编程串行接口芯片8251A”和“可编程定时器/计数器接口芯片8253”等3章，采用比较实际的例子对相关内容进行了扩充。这样处理以后，使教材的各章节内容既相对独立又相互衔接，形成层次化和模块化的知识体系，便于教学的取舍。全书注重将知识性、先进性和系统性融为一体，通过大量的实例来阐述各章的知识点，保证学习的兴趣和效果。

本教材的教学参考学时为80～90学时（包括实训），各校可按照实际教学情况进行教学内容上的调整。全书共计14章：

第1章概述了微型计算机的基础知识。

第2章介绍了典型微处理器的内、外部结构和功能特点。

第3章分析了寻址方式与指令系统。

第4章阐述了汇编语言及其程序设计。

第5章介绍了微型计算机的总线技术。

第6章介绍了存储器系统。

第7章讲解了输入/输出接口技术。

第8章分析了可编程DMA控制器8237A及其应用。

第9章介绍了中断技术。

第10章讨论了可编程并行接口芯片8255A及其应用。

更多资源请关注网易博客
http://shujihui.blog.163.com/

第 11 章讨论了串行通信与可编程串行接口芯片 8251A 及其应用。

第 12 章讨论了可编程定时器/计数器接口芯片 8253 及其应用。

第 13 章介绍了常用的人机交互设备及接口。

第 14 章介绍了 D/A 及 A/D 转换器的有关知识及其应用。

本书由杨立担任主编，邓振杰、荆淑霞任副主编。各章编写分工如下：杨立负责编写第 1～5 章及附录；邓振杰负责编写第 6～9 章；荆淑霞负责编写第 10～13 章；蒋天伟负责编写第 14 章。参加本书大纲讨论和部分内容编写工作的还有：曲凤娟、金永涛、李京辉、马玉戈、王静、邵温、邹澎涛、朱蓬华等。全书由杨立负责组织、审校与定稿。

由于编者水平有限，书中不足之处在所难免，敬请广大读者批评指正。

编　者

2006 年 6 月

更多资源请关注网易博客
http://shujihui.blog.163.com/

目录

CONTENTS

第 1 章　微型计算机概述 1
1.1　微型计算机的发展与应用 1
1.1.1　微处理器的产生和发展 1
1.1.2　微型计算机的分类 3
1.1.3　微型计算机的特点与性能指标 4
1.1.4　微型计算机的应用 5
1.2　微型计算机的硬件结构及系统组成 7
1.2.1　微型计算机硬件的基本结构 8
1.2.2　微型计算机的系统组成 11
1.3　计算机中的信息表示 12
1.3.1　计算机中的数制及其转换 12
1.3.2　计算机中数值数据的表示 15
1.3.3　计算机中常用的编码 18
本章小结 20
思考与练习题 21
第 2 章　典型微处理器 22
2.1　微处理器性能简介 22
2.1.1　典型微处理器的主要性能指标 22
2.1.2　微处理器的基本功能 23
2.2　Intel 8086 微处理器的内、外部结构特性 23
2.2.1　Intel 8086 微处理器内部组成结构 23
2.2.2　8086 微处理器的寄存器结构 26
2.2.3　8086 微处理器的外部引脚特性 29
2.3　存储器和 I/O 组织 31
2.3.1　存储器组织 31
2.3.2　I/O 端口组织 35
2.4　8086 微处理器的总线周期和工作方式 36
2.4.1　8284A 时钟信号发生器 36
2.4.2　8086 总线周期 36
2.4.3　8086 微处理器的最小/最大工作方式 38
2.5　32 位微处理器简介 40
2.5.1　80386 微处理器 40
2.5.2　80486 微处理器 41
2.5.3　Pentium 系列微处理器 43

更多资源请关注网易博客
http://shujihui.blog.163.com/

本章小结 48
思考与练习题 48
第 3 章 寻址方式与指令系统 50
3.1 指令格式及寻址 50
3.1.1 指令系统与指令格式 50
3.1.2 寻址及寻址方式的概念 51
3.2 8086 指令系统的寻址方式 51
3.2.1 与操作数有关的寻址方式 51
3.2.2 与 I/O 端口有关的寻址方式 54
3.3 8086 指令系统 55
3.3.1 数据传送类指令 55
3.3.2 算术运算类指令 59
3.3.3 逻辑运算与移位类指令 62
3.3.4 串操作类指令 64
3.3.5 控制转移类指令 67
3.3.6 处理器控制类指令 71
3.4 DOS 和 BIOS 中断调用 72
3.4.1 DOS 功能调用 72
3.4.2 BIOS 中断调用 73
3.5 Pentium 微处理器新增寻址方式和指令 73
3.5.1 Pentium 微处理器的内部寄存器 73
3.5.2 Pentium 微处理器的新增寻址方式 74
3.5.3 Pentium 系列微处理器专用指令 75
3.5.4 Pentium 系列微处理器控制指令 76
本章小结 76
思考与练习题 77
第 4 章 汇编语言 79
4.1 汇编语言简述 79
4.1.1 汇编语言语句格式 79
4.1.2 汇编语言程序结构 83
4.1.3 汇编语言常用伪指令 84
4.1.4 汇编语言程序上机过程 91
4.2 汇编语言程序设计 92
4.2.1 程序设计的基本步骤及程序基本结构 92
4.2.2 顺序结构程序设计 93
4.2.3 分支结构程序设计 95
4.2.4 循环结构程序设计 98
4.2.5 子程序设计 101

更多资源请关注网易博客
http://shujihui.blog.163.com/

本章小结 102
思考与练习题 103
第 5 章 微型计算机的总线技术 104
5.1 总线技术概述 104
5.2 系统总线 105
5.2.1 STD 总线 106
5.2.2 ISA 总线 110
5.2.3 EISA 总线 116
5.3 局部总线 117
5.3.1 VESA 总线 117
5.3.2 PCI 总线 119
5.3.3 AGP 总线 121
5.4 其他总线介绍 122
5.4.1 USB 总线 122
5.4.2 IEEE1394 125
5.4.3 I^2C 总线 128
本章小结 130
思考与练习题 130
第 6 章 存储器系统 132
6.1 存储器概述 132
6.1.1 存储器的分类 132
6.1.2 存储器的性能指标 133
6.1.3 存储系统的层次结构 134
6.2 半导体存储器 135
6.2.1 半导体存储器的基本结构 136
6.2.2 随机存取存储器（RAM） 137
6.2.3 动态 RAM（DRAM） 140
6.3 只读存储器（ROM） 142
6.3.1 掩膜 ROM 142
6.3.2 可编程 PROM 143
6.3.3 光可擦除 EPROM 143
6.3.4 电可擦除 E^2PROM 144
6.3.5 闪速存储器 144
6.4 存储器的扩展与寻址 145
6.4.1 位扩展 145
6.4.2 字位扩展 146
6.4.3 存储器的寻址 146
6.5 存储器与微处理器的连接 147

更多资源请关注网易博客
http://shujihui.blog.163.com/

6.5.1 连接时应注意的问题 147
6.5.2 典型微处理器与存储器的连接 148
6.6 辅助存储器 149
6.6.1 软盘存储器及其接口 150
6.6.2 硬盘存储器及其接口 152
6.6.3 光盘存储器及其接口 156
6.7 新型存储器技术 158
6.7.1 多体交叉存储器 158
6.7.2 高速缓冲存储器（Cache） 159
6.7.3 虚拟存储器 162
本章小结 164
思考与练习题 164
第 7 章 输入/输出接口技术 166
7.1 输入/输出接口技术概述 166
7.1.1 输入/输出接口的结构与功能 166
7.1.2 CPU 与 I/O 接口之间传递的信息类型 168
7.1.3 I/O 端口的编址方式 169
7.2 I/O 数据传送方式 170
7.2.1 无条件传送方式 170
7.2.2 查询传送方式 172
7.2.3 中断传送方式 174
7.2.4 DMA 传送方式 175
本章小结 176
思考与练习题 177
第 8 章 可编程 DMA 控制器 8237A 178
8.1 可编程 DMA 控制器 8237A 概述 178
8.2 8237A 的内部结构及引脚 179
8.2.1 8237A 的内部结构 179
8.2.2 8237A 的引脚 180
8.3 8237A 工作方式 182
8.3.1 单字节传送方式 182
8.3.2 块传送方式 182
8.3.3 请求传送方式 183
8.3.4 级联传送方式 183
8.4 8237A 内部寄存器功能及格式 184
8.4.1 当前地址寄存器 184
8.4.2 当前字节计数寄存器 184
8.4.3 基地址寄存器 184

更多资源请关注网易博客
http://shujihui.blog.163.com/

8.4.4 基字节数寄存器 185
8.4.5 命令寄存器 185
8.4.6 工作方式寄存器 186
8.4.7 请求寄存器 187
8.4.8 屏蔽寄存器 188
8.4.9 状态寄存器 189
8.4.10 暂存寄存器 189
8.4.11 软件命令 189
8.5 8237A 的编程及应用 190
本章小结 193
思考与练习题 193
第 9 章 中断技术 194
9.1 中断技术概述 194
9.1.1 中断技术的优点 195
9.1.2 中断源 195
9.1.3 中断处理过程 196
9.1.4 中断优先权 199
9.2 8086 的中断结构 201
9.2.1 中断类型 201
9.2.2 中断向量表 203
9.3 可编程中断控制器 8259A 及其应用 205
9.3.1 8259A 的内部结构及引脚 205
9.3.2 8259A 的中断管理方式 208
9.3.3 8259A 的编程 211
本章小结 218
思考与练习题 218
第 10 章 可编程并行接口芯片 8255A 220
10.1 并行接口概述 220
10.1.1 并行接口的分类 220
10.1.2 并行接口的特点 221
10.2 并行接口芯片 8255A 221
10.2.1 内部结构及引脚 221
10.2.2 8255A 的工作模式 224
10.2.3 8255A 的编程及应用 230
本章小结 235
思考与练习题 235
第 11 章 可编程串行接口芯片 8251A 236
11.1 串行传输的基本概念 236

更多资源请关注网易博客
http://shujihui.blog.163.com/

11.1.1 串行通信的基本概念 236
11.1.2 信号的调制与解调 238
11.2 可编程串行接口芯片 8251A 239
11.2.1 8251A 的基本性能 239
11.2.2 8251A 的基本结构 239
11.2.3 8251A 的编程控制 243
11.2.4 8251A 的初始化和编程应用 244
11.3 PC 串行异步通信接口 248
11.3.1 串行异步通信的基本概念 248
11.3.2 BIOS 串行异步通信接口的功能调用 250
本章小结 253
思考与练习题 253
第 12 章 可编程定时器/计数器接口芯片 8253 255
12.1 定时器/计数器的工作原理 255
12.1.1 微机系统中的定时 255
12.1.2 外部定时法 255
12.1.3 可编程定时器/计数器的工作原理 256
12.2 可编程定时器/计数器 8253 256
12.2.1 8253 的特点 256
12.2.2 8253 内部结构和引脚功能 256
12.2.3 8253 的初始化 259
12.2.4 8253 的工作方式 261
12.2.5 8253 的应用 268
本章小结 270
思考与练习题 271
第 13 章 人机交互设备及接口 272
13.1 人机交互设备概述 272
13.2 键盘与鼠标 272
13.2.1 键盘及接口电路 272
13.2.2 PC 键盘接口及其应用 276
13.2.3 鼠标及接口电路 279
13.3 显示器及接口 280
13.3.1 CRT 显示器 281
13.3.2 CRT 显示器接口电路编程方法 286
13.3.3 LED 与 LCD 显示 290
13.4 打印机接口 294
13.4.1 常用打印机及工作原理 294
13.4.2 主机与打印机的接口 295

更多资源请关注网易博客
http://shujihui.blog.163.com/

13.4.3 打印机的中断调用 296
13.5 其他外设简介 297
13.5.1 扫描仪原理及其应用 297
13.5.2 数码相机原理及其应用 298
13.5.3 触摸屏原理及其应用 299
本章小结 300
思考与练习题 300
第 14 章 D/A 及 A/D 转换器 301
14.1 D/A 及 A/D 转换器概述 301
14.2 典型 D/A 转换器芯片 302
14.2.1 D/A 转换器工作原理 302
14.2.2 D/A 转换器的主要参数 303
14.2.3 8 位 D/A 转换器 DAC0832 及应用 304
14.3 典型 A/D 转换器芯片 310
14.3.1 A/D 转换器工作原理 310
14.3.2 ADC 的性能参数 311
14.3.3 典型 A/D 转换器件 ADC0809 311
14.3.4 A/D 转换器的选择原则 318
14.4 DAC 及 ADC 应用实例 318
本章小结 320
思考与练习题 320
附录 A 8086 指令系统 321
附录 B DEBUG 调试程序的应用 325
附录 C DOS 系统功能调用（INT 21H） 326
附录 D BIOS 功能调用 333
参考文献 336

每日更新 收藏必备http://wangchuang.32666.com

第 1 章 微型计算机概述

本章阐述了微处理器的产生和发展过程；分析了微型计算机的特点、性能指标以及分类和应用；介绍了微型计算机的基本结构、工作原理以及系统组成；讨论了计算机中常用的数制及其转换、带符号数的表示、字符编码和汉字编码等基本知识。

通过本章的学习，要求读者能够理解和掌握微型计算机的发展过程和前景；熟悉微型计算机的工作特点和系统组成；掌握计算机中数制的表示和相互转换、无符号数和带符号数的表示、字符编码与汉字编码等信息的表示以及应用等相关知识，为后续内容的学习打下良好的基础。

1.1 微型计算机的发展与应用

1946 年 2 月，世界上第一台电子数字计算机 ENIAC（Electronic Numerical Integrator and Calculator，电子数字积分计算机）在美国宾夕法尼亚大学研制成功，从此，计算机的发展随着其主要电子部件的演变经历了电子管、晶体管、中小规模集成电路、大规模集成电路和超大规模集成电路等 5 个时代。

在 50 多年的发展历程中，计算机技术突飞猛进，特别是进入 20 世纪 70 年代以后，微型计算机的出现更为计算机的广泛应用开拓了更加广阔的前景。其存储容量、运算速度、可靠性、性能价格比等都有很大的突破，同时推出了各种系统软件和应用软件，使得其功能不断增强。

目前，微型计算机已经渗透到国民经济的各个领域，极大地改变了人们的工作、学习及生活方式，成为信息时代的主要标志。

本节重点介绍微处理器的产生和发展以及微型计算机的分类和应用。

1.1.1 微处理器的产生和发展

微处理器诞生于 20 世纪 70 年代初，它是将传统计算机的运算器和控制器等部件集成在一块大规模集成电路芯片上作为中央处理部件，简称为微处理器（Microprocessor）。微型计算机就是以微处理器为核心，再配上存储器、接口电路等芯片构成的。微型计算机以其体积小、重量轻、价格低廉、可靠性高、结构灵活、适应性强和应用面广等一系列优点，占领了世界计算机市场并得到广泛的应用，成为现代社会中不可缺少的重要工具。

自从微处理器和微型计算机问世以来，短短的 30 多年时间里，几乎每两年时间集成度就提高 1 倍，每 3~5 年就更新换代一次。按照微型计算机的 CPU 字长和功能划分，它经历了以下 6 代的演变。

1. 第一代：4 位和 8 位低档微处理器

美国 Intel 公司在 1971 年开发出全球第一块字长为 4 位的微处理器芯片 4004，它集成了 2300 多个晶体管，时钟频率为 108kHz，每秒可进行 6 万次运算，寻址空间只有 640 B（字节），

每日更新 收藏必备http://wangchuang.32666.com

指令系统比较简单，价格较低廉，由它组成的 MCS-4 计算机是世界上第一台微型计算机。

随后 Intel 公司研制出字长 8 位的 8008 微处理器，它采用 PMOS 工艺，基本指令 48 条，时钟频率为 500kHz，集成度为 3 500 晶体管/片，以它为核心组成了 MCS-8 微型计算机。

这一阶段的微型计算机主要用于处理算术运算、家用电器以及简单的控制等。

2. 第二代：8 位中高档微处理器

1974 年，Intel 公司推出了新一代 8 位微处理器芯片 8080，它采用 NMOS 工艺，集成了 6 000 个晶体管，时钟频率为 2MHz，其指令系统比较完善，寻址能力有所增强，运算速度提高了一个数量级。

这一阶段的微型计算机主要用于教学和实验、工业控制、智能仪器等。

3. 第三代：16 位微处理器

1978 年，Intel 公司于推出了 16 位的微处理器芯片 8086，采用 HMOS 工艺，内部集成了 29 000 个晶体管，时钟频率达 5MHz/8MHz/10MHz，寻址空间达到 1MB。其间，Intel 公司又推出了 8086 的一个简化版本 8088，其时钟频率为 4.77MHz，将 8 位数据总线独立出来，减少了管脚，成本也较低。

1979 年，IBM 公司采用 Intel 的 8086 与 8088 作为个人计算机（IBM PC）的 CPU，PC（个人计算机）时代从此诞生。

1982 年 2 月，Intel 公司推出了超级 16 位微处理器芯片 80286，其集成度达到 13.4 万晶体管/片，时钟频率达 20MHz，各方面的性能有了很大的提高，它的 24 位地址总线可以寻址 16MB 地址空间，还可以访问 1GB 的虚拟地址空间，能够实现多任务并行处理。

这一阶段的微型计算机在数值计算、数据处理、信息管理、过程控制和智能化仪表等诸多方面都得到了广泛的应用。

4. 第四代：32 位微处理器

1985 年 10 月，Intel 公司推出了 32 位的微处理器芯片 80386，它集成了 27.5 万个晶体管，时钟频率达到 33MHz，数据总线和地址总线均为 32 位，具有 4GB 的物理寻址能力。由于在芯片内部集成了分段存储管理部件和分页存储管理部件，它能管理高达 64TB 的虚拟存储空间。

1989 年 4 月，Intel 公司推出了 80486 微处理器，其芯片内集成了 120 万个晶体管，它不仅把浮点运算部件集成进芯片内，同时还把一个规模大小为 8KB 的一级高速缓冲存储器 Cache 也集成进了 CPU 芯片。同时，80486 微处理器的兼容性也得到了更大的提高。

5. 第五代：超级 32 位 Pentium 微处理器

1993 年 3 月，Intel 公司推出 Pentium 微处理器芯片（俗称 586）。其内部集成了 310 万个晶体管，采用了全新的体系结构，性能大大高于 Intel 系列其他微处理器。Pentium 系列 CPU 的主频从 60MHz 到 100MHz 不等，它支持多用户、多任务，具有硬件保护功能，支持构成多处理器系统。

1996 年，Intel 公司推出了高能奔腾（Pentium Pro）微处理器，它集成了 550 万个晶体管，内部时钟频率为 133MHz，采用了独立总线和动态执行技术，处理速度大大提高。

1996 年底，Intel 公司又推出了多能奔腾（Pentium MMX）微处理器，MMX（Multi Media eXtension）技术是 Intel 公司最新发明的一项多媒体增强指令集技术，它为 CPU 增加了 57 条

每日更新 收藏必备http://wangchuang.32666.com

MMX 指令，此外，还将 CPU 芯片内的高速缓冲存储器 Cache 由原来的 16KB 增加到 32KB，使处理多媒体的能力大大提高。

1997 年 5 月，Intel 公司推出了 Pentium II 微处理器，它集成了 750 万个晶体管，8 个 64 位的 MMX 寄存器，时钟频率达 450MHz，二级高速缓冲存储器 Cache 达到 512KB，它的浮点运算性能、MMX 性能都是最出色的。

1999 年 2 月，Intel 公司推出了 Pentium III微处理器。它集成了 950 万个晶体管，时钟频率为 500 MHz。随后，又推出了新一代高性能 32 位 Pentium 4 微处理器，它采用了 NetBurst 的新式处理器结构，可以更好地处理互联网用户的需求，在数据加密、视频压缩和对等网络等方面的性能都有较大幅度的提高。Pentium 4 微处理器为因特网、图形处理、数据流视频、语音、3D 和多媒体等多种应用模式提供了强大的功能。

6. 第六代：新一代 64 位微处理器 Merced

在不断完善 Pentium 系列处理器的同时，Intel 公司与 HP 公司联手开发了更先进的 64 位微处理器——Merced。

Merced 采用全新的结构设计，这种结构称为 IA-64（Intel Architecture-64），IA-64 不是原来 Intel 32 位 X86 结构的 64 位扩展，也不是 HP 公司的 64 位 PA-RISC 结构的改造。IA-64 是一种采用长指令字（LIW）、指令预测、分支消除、推理装入和其他一些先进技术从程序代码提取更多并行性的全新结构。

随着微处理器的不断升级，微型计算机也在不断发展，其功能不断完善，应用领域扩展到了国民经济和人们生活中的各个方面。

总体来看，未来计算机的发展趋势有如下几个方面。

（1）未来的计算机朝着微型计算机和巨型计算机两级方向发展。微型计算机的发展反映了计算机的应用普及程度，巨型计算机的发展则代表了计算机科学的发展水平。

（2）未来的计算机开发和研究的热点是多媒体计算机。研究的关键技术是处理视频和音频数据的压缩、解压缩、多媒体数据的通信以及各种接口的实现方案等。

（3）未来计算机发展的总趋势是智能化计算机。它突出人工智能方法和技术的应用，具有在某种程度上模仿人的推理、联想、学习等思维功能，并具有声音识别和图像识别等能力。

1.1.2　微型计算机的分类

通常情况下，微型计算机可以按照 CPU 的字长、使用形态等划分类别。

1. 按照 CPU 的字长来分类

微型计算机的性能主要取决于微处理器，按照微处理器能够处理的数据字长作为分类标准，有以下几种类别。

（1）4 位微型计算机：CPU 的字长为 4 位，系统传送的数据位为 4 位。

（2）8 位微型计算机：CPU 的字长为 8 位，系统并行传送的数据位为 8 位。在计算机中，通常将 8 位二进制数称为一个字节（Byte）。

（3）16 位微型计算机：采用高性能的 16 位微处理器作为其 CPU，系统并行传送的数据位数为 16 位。

每日更新 收藏必备http://wangchuang.32666.com

（4）32 位微型计算机：采用 32 位微处理器组成，这是当前使用最多的微型计算机，系统并行传送的数据位可以达到 32 位。

（5）64 位微型计算机：采用 64 位微处理器组成，这是当前性能最优的微型计算机，系统采用了被称为“显示并行指令计算”的指令架构。

2. 按照微型计算机的利用形态来分类

（1）单片微型计算机：这是在一个芯片上包括 CPU、RAM、ROM 及 I/O 接口电路的完整计算机功能的电路。由于集成度的关系，其存储容量有限，I/O 电路也不多，所以通常用于一些专用的小系统中。

（2）单板微型计算机：这是一种将微处理器和一定容量的存储器芯片及 I/O 接口电路等大规模集成电路组装在一块印刷电路板上的微型计算机。通常，在这块板上还包含固化在 ROM 中的容量不大的监控程序，以及配置一些典型的外部设备。

（3）位片式微型计算机：这是采用多片双极型位片组合而成的 CPU，处理速度较高。由于双极型工艺集成度较低，功耗较大，因此在一个单片上的位数不可能做得很多。位片式微处理器以位为单位构成 CPU 芯片，常用多片位片式微处理器构成高速、分布式系统和阵列式系统。

（4）微型计算机系统：这是将包含 CPU、RAM、ROM 和 I/O 接口电路的主板及其他若干块印刷板电路，如存储器扩展板、外设接口板、电源等组装在一个机箱内，构成一个完整的、功能更强的计算机装置，称为微型计算机系统。在这种系统中，通常还配有磁盘、光盘等外部存储器，配有键盘、屏幕显示器等人机对话工具和打印机、扫描仪等外部设备，并且有丰富的软件支持。

1.1.3 微型计算机的特点与性能指标

1. 微型计算机的特点

微型计算机采用了许多先进的加工工艺和制造技术，其硬件和软件的有机组合，显示出许多突出优点，使得微型计算机从问世以来就得到了极其迅速的发展和广泛的应用，其特点可以概括如下。

（1）功能强：微型计算机运算速度快、计算精度高，而且都配有一整套支持其工作的软件，使得微型计算机的处理功能大大增强，满足了各行各业的实际应用。

（2）可靠性高：微处理器及其配套系列芯片上可以集成上百万个元件，减少了大量的焊点、连线、接插件等不可靠因素，使其可靠性大大增加。

（3）价格低：由于微处理器及其配套系列芯片集成度高，适合大批量生产，因此产品成本低，低廉的价格对于微型计算机的推广和普及是十分有利的。

（4）适应性强：由于硬件扩展很方便，在相同的配置情况下，只要对硬件和软件作某些变动就可以适应不同用户对微型计算机系统的要求。而且，微处理器的制造厂家除生产微处理器芯片外，还生产各种与之相配套的支持芯片和提供许多相关的支持软件，为计算机用户根据实际需求组成微型计算机的应用系统创造了十分有利的条件。

（5）体积小、重量轻：通常微处理器及其配套的支持芯片尺寸都比较小，尤其近几年在微型计算机中还大量采用了大规模集成专用芯片（ASIC）和通用可编程门阵列（GAL）器

每日更新 收藏必备http://wangchuang.32666.com

件，使得微型计算机的体积明显缩小，而且重量减轻。随着超大规模集成电路技术的不断发展，今后推出的微处理器集成度将更高，体积将更小，功能将更强。

（6）维护方便：微型计算机已逐渐趋于标准化、模块化和系列化，从硬件结构到软件配置都作了比较全面的考虑，采用自检、诊断及测试等技术可以及时发现系统故障。发现故障以后排除故障也比较容易，例如可以迅速地更换标准化模块或芯片等。

2. 微型计算机的性能指标

为了衡量微型计算机的性能好坏，可以采用以下的指标来描述。

（1）位（bit）：在计算机中是指一个二进制位，由“0”和“1”两种状态构成，若干个二进制位的组合可以表示出计算机中的各种信息。

（2）字长：是指微处理器内部寄存器、运算器、内部数据总线等部件之间传输数据的宽度或位数，它是微处理器数据处理能力的重要指标。字长（二进制数的位数）应该是字节的整数倍，如 16 位、32 位、64 位等，字长越长，精度越高，主存容量也越大。

（3）字节（Byte）：这是计算机中通用的基本存储和处理单元，它由 8 个二进制位组成。

（4）字：这是计算机内部进行数据处理的常用单位。16 位二进制为一个字，即由两个字节组成一个字。如果是 32 位的微型计算机则由 4 个字节组成一个双字。

（5）主频：也称时钟频率，单位为 MHz（兆赫），它决定了微型计算机的处理速度。Pentium 系列微型计算机的主频可达到上千兆赫。

（6）主存容量：指主存储器中 RAM 和 ROM 的总和，是衡量微型计算机处理数据能力的一个重要指标。

（7）可靠性：指计算机在规定的时间和工作条件下正常工作不发生故障的概率。故障率越低说明可靠性越高。

（8）兼容性：指计算机的硬件和软件可用于其他多种系统的性能。主要体现在数据处理、I/O 接口、指令系统等的可兼容性。

（9）性能价格比：是衡量计算机产品优劣的综合性指标，包括计算机的硬软件性能与售价的关系，通常希望以最小的成本获取最大的效益。

1.1.4　微型计算机的应用

由于微型计算机具有价格低廉、体积小、重量轻、功耗低、可靠性高、使用灵活等优点，所以微型计算机的应用日益深入到各行各业。从仪器仪表和家电的智能化，再到科学计算、自动控制、办公自动化、生产自动化、数据和事务处理、计算机辅助设计、数据库应用、计算机网络应用、人工智能、计算机模拟、计算机辅助教育等各个领域均得到了广泛的应用。微型计算机在当今信息社会已经成为不可缺少的重要工具。

概括起来，微型计算机的应用可以归纳为以下几个方面。

1. 数值计算

数值计算是指用计算机来解决科学研究和工程技术中复杂的数学及数值计算问题。计算机不仅计算速度快而且精度高，许多人工难以完成的复杂计算都可以通过计算机来解决。在现代科学研究和工程设计中计算机已成为必不可少的计算工具。例如人造卫星轨道的计算、宇宙飞船的制导、可控热核反应、新材料的研制、原子能的研究、气象预报等都是借助计算

每日更新 收藏必备http://wangchuang.32666.com

机来进行运算处理的。

2. 办公自动化

办公自动化简称为 OA（Office Automation）。它是计算机、通信与自动化技术相结合的产物，也是当前最为广泛的一类应用。主要包括：电子数据处理系统 EDP（Electronic Data Process），如公文的编辑打印、报表的填写与统计、文档检索、活动安排及其他数据处理等；管理信息系统 MIS（Management Information System），这是采用计算机对企事业单位或政府机关实行全面管理的信息处理系统，如人事管理、财务管理、计划管理、统计管理等；决策支持系统 DSS（Decision Supporting System），包括数据库、知识库、模型库和方法库，它通过对大量历史数据和当前数据的统计、分析，预测在不同对策下可能产生的结果。

3. 数据库应用

数据库是在计算机存储设备中按照某种关联方式存放的一批数据。借助数据库管理系统 DBMS（Database Management System），可对其中的数据实施控制、管理和使用，如科技情报检索系统、银行储户管理系统、飞机票订票系统等。根据数据存放的差异，可以将数据库分为集中式和分布式两类：集中式数据库将数据集中在一台计算机上；分布式数据库则将数据分散在多台计算机内。

4. 多媒体技术

这是一种交互式地综合处理各种不同感觉媒体（如语言、音乐、文字、数值、图画、活动图像）的信息处理技术，具有这种功能的计算机就是多媒体计算机，多媒体技术的发展使得计算机更有效地进入了人类生活的各个领域，促进了全新的信息产品制造业与信息服务业的进步。

5. 过程控制

过程控制是指用传感器在现场采集受控对象的数据，通过比较器求出与设定数据的偏差，由计算机按控制模型进行计算，产生相应的控制信号，驱动伺服装置对受控对象进行控制和调整。它不仅能通过连续监控提高生产的安全性和自动化水平，同时也提高了产品的质量，降低了成本，减轻了劳动强度。过程控制在石油化工生产、钢铁及有色金属冶炼、环境保护监测系统、数控机床和精密机械制造、交通运输中的行车调度、家用电器中的自动控制功能等方面都得到广泛的应用，在军事上也常用计算机控制导弹等武器的发射与导航，自动修正导弹在飞行中的航向控制。

6. 计算机辅助处理

这种方式主要包括以下几个方面。

（1）计算机辅助设计 CAD（Computer Aided Design）：具有快速改变产品设计参数，优化设计方案，动态显示产品投影图、立体图，输出图纸等功能，降低了产品的设计成本，缩短了产品的设计周期。

（2）计算机辅助制造 CAM（Computer Aided Manufacturing）：根据加工过程编写数控加工程序，由程序控制数控机床来完成工件的自动加工，并能在加工过程中自动换刀及给出数据，一次自动完成多种复杂的工序。

（3）计算机集成制造系统 CIMS（Computer Integrated Manufacturing System）：是集设计、制造、管理三大功能于一体的现代化工厂生产系统，代表一种新型的生产模式，具有生产效

每日更新 收藏必备http://wangchuang.32666.com

率高、生产周期短等特点。生产过程自动化是计算机在现代生产领域特别是在制造业中的典型应用，不仅提高了自动化水平，而且使传统的生产技术发生了革命性的变化。

（4）计算机仿真（Simulation）：这是采用编制好的程序和配备相关环境在计算机上进行的模型试验，能够大大减少投资、避免风险。例如：武器的研制、飞机的设计、农作物生产模拟等。除此之外，计算机仿真还经常用于制造训练环境，进行人员培训和教育教学等。

7. 网络与信息化

计算机网络是利用通信设备和线路等与不同的计算机系统互联起来，并在网络软件支持下实现资源共享和传递信息的系统。根据计算机之间距离的远近、覆盖范围的多少，可分为局域网 LAN（Local Area Network）、广域网 WAN（Wide Area Network）、城市网 CAN（City Area Network）和因特网（Internet）。网络应用使人类进入了信息化社会，人们可以在网上浏览、检索信息、下载软件，充分享受网络资源；收发电子邮件（E-mail）、传真（FAX）、传送文件（FTP）、发布公告（BBS）、参加网上会议（Netmeeting），参加各种网上论坛；在网上开展电子商务和电子数据交换等。

远程教育是建立在互联网上的一种教学环境。它以现代化的信息技术为手段，以适合远程传输和交互式学习的教学资源为教材构成开放式教育网络。任何人在任何时间、任何地点都可选择自己需要的内容进行学习，从而为人们提供了一个终身学习的系统。

在信息化社会中，计算机总是和各种信息的加工、处理、存储、检索、识别、控制、分析和使用分不开。随着信息化社会的发展，信息的快速获取和共享已成为一个国家经济发展和社会进步的重要制约因素。

8. 人工智能

人工智能研究目前最具有代表性的两个领域是专家系统和机器人。

（1）专家系统：这是一个具有大量专门知识的计算机程序系统。专家系统总结了某个领域中专家的大量知识，根据这些专门的知识，系统可以对输入的原始数据进行推理，作出判断和决策，回答用户的咨询。目前专家系统已广泛用于化学结构研究、医疗诊断、遗传工程、空中交通控制和商业等领域。

（2）机器人：这是一种能够模仿人类智能和肢体功能的计算机操作装置。自从微处理器问世后，机器人开始进入大量生产和使用阶段。机器人不仅能提高工作质量和生产效率、降低成本、代替人完成有害环境中的工作，而且还因为它是人工智能各学科的综合应用，所以具有重大的科学实验价值。

1.2　微型计算机的硬件结构及系统组成

微型计算机系统包括硬件和软件两大部分。硬件系统是由电子部件和机电装置所组成的计算机实体，其基本功能是接受计算机程序，并在程序控制下完成信息输入、处理和结果输出等任务。软件系统是指为计算机运行服务的全部技术资料和各种程序，以保证计算机硬件的功能得以充分发挥。

每日更新 收藏必备http://wangchuang.32666.com

1.2.1 微型计算机硬件的基本结构

通用微型计算机的硬件系统由微处理器、内存储器、外存储器、总线、接口电路、输入/输出设备等部件组成，如图 1-1 所示。

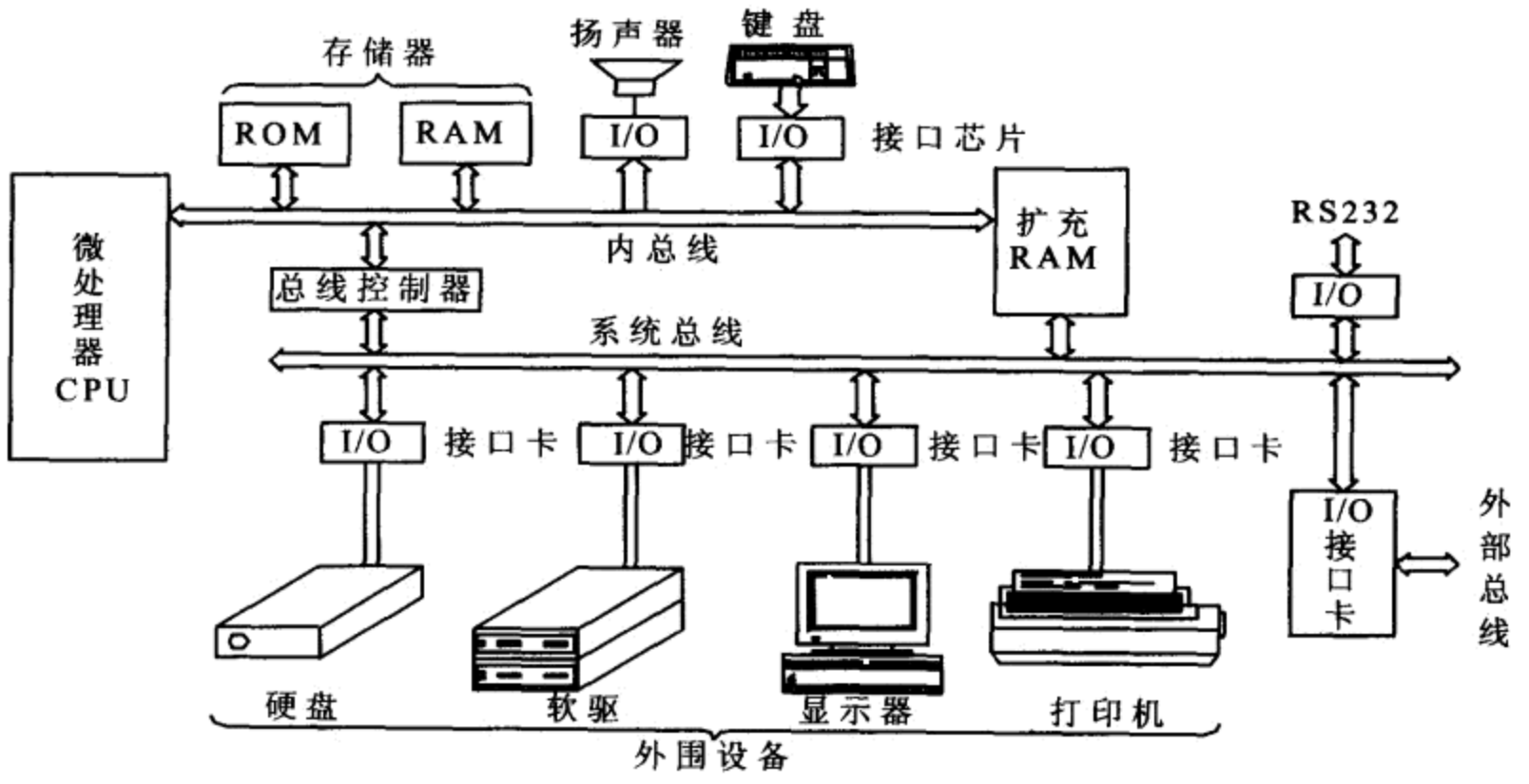


图 1-1 通用微型计算机的硬件系统结构

下面对各主要组成部件的功能进行简要分析。

1. 微处理器

微处理器也称为中央处理器（Control Processing Unit，CPU），是微型计算机的核心部件，它是包含有运算器、控制器、寄存器组以及总线接口等部件的一块大规模集成电路芯片，负责对计算机系统的各个部件进行统一的协调和控制。

（1）运算器（Arithmetic Logic Unit，ALU）：又称为算术逻辑单元，是计算机中加工和处理各种数据的部件，主要完成算术运算和逻辑运算。

（2）控制器（Control Unit）：是计算机工作的指挥与控制中心，它能自动地从内存储器中取出指令并将指令转换成控制信号，指挥各部件协同工作。

（3）寄存器组：是用来暂存数据和指令等信息的逻辑部件，包括通用寄存器和专用寄存器。由于寄存器的访问速度要比存储器快，所以主要起到数据准备、调度和缓冲的作用。

2. 主存储器

主存储器是微型计算机中存储程序、原始数据、中间结果和最终结果等各种信息的部件。按其功能和性能，可以分为随机存储器和只读存储器，两者共同构成主存储器。通常说内存容量时主要是指随机存储器，不包括只读存储器在内。

（1）随机存储器 RAM（Random Access Memory）：又称为读/写存储器。用于存放当前参与运行的程序和数据。其特点是：信息可读可写，存取方便，但信息不能长期保留，断电会丢失。关机前要将 RAM 中的程序和数据转存到外存储器上。

（2）只读存储器 ROM（Read Only Memory）：用于存放各种固定的程序和数据，由生

每日更新 收藏必备http://wangchuang.32666.com

产厂家将开机检测、系统初始化、引导程序、监控程序等固化在其中。它的特点是：信息固定不变，只能读出不能重写，关机后原存储的信息不会丢失。

3. 系统总线

系统总线是 CPU 与其他部件之间传送数据、地址和控制信息的公共通道。各个部件直接用系统总线相连，信号通过总线相互传送。根据传送内容的不同，可以分成以下 3 种总线。

（1）数据总线 DB（Data Bus）：用于 CPU 与主存储器、CPU 与 I/O 接口之间传送数据。数据总线一般为双向总线，其宽度等于计算机的字长。

（2）地址总线 AB（Address Bus）：用于 CPU 访问主存储器和外部设备时，传送相关的地址信号。在计算机中，存储器、输入设备、输出设备等都有各自的地址，地址总线的宽度决定 CPU 的寻址能力。

（3）控制总线 CB（Control Bus）：用于传送 CPU 对主存储器和外部设备的控制信号。控制总线是控制器发送控制信号的通道，控制信号通过控制总线通往各个设备，使这些设备完成指定的操作。

4. 输入/输出接口电路

输入/输出接口电路也称为 I/O（Input /Output）电路，它是微型计算机外部设备交换信息的桥梁。对输入/输出接口电路的特点有如下描述。

（1）接口电路一般由寄存器组、专用存储器和控制电路 3 部分组成，当前的控制指令、通信数据及外部设备的状态信息等分别存放在专用存储器或寄存器组中。

（2）所有外部设备都通过各自的接口电路连接到微型计算机的系统总线上去。

（3）接口电路的通信方式分为并行通信和串行通信，并行通信是将数据各位同时传送，串行通信则使数据一位一位地顺序传送。

5. 主机板

由 CPU、RAM、ROM、I/O 接口电路及系统总线组成的计算机装置简称“主机”。主机的主体是主机板，也称为系统主板或简称主板，CPU 就安装在它的上面。主机板上有 CPU 插座（或插槽）、内存槽、扩展槽、主板电源插座、磁盘接口、主控芯片组、BIOS 芯片、CMOS 芯片、跳线或 DIP 开关、电池、各种外围设备输入/输出端口等，如图 1-2 所示。

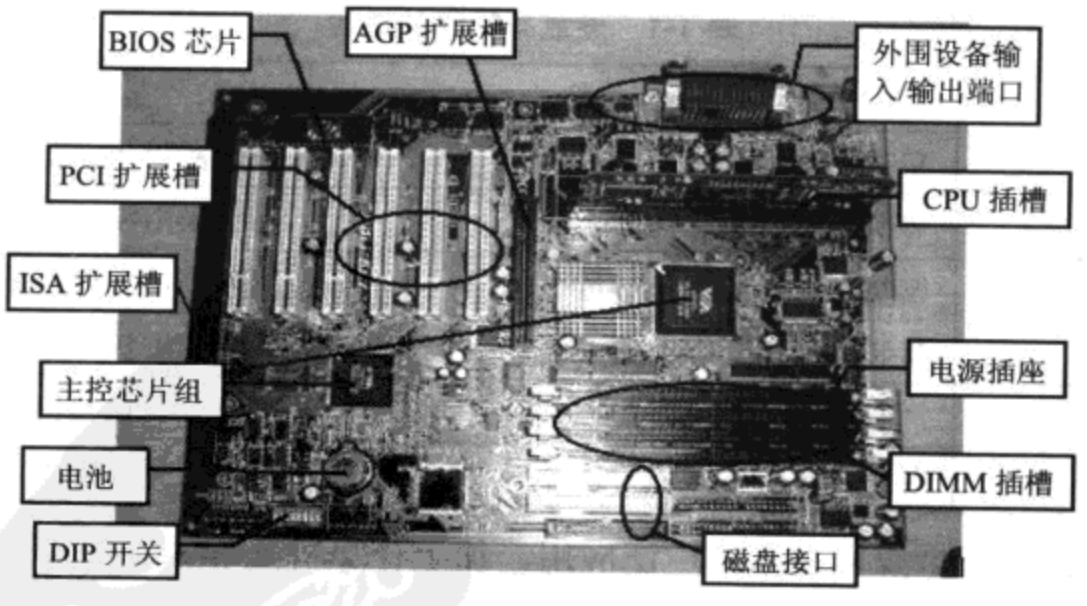


图 1-2　常见微型计算机的主版结构

每日更新 收藏必备http://wangchuang.32666.com

（1）内存槽（DIMM）：用来插入内存条。一个内存条上安装有多个 RAM 芯片。

（2）扩展槽：用来插入各种外部设备的适配卡。选择主板时要注意它的扩展槽数量和总线标准。前者反映计算机的扩展能力，后者表达对 CPU 的支持程度及对适配卡的要求。

（3）跳线及跳线开关（DIP）：跳线是一种起“短接”作用的微型开关，它与多孔微型插座配合使用，可以调整某些相关的参数，以扩大主板的通用性。跳线开关是一组微型开关，可实现跳线的短路、开路作用。

（4）主要辅助部件包括下列几个。

- CMOS 电路：保存存储器和外部设备的种类、规格、当前日期、时间等大量参数，为系统的正常运行提供所需数据。当 CMOS 中的数据出现问题或需要重新设置时，可以在系统启动阶段按【Del】键启动 SETUP 程序，进入修改状态。
- ROM BIOS 芯片：BIOS 是指在 ROM 中固化的“基本输入/输出系统”程序。BIOS 程序的性能对主板影响较大，好的 BIOS 程序能够充分发挥主板各种部件的功能，并能在不同的硬件环境下，方便地兼容运行多种应用软件。
- 外部高速缓冲存储器（Cache）芯片：高速缓冲存储器采用静态读/写存储器（SROM），由两部分组成：一部分存放数据，另一部分是此数据的标记。
- 芯片组：是成套使用的一组芯片，负责将 CPU 运算或处理的结果及其他信息传送到相关部件并实现这些部件的控制，芯片组是 CPU 与所有部件的硬件接口。
- 振荡晶体：产生 CPU 主频所要求的固定频率。有的主板采用可调式振荡晶体，利用跳线生成多种频率，以适应不同的 CPU。

6. 外存储器

目前微型计算机使用的外部存储器大多是磁盘存储器，由磁盘、磁盘驱动器和驱动器接口电路组成。通常，外部存储器分为软盘、硬盘和光盘 3 种。

（1）软盘：由盘片、盘套组成，盘片与盘轴连接，上有读/写定位机构，在盘套上设有读/写窗口和写保护块。目前比较常见的软盘是 3.5 英寸双面高密度磁盘，其容量为 1.44MB。

（2）硬盘：采用金属为基底，表面涂覆有磁性材料，由于刚性较强，所以称为硬盘。

硬盘与软盘的比较如表 1-1 所示。

表 1-1 硬盘与软盘的比较

比较内容	软　盘	硬　盘
结构特点	单片，划分有磁道和扇区	多片密封，组合成柱面和扇区
读/写方式	磁头直接接触磁盘	磁头采用浮动方式工作
存储容量	密度低，单片存储容量小	密度高，存储容量大
存取速度	旋转速度慢，存取速度慢	旋转速度快，存取速度快
使用方式	随身携带，使用方便	多安装在机箱内部
使用寿命和价格	易损坏，寿命短，价格低	不易损坏，寿命长，价格高

（3）光盘：由于多媒体技术的广泛应用，加上计算机处理大量数据、图形、文字、声像等多种信息能力增强，磁盘存储器容量不足的矛盾日益突出。为解决这个问题，人们又研制出了一种新型的“光盘存储器”，它使用激光进行读/写，比磁盘存储器具有更大的存储容

每日更新 收藏必备http://wangchuang.32666.com

量；由于激光头与介质无接触、没有退磁问题，所以信息保存时间长。

7. 输入/输出设备

计算机中最常用的输入设备是键盘和鼠标，最常用的输出设备是显示器和打印机。

（1）键盘：主要用于输入数据、文本、程序和命令。按照各类按键的功能和排列位置，可将键盘分为 4 个主要部分：打字机键盘、功能键、编辑键和数字小键盘。

（2）鼠标：是一种屏幕标定装置。目前常用的有线鼠标分机械式和光电式两种。机械式鼠标利用其下面滚动的小球在桌面上移动，使屏幕上的光标随着移动，这种鼠标价格便宜，但易沾灰尘，影响移动速度，要经常清洗；光电式鼠标通过接收其下面光源发出的反射光，并转换为移动信号送入计算机，使屏幕光标随着移动。光电式鼠标功能优于机械式鼠标。此外还有无线鼠标。

（3）显示器：是微型计算机中最重要的输出设备，是人机交互的桥梁。它能以数字、字符、图形、图像等形式显示各种设备的状态和运行结果，编辑各种程序、文件和图形。显示器通过显示卡连接到系统总线上，显示卡负责把需要显示的图像数据转换成视频控制信号，控制显示器显示该图像。

（4）打印机：也是重要的输出设备，它可以将计算机的运行结果、信息等打印在纸上。打印机按输出方式可分为行式打印机和串式打印机，前者按“点阵”逐行打印，后者按“字符”逐行打印。目前，比较常用的有针式打印机、喷墨打印机、激光打印机等。

1.2.2　微型计算机的系统组成

前面讨论了微型计算机的硬件结构，硬件的基本功能是接受计算机程序，并在程序的控制下完成数据输入、数据处理和输出结果等任务。在硬件基础上合理地配置软件系统后就构成了微型计算机系统，其组成示意图如图 1-3 所示。

软件系统是指为计算机运行工作服务的全部技术资料和各种程序，它可以保证计算机硬件的功能得以充分发挥，并为用户提供一个宽松的工作环境。

计算机软件由系统软件和应用软件组成。系统软件的主要特点是简化计算机操作，支持应用软件的运行并提供服务，通常包括操作系统、实用程序和语言处理程序；应用软件是用来为用户解决某种应用问题的程序及有关的文件和资料。

常用软件的功能和特点简介如下。

（1）操作系统（Operating System）：是用户和计算机系统之间的接口，用户通过操作系统提供的命令调用有关程序来使用计算机。

操作系统本身又由许多程序组成，这些程序分别管理磁盘、输入/输出、CPU、内存储器以及处理中断等。此外，各种实用程序、语言处理程序以及应用程序都在操作系统的管理和控制下运行，使计算机各个部件能够协调地按要求发挥自己的作用。操作系统通常都由生产厂家提供，例如单用户操作系统 MS-DOS，分时、多用户 UNIX 系统以及基于图形用户界面的 Windows 2000 等。

（2）语言处理程序：计算机语言是人与计算机进行交流的工具。计算机能直接识别机器语言，也能经过编译处理用汇编语言和高级程序设计语言编制的程序。

（3）应用软件：是为了解决各类应用问题而编写的程序。它直接面向用户，为用户服

每日更新 收藏必备http://wangchuang.32666.com

务。例如：财务管理软件、汉字处理软件、Office 办公软件等。

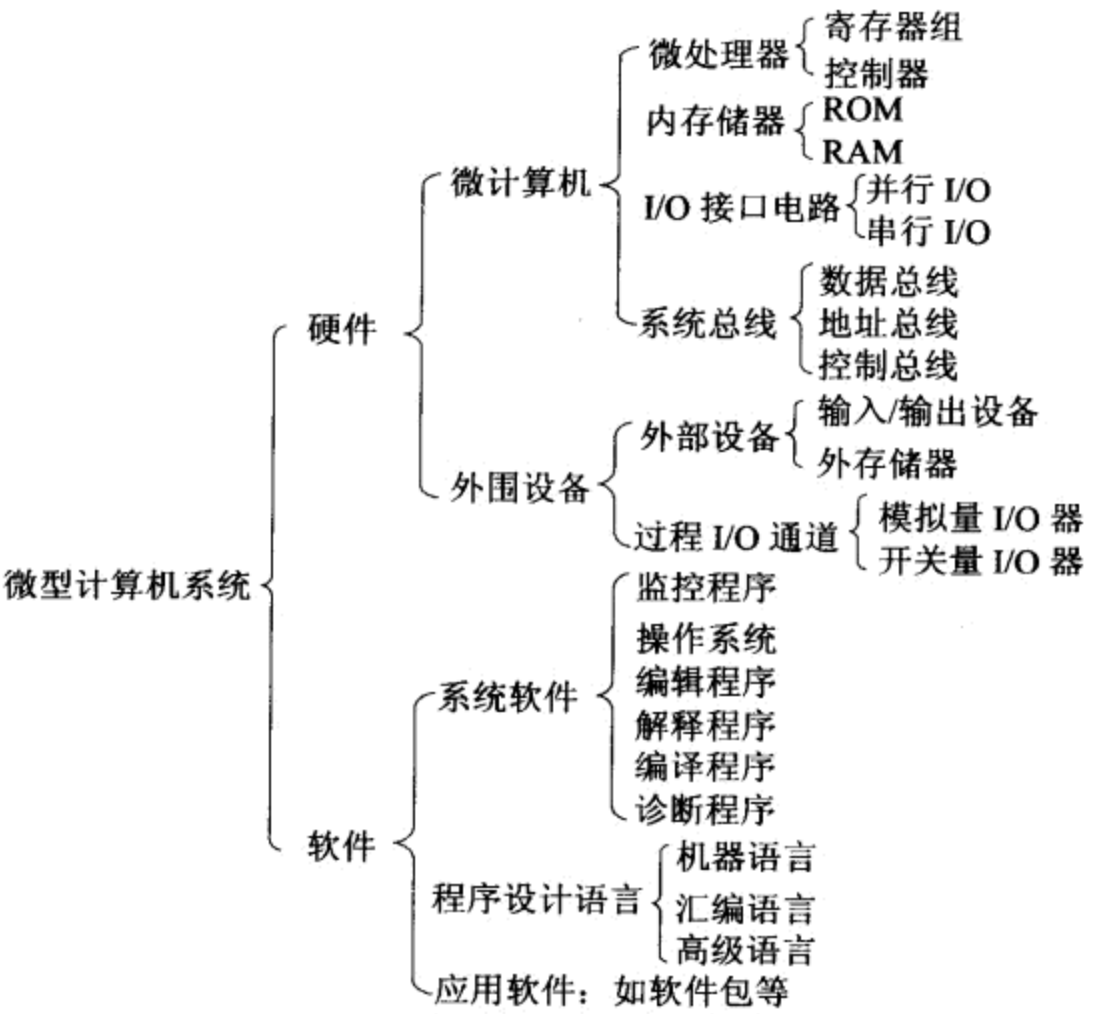


图 1-3　微型计算机系统的组成

1.3　计算机中的信息表示

计算机的基本功能是对数据进行加工，计算机内的数字、字符、指令、控制状态、图形和声音等信息都采用二进制编码形式来表示。在使用上通常把计算机中的数据分为两类：一类是用来表示量的大小的数，能够进行算术等运算；另一类是编码，在计算机中用来描述某种信息。

1.3.1　计算机中的数制及其转换

1. 数制的基本概念

十进制数是人们最熟悉和最常用的数，它采用 0~9 共 10 个数字及其进位来表示数的大小。0~9 称为“数码”，全部数码的个数称为“基数”，用“逢基数进位”的原则进行计数称为进位计数制。十进制数的基数是 10，其计数原则是“逢十进一”。代表不同数值的各位有不同的“位权”，位权与基数的关系是：位权的值等于基数的若干次幂。

例如：十进制数 125.62 可以展开为以下多项式：

$125.62=1\times10^{2}+2\times10^{1}+5\times10^{0}+6\times10^{-1}+2\times10^{-2}$

式中的 10^{2}、10^{1}、10^{0}、10^{-1}、10^{-2} 等即为该位的位权，该位的数值等于该位数码与该位权的乘积。

对于任何一种数制表示的数，都可以写成按位权展开的多项式之和，其一般形式为：

每日更新 收藏必备http://wangchuang.32666.com

$$N=d_{n-1}b^{n-1}+d_{n-2}b^{n-2}+d_{n-3}b^{n-3}+\cdots+d_{-m}b^{-m}$$

式中：

n——整数的总位数。

m——小数的总位数。

$d_{下标}$——表示该位的数码。

b——表示进位制的基数。

$b^{上标}$——表示该位的位权。

为了区分各种计数制的数据，可以采用以下两种方法进行书写表达。

（1）在数字后面加写相应英文字母作为标识。如：B（Binary）表示二进制数； D（Decimal）表示十进制数（可省略）；H（Hexadecimal）表示十六进制数。

（2）在数字的括号外面加计数制下标，此种方法比较直观。如：二进制的 11010011 可以写成$(11010011)_2$。

表 1-2 给出了计算机中二、十、十六进制的基数、数码、进位关系和表示方法。

表 1-2　计算机中二、十、十六进制的基数、数码、进位关系和表示方法

计 数 制	基　数	数　　码	进位关系	表示方法
二进制	2	0、1	逢 2 进 1	1010B 或$(1010)_2$
十进制	10	0、1、2、3、4、5、6、7、8、9	逢 10 进 1	598D 或$(598)_{10}$
十六进制	16	0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F	逢 16 进 1	7C2F H 或$(7C2F)_{16}$

2. 数制之间的转换

为了使用方便，在计算机中有时需要将不同的数制进行转换，其规律总结如表 1-3 所示。

表 1-3　各种计数制之间的转换规律

计数制转换要求	相应转换遵循的规律
十进制整数转换为二进制（或十六进制）整数	采用基数 2（或基数 16）连续去除该十进制整数，直至商等于“0”为止，然后逆序排列所得到的余数
十进制小数转化为二进制（或十六进制）小数	连续用基数 2（或基数 16）去乘以该十进制小数，直至乘积的小数部分等于 0，然后顺序排列每次乘积的整数部分
二、十六进制数转换为十进制数	用其各位所对应的系数，按照“位权展开求和”的方法就可以得到
二进制数转换为十六进制数	从小数点开始分别向左或向右，将每 4 位二进制数分成 1 组，不足 4 位的补 0，然后将每组用一位十六进制数表示即可
十六进制数转换为二进制数	将每位十六进制数用 4 位二进制数表示即可

下面用一些例题来分析各种数制之间的转换方法。

【例 1.1】将十进制整数$(103)_{10}$转换为二进制整数。按照转换规律，采用“除 2 倒取余”的方法，过程如下：

每日更新 收藏必备http://wangchuang.32666.com

2	103	
2	51	余数为 1
2	25	余数为 1
2	12	余数为 1
2	6	余数为 0
2	3	余数为 0
2	1	余数为 1
	0	余数为 1

所以，$(103)_{10}=(1100111)_2$

【例 1.2】将十进制小数$(0.8125)_{10}$转换为二进制小数。按照转换规律，采用“乘 2 顺取整”的方法，过程如下：

0.8125×2=1.625　　取整数位 1

0.625×2=1.25　　取整数位 1

0.25×2=0.5　　取整数位 0

0.5×2=1.0　　取整数位 1

所以，$(0.8125)_{10}=(0.1101)_2$

若出现乘积的小数部分一直不为“0”，则可以根据计算精度的要求截取一定的位数即可。

【例 1.3】将十进制整数$(2347)_{10}$转换为十六进制整数。按照转换规律，采用“除 16 倒取余”的方法，过程如下：

16	2347	
16	146	余数为 11（十六进制数为 B）
16	9	余数为 2
	0	余数为 9

所以，$(2347)_{10}=(92B)_{16}$

【例 1.4】将十进制小数$(0.8129)_{10}$转换为十六进制小数。按照转换规律，采用“乘 16 顺取整”的方法，过程如下：

0.8129×16=13.0064　　取整数位 13（十六进制数为 D）

0.0064×16=0.1024　　取整数位 0

0.1024×16=1.6384　　取整数位 1

0.6384×16=10.2144　　取整数位 10（十六进制数为 A）

本例取到该数据的计算精度为小数点后 4 位数。

所以，$(0.8129)_{10}=(0.D01A)_{16}$

【例 1.5】将二进制数$(1011001.101)_2$转换为十进制数。采用按位权展开求和的方法，过程如下：

$$(1011001.101)_2=1\times2^6+1\times2^4+1\times2^3+1\times2^0+1\times2^{-1}+1\times2^{-3}$$
$$=64+16+8+1+0.5+0.125$$
$$=(89.625)_{10}$$

【例 1.6】将十六进制数$(2D7.A)_{16}$转换为十进制数，过程如下：

每日更新 收藏必备http://wangchuang.32666.com

$(2D7.A)_{16}=2\times16^2+13\times16^1+7\times16^0+10\times16^{-1}$

$=512+208+7+0.625$

$=(727.625)_{10}$

【例 1.7】将二进制数$(1110110010110.010101101)_2$转换为十六进制数，从小数点开始分别向左或向右，将每 4 位二进制数分成一组，过程如下：

0001　1101　1001　0110.　0101　0110　1000

↓　↓　↓　↓　↓　↓　↓

1　D　9　6.　5　6　8

所以，$(1110110010110.010101101)_2=(1D96.568)_{16}$

【例 1.8】将十六进制数$(72A3.C69)_{16}$转换为二进制数，将每位十六进制数用 4 位二进制数表示，过程如下：

7　2　A　3.　C　6　9

↓　↓　↓　↓　↓　↓　↓

0111　0010　1010　0011.　1100　0110　1001

所以，$(72A3.C69)_{16}=(1110010100011.110001101001)_2$

1.3.2　计算机中数值数据的表示

1. 基本概念

在计算机内部表示二进制数的方法称为数值编码，把一个数及其符号在机器中的表示加以数值化，这样的数称为机器数。机器数所代表的数称为该机器数的真值。要完整地表示一个机器数，应考虑 3 个因素：机器数的范围；机器数的符号；机器数中小数点的位置。

（1）机器数的范围：由计算机的 CPU 字长来决定。

当使用 8 位寄存器时，字长为 8 位，所以一个无符号整数的最大值是：

(11111111)B=(255)D，此时机器数的范围是 0~255。

当使用 16 位寄存器时，字长为 16 位，所以一个无符号整数的最大值是：

(1111111111111111)B=(FFFF)H=(65535)D，此时机器数的范围是 0~65 535。

（2）机器数的符号：在算术运算中，数据是有正有负的，称之为带符号数。为了在计算机中正确地表示带符号数，通常规定每个字长的最高位为符号位，并用“0”表示正数，用“1”表示负数。

例如：字长为 8 位二进制时，D7 为符号位，其余 D6~D0 为数值位；字长为 16 位二进制数时，D15 为符号位，其余 D14~D0 为数值位。

（3）机器数中小数点的位置：在机器数中，小数点的位置通常有两种约定，一种规定小数点的位置固定不变，这时的机器数称为“定点数”；另一种规定小数点的位置可以浮动，这时的机器数称为“浮点数”。

2. 带符号数的原码、反码、补码表示

（1）原码：规定正数的符号位为 0，负数的符号位为 1，其他位按照一般的方法来表示数的绝对值。

例如，当机器字长为 8 位二进制数时：

X=+1011011　　[X]原码=01011011

Y=−1011011　　[Y]原码=11011011

原码表示的整数范围是$-(2^{n-1}-1)$～$+(2^{n-1}-1)$，其中 n 为机器字长。

通常：8 位二进制原码表示的整数范围是−127～+127，

16 位二进制原码表示的整数范围是−32 767～+32 767。

（2）反码：对于一个带符号的数来说，正数的反码与其原码相同，负数的反码为其原码除符号位以外的各位按位取反。

例如，当机器字长为 8 位二进制数时：

X=+1011011　　[X]原码=01011011　　[X]反码=01011011

Y=−1011011　　[Y]原码=11011011　　[Y]反码=10100100

负数的反码与负数的原码有很大的区别，反码通常用做求补码过程中的中间形式。反码表示的整数范围与原码相同。

（3）补码：正数的补码与其原码相同，负数的补码为其反码在最低位加 1。

例如：X= +1011011　　[X]原码=01011011　　[X]补码=01011011

Y= −1011011　　[Y]原码=11011011　　[Y]补码=10100101

补码表示的整数范围是-2^{n-1}～$+(2^{n-1}-1)$，其中 n 为机器字长。

则：8 位二进制补码表示的整数范围是−128～+127，

16 位二进制补码表示的整数范围是−32 768～+32 767。

对于 8 位字长的二进制数，其原码、反码、补码的对应关系如表 1-4 所示。

表 1-4　8 位二进制数的原码、反码、补码的对应关系

二进制数	无符号数	带符号数		
		原　码	反　码	补　码
00000000	0	+0	+0	+0
00000001	1	+1	+1	+1
…	…	…	…	…
01111111	127	+127	+127	+127
10000000	128	−0	−127	−128
…	…	…	…	…
11111110	254	−126	−1	−2
11111111	255	−127	−0	−1

（4）补码与真值之间的转换：已知某数的真值，可以通过补码的定义来完成真值到补码的转换；反之，若已知某数的补码，也可以通过以下方法来求出其真值。

- 对于正数的补码，其真值等于补码的本身；
- 对于负数的补码，求其真值时可以除符号位以外将补码的有效值按位求反后在末位加 1，即可得到该负数补码对应的真值。

【例 1.9】已知$[X_1]_{补码}$=01011001B，求真值 X_1；已知$[X_2]_{补码}$=11011010B，求真值 X_2。

（1）由于$[X_1]_{补码}$代表的数是正数，则其真值：

X_1=+1011001B

$=+(1\times2^6+1\times2^4+1\times2^3+1\times2^0)$

=+(64+16+8+1)

=+89D

（2）由于$[X_2]_{补码}$代表的数是负数，则其真值：

$X_2=-([1011010]_{求反}+1)$B

=-(0100101+1)B

=-(0100110)B

$=-(1\times2^5+1\times2^2+1\times2^1)$

=-(32+4+2)

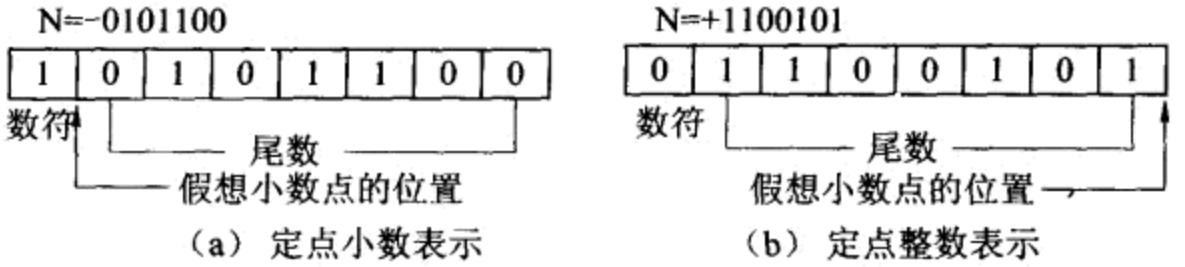
=-38D

3. 定点数和浮点数的表示

（1）定点数：如果数据采用整数表示，那么将小数点约定在最低位的右边，称之为定点整数；如果数据采用纯小数表示，那么将小数点约定在符号位之后，称之为定点小数。

这两种定点数的表示方法在计算机中均可采用。需要说明的是，计算机中小数点的位置是假想位置，计算机厂家在机器设计时将数的表示形式约定好，各种部件及运算线路均按约定的形式进行设计。

例如，对于 8 位字长的计算机，假想小数点的位置可表示为如图 1-4 所示。



（a）定点小数表示　　（b）定点整数表示

图 1-4　定点数的表示

（2）浮点数：如果要处理的数据既有整数部分又有小数部分，就可以采用浮点数表示。该方法中小数点的位置是不固定的，可以表示的数值范围要比定点数大。

例如，以下十进制数可以表示为：

$1428.25=0.142825\times10^4$ ；$-0.000012=-0.12\times10^{-4}$

可见，在原数字中无论小数点前后各有几位数，它们都可以用一个纯小数与 10 的整数次幂的乘积来表示，这就是浮点数的表示方法，也称为科学计数法。

通常，一个二进制数 N 可以表示为：$N=\pm2^{\pm P}\times S$

式中 S 称为 N 的尾数，即全部的有效数字，2 前面的“±”号是尾数的符号；P 称为 N 的阶码，指明小数点的实际位置，2 的右上方的“±”号是阶码的符号。

一般情况下，浮点数在机器中的表示分成两部分，格式如下：

阶符	阶码 P	尾符	尾码 S

阶码 P 用来表示数的实际小数点相对机器中约定小数点位置的浮动方向：如果阶符为负，

则实际小数点在约定小数点的左边，反之在右边，其位置由阶码值来确定，而尾数符号则代表了浮点数的符号。所以阶符和阶码指明了小数点的位置，小数点随着 P 的符号和大小而浮动。

1.3.3 计算机中常用的编码

计算机除了用于数值计算之外，还要进行大量的文字信息处理，也就是要对表达各种文字信息的符号进行加工。例如，计算机和外设的键盘、显示器、打印机之间的通信都是采用字符方式输入/输出的。计算机中目前最通用的两种字符编码分别是美国信息交换标准代码（ASCII 码）和二—十进制编码（BCD 码）。

1. 美国信息交换标准代码（ASCII 码）

ASCII（American Standard Code for Information Interchange）码是美国信息交换标准代码的简称，用于给西文字符编码，包括英文字母的大小写、数字、专用字符、控制字符等。

这种编码由 7 位二进制数组合而成，可以表示 128 种字符，其中 34 个起控制作用的称为“功能码”，其余 94 个符号供书写程序和描述命令之用，称为“信息码”。ASCII 码的编码内容如表 1-5 所示。

表 1-5 7 位 ASCII 码编码表

低 4 位代码		高 3 位代码							
		0	1	2	3	4	5	6	7
		000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	、	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	|
D	1101	CR	GS	—	=	M	]	m	}
E	1110	SO	RS	.	>	N	↑	n	~
F	1111	SI	US	/	?	O	←	o	DEL

表中用英文字母缩写表示的“控制字符”在计算机系统中起各种控制作用，它们在表中

占前两列，加上“SP”和“DEL”，共 34 个；其余的是 10 个阿拉伯数字、52 个英文大小写字母、32 个专用符号等“图形字符”，可以显示或打印出来，共 94 个。

表 1-5 中“控制字符”的含义如表 1-6 所示。

表 1-6　ASCII 编码表中控制字符的含义

字　符	功　能	字　符	功　能	字　符	功　能
NUL	空	FF	走纸控制	CAN	注销
SOH	标题开始	CR	回车	EM	纸尽
STX	正文结束	SO	移位输出	SUB	减
ETX	本文结束	SI	移位输入	ESC	换码
EOT	传输结束	DLE	数据链换码	FS	文字分隔符
ENQ	询问	DC1	设备控制 1	GS	组分隔符
ACK	承认	DC2	设备控制 2	RS	记录分隔符
BEL	报警符	DC3	设备控制 3	US	单元分隔符
BS	退一格	DC4	设备控制 4	SP	空格
HT	横向列表	NAK	否定	DEL	删除
LF	换行	SYN	空转同步		
VT	垂直制表	ETB	信息组传输结束		

ASCII 码是 7 位二进制编码，而计算机的基本存储单位是字节（Byte），一个字节包含 8 个二进制位（bit）。因此，ASCII 码将最高位作为奇偶校验位，用来检验代码在存储、处理和传送过程中是否发生错误。奇校验时，每个代码的二进制形式中应有奇数个“1”；偶校验时，每个代码的二进制形式中应有偶数个“1”。

后来 IBM 公司将 ASCII 码的位数增加了一位，用 8 位二进制数构成一个字符编码，共有 256 个符号。扩展后的 ASCII 码除了原先的 128 个字符外，又增加了一些常用的科学符号和表格线条。

2. 二—十进制编码——BCD 码

计算机中的数采用二进制形式表示，但人们常常习惯用十进制数来进行数据的输入/输出，BCD（Binary-Coded Decimal）码就是专门解决用二进制数表示十进数的问题。BCD 码又称为“二—十进制编码”，最常用的是 8421-BCD 编码，其方法是采用 4 位二进制数来表示 1 位十进制数，自左至右每一个二进制位对应的位权是 8、4、2、1。

由于 4 位二进制数有 0000B~1111B 共 16 种状态，而十进制数 0~9 只取 0000B~1001B 的 10 种状态，其余 6 种状态闲置不用。一般情况下，BCD 码有压缩 BCD 码和非压缩 BCD 码两种表示形式。

（1）压缩 BCD 码。压缩 BCD 码的每一位数采用 4 位二进制数来表示，即一个字节表示两位十进制数。例如，十进制数 59D，采用压缩 BCD 码表示为二进制数是 01011001B。

（2）非压缩 BCD 码。非压缩 BCD 码的每一位数采用 8 位二进制数来表示，即一个字节表示 1 位十进制数，而且只用每个字节的低 4 位来表示 0~9，高 4 位为 0。例如，十进制

数 87D，采用非压缩 BCD 码表示为二进制数是 00001000 00000111B。

8421-BCD 编码如表 1-7 所示。

表 1-7 8421-BCD 编码表

十进制数	8421-BCD 编码	十进制数	8421-BCD 编码
0	0000	8	1000
1	0001	9	1001
2	0010	10	0001 0000
3	0011	11	0001 0001
4	0100	12	0001 0010
5	0101	13	0001 0011
6	0110	14	0001 0100
7	0111	15	0001 0101

3. 汉字编码

具有汉字信息处理能力的计算机系统，除了配备必要的汉字设备和接口外，还应该装配支持汉字信息输入、输出和处理的操作系统。计算机处理汉字信息的前提条件是对每个汉字进行编码，这些编码统称为汉字代码。目前计算机中常用的几种汉字代码有如下几种。

（1）汉字输入码：用于外部输入汉字，也称为外码。它位于人机界面上，面向用户，其编码原则是简单易记、操作方便、有利于提高输入速度。目前使用较多的有顺序码——将汉字按一定顺序排好，然后逐个赋予 1 个号码作为该汉字的编码，例如区位码；音码——根据汉字的读音进行编码，例如拼音码；形码——根据汉字的字形进行编码，例如五笔字型；音形码——根据汉字的读音和字形进行编码，例如双拼码。

（2）汉字机内码：是汉字处理系统内部存储、处理汉字而使用的编码，简称内码。内码与国家标准 GB2312—80 汉字字符集有简明的一一对应关系。

（3）汉字交换码：是汉字信息处理系统之间或通信系统之间传输信息时，对每个汉字所规定的统一编码。我国已指定了汉字交换码的国家标准“信息交换用汉字编码字符集——基本集”，代号 GB2312—80，又称“国标码”。国标码字符集共收录汉字和图形符号 7 445 个。其中：一级常用汉字 3 755 个；二级非常用汉字和偏旁部首 3 008 个；图形符号 682 个。国标码是所有汉字编码都应该遵循的标准，自公布这一标准后，汉字机内码的编码、汉字字库的设计、汉字输入码的转换、输出设备的汉字地址码等，都以此标准为基础。

本章小结

本章从微处理器的产生和发展开始，对微型计算机的基本概念、硬件结构、系统组成、应用特点等各类知识作了相应的概述。通过学习，读者要明确微型计算机的硬件系统主要由 CPU、存储器、系统总线、接口电路及 I/O 设备等部件组成；软件由各种程序和数据组成，可以提高计算机使用效率、扩大计算机功能。系统软件是对硬件功能的扩充与完善，而操作系统是配置在硬件上的第一层软件，它是对硬件功能的第一次扩充，所有系统实用程序以及

更上层的应用程序都在操作系统上运行，受操作系统的统一管理和控制。

目前采用 Pentium 4 芯片为主流机型的微型计算机在市场上占据了重要位置。购买和配置一台微型计算机要综合考虑系统主板、内存容量、外存储器的种类、显示器的类型、软件配置以及价格等因素。

计算机中数据的常用表示方法有二、十、十六进制数，各类数制之间可以相互转换并有一定的规律可循，在机器内部有无符号数和带符号数的表示、定点数和浮点数的表示，此外还有字符编码和汉字编码等。

通过本章的学习，要了解微型计算机的发展历史和应用场合，关注当前微型计算机的发展动向，尤其是微处理器芯片的更新换代，以及相关软件的应用。要熟悉微型计算机系统组成，理解微型计算机硬件和软件各主要模块的功能和在系统中所处的地位；要掌握计算机内部的信息处理方法和特点，熟悉各类数制之间的相互转换，理解无符号数和带符号数的表示方法，掌握 BCD 码、字符的 ASCII 码及汉字编码和应用，为后续内容的学习打下一个良好的基础。

思考与练习题

1. 微处理器和微型计算机的发展经历了哪些阶段？各典型芯片具备哪些特点？
2. 微型计算机硬件结构由哪些部分组成？各部分的主要功能和特点是什么？
3. 什么是系统的主机板？它由哪些部件组成？
4. 微型计算机系统软件的主要特点是什么？包括哪些内容？
5. 微型计算机的主要特点有哪些？简述微型计算机主要性能指标的含义。
6. 什么是微型计算机的系统总线？说明微处理器三大总线的作用。
7. 计算机中常用的数制和码制有哪些？如何进行数制之间的转换？
8. 将下列十进制数分别转换为二进制数、十六进制数和压缩 BCD 码。
   （1）25.82　　（2）412.15　　（3）513.46　　（4）69.136
9. 将下列二进制数分别转换为十进制数和十六进制数。
   （1）111001.101　　（2）110010.1101　（3）1011.11011　（4）101101.0111
10. 将下列十六进制数分别转换为二进制数、十进制数和 BCD 码。
   （1）7B.21　　（2）127.1C　　（3）6A1.41　　（4）2DF3.4
11. 写出下列十进制数的原码、反码、补码表示（采用 8 位二进制数）。
   （1）96　　（2）31　　（3）-42　　（4）-115
12. 已知下列补码求出其原值的十进制表示。
   （1）92H　　（2）8DH　　（3）B2H　　（4）4C26H
13. 按照字符所对应的 ASCII 码表示，查表写出下列字符的 ASCII 码。
   a 、K 、G、+、DEL、SP、CR 、$

第 2 章 典型微处理器

微型计算机的核心部件是微处理器，它决定了微型计算机的结构。要掌握微型计算机原理就应该熟悉微处理器的内部组成及其相应功能，理解各个部件的工作原理和具备的特点。

本章先针对 Intel 8086 微处理器的体系结构，分析其内、外部特性和工作方式，然后再讨论 Intel 80X86、Pentium 等 32 位高档微处理器的组成结构、典型部件及其特点。

通过本章的学习，读者应掌握典型微处理器的内部组成、寄存器结构、外部引脚特性和作用、存储器和 I/O 组织、时序和总线操作以及系统的工作方式和特点等知识。

2.1 微处理器性能简介

微处理器是微型计算机的心脏，也是整个硬件系统的控制指挥中心。微处理器的职能是执行各种运算和信息处理，控制各个计算机部件自动协调地完成系统规定的各种操作。微处理器的性能与它的内部结构和硬件配置直接相关，并反映出它所配置的微型计算机特有的状态。不同型号的微型计算机性能的差别首先在于微处理器性能的不同。

2.1.1 典型微处理器的主要性能指标

在 1.1.3 小节中已经介绍了微型计算机的性能指标，除此以外，典型微处理器还有以下主要性能指标。

（1）主频、外频、倍频：主频是微处理器的时钟频率；外频是指系统总线的工作频率，即主板的工作频率，它可以衡量微型计算机外设的工作速度；而倍频则是指微处理器外频与主频相差的倍数。三者的关系为：主频＝外频×倍频。

（2）内存总线速度：存放在外存上的信息都要通过内存再进入微处理器进行处理，内存总线的速度对整个系统性能显得很重要。由于内存和微处理器之间的运行速度存在差异，通常采用二级缓存来协调，这里的内存总线速度就是指微处理器与二级高速缓存和内存之间的通信速度。

（3）扩展总线速度：扩展总线指的是安装在微型计算机系统上的局部总线，如 VESA 和 PCI 总线等，它是微处理器联系各种外部设备的桥梁。扩展总线速度是指 CPU 与扩展设备之间的数据传输速度。

（4）地址总线宽度：决定了微处理器可以访问的物理地址空间，即微处理器能够使用多大容量的内存。

（5）数据总线宽度：决定了微处理器与二级高速缓存、内存以及输入/输出设备之间一次数据传输的信息量，以二进制为单位。

（6）高速缓存：高速缓存（Cache）也是影响微处理器性能的一个重要因素，在微处理器中内置高速缓存可以提高微处理器的运行效率。Cache 的存取速度与微处理器的主频相匹配。

2.1.2　微处理器的基本功能

微处理器对整个计算机系统的运行是极为重要的，一般情况下，它应该具有以下 4 个方面的基本功能。

（1）指令控制：也称为程序的顺序控制，使计算机中的指令或程序严格按照规定的顺序执行。

（2）操作控制：将计算机指令所产生的一系列控制信号分别送往相应的部件，从而控制这些部件按指令的要求完成规定的工作。

（3）时间控制：使计算机中各类控制信号严格按照时间上规定的先后顺序进行操作，以完成时序控制和总线操作

（4）数据加工：是指对数据进行算术运算和逻辑运算等操作，或对其他信息进行的处理。

2.2　Intel 8086 微处理器的内、外部结构特性

1978 年 Intel 公司推出的 8086 是一种具有代表性的 16 位微处理器，后续推出的各种微处理器均保持与之兼容。

8086 微处理器使用+5V 电源，40 条引脚双列直插式封装，时钟频率为 5MHz~10MHz，基本指令执行时间为 0.3μs~0.6μs。有 16 根数据线和 20 根地址线，可寻址的内存地址空间达 1MB（2^{20}B）。8086 微处理器的特点是：采用并行流水线工作方式，通过设置指令预取队列实现；对内存空间实行分段管理，将内存分为 4 个段并设置段寄存器，以实现对 1MB 空间的寻址；支持多处理器系统；有最小模式和最大模式两种工作模式。

2.2.1　Intel 8086 微处理器内部组成结构

8086 微处理器从功能上可以划分为两个逻辑单元，即执行部件 EU（Execution Unit）和总线接口部件 BIU（Bus Interface Unit），其内部组成结构如图 2-1 所示。

1. 执行部件 EU

执行部件 EU 中包含 1 个 16 位的算术逻辑单元（ALU），8 个 16 位的通用寄存器，1 个 16 位的状态标志寄存器，1 个数据暂存寄存器和执行部件的控制电路。

EU 的主要功能是：从 BIU 的指令队列中取出指令代码，经指令译码器译码后执行指令规定的全部功能。执行结果或执行指令所需的数据，都由 EU 向 BIU 发出命令，对存储器或 I/O 接口进行读/写操作。

EU 中的各个部件通过 16 位的 ALU 总线连接在一起，在内部实现快速数据传输。EU 从 BIU 的指令队列中获得指令，当指令要求访问存储器单元或外部设备时，EU 就向 BIU 发出操作请求，并提供访问的数据和地址，由 BIU 完成相应的操作。

EU 中各个组成部件的功能分析如下。

（1）算术逻辑单元 ALU：可用于算术、逻辑运算，也可按指令的寻址方式计算出寻址单元的 16 位偏移地址（有效地址 EA），并将其送到 BIU 中形成 1 个 20 位的实际地址，实现对 1MB 的存储空间寻址。在算术、逻辑运算时，数据先传送到暂存寄存器中，再经 ALU 运算处理。运算后的结果经内部总线送回到累加器或其他寄存器、存储单元中。

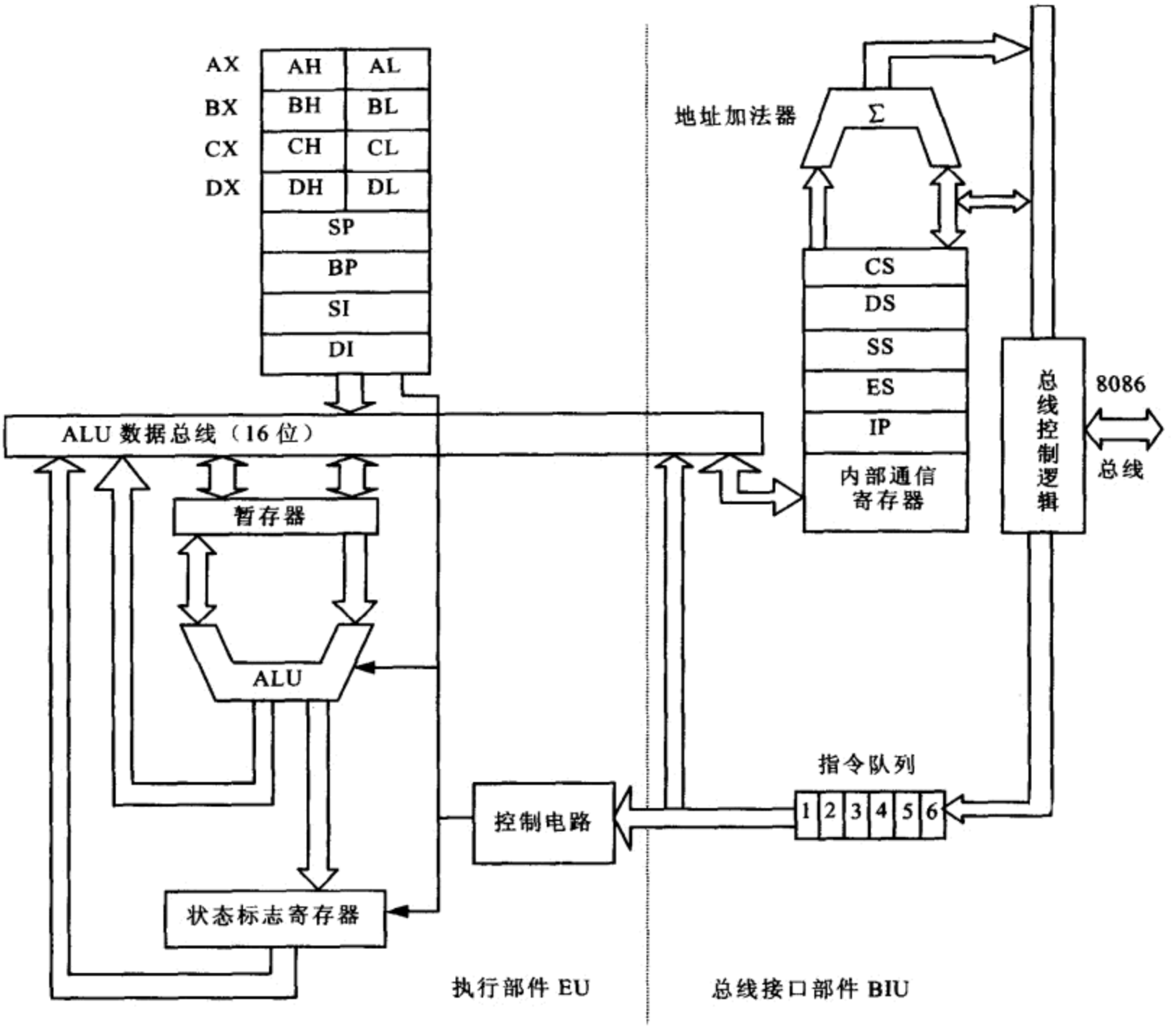


图 2-1　8086 微处理器内部结构框图

（2）标志寄存器 FLAG：用来反映 CPU 最近一次运算结果的状态特征或存放控制标志。标志寄存器的各标志位记录了指令执行后的各种状态。

（3）数据暂存寄存器：协助 ALU 完成运算，暂存参加运算的数据。

（4）通用寄存器组：包括 4 个 16 位数据寄存器 AX、BX、CX、DX，可以用来寄存 16 位或 8 位数据；4 个 16 位地址指针与变址寄存器 SP、BP、SI、DI。

（5）EU 控制电路：是控制、定时与状态逻辑电路，接收从 BIU 指令队列中取来的指令，经过指令译码形成各种定时控制信号，对 EU 的各个部件实现特定的定时操作。

2. 总线接口部件 BIU

BIU 内部设有 4 个 16 位段地址寄存器，即代码段寄存器 CS、数据段寄存器 DS、堆栈段寄存器 SS 和附加段寄存器 ES；1 个 16 位指令指针寄存器 IP；1 个 6 字节指令队列缓冲器；以及 20 位地址加法器和总线控制电路。

BIU 的主要功能是：根据 EU 的请求，完成 CPU 与存储器或 I/O 设备之间的数据传送。BIU 提供从存储器的指定单元取出指令，送至指令队列中或直接传送给 EU 去执行；从存储

器的指定单元和外设端口中取出指令规定的操作数传送给 EU，或者把 EU 的操作结果传送到指定的存储单元和外设端口中。

BIU 中相关部件的功能分析如下。

（1）指令队列缓冲器：8086 的指令队列可存放 6 个字节的指令代码，按“先进先出”的原则进行存取操作，其工作顺序如下。

- BIU 从存储器取出的指令按字节顺序存放在指令队列缓冲器中，EU 按顺序取出，译码后执行。
- 当 EU 正在执行指令且不需要占用总线时，一旦指令队列缓冲器空出 2 个字节，BIU 会自动进行预取下一条指令的操作，将取得的指令按先后次序存入指令队列缓冲器中，然后再由 EU 按顺序取出来执行。
- 在 EU 执行指令的过程中，指令需要对存储器或 I/O 设备进行数据存取时，BIU 将在执行完现行取指的存储器周期后的下一个存储器周期，对指定的存储器单元或 I/O 设备进行存取操作，交换的数据经 BIU 送至 EU 进行处理。
- 当 EU 执行完转移、调用和返回指令时，BIU 会自动清除指令队列缓冲器，将指令队列中的尚存指令作废，并要求 BIU 从新的地址重新开始取指令，新取的第一条指令将直接经指令队列送到 EU 去执行，随后取来的指令将填入指令队列缓冲器。

（2）地址加法器和段寄存器：地址加法器和段寄存器用于形成存储器的物理地址，完成从 1 个 16 位的存储器逻辑地址（由段基址与段内偏移地址两部分组成）到 1 个 20 位的实际存储器地址（物理地址）的转换运算。

（3）指令指针寄存器 IP：IP 用于存放 BIU 要取的下一条指令的段内偏移地址。程序不能直接对 IP 进行存取，但能在程序运行中自动修正，使之指向要执行的下一条指令。某些转移、调用、中断和返回等指令能使 IP 的值改变。

（4）总线控制电路与内部通信寄存器：总线控制电路用于产生外部总线操作时的相关控制信号，是连接 CPU 外部总线与内部总线的中间环节，而内部通信寄存器用于暂存总线接口单元 BIU 与执行单元 EU 之间交换的信息。

微处理器在执行程序时依次先从存储器中取出一条指令，然后读出操作数，最后执行指令。即取指令和执行指令是串行进行的，取指令期间 CPU 必须等待，其过程如图 2-2 所示。

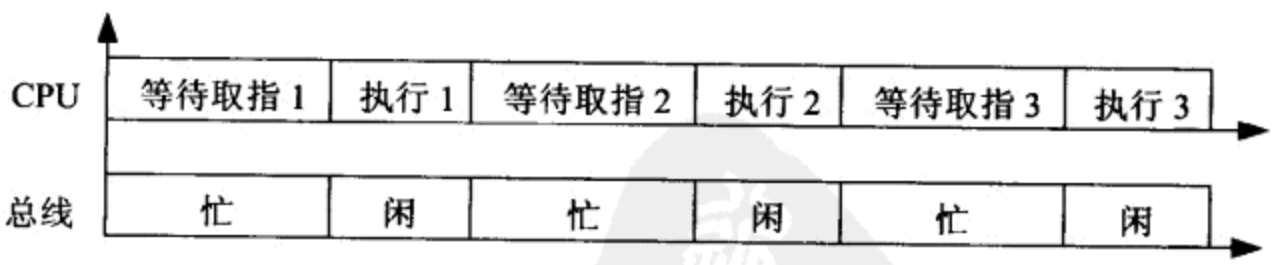


图 2-2　传统微处理器的指令执行过程

在 8086 中，取指令和执行指令分别由总线接口部件 BIU 和执行部件 EU 来完成，并且存在指令队列缓冲器中，使 BIU 和 EU 可以并行工作，执行部件负责执行指令，总线接口部件负责提取指令、读出操作数和写入结果。这两个部件能互相独立地工作。

在大多数情况下，取指令和执行指令可以重叠进行，即在执行指令的同时进行取指令的操作，如图 2-3 所示。

8086 中的 BIU 和 EU 采用并行工作方式，减少了 CPU 为取指令而等待的时间，使整个程序运行期间 BIU 总是忙碌的，充分利用了总线，极大地提高了 CPU 的工作效率，加快了整机的运行速度，也降低了 CPU 对存储器存取速度的要求，这成为 8086 的突出优点。

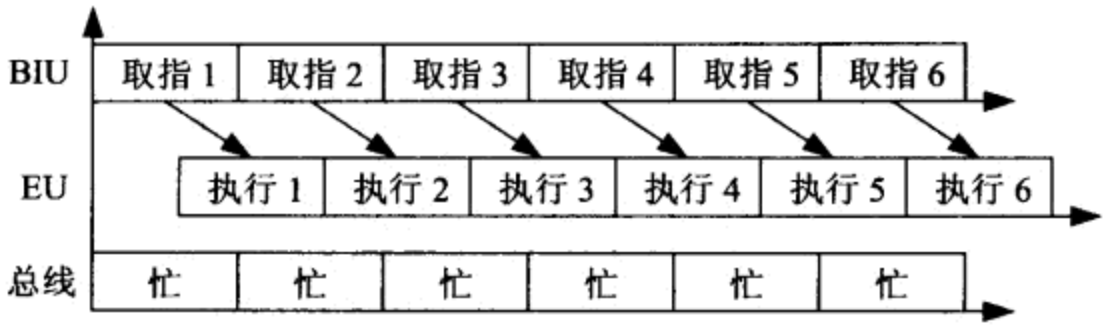


图 2-3　8086 微处理器的指令执行过程

2.2.2　8086 微处理器的寄存器结构

CPU 中大部分指令都是在寄存器中实现对操作数的预定功能，这些寄存器用来暂时存放参加运算的操作数和运算过程中的中间结果，使得在程序执行的过程中不必每时每刻都要到存储器中存取数据。

8086 微处理器中可供编程使用的有 14 个 16 位寄存器，按其用途可分为 3 类：通用寄存器、控制寄存器、段寄存器，如图 2-4 所示。

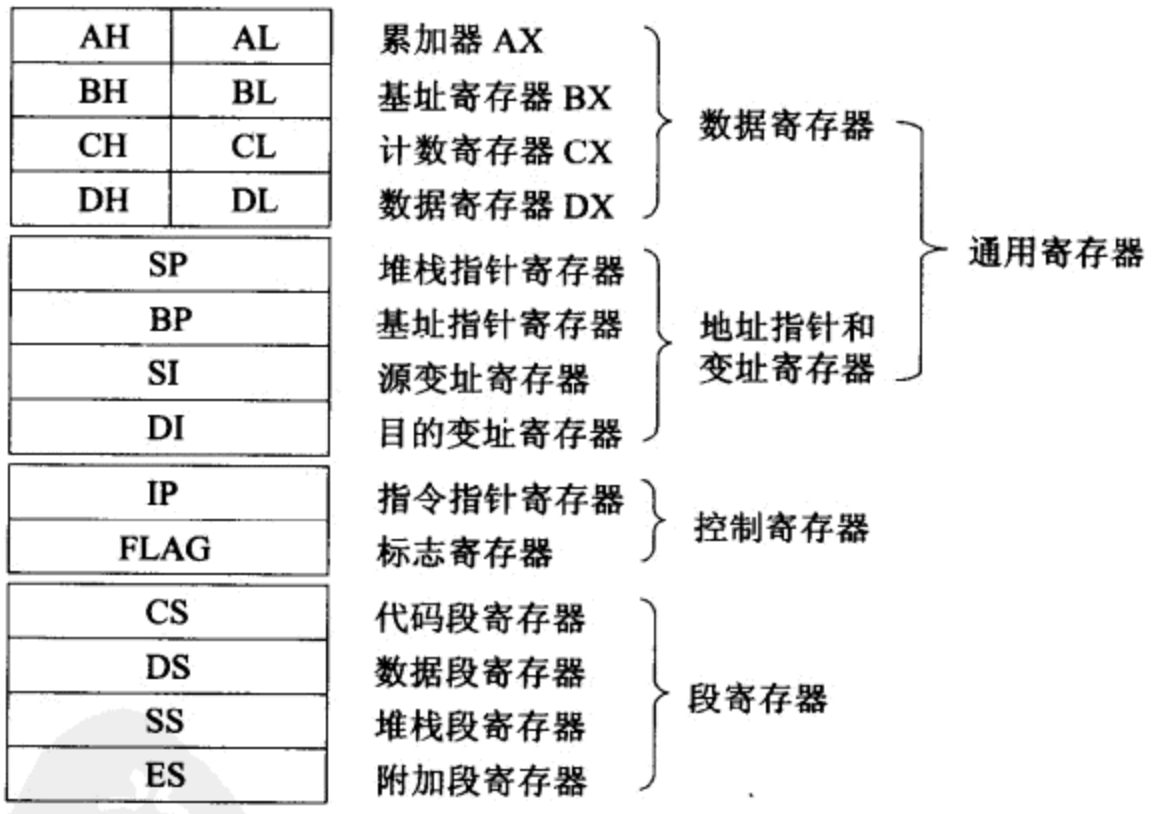


图 2-4　8086 微处理器内部寄存器结构

1. 通用寄存器

8086 微处理器中设置了一些通用寄存器，操作数可以直接存放在这些寄存器中，既可减少访问存储器的次数，又可缩短程序的长度，占用内存空间少，提高了数据处理速度。

8086 的通用寄存器分为数据寄存器、地址指针和变址寄存器两组。

（1）数据寄存器：用来存放操作数或中间结果。包括 4 个 16 位的寄存器 AX、BX、CX

和 DX，还可将其分成独立的两个 8 位寄存器来使用，即 AH、BH、CH、DH 和 AL、BL、CL、DL 两组。16 位数据寄存器主要用于存放数据和地址，而 8 位寄存器只能用于存放数据。

（2）地址指针和变址寄存器：用来存放地址的偏移量（被寻址存储单元相对于段起始地址的距离，或称偏移地址）。包括 16 位的指针寄存器 SP、BP 和变址寄存器 SI、DI。SP 和 BP 用于堆栈段操作，入栈（PUSH）和出栈（POP）指令由 SP 给出栈顶偏移地址，称为堆栈指针寄存器；BP 存放位于堆栈段中数据区基址的偏移地址，称做基址指针寄存器。SI 和 DI 用来存放当前数据段的偏移地址。源操作数的偏移地址存放于 SI 中，称其为“源变址寄存器”；目的操作数的偏移地址存放于 DI 中，称其为“目的变址寄存器”。

通用寄存器在一定场合有其特定的用法。如：AX 作累加器；BX 作基址寄存器；CX 作计数寄存器；DX 作数据寄存器等。

表 2-1 中给出了一些寄存器在指令中的特定用法。

表 2-1　通用寄存器的特定用法

寄存器名称	寄存器含义	常用的操作功能
AX	16 位累加器	字乘，字除，字 I/O 处理
AL	8 位累加器	字节乘，字节除，字节 I/O 处理，查表转换，十进制运算
AH	8 位累加器	字节乘，字节除
BX	16 位基址寄存器	查表转换
CX	16 位计数寄存器	数据串操作指令，循环指令
CL	8 位计数寄存器	变量移位，循环移位
DX	16 位数据寄存器	字乘，字除，间接 I/O 处理
SP	16 位堆栈指针寄存器	堆栈操作
SI	16 位源变址指针寄存器	数据串操作指令
DI	16 位目的变址指针寄存器	数据串操作指令

2. 控制寄存器

8086 微处理器的控制寄存器主要有指令指针寄存器 IP 和标志寄存器 FLAG。

（1）指令指针寄存器 IP：IP 是一个 16 位的寄存器，存放 EU 要执行的下一条指令的偏移地址，用以控制程序中指令的执行顺序。正常运行时，BIU 可修改 IP 中的内容，使它始终指向 BIU 要取的下一条指令的偏移地址。

一般情况下，每取一次指令操作码 IP 就自动加 1，从而保证指令按顺序执行。应当注意，IP 实际上是指令机器码存放单元的地址指针，不能用指令取出 IP 或给 IP 设置给定值，但可以通过某些指令修改 IP 的内容，例如转移类指令就可以自动将转移目标的偏移地址写入 IP 中，实现程序转移。

（2）标志寄存器 FLAG：标志寄存器 FLAG 是一个 16 位的寄存器，共 9 个标志，其中 6 个用做状态标志，3 个用做控制标志，如图 2-5 所示。

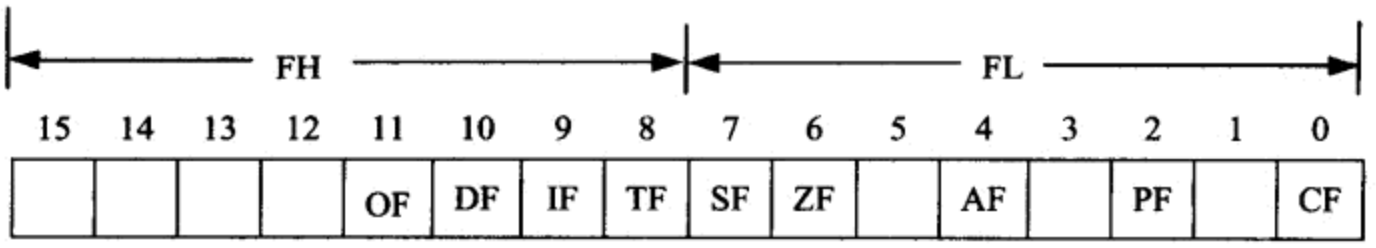


图 2-5　8086 微处理器的标志寄存器 FLAG

FLAG 中的状态标志用来反映 EU 执行算术运算和逻辑运算以后的结果特征，这些标志常常作为条件转移类指令的测试条件，以控制程序的运行方向；控制标志用来控制 CPU 的工作方式或工作状态，它一般由程序设置或由程序清除。这 9 个标志位的含义和特点如表 2-2 所示。

表 2-2　标志寄存器 FLAG 中标志位的含义和特点

标志类别	标志位	含　义	特　点	应用场合
状态标志	CF（Carry Flag）	进位标志	CF=1：结果在最高位产生一个进位或借位；CF=0：无进位或借位	主要用于加、减运算，移位和循环指令
	PF（Parity Flag）	奇偶标志	PF=1：结果中有偶数个 1；PF=0：结果中有奇数个 1	用于检查在数据传送过程中是否有错误发生
	AF（Auxiliary Carry Flag）	辅助进位标志	AF=1：结果中的低 4 位产生了一个进位或借位；AF=0：无进位或借位	主要用于实现 BCD 码算术运算结果的调整
	ZF（Zero Flag）	零标志	ZF=1：运算结果为零；ZF=0：运算结果不为零	用于判断运算结果和进行控制转移
	SF（Sign Flag）	符号标志	SF=1：运算结果为负数；SF=0：运算结果为正数	用于判断运算结果和进行控制转移
	OF（Overflow Flag）	溢出标志	OF=1：带符号数运算时产生算术溢出；OF=0：无溢出	主要用于判断运算结果的溢出情况
控制标志	TF（Trap Flag）	陷阱标志	TF=1：CPU 处于单步工作方式；TF=0：CPU 正常执行程序	用于程序调试
	IF（Interrupt-Enable Flag）	中断允许标志	IF=1：允许接受 INTR 发来的可屏蔽中断请求信号；IF=0：禁止接受可屏蔽中断请求信号	用于控制可屏蔽中断
	DF（Direction Flag）	方向标志	DF=1：字符串操作指令按递减顺序从高到低的方向进行处理；DF=0：按递增的顺序从低到高的方向进行处理	用于控制字符串操作指令的步进方向

对于状态标志，CPU 在进行算术/逻辑运算时，根据操作结果自动将状态标志位置位（等于 1）或复位（等于 0）；对于控制标志，事先用指令设置，在程序执行时检测这些标志，用以控制程序的转向。

3. 段寄存器

8086 最大可寻址 1MB 的存储空间，但是，包含在指令中的地址以及在指针和变址寄存

器中的地址只有 16 位长，不可能寻址到 1MB 的存储空间。为此，采用存储器分段技术来实现。8086CPU 把这 1MB 的存储空间分成若干个逻辑段，每个逻辑段的长度不超过 64KB。这些逻辑段是相互独立的，可以在整个空间浮动。

8086CPU 共有 4 个 16 位的段寄存器，用来存放每一个逻辑段的段起始地址。

（1）代码段寄存器 CS（Code Segment）：用来给出当前的代码段起始地址，存放 CPU 可以执行的指令，CPU 执行的指令将从代码段取得。

（2）数据段寄存器 DS（Data Segment）：指向程序当前使用的数据段，用来存放数据，包括参加运算的操作数和中间结果。

（3）堆栈段寄存器 SS（Stack Segment）：给出程序当前所使用的堆栈段，即在存储器中开辟的堆栈区，堆栈操作的执行地址就在该段。

（4）附加段寄存器 ES（Extra Segment）：指出程序当前所使用的附加段，通常也用来存放数据，典型用法是存放处理以后的数据。

2.2.3　8086 微处理器的外部引脚特性

8086 微处理器采用双列直插式的封装形式，具有 40 条引脚，如图 2-6 所示。

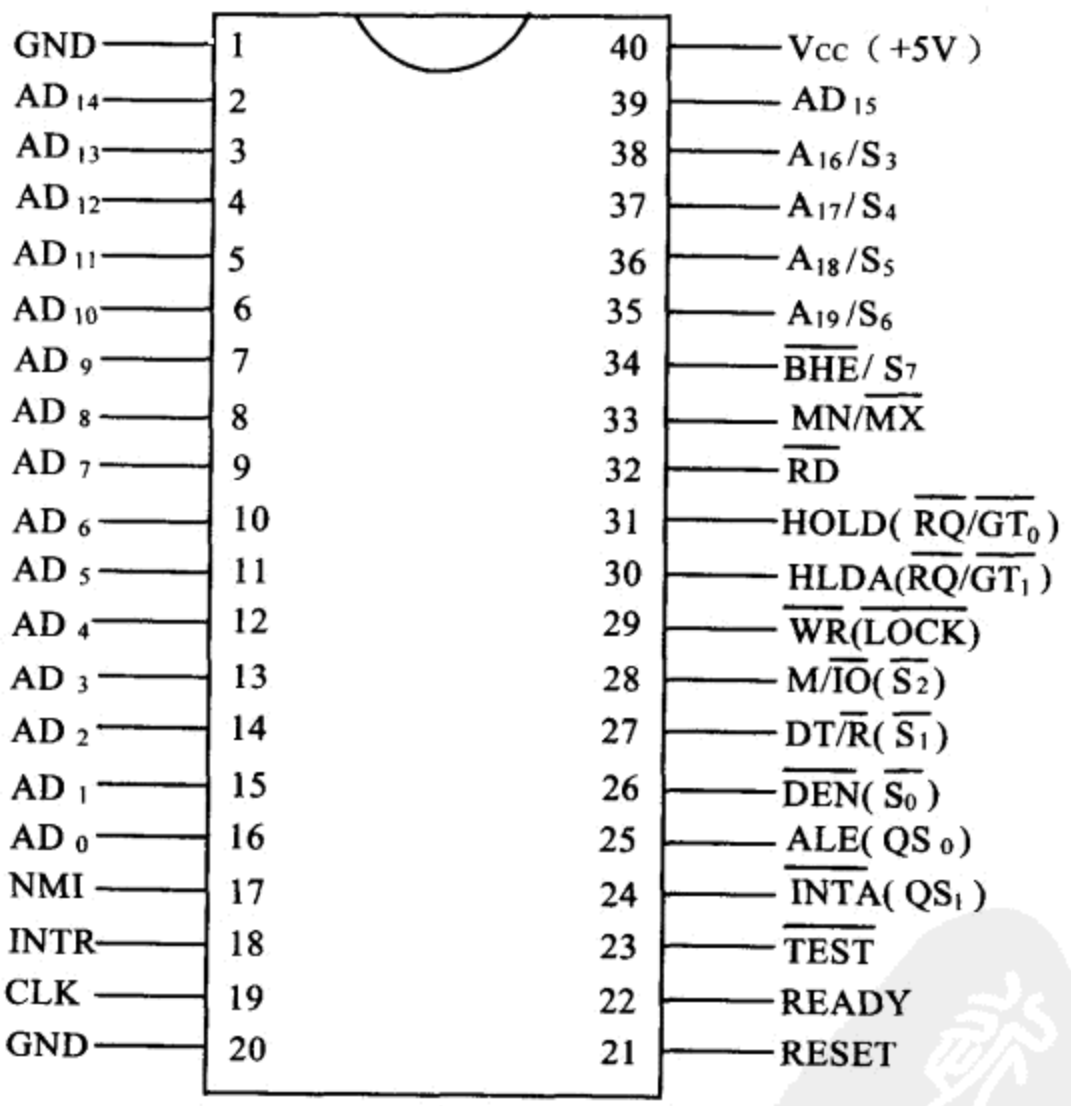


图 2-6　8086 微处理器引脚图

8086 微处理器数据总线为 16 条，地址总线为 20 条，采用分时复用的地址/数据总线，即一部分引脚具有双重功能。如 AD_{15}~AD_0 这 16 个引脚，有时输出地址信号，有时可传送数据信号。

为了适应不同的应用环境，8086 微处理器有两种工作方式：最大工作方式（MN/$\overline{MX}$ =0）和最小工作方式（MN/$\overline{MX}$ =1），可以通过第 33 条引脚 MN/$\overline{MX}$ 来控制。最大工作方式适用于多微处理器组成的大系统，最小工作方式适用于单微处理器组成的小系统。

8086 微处理器的引脚按其作用可分为以下 5 类。

1. 地址/数据总线 AD_{15}~AD_0（双向，三态）

这是分时复用的存储器或 I/O 端口的地址/数据总线。传送地址时三态输出，传送数据时可双向三态输入/输出。所谓“三态”是指除“0”、“1”两种状态外，还有一种悬浮（高阻）状态，通常采用三态门进行控制。

CPU 访问存储器或 I/O 端口一次所需的时间称为总线周期。一个总线周期，通常包括 T_1，T_2，T_3，T_4 状态，即 4 个时钟周期。作为地址和数据总线的复用引脚，在总线周期的第一个时钟周期（T_1 状态），输出存储器或 I/O 端口的低 16 位地址（记为 A_0~A_{15}），而在总线周期的其他时钟周期内，用于传送数据。当 CPU 处于“保持响应”状态时，这些引脚处于高阻隔离状态（即悬浮状态）。

2. 地址/状态线 A_{19}/S_6~A_{16}/S_3（输出，三态）

A_{19}~A_{16} 是地址总线的高 4 位，S_6~S_3 是状态信号，这些引线也采用多路开关的分时输出，在存储器操作的总线周期的第一个时钟周期（T_1 状态）时，输出 20 位地址的高 4 位 A_{19}~A_{16}，与 A_{15}~A_0 组成 20 位地址信号。访问 I/O 时，不使用这 4 条线，即 A_{19}~A_{16} 等于 0，在其他时钟周期输出状态信号。S_3 和 S_4 的组合表示正在使用的寄存器名，如表 2-3 所示。S_5 表示 IF 的当前状态，S_6 则始终输出低电平“0”，以表示 8086 当前连接在总线上。

表 2-3 S_4~S_3 的代码组合与对应的状态

S_4	S_3	状　态
0	0	当前正在使用 ES（可修改数据）
0	1	当前正在使用 SS
1	0	当前正在使用 CS，或未使用任何段寄存器
1	1	当前正在使用 DS

3. 控制总线

（1）总线高字节允许/状态 $\overline{BHE}/S_7$（输出，三态）：在总线周期的第一个时钟周期（T_1），输出总线高字节允许信号 $\overline{BHE}$，表示高 8 位数据线上的数据有效，其余时钟周期输出状态 S_7。若 $\overline{BHE}$ =1，表示数据传送只在 AD_0~AD_7 上进行。$\overline{BHE}$ 和 A_0 配合用来产生存储体的选择信号。

（2）读控制信号线 $\overline{RD}$（输出，三态）：当 $\overline{RD}$ 有效（低电平）时，表示 CPU 正在进行读存储器或读 I/O 端口的操作。CPU 是读取内存单元还是读取 I/O 端口的数据，取决于 M/$\overline{IO}$ 信号。在一个读操作的总线周期中，$\overline{RD}$ 信号在 T_2、T_3 和 T_W 状态均为低电平。在系统总线进入“保持响应”期间，$\overline{RD}$ 被浮空。

（3）准备就绪信号 READY（输入）：该信号高电平有效，是由被访问的存储器或 I/O 端口发来的响应信号。READY=1 时，表示所寻址的存储单元或 I/O 端口已准备就绪。CPU 在每个总线周期的 T_3 状态开始对 READY 信号进行检测。如果检测到 READY=0，表示寻址对象尚未准备就绪，要求 CPU 自动插入一个或几个等待状态（T_W），然后在 T_W 状态再次检查 READY

信号，若还是无效，则继续插入一个新的 T_W，直到 READY 线出现高电平才进行数据传送。

（4）测试信号 $\overline{TEST}$（输入）：测试信号是由 WAIT 指令来检查的。当 CPU 执行 WAIT 指令时，每隔 5 个时钟周期对该线的输入进行一次测试。若 $\overline{TEST}$ =1，CPU 停止取下一条指令而进入等待状态，重复执行 WAIT 指令，CPU 空闲等待，直到 $\overline{TEST}$ =0，等待状态结束，CPU 继续执行被暂停的指令。等待期间允许外部中断。$\overline{TEST}$ 引脚信号用于多处理器系统中实现 CPU 与协处理器或其他处理器的同步协调。

（5）可屏蔽中断请求信号 INTR（输入）：高电平有效。当 INTR=1 时，表示外设向 CPU 提出了中断请求，CPU 在每个指令周期的最后一个 T 状态采样该信号。若 IF=1（即中断未屏蔽），则 CPU 响应中断，暂停正在执行的程序，CPU 通过执行中断响应周期转去执行中断服务程序；若 IF=0（即中断被屏蔽），则 CPU 继续执行指令队列中的下一条指令。

（6）非屏蔽中断请求信号 NMI（输入）：该信号上升沿有效，它不受中断允许标志 IF 状态的影响，即不能用软件进行屏蔽。只要 NMI 出现，CPU 就会在结束当前指令后，进入相应的中断服务程序。NMI 比 INTR 的优先级别高。

（7）复位信号 RESET（输入）：使 CPU 立即结束当前正在进行的操作。CPU 要求复位信号至少要保持 4 个时钟周期的高电平，才能结束它正在进行的操作。随着 RESET 信号变为低电平，CPU 就开始执行再启动过程。CPU 复位之后，将从 FFFF0H 单元开始取出指令，这个单元在 ROM 区域中，那里通常放置一条转移指令，它所指向的目的地址就是系统程序的实际起始地址。

（8）系统时钟 CLK（输入）：为 CPU 提供基本的时钟脉冲，通常与时钟发生器 8284A 的时钟输入端相连。

4. 电源线 V_{CC} 和地线 GND

电源线 V_{CC} 接入的电压为+5V±0.5V，两条地线 GND 均应接地。

5. 其他控制线：24 脚~31 脚

这些控制线的性能将根据方式控制线 MN/$\overline{MX}$ 所处的状态而定。

2.3　存储器和 I/O 组织

2.3.1　存储器组织

1. 存储器的内部结构及访问方法

由于 8086 有 20 根地址线，所以可寻址的存储器空间为 1MB（2^{20}B），地址范围为 0~2^{20}-1（00000H~FFFFFH）。

存储器内部按字节进行组织，两个相邻的字节称为一个字。存放的信息若以字节为单位，则将其在存储器中按顺序排列存放；若存放的数据为一个字则将每一个字的低字节存放在低地址中，高字节存放在高地址中，并以低地址作为该字的地址。

在 8086 存储器中，从偶地址开始存放的字，称为规则字或对准字，从奇地址开始存放的字，称为非规则字或非对准字。规则字的存取可在一个总线周期内完成，非规则字的存取需两个总线周期。

8086 存储器 1MB 的存储空间被分成两个 512KB 的存储体，分别叫高位库和低位库。低位库固定与 CPU 低位字节数据线 D_7~D_0 相连，称为低字节存储体，该存储体中的每个地址均为偶地址；高位库与 CPU 的高位字节数据线 D_{15}~D_8 相连，称为高字节存储体，该存储体中的每个地址均为奇地址。两个存储体之间采用字节交叉编址方式，如图 2-7 所示。

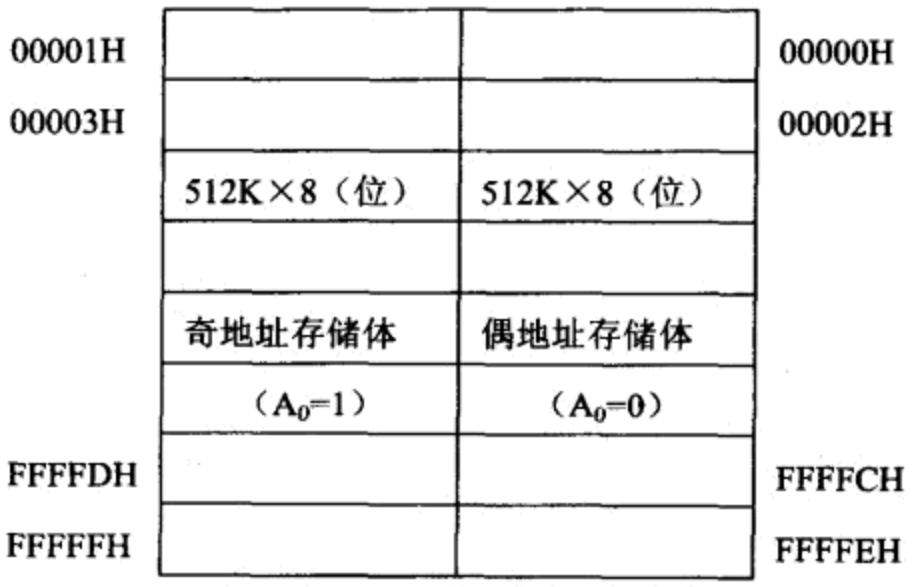


图 2-7　8086 存储器的分体结构

对于任何一个存储体，只需要 19 位地址码 A_{19}~A_1 就够了，最低位地址码 A_0 用以区分当前访问哪一个存储体。当 A_0=0，表示访问偶地址存储体；A_0=1，表示访问奇地址存储体。

8086 系统设置了一个总线高位有效控制信号 $\overline{BHE}$。$\overline{BHE}$ 与 A_0 相互配合，使得 CPU 可以访问两个存储体中的一个字信息。$\overline{BHE}$ 和 A_0 的组合控制作用如表 2-4 所示。

表 2-4　$\overline{BHE}$ 和 A_0 的组合控制作用

$\overline{BHE}$	A_0	操 作 功 能
0	0	同时访问两个存储体，读/写一个对准字信息
0	1	只访问奇地址存储体，读/写高字节信息
1	0	只访问偶地址存储体，读/写低字节信息
1	1	无操作

两个存储体与 CPU 总线之间的连接如图 2-8 所示。奇地址存储体的片选端 $\overline{SEL}$ 受控于 $\overline{BHE}$ 信号，偶地址存储体的片选端受控于地址线 A_0。

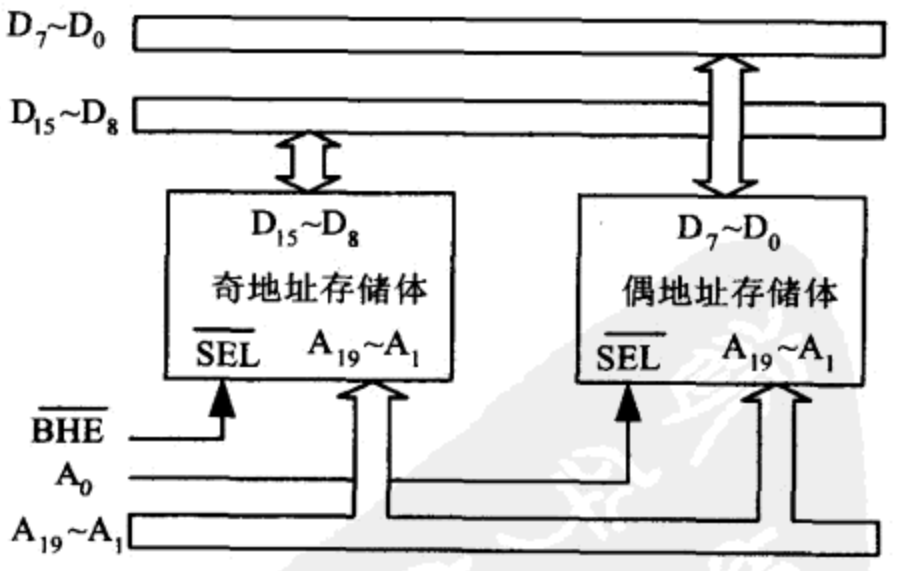


图 2-8　存储体与总线的连接示意图

当访问存储器中某个字节时：指令中的地址码经变换后得到 20 位的物理地址。如果是偶地址（A_0=0，$\overline{BHE}$=1），这时可由 A_0 选定偶地址存储体，A_{19}~A_1 从偶地址存储体中选定某个字节地址，并启动该存储体，读/写该地址中一个字节的信息，通过数据总线的低 8 位传送数据；如果是奇地址（A_0=1），则偶地址存储体不会被选中，系统将自动产生 $\overline{BHE}$=0，作为奇地址存储体的选择信号，与 A_{19}~A_1 一起选定奇地址存储体中的某个字节地址，并读/写该地址中一个字节的信息，通过数据总线的高 8 位传送数据。

当访问存储体中某个字时：一种情况是需要访问从偶地址开始的一个字（即高字节在奇地址中，低字节在偶地址中），可一次访问存储器读/写一个字信息，这时 A_0=0，$\overline{BHE}$=0。另一种情况是需要访问从奇地址开始的一个字（即高字节在偶地址中，低字节在奇地址中），这时需要访问两次存储器才能读/写这个字的信息，第一次访问存储器读/写奇地址中的字节，第二次访问存储器读/写偶地址中的字节。显然，为了加快程序的运行速度，希望访问存储器的字地址为偶地址。

2. 存储器分段

在 8086 系统中，可寻址的存储空间达 1MB。要对整个存储器空间寻址，则需要 20 位长的地址码，而 8086 系统内所有的寄存器都只有 16 位，只能寻址 64KB（2^{16}B）。因此把整个存储空间分成许多逻辑段，这些逻辑段容量最多为 64KB。8086 微处理器允许它们在整个存储空间中浮动，各个逻辑段之间可以紧密相连，也可以相互重叠，还可以分开一段距离，如图 2-9 所示。

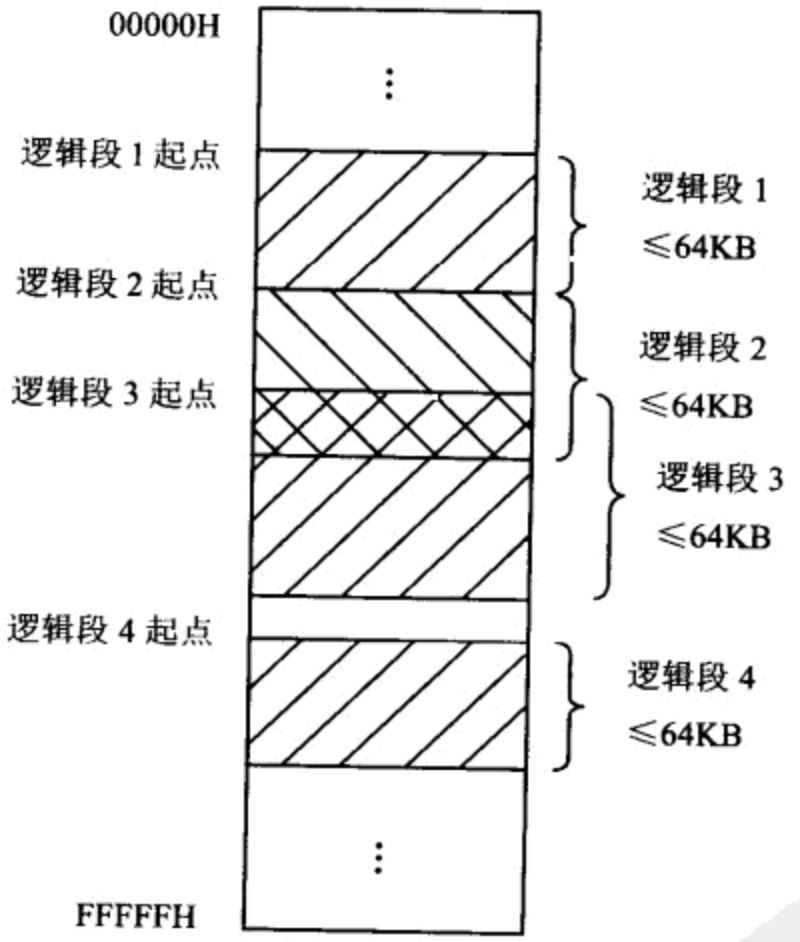


图 2-9　存储器分段示意图

16 字节的存储空间称为一节（Paragraph），一般要求各个逻辑段从字节的整数边界开始，即尽量保证段起始地址的低 4 位地址码为“0”。

存储器采用分段编码方法进行组织，带来了一系列的好处。首先，程序中的指令只涉及16位地址，缩短了指令长度，提高了程序执行的速度。尽管8086的存储器空间多达1MB，但在程序执行过程中，不需要在1MB空间中去寻址，多数情况下只在一个较小的存储器段中运行。而且大多数指令运行时，并不涉及段寄存器的值，只涉及16位的偏移量。

3. 存储器地址

8086在访问存储器时管理1MB的内存空间，其20根地址线都有效；在访问外设时管理64 KB的I/O端口空间，仅16根地址线有效。下面对用到的各类地址进行描述。

（1）段地址（Segment Address）：为一个无符号数的16位二进制数，描述要寻址的逻辑段在内存中的起始位置。段地址保存在16位的CS、SS、DS和ES段寄存器中。

（2）偏移地址（Offset Address）：为无符号数的16位二进制数，描述要寻址的内存单元距本段段首的偏移量。在编程中常被称做“有效地址EA（Effective Address）”。

（3）逻辑地址（Logic Address）：为无符号的16位二进制数，是在程序中使用的地址，由段地址和偏移地址两部分组成。表示形式为“段地址：偏移地址”。

（4）物理地址（Physical Address）：是存储器的实际地址，由CPU提供的20位地址码来表示，是唯一能代表存储空间每个字节单元的地址。

逻辑地址到物理地址的转换由BIU中20位的地址加法器自动完成，如图2-10所示。实际上物理地址是段地址左移4位加偏移地址形成的。

其计算公式为：

物理地址=段地址×10H＋偏移地址

访问存储器时，段地址是由段寄存器提供的。8086通过4个段寄存器来访问不同的段。用程序对段寄存器的内容进行修改，可实现访问所有段。

不同的操作使用的段地址和偏移地址的来源不同，表2-5给出了各种访问存储器操作所使用的段寄存器和段内偏移地址的来源。

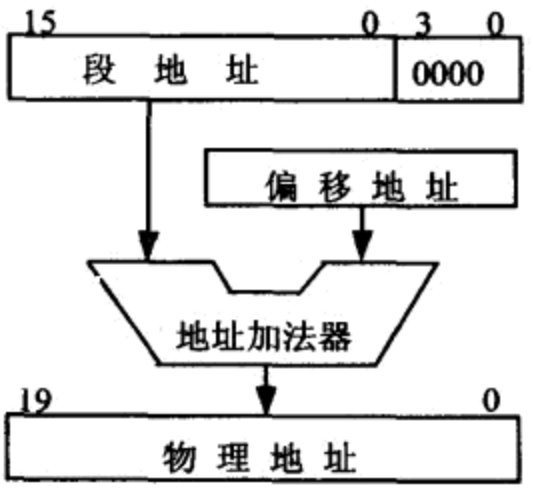


图2-10 物理地址的形成过程

一般情况下，段寄存器的作用由系统约定，有些操作除了约定的段寄存器外，还可指定其他段寄存器。如通用数据存取，除由约定的DS给出段基址外，还可指定CS、SS和ES；有些操作只能使用约定的段寄存器，不允许指定其他段寄存器，如取指令只使用CS。

表2-5 各种访问存储器的段地址和偏移地址

类 型	约定的段寄存器	可指定的段寄存器	偏移地址
取指令	CS	无	IP
堆栈操作	SS	无	SP
串指令（源）	DS	CS、ES、SS	SI
串指令（目的）	ES	无	DI
用BP作基址	SS	CS、ES、SS	有效地址EA
通用数据读/写	DS	CS、ES、SS	有效地址EA

4. 专用和保留的存储器单元

Intel 公司为了保证与未来的产品的兼容性，规定在存储区的最低地址区和最高地址区保留一些单元供 CPU 的某些特殊功能专用。其中主要有如下功能。

（1）00000H~003FFH（共 1KB）：存放中断向量表，每个中断向量占 4 个字节，前 2 个字节存放中断处理服务程序入口的偏移地址，后 2 个字节存放中断服务程序入口的段地址。因此，1KB 区域可以存放 256 个中断处理服务程序的入口地址。

（2）B0000H~B0FFFH（共 4KB）：单色显示器的视频缓冲区，存放单色显示器当前屏幕显示字符所对应的 ASCII 码及其属性。

（3）B8000H~BBFFFH（共 16KB）：彩色显示器的视频缓冲区，存放彩色显示器当前屏幕像素点所对应的代码。

（4）FFFF0H~FFFFFH（共 16B）：存放一条无条件转移指令，使系统在上电或复位时，会自动跳转到系统的初始化程序。这个区域被包含在系统的 ROM 范围内，在 ROM 中驻留着系统的基本 I/O 系统程序，即 BIOS。

2.3.2　I/O 端口组织

8086 微处理器和外部设备之间是通过 I/O 接口电路进行联系，以达到相互间传输信息的目的。每个 I/O 接口都有一个端口或几个端口，所谓端口是指 I/O 接口电路中供 CPU 直接存取访问的那些寄存器或某些特定电路。一个 I/O 接口总要包括若干个端口，如数据、命令、状态、方式端口等，微机系统要为每个端口分配一个地址号，称为端口地址。各个端口地址和存储单元地址一样，应具有唯一性。

8086 微处理器用地址总线的低 16 位作为对 8 位 I/O 端口的寻址线，所以 8086 微处理器可访问的 8 位 I/O 端口有 65 536（2^{16}）个。两个编号相邻的 8 位端口可以组成一个 16 位的端口。一个 8 位的 I/O 设备既可以连接在数据总线的高 8 位上，也可以连接在数据总线的低 8 位上。

8086 的 I/O 端口有以下两种编址方式。

1. 统一编址

统一编址也称“存储器映射方式”。在这种编址方式下，端口和存储单元统一编址，即将 I/O 端口地址置于 1MB 的存储器空间中，在整个存储空间中划出一部分空间给外设端口，把它们看作存储器单元对待。CPU 访问存储器的各种寻址方式都可用于寻址端口，访问端口和访问存储器的指令在形式上完全一样。

统一编址的主要优点是无需专门的 I/O 指令，对端口操作的指令类型多，从而简化了指令系统的设计。不仅可以对端口进行数据传送，还可以对端口内容进行算术/逻辑运算和移位等操作，端口操作灵活，有比较大的编址空间。缺点是端口占用存储器的地址空间，使存储器容量更加紧张，同时端口指令的长度增加，执行时间较长，端口地址译码器较复杂。

2. 独立编址

独立编址也称“I/O 映射方式”。这种方式的端口单独编址构成一个 I/O 空间，不占用存储器地址，故称“独立编址”方式。CPU 设置了专门的输入和输出指令（IN 和 OUT）来访问端口。

8086 使用 A_{15}~A_0 这 16 条地址线作为端口地址线，可访问的 I/O 端口最多可达 64K 个 8 位端口或 32K 个 16 位端口。在这种方式下，端口所需的地址线较少，地址译码器较简单，

采用专用的 I/O 指令，执行时间少，指令长度短。端口操作指令形式上与存储器操作指令有明显区别，使程序编制与阅读较清晰。缺点是输入/输出指令类别少，一般只能进行传送操作。

在采用独立编址方式时，CPU 必须提供控制信号以区别是寻址内存还是寻址 I/O 端口。8086 微处理器在执行访问存储器指令时，M/$\overline{IO}$ 信号为高电平，通知外部电路 CPU 访问存储器，当 8086 微处理器执行输入/输出指令时，M/$\overline{IO}$ 为低电平，以表示 CPU 在访问 I/O 端口。

2.4 8086 微处理器的总线周期和工作方式

8086 微处理器由外部的一片 8284A 时钟信号发生器提供主频为 5MHz 的时钟信号，在时钟节拍作用下，CPU 一步一步顺序地执行指令，因此，时钟周期是 CPU 指令执行时间的刻度。执行指令的过程中，凡需执行访问存储器和访问 I/O 端口的操作都统一交给 BIU 的外部总线完成，每一次访问都称为一个“总线周期”。若执行的是数据输出，称为“写总线周期”，若执行的是数据输入，称为“读总线周期”。

2.4.1 8284A 时钟信号发生器

8284A 是 Intel 公司专为 8086 设计的时钟信号发生器，除提供恒定的时钟信号外，还对外界输入的就绪信号 RDY 和复位信号 $\overline{RES}$ 进行同步。8284A 的引脚特性如图 2-11 所示。

8284A 的工作原理为：当外界就绪信号 RDY 输入 8284A，经时钟下降沿同步后，输出 READY 信号作为 8086 的就绪信号 READY；外界的复位信号 $\overline{RES}$ 输入 8284A，经整形并由时钟下降沿同步后，输出 RESET 信号作为 8086 的复位信号 RESET，其宽度不得小于 4 个时钟周期。外界的 RDY 和 $\overline{RES}$ 可以在任何时候发出，但送至 CPU 的信号都是经时钟同步后的信号。

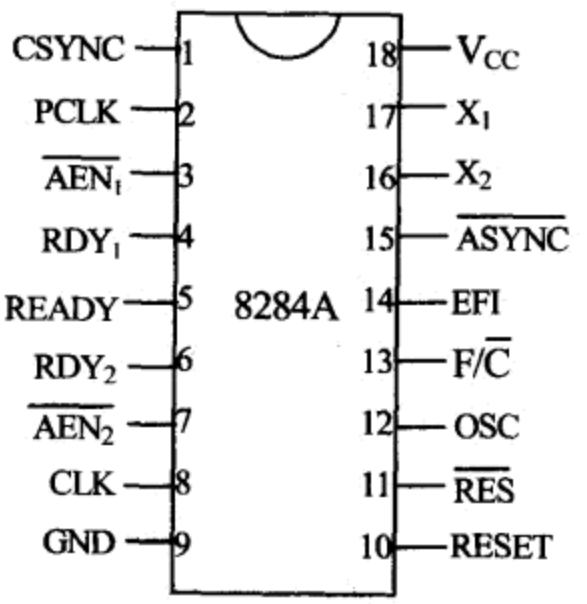


图 2-11 8284A 引脚特性

根据不同的振荡源，8284A 有两种不同的连接方法。

（1）采用脉冲发生器作为振荡源，这时只需将脉冲发生器的输出端和 8284A 的 EFI 端相连即可，同时将 F/$\overline{C}$ 接为高电平。

（2）采用石英晶体振荡器作为振荡源，这时只需将晶体振荡器连在 8284A 的 X_1 和 X_2 两端，同时将 F/$\overline{C}$ 接地。

不管用哪种方法，8284A 输出的时钟 CLK 的频率均为振荡源频率的 1/3，振荡源频率经 8284A 驱动后，由 OSC 端输出，可供系统使用。

2.4.2 8086 总线周期

8086 微处理器与存储器或外部设备通信，是通过 20 位分时多路复用地址/数据总线来实现的。为了取出指令或传输数据，CPU 要执行一个总线周期。8086 微处理器经外部总线对存储器或 I/O 端口进行一次信息的输入或输出过程，称为总线操作。执行该操作所需要的时间，称为总线周期。由于总线周期全部由 BIU 来完成，所以也把总线周期称为 BIU 总线周期。

8086 的总线周期至少由 4 个时钟周期组成。每个时钟周期称为 T 状态，用 T_1、T_2、T_3 和 T_4 表示。时钟周期是 CPU 的基本时间计量单位，由主频决定。例如 8086 的主频为 5MHz，1 个时钟周期就是 100ns。典型的总线周期波形如图 2-12 所示。

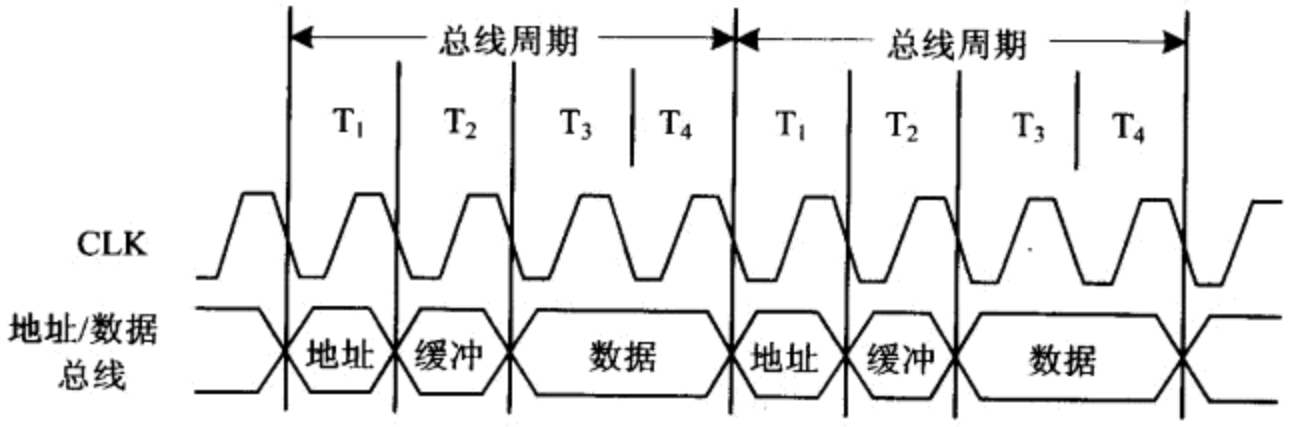


图 2-12　典型的 8086 总线周期波形图

在 T_1 状态期间，CPU 将存储地址或 I/O 端口的地址置于总线上。若要将数据写入存储器或 I/O 设备，则在 T_2~T_4 这段时间内，要求 CPU 在总线上一直保持要写的数据；若要从存储器或 I/O 设备读入信息，则 CPU 在 T_3~T_4 期间接收由存储器或 I/O 设备置于总线上的信息。T_2 时总线浮空，允许 CPU 有个缓冲时间，把输出地址的写方式转换为输入数据的读方式。可见，AD_0~AD_{15} 和 A_{16}/S_3~A_{19}/S_6 在总线周期的不同状态传送不同的信号，这就是 8086 的分时多路复用地址/数据总线。

BIU 只在下列情况下，执行一个总线周期。

（1）在指令的执行过程中，根据指令的需要，由 EU 请求 BIU 执行一个总线周期。例如，取操作数或存放指令执行结果等。

（2）当指令队列寄存器已经空出 2 个字节，BIU 必须填写指令队列的时候。

这样，在这两种总线操作周期之间，就有可能存在着 BIU 不执行任何操作的时钟周期。

1. 空闲状态 T_I（Idle State）

总线周期只用于 CPU 和存储器（或 I/O 端口）之间传送数据和填充指令队列。若在两个总线周期之间，存在着 BIU 不执行任何操作的时钟周期，那么这些不起作用的时钟周期称为空闲状态，用 T_I 表示。在系统总线处于空闲状态时，可以包含 1 个或多个时钟周期。这期间在高 4 位的总线上，CPU 仍然输出前一个总线周期的状态信号 S_3~S_6；而在低 16 位总线上，则视前一个总线周期是写周期还是读周期来确定。若前一个总线周期为写周期，CPU 会在总线的低 16 位继续输出数据信息；若前一个总线周期为读周期，CPU 则使总线的低 16 位处于浮空状态。

引起空闲状态有多种情况。例如：当 8086 微处理器把总线的主控权交给协处理机的时候；当 8086 执行一条长指令（如 16 位的乘法指令 MUL 或除法指令 DIV）时，BIU 有相当长的一段时间不执行任何操作，其时钟周期处于空闲状态。

2. 等待状态 T_W（Wait State）

8086 微处理器与慢速的存储器和 I/O 接口交换信息时，为了防止丢失数据，会由存储器或外设通过 READY 信号线，在总线周期的 T_3 和 T_4 之间插入 1 个或多个必要的等待状态 T_W，用来给予必要的时间补偿。在等待状态期间，总线上的信息保持 T_3 状态时的信息不变，其他一些控制信号也都保持不变。包含了 T_I 与 T_W 状态的典型总线周期，如图 2-13 所示。

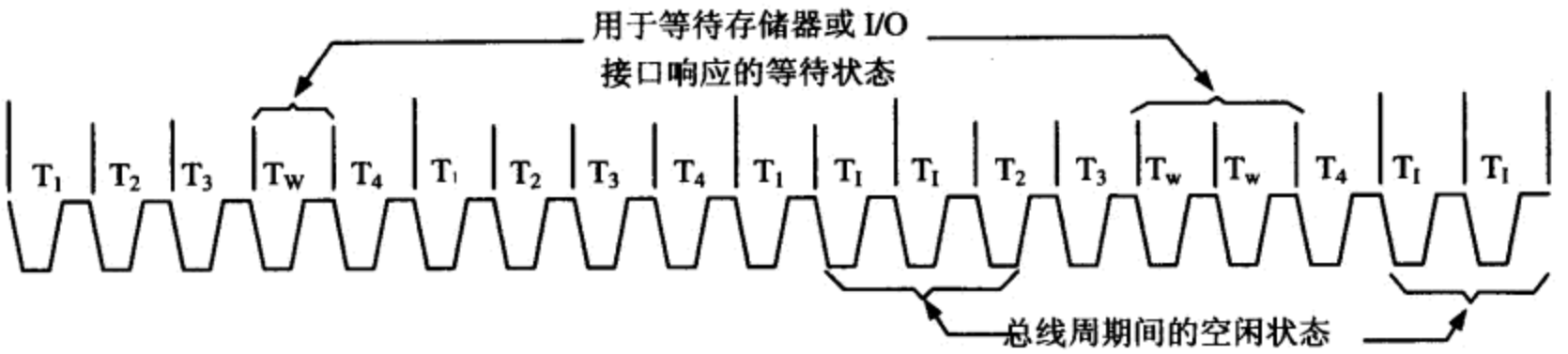


图 2-13　典型的总线周期序列

当存储器或外设完成数据的读/写准备时，便在 READY 线上发出有效信号，CPU 接到此信号后，会自动脱离 T_W 而进入 T_4 状态。

2.4.3　8086 微处理器的最小/最大工作方式

通常，系统总线上所挂接的存储器、I/O 接口等部件越多，计算机的功能就越强，规模就越大。为了构成不同规模的微型计算机，适应各种各样的应用场合，Intel 公司在设计 8086 微处理器芯片时，规定了可以在两种模式下工作，即最小工作方式和最大工作方式。通过第 33 条引脚 MN/$\overline{MX}$ 来控制：最小工作方式时 MN/$\overline{MX}$ =1，适用于单微处理器组成的小系统；最大工作方式时 MN/$\overline{MX}$ =0，适用于多微处理器组成的大系统。

1. 最小工作方式

最小工作方式是指系统中只有 8086 一个微处理器，是一个单微处理器系统。在这种系统中，所有的总线控制信号都直接由 8086 微处理器产生，总线控制逻辑电路被减到最少。

最小工作方式时，8086 微处理器中 8 条控制引脚 24~31 的功能定义如下。

（1）中断响应信号 $\overline{INTA}$（输出，低电平有效）：用于 CPU 对外设的中断请求作出响应。8086 的 $\overline{INTA}$ 信号实际上是两个连续的负脉冲，在每个总线周期的 T_2、T_3 和 T_W 状态下，$\overline{INTA}$ 为低电平。其第 1 个负脉冲通知外设接口，它发出的中断请求已经得到允许；第 2 个负脉冲期间，外设接口往数据总线上放中断类型码，从而使 CPU 得到了有关此中断请求的详尽信息。

（2）地址锁存信号 ALE（输出，高电平有效）：在任何一个总线周期的时钟周期 T_1 时，ALE 输出高电平，以表示当前在地址/数据复用总线上输出的是地址信息，地址锁存器 8282/8283 将 ALE 作为锁存信号，对地址进行锁存。

（3）数据允许信号 $\overline{DEN}$（输出，三态，低电平有效）：$\overline{DEN}$ 作为总线收发控制器 8286/8287 的选通信号，在 CPU 访问存储器或 I/O 端口的总线周期的后半段时间内该信号有效，表示 CPU 准备好接收或发送数据，允许数据收发器工作。

（4）数据发送/接收控制信号 DT/$\overline{R}$（输出，三态）：用于在系统使用 8286/8287 作为数据总线收发器时，控制其数据传送的方向，如果 DT/$\overline{R}$ 为高电平，则进行数据发送，否则进行数据接收。

（5）存储器/输入输出信号 M/$\overline{IO}$（输出，三态）：用来表示 CPU 是访问存储器还是访问输入/输出设备，信号接至存储器芯片或 I/O 接口芯片的片选端。高电平时表示 CPU 要访问存储器；低电平时表示 CPU 访问 I/O 端口。

（6）控制信号 $\overline{WR}$（输出，低电平有效，三态）：该信号有效时表示 CPU 正在对存储器或 I/O 端口执行写操作。在任何写周期，$\overline{WR}$ 只在 T_2、T_3 和 T_W 有效。

（7）总线保持请求信号 HOLD（输入，高电平有效）：是系统中其他总线主控部件（如协处理器、DMA 控制器等）向 CPU 发出的请求占用总线的控制信号。当 CPU 从 HOLD 线上收到请求信号时，如果 CPU 允许让出总线，就在当前总线周期完成时，在 T_4 状态使 HLDA 输出高电平作为响应信号，且同时使地址/数据总线和控制总线处于浮空状态。总线请求部件收到 HLDA 信号后获得总线控制权，HOLD 和 HLDA 都保持高电平。当请求部件完成对总线的占用后，将把 HOLD 信号变为低电平，CPU 收到该无效信号后也将 HLDA 变为低电平，从而恢复对地址/数据总线和控制总线的占有权。

（8）总线保持响应信号 HLDA（输出，高电平有效）：在 HLDA 信号有效期间，CPU 让出总线控制权，总线请求部件收到 HLDA 信号后，获得总线控制权。这时，CPU 使地址/数据总线与所有具有三态的控制线都处于高阻隔离状态，CPU 处于“保持响应”状态。

2. 最大工作方式

最大工作方式主要用在中等或大规模的 8086 系统中。该方式是多微处理器系统，其中必有一个主处理器 8086，其他的处理器称为协处理器。和 8086 匹配的协处理器主要有两个：一个是专用于数值运算的处理器 8087，另一个是专用于输入/输出处理的协处理器 8089。此时 8086 不直接提供用于存储器或 I/O 端口读/写命令等控制信号，而是由总线控制器 8288 产生相应的控制信号。最大工作方式时 CPU 的 24~31 控制引脚的功能重新定义如下。

（1）总线周期状态信号 $\overline{S_2}$、$\overline{S_1}$、$\overline{S_0}$（输出，三态）：表示 CPU 总线周期的操作类型。使用总线控制器 8288 后 CPU 就可以对 $\overline{S_2}$、$\overline{S_1}$、$\overline{S_0}$ 状态信息进行译码，产生相应的控制信号。$\overline{S_2}$、$\overline{S_1}$、$\overline{S_0}$ 对应的总线操作和 8288 产生的控制命令如表 2-6 所示。

表 2-6　$\overline{S_2}$、$\overline{S_1}$、$\overline{S_0}$ 与总线操作、8288 控制命令的对应关系

状态输入			CPU 总线操作	8288 控制命令
$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$		
0	0	0	中断响应	$\overline{INTA}$
0	0	1	读 I/O 端口	$\overline{IORC}$
0	1	0	写 I/O 端口	$\overline{IOWC}$、$\overline{AIOWC}$
0	1	1	暂停	无
1	0	0	取指令周期	$\overline{MRDC}$
1	0	1	读存储器周期	$\overline{MRDC}$
1	1	0	写存储器周期	$\overline{MWTC}$、$\overline{AMWC}$
1	1	1	无源状态	无

（2）指令队列状态信号 QS_1、QS_0（输出）：用来提供 8086 内部指令队列的状态。内部在执行当前指令的同时从存储器预先取出后面的指令，并将其放在指令队列中。由 QS_1、QS_0 提供指令队列的状态信息，以便跟踪 8086 内部指令序列。QS_1、QS_0 表示的状态情况如表 2-7 所示。

表 2-7 QS_1、QS_0 与队列状态

QS_1	QS_0	队列状态
0	0	无操作，队列中指令未被取出
0	1	从队列中取出当前指令的第 1 个字节
1	0	指令队列空
1	1	从队列中取出当前指令的第 2 字节以后部分

（3）总线封锁信号 $\overline{LOCK}$（输出，三态）：当 $\overline{LOCK}$ 为低电平时，表示 CPU 要独占总线。为了保证 CPU 在一条指令的执行中总线使用权不会被其他主设备打断，可以在指令的前面加上一个 $\overline{LOCK}$ 前缀。这条指令执行时就会使 CPU 产生一个 $\overline{LOCK}$ 信号，CPU 封锁其他主控设备使用总线，直到这条指令结束为止。

（4）总线请求/总线请求允许信号 $\overline{RQ}/\overline{GT_1}$、$\overline{RQ}/\overline{GT_0}$（双向）：用于裁决总线使用权，可供 CPU 以外的两个总线主控设备发出使用总线的请求信号，或接收 CPU 对总线请求信号的响应信号。该信号为输入时，表示其他设备向 CPU 发出请求使用总线；该信号为输出时，表示 CPU 对总线请求的响应信号。总线请求信号和允许信号在同一引线上传输，但方向相反。其中 $\overline{RQ}/\overline{GT_0}$ 比 $\overline{RQ}/\overline{GT_1}$ 的优先级要高。

总线请求和允许的操作过程如下：另一总线主控设备输送一个脉冲给 8086，表示总线请求，在 CPU 的下一个 T_4 或 T_1 期间，CPU 输出一个脉冲给请求总线的设备，作为总线响应信号。当总线使用完毕后，总线请求主设备输出一个脉冲给 CPU，表示总线请求的结束，每次操作都需要这样的 3 个脉冲。

2.5 32 位微处理器简介

2.5.1 80386 微处理器

1985 年 10 月 Intel 公司推出高性能 32 位微处理器 80386，该芯片内部集成有 27.5 万个晶体管，整个芯片采用 132 引脚的陶瓷网格阵列（PGA）封装．具有高可靠性和紧密性。

1. 80386 的主要特性

80386 主要有如下特性。

（1）提供 32 位指令，支持 8 位、16 位和 32 位的数据类型，具有 8 个通用 32 位寄存器，ALU 和内部总线的数据通路均为 32 位，具有片内地址转换的高速缓冲存储器 Cache。

（2）提供 32 位外部总线接口，最大数据传输速率为 32Mb/s。由于采用了流水线方式，可同高速 DRAM 芯片接口，支持动态总线宽度控制，能动态地切换 32 位/16 位数据总线。

（3）具有片内集成存储器管理部件 MMU，可支持虚拟存储和特权保护，保护机构采用 4 级特权层，可选择片内分页单元。片内具有多任务机构，能快速完成任务的切换。

（4）具有实地址方式、保护方式和虚拟 8086 三种工作方式。实地址方式和虚拟 8086 方式与 8086 相同，保护方式可支持虚拟存储、保护和多任务。

（5）可直接寻址 4GB（232）的物理存储空间，虚拟存储空间达 64TB。存储器采用分段结构，一个段最大可为 4GB。

（6）通过配用 80287、80387 数值协处理器可支持高速数值处理。

（7）时钟频率为 12.5MHz、16MHz、20MHz、25MHz 和 33MHz 等。

2. 80386 的内部结构

80386 微处理器的内部结构如图 2-14 所示，由总线接口部件、指令预取部件、指令译码部件、控制部件、数据部件、保护测试部件、分段部件和分页部件等 8 个功能部件组成。控制部件、数据部件和保护测试部件共同组成执行部件，分段部件和分页部件合在一起称为存储器管理部件。

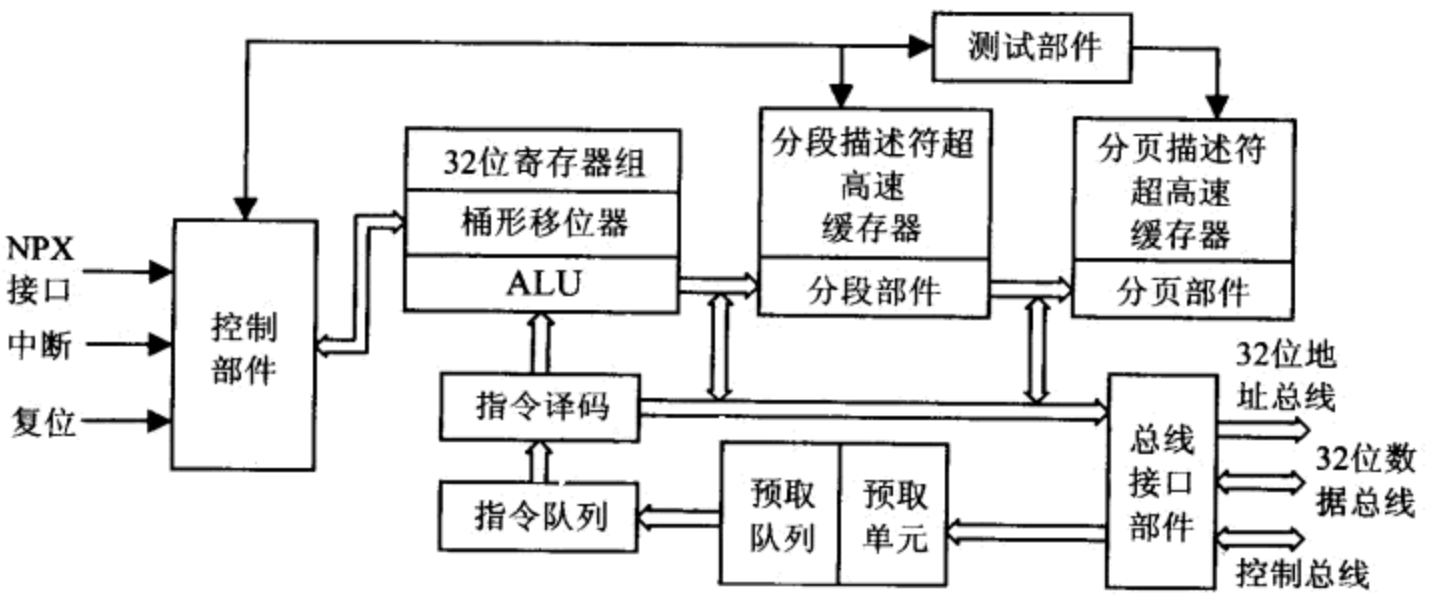


图 2-14　80386 微处理器的内部结构框图

主要部件的功能分析如下。

（1）总线接口部件：提供 CPU 和系统之间的高速接口，负责外部总线与内部部件之间的信息交换。此外产生一些执行 CPU 总线周期所需要的信号，包括地址、数据、控制信号等，用来与存储器和输入/输出部件通信。

（2）指令预取部件：负责从存储器中取出指令并存放在 16 字节的指令队列中。它还管理一个线性地址指针和一个段预取界限，这两项内容是从分段部件获得的，分别作为预取指令指针和检查是否违反分段界限。

（3）指令译码部件：负责对指令进行译码，可以完成从指令到微指令的转换，译码后的指令放在译码器的指令队列中，供执行部件使用。

（4）执行部件：负责执行指令，其中控制部件用于控制 ROM；译码器给控制部件提供微代码的起始地址，控制部件按此微代码执行相应的操作；数据部件包括寄存器组和算术/逻辑部件，负责进行算术/逻辑运算。

（5）存储器管理部件：由分段部件和分页部件组成。分段部件将逻辑地址按执行部件的要求变换成线性地址，实现有效地址的计算；分页部件提供了对物理地址空间的管理，将物理地址送给总线接口部件，执行存储器或 I/O 存取。

2.5.2　80486 微处理器

80486 是 Intel 公司 1989 年 4 月推出的，它采用 1μm CHMOS 工艺，芯片内集成了 120 万个晶体管，时钟频率为 25~50MHz。寄存器仍为 32 位，数据总线和地址总线也皆为 32 位。

80486 在 80386 原有 6 个部件的基础上，又新增了两个部件：高性能浮点运算部件 FPU 和高速缓冲存储器 Cache。它把浮点运算部件和高速缓冲存储器 Cache 集成在芯片内，使运算速度和数据存取速度得到大大提高。

1. 80486 的主要特性

80486 以提高速度和支持多处理器机构为目标，采用了容易实现多处理器的硬件部件，增加了在禁止其他处理器访问的同时，访问并更改共享存储器的指令。

（1）在 CISC（复杂指令集计算机）技术的基础上，首次采用了 RISC（精简指令集计算机）技术，有效地减少了指令的时钟周期个数。

（2）芯片上集成部件多。包括了 8KB 的指令和数据高速缓存、浮点运算部件、分页虚拟存储管理和 80387 数值协处理器等多个部件，并且集 Cache 与 CPU 为一体，提高了微处理器的处理速度。

（3）高性能的设计。在以主频 33MHz 工作时，8KB 的指令和数据兼用的高速缓冲存储器与 106Mb/s 的猝发总线传输率相结合，确保高速的系统处理能力。由于 80486 采用了猝发式总线与内存进行高速数据交换，从而大大加快了微处理器与内存交换数据的速度。

（4）完全的 32 位体系结构。地址和数据总线均为 32 位，寄存器也是 32 位。

（5）增加了多处理器指令，增强了多重处理系统，片上硬件确保了超高速缓存一致性协议，并支持多级超高速缓存结构。

（6）具有机内自测试功能，可以广泛地测试片上逻辑电路、超高速缓存和片上分页转换高速缓存。

2. 80486 的基本结构

80486 微处理器的内部结构如图 2-15 所示，包括总线接口、片内高速缓冲存储器 Cache、指令预取、指令译码、控制/保护、整数、浮点运算、分段和分页等 9 个功能部件。80486 将这些部件集成在一块芯片上，除减少主板空间外，还提高了 CPU 的执行速度。

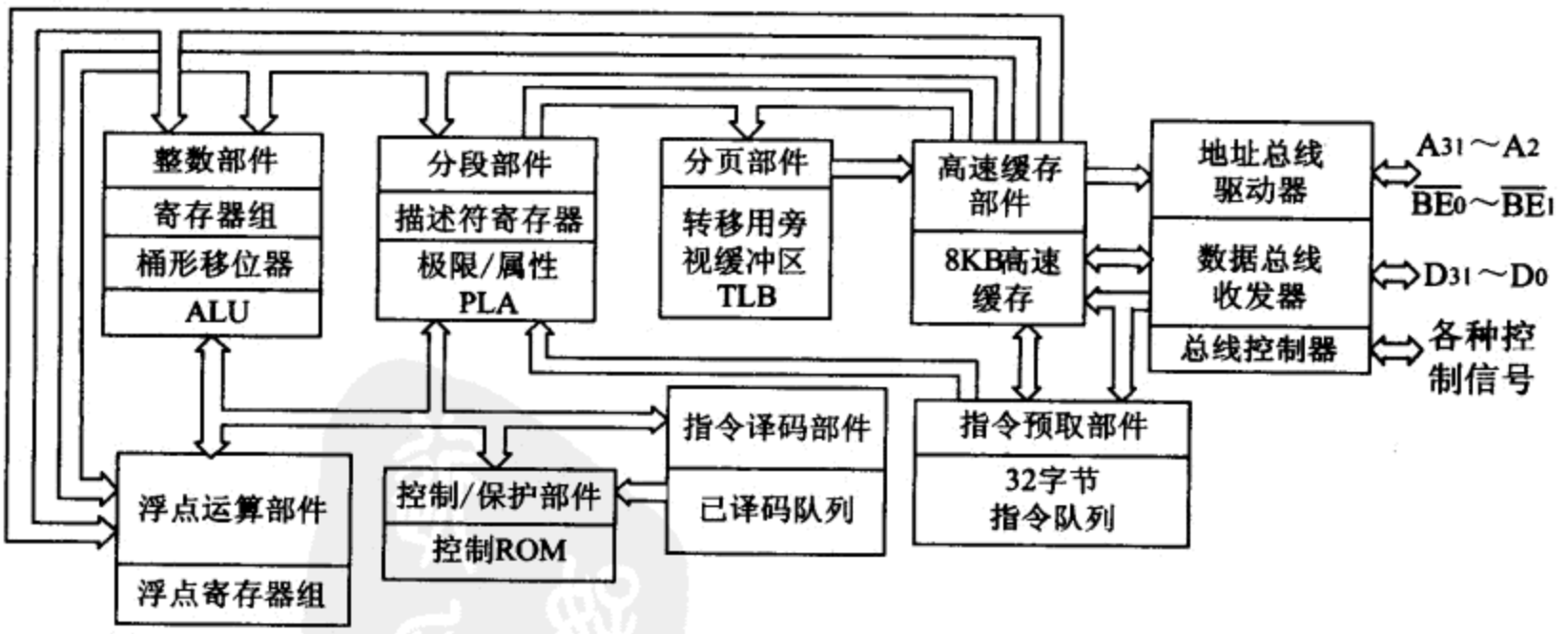


图 2-15　80486 微处理器内部结构

各主要部件的功能分析如下。

（1）浮点运算部件 FPU：由指令接口、数据接口、运算控制单元、浮点寄存器和浮点运算

器组成，可以处理一些超越函数和复杂的实数运算，以极高的速度进行单精度或双精度的浮点运算。

（2）总线接口部件：根据优先级的高低协调数据传输、指令预取操作。在内部，它通过 3 个 32 位总线与指令预取部件和高速缓存进行通信；在外部，它产生微处理器总线周期所必需的各种信号。总线接口部件还配备一个暂时存储器，用来暂时存放要写到主存储器中的 4 个 32 位数据，它起缓冲器的作用。

（3）指令预取部件：先到 Cache 中去取几条要用的指令，如果在 Cache 中没有找到所需的指令，就到主存储器中去取这几条指令。80486 有 32 个字节的指令队列，取出的指令放在指令队列中。

（4）指令部件：用于把从预取队列中取出的指令转换成低级的控制信号和微代码指令入口。译码的指令存放在指令队列中，一旦控制器发出请求，就将其发送给控制/保护部件。

（5）控制/保护部件：是把指令转换成微代码指令，这些微代码指令通过内部总线直接送入各执行部件去执行。它负责解释指令译码器收到的控制信号和微代码入口，并根据译码后的指令来指挥整数部件和浮点部件、存储器管理部件等的一切活动。

（6）整数部件：包括 ALU、桶形移位器、寄存器组等部分。负责执行控制器指定的全部算术和逻辑运算，可在一个时钟周期内执行加载、存储、加减、逻辑和移位等单条指令。

（7）存储器管理部件 MMU：由分段部件 SU 和分页部件 PU 组成。通过其中的分段部件将每一个内部逻辑地址转换成线性地址，再由分页部件将线性地址转换成物理地址。

2.5.3　Pentium 系列微处理器

Pentium 系列微处理器从 Pentium、Pentium Pro、Pentium MMX 到 Pentium Ⅱ、Pentium Ⅲ、Pentium 4 等，Intel 公司通过改变 CPU 的工作频率、二级缓存的大小、产品制造工艺等来不断提高微处理器的性能，内部结构和功能也在不断地扩充。

1. Pentium 系列微型计算机的主要特点

（1）高集成度，片内集成有 310 万个晶体管。

（2）时钟频率高，从 60MHz 或 66MHz 发展到 500MHz，700MHz 和 1 500MHz。

（3）数据总线带宽增加，内部总线为 32 位，外部数据总线宽度为 64 位。

（4）片内采用分立的指令 Cache 和数据 Cache 结构，可无冲突地同时完成指令预取和数据读/写。

（5）采用 RISC 型超标量结构。超标量是指微处理器内具有多条指令执行流水线，以增加每个时钟周期内可以执行的指令数，从而使微处理器的运行速度成倍提高。

（6）高性能的浮点运算器。Pentium 采用全新设计的增强型浮点运算器（FPU），即 FPU 采用了超级流水线技术，使得它的浮点运算速度比 80486DX 要快 3~5 倍。

（7）双重分离式高速缓存。将指令高速缓存与数据高速缓存分离，各自拥有独立的 8KB 高速缓存。而且数据高速缓存采用回写方式，以适应共享主存储器多机系统的需要，抑制存取总线次数，使其能全速执行，减少等待及传送数据时间。

（8）增强了错误检测与报告功能。内部增强了错误检测与报告功能，特别引进了片功能冗余检测（FRC），并采用了一种能降低出错的六晶体管存储单元。

（9）64 位数据总线。Pentium 为了大幅度提高数据传输速度而使用 64 位的数据总线。

（10）分支指令预测。处理器内部采用了分支预测技术，大大提高了流水线执行效率。

（11）常用指令固化及微代码改进。把一些常用的指令（如 MOV、INC、DEC、PUSH 等）改用硬件实现，不再使用微代码操作，使指令执行速度进一步提高。

（12）系统管理方式。具有实地址方式、保护方式、虚拟 8086 方式及具有特色的 SMM（系统管理方式）。与其他高性能微处理器一样，复位时自动进入实地址方式，可以通过机器内部的装有系统级程序代码的 ROM 来控制，并可以从一种方式切换到另一种方式。

（13）软件向上兼容 80386/80486，可以在 MS-DOS，Windows 95，Windows NT，OS/2，UNIX 和 Solaris 等操作系统下运行。

2. Pentium 微处理器的内部结构

Pentium 微处理器的主要部件包括总线接口部件、指令高速缓存器、数据高速缓存器、指令预取部件（指令预取缓冲器）与转移目标缓冲器、寄存器组、指令译码部件、具有两条流水线的整数处理部件（U 流水线和 V 流水线）、拥有加乘除运算且具有多用途电路的流水浮点处理部件 FPU 等。Pentium 微处理器的内部结构框图如图 2-16 所示。

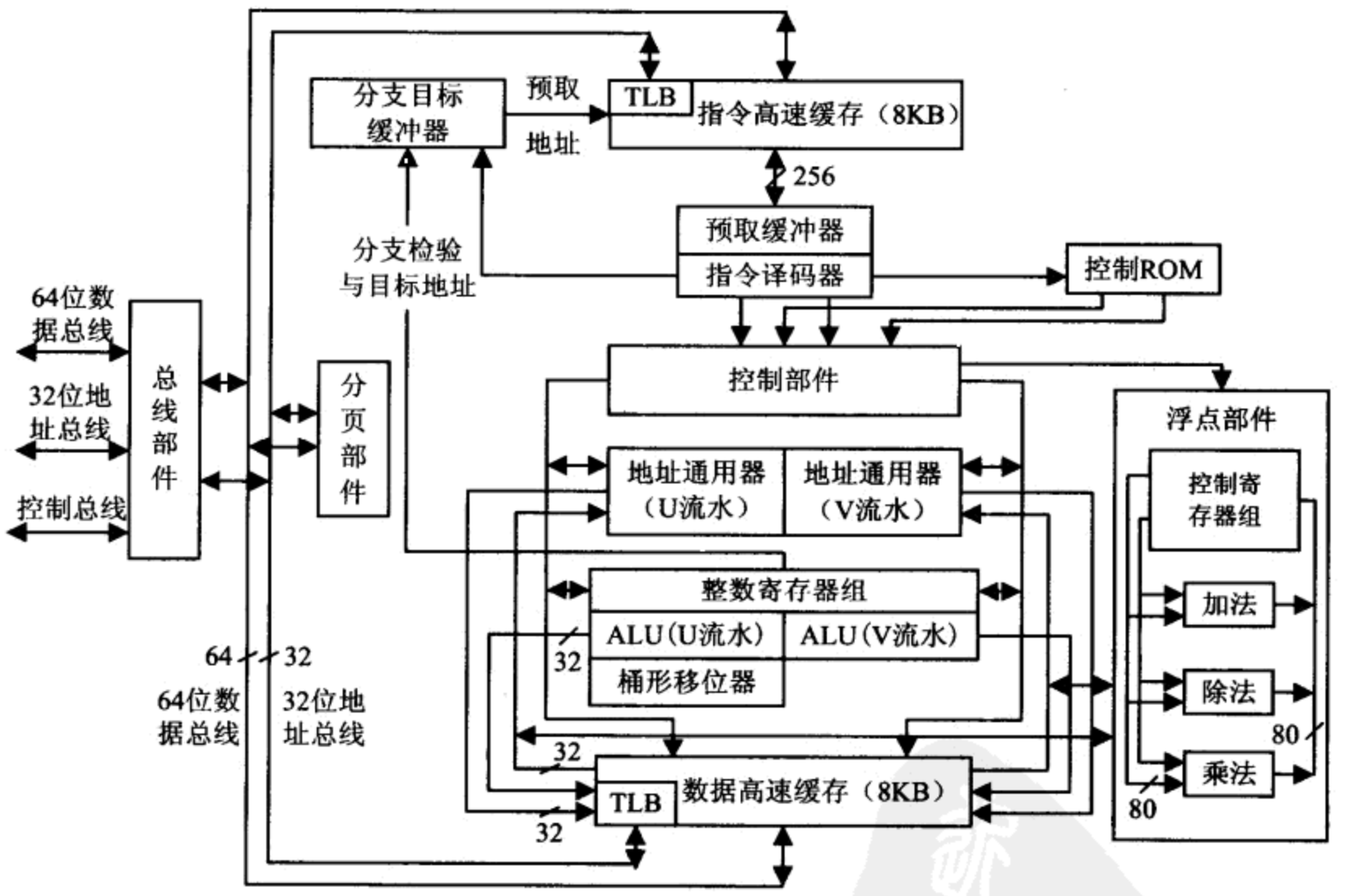


图 2-16　Pentium 微处理器内部结构

主要部件的功能分析如下。

（1）超标量整数处理部件：所谓超标量是指微处理器具有多条流水线，以增加每个时钟周期可以执行的指令数，从而使微处理器的运行速度成倍提高。

Pentium 微处理器有两条指令流水线，一条是 U 流水线，另一条是 V 流水线。两条流水

线都可以执行整数指令，但只有 U 流水线执行浮点指令。因此，能够在每个时钟周期内执行两条整数指令，或在每个时钟周期内执行一条浮点指令。每条流水线都有自己独立的地址生成部件、算术逻辑部件和数据超高速缓存接口。每条流水线都采用与 80486 相同的 5 级整数流水线，指令在其中分级执行。这 5 级流水线的内容如下。

- 指令预取（IP），指通过一条 256 位的内部总线将指令从指令高速缓存送到预取部件。
- 指令译码（ID），指对取来的指令进行译码分析。
- 地址生成和取操作数（AG）。
- 指令执行（IE），指由算术/逻辑运算部件 ALU 执行指令。
- 回写（WB），指将结果写回到合适的寄存器或存储单元中。相对于通写方式而言，可以减少访问主存储器的次数。

（2）超标量流水线浮点处理部件：Pentium 微处理器中的浮点操作被高度流水线化，并与整数流水线集成在一起。Pentium 微处理器的浮点流水线是超级流水线，微处理器内部流水线进一步分割成若干个小而快的级段，使指令能在其中以更快的速度通过。每一个超级流水线级段都以数倍于时钟周期的速度运行。

（3）独立的数据和指令高速缓存 Cache：Pentium 加进了第二个整数 ALU，使得整数单元的潜在处理能力增加了一倍，这同时也要求处理器进行双倍的指令与数据存取。在 Pentium 微处理器中，有两个独立的 8KB 指令高速缓存和 8KB 数据高速缓存，并可以扩展到 12KB。一个专门用于指令，另一个专门用于数据，允许两个 Cache 同时存取，使得内部传输效率更高。这两个 Cache 采用双路相关联的结构，每路有 128 个高速缓存行，每行可存放 32 个字节（256 位）的信息，即每路 4KB。数据高速缓存有两个端口，分别用于 U、V 两条流水线。每一个高速缓存都有一个专用的转换后援缓冲器（TLB），用来把线性地址快速地转换成高速缓存所用的物理地址。

（4）Pentium 微处理器的指令集与指令预取：Pentium 有两个 32 字节的指令预取缓冲器，通过一个预取缓冲器顺序地处理指令地址，直到它取到一条分支指令，此时存放有关分支历史信息的分支目标缓冲器 BTB，将对预取到的分支指令是否导致分支进行预测。

（5）分支预测：Pentium 在指令预取处理中增加了分支预测逻辑，提供分支目标缓冲器 BTB 的小 Cache 来预测程序转移。当产生一次程序转移时，BTB 就将该指令和转移目标地址存起来，BTB 中记录着正在执行的程序内所发生的几次转移，它可以利用存放在其中的转移记录，来预测下一次程序转移。

3. Pentium Pro 微处理器

Pentium Pro 又称为高能奔腾，是 Intel 公司继 Pentium 之后于 1995 年末推出的又一种新型高性能奔腾微处理器。它比 Pentium 增加了 8 条指令，对 X86 处理器向下兼容，采用 387 个引脚的 PGA 封装。

Pentium Pro 的主要特点如下。

（1）微处理器集成了 550 万只晶体管，高速缓存器集成了 1 550 万只晶体管。

（2）三路发布超级标量微结构，14 级超流水线，使一个时钟周期内可同时执行 3 条指令。

（3）具有 5 个并行处理单元：2 个整数运算部件，1 个装入部件，1 个存储部件，1 个

浮点运算部件。

（4）8KB 两路组相关指令高速缓存，8KB 四路组相关数据高速缓存。

（5）专用全速总线上的 256KB SRAM 二级高速缓存与微处理器紧密相连。

（6）事务处理 I/O 总线和非封锁高速缓存分级结构。

（7）错序执行，动态分支预测和推理执行。

（8）工作电压 2.9V，0.6μm 结构，4 层金属 BiVMOS 工艺，微处理器硅片 306mm^2。

（9）主要用于具有 32 位操作系统的服务器中。

（10）采用了 RISC 技术，超标量与超流水线相结合的核心机构，实现了动态执行技术。

4. MMX 及 Pentium MMX 微处理器

MMX 为 Multi-Media eXtended（多媒体扩展技术）的简称，在微处理器内部除常用指令系统的指令外，增加了支持多媒体的指令集，使微处理器性能大大增强。

Intel 公司于 1997 年初推出 Pentium MMX，称为多能奔腾处理器，是一种充分改善多信息应用程序性能的微处理器。具有 MMX 技术的用于台式系统的多能奔腾处理器的主频主要有 166MHz、200MHz 和 233MHz。

多能奔腾微处理器可以全面提高计算机的综合性能，主要包括整数运算、浮点运算及多媒体应用 3 个方面。与相同速度的奔腾处理器相比，使用 MMX 技术的多能奔腾处理器性能可提高 60%以上。基于 MMX 技术的新微处理器芯片增加了 57 条多媒体指令，加快了运行多媒体和视频应用程序的速度，而且把芯片的高速缓存器加大一倍，并给予更强的分支预测能力，使其计算技能在执行所有任务中获得成功。

5. Pentium Ⅱ微处理器

1997 年 5 月，Intel 公司推出了 Pentium Ⅱ，它采用了与 Pentium Pro 相同的核心结构，从而继承了原有 Pentium Pro 处理器优秀的 32 位微处理器性能。与此同时，它加快了对段寄存器写操作的速度，增加了对多媒体的支持和对 16 位代码优化的特性，它能够同时处理两条 MMX 指令。

Pentium Ⅱ首次采用单边连接盒的独立接插式标准（Slotl），用一块带金属外壳的印刷电路板，不但集成了处理器部件，还包括了 32KB 的 L1 Cache，并且处理器封装与 512KB~2MB 的 L2 Cache 等置于同一个底座共 242 个引脚，可直接插入主板的相应插座中。

Pentium Ⅱ处理器的主要特点如下。

（1）双重独立总线（DIB）体系结构，能同时使用具有纠错功能的 64 位系统总线和具有可选纠错功能的 64 位 Cache 总线。

（2）多重跳转分支预测。通过多条分支预测程序执行，加快了指令向处理器的流动。

（3）数据流分析。分析并重排指令，使指令以优化的顺序执行，与原始程序的顺序无关。

（4）指令推测执行。通过预先查看程序计数器 PC，并执行那些将要执行的指令，提高了程序运行速率。

（5）采用 Intel MMX 技术。包括了 57 条增强的 MMX 指令技术，可处理视频、声频及图像数据。

6. Pentium III微处理器

1999 年 Intel 推出了 Pentium III微处理器，总线频率为 100/133MHz，内部核心部分集成了 950 万只晶体管，具有单指令多数据（SIMD）浮点运算部件，SIMD 技术使 Pentium III微处理器用一条指令就能完成以往需要 4 条指令才能完成的浮点数据运算。Pentium III微处理器又增加了 70 条多媒体指令，仍采用 Slot1 封装。现已研制出 1 000MHz（1GHz）的 Pentium III处理器，芯片内集成了 2 810 万个晶体管，体积更小，功耗更低而性能更强。

Pentium III的主要特点如下。

（1）主频 450MHz 以上。

（2）总线频率 100MHz/133MHz。

（3）新增加 70 条 SSE 指令。

（4）2.0V 供电，0.25μm 工艺制造。

（5）32KB 的 L1 以主频速度工作，512KB 的 L2 以主频一半速度工作。

Pentium III微处理器除时钟频率提高外，它的整体性能也大大提高，如语音处理能力提高 37%，图形图像处理能力提高 64%。其内部还内置了序列号，在连接 Internet 时，可借助该序列号识别上网者的合法身份。增加的 70 条多媒体指令（Streaming SIMD Extensions，SSE）可提高 3D、语音、图形图像、互联网软件运行速度。

7. Pentium 4 微处理器

2000 年 3 月，Intel 公司推出了新一代高性能 32 位 Pentium 4 微处理器，它采用了 NetBurst 的新式处理器结构，可以更好地处理互联网用户的需求，在数据加密、视频压缩和对等网络等方面的性能都有较大幅度的提高。

而目前市场上流行的是 Intel 公司新一代高性能 32 位 Pentium 4 微处理器，它采用了 NetBurst 的新式处理器结构，可以更好地处理互联网用户的需求，在数据加密、视频压缩和对等网络等方面的性能都有较大幅度的提高。

Pentium 4 微处理器有以下的主要特征和处理能力。

（1）采用超级流水线技术，使 CPU 指令的运算速度成倍增长，在同一时间内可以执行更多的指令，显著提高了处理器时钟频率以及其他性能。

（2）快速执行引擎使处理器的算术逻辑单元达到了双倍内核频率，可以用于频繁处理诸如加、减运算之类的重复任务，实现了更高的执行吞吐量，缩短了等待时间。

（3）执行追踪缓存，用来存储和转移高速处理所需的数据。

（4）高级动态执行，可以使微处理器识别平行模式，并且对要执行的任务区分先后次序，以提高整体性能。

（5）具备 400 MHz 的系统总线，可以使数据以更快的速度进出微处理器，此总线在 Pentium 4 微处理器和内存控制器之间提供了 3.2GB 的传输速度，是现有的最高带宽台式机系统总线，具备了响应更迅速的系统性能。

（6）增加了 114 条新指令，主要用来增强微处理器在视频和音频等方面的多媒体性。

（7）为用户提供了更加先进的技术，使之能够获得丰富的互联网体验。

Pentium 4 微处理器为因特网、图形处理、数据流视频、语音、3D 和多媒体等多种应用模式提供了强大功能，可谓是数字时代的核“芯”动力。融合众多先进技术的 Pentium 4

NetBurst 架构带来更逼真的三维环境、更流畅的动画，同时，极大地加速了视频编辑与处理过程。

本章小结

本章针对 8086 微处理器及其体系结构作了详细介绍。8086 微处理器从功能结构上可以划分为执行部件和总线接口部件两大部分，这种并行工作方式减少了 CPU 等待取指令的时间，充分利用了总线，有力地提高了 CPU 的工作效率，加快了整机的运行速度，也降低了 CPU 对存储器存取速度的要求，成为 8086 的突出优点。

8086 微处理器的寄存器使用非常灵活，8086 微处理器可供编程使用的有 14 个 16 位寄存器，按其用途可分为 3 类：通用寄存器、段寄存器、指针和标志寄存器。8086 微处理器的存储器和 I/O 组织是本章的重点内容，应掌握存储器的分段管理、物理地址和逻辑地址的换算及 I/O 端口的编址方式。

本章还针对 32 位的 80X86 系列产品 80386、80486 以及 Pentium 系列等高档微处理器的特点及基本结构作了介绍。Pentium 系列微型计算机是目前 PC 市场占有份额最多的一种，Pentium、Pentium Pro、Pentium MMX、Pentium II、Pentium III、Pentium 4 等微处理器是普遍使用的微处理器。它们不仅增加了数据总线、地址总线的位数，而且采用了许多新技术，如指令高速缓存与数据高速缓存分离，分支预测，超标量流水线技术，RISC 结构，增加了支持多媒体的指令集，使微处理器性能大大增强。

思考与练习题

1. 8086 微处理器由哪两部分组成，它们的主要功能是什么？
2. 8086 微处理器数据总线和地址总线各是多少，最大的存储空间是多少？
3. 8086 微处理器中的指令队列起什么作用，其长度是多少字节？
4. 8086 的标志寄存器有哪些标志位，它们的含义和作用是什么？
5. 数据在存储器中存放有何规定，什么是对准字，什么是非对准字？
6. 什么是逻辑地址，它由哪两部分组成？8086 的物理地址是如何形成的？
7. 8086 微机系统中存储器为什么要分段，各逻辑段之间的关系如何？
8. I/O 端口有哪两种编址方式，8086 的最大 I/O 寻址空间是多少？
9. 8086 的最大工作方式模式和最小工作方式的主要区别是什么？他们分别应用在何种场合？
10. 有一个由 10 个字组成的数据区，其起始地址为 1200H:0120H。试写出该数据区的首末存储单元的实际地址。
11. 若一个程序段开始执行之前，(CS)=33A0H，(IP)=0130H，试问该程序段启动执行指令的实际地址是什么？
12. 有两个 16 位的字 31DAH 和 5E7FH，它们在 8086 系统存储器中的地址分别为 00130H 和 00134H，试画出它们的存储示意图。

13. 将字符串“Good！”的 ASCII 码依次存入从 01250H 开始的字节单元中，画出它们存放的内存单元示意图。
14. 8086 微处理器读/写总线周期各包含多少个时钟周期？什么情况下需要插入 T_W 等待周期？应插入多少个 T_W，取决于什么因素？什么情况下会出现空闲状态 T_I？
15. 80286 微处理器寄存器结构中比 8086 增加的部分有哪些，其主要用途是什么？
16. 简述实地址方式和虚拟 8086 方式的区别。
17. 简述 Pentium 微处理器的主要特点。

第 3 章 寻址方式与指令系统

本章从指令格式、寻址的概念出发，主要讨论了 8086 微处理器采用的指令格式和寻址方式，详细分析了 8086 指令系统中各类指令的功能、特点和应用，通过一些实例来说明指令的使用方法。此外，还讨论了 Pentium 微处理器的内部寄存器和指令格式，新增的寻址方式和专用指令等。本章为后面汇编语言程序设计的学习和理解打下坚实的基础。

通过本章的学习，读者应掌握 8086 微处理器指令系统的寻址方式及地址计算方法、8086 微处理器指令及其应用、DOS 和 BIOS 中断调用；熟悉 Pentium 微处理器增加的寄存器、寻址方式和专用指令等。

3.1 指令格式及寻址

3.1.1 指令系统与指令格式

1. 指令与指令系统

计算机的工作过程就是执行指令的过程。指令是要求计算机执行特定操作的命令，通常一条指令对应一种特定操作，例如加、减、传送、移位等。指令的执行是在计算机的 CPU 中完成的，每条指令规定的运算及基本操作都是简单的、基本的，它和计算机硬件所具备的能力相对应。

计算机所能执行的全部指令的集合称为指令系统。指令系统是计算机硬件和软件之间的桥梁，是汇编语言程序设计的基础，它与微处理器的性能密切相关，性能优越的指令系统可以更快更好地运行各种程序，实现更多的微处理器性能。

指令以二进制编码的形式存放在存储器中，用二进制编码形式表示的指令称为机器指令。CPU 可以直接识别机器指令。对于用户来说，机器指令记忆、阅读比较困难，为此将每一条指令都用统一规定的符号和格式来表示。用符号表示的指令称为符号指令。符号指令具有直观、易理解、可帮助记忆的特点。汇编语言程序中的指令就是符号指令。在计算机中，符号指令与机器指令具有一一对应的关系。

2. 指令格式

计算机的指令格式与机器的字长、存储器的容量及指令的功能有很大的关系，从便于程序设计、增加操作并行性、提高指令功能的角度出发，指令中包含的信息越多越好，但在某些指令中，其中一部分信息可能无用，必将浪费指令所占的存储空间，同时增加访问存储器的次数，从而影响执行速度。所以，如何合理、科学地设计指令格式，使指令既能够给出足够的信息，指令长度又尽可能地与机器的字长相匹配，以节省存储空间、缩短取指时间、提高机器性能，是指令系统设计中需要考虑的问题。

计算机是通过执行指令来处理数据的，为了指出数据的来源、操作结果的去向以及所执行的操作，计算机中的指令应该由操作码字段和操作数字段两部分组成。

（1）操作码字段：规定指令的操作类型，说明计算机要执行的具体操作，如传送、运算、移位、跳转等操作，同时还指出操作数类型、操作数传送方向、寄存器编码或符号扩展等，是指令中必不可少的组成部分。

（2）操作数字段：说明在指令执行的过程中需要的操作数，它可以是操作数本身，也可以是操作数地址或是地址的一部分，还可以是指向操作数的地址指针或其他有关操作数据的信息。单地址指令的操作只需一个操作数，如加 1 指令：INC AX。大多数运算型指令都需要两个操作数，如加法指令：ADD AX，BX；运算的结果送到 AX 中，AX 称为目的操作数，BX 称为源操作数。

由于在计算机中存储的命令和程序是由二进制代码组成的，不易理解和记忆，容易出错，人们就采用一些助记符——通常用指令功能的英文单词的缩写来代替操作码，如数据传送指令用助记符 MOV（MOVE 的缩写）表示。

3.1.2　寻址及寻址方式的概念

对于一条指令来说，要解决两个问题：一是要指出进行什么操作，这由指令操作码来表明；二是要指出操作数的来源，这就是操作数的寻址方式。

计算机中大多数指令采用一个或两个操作数。操作数有 3 种表示形式，分别称为立即数、寄存器操作数和存储器操作数。

指令指定操作数的位置，即给出地址信息，在执行时需要根据这个地址信息找到需要的操作数，这种寻找操作数的过程称为寻址。寻址方式就是寻找操作数或操作数地址的方式。根据寻址方式可以方便地访问各类操作数。

8086 指令中的操作数有以下 3 种可能的存放位置。

（1）立即操作数在指令中，即指令的操作数部分就是操作数本身。

（2）寄存器操作数包含在 CPU 的某个内部寄存器中，这时指令的操作数部分是 CPU 内部寄存器的一个编码。

（3）存储器操作数在内存的数据区中，这时指令的操作数部分包含此操作数所在的内存地址。

3.2　8086 指令系统的寻址方式

8086 提供了与操作数有关和与 I/O 端口地址有关的两类寻址方式。其中，与操作数有关的寻址方式共 7 种，分别是立即数寻址、寄存器寻址、直接寻址、寄存器间接寻址、寄存器相对寻址、基址变址寻址和相对基址变址寻址方式；与 I/O 端口有关的寻址方式分别是直接端口寻址和间接端口寻址方式。

3.2.1　与操作数有关的寻址方式

1. 立即数寻址方式

操作数直接存放在指令中，并紧跟在操作码之后的寻址方式就是立即数寻址方式。该寻址方式主要用来对存储器或寄存器赋值，且只能用于源操作数字段，不能用于目的操作数字

段，其显著特点是速度快。可以用二、十、十六进制数来表示立即数。如果是 16 位操作数，则低位字节存放在低地址单元中，高位字节存放在高地址单元中。在非十进制的立即数末尾需要使用字母加以标识。

```
例如：MOV  AL,11001010B          ;将二进制数 11001010B 送寄存器 AL
      MOV  AL,0FH                ;将十六进制数 0FH 送 AL
      MOV  AX,1234H              ;将 1234H 送 AX,AH 中为 12H,AL 中为 34H
      MOV  AL,10                 ;将十进制数 10 送 AL
```

2. 寄存器寻址

操作数存放在 CPU 的内部寄存器时，可在指令中指出寄存器名，这种寻址方式就叫寄存器寻址方式。

8 位操作数可采用 AH、AL、BH、BL、CH、CL、DH、DL 等寄存器保存；16 位操作数可采用 AX、BX、CX、DX、SI、DI、SP、BP 等寄存器保存。

```
例如：MOV   AX,BX              ;将 BX 的内容送 AX
      ADD   AX,BX              ;执行(AX)←(AX)+(BX)
```

寄存器寻址方式的指令在 CPU 内部进行操作，不需要使用总线周期，因此执行速度快。该方式可用于源操作数，也可用于目的操作数，或者两者都用寄存器寻址方式。

3. 存储器寻址方式

由于寄存器数量有限，所以程序中的大多数操作数需要从内存中获得。内存的寻址方式有多种，最终都将得到存放操作数的物理地址。

8086 指令系统提供了以下 5 种针对存储器的寻址方式。

（1）直接寻址

指令中给出的地址码即为操作数的有效地址，称为直接寻址方式。若操作码所需的操作数存放在存储器中，则指令中需要给出操作数的地址信息。操作数的有效地址（EA）的计算方法和寻址方式有密切联系，物理地址 PA 的计算和操作数的位置有关。

由于 8086 的有效地址为 16 位，所以这种指令的地址码要占两个字节，其存储方式要按“字”存放。指令中给出的地址码即为操作数的有效地址（EA），它是一个 8 位或 16 位的位移量数据。

【例 3.1】执行指令 MOV AX，[0002H]，其中(DS)=2000H。

给定数据段数据存储形式如图 3-1 所示。

该指令执行时：

有效地址为 EA=0002H；

物理地址为 PA=(DS)×10H+EA=20002H。

由于指令中目的操作数是 16 位寄存器，因此要取出 20002H 和 20003H 两个单元中的数据，即 20002H 单元中的 30H 送给 AL，20003H 单元中的数据 56H 送给 AH。指令执行后的结果为(AX)=5630H。

偏移地址	数据段	物理地址
0000H	12H	20000H
0001H	23H	20001H
0002H	30H	20002H
0003H	56H	20003H
0004H	0AH	20004H

图 3-1　数据段内容存放示意图

8086 指令系统中规定，存储器直接寻址方式如果不加说明，操作数一定在数据段 DS。若操作数在规定段以外的其他段，则必须在地址前加以说明。这种说明称为“段超越前缀”。

例如：MOV AL，ES:[0002H]

源操作数指定存放于附加段 ES 中，则物理地址 PA=(ES)×10H+EA。

（2）寄存器间接寻址方式

这种寻址方式是在指令中给出寄存器，寄存器中的内容为操作数的有效地址。如果指令中指定的寄存器是 BX、SI 和 DI，操作数以 DS 段寄存器中的内容作为段地址。此时，操作数的物理地址为：PA=(DS)×10H +(BX)或(SI)或(DI)

如果指令中指定的寄存器是 BP，操作数以 SS 段寄存器中的内容作为段地址。此时，操作数的物理地址为：PA=(SS)×10H+(BP)

【例 3.2】执行指令 MOV AX，[BX]；若(DS)=2000H，(BX)=1000H

则：PA=2000H×10H+1000H

=21000H

即从 21000H 单元取出低字节数据，从 21001H 单元取出高字节数据，组合成 1 个字数据后送到寄存器 AX 中。

（3）寄存器相对寻址方式

这种寻址方式是在指令中给定一个基址寄存器（或变址寄存器）和一个 8 位或 16 位的相对偏移量，两者之和作为操作数的有效地址。使用 BX、SI、DI 这 3 个间址寄存器指示的是数据段中的数据，使用 BP 作间址寄存器指示的是堆栈段中的数据。

有效地址计算为：EA=(R)+8 位（或 16 位）偏移量；其中 R 为给定寄存器。

物理地址计算为：PA=(DS)×10H+EA（使用 BX、SI、DI 间址寄存器）

PA=(SS)×10H+EA（使用 BP 作为间址寄存器）

【例 3.3】执行指令 MOV AX，[BX+10H] ；传送指定地址中的字数据到累加器中。

若给定(BX)=1200H，(DS)=2000H，(21210H)=21H，(21211H)=43H

则：EA=(BX)+10H=1210H

PA=(DS)×10H+EA=21210H

指令执行后(AX)=4321H。

（4）基址变址寻址

这种方式是在指令中给出一个基址寄存器和一个变址寄存器，两者内容之和作为操作数的有效地址。

基址寄存器可取 BX 或 BP，变址寄存器可取 SI 或 DI，但指令中不能同时出现两个基址寄存器或两个变址寄存器。如果基址寄存器为 BX，则段寄存器使用 DS；如果基址寄存器用 BP，则段寄存器用 SS。

【例 3.4】给定指令 MOV AX，[BX+SI]；寄存器内容为(DS)=1200H，(BX)=0100H，(SI)=0050H，计算操作数地址并分析指令的执行情况。

给定指令为基址变址寻址方式，其功能是将指定的内存单元内容传送到 AX 中。

操作数的物理地址为：PA =(DS)×10H+(BX)+(SI)

=1200H×10H +0100H+0050H

=12150H

指令的执行结果是将内存 12150H 单元的内容传送到寄存器 AX 中。如果内存 12150H 单

元开始保存的是字数据 1122H，则指令的执行完毕结果为：(AX)=1122H。指令的操作情况如图 3-2 所示。

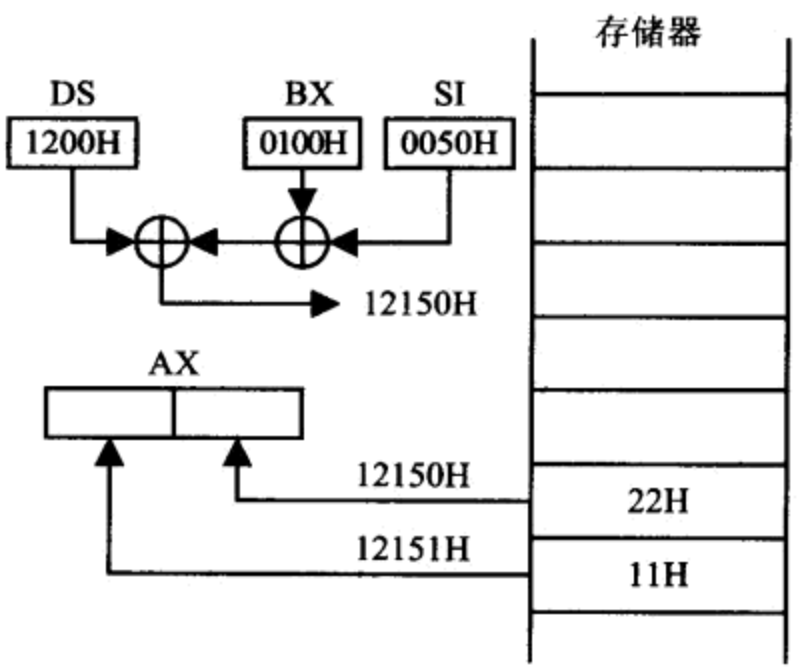


图 3-2　基址变址寻址操作过程分析

（5）相对基址变址寻址

这种寻址方式是在指令中给出一个基址寄存器、一个变址寄存器和 8 位或 16 位的偏移量，三者之和作为操作数的有效地址。

基址寄存器可取 BX 或 BP，变址寄存器可取 SI 或 DI，如果基址寄存器为 BX，则段寄存器使用 DS；基址寄存器用 BP，则段寄存器用 SS。

其物理地址计算为：PA=(DS)×10H+(BX)+(SI)或（DI）+偏移量

PA=(SS)×10H+(BP)+(SI)或（DI）+偏移量

【例 3.5】已知(DS)=2100H，(BX)=0110H，(DI)=0020H，偏移量=0050H，用相对基址变址寻址方式计算操作数的有效地址和物理地址如下。

有效地址为：EA=(BX)+(DI)+偏移量

=0110H+0020H+0050H

=0180H

物理地址为：PA=(DS)×10H+EA

=2100H×10H +0180H

=21080H

3.2.2　与 I/O 端口有关的寻址方式

8086 微处理器采用独立编址的 I/O 端口，有专门的输入指令 IN 和输出指令 OUT，寻址方式有如下两种。

1. 直接端口寻址

直接端口寻址是在指令中直接给出要访问的端口地址，一般采用 2 位十六进制数表示，也可以用符号表示，可访问的端口范围为 0～255。

例如：IN AL，25H

表示从 I/O 端口地址为 25H 的端口中取数据送到寄存器 AL 中。

2. 间接端口寻址

若访问的端口地址值大于 255，则必须用 I/O 端口的间接寻址方式。它是把 I/O 端口的地址先送到 DX 中，用 DX 作为间接寻址寄存器。此种方式可访问的端口范围为 0～65 535。

例如：
```
MOV DX, 285H    ;将端口地址 285H 送到 DX 寄存器
OUT DX, AL      ;将 AL 中的内容输出到 DX 指定的端口
```

3.3　8086 指令系统

8086 指令系统是 80X86/Pentium 的基本指令集。指令的操作数可以是 8 位或 16 位操作数，偏移地址是 16 位地址。

8086 指令系统按功能可以分为以下 6 类指令：

- 数据传送类指令
- 算术运算类指令
- 逻辑运算（位操作）类指令
- 串操作类指令
- 控制转移类指令
- 处理器控制类指令

学习指令时要注意掌握各类指令的书写格式、指令功能、寻址方式、指令对标志位的影响等方面。全面准确地理解每条指令的功能和用法是编写程序的关键。

3.3.1　数据传送类指令

数据传送类指令是计算机中最基本、最重要也是最常用的一种指令，其功能是把数据、地址或立即数传送到寄存器或存储单元。此类指令除了 SAHF 和 POPF 外均不影响标志寄存器的内容。按照传送的内容和功能的不同，可将数据传送指令分为 3 组，如表 3-1 所示。其中 dst 表示目的操作数，src 表示源操作数。

表 3-1　数据传送类指令

指令类型	指令格式	指令功能
通用数据传送	MOV dst，src	字节或字传送
	PUSH src	字压入堆栈
	POP dst	字弹出堆栈
	XCHG dst，src	字节或字交换
	XLAT	换码指令
地址传送	LEA dst，src	装入有效地址
	LDS dst，src	装入 DS 寄存器
	LES dst，src	装入 ES 寄存器
标志位传送	LAHF	将 FLAG 低字节装入 AH 寄存器
	SAHF	将 AH 内容装入 FLAG 低字节
	PUSHF	将 FLAG 内容压栈
	POPF	从堆栈中弹出一个字给 FLAG

1. 通用数据传送指令

（1）MOV 传送指令

指令格式：MOV dst, src

该指令把源操作数 src 传送至目的操作数 dst。指令执行后源操作数内容不变，目的操作数内容与源操作数内容相同。

源操作数可以是通用寄存器、段寄存器、存储器以及立即数，目标操作数可以是通用寄存器、段寄存器（CS 除外）或存储器。各类数据的传送关系如图 3-3 所示。

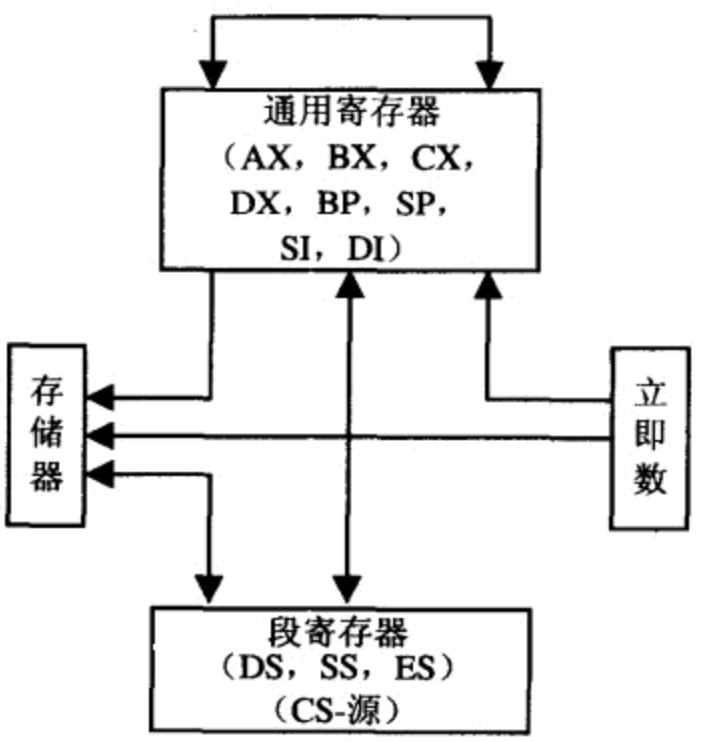


图 3-3 MOV 指令中各类数据的传送关系

【例 3.6】MOV 指令的应用形式分析如下：

```
MOV  AL,35H              ;8 位立即数 35H 送 AL
MOV  DX,1234H            ;16 位立即数 1234H 送 DX
MOV  DL,'B'              ;字符 B 的 ASCII 码送 DL
MOV  CL,BL               ;8 位寄存器之间传送
MOV  AX,BX               ;16 位寄存器之间传送
MOV  AX,[2100H]          ;存储单元数据送 AX
MOV  [0210H],SI          ;SI 中内容送指定存储单元
MOV  DS,AX               ;AX 中内容送段寄存器 DS
MOV  [SI],DS             ;DS 的内容送 SI 所指单元
MOV  ES,[BX]             ;BX 所指的存储单元内容送 ES
```

MOV 指令使用时应该注意：目的操作数不允许用立即数寻址，也不允许用 CS 寄存器；源操作数和目的操作数不能同时为存储器操作数；两个段寄存器之间不允许直接传送数据。

（2）PUSH/POP 堆栈操作指令

堆栈是存储器中的一个特殊区域，以“先进后出”的方式进行数据操作。8086 的堆栈组织是从高地址向低地址方向生长的。它只有一个出入口，堆栈指针寄存器 SP 始终指向堆栈的栈顶单元。堆栈组织示意参见图 3-4。

入栈指令格式：PUSH　src

执行的操作为：SP ←(SP)−2

(SP)←(src)

操作时 SP 的内容首先减 1，操作数的高位字节送入当前 SP 所指示的单元中；然后 SP 中的内容再减 1，操作数的低位字节送入当前 SP 所指示的单元中。src 是入栈的字操作数，除了不允许用立即数外，通用寄存器、段寄存器和存储器操作数都能入栈。

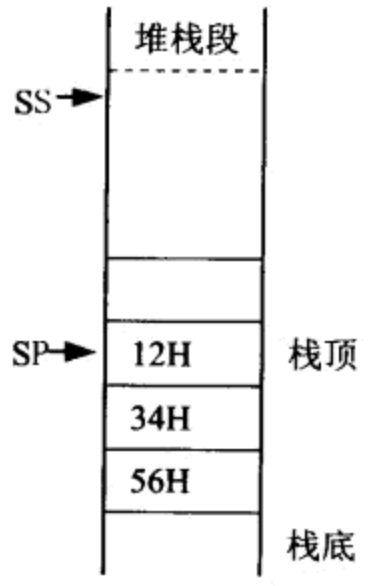


图 3-4　堆栈示意图

【例 3.7】执行指令 PUSH AX

若给定(SP)=00F8H，(SS)=2500H，(AX)=5120H

则指令执行后(SP)=00F6H，(250F6H)=5120H。

出栈指令格式：POP dst

执行操作为：dst ←((SP))

(SP)←(SP)+2

dst 是出栈操作的目的地址，长度必须为 16 位，除了立即数和 CS 段寄存器以外，通用寄存器、段寄存器和存储器都可以作为出栈的目的地址。

操作时首先将 SP 所指的栈顶单元内容送入 dst 低位字节单元，SP 的内容加 1，然后将 SP 所指栈顶单元内容送入 dst 的高位字节单元，SP 的内容再加 1。

【例 3.8】执行指令 POP BX

若给定(SS)=2000H，(SP)=0100H，(BX)=78C2H，(20100H)=6B48H

则指令执行后(BX)=6B48H，(SP)=0102H。

（3）XCHG 交换指令

指令格式：XCHG opr1,opr2

其中 opr1 和 opr2 表示操作数。

该指令中必须有一个操作数是在寄存器中，可以实现字节数据交换，也可以实现字数据交换。指令执行结果不影响标志位。

【例 3.9】执行指令 XCHG AX,BX

将寄存器 AX 内容与 BX 内容互相交换位置。

若给定(AX)=325AH，(BX)=2130H

则指令执行后(AX)=2130H，(BX)=325AH。

2. 累加器专用传送指令

累加器 AX 通常作为数据传输的核心，8086 指令系统中的输入/输出指令和换码指令是专门通过累加器来执行的。

（1）IN/ OUT 输入/输出指令

输入指令格式为：IN Acc,src

表示 Acc 为 8 位或 16 位累加器。

例如：
```
IN AL,30H       ;将 30H 端口中的数据读入到 AL 中
IN AX,20H       ;将 20H 端口中的数据读入到 AL 中，将 21H 端口中的数据读入到 AH 中
```

输出指令格式为：OUT dst,Acc

例如：OUT DX,AL　　;AL 中的内容输出到 DX 所指示的字节端口

（2）XLAT 换码指令

指令格式为：XLAT

执行操作 AL←(BX+AL)

换码指令可将累加器 AL 中的一个值转换为内存表格中的某一个值，再送回 AL 中。XLAT 一般用来实现码制之间的转换，又称为查表转换指令。

3. 地址传送指令

（1）LEA 有效地址送寄存器指令

将存储器操作数 src 的有效地址传送到 16 位的通用寄存器。

例如：LEA BX,[SI+BP]

执行后，将(SI)+(BP)的结果送到 BX 中。

（2）LDS 地址指针送寄存器指令

此指令执行的操作是将 src 指示的前两个字节单元的内容送入指令中指定的通用寄存器中，src+2 所指示的两个字节的内容送入 DS 中。

【例 3.10】执行指令 LDS BX, ARR[SI]

如果给定 ARR＝0010H，(SI)＝0020H，(DS)＝2000H，(BX)＝6AE0H，(20030H)＝0080H，(20032H)＝4000H

则指令执行后：(BX)＝0080H，(DS)＝4000H。

（3）LES 指针送寄存器指令

指令执行的操作与 LDS 指令大致相似，不同之处是以 ES 代替 DS。

【例 3.11】执行指令 LES DI,[BX]

如果给定(DS)＝2000H，(BX)＝0020H，(20020H)＝18H，(20021H)＝A5H，(20022)＝00H，(20023H)＝50H，(ES)＝4000H

则指令执行后：(DI)＝A518H，(ES)＝5000H。

4. 标志寄存器传送指令

标志寄存器传送指令共有 4 条，这些指令都是单字节指令，指令的操作数以隐含形式规定，字节操作数隐含为 AH 寄存器。

（1）LAHF 取标志指令

将标志寄存器 FLAG 中的 5 个状态标志位 SF、ZF、AF、PF 以及 CF 分别取出传送到累加器 AH 的对应位。

（2）SAHF 置标志位指令

指令的传送方向与 LAHF 方向相反，FLAG 寄存器中的 SF、ZF、AF、PF 和 CF 将被修改成 AH 寄存器中对应位的值。

（3）PUSHF 标志压入堆栈指令

先将 SP 减 2，然后将标志寄存器 FLAG 中的内容压入堆栈中。

（4）POPF 标志弹出堆栈指令

指令的操作与 PUSHF 指令相反，它将堆栈内容弹出到标志寄存器 FLAG，然后加 2。

PUSHF 和 POPF 指令可用来保护调用过程以前标志寄存器的值，过程返回后再恢复此标志位，或用来修改标志寄存器中相应标志位的值。

3.3.2　算术运算类指令

8086 的算术运算类指令包括加、减、乘、除 4 种基本运算指令，以及进行 BCD 码十进制数运算的指令。

算术运算指令涉及无符号数和带符号数两种类型的数据。加减运算采取同一套指令，乘除运算有各自不同的指令。加减法运算在执行过程中有可能产生溢出，对于无符号数，如果加法运算最高位向前产生进位，减法运算最高位向前有借位，则表示出现溢出，用 CF 标志位可检测无符号数是否溢出。对于带符号数，采用补码运算，符号位参加运算，溢出则表示运算结果发生错误，用 OF 标志位可检测带符号数是否溢出。

算术运算指令会影响标志位，其规则如下：

- 运算结果向前产生进位或借位时，CF=1。
- 最高位向前进位和次高位向前进位不同时，OF=1。
- 若运算结果为 0，ZF=1。
- 若运算结果最高位为 1，SF=1。
- 若运算结果中有偶数个 1，PF=1。

表 3-2 给出了算术运算指令的名称、助记符和它们对标志位的影响。

表 3-2　算术运算类指令

类　别	指令名称	指令书写格式（助记符）	状态标志					
			OF	SF	ZF	AF	PF	CF
加法	加法（字节/字）	ADD dst,src	¤	¤	¤	¤	¤	¤
	带进位加法（字节/字）	ADC dst,src	¤	¤	¤	¤	¤	¤
	加 1（字节/字）	INC dst	¤	¤	¤	¤	¤	—
减法	减法（字节/字）	SUB dst,src	¤	¤	¤	¤	¤	¤
	带借位减法（字节/字）	SBB dst,src	¤	¤	¤	¤	¤	¤
	减 1	DEC dst	¤	¤	¤	¤	¤	—
	取补	NEC dst	¤	¤	¤	¤	¤	¤
	比较	CMP dst,src	¤	¤	¤	¤	¤	¤
乘法	不带符号乘法（字节/字）	MUL src	¤	※	※	※	※	¤
	带符号乘法（字节/字）	IMUL src	¤	※	※	※	※	¤
除法	不带符号除法（字节/字）	DIV src	※	※	※	※	※	※
	带符号除法（字节/字）	IDIV src	※	※	※	※	※	※
	字节扩展	CBW	—	—	—	—	—	—
	字扩展	CWD	—	—	—	—	—	—
十进制调整	非组合 BCD 码的加法调整	AAA	※	※	※	¤	※	¤
	组合 BCD 码的加法调整	DAA	※	¤	¤	¤	¤	¤
	非组合 BCD 码的减法调整	AAS	※	※	※	¤	※	¤
	组合 BCD 码的减法调整	DAS	※	¤	¤	¤	¤	¤
	非组合 BCD 码的乘法调整	AAM	※	¤	¤	※	¤	※
	组合 BCD 码的乘法调整	AAD	※	¤	¤	※	¤	※

注：表中“¤”表示运算结果影响标志位；“—”表示运算结果不影响标志位；“※”表示标志位为任意值

1. 加、减法指令及其应用

【例 3.12】分析给定加、减法指令的格式及操作功能

```
ADD  AL,BL                ;两个寄存器字节数据相加
ADD  AL,[0210H]           ;内存单元与寄存器字节数据相加
ADD  [SI],AX              ;寄存器与内存单元字数据相加
INC  AL                   ;执行(AL)←(AL)+1
INC  CX                   ;执行(CX)←(CX)+1
SUB  AX,BX                ;执行(AX)←(AX)-(BX)
SBB  DX,CX                ;执行(DX)←(DX)-(CX)-CF
DEC  CX                   ;执行(CX)←(CX)-1
```

使用注意事项：

（1）ADD 指令中，两个存储器操作数不能直接相加，段寄存器不能参加运算。还要注意两个操作数的类型保持一致。

（2）INC 指令的操作对象只能为通用寄存器或存储器操作数，不能为立即数，也不能是段寄存器，该指令常用在循环程序中修改指令或用做循环计数器。

（3）NEG 求补指令将操作对象中的内容取 2 的补码，相当于将操作对象中的内容按位取反后，末位加 1。

（4）CMP 比较指令与 SUB 指令一样执行减法操作，但该指令不保存结果，即指令执行后，两个操作数的内容不会改变。这条指令是根据操作的结果设置状态标志位，按比较结果使程序产生条件转移。

```
例如：CMP AL,0            ;AL 和 0 进行比较
      JGE NEXT            ;若 AL≥0 则转到 NEXT 位置执行
```

2. 乘、除法指令及其应用

（1）乘法指令包括对无符号数和带符号数相乘的指令。进行乘法运算时，如果两个 8 位数相乘，结果为 16 位；如果两个 16 位数相乘，结果为 32 位。乘法指令中有两个操作数，但指令中只给出乘数，被乘数隐含给出。8 位数相乘时被乘数先放入 AL 中，乘积存放到 AX 中；16 位数相乘时被乘数先放入 AX 寄存器中，乘积放到 DX 和 AX 两个寄存器中，且 DX 中存放高 16 位，AX 中存放低 16 位。

无符号数乘法指令“MUL src”和带符号数乘法指令“MUL src”中，src 可采用寄存器操作数、存储器操作数，但不能使用立即数和段寄存器。

```
例如：MUL AL              ;完成(AL)×(AL),结果送 AX
      MUL BX              ;完成(AX)×(BX),结果送 DX、AX
```

（2）8086 微处理器执行除法时规定：除数长度只能是被除数长度的一半。当被除数为 16 位时，除数应为 8 位；当被除数为 32 位时，除数应为 16 位。

DIV 无符号数除法指令的被除数、除数、商和余数全部为无符号数；IDIV 带符号数除法指令的被除数、除数、商和余数均为带符号数，且余数的符号位与被除数相同。

src 为字节数据时：AL ←(AX)/(src)　　表示保存商；
　　AH ←(AX)/(src)　　表示保存余数；
src 为字数据时：AX ←(DX)(AX)/(src)　　表示保存商；
　　DX ←(DX)(AX)/(src)　　表示保存余数。

（3）符号扩展指令是指用一个操作数的符号位形成另一个操作数，后一个操作数的各位是全 0（正数）或全 1（负数），其结果是使数据位数加长，但数据大小并没有改变。符号扩展指令的执行不影响标志位。

CBW 字节转换为字指令：将 AL 中的符号位扩展到 AH 中。若 AL 中的 D_7=0，则(AH)=00H；若 AL 中的 D_7=1，则(AH)=FFH。

CWD 字转换为双字指令：将 AX 中的符号位扩展到 DX 中。若 AX 中的 D_{15}=0，则(DX)=0000H，若 AX 中的 D_{15}=1，则(DX)=FFFFH。

3. 十进制调整指令

算术运算中如果使用了 BCD 数，其结果要用十进制调整指令进行处理。

（1）DAA 组合十进制数加法调整指令：指令执行前先执行加法指令（ADD 或 ADC）将两个组合十进制数相加，结果存放在 AL 寄存器中。DAA 指令紧跟在加法指令之后，先对结果进行测试，若结果中的低 4 位是十六进制数码 A～F，或者 AF=1，则 AL 寄存器内容加 06H；如果 AL 寄存器中高 4 位是十六进制数码 A～F，则 AL 寄存器的内容加 60H。

【例 3.13】执行指令：ADD AL,BL

DAA

若给定(AL)=28H，(BL)=69H，执行 ADD 指令后，(AL)=91H，AF=1。

执行 DAA 指令，因 AF=1，调整操作 AL←(AL)+06H，则 AL 中的内容为 97H，高 4 位≤09H，不必进行调整。

（2）AAA 非组合十进制数加法调整指令：指令执行之前先执行 ADD 或 ADC 指令，把两个非组合十进制数相加，将结果存放在 AL 寄存器中，调整后的结果放到 AX 中。AAA 指令的调整原则是，若 AL 的低 4 位＞9，或 AF=1，则 AL←(AL)+06H；CF=AF=1；同时 AH←(AH)+1；AL 的高 4 位为 0。

【例 3.14】执行指令：ADD AL,BL

AAA

若给定(AX)=0505H，(BL)=09H。ADD 指令执行后，(AL)=0EH，不是非组合 BCD 码；执行 AAA 指令后，(AX)=0104H，为 14 的非组合 BCD 码。

（3）DAS 组合十进制减法调整指令：指令执行之前先执行减法指令（SUB 或 SBB 指令）把两个组合十进制数相减，并将结果存放在 AL 寄存器中。DAS 指令的调整方法是，如果 AF=1，或者 AL 寄存器的低 4 位是十六进制的 A～F，则 AL 寄存器中的内容减 06H，且将 AF 标志位置 1。如果 CF=1，或者 AL 寄存器的高 4 位是十六进制的 A～F，则 AL 寄存器中的内容减 60H，并将 CF 标志位置 1。

【例 3.15】执行指令：SUB AL,AH

DAS

若给定(AL)=97H，(AH)=39H，SUB 指令执行后，(AL)=5EH，AF=1，AL 中的内容不是组合 BCD 码格式，需进行调整。执行 DAS 指令，完成 AL←(AL)−06H 后，(AL)=58H，CF=0，且 AL 中高 4 位≤09H，不必进行调整。

（4）AAS 非组合十进制数减法调整指令：指令执行前先执行 SUB 或 SBB 指令把两个非组合十进制数相减，将结果存放在 AL 寄存器中，调整后的结果放到 AX 中。指令执行后，

只影响 AF 和 CF 标志位。AAS 指令的调整原则是：若 AL 的低 4 位>9，或 AF=1，则 AL←(AL)-06H；CF=AF=1；同时 AH←(AH)-1；AL 的高 4 位为 0。

【例 3.16】执行指令：SUB AL,CL

AAS

若给定(AX)=0205H，(CL)=08H，SUB 指令执行后 AF=1，CF=1，(AL)=0FDH，AAS 指令执行后，(AX)=0107H。

3.3.3 逻辑运算与移位类指令

8086 指令系统的逻辑运算与移位类指令的类别、书写格式和对标志位的影响如表 3-3 所示。

表 3-3 逻辑运算与移位类指令

类 别	指令名称	指令书写格式（助记符）	状态标志					
			OF	SF	ZF	AF	PF	CF
逻辑运算	“非”（字节/字）	NOT dst	—	—	—	—	—	—
	“与”（字节/字）	AND dst，src	0	¤	¤	※	¤	0
	“或”（字节/字）	OR dst	0	¤	¤	※	¤	0
	“异或”（字节/字）	XOR dst，src	0	¤	¤	※	¤	0
	“测试”（字节/字）	TEST dst，src	0	¤	¤	※	¤	0
一般移位	逻辑左移（字节/字）	SHL dst，计数值	¤	¤	¤	※	¤	¤
	算术左移（字节/字）	SAL dst，计数值	¤	¤	¤	※	¤	¤
	逻辑右移（字节/字）	SHR dst，计数值	¤	¤	¤	※	¤	¤
	算术右移（字节/字）	SAR dst，计数值	¤	¤	¤	※	¤	¤
循环移位	循环左移（字节/字）	ROL dst，计数值	¤	—	—	※	—	¤
	循环右移（字节/字）	ROR dst，计数值	¤	—	—	※	—	¤
	带进位循环左移（字节/字）	RCL dst，计数值	¤	—	—	※	—	¤
	带进位循环右移（字节/字）	RCR dst，计数值	¤	—	—	※	—	¤

注：表中“¤”表示运算结果影响标志位；“—”表示运算结果不影响标志位；“※”表示标志位为任意值；“0”表示将标志位置“0”

1. 逻辑运算指令及其应用

逻辑运算指令可对 8 位或 16 位数进行逻辑运算，并且是按位进行操作的。

（1）利用 AND 指令可将操作数中的某些位保持不变，而使其他一些数位清 0，称之为屏蔽；利用 OR 指令可将操作数中的某些位保持不变，而使其他一些位置 1。

（2）利用 XOR 指令可将操作数中的某些位保持不变，而使其他一些数位按位取反。

（3）NOT 指令为单操作数指令，对给定的数逐位取反，该指令执行后不影响任何标志位。

（4）测试指令 TEST 中的两个操作数执行“与”操作后，结果不回送，只影响状态标志

位 PF、SF、ZF，使 CF=0，OF=0，利用该指令，可以在不改变原有操作数的情况下，用来检测某一位或某几位是“0”还是“1”。

【例 3.17】逻辑运算指令使用分析如下：

```
AND  AL,0FH  ;若(AL)=38H,指令执行后(AL)=08H,屏蔽了高 4 位
OR   AL,80H  ;若(AL)=25H,指令执行后(AL)=A5H,将位 7 置 1
XOR  AL,CL   ;若(AL)=69H,(CL)=8AH,指令执行后(AL)=E3H
             ;且 CF=0,OF=0
NOT  AL      ;若(A)=11110000B,指令执行后(AL)=00001111B
```

2. 移位指令及其应用

移位操作类指令可以对字节或字中的各位数据进行算术移位、逻辑移位或循环移位。图 3-5 为各种移位操作的功能示意。其中，C 表示进位标志，M 表示最高位（符号位）。

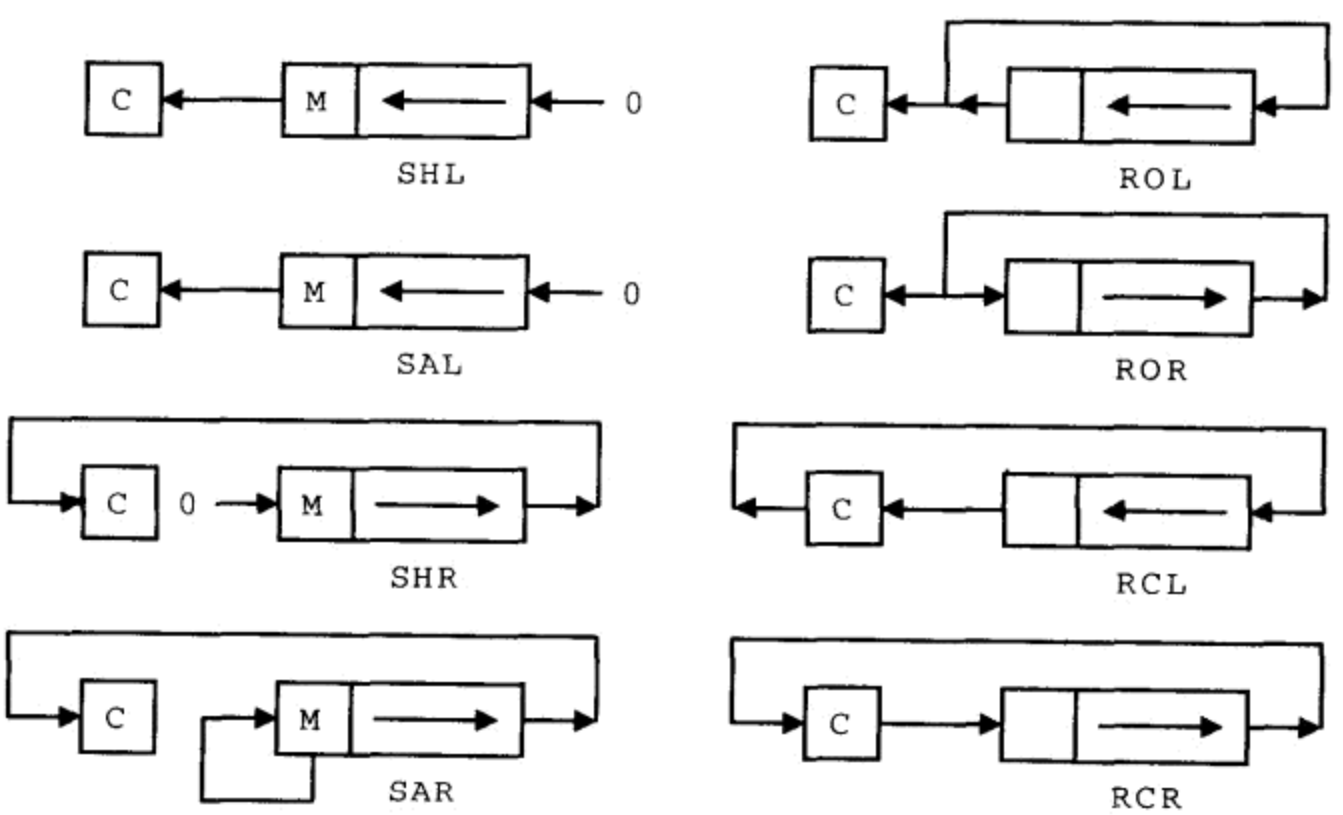


图 3-5　移位操作的功能示意

（1）一般移位指令

- 逻辑左移指令：SHL dst,1/ CL

指令功能为：将 dst 中的二进制位向左移动 1 位或 CL 中指定的位数。左移 1 位时，操作数的最高位移出到 CF 中，最低位补 0。

其中 dst 可以是通用寄存器或存储器操作数，但不能是立即数和段寄存器。操作数可以是 8 位或 16 位。如果移动的位数超过 1 位，则移位次数应放入 CL 中。

SHL 指令将影响 CF、OF、SF、ZF、PF 标志位。利用左移 1 位操作可实现将操作数乘 2 的运算。

例如：

```
SHL AL,1     ;将 AL 中的内容向左移动 1 位,相当于(AL)×2
MOV CL,4     ;移位次数送 CL
SHL AL,CL    ;将 AL 中的内容向左移动 4 位,空出位补 0
```

• 逻辑右移指令：SHR dst,1/ CL

指令功能为：将 dst 中的二进制位向右移动 1 位或 CL 寄存器中指定的位数。右移 1 位时，操作数的最低位移出送到 CF 中，最高位补 0。

SHR 指令将影响 CF、OF、SF、ZF、PF 标志位。利用逻辑右移 1 位操作可实现将无符号操作数除以 2 的运算。

例如：

```
SHR BL,1  ;将 BL 中的内容向右移动 1 位,相当于(BL)/2
```

• 算术左移指令：SAL dst,1/ CL

指令功能为：将 dst 中的二进制位向左移动 1 位或 CL 寄存器中指定的位数。左移 1 位时，操作数的最高位移出送到 CF 中，最低位补 0。

• 算术右移指令：SAR dst,1/ CL

指令功能为：将 dst 中的二进制位向右移动 1 位或 CL 寄存器中指定的位数。右移 1 位时，操作数的最低位移出送到 CF 中，最高位补符号位。

SAR 指令将影响 ZF、SF、PF、OF、CF 标志位。因为执行算术右移操作后，符号位保持不变，所以 OF=0。移出的二进制位送入 CF 中，其他标志位根据结果进行设置。利用算术右移 1 位操作可实现将带符号操作数除以 2 的运算。

（2）循环移位指令

有以下 4 种循环移位指令，它们均影响 CF、OF、SF、ZF、PF 标志位。

• 循环左移指令：ROL dst,1/ CL

指令功能为：将 dst 中的二进制位向左移动 1 位或 CL 寄存器中指定的位数。左移 1 位时，操作数的最高位移出送到 CF 中，同时送至最低位。

• 循环右移指令：ROR dst,1/ CL

指令功能为：将 dst 中的二进制位向右移动 1 位或 CL 寄存器中指定的位数。右移 1 位时，操作数的最低位移出送到 CF 中，同时送至最高位。

• 带进位的循环左移指令：RCL dst,1/ CL

指令功能为：将 dst 中的二进制位向左移动 1 位或 CL 寄存器中指定的位数。左移 1 位时，操作数的最高位移出送到 CF 中，CF 中原有的内容送至最低位。

• 循环右移指令：RCR dst,1/ CL

指令功能为：将 dst 中的二进制位向右移动 1 位或 CL 寄存器中指定的位数。右移 1 位时，操作数的最低位移出送到 CF 中，CF 中原有的内容送至最高位。

3.3.4 串操作类指令

8086 指令系统中设置了串操作指令，其操作对象是内存中地址连续的字节串或字串。在每次基本操作后，能够自动修改地址指针，为下一次操作做准备。基本串操作指令有串传送（MOVS）、串比较（CMPS）、串扫描（SCAS）、串存取（LOPS、STOS）等。任何一个基本串操作指令的前面都可以加一个操作重复前缀使指令操作重复，这样，在处理长数据串时就比用循环软件处理快得多。

串操作指令具有以下几个共同的特点。

（1）约定以 DS:SI 寻址源串，以 ES:DI 寻址目标串。

（2）用方向标志规定串处理方向。若标志 DF=0，则从低地址向高地址方向处理，即字节操作时地址指针加 1，字操作时地址指针加 2；若 DF=1，地址指针减量，即字节操作时地址指针减 1，字操作时地址指针减 2。

（3）可以在串操作指令前加重复前缀对一个以上的串数据进行操作。但这时必须用 CX 作为重复次数计数器，串操作指令每执行一次，CX 值自动减 1，直至减为 0 则停止串操作。

1. 串传送指令

指令格式：MOVS dst,src

MOVSB　　表示字节传送；

MOVSW　　表示字传送。

MOVS 指令给出源操作数和目的操作数，此时指令执行字节操作还是字操作，取决于这两个操作数定义时的类型。

MOVSB 和 MOVSW 指令操作功能为：(DI)←((SI))，表示将 SI 所指示单元的内容传送到 DI 所指示单元，然后修改地址指针。

字节操作时：SI←(SI)±1，DI←(DI)±1

字操作时：　SI←(SI)±2，DI←(DI)±2

当方向标志位 DF=0 时用“+”，DF=1 时用“-”。该指令的执行不影响标志位。

2. 串存储指令

指令格式：STOS dst

STOSB　　表示字节存储；

STOSW　　表示字存储。

操作功能为：

字节操作：(DI)←(AL)　　表示 AL 中的内容送到 DI 所指示的字节单元；

DI ← (DI)±1　　表示 DI 指针加 1 或减 1 调整。

字操作：(DI)←(AX)　　表示 AX 中的内容送到 DI 所指示的字单元；

DI←(DI)±2　　表示 DI 指针加 2 或减 2 调整。

该指令把 AL 或 AX 中的内容存入由 DI 指示的附加段，并根据 DF 的值及数据类型修改 DI 中的内容。该指令执行前要将存入的内容预先放到 AL 或 AX 中，并设置 DF、DI 初始值。

STOS 指令的执行不影响标志位。这条指令和 REP 指令配合使用，可用来将存储区中的某一连续区域放入相同的内容。

3. 取串指令

指令格式：LODS dst

LODSB　　表示取字节串；

LODSW　　表示取字串。

操作功能为：

字节操作：AL←((SI))　　表示 SI 所指示字节单元内容送至 AL；

SI←(SI)±1　　表示 SI 指针加 1 或减 1 调整。

字操作：　AX←((SI))　　表示 SI 所指示字单元内容送至 AX；

　　SI←(SI)±2　　表示 SI 指针加 2 或减 2 调整。

该指令把 SI 指示的数据段中的字节数据或字数据传送至 AL 或 AX，并根据 DF 的值及数据类型来调整 SI 中的内容。该指令的执行不影响标志位。

LODS 指令用来从数据段取出一个字或字节，一般不和重复操作前缀指令配合使用。

4. 串比较指令

指令格式：CMPS　dst,src

　　CMPSB　　表示字节串比较；

　　CMPSW　　表示字串比较。

操作功能为：

字节操作：((SI))—((DI))　　表示 SI 所指示单元的内容减去 DI 所指示单元内容；

　　SI←(SI)±1　　表示 SI 指针加 1 或减 1 调整；

　　DI←(DI)±1　　表示 DI 指针加 1 或减 1 调整。

字操作：((SI))-((DI))　　表示 DI 所指示单元的内容减去 SI 所指示单元内容；

　　SI←(SI)±2　　表示 SI 指针加 2 或减 2 调整；

　　DI←(DI)±2　　表示 DI 指针加 2 或减 2 调整。

该指令完成两个字节数据或字数据相减，结果不回送，只影响状态标志位，并根据 DF 的值及数据类型来修改 DI 的内容。设置 SI 指向被减数，DI 指向减数，并设置 DF 值。

CMPS 和 REPE/PEPZ 或 REPNE/REPNZ 指令配合使用可用来判断两个字符串是否相等。

【例 3.18】已知在内存中有两个字符串 STR1 和 STR2，比较两个字符串是否相等，若相等，将 FLAG 单元置 1，否则置 0。程序段如下。

```
        MOV   FLAG,1              ;标志单元首先置 1
        LEA   SI, STR1            ;SI 指针指向源字符串首单元
        LEA   DI, STR2            ;DI 指针指向目标字符串首单元
        MOV   CX,CN               ;字符串长度送 CX
        CLD                       ;DF 标志位清 0
        CMPSB                     ;按字节进行比较
        JZ    NEXT                ;字符串相等，转至 NEXT 位置
        MOV   FLAG,0              ;否则将 FLAG 单元清 0
NEXT:   HLT
```

5. 串搜索指令

指令格式：SCAS dst,src

　　SCASB　　表示字节串搜索；

　　SCASW　　表示字串搜索。

操作功能为：

字节操作：　AL—((DI))　　表示 AL 中的内容减去 DI 所指示字节单元内容；

　　DI←(DI)±1　　表示 DI 指针加 1 或减 1 调整。

字操作：　AX—((DI))　　表示 AL 中的内容减去 DI 所指示字单元内容；

　　DI←(DI)±2　　表示 DI 指针加 2 或减 2 调整。

该指令完成 AL 或 AX 中内容减去字节数据或字数据，结果不回送，只影响状态标志位，

并根据 DF 的值及数据类型来调整 DI 的内容。该指令执行前，AL 或 AX 中设置被搜索的内容，DI 指向被搜索的字符串的首单元，并设置 DF 值。

6. 方向标志处理指令

（1）方向标志清除指令 CLD：该指令使 DF=0，在执行串操作指令时，可使地址自动增量。

（2）设置方向标志指令 STD：该指令使 DF=1，在执行串操作指令时，可使地址自动减量。

7. 重复操作前缀指令

（1）重复操作前缀

指令格式：REP String Primitive

其中：String Primitive 可为 MOVS、STOS 或 LODS 串操作

操作功能：REP 前缀使后面的串指令重复执行，其次数预先送入 CX 寄存器中，每执行一次串操作指令，CX 寄存器中的内容自动减 1，一直重复到(CX)=0 指令才结束。

（2）相等/为零时重复操作前缀

指令格式：REPE/REPZ String Primitive

其中：String Primitive 可为 CMPS、SCAS 串操作。

操作功能：当(CX)≠0 且比较相等的情况下重复执行串指令，最大的比较次数送入 CX 寄存器中。

（3）不相等/不为零时重复操作前缀

指令格式：REPNE/REPNZ String Primitive

其中：String Primitive 可为 CMPS、SCAS 串操作。

执行操作：当(CX)≠0 且比较不相等的情况下重复执行串指令，最大的比较次数送入 CX 寄存器中。

3.3.5　控制转移类指令

一般情况下，程序是按指令顺序地逐条执行的，但实际上经常需要改变程序的执行流程。控制转移类指令用来改变程序执行的方向，即修改 IP 和 CS 的值。

按转移位置可将转移指令分为段内转移和段间转移。若指令给出改变 IP 中内容的信息，转移的目标位置和转移指令在同一个代码段，则称为段内转移；若指令给出改变 IP 中内容的信息，又给出改变 CS 中内容的信息，转移的目标位置和转移指令不在同一个代码段，则称为段间转移。

根据转移指令的功能，可分为无条件转移指令、条件转移指令、循环控制指令、子程序调用和返回指令等。

1. 无条件转移指令

无条件转移（JMP）指令中作为转移目的地址的操作数，在汇编语言格式中可直接使用符号地址。在汇编之后可得到相应的段地址及偏移量。

JMP 指令控制处理机转移到指定的位置去执行程序，指令中必须给出转移位置的目标地址，通常有以下 5 种形式。

（1）JMP SHORT opr　　表示段内直接短转移指令；
（2）JMP NEAR PTR opr　　表示段内直接近转移指令；
（3）JMP WORD PTR opr　　表示段内间接转移指令；
（4）JMP FAR PTR opr　　表示段间直接转移指令；
（5）JMP DWORD PTR opr　　表示段间间接转移指令。

opr 为转移目标地址，可直接使用符号地址（标号）。属性运算符 SHORT 指示汇编程序将符号地址汇编成 8 位偏移量，指令执行时，转移的目标地址为当前的 IP 值与指令中给定的 8 位偏移量之和。属性运算符 NEAR PTR 指示汇编程序将符号地址汇编成 16 位偏移量，指令执行时，转移的目标地址为当前的 IP 值与指令中给定的 16 位偏移量之和。偏移地址紧跟在指令操作码之后，由汇编程序计算得出。

有效地址 EA 的值是由 opr 的寻址方式决定的。若是寄存器寻址，指令中直接给出寄存器号，寄存器中的内容送到 IP 中；若是存储器寻址，按存储器寻址方式计算出有效地址和物理地址，用这个物理地址去读取内存中的数据送给 IP 指针。

【例 3.19】执行段内间接转移指令：JMP WORD PTR[BX]

若给定(IP)=0012H，(BX)=0100H，(DS)=2000H，(20100H)=80H，(20101H)=00H，目标地址为存储器寻址。

有效地址计算：EA=(BX)=0100H，物理地址计算：PA=DS×10H+EA=20100H

所以，指令执行后(IP)=0080H。

如果是段间直接转移，则执行操作：IP←opr 的偏移地址；CS←opr 所在段的段地址。

如果是段间间接转移，则执行操作：IP←(EA)；CS←(EA+2)。EA 的值是由 opr 的寻址方式决定的，按寻址方式计算出有效地址和物理地址，用物理地址去读取内存中连续的两个字数据，其中低位字数据送给 IP，高位字数据送给 CS。

【例 3.20】执行段间间接转移指令：JMP DWORD PTR[BX+20H]

若给定(CS)=3000H，(IP)=0012H，(BX)=0100H，(DS)=2000H，(20120H)=A0H，(20121H)=00H，(20122H)=00H，(20123H)=50H

有效地址计算：EA=(BX)+20H=0120H；物理地址计算：PA=DS×10H+EA=20120H

指令执行后：(IP)=0080H，(CS)=5000H。

2. 条件转移指令

8086 指令系统的条件转移指令以某些标志位的状态或有关标志位的逻辑运算结果作为依据，以此决定是否转移。这些标志位通常由条件转移指令的上一条指令所设置。条件转移指令将根据这些标志位的状态，判断是否满足对应的测试条件。若满足条件，则转移到指令指定的地方，否则继续执行条件转移指令之后的指令。此外还有无符号数比较转移指令和带符号数比较转移指令两类，这两类指令一般在 CMP 指令后使用。

条件转移指令的助记符、指令名称、转移条件等见表 3-4。

表 3-4　条件转移指令

指令助记符	指令名称	转移条件	指令助记符	指令名称	转移条件
JZ/JE	为 0/相等	ZF=1	JC/JB/JNAE	进位/低于/不高于等于	CF=1
JNZ/JNE	不为 0/不相等	ZF=0	JNC/JNB/JAE	无进位/不低于/高于等于	CF=0
JS	符号为负	SF=1	JBE/JNA	低于等于/不高于	CF=1 或 ZF=1
JNS	符号不为负	SF=0	JNBE/JA	不低于等于/高于	CF=0 且 ZF=0
JO	结果溢出	OF=1	JL/JNGE	小于/不大于等于	SF≠OF
JNO	结果无溢出	OF=0	JNL/JGE	不小于/大于等于	SF=OF
JP/JPE	1 的个数为偶数	PF=1	JLE/JNG	小于等于/不大于	ZF≠0 或 ZF=1
JNP/JPO	1 的个数为奇数	PF=0	JNLE/JG	不小于等于/大于	SF=O 且 ZF=0

【例 3.21】已知在内存中有两个无符号字节数据 X1 和 X2，比较两个数是否相等，若相等，则将 RESULT 单元置 1，否则置 0。程序段如下：

```
MOV   AL,X1              ;将第一个数取出送至 AL 中
CMP   AL,X2              ;和第二个数进行比较
JZ    NEXT               ;相等则转到 NEXT 位置执行
MOV   RESULT,0           ;否则,将 RESULT 单元置 0
JMP   EXIT               ;然后转到 EXIT 位置
NEXT: MOV   RESULT,1     ;将 RESULT 单元置 1
EXIT: HLT
```

【例 3.22】已知在内存中有两个无符号数字节数据 NUM1 和 NUM2，找出其中的最大数送到 MAX 单元。程序段如下：

```
      MOV   AL,NUM1      ;将第一个数取出送到 AL 中
      CMP   AL,NUM2      ;和第二个数进行比较
      JA    NEXT         ;第一个数大于第二个数则转到 NEXT 位置
      MOV   AL,NUM2      ;否则将第二个数取出送到 AL 中
NEXT: MOV   MAX,AL       ;AL 中为最大数送到 MAX 单元
```

本题若改为带符号数，则程序段应为：

```
      MOV   AL,NUM1      ;将第一个数取出送到 AL 中
      CMP   AL,NUM2      ;和第二个数进行比较
      JG    NEXT         ;第一个数大于第二个数则转到 NEXT 位置
      MOV   AL,NUM2      ;否则将第二个数取出送到 AL 中
NEXT: MOV   MAX,AL       ;AL 中为最大数,送到 MAX 单元
```

3. 循环控制指令

循环程序是一种常用的程序结构，为了加快对循环程序的控制，8086 系统专门设置了一组循环控制指令，如表 3-5 所示，循环计数值在 CX 中。

表 3-5 循环控制指令

指令名称	助 记 符	测试条件
循环	LOOP 目标标号	CX←CX-1，CX≠0
等于/结果为 0 循环	LOOPE/LOOPZ 目标标号	CX←CX-1，ZF=1 且 CX≠0
不等于/结果不为 0 循环	LOOPNE/LOOPNZ 目标标号	CX←CX-1，ZF=0 且 CX≠0
CX 内容为 0 转移	JCXZ 目标标号	CX=0

循环控制指令只是根据结果的标志位状态进行控制操作，指令本身不影响标志位。循环控制指令用于控制程序的循环，其控制转向的目的地址是在以当前指令指针 IP 内容为中心的 -128～+127 的范围内，指令采用 CX 作为计数器，每执行一次循环，CX 内容减 1，直到为 0，循环结束。

LOOP 指令是以寄存器 CX 的内容作为计数控制，作(CX)←(CX)-1 的操作，并判断 CX；当 CX≠0 时，转移到由操作数指示的目的地址，即(IP)←(IP)+位移量，进行循环；当 CX=0 时，结束循环。

LOOPZ/LOOPE 指令可完成当 ZF=1 且 CX≠0 条件下的循环操作。在 LOOPZ 或 LOOPE 所做的控制循环操作过程中，除了进行(CX)←(CX) -1 的操作，还要判断(CX)是否为零。此外，还将判断标志位 ZF 的值。

LOOPNZ 或 LOOPNE 指令可完成当 ZF=0 且(CX)≠0 的条件下的循环控制操作。其操作过程类似于 LOOPZ 或 LOOPE 指令。

【例 3.23】在内存中有一个具有 CN 个字节的数据串，首单元地址为 DATA-BUF，找出第一个不为 0 的数据的地址并送到 ADDR 单元中。程序段如下：

```
    MOV   SI,OFFSET  DATA-BUF      ;取首单元地址
    MOV   CX,CN                    ;计数器初值
    MOV   AL,0                     ;AL 清零
    DEC   SI                       ;循环初始化
LP: INC   SI                       ;指针增 1
    CMP   AL,[SI]                  ;内存中的数据和 AL 中的内容比较
    LOOPZ  LP                      ;为 0 且未比较到末尾转 LP 位置继续
    JZ    EXIT                     ;否则判断 ZF,若为 1,转 EXIT
    MOV   ADDR,SI                  ;ZF 为 0,SI 中的内容送 ADDR
EXIT: HLT
```

该程序段中循环结束的情况有两种：一种是找到了不为 0 的数据，一种是 CN 个数据比较结束后未找到。所以，退出循环后要判断 ZF 是否为 1，ZF=1 说明所有数据都为 0，否则就是找到了不为 0 的数据，并且由 SI 指示其存储地址。

4. 子程序调用和返回指令

在程序设计过程中，通常把功能分解为若干个小的模块。每一个小功能模块对应一个过程。在汇编语言中，过程又称为子程序。程序中可由调用程序（称为主程序）调用这些子程序，子程序执行完毕后返回主程序继续执行。

子程序调用分为段内和段间调用。

指令格式为：CALL NEAR PTR opr　　表示段内调用；

CALL FAR PTR opr　　表示段间调用；
RET　　表示子程序返回。

其中，opr 为子程序名（即子程序第一条指令的符号地址）。

【例 3.24】在主程序中执行一条段内调用语句，调用形式如下：

```
MAIN PROC FAR                    ;定义主程序
     MOV AX,DATA                 ;DS 初始化
     MOV DS,AX
          ⋮
     CALL DISPLAY                ;调用子程序 DISPLAY
          ⋮
DISPLAY PROC NEAR                ;定义子程序
     PUSH AX                     ;保护现场
     PUSH BX
          ⋮
     RET                         ;子程序返回
```

3.3.6　处理器控制类指令

这类指令主要用于修改状态标志位、控制 CPU 的功能，如使 CPU 暂停、等待、空操作等，如表 3-6 所示。

表 3-6　处理器控制类指令

类　别	操作码（助记符）	名　称
标志操作	STC	进位标志置 1，即 CF=1
	CLC	进位标志置 0，即 CF=0
	CMC	进位标志取反
	STD	方向标志置 1，即 DF=1
	CLD	方向标志置 0，即 DF=0
	STI	中断标志置 1，即 IF=1
	CLI	清除中断标志，即 IF=0
CPU 控制	HLT	停机
	WAIT	等待
	NOP	空操作
	LOCK	封锁总线
	ESC	交权

在 CPU 控制指令中，HLT 指令使程序暂停执行，这时 CPU 不进行任何操作，当 CPU 发生复位或发生外部中断时，CPU 脱离暂停状态，HLT 指令可用于程序中等待中断；WAIT 指令在 CPU 测试引脚为高电平（无效）时，使 CPU 进入等待状态。这时，CPU 不做任何操作。当 CPU 测试引脚为低电平（有效）时，CPU 脱离等待状态，继续执行 WAIT 指令后面的指令；NOP 指令只做空操作，不执行其他任何操作，占用一个字节存储单元，空耗一个指令执

行周期，该指令常用于程序调试，还可以实现软件延时；LOCK 指令是一个前缀，可以加在任何指令之前，CPU 执行该指令时封锁总线；ESC 指令将处理器的控制权交给协处理器，如在 8086 系统中可加入浮点运算协处理器 8087，若 8086 微处理器发现是一条浮点指令时，就利用 ESC 指令将浮点指令交给 8087 执行。

3.4 DOS 和 BIOS 中断调用

3.4.1 DOS 功能调用

DOS 功能调用可完成对文件、设备、内存的管理。对用户来说，这些功能模块就是几十个独立的中断服务程序，这些程序的入口地址已由系统置入中断向量表中，在汇编语言程序中可用中断指令直接调用。

1. 系统功能调用的方法

要完成 DOS 系统的功能调用，按如下基本步骤操作：

（1）将入口参数送到指定寄存器中。

（2）将子程序功能号送入 AH 寄存器中。

（3）使用“INT 21H”指令转入子程序入口执行相应操作。

2. 常用的几种系统功能调用

（1）AH=01H 表示带显示的键盘输入。

01 号功能调用是系统扫描键盘并等待键盘输入单个字符，当有键按下时，先检查是否是 Ctrl+Break 键，若是则将字符的键值（ASCII 码）送入 AL 寄存器中，并在屏幕上显示该字符。此调用没有入口参数。

使用格式如下：

```
MOV AH,01H
INT 21H
```

（2）AH=02H 表示从显示器上输出单个字符。

02 号功能调用的入口参数：被显示字符的 ASCII 码送入 DL 寄存器。

【例 3.25】要在屏幕上显示“$”符号，可用以下指令序列：

```
MOV DL,'$'
MOV AH,02H
INT 21H
```

（3）AH=09H 表示在显示器上输出字符串。

09 号功能调用是将指定的字符串送显示器显示，字符串事先存放在内存的数据缓冲区中，字符串的首地址送入指定位置，并要求字符串必须以'$'结束。

入口参数：DS:DX 指向缓冲区中字符串的首单元。

（4）AH=0AH 表示字符串输入到缓冲区。

0A 号功能调用是将键盘输入的字符串写入内存缓冲区中。为了接收字符，首先在内存区中定义一个缓冲区，其中第一个字节为缓冲区的字节个数，第二个字节用做系统填写实际键入的字符总数，从第三个字节开始存放字符串。输入的字符以回车键结束，如果实际键入的字符不足以填满缓冲区，则其余字节补 0；若输入的字符个数大于定义长度，则超出的字符将丢

失，并且响铃警告。

【例 3.26】从键盘输入一个字符，并在屏幕上显示输出，以键入字符'$'作为停止操作的标志。程序段如下：

```
START: MOV AH,01H              ;输入单个字符
       INT 21H
       CMP AL,'$'              ;判断是否是结束标志
       JZ NEXT                 ;若是则转 NEXT
       MOV DL,AL
       MOV AH,02H              ;显示输出字符
       INT 21H
       JMP START
NEXT: HLT
```

DOS 系统的功能调用详细内容可参见后续章节及附录 C。

3.4.2　BIOS 中断调用

IBM PC 系列微机在只读存储器中提供了 BIOS 基本输入/输出系统，它占用系统板上 8KB 的 ROM 区域，又称为 ROM BIOS。

BIOS 为用户程序和系统程序提供主要外设的控制功能，如系统加电自检、引导装入及对键盘、磁盘、磁带、显示器、打印机、异步串行通信口等的控制。计算机系统软件就是利用这些基本的设备驱动程序，完成各种功能操作。每个功能模块的入口地址都在中断矢量表中，通过中断指令 INT n 可以直接调用。n 是中断类型号，每个类型号 n 对应一种 I/O 设备的中断调用，每个中断调用又以功能号来区分其控制功能。

有关 BIOS 中断调用的类型号、功能、入口参数和出口参数可参见后续章节及附录 D。

3.5　Pentium 微处理器新增寻址方式和指令

目前在市场上使用较多的是 Pentium 微处理器，它拥有全新的结构与功能，在指令系统和寻址方式等方面也进行了扩展。下面简要分析 Pentium 微处理器与 8086、80X86 系列芯片在指令及寻址方式等方面中的不同和特点。

3.5.1　Pentium 微处理器的内部寄存器

由于 Pentium 微处理器采用了 32 位指令，其内部寄存器和指令格式与 16 位微处理器不同，主要体现在以下几方面。

（1）指令的操作数可以是 8 位、16 位或 32 位的二进制数。

（2）根据指令的不同，操作数字段可以是 0～3 个。当选用 3 个操作数时，最左边的操作数为目的操作数，右边两个操作数均为源操作数。

（3）在部分不存放结果的单操作数指令中，可以采用立即数作为操作数，且某些指令对操作数的数据类型不是简单地要求一致，而是要有不同的匹配关系。

（4）立即数寻址方式中操作数可以是 32 位的立即数，寄存器寻址方式中操作数可以是 32 位通用寄存器，存储器操作数寻址方式既可采用 16 位的地址寻址方式，也可采用 32 位的扩展地址寻址方式。

（5）16 位微处理器原有的 4 个通用数据寄存器扩展为 32 位，更名为 EAX、EBX、ECX 和 EDX。

（6）原有的 4 个用于内存寻址的通用地址寄存器扩展为 32 位，更名为 ESI、EDI、EBP、ESP。

（7）指令指针寄存器扩展为 32 位，更名为 EIP，实地址方式下仍然可以使用它的低 16 位 IP。

（8）在原有 4 个段寄存器基础上增加 2 个新的段寄存器 FS 和 GS，段寄存器长度仍然为 16 位，但是它存放的不再是“段基址”，而是代表这个段编号的 13 位二进制数，称为“段选择字”。

（9）增加了 4 个系统地址寄存器，分别是存放“全局段描述符表”首地址的 GDTR，存放“局部段描述符表”选择字的 LDTR，存放“中断描述符表”首地址的 IDTR，存放“任务段”选择字的“任务寄存器”TR。

（10）标志寄存器扩展为 32 位，更名为 EFLAGS，除了原有的状态、控制标志外，还增加了表示 IO 操作特权级别的 IOPL，表示进入虚拟 8086 方式的 VM 标志等。

（11）新增 5 个 32 位的控制寄存器，命名为 CR_0～CR_4，CR_0 寄存器的 PE=1 表示目前系统运行在“保护模式”，PG=1 表示允许进行分页操作。CR_3 寄存器存放“页目录表”的基地址。

（12）新增 8 个用于调试的寄存器 DR_0～DR_7，2 个用于测试的寄存器 TR_6～TR_7。

3.5.2 Pentium 微处理器的新增寻址方式

Pentium 微处理器与 8086 相比新增加了 3 种寻址方式，分别是比例变址寻址方式、基址加比例变址寻址方式和带位移量的基址加比例变址寻址方式。

1. 比例变址寻址方式

其有效地址为： EA=[变址寄存器]×比例因子+位移量

该方式下的 EA 是变址寄存器的内容乘以指令中指定的比例因子再加上位移量之和，乘比例因子的操作是在 CPU 内部由硬件完成的。

这种寻址方式与寄存器相对寻址相比增加了比例因子，其优点在于：对于元素大小为 2、4、8 字节的数组，可以在变址寄存器中给出数组元素下标，而由寻址方式控制直接用比例因子把下标转换为变址值。

例如：

```
MOV EAX,COUNT[ESI×4]
```

2. 基址加比例变址寻址方式

其有效地址为：EA=[基址寄存器]+[变址寄存器]×比例因子

该方式下 EA 是变址寄存器的内容乘以比例因子再加上基址寄存器的内容之和，这种寻址方式与基址变址寻址方式相比增加了比例因子。

例如：

```
MOV ECX,[EAX][EDX×8]
```

3．带位移量的基址加比例变址寻址方式

其有效地址为：EA=[基址寄存器]+[变址寄存器]×比例因子+位移量

操作数的 EA 是变址寄存器的内容乘以比例因子，加上基址寄存器的内容，再加上位移量之和，所以有效地址由四部分组成。在寻址过程中，变址寄存器内容乘以比例因子的操作也是在 CPU 内部由硬件来完成的。

这种寻址方式比相对基址变址寻址方式增加了比例因子，便于对元素为 2、4、8 字节的二维数组的处理。

例如：

```
MOV EAX,TABLE[EBP][EDI × 4]
```

在对操作数进行寻址时要注意以下几点。

（1）Pentium 微处理器在实地址方式下，一个段的最大长度仍然为 64KB，段基址是 16 的倍数，用段寄存器存放段基址。

（2）Pentium 有 6 个段寄存器，在寻址内存操作数时，指令给出的内存操作数的地址均为有效地址。

（3）内存操作数有效地址可由一个 32 位基址寄存器、一个可乘上比例因子 1、2、4、8 的 32 位变址寄存器和一个不超过 32 位的常数偏移量组成。

（4）32 位寻址情况下，8 个 32 位通用寄存器均可作基址寄存器，其中 ESP、EBP 以 SS 为默认段寄存器，其余 6 个通用寄存器均以 DS 为默认段寄存器。

（5）基址字段、变址字段、偏移量字段可任意省略其一或其二。

（6）比例因子要与变址字段联合使用，如省略了变址字段则比例因子不能独立存在。

3.5.3　Pentium 系列微处理器专用指令

Pentium 系列处理器的指令集向上兼容，它保留了 8086 和 80X86 微处理器系列的所有指令，因此，所有早期的软件可直接在 Pentium 机上运行。

从微处理器的指令系统中可以看出，自 1985 年 Intel 公司推出 32 位微处理器 80386 以来，始终使用着几乎一样的指令系统，只是每提高一代便追加很少几条指令。Pentium 微处理器的指令集与 80486 相比变化不大，其主要特色是拥有能使系统程序员实现多路处理 Cache 一致性协议的新指令，以及一条 8 字节比较交换指令和一条微处理器识别指令。

Pentium 处理器指令集中新增加了以下 3 条专用指令。

1．比较和交换 8 字节数据指令——CMPXCHG8B

指令格式：CMPXCHG8B opr1,opr2

该指令执行 64 位数据的比较和交换操作。执行时将存放在 opr1（64 位存储器）中的目的操作数与累加器 EDX:EAX 的内容进行比较，如果相等，则 ZF=1，并将源操作数 opr2（规定为 EDX:EAX）的内容送入 opr1；否则 ZF=0，并将 opr1 送到相应的累加器。

例如：

```
CMPXCHG8B mem,ECX:EBX
```

指令执行后，如果 EDX:EAX=[mem]，则 ECX:EBX→[mem]，ZF=1

否则[mem]→EDX:EAX 且 ZF=0

2. CPU 标识指令——CPUID

指令格式：CPUID

该指令执行后可以将有关 Pentium 处理器的型号和特点等系列信息返回到 EAX 中。在执行 CPUID 指令前，EAX 寄存器必须设置为 0 或 1，根据 EAX 中设置值的不同，软件会得到不同的标志信息。

3. 读时间标记计数器指令——RDTSC

指令格式：RDTSC

在 Pentium 处理器有一个片内 64 位计数器，称为时间标记计数器 TSC。计数器的值在每个时钟周期都自动加 1，执行 RDTSC 指令可以读出计数器 TSC 中的值，并送入寄存器 EDX:EAX 中，EDX 保存 64 位计数器中的高 32 位，EAX 保存低 32 位。

一些应用软件需要确定某个事件已执行了多少个时钟周期，在执行该事件之前和之后分别读出时钟标志计数器的值，计算两次值的差就可得出时钟周期数。

3.5.4 Pentium 系列微处理器控制指令

Pentium 处理器指令集中新增加了 3 条系统控制指令。

1. 读专用模式寄存器指令——RDMSR

RDMSR 指令使软件可访问专用模式寄存器的内容，执行指令时在访问的模式专用寄存器与寄存器组 EDX:EAX 之间进行 64 位的读操作。

2. 写专用模式寄存器指令——WRMSR

WRMSR 指令执行时在访问的专用模式寄存器与寄存器组 EDX:EAX 之间进行 64 位的写操作。

Pentium 处理器有两个专用模式寄存器，即机器地址检查寄存器（MCA）和机器类型检查寄存器（MCT）。

如果要访问机器地址检查寄存器 MCA，指令执行前需将 ECX 置为 0；而为了访问机器类型检查寄存器 MCT，需要将 ECX 置为 1。

3. 恢复系统管理模式指令——RSM

Pentium 处理器有一种称为系统管理模式（SMM）的操作模式，这种模式主要用于执行系统电源管理功能。外部硬件的中断请求使系统进入 SMM 模式，执行 RSM 指令后返回原来的实模式或保护模式。

本章小结

本章针对 8086 寻址方式和指令系统分别介绍了基本概念和各类指令的使用。8086 微处理器指令按照操作数的设置可分为隐含操作数指令、单操作数指令和双操作数指令 3 种；按操作数的存放位置有立即数、寄存器操作数、存储器操作数和输入/输出端口操作数 4 种类型。

指令通常并不直接给出操作数，而是给出操作的存放地址。寻找操作数地址的方式称为寻址方式。8086 有立即数寻址、寄存器寻址、直接寻址、寄存器间接寻址、相对寻址、基址变址寻址 6 种基本寻址方式。要明确各种寻址方式的区别和特点，重点掌握存储器寻址方式

中的有效地址和物理地址的计算方法。

指令系统是程序设计的基础，Intel 8086 微处理器的指令系统按功能分为数据传送类、算术运算类、逻辑运算（位操作）类、串操作类、控制转移类、处理器控制类等指令。在学习这些指令时，应当从指令的功能、操作数的个数、源操作数与目的操作数可以采用的寻址方式及是否影响标志位等几方面加以理解，为汇编语言的学习提供基础。

本章还介绍了 Pentium 微处理器的新增指令和寻址方式，以供读者在实际应用中作为参考。

思考与练习题

1. 什么叫寻址和寻址方式？8086 指令系统有哪几种寻址方式？各自有何特点？
2. 指出下列指令中源操作数和目的操作数的寻址方式。

   （1）MOV AX,100H　　（2）MOV CX,AX

   （3）ADD [SI],1000　　（4）SUB BX,[SI+100]

   （5）MOV [BX+300],AX　　（6）AND BP,[DI]
3. 分析下列指令的正误，对错误指令说明出错误原因并加以改正。

   （1）MOV [1200],23H　　（2）MOV 1020H,CX

   （3）MOV [1000H],[2000H]　　（4）MOV IP,000H

   （5）PUSH AL　　（6）OUT CX,AL

   （7）IN AL,[80H]　　（8）MOV CL,3300H
4. 给定 (DS)=2000H，(BX)=0100H，(SI)=0002H，(20100H)=12H，(20101H)=34H，(20102H)=56H，(20103H)=78H，(21200H)=2AH，(21201H)=4CH，(21202H)=B7H，(21203H)=65H。

   试分析下列指令执行后，AX 寄存器中的内容。

   （1）MOV AX,1200H　　（2）MOV AX,BX

   （3）MOV AX,[1200H]　　（4）MOV AX,[BX]

   （5）MOV AX,1100H[BX]　　（6）MOV AX,[BX＋SI]

   （7）MOV AX,[1100H+BX+SI]
5. 已知(AX)=75A4H，CF=1，分别写出下列指令执行后的结果：

   （1）ADD AX,08FFH　　（2）INC AX

   （3）SUB AX,4455H　　（4）AND AX,0FFFH

   （5）OR AX,0101H　　（6）SAR AX,1

   （7）ROR AX,1　　（8）ADC AX,5
6. 给定(SS)=8000H，(SP)=2000H，(AX)=7A6CH，(DX)=3158H。执行下列程序段，画出每条指令执行后寄存器的内容和堆栈存储内容的变化情况。

```
PUSH AX
PUSH DX
POP BX
POP CX
```

7. 设 AX、BX 中保存有带符号数，CX、DX 中保存无符号数，请写出实现以下功能的指令或程序段。

若(CX)<(DX)，则转移到 NEXT1。

若(AX)>(BX)，则转移到 NEXT2。

若(CX)=0，则转移到 NEXT3。

若 AX 中内容为负，则转移到 NEXT4。

8. 试分析下面程序段执行完后，BX 的内容是什么。

```
MOV BX,1030H
MOV CL,3
SHL BX,CL
DEC BX
```

第4章 汇编语言

本章主要讲解汇编语言的语句格式以及汇编语言程序设计的基本步骤和基本方法。概述汇编语言中常用的伪指令，通过实例来说明程序的组成结构，重点讨论顺序结构程序、分支结构程序、循环结构程序和子程序的基本结构及设计方法。

读者通过本章的学习，应该建立起汇编语言程序设计的整体思路并掌握设计方法，灵活运用 CPU 指令系统和 MASM 中提供的伪指令进行程序设计。在熟悉相关类别程序的结构和组成基础上，不断提高汇编语言程序设计的技巧，为后期的开发应用打下良好基础。

4.1 汇编语言简述

4.1.1 汇编语言语句格式

1. 汇编语言和汇编程序的基本概念

汇编语言是一种面向 CPU 指令系统的程序设计语言，它采用指令助记符来表示操作码和操作数，用符号地址表示操作数地址。用汇编语言编写的程序能够直接利用硬件系统的特性，直接对位、字节、字寄存器、存储单元、I/O 端口等进行处理，同时也能直接使用 CPU 指令系统和指令系统提供的各种寻址方式编制出高质量的程序，这种程序不但占用内存空间少，而且执行速度快。

汇编语言编写的源程序在输入计算机后，需要翻译成目标程序，计算机才能执行相应指令，这个翻译过程称为汇编，完成汇编任务的程序称为汇编程序。汇编程序是将汇编语言源程序翻译成机器能够识别和执行的目标程序的一种系统程序。

汇编程序是最早也是最成熟的一种系统软件，它除了能够将汇编语言源程序翻译成机器语言程序，还能根据用户的要求自动分配存储区域，包括程序区、数据区、暂存区等；自动把各种进制数转换成二进制数，把字符转换成 ASCII 码，计算表达式的值等；自动对源程序进行检查，给出错误信息，如非法格式、未定义的助记符、标号、漏掉的操作数等。具有这些功能的汇编程序称为基本汇编 ASM（Assembler）。

在基本汇编的基础上，进一步允许在源程序中把一个指令序列定义为一条宏指令，并包含大量伪指令的汇编程序，叫做宏汇编 MASM（MacroAssembler）。它包含全部基本汇编 ASM 的功能，还增加了宏指令、结构、记录等高级汇编语言功能。

汇编程序以汇编语言源程序文件作为输入，并由它产生两种输出文件：目标程序文件和源程序列表文件。目标程序文件经连接定位后由计算机执行；源程序列表文件将列出源程序、目标程序的机器语言代码及符号表。符号表是汇编程序所提供的一种诊断手段，它包括程序中所用的所有符号和名字，以及这些符号和名字所指定的地址。如果程序出错，可以较容易地从这个符号表中检查出错误。

在编写源程序时，要严格遵守汇编语言程序的书写规范，否则就会出错。

2. 汇编语言语句格式

汇编语言的语句可以由 1~4 部分组成：[名字]　操作符　[操作数]　[;注释]

其中带方括号的部分表示任选项。

（1）名字字段：名字是一个符号，表示本条语句的符号地址。名字可以是标号和变量，统称为标识符。它是由字母打头的字符串。在汇编语言程序中，指令语句的名字之后要用冒号“:”，而伪指令语句中名字之后不要加冒号“:”。

标号和变量具备下面 3 种属性。

- 段属性：该属性定义了标号和变量的段起始地址，其值必须在一个段寄存器中。标号的段是它所出现的对应代码段，由 CS 指示。变量的段通常由 DS 或者 ES 指示。
- 偏移属性：该属性表示标号和变量相距段起始地址的字节数，该数是一个 16 位无符号数。
- 类型属性：该属性对于标号而言，用于指出该标号是在本段内引用还是在其他段中引用。标号的类型有 NEAR（段内引用）和 FAR（段间引用）。对于变量，其类型属性说明变量有几个字节长度，这一属性由定义变量的伪指令确定。

（2）操作符字段：操作符可以是机器指令、伪指令和宏指令的助记符。

机器指令是 CPU 指令系统中的指令，汇编程序将其翻译成对应的机器码。伪指令则不能翻译成对应的机器码，它只是在汇编过程中完成相应的控制操作，又称为汇编控制指令。宏指令则是有限的一组指令（机器指令、伪指令）定义的代号，汇编时将根据其定义展开成相应的指令。

（3）操作数字段：是操作符的操作对象。当有两个或两个以上的操作数时，各操作数之间用逗号隔开。操作数一般有常数、寄存器、标号、变量和表达式等几种形式。

- 常数：常数是没有属性的纯数，它的值在汇编时已完全确定，程序运行过程中，不会发生变化。在 8086 宏汇编中，允许有二进制、八进制、十进制、十六进制常数和字符串常数。在指令中，常数通常被称为立即数，它只能用做源操作数，不能作为目标操作数。它的允许取值范围由指令中的目标操作数的形式自动确定为 8 位或 16 位。
- 存储器操作数：包括标号和变量。它可以作为源操作数，也可以作为目标操作数，但不能同时充当源操作数和目标操作数。标号是可执行的指令性语句的符号地址，它可以作为 JMP 和 CALL 指令的转向目标操作数。变量是指存放在某些存储单元中的数据，这些数据在程序运行期间是可以改变的。变量通过标识符来引用，可以作为存储器访问指令的源操作数和目标操作数。
- 表达式：由常数、寄存器、标号、变量与一些运算符相结合而成，一般有数字表达式和地址表达式两种。

（4）注释字段：以“;”开头的说明部分，是语句的非执行部分，可以根据需要来写。一般情况下，注释用来说明一段程序或几条语句的功能，以增加程序的可读性，便于修改和调试。

3. 汇编语言中的标识符、表达式和运算符

（1）标识符

汇编语言语句格式第一个字段是它的名字字段，名字可以是标号或变量，这两者又称为标识符。

（2）表达式和运算符

在表达式中，运算符充当着重要的角色。8086 宏汇编有算术运算符、逻辑运算符、关系运算符、分析运算符和综合运算符共 5 种，如表 4-1 所示。

表 4-1　8086 汇编语言中的运算符

算术运算符	逻辑运算符	关系运算符	分析运算符	综合运算符
+（加法）	AND（与）	EQ（相等）	SEG（求段基值）	PTR
-（减法）	OR（或）	NE（不相等）	OFFSET（求偏移量）	段属性前缀
×（乘法）	XOR（异或）	LT（小于）	TYPE（求变量类型）	THIS
/（除法）	NOT（非）	GT（大于）	LENGTH（求变量长度）	SHORT
MOD（求余）		LE（小于或等于）	SIZE（求字节数）	HIGH
SHL（左移）		GE（大于或等于）		LOW
SHR（右移）				

算术运算符用于完成算术运算，其中加、减、乘、除运算都是整数运算，结果也是整数。除法运算得到的是商的整数部分。求余运算是指两数整除后所得到的余数。

逻辑运算符的作用是对操作数进行按位操作。它与指令系统中的逻辑运算指令不同，运算后产生一个逻辑运算值，供给指令操作数使用，它不影响标志位。其中 NOT（非）是单操作数运算符，其他 3 个逻辑运算符为双操作数运算符。

关系运算符都是双操作数运算符，它的运算对象只能是两个性质相同的项目。关系运算的结果只能是两种情况：关系成立或不成立。当关系成立时，运算结果为 1，否则为 0。

分析运算符是对存储器地址进行运算的。它可以将存储器地址的 3 个重要属性，即段、偏移量和类型分离出来，返回到所在的位置做操作数使用。因此分析运算符又称为数值返回运算符。

分析运算符的相关功能如下。

- SEG 运算符可以得到一个标号或变量的段基址。
- OFFSET 运算符可以得到一个标号或变量的偏移量。
- TYPE 运算符可加在变量、结构或标号的前面，所求出的是这些存储器操作数的类型部分。
- LENGTH 运算符放在数组变量的前面，可以求出该数组中所包含的变量或结构的个数。
- SIZE 运算符在变量已经用重复操作符 DUP 加以说明后，可以得到分配给该变量的字节总数。如果未用 DUP 加以说明，则得到的结果是 TYPE 运算的结果。

综合运算符可以用来建立和临时改变变量或标号的类型及存储器操作数的存储单元类型，而忽略当前的属性，所以又称为属性修改运算符。

综合运算符的相关功能如下。

- PTR 运算符用来指定或修改存储器操作数的类型，但它本身并不实际分配存储器。
- 段属性前缀用于需要进行段超越寻址的场合。

- SHORT 运算符用来修饰 JMP 指令中跳转地址的属性，指出跳转地址是在下一条指令地址的-128~+127 个字节范围之内。
- THIS 运算符和 PTR 运算符一样，可以用来建立一个特殊类型的存储器地址操作数，而不实际为它分配新的存储单元。用 THIS 建立的存储器地址操作数的段和偏移量部分与目前所能分配的下一个存储单元的段和偏移量相同，但类型由 THIS 指定。
- HIGH 和 LOW 被称为字节分离运算符，它们将一个 16 位的数或表达式的高字节和低字节分离出来。

【例 4.1】PTR 运算符应用举例。

```
VAR1  DB  30H,40H
VAR2  DW  2050H
  …
MOV  AX,WORD  PTR  VAR1
MOV  BL,BYTE  PTR  VAR2
```

在此例中，VAR1 为字节变量，对应 VAR1 存储单元保存的数据为 30H，对应 VAR1+1 存储单元保存的数据为 40H；VAR2 为字变量，对应 VAR2 存储单元保存的数据为 2050H。

在传送指令中，从字节变量 VAR1 存储单元和 VAR1+1 存储单元中取出一个字数据，赋给字寄存器 AX；从字变量 VAR2 存储单元中取出一个字节数据，赋给字节寄存器 BL。

则有：(AX)=4030H,(BL)=50H

【例 4.2】对同一个数据区，要求既可以字节为单位，又可以字为单位进行存取。

```
AREA1 EQU THIS WORD
AREA2 DB 100 DUP(?)
```

此例中，AREA1 和 AREA2 实际上代表同一个数据区，共有 100 个字节，但 AREA1 的类型为 WORD，而 AREA2 的类型为 BYTE。

将算术运算符、逻辑运算符、关系运算符和分析运算符及综合运算符和常数、寄存器名、标号、变量一起组成表达式，放在语句的操作数字段中。

汇编过程中，汇编程序先计算表达式的值，然后再翻译指令。在计算表达式的值时，如果一个表达式同时具有多个运算符，则按以下规则运算。

- 优先级高的先运算，优先级低的后运算。
- 优先级相同时，按表达式中从左到右的顺序运算。
- 括号可以提高运算的优先级，括号内的运算总是在相邻的运算之前进行。
- 各类运算符从高到低的优先级排列顺序如表 4-2 所示。

表 4-2 各类运算符的优先级别

优先级别	运 算 符
1	LENGTH，SIZE，WIDTH，MASK，()，[]，<>
2	.（结构变量名后面的运算符）
3	:（段超越运算符）
4	PTR，OFFSET，SEG，TYPE，THIS
5	HIGH，LOW
6	+，-（一元运算符）

续上表

优先级别	运 算 符
7	*，/，MOD，SHL，SHR
8	+，-（二元运算符）
9	EQ，NE，LT，LE，GT，GE
10	NOT
11	AND
12	OR，XOR
13	SHORT

4.1.2 汇编语言程序结构

为便于分析汇编语言源程序的结构，先看下面给出的一个完整的汇编语言源程序的例子。

【例4.3】要求从内存中存放的10个无符号字节整数数组中找出最小数，将其值保存在AL寄存器中。汇编语言源程序如下：

```
DATA    SEGMENT                                          ;定义数据段
        BUF  DB  23H,16H,08H,20H,64H,8AH,91H,35H,2BH,FFH    ;定义数据区
        CN   EQU $-BUF
DATA    ENDS
STACK   SEGMENT                                          ;定义堆栈段
        STA  DB  10 DUP(?)
        TOP  EQU $-STA
STACK   ENDS
CODE    SEGMENT                                          ;定义代码段
        ASSUME   CS:CODE,DS:DATA,SS:STACK
START:  PUSH DS                                          ;汇编开始
        XOR  AX,AX
        PUSH AX
        MOV  AX,DATA
        MOV  DS,AX                                       ;初始化DS
        MOV  BX,OFFSET BUF
        MOV  CX,CN
        DEC  CX
        MOV  AL,[BX]
        INC  BX
LP:     CMP  AL,[BX]                                     ;两数比较
        JBE  NEXT                                        ;若(AL)<[BX],转NEXT
        MOV  AL,[BX]                                     ;将较小数存入AL中
NEXT:   INC  BX
        DEC  CX
        JNZ  LP
        MOV  AH,4CH                                      ;返回DOS
        INT  21H
CODE    ENDS
        END  START                                       ;汇编结束
```

从上面例子可以看出，汇编语言源程序是分段结构形式。一个汇编语言源程序是由若干个逻辑段组成的，每个逻辑段以 SEGMENT 语句开始，以 ENDS 语句结束。整个源程序以 END 语句结束。每个逻辑段内有若干条语句，一个汇编源程序是由完成某种特定操作功能的语句组成的。

通常，一个汇编源程序由 3 个逻辑段组成，即数据段、堆栈段和代码段。

（1）数据段用来在内存中建立一个适当容量的工作区，以存放常数、变量等程序需要对其进行操作的数据。

（2）堆栈段用来在内存中建立一个适当的堆栈区，以便在中断、子程序调用时使用。堆栈段一般可以有几十个字节至几千字节。如果太小，则可能导致程序执行中的堆栈溢出错误。

（3）代码段包括了许多以符号表示的指令，其内容就是程序要执行的指令。

作为一个汇编源程序的主模块，下面几部分是不可缺少的。

- 必须用 ASSUME 伪指令告诉汇编程序，某一段地址应该放入哪一个段寄存器。这样对源程序模块进行汇编时，才能确定段中各项的偏移量。
- DOS 的装入程序在执行时，将把 CS 初始化为正确的代码段地址，把 SS 初始化为正确的堆栈段地址，在源程序中不需要再对它们进行初始化。因为装入程序已经将 DS 寄存器留作他用，这是为了保证程序段在执行过程中数据段地址的正确性，所以在源程序中应该有以下两条指令，对它进行初始化。

```
MOV  AX,DATA
MOV  DS,AX
```

- 在 DOS 环境下，通常采用 DOS 的 4CH 号中断功能调用使汇编语言返回 DOS，即采用如下两条指令：

```
MOV  AH,4CH
INT  21H
```

如果不是主模块，这两条指令可以不用。

由于 8086 的 1MB 存储空间是分段管理的，汇编语言源程序存放在存储器中，无论是取指令码还是存取操作数，都要访问内存。因此，汇编语言源程序的编写必须遵照存储器分段管理的规定，分段进行编写。

4.1.3 汇编语言常用伪指令

通常所说的机器指令是发送给 CPU 的命令，在运行时由 CPU 执行，每条指令对应 CPU 的一种特定的操作，例如传送、加法、减法等；而伪指令是发送给汇编程序的命令，在汇编过程中由汇编程序进行处理。例如定义数据、分配存储区、定义段及定义过程等。汇编以后，每条 CPU 指令产生一一对应的目标代码；而伪指令则不产生与之相应的目标代码。

宏汇编程序 MASM 提供了约几十种伪指令，根据伪指令的功能，大致可以分为以下几类：数据定义、符号定义、段定义、过程定义、宏处理、模块定义与连接、处理器方式、条件、列表、其他伪指令等。

下面介绍一些常用的基本伪指令。

1. 数据定义伪指令

数据定义伪指令用来定义一个变量的类型，并将所需要的数据放入指定的存储单元中，可以只给变量分配存储单元，而不赋予特定的值。

数据定义伪指令的一般格式为：

[变量名]　伪指令　操作数 [,操作数,…][;注释]

方括号中的变量名为任意选项，它代表所定义的第一个单元的地址。变量名后面不要跟冒号“:”。伪指令后面的操作数可以不止一个，如果有多个操作数时，相互之间应该用逗号“,”分开。注释项也是任选的。

有以下 5 种数据定义类型：

[变量名]　DB　表达式　　表示定义字节；
[变量名]　DW　表达式　　表示定义字；
[变量名]　DD　表达式　　表示定义双字；
[变量名]　DQ　表达式　　表示定义 8 个字节；
[变量名]　DT　表达式　　表示定义 10 个字节；

这里的表达式是赋给变量的初始值，可以有一个，也可以有多个，给变量赋初值时，如果使用字符串，则字符串必须放在单引号中。

数据定义伪指令后面的操作数可以是常数、表达式或字符串，但每项操作数的值不能超过由伪指令所定义的数据类型限定的范围。例如，DB 伪指令定义数据的类型为字节，则其范围应该是，无符号数：0～255；带符号数：-128～+127。

给变量赋初值时，如果使用字符串，则字符串必须放在单引号中。另外，超过两个字符的字符串只能用 DB 伪指令定义。

【例 4.4】在给定的数据段中，分析数据定义伪指令的使用和存储单元的初始化。

```
DATA  SEGMENT                    ;定义数据段
   B1 DB 10H,30H                 ;存入两个字节 10H,30H
   B2 DB 2×3+5                   ;存入表达式的值 0BH
   S1 DB 'GOOD! '                ;存入 5 个字符
   W1 DW 1000H,2030H             ;存入两个字 1000H,2030H
   W2 DD 12345678H               ;存入双字 1234H,5678H
   S2 DB 'AB'                    ;存入两个字符的 ASCII 码 41H,42H
   S3 DW 'AB'                    ;存入 42H,41H
DATA ENDS                        ;数据段结束
```

在数据定义的第 1、2 条指令中分别将常数和表达式的值赋予一个字节变量；第 3 条指令的操作数是包含 5 个字符的字符串；第 4、5 条指令分别给字变量和双字变量赋初值；在第 6、7 条指令中要注意 DB 和 DW 的区别，虽然操作数均为“AB”两个字符，但存入变量的内容各不相同。

除了常数、表达式和字符串外，问号“？”也可以作为数据定义伪指令的操作数，此时仅给变量保留相应的存储单元，而不赋予变量某个确定的初值。

当同样的操作数重复多次时，可以采用重复操作符“DUP”来表示。

其使用格式为：n DUP(初值[,初值,…])

圆括号中为重复的内容，n 为重复次数。如果用“n DUP(?)”作为数据定义伪指令的唯

一操作数，则汇编程序产生一个相应的数据区，但不赋予任何初始值。此外，重复操作符“DUP”可以嵌套。

【例 4.5】在如下所示的数据段中，分析重复操作符“DUP”的使用和存储单元的初始化。

```
DATA SEGMENT                          ;定义数据段
     BUF1 DB ?                        ;分配字节变量存储单元,不赋初值
     BUF2 DB 8  DUP(0)                ;分配字节变量存储单元,赋初值为 0
     BUF3 DW 5  DUP(?)                ;分配字变量存储单元,不赋初值
     BUF4 DW 10 DUP(0,1,?)            ;分配字变量存储单元,对其初始化
     BUF5 DB 50 DUP(2,2 DUP(4),6)     ;分配字节变量存储单元,对其初始化
DATA ENDS                             ;数据段结束
```

数据段中的第 2 条指令给字节变量 BUF2 分配 8 个存储单元，并赋初值为 0；第 3 条指令给字变量 BUF3 分配 5 个字单元，即 10 个存储单元，不预先赋初值；第 4 条指令给字变量 BUF4 分配初始数据为 0、1、？且重复次数为 10 的存储空间，共占 30 个字节；第 5 条指令给字节变量 BUF5 定义为一个数据区，其中包含重复 50 次的内容：2、4、4、6，共占 200 个字节。

2. 符号定义伪指令

符号定义伪指令的用途是给一个符号重新命名，或定义新的类型属性等。这些符号可以包括汇编语言的变量名、标号名、过程名、寄存器名及指令助记符等。

常用的符号定义伪指令有 EQU、=、LABLE。

- EQU 伪指令：其作用是将表达式的值赋予一个名字，可以用这个名字来代替给定表达式。表达式可以是一个常数、变量、寄存器名、指令助记符、数值表达式或地址表达式等。

【例 4.6】分析 EQU 伪指令的作用。

```
COUNT   EQU  100                  ;COUNT 代替常数 100
VAL     EQU  ASCII-TABLE          ;VAL 代替变量 ASCII-TABLE
SUM     EQU  30+25                ;SUM 代替数值表达式
ADR     EQU  ES:[BP+DI+10]        ;ADR 代替地址表达式 ES:[BP+DI+10]
C       EQU  CX                   ;C 代替寄存器 CX
M       EQU  MOV                  ;M 代替指令助记符 MOV
```

需要注意的是，一个符号一经 EQU 伪指令赋值后，在整个程序中，不允许再对同一符号重新赋值。

- =（等号）伪指令：其功能与 EQU 伪指令基本相同，主要区别在于它可以对同一个名字重复定义。

例如：

```
COUNT  EQU  10                    ;正确,COUNT 代替常数 10
COUNT  EQU  10+20                 ;错误,COUNT 不能再次定义
COUNT =10                         ;正确,COUNT 代替常数 10
COUNT =10+20                      ;正确,COUNT 可以重复定义
```

- LABLE 伪指令：其用途是在原来标号或变量的基础上定义一个类型不同的新的标号或变量。

变量的类型可以是 BYTE、WORD、DWORD，标号的类型可以是 NEAR、FAR。

利用 LABLE 伪指令可以使同一个数据区兼有 BYTE 和 WORD 两种属性，这样，在以后

的程序中可根据不同的需要分别以字节为单位，或以字为单位存取其中的数据。

【例4.7】用LABLE伪指令定义变量和标号。

```
VAL1 LABLE BYTE                ;VAL1是字节型变量
VAL2 DW 20 DUP(?)              ;VAL2是字型变量
```

VAL1和VAL2变量的存储地址相同，但类型不同。

```
NEXT1: LABLE  FAR              ;NEXT1为FAR型标号
NEXT2: MOV AX,1200H            ;NEXT2为NEAR型标号
       …
       JMP  NEXT2              ;段内转移
       JMP  NEXT1              ;段间转移
```

3. 段定义伪指令

段定义伪指令的用途是在汇编语言程序中定义逻辑段，用它来指定段的名称和范围，并指明段的定位类型、组合类型及类别。

常用的段定义伪指令有：

- SEGMENT/ENDS伪指令

使用格式：　段名　[定位类型]　[组合类型]　['类别']

　　…（段内语句系列）

　　段名　ENDS

SEGMENT伪指令用于定义一个逻辑段，给逻辑段赋予一个段名，并以后面的任选项规定该逻辑段的其他特性。SEGMENT伪指令位于一个逻辑段的开始，ENDS伪指令则表示一个逻辑段的结束。这两个伪操作总是成对出现，缺一不可，两者前面的段名必须一致。

SEGMENT伪指令后面还有3个任选项，三者的顺序必须符合格式中的规定。这些任选项是给汇编程序和连接程序的命令，它告诉汇编程序和连接程序，如何确定解决边界，以及如何组合几个不同的段等。

（1）定位类型：定位类型选项告诉汇编程序如何确定逻辑段的边界在存储器中的位置，用来规定对段起始边界的要求。有以下4种定位类型。

BYTE：表示逻辑段从字节的边界开始，即可以从任何地址开始。此时本段的起始地址紧接在前一个段的后面。

WORD：表示逻辑段从字的边界开始。两个字节为一个字，此时本段的起始地址最低一位必须是0，即从偶地址开始。

PARA：表示逻辑段从一个节的边界开始。通常16个字节称为一个节，故本段的起始地址最低4位必须为0，应为××××0H。

PAGE：表示逻辑段从页边界开始。通常256个字节称为一页。故本段的起始地址最低8位必须为0，应为×××00H。

如果省略定位类型任选项，则默认值为PARA。

（2）组合类型：SEGMENT伪指令的第2个任选项是组合类型，它告诉汇编程序，当装入存储器时各个逻辑段如何进行组合。共有以下6种组合类型。

NONE：表示本段与其他逻辑段不发生关系，每段都有自己的基地址。这是任选项默认的组合类型。

PUBLIC：连接时，对于不同程序模块中的逻辑段，只要具有相同的类别名，就把这些

段顺序连接成为一个逻辑段装入内存。

STACK：组合类型为 STACK 时，其含义与 PUBLIC 基本一样，即不同程序中的逻辑段，如果类别名相同，则顺序连接成为一个逻辑段。不过组合类型 STACK 仅限于作为堆栈区域的逻辑段使用。

COMMON：连接时，对于不同程序中的逻辑段，如果具有相同的类别名，则都从同一个地址开始装入，因而各个逻辑段将发生重叠。最后，连接以后的段的长度等于原来的逻辑段的长度，重叠部分的内容是最后一个逻辑段的内容。

MEMORY：几个逻辑段连接时，连接程序将把本段定位在被连接在一起的其他所有段之上，如果被连接的逻辑段中有多个段的组合类型都是 MEMORY，则汇编程序只将首先遇到的段作为 MEMORY 段，而其余的段均当作 COMMON 段来处理。

AT 表达式：这种组合类型表示本逻辑段根据表达式求值的结果定位段基址。例如 AT 5800H，表示本段的段基址为 5800H，偏移地址为 0000H，则本段从存储器的物理地址 58000H 开始装入。

（3）类别：SEGMENT 伪指令的第 3 个任意选项是类别，类别必须放在单引号内。类别的作用是在连接时决定各逻辑段的装入顺序。当几个程序模块进行连接时，其中具有相同类别名的逻辑段被装入连续的内存区，类别名相同的逻辑段，按出现的先后顺序排列。没有类别名的逻辑段，与其他无类别名的逻辑段一起连续装入内存。

- ASSUME 伪指令

使用格式：ASSUME　段寄存器名:段名[,段寄存器名:段名[,...]]

ASSUME 伪指令告诉汇编程序，将某一个段寄存器设置为某一个逻辑段的段址，即明确指出源程序中的逻辑段与物理段之间的关系，当汇编程序汇编一个逻辑段时，可利用相应的段寄存器寻址该逻辑段中的指令或数据。

4. 过程定义伪指令

在程序设计中，经常将一些重复出现的语句组定义为子程序，又称为过程，可以采用 CALL 指令来调用。

过程定义伪指令 PROC/ENDP

使用格式：过程名　　PROC　　[NEAR]/FAR

...(语句系列)

RET

...(语句系列)

过程名　　ENDP

其中，PROC 伪指令定义一个过程，赋予过程一个名字，并指出该过程的类型属性为 NEAR 或 FAR。如果没有特别指明类型，则认为过程的类型是 NEAR。伪指令 ENDP 标志过程的结束。上述两个伪指令前面的过程名必须一致。

当一个程序段被定义为过程后，程序中其他地方就可以用 CALL 指令来调用这个过程。调用一个过程的格式为：CALL 过程名。

过程名实质上是过程入口的符号地址，它和标号一样，也有 3 种属性：段属性、偏移量属性、类型属性。过程的类型属性可以是 NEAR 或者 FAR。

一般来说，被定义为过程的程序段中应该有返回指令 RET，但不一定是最后一条指令，也可以有不止一条 RET 指令。执行 RET 指令后，控制返回到原来调用指令的下一条指令。过程的定义和调用均可以嵌套。

5. 结构定义伪指令

结构就是相互关联的一组数据的某种组合形式。使用结构，需要进行以下几方面的工作。

（1）结构的定义

用伪指令 STRUC 和 ENDS 把相关数据定义语句组合起来，便构成一个完整的结构。

使用格式：结构名　STRUC

（数据定义语句序列）

结构名　ENDS

【例 4.8】用结构制作一张学生成绩表，学生的信息包括姓名、学号、各门课成绩。

```
STUDENT  STRUC
   NAME1     DB 'WANG'
   NUMBER    DB  ?
   ENGLISH   DB  ?
   MATHS     DB  ?
   COMPUTER  DB  ?
STUDENT  ENDS
```

此例中，STUDENT 称为结构名，数据定义语句序列中的变量名叫做结构字段名。

（2）结构的预置

结构的定义完成以后，就如同在某些高级语言中完成了某些数据类型的定义，在汇编语言中，结构这个数据类型是通过结构变量来使用的。

对结构进行预置的格式如下：结构变量名　结构名 (字段值表)

其中：

- 结构名是结构定义使用的名字。
- 结构变量名是程序中具体使用的变量，它与具体的存储空间及数据相联系，程序中可直接引用它。
- 字段值表用来给结构变量赋初值，表中各字段的排列顺序及类型应该与结构定义时一致，各字段之间以逗号分开。

通过结构预置语句，可以对结构中某些字段进行初始化。但通过预置进行结构变量的初始化有一定的限制和规定：在结构定义中具有一项数据的字段才能通过预置来代替初始定义的值，而用 DUP 定义的字段或一个字段后有多个数据项的字段，则不能在预置时修改其定义时的值。

【例 4.9】结构定义中的结构变量初值的预置。

```
DATA  STRUC
   A1  DB  30H                ;简单元素,可以修改
   A2  DB  10H, 20H           ;多重元素,不能修改
   A3  DW  ?                  ;简单元素,可以修改
   A4  DB  'ABCD'             ;可用同长度的字符串修改
   A5  DW  10 DUP(?)          ;多重元素,不能修改
DATA ENDS
```

若有些字段的内容采用定义时的初值，则在预置语句中这些字段的位置仅写一个逗号即可。若所有的字段都如此，则仅写一对尖括号即可。

【例 4.10】对前面定义的 STUDENT 结构，采用结构变量来代表学生的信息。设有 3 个学生，则可有：

```
S1 STUDENT <'ZHANG',11,87,90,89>
S2 STUDENT <'WANG',12,68,83,71>
32 STUDENT <'LI',13,92,86,95>
```

这样，就在存储器中为 3 个学生建立了成绩档案，把他们的姓名、学号及 3 门课成绩都放在了指定的位置。

（3）结构的引用

程序中引用结构变量，和其他变量一样，可直接写结构变量名。若要引用结构变量中的某一字段，则采用如下形式：

结构变量名·结构字段名

或者，先将结构变量的起始地址的偏移量送到某个地址寄存器，然后再用：

[地址寄存器]·结构字段名

例如,若要引用结构变量 S1 中的 ENGLISH 字段，以下两种用法都是正确的。

（1）MOV AL, S1·ENGLISH

（2）MOV BX, OFFSET S1

MOV AL, [BX]·ENGLISH

6. 模块定义与连接伪指令

在编写规模较大的汇编语言源程序时，可以将整个程序划分为几个独立的源程序，称之为模块，然后将各模块分别进行汇编，生成各自的目标程序，最后将它们连接成为一个完整的可执行程序。为了进行模块之间连接和实现相互的符号访问，以便进行变量传送，通常使用以下几个伪指令。

- NAME 伪指令：用于给源程序汇编以后得到的目标程序指定一个模块名，连接时需要使用这个目标程序的模块名。
- END 伪指令：表示源程序到此结束，指示汇编程序停止汇编，对于 END 后面的语句可以不予理会。
- PUBLIC 伪指令：说明本模块中的某些符号是公共的，即这些符号可以提供给将被连接在一起的其他模块使用。
- EXTRN 伪指令：说明本模块中所用的某些符号是外部的，即这些符号在将被连接在一起的其他模块中定义，在定义这些符号的模块中还必须用 PUBLIC 伪指令加以说明。

7. 程序计数器$和 ORG 伪指令

（1）程序计数器$：字符“$”在 8086 宏汇编中具有一种特殊的意义，称为程序计数器。汇编程序在对段定义的处理过程中，每遇到一个新的段名，就在段表中填入该段名，同时为该段设置一个初值为 0 的位置计数器。然后，对该段进行汇编，对申请分配存储器的语句及产生目标代码的语句，都将其占用的存储器的字节数累加在该段的位置计数器中。随着汇编

的进行，位置计数器的值不断变化，字符“$”便是表示位置计数器的当前值，它可以在数值表达式中使用。在程序中，“$”出现在表达式里，它的值为程序下一个所能分配的存储单元的偏移地址。

（2）ORG 伪指令：ORG 是起始位置设定伪指令，用来指出源程序或数据块的起点。段内存储器的分配是从 0 开始依次顺序分配的。因此，位置计数器的值是从 0 开始递增累计的。但在程序设计中，若需要将存储单元分配在指定位置，而不是从位置计数器的当前值开始，便可以使用 ORG 语句，利用 ORG 伪指令可以改变位置计数器的值。

【例 4.11】已知数据段中 VAR1 的偏移量为 2，占 3 个字节，初始数据为 20H、30H、40H；VAR2 的偏移量为 8，占 2 个字节，初始数据为 5678H。VAR1 和 VAR2 之间有 3 字节的距离，采用 ORG 完成数据段存储器的分配。

```
DATA SEGMENT
     ORG 2                      ;预置 VAR1 的偏移量为 2
     VAR1 DB 20H,30H,40H        ;VAR1 的初始数据
     ORG $ + 3                  ;预置 VAR2 的偏移量为 8
     VAR2 DW 5678H              ;VAR2 的初始数据
DATA ENDS
```

4.1.4　汇编语言程序上机过程

1. 汇编语言的工作环境

硬件环境：8086 汇编语言程序一般多在 IBM PC/XT 及其兼容机上运行，因此要求机器具有一些基本配置就可以了，汇编语言对机器硬件环境没有特殊要求。

软件环境：指支持汇编语言程序运行和帮助建立汇编语言源程序的一些软件，主要包括以下几个方面。

（1）DOS 操作系统：汇编语言程序的建立和运行都是在 DOS 操作系统的支持下进行的。

（2）编辑程序：编辑程序是用来输入和建立汇编语言源程序的一种通用的系统软件，通常源程序的修改也是在编辑状态进行的。

（3）汇编程序：8086 的汇编程序有基本汇编 ASM.EXE 和宏汇编 MASM.EXE 两种。基本汇编不支持宏操作，因此，一般选用宏汇编 MASM.EXE。

（4）连接程序：8086 汇编语言使用的连接程序是 LINK.EXE。

（5）调试程序：这类程序作为一种辅助工具，帮助编程者进行程序的调试，通常用动态调试程序 DEBUG.COM，其相关内容可参见附录 B。

2. 汇编语言上机步骤

一般情况下，在计算机上运行汇编语言程序的步骤如下。

（1）用编辑程序（例如 EDIT.COM）建立扩展名为.ASM 的汇编语言源程序文件。

（2）用汇编程序（例如 MASM.EXE）将汇编语言源程序文件汇编成用机器码表示的目标程序文件，其扩展名为.OBJ。

（3）如果在汇编过程中出现语法错误，根据错误的信息提示（如错误位置、错误类型、错误说明），用编辑软件重新调入源程序进行修改。没有错误时采用连接程序（例如 LINK.EXE）把目标文件转化成可执行文件，其扩展名为.EXE。

（4）生成可执行文件后，在 DOS 命令状态下直接键入文件名就可以执行该文件。

4.2 汇编语言程序设计

4.2.1 程序设计的基本步骤及程序基本结构

1. 程序设计的基本步骤

用汇编语言设计程序，一般按下述步骤进行。

（1）分析问题，抽象出数学模型：其目的就是求得问题一个确切的理解，明确问题的环境限制，弄清已知条件、原始数据、输入信息、对运算精度的要求、对处理速度的要求，以及最后应获得的结果。

（2）确定算法或解题思想：所谓算法就是确定解决问题的方法和步骤。一类问题可以同时存在几种算法，评价算法好坏的指标是程序执行的时间和占用存储器的空间，设计该算法和编写程序所投入的人力，理解该算法的难易程度，以及可扩充性和可适应性等。

（3）绘制流程图：流程图是一种用特定的图形符号加上简单的文字说明来表示数据处理过程的步骤。它指出了计算机执行操作的逻辑次序，表达简洁、清晰，设计者可以从流程图上直接了解系统执行任务的全部过程及各部分之间的关系，便于排除设计错误。

（4）存储空间和工作单元初始化：8086 存储器结构要求存储空间分段使用，因此要分别定义数据段、堆栈段、代码段及附加段。工作单元可以设置在数据段和附加段中的某些存储单元中，也可以设置在 CPU 内部的数据寄存器中。

（5）程序编制：应严格按规定的语法规则书写，程序结构尽可能简单、层次清楚，合理分配寄存器的用途，选择常用、简单、直接、占用内存少、运行速度快的指令序列；采用结构化程序设计方法；尽量提高源程序的可读性和可维护性，必要时提供注释。

（6）静态检查：查看程序是否具备所要求的功能，选用的指令是否合适，程序的语法和格式上是否有错误，指令中引用的语句标号名称和变量名是否定义正确，程序执行流程是否符合算法等。静态检查可以及时发现问题，及时进行修改。

（7）动态调试：汇编程序可以检查源程序中的语法错误，可按指出的语法错误修改程序，直至无误，再利用 DEBUG 调试工具检查程序运行以后是否能达到预期的结果。

总之，程序结构清晰、可读性强、占用存储空间少、运行速度快是每一个编程人员追求的目标。为了做到这一点，不仅要理解指令的基本功能，而且要多读、多写、多上机调试各种程序，通过实践不断总结经验，加深对汇编语言的语句和程序设计方法的理解，逐步掌握各种程序设计方法和技巧。

2. 程序的基本结构

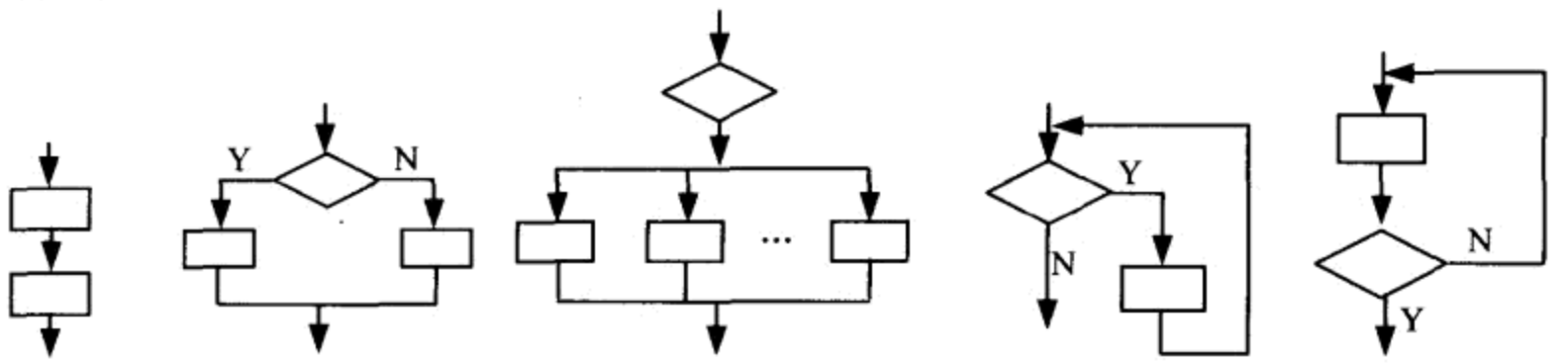
程序一般可以由顺序结构、分支结构和循环结构组合而成。每一个结构只有一个入口和一个出口，3 种结构的任意组合和嵌套就构成了结构化的程序。

（1）顺序结构：是按照语句的先后次序执行一系列的顺序操作，如图 4-1（a）所示。

（2）分支结构：也叫条件选择结构，根据不同情况做出判断和选择，以便执行不同的程序段。分为双分支结构和多分支结构，如图 4-1（b）和图 4-1（c）所示。

（3）循环结构：循环实际上是分支结构的一种扩展，循环是否继续是依靠条件判断语句来决定的。按照条件判断的位置，可以把循环分为“当型循环”和“直到型循环”，如图

4-1（d）和图 4-1（e）所示。



（a）顺序结构　（b）双分支结构　（c）多分支结构　（d）当型循环　（e）直到型循环

图 4-1　程序的基本结构

4.2.2　顺序结构程序设计

顺序结构程序从执行开始到最后一条指令为止，指令指针 IP 中的内容呈线性增加；从流程图上看，顺序结构的程序只有一个起始框，一个或几个执行框和一个终止框。程序无分支、无循环，按直线形式顺序执行。设计这种程序的方法十分简单，只要遵照算法步骤依次写出相应的指令即可。在进行顺序结构程序设计时，主要考虑的是如何选择简单有效的算法，如何选择存储单元和工作单元。

下面通过一些实例来分析顺序结构程序的设计过程。

【例 4.12】要求从键盘输入一个小写英文字母，将其转换成大写英文字母后在显示器上显示出来。编写该程序。

分析：本例由于涉及到对输入/输出设备的操作，需要利用 DOS 功能调用的 INT 21H 指令来实现。键盘输入一个字符可用 01H 调用，显示器上显示字符可用 02H 调用。另外，从 ASCII 码表中可知，将小写字母转换成大写字母只要将其 ASCII 码减去 32 即可。

为了简化程序设计，假设从键盘输入的字符一定是小写字母。

设计的源程序如下：

```
STACK   SEGMENT STACK                    ;定义堆栈段
        DB 100 DUP(?)                    ;开辟 100 个存储单元
STACK   ENDS                             ;堆栈段结束
CODE    SEGMENT                          ;定义代码段
        ASSUME CS:CODE ,SS:STACK
START:  MOV  AH ,01H                     ;1 号调用,从键盘输入一字符存入 AL
        INT  21H
        SUB  AL,20H                      ;将 AL 中字符的 ASCII 码减去 20H 变成大写字母
        MOV  DL,AL                       ;结果送 DL
        MOV  AH,02H                      ;2 号调用,在显示器上显示 DL 中的内容
        INT  21H
        MOV  AH,4CH
        INT  21H                         ;返回 DOS
CODE    ENDS                             ;代码段结束
        END  START                       ;程序汇编结束
```

【例 4.13】试计算 S=(W-(X×Y+Z-340))/X，设 W、X、Y、Z 均为 16 位带符号数。要求计算完成后，商和余数存入 S 指定的 RESULT 单元中。

分析：本例是完成表达式的计算，将各变量进行定义并赋初始值，然后按照计算顺序从内到外依次操作，最后保存结果。

完整的汇编语言源程序如下：

```
DATA    SEGMENT                          ;定义数据段
   W    DW -245                          ;变量初始化
   X    DW 15
   Y    DW -32
   Z    DW 280
        RESULT DW 2 DUP(?)               ;定义结果保存单元
DATA    ENDS                             ;数据段结束
CODE    SEGMENT                          ;定义代码段
        ASSUME CS:CODE,DS:DATA
START:  MOV AX,DATA                      ;初始化 DS
        MOV DS,AX
        MOV AX,X                         ;取出被乘数 X 存入 AX 中
        IMUL Y                           ;执行 X×Y 操作
        MOV CX,AX                        ;乘积的低位字转存至 CX
        MOV BX,DX                        ;乘积的高位字转存至 BX
        MOV AX,Z                         ;取出加数 Z 存入 AX 中
        CWD                              ;将 Z 扩展成双字
        ADD CX,AX                        ;X×Y 的低位字与 Z 的低位字相加
        ADC BX,DX                        ;X×Y 的高位字与 Z 的高位字及进位相加
        SUB BX,340                       ;X×Y+Z 的低位字减 340
        SBB BX,0                         ;X×Y+Z 的高位字减低位的借位
        MOV AX,W                         ;取出 W 存入 AX
        CWD                              ;将 W 扩展成双字
        SUB AX,CX                        ;(W-(X×Y+Z)-340)低位运算
        SBB DX,BX                        ;(W-(X×Y+Z)-340)高位运算
        IDIV X                           ;(W-(X×Y+Z)-340)/X
        MOV RESULT,AX                    ;商存入 RESULT 单元
        MOV RESULT+2,DX                  ;余数存入 RESULT+2 单元
        MOV AH,4CH
        INT 21H                          ;返回 DOS
CODE    ENDS                             ;代码段结束
        END  START                       ;汇编结束
```

【例 4.14】已知在内存中从 TABLE 单元起存放 0～100 的平方值。在 X 单元中有一个待查数据，用查表的方法求出 X 的平方值送到 Y 单元中。程序段如下：

```
DATA    SEGMENT
        TABLE DW 0,1,4,9,16,25…
        X DB 4
        RESU DW ?
DATA    ENDS
CODE    SEGMENT
        ASSUME DS:DATA,CS:CODE
START:  MOV AX,DATA
        MOV DS,AX                        ;初始化 DS
        MOV BX,OFFSET TABLE              ;BX 指向表格首单元
        MOV AL,X                         ;X 中内容取出送至 AL 中
        MOV AH,0                         ;X 中的值扩展成字
```

```
        SHL AX,1                    ;计算X×2
        ADD BX,AX                   ;BX指向要查找的位置
        MOV DL,[BX]                 ;取出要查找的内容的低位字节
        MOV DH,[BX+1]               ;取出要查找的内容的高位字节
        MOV RESU,DX                 ;结果保存到内存中
        MOV AH,4CH
        INT 21H                     ;返回DOS
CODE    ENDS
        END START                   ;汇编结束
```

【例4.15】已知在内存NUM单元中放有1个字节的二进制数，将其以十六进制形式在显示器上显示出来。

源程序设计如下：

```
STACK   SEGMENT STACK
  DB 200 DUP(?)
STACK   ENDS
DATA    SEGMENT
  NUM   DB  01100101B                   ;定义变量并赋初值
  BUF1  DB  0AH, 0DH ,'(NUM) = '        ;调用回车换行,显示字符串
  BUF2  DB  4  DUP (?)                  ;定义结果的保存单元
DATA    ENDS
CODE    SEGMENT
        ASSUME DS:DATA, CS:CODE, SS:STACK
BEGIN:  MOV AX, DATA                    ;初始化DS
        MOV DS, AX
        LEA DI, BUF2                    ;取BUF2的有效地址
        MOV AL, NUM                     ;取内存中的二进制数
        MOV CL, 4                       ;计数初值
        SHR AL, CL                      ;逻辑右移
        OR  AL, 30H                     ;逻辑或运算
        MOV [DI], AL                    ;保存结果
        INC DI                          ;地址加1
        MOV AL, NUM                     ;取内存中的二进制数
        AND AL, 0FH                     ;逻辑与运算
        OR AL, 30H                      ;逻辑或运算
        MOV [DI], AL                    ;保存结果
        INC DI                          ;地址加1
        MOV BYTE PTR [DI], 'H'          ;送字符H
        INC DI                          ;地址加1
        MOV BYTE PTR [DI] ,'$'          ;送字符$
        MOV DX ,OFFSET BUF1             ;取偏移地址
        MOV AH ,09H                     ;09号调用显示字符串
        INT 21H
        MOV AH ,4CH
        INT 21H                         ;返回DOS
CODE    ENDS
        END BEGIN                       ;程序结束
```

4.2.3 分支结构程序设计

分支结构的程序可以有两种形式：双分支结构和多分支结构。双分支结构是组成其他复

杂程序的基本结构。首先要明确需要判断的条件；要用哪一个条件转移语句；条件成立的分支要完成什么任务，条件不成立的分支要完成哪一些操作。多分支程序适用于有多种条件的情况。

【例 4.16】已知在内存中有一个字节单元 X，存有带符号数据，要求计算出它的绝对值后，放入 RESULT 单元中。

题目分析：根据数学中绝对值的概念可知，一个正数的绝对值是它本身，而一个负数的绝对值是它的相反数；要计算一个数的相反数，需要完成减法运算，即用 0 减去这个数。8086 指令系统中有专门的求相反数的指令 NEG，程序流程图如图 4-2 所示。

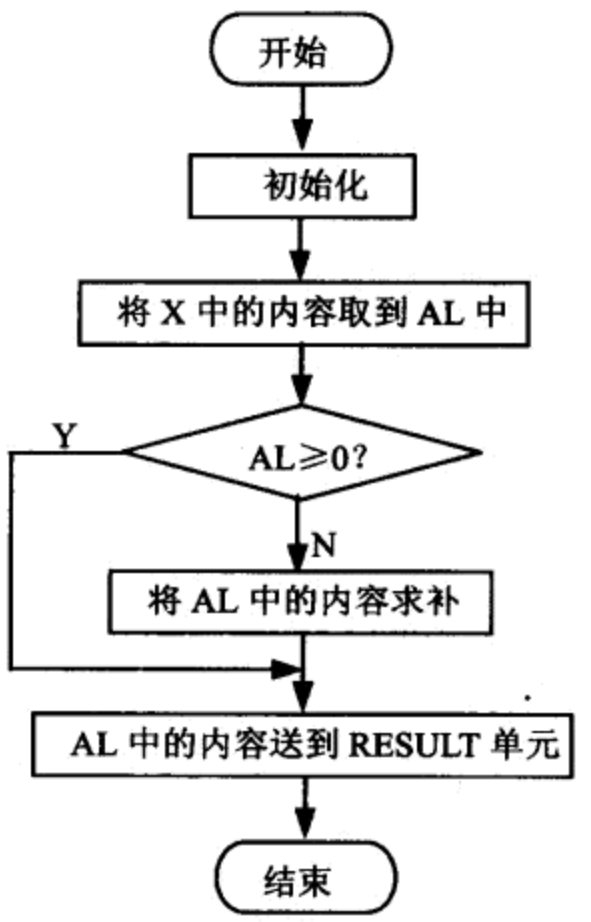


图 4-2　例 4.16 程序流程图

源程序设计如下：

```
DATA    SEGMENT
        X DB -25                ;定义变量
        RESULT  DB ?            ;定义结果保存单元
DATA    ENDS
CODE    SEGMENT
        ASSUME DS: DATA,CS: CODE
START:  MOV AX,DATA
        MOV DS,AX               ;初始化 DS
        MOV AL,X                ;X 取到 AL 中
        TEST AL,80H             ;测试 AL 正负
        JZ NEXT                 ;为正，转 NEXT
        NEG AL                  ;否则 AL 求补
NEXT:   MOV RESULT,AL           ;送结果
        MOV AH,4CH
        INT 21H                 ;返回 DOS
CODE    ENDS
        END START               ;汇编结束
```

【例 4.17】设变量 X 为带符号整数，试编写一个程序，完成下面的分段函数计算：

$$Y=\begin{cases}0 & |X|\geqslant 5\\ 1-X & |X|<5\end{cases}$$

题目分析：为了实现上述分段函数的计算，首先要利用上一个例题求出 X 的绝对值，然后根据 X 的绝对值判断 Y 的取值，流程图如图 4-3 所示。程序段如下：

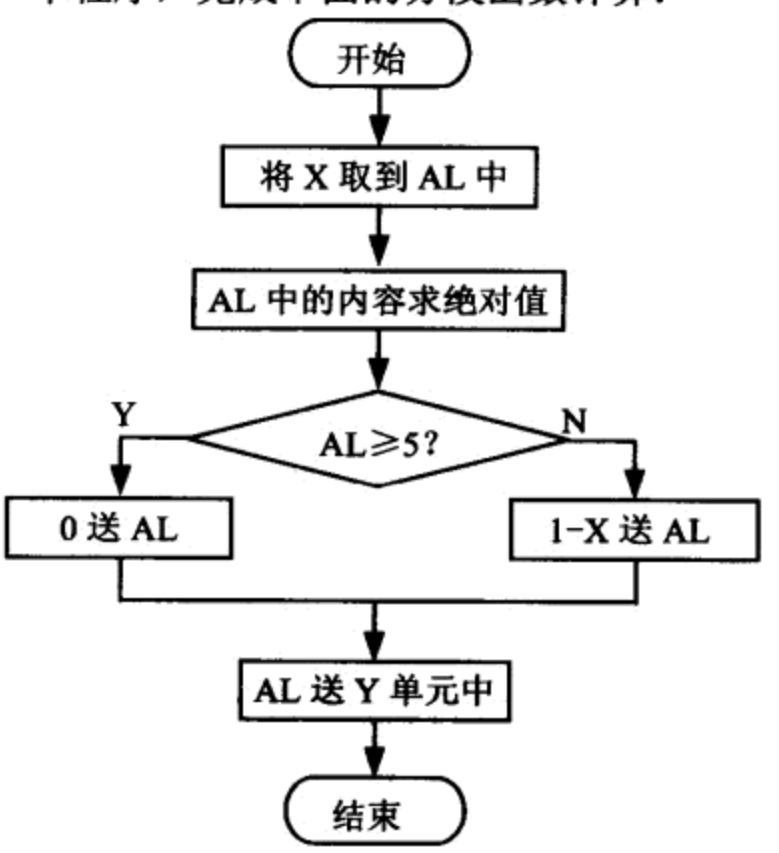


图 4-3　例 4.17 程序流程图

```
DATA    SEGMENT
        X DB -36
        Y DB ?
DATA    ENDS
CODE    SEGMENT
        ASSUME CS: CODE,DS: DATA
START:  MOV AX,DATA
        MOV DS,AX
        MOV AL,X
        CMP AL,0
```

```
        JGE NEXT                ;为正,转 NEXT
        NEG AL                  ;AL 中的内容求补
NEXT:   CMP AL,5                ;|X|和 5 比较
        JGE P1                  ;≥5,转 P1
        MOV AL,1                ;否则 1 送 AL
        MOV BL,X                ;X 送 BL 中
        SUB AL,BL               ;结果送 AL 中
        JMP EXIT                ;转 EXIT
P1:     MOV AL,0                ;0 送 AL 中
EXIT:   MOV Y,AL                ;结果送 Y 单元
        MOV AH,4CH
        INT 21H                 ;返回 DOS
CODE    ENDS
        END START               ;汇编结束
```

【例 4.18】编写程序，完成下面的分段函数的计算（X 为单字节带符号数据）。

$$Y=\begin{cases}1 & X>0\\ 0 & X=0\\ -1 & X<0\end{cases}$$

题目分析：这是 1 个 3 路分支的程序设计，X 为内存中的一个带符号数，首先判断其正负，若为负，-1 作为函数值；若为正，再判断是否为 0，如果为 0，函数返回值为 0，否则返回值为 1，流程图如图 4-4 所示。

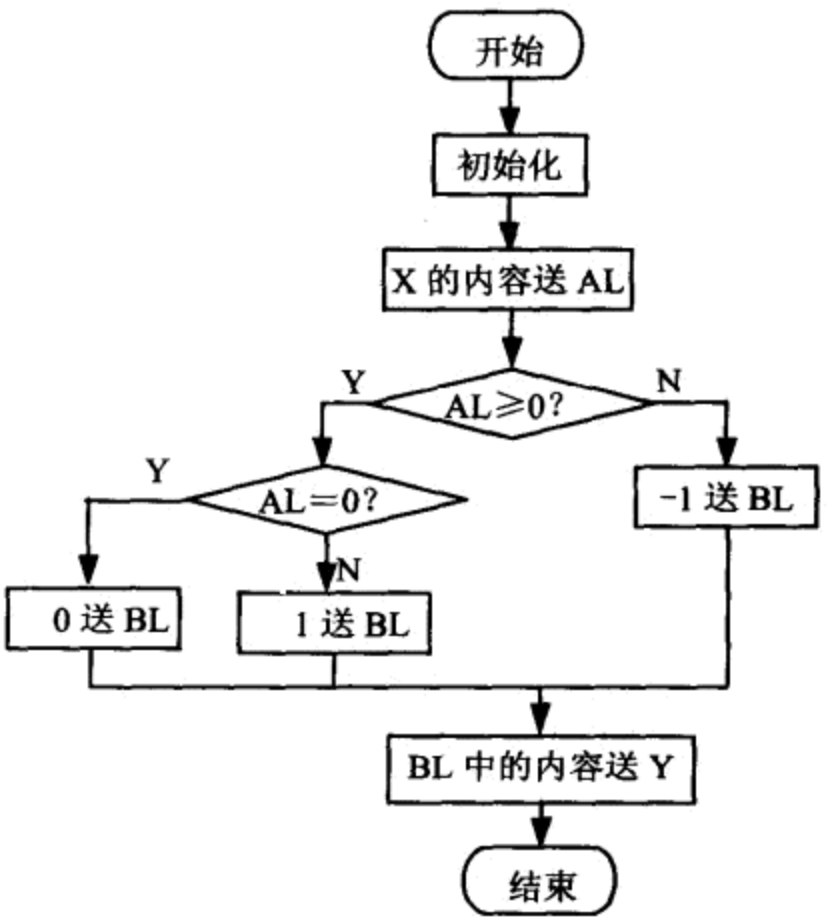


图 4-4　例 4.18 程序流程图

程序段如下：

```
DATA    SEGMENT
        X  DB  -25
        Y  DB  ?
DATA    ENDS
CODE    SEGMENT
        ASSUME   CS:CODE,DS:DATA
START:  MOV  AX,DATA
        MOV  DS,AX              ;初始化 DS
        MOV  AL,X               ;X 取到 AL 中
        CMP  AL,0               ;Al 和 0 比较
        JGE  BIG                ;≥0,转 BIG
        MOV  BL,-1              ;否则 - 1 送 BL
        JMP  EXIT               ;转到结束位置
BIG:    JE   EE                 ;Al=0 转 EE
        MOV  BL,1               ;否则 1 送 BL
        JMP  EXIT               ;转到结束位置
EE:     MOV  BL,0               ;0 送 BL
EXIT:   MOV  Y,BL               ;BL 中内容送 Y 单元
        MOV  AH,4CH
        INT  21H
CODE    ENDS
        END  START              ;汇编结束
```

【例 4.19】设计一个程序，统计在内存 W 单元保存的无符号字变量中有多少个二进制“1”，统计结果送内存 N 单元保存。

题目分析：要统计内存 W 单元保存的无符号字变量中有多少个二进制“1”，可以将该数据送寄存器 AX 中，通过移位指令将 AX 中的每一位依次移入进位标志 CF 中，若 CF=1 则计数器 CL 的值加 1，这样就可以统计出 AX 中 1 的个数。

设计源程序如下：

```
DATA    SEGMENT
   W    DW 00FFH                    ;定义字数据
   N    DB ?                        ;定义结果保存单元
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA
START:  MOV AX,DATA                 ;初始化 DS
        MOV DS,AX
        MOV CL,0                    ;计数器清零
        MOV AX,W                    ;取数到 AX
   AA:  AND AX,AX                   ;逻辑与，判 AX=0?
        JZ  CC                      ;结果为 0 转移到 CC
        SHL AX,1                    ;逻辑左移 1 位
        JNC BB                      ;无进位转 BB
        INC CL                      ;有"1"则计数
   BB:  JMP AA                      ;无条件转 AA
   CC:  MOV N,CL                    ;结果送 N 单元
        MOV AH,4CH
        INT 21H                     ;返回 DOS
CODE    ENDS
        END START                   ;汇编结束
```

4.2.4 循环结构程序设计

1. 循环结构程序的基本组成

循环结构程序是经常用到的，主要由以下 4 个部分组成。

（1）初始化部分：在进入循环程序之前，要进行循环程序的初始状态的设置，包括循环计数器初始化，地址指针初始化，存放运算结果的寄存器或内存单元的初始化。

（2）循环体：是完成循环工作的主要部分，使用循环程序的目的就是要重复执行这段操作。不同的程序要解决的问题不同，因此循环体的具体内容也有所不同。

（3）参数修改部分：为保证每次循环的正常执行，相关信息（如计数器的值、操作数的地址指针等）要发生有规律的变化，为下一次循环作准备。

（4）循环控制部分：是循环程序设计的关键。每个循环程序必须选择一个恰当的循环控制条件来控制循环的运行和结束。有时循环次数是已知的，可使用计数器来控制；有时循环次数是未知的，应根据具体情况设置控制循环结束的条件。

常见的循环结构有两种：一种是先执行循环体，然后判断循环是否继续进行；另一种是先判断是否符合循环条件，符合则执行循环体，否则退出循环。如果循环次数是已知的，则可以采用计数控制的方法；如果循环次数预先是不确定的，可以通过测试某些条件是否成立来实现对循环的控制。

2. 单循环结构程序设计

【例 4.20】编制程序完成求 1~50 之间的自然数中的奇数累加和，结果送到 RESULT 单元中。

该题属于循环次数已知，即将 50 以内的 25 个奇数进行累加和计算，可采用递增计数法来实现求累加和，程序设计如下：

```
DATA    SEGMENT
        RESULT  DW  ?           ;定义结果保存单元
        CN   EQU 25             ;定义计数次数变量
DATA    ENDS
CODE    SEGMENT
        ASSUME  DS:DATA,CS:CODE
START:  MOV  AX,DATA
        MOV  DS,AX              ;初始化 DS
        MOV  AX,0               ;循环初始化
        MOV  CX,CN
        MOV BX,1
NEXT:   ADD  AX,BX              ;求累加和
        ADD BX,2                ;(BX) ← (BX)+2
        LOOP  NEXT              ;(CX)-1 ≠ 0 转 NEXT
        MOV  RESULT,AX          ;计算完毕保存结果
        MOV  AH,4CH
        INT  21H                ;程序结束
CODE    ENDS
        END  START              ;汇编结束
```

【例 4.21】编制程序完成求 1+2+3+…+N 的累加和，直到累加和超过 1 000 为止。统计被累加的自然数的个数送入 CN 单元，累加和送入 SUM 单元。

该题的循环次数预先是不确定的，只能按照循环过程中的某个特定条件来决定循环是否继续执行。可通过测试条件是否成立来实现对循环的控制。

流程图如图 4-5 所示，程序设计如下：

```
DATA    SEGMENT
        SUM DW  ?
        CN  DW  ?
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA
START:  MOV AX,DATA
        MOV DS,AX               ;初始化 DS
        MOV AX,0                ;0 送 AX
        MOV BX,0                ;0 送 BX
LP:     INC BX                  ;BX 加 1
        ADD AX,BX               ;求累加和
        CMP AX,1000             ;比较
        JBE LP                  ;结果≤1000 转 LP
        MOV SUM,AX
        MOV CN,BX               ;送结果
        MOV AH,4CH
```

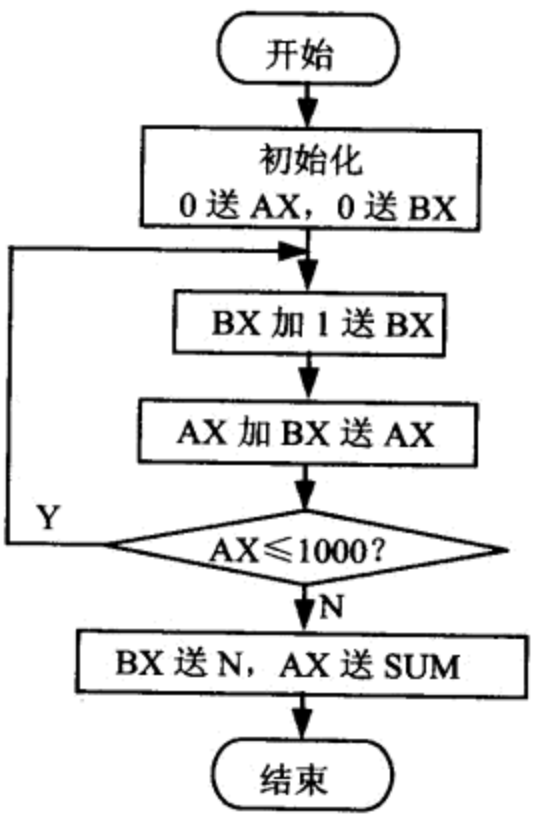


图 4-5　例 4.21 程序流程图

```
        INT 21H                 ;返回 DOS
CODE    ENDS
        END START               ;汇编结束
```

3. 多重循环结构程序设计

循环程序可以有多重循环，又称为循环嵌套。在使用多重循环时要特别注意以下几点。

（1）内循环应完整地包含在外循环内，内外循环不能相互交叉。

（2）内循环在外循环中的位置可根据需要任意设置，在分析程序流程时要避免出现混乱。

（3）内循环既可以嵌套在外循环中，也可以几个循环并列存在。可以从内循环直接跳到外循环，但不能从外循环直接跳到内循环。

（4）防止出现死循环，无论是内循环还是外循环，不要使循环回到初始化部分，否则将出现死循环。

（5）每次完成外循环再次进入内循环时，初始条件必须重新设置。

【例 4.22】数据段中有一组带符号数据，存放在从 A 单元开始的区域中，试编程序实现将它们按从小到大的顺序排序。要求排序后依然放在原来的存储区中。

这里采用冒泡法来设计该程序。从第一个数开始依次对两两相邻的两个数进行比较，如果次序符合要求（即第 i 个数小于第 i+1 个数），不做任何操作；否则两数交换位置。这样经过第一轮的两两比较（N-1 次），最大数放到了最后。第二轮对前 N-1 个数做上面的工作，则把次大数放到了倒数第二个单元……依此类推，做 N-1 轮同样的操作，就完成了从小到大排序。

通过上述分析可以知道，该算法要用双重循环实现。外循环次数为 N-1 次，内循环次数分别为 N-1 次、N-2 次、N-3 次、……、2 次、1 次。所以内循环的循环次数和外循环的计数器值有关，即等于外循环计数器的值。

程序设计如下：

```
DATA    SEGMENT
        A    DB  23,-15,34,67,-19,0,-12,89,120,55
        CN   EQU $-A
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA
START:  MOV   AX,DATA
        MOV   DS,AX             ;初始化 DS
        MOV   CX,CN-1           ;外循环次数送计数器 CX
LP1:    MOV   SI,0              ;数组起始下标 0 送 SI
        PUSH  CX                ;外循环计数器入栈
LP2:    MOV   AL,A[SI]          ;A[SI]取出送 AL
        CMP   AL,A[SI+1]        ;A[SI]和 A[SI+1]比较
        JLE   NEXT              ;小于或等于转 NEXT
        XCHG  AL,A[SI+1]        ;否则 A[SI]和 A[SI+1]交换
        MOV   A[SI],AL
NEXT:   INC   SI                ;数组下标加 1
        LOOP  LP2               ;CX-1 不为 0 转 LP2
        POP   CX                ;否则退出内循环，将 CX 出栈
        LOOP  LP1               ;CX-1 不为 0 转 LP1
```

```
        MOV   AH,4CH
        INT   21H                    ;返回 DOS
CODE    ENDS
        END   START                  ;汇编结束
```

4.2.5 子程序设计

在程序设计中，常把多处用到的同一个程序段或者具有一定功能的程序段单独存放在某一存储区域中，需要执行的时候，使用调用指令转到这段程序来执行，执行完再返回原来的程序，这个程序段就称为子程序。

调用子程序的程序段称为主程序。主程序中调用指令的下一条指令的地址称为返回地址，有时也称为断点。

1. 主—子程序调用及功能分析

下面通过一段程序实例，来说明子程序的基本结构和主程序与子程序之间的调用关系。

【例 4.23】 设计一个子程序，完成统计一组字数据中的正数和 0 的个数。

```
DATA    SEGMENT
  ARR   DW  -123,456,67,0,-34,-90,89,67,0,256
  CN    EQU (S-ARR)/2
  ZER   DW  ?
  PLUS  DW  ?
DATA    ENDS
CODE    SEGMENT
  ASSUME  DS:DATA,CS:CODE
START:  MOV  AX,DATA
MOV  DS,AX                       ;初始化 DS
MOV  SI,OFFSET ARR               ;数组首地址送 SI
MOV  CX,CN                       ;数组元素个数送 CX
CALL PZN                         ;调用近过程 PZN
MOV  ZER,BX                      ;0 的个数送 ZER
MOV  PLUS,AX                     ;正数的个数送 PLUS
MOV  AH,4CH
INT  21H                         ;返回 DOS
;子程序名: PZN
;子程序功能: 统计一组字数据中的正数和 0 的个数
;入口参数: 数组首地址在 SI 中,数组个数在 CX 中
;出口参数: 正数个数在 AX 中,0 的个数在 BX 中
;使用寄存器: AX、BX、CX、DX、SI 及 PSW
PZN PROC  NEAR
    PUSH  SI
    PUSH  DX
    PUSH  CX                     ;保护现场
    XOR   AX,AX
    XOR   BX,BX                  ;计数单元清 0
PZN0:MOV  DX,[SI]                ;取一个数组元素送 DX
    CMP   DX,0                   ;DX 中内容和 0 比较
    JL    PZN1                   ;小于 0 转 PZN1
    JZ    ZN                     ;等于 0 转 ZN
    INC   AX                     ;否则为正数,AX 中内容加 1
    JMP   PZN1                   ;转 PZN1
```

```
        ZN:     INC  BX                    ;为 0,BX 中内容加 1
        PZN1:   ADD  SI,2                  ;数组指针加 2 调整
                LOOP PZN0                  ;循环控制
                POP  CX
                POP  DX
                POP  SI                    ;恢复现场
                RET                        ;返回主程序
        PZN     ENDP                       ;子程序定义结束
CODE    ENDS                               ;代码段结束
        END     START                      ;汇编结束
```

从本例可以看出子程序的基本结构包括以下几个部分。

（1）子程序说明：用来说明子程序的名称、功能、入口参数、出口参数、占用工作单元的情况，明确该子程序的功能和调用方法。

（2）现场保护及恢复：由于汇编语言所处理的对象主要是 CPU 寄存器或内存单元，主程序在调用子程序时已经占用了一定的寄存器，子程序执行时又要用到这些寄存器，执行完毕返回主程序后，为了保证主程序按原有的状态继续正常执行，需要对这些寄存器的内容加以保护，这就是保护现场；子程序执行完毕后再恢复这些被保护的寄存器的内容，称为恢复现场。现场保护及恢复通常采用堆栈操作。

（3）子程序体：这一部分内容用来实现相应的子程序功能。

（4）子程序返回：子程序的返回语句 RET 和主程序中的调用语句 CALL 相互对应，才能正确实现子程序的调用和返回。返回指令用来恢复被中断位置的地址。

2. 子程序的参数传递

主程序在调用子程序之前，必须把需要加工处理的数据传递给子程序，这些被加工处理的数据称为输入参数；当子程序执行完毕返回主程序时，应把本次加工处理的结果传递给主程序，这些结果称为输出参数。主程序向子程序传递输入参数及子程序向主程序传递输出参数称为主程序和子程序间的参数传递。

汇编语言中实现参数传递的方法主要有 3 种：寄存器传递、堆栈传递和存储器传递。

（1）寄存器传递：这种方式适合于需要传递的参数较少的情况。具体方法是在调用子程序之前，把参数放到规定的寄存器中，由这些寄存器将参数带入子程序中，执行子程序结束后的结果也放到规定的寄存器中带回主程序。

（2）堆栈传递：这是通过堆栈这个临时存储区来实现参数传递，主程序将入口参数压入堆栈，子程序从堆栈中取出参数；子程序将出口参数压入堆栈，主程序从堆栈中取出参数。在编译程序处理参数传递及汇编语言与高级语言混合编程时经常采用。

（3）存储器传递：若调用程序和子程序在同一个程序模块内，采用存储器传递参数是最简单的方法。通常在数据段内定义出要传送的数据变量，也称为数据缓冲区，子程序对这些定义的变量直接访问即可。

本章小结

汇编语言是面向机器的程序设计语言，它使用指令助记符、符号地址及标号编制程序，要熟悉汇编语言源程序的基本格式，正确运用语句格式来书写程序段，掌握伪指令的功能和

应用，并通过上机操作，熟悉编辑程序、汇编程序、连接程序和调试程序等软件工具的使用，掌握源程序的建立、汇编、连接、运行、调试等技能。

本章还介绍了汇编语言程序设计的一般步骤，举例说明了 3 种基本的程序结构的设计方法。顺序结构是按照语句实现的先后次序执行一系列的操作，是最简单的一种结构。分支结构是程序设计中常用的结构之一，它有双分支和多分支两种形式。循环结构用来实现需要重复执行的操作，通常由循环初始化、循环体部分、参数修改部分和循环控制部分共 4 部分组成。子程序设计是把多处用到的同一个程序段或者具有一定功能的程序段单独存放在某一存储区域中，需要执行的时候，使用调用指令转到这段程序来执行，执行完再返回原来的程序。这样做提高了程序段的利用率，也有利于模块化的程序设计和开发。

熟悉各种程序的结构和编程技巧对汇编语言程序设计有着积极的促进作用。

思考与练习题

1. 编程实现求 $S=(X^2+Y^2)/Z$ 的值，并将结果放入 RESULT 单元。
2. 编程序完成计算 S=(A+B)/2-2(A AND B)。
3. 试编程实现将内存中的若干个无符号字数据中的最大值和最小值找出来，并开辟独立的存储区域保存结果。
4. 编写程序，计算下面函数的值。

$$S=\begin{cases} 2X & (X<0) \\ 3X & (0\leqslant X\leqslant 10) \\ 4X & (0>10) \end{cases}$$

5. 从键盘输入一系列字符，以回车符结束，编程统计其中非数字字符的个数。
6. 试编程实现将键盘连续输入的小写字母用大写字母显示出来，小写字母与大写字母之间采用“—”号分隔，当输入非小写字母时程序停止处理。
7. 在数据段中有一个字节数组，编程统计其中正数的个数放入 A 单元保存，统计负数的个数放入 B 单元保存。

第 5 章 微型计算机的总线技术

总线是微型计算机系统的重要组成部分，是系统中传递各类信息的通道，也是微型计算机系统中各模块间的物理接口，它负责在 CPU 和其他部件之间进行信息的传递。总线的性能好坏直接影响到微型计算机系统的工作效率、可靠性、可扩展性、可维护性等多项系统性能。

本章从总线的基本概念入手，分析常用的系统总线、局部总线和外部设备总线，阐述各类总线的特点和功能。

通过本章的学习，读者应该理解总线的基本概念，熟悉微机总线的组成结构，掌握常用系统总线、局部总线、外部设备总线以及 USB 总线的结构和引脚特性，并侧重其应用。

5.1 总线技术概述

总线是微型计算机系统中多个部件之间公用的一组连线，由它构成芯片、插件或系统之间的标准信息通路。它是系统中各个部件信息交换的公共通道，各部件之间的联系都是通过总线实现的。微型计算机采用总线技术的目的是为了简化硬、软件的系统设计，在硬件方面，设计者只需按总线规范设计插件板，保证它们具有互换性与通用性，支持系统的性能及系列产品的开发；在软件方面，接插件的硬件结构带来了软件设计的模块化。用标准总线连接的计算机系统结构简单清晰，便于扩充与更新。

按各类总线的结构和连接对象及范围的不同，通常可分为以下 3 类总线。

1. 微处理器片内总线

片内总线是指在微处理器芯片内部各单元之间传输信息的总线，它主要用于芯片级的互连。片内总线对微处理器来讲就是它的引脚信号，例如微处理器内部的控制器、运算器、寄存器之间，以及系统主板上的 CPU、存储器、接口电路之间的连接，就是采用片内总线来实现的，有时也称其为局部总线。

2. 内总线

内总线用以实现微机系统与各种扩展插件板之间的相互连接，是微机系统所特有的总线，一般用于模板之间的连接。内总线有时也称为板极总线或系统总线，目前这类总线已基本上实现了标准化，例如 STD 总线、ISA 总线、VESA 总线、PCI 总线等。在微型计算机主板上的各种扩展插槽都属于内总线。

3. 外部总线

这是微型计算机之间或微型计算机与外部设备之间进行通信的总线，主要用于设备级的互联。外部总线也称为通信总线。其种类比较多，通常与特定的设备有关，例如：RS-232C 总线、RS-458 总线、USB 总线等。

图 5-1 所示即为常见的微型计算机总线层次的典型结构。

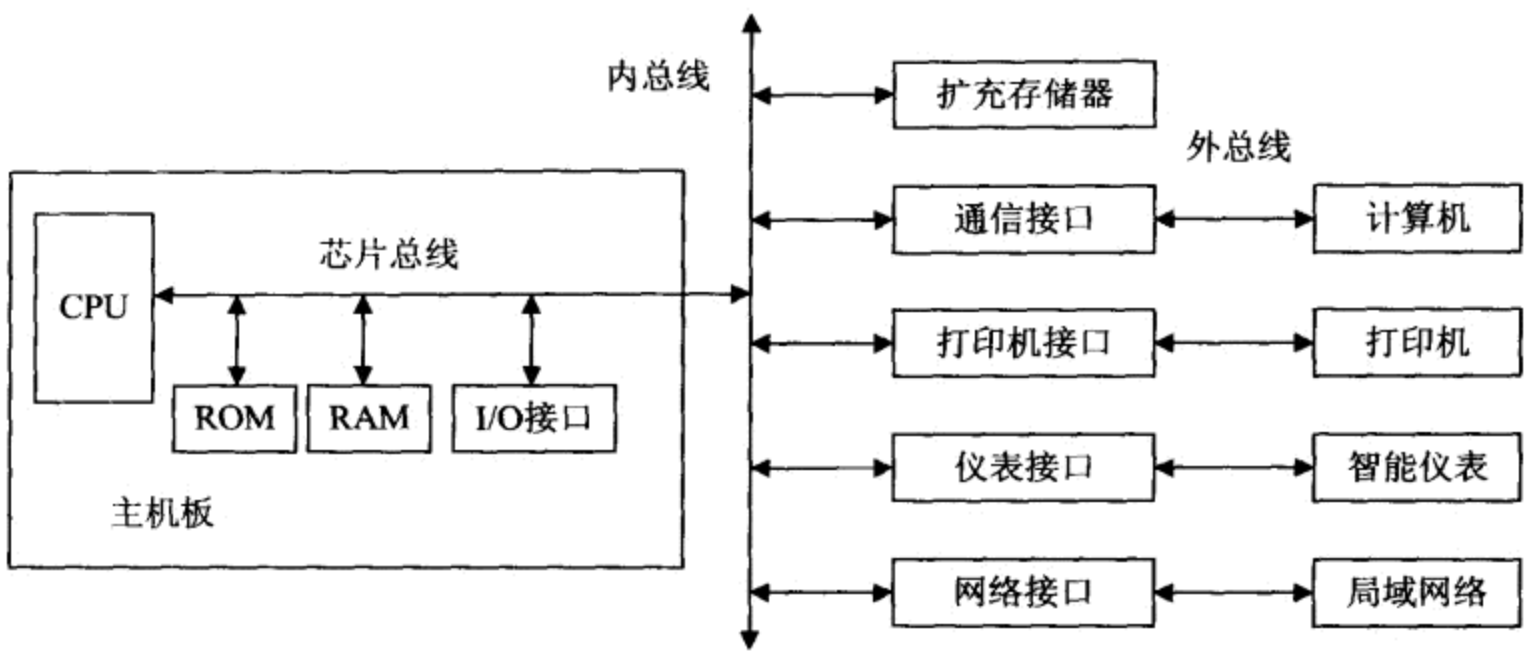


图 5-1　微型计算机总线层次结构示意图

4. 标准总线的特点

为了使微型计算机应用系统朝模块化、标准化的方向发展，通常要求微型计算机系统的总线结构应该具备标准化模式。标准总线具有简化系统设计、简化系统结构、易于系统扩展、便于系统更新以及便于系统调试和维修等特点。

标准总线不仅在电气上规定了各种信号的标准电平、负载能力和定时关系，也在结构上规定了插件的尺寸规格和引脚定义，各模块可实现标准连接。目前总线标准有两类：一类是 IEEE（美国电气及电子工程师协会）标准委员会定义与解释的标准，如 IEEE-488 总线和 RS-232C 串行接口标准等；另一类是因广泛应用而被大家接受与公认的标准，如 S-100 总线、IBM PC 总线、ISA 总线、EISA 总线、PCI 总线等。

总线的性能主要体现在总线宽度和总线频率上。

（1）总线宽度：是指可以同时传输的数据位数，位数越多，一次传输的信息就越多。如 EISA 总线宽度为 16 位，PCI 总线宽度为 32 位，PCI-2 总线宽度可达到 64 位。

（2）总线频率：总线通常都有一个基本时钟，总线上其他信号都以这个时钟为基准，这个时钟的频率也是总线工作的最高频率。时钟频率越高，单位时间内传输的数据量就越大。如 EISA 总线时钟频率为 8MHz，PCI 总线为 33.3MHz，PCI-2 总线可达 66MHz。

随着微型计算机的发展，总线技术也在不断地发展与完善，并且已经出现了一系列的标准化总线，这些标准化总线的广泛使用，对微型计算机系统在各个领域的普及和应用起到了积极的推动作用。

5.2　系统总线

系统总线是微型计算机主板上微处理器和外部设备之间进行通信时所采用的数据通道，可以支持各种端口、处理器、RAM 和其他部件。当采用键盘或鼠标输入数据时，数据经过系统总线进入 RAM，然后再进入 CPU 进行相应处理。

为了使总线具有可扩展性，通常采用开放系统总线的方式，在主板上预留一些扩展插槽来提供系统总线，新的计算机硬件设备通过插在扩展槽上被装备到计算机中，实现与主板上

其他部分之间的数据通信。

为了满足一些高传输率扩展卡的需要，从系统总线中又分离出了局部总线。局部总线具有较好的传输速率，可以保证系统总线的性能，目前已经得到广泛的应用。

微型计算机的系统总线从性能上可以分为低端总线和高端总线。

低端总线一般支持 8 位、16 位的微处理器，主要功能是进行 I/O 处理，总线信号依赖微处理器芯片，有的总线实际上就是微处理器引脚的延伸，比如 ISA 总线、STD 总线等。

高端总线可以支持 32 位、64 位微处理器，它提高了数据传输率和处理能力，对微处理器的依赖性在减小，同时具备良好的兼容性、支持高速缓存 Cache、支持多微处理器、可自动配置等特点，比如 EISA 总线、VESI 总线、PCI 总线等。

表 5-1 所示是一些常见的微型计算机总线性能比较，从中可以了解到各种系统总线和局部总线的总体状况。

表 5-1 常见的微型计算机总线性能比较

总线类型	通用机型	总线宽度（bit）	总线工作速率（MHz）	最大传输速率（Mb/s）
PC/XT	8086	8	4	4
ISA	80286、386、486 系列	16	8	16
EISA	80286、386、586 系列	32	8.33	33.3
STD	V20、V40、IBM 系列	8	2	2
MCA	IBM PC、工作站	32	8	33
PCI	Pentium 系列 PC	32、64	33	132、264
AGP	Pentium 系列 PC	64	66 以上	264 以上

5.2.1 STD 总线

STD（Standard）总线是用于工业控制微型计算机的标准系统总线，以其优越的性能和强大的生命力在工业控制领域中受到了广泛的欢迎并得到迅速的发展，被定名为 IEEE961。

STD 总线开始推出时是针对当时的 8 位微型计算机，经过不断修订和改进，利用复用技术在原来定义的 56 个总线信号之下实现了支持 20 位地址、寻址 1MB 的直接寻址能力。在保证现有 I/O 插件板兼容的条件下，提供全 16 位数据的传输能力，20 世纪 80 年代末 STD 总线已由 56 个信号发展到 114、136 个信号。1989 年美国 EAITECH 公司开发出了 32 位的 STD32，为高档的 STD 微型计算机系统的发展提供了有利的条件。

1. STD 总线的特点

- 具有较好的兼容性，可以向上或向下兼容。例如，8 位的 STD 产品可以与新标准的 16 位或 32 位 STD 产品一起工作。
- STD 总线的电路板采用小板结构，高度模块化，开发周期短，并且可以根据用户要求组成各种规模的微型计算机应用系统。
- 采用一整套高可靠性措施，使该总线构成的工业控制机可以长期可靠地工作在恶劣环境之下。

- 结构简单，并且能够支持多微处理器系统，是一种规模小并且性能好的系统总线。
- STD 总线系统模板品种齐全，价格低廉，性能良好，易于选用，为推广 STD 总线产品创造了有利条件。

2. STD 总线信号定义

表 5-2 给出了 STD 总线信号的定义，总计 56 条信号，其中电源线 10 条，地址线 24 条，数据线 16 条，控制线 22 条，地址线与数据线采用分时复用方式进行工作。

表 5-2　STD 总线信号定义

总线类别	元件面			焊接面		
	引　脚	信号名称	功　能	引　脚	信号名称	功　能
	1	Vcc	逻辑电源+5V	2	Vcc	逻辑电源+5V
逻辑电源总线	3	GND	逻辑接地	4	GND	逻辑接地
	5	VBB.1/VBAT	偏置 1/电池	6	VBB.2/VBAT	偏置 2/掉电电源
数据总线	7	D_3/A_{19}	数据总线/地址扩展总线	8	D_7/A_{23}	数据总线/地址扩展总线
	9	D_2/A_{18}		10	D_6/A_{22}	
	11	D_1/A_{11}		12	D_5/A_{21}	
	13	D_0/A_{16}		14	D_4/A_{20}	
地址总线	15	A_7	地址总线	16	A_{15}/D_{15}	地址总线/数据扩展总线
	17	A_6		18	A_{14}/D_{14}	
	19	A_5		20	A_{13}/D_{13}	
	21	A_4		22	A_{12}/D_{12}	
	23	A_3		24	A_{11}/D_{11}	
	25	A_2		26	A_{10}/D_{10}	
	27	A_1		28	A_9/D_9	
	29	A_0		30	A_8/D_8	
控制总线	31	$\overline{WR}$	写命令	32	$\overline{RD}$	读命令
	33	$\overline{IORQ}$	I/O 地址选择	34	$\overline{MEMRQ}$	存储器地址选择
	35	$\overline{IOEXP}$	I/O 扩展	36	$\overline{MEMEX}$	存储器扩展
	37	$\overline{REFRESH}$	刷新定时	38	$\overline{MCSYNC}$	CPU 周期同步
	39	$\overline{STATUS_1}$	CPU 状态	40	$\overline{STATUS_0}$	CPU 状态
	41	$\overline{BUSAK}$	总线响应	42	$\overline{BUSRQ}$	总线请求
	43	$\overline{INTAK}$	中断响应	44	$\overline{INTRQ}$	中断请求
	45	$\overline{WAITRQ}$	等待请求	46	$\overline{NMIRQ}$	非屏蔽中断请求

续上表

总线类别	元件面			焊接面		
	引 脚	信号名称	功 能	引 脚	信号名称	功 能
控制总线	47	$\overline{\text{SYSRESET}}$	系统复位	48	$\overline{\text{PBRESET}}$	按钮复位
	49	$\overline{\text{CLOCK}}$	处理器时钟	50	$\overline{\text{CNTRL}}$	辅助定时
	51	PCO	优先级链输出	52	PCI	优先级链输入
辅助电源总线	53	AUXGND	辅助接地	54	AUXGND	辅助接地
	55	AUX+V	电源+12V	56	AUX−V	辅助电源-12V

3. STD 总线的优先级控制

STD 总线支持多微处理器系统，更广义地说是支持多总线控制器系统。当 STD 系统总线上有多个总线控制器时，必须保证在每一时刻要将总线让给提出请求的优先级最高的总线控制器使用。

利用以下串、并联两种总线优先级的安排可以实现 STD 总线的优先级控制。

（1）串联总线优先级控制：这种方式是在每一个总线控制器模板上，建立总线优先级逻辑电路。

【例 5.1】采用串联优先级控制链的形式进行总线的优先级控制。如图 5-2 所示，在 STD 总线上用 3 个优先级逻辑电路和相应控制线组成。

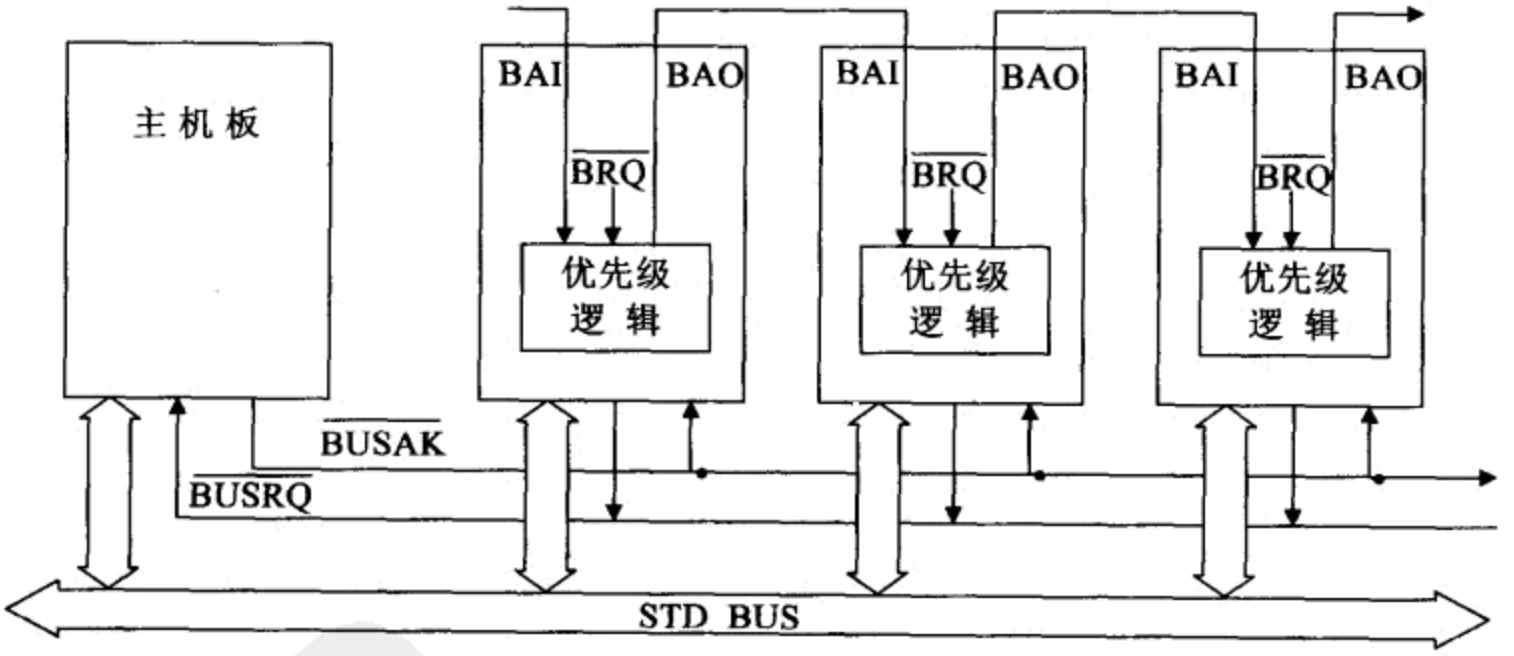


图 5-2 STD 的串联总线优先级控制

在图 5-2 中，总线优先级控制的工作原理如下。

- 当 BAI 为高电平时，允许总线控制器向处理器提出总线请求，一旦响应，则 BAO 为低电平，阻止其他总线控制器提出总线请求。
- 当 $\overline{\text{BRQ}}$ 为低电平时，为模板总线请求。
- 当 $\overline{\text{BUSAK}}$ 为低电平时，$\overline{\text{BUSRQ}}$ 就不会发出有效信号。
- 只有 $\overline{\text{BUSAK}}$ 为高电平时，且 BAI 也为高电平时，$\overline{\text{BRQ}}$ 才能产生 $\overline{\text{BUSRQ}}$ 有效信号。
- 一旦 BAI 为高电平、$\overline{\text{BUSAK}}$ 为低电平时，它就取得了总线的控制权。

（2）并联总线优先级控制：这种方法是将优先级控制逻辑电路放在单独一块模板上，其他各个总线控制器均通过这块模板提出总线请求。

【例 5.2】总线的并联优先级控制分析。并联总线优先级控制如图 5-3 所示。

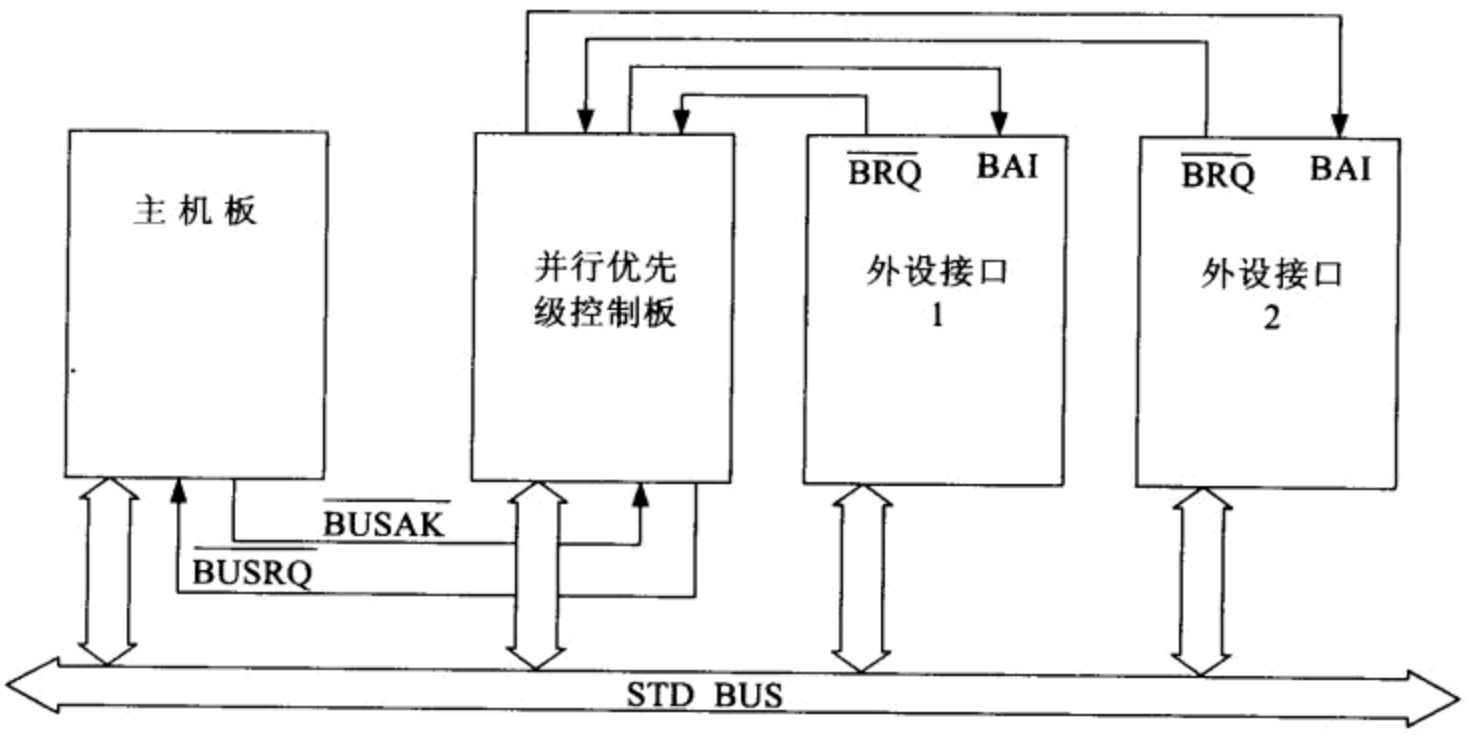


图 5-3　STD 的并联总线优先级控制

在图 5-3 中，单独模板上的并联总线优先级控制电路对各总线控制器分配不同的优先级，这样，在 STD 总线工作时，可以使各个部件按规定的操作流程有序地进行。

4. 利用 STD 总线进行存储器扩展

从 STD 总线信号定义中可以看到，构成 STD 系统时如果 CPU 能够提供更多的地址信号，那么，存储器的寻址范围可以达到 1～16MB。如果 CPU 只能提供 16 条地址线，那么在一般情况下存储器的寻址范围只能达到 64KB。对于只有 64KB 的内存空间来讲，在某些特定的场合，尤其是数据量比较大时是不够用的，需要采取一些相应的措施来扩展内存。在 STD 总线系统中可以采用以下措施来实现。

（1）利用 $\overline{\text{MEMEX}}$ 信号选择存储器组

如果在设计 STD 总线时锁存并控制存储器扩展 $\overline{\text{MEMEX}}$ 信号的电平，就可以在 $\overline{\text{MEMEX}}$ 为低电平时，使处理器寻址 64KB 的一组存储器，而当 $\overline{\text{MEMEX}}$ 为高电平时，寻址另外一组 64KB 的存储器，从而使系统达到能够寻址 128KB 的能力。

（2）输出接口存储器组选择

【例 5.3】存储器容量的扩充还可以利用图 5-4 所示的输出接口的输出来选择存储器组。其中，每一位输出可选择一个 64KB 的存储器组。图 5-4 使用 8 位锁存器将使存储器扩大到 512KB，而且用起来比较方便。如果增加图中锁存器的个数，则可以继续扩大存储器的容量。

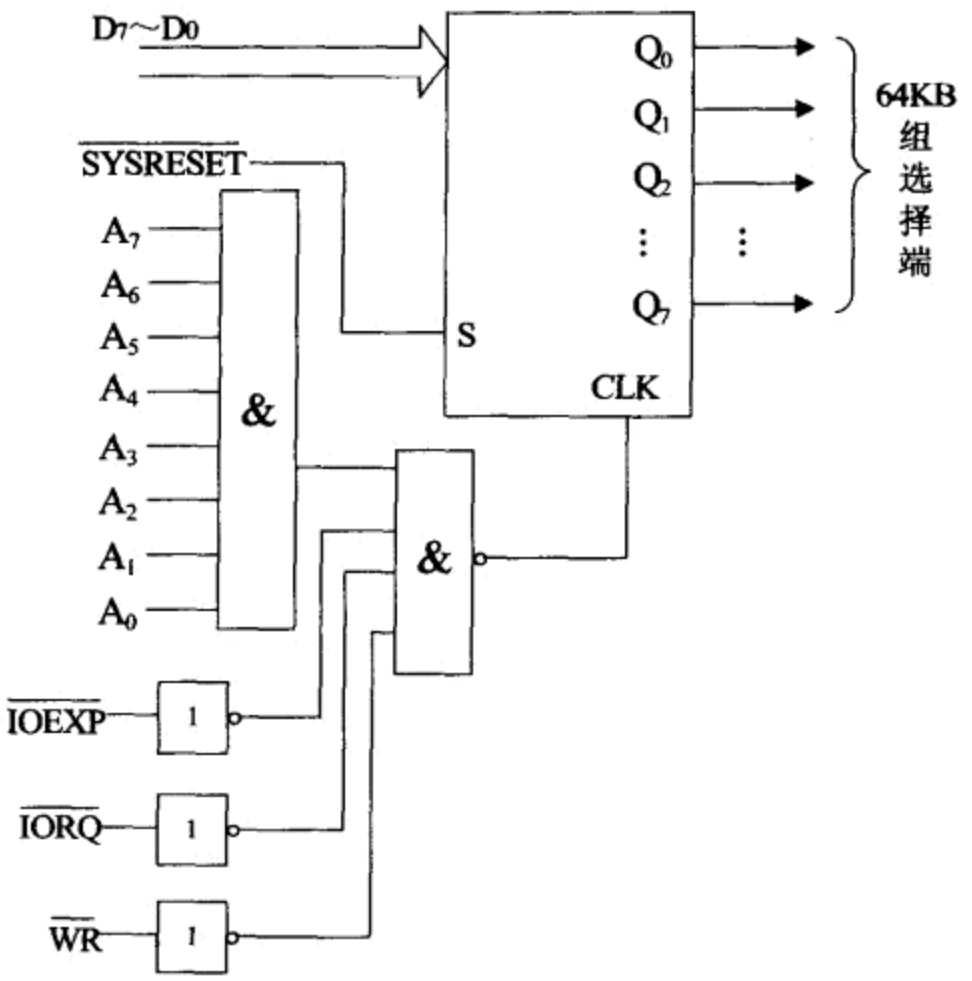


图 5-4　8 位锁存器用于选择存储器组

5.2.2　ISA 总线

ISA（Industry Standard Architecture，工业标准体系结构）总线是早期比较有代表性的总线。自从 1981 年 IBM 推出 PC 以来，它的 PC 产品经历了巨大的变革。早期的 PC/XT 微机成为当时 8 位机的典型代表。随着 PC/XT 微机的普及，XT 总线成为许多应用系统设计的标准总线，它定义了 62 个总线信号，带宽为 8 位。在 XT 总线系统里，CPU 不仅要处理数据和指令，还要管理扩展总线。1984 年 IBM PC/AT 微机出现时采用了 AT 总线。AT 总线与 80286 CPU 的性能相匹配，它使用 16 位技术支持更宽的地址总线和数据总线，保留了原有 XT 总线的全部 62 个信号，并扩展了 36 个信号，形成 62 引脚大插槽附加 36 引脚小插槽的特殊结构。AT 总线的速度与 80286 相同，为 6 MHz 或 8 MHz。

IBM 公司推出 PC/XT 和 PC/AT 个人计算机后，IEEE（Institute of Electronic and Electronic Engineers 电气电子工程师协会）在 1987 年定义了工业标准体系结构（ISA），将 PC/XT 总线定义为 8 位 ISA，将 PC/AT 总线定义为 16 位 ISA。

一般来说，ISA 总线具有以下特点。

- 既支持 8 位数据操作，也支持 16 位数据操作，为早期的微型计算机提供了良好的兼容性。
- 可以将 PC/AT 总线和 PC/XT 总线的运行速度提升至 8MHz，还可提供最大为 8MB/s 的数据传输率。
- 强调 I/O 处理能力，提供了 1KB 的 I/O 空间、15 级的硬件中断、7 级的 DMA 通道、8 个设备的负载能力。
- 总线中的地址、数据线采用非多路复用形式，使系统的扩展设计更为简便。

- ISA 是一种多主控设备总线，除主 CPU 外，DMA 控制器、DRAM 刷新控制器、带处理器的智能卡都可成为 ISA 的主控设备。
- 由于 IBM PC 广泛流行，可供选择的 ISA 插件卡品种较多，这有利于用户根据需要快速构成相应的微型计算机应用系统。

1. 8 位 ISA 总线

8 位 ISA 总线是最早的 PC 系统总线，也叫做 PC 总线。它支持 8 位数据传输和 10 位寻址空间，其特点是把 CPU 视为总线的唯一总控设备，其余外围设备均为从属设备。

8 位 ISA 总线是一种开放式的结构总线，在总线母板上有 8 个系统插槽，用于 I/O 设备和 PC 的连接。由于该总线具有价格低、可靠性好、使用灵活等特点，并且对插板兼容性好，因此，有许多厂家生产该总线的兼容产品，品种范围比较广泛，供用户选择的余地较大。最初的 ISA 总线产品主要用于办公自动化，随着使用的推广，其产品很快扩大到实验室及工业环境下的数据采集和控制。

8 位 ISA 总线引脚信号定义如表 5-3 所示，总共有 62 条。通过一个 31 脚分为 A、B 两面的连接插槽来实现，其中，A 面为元件面，B 面为焊接面。符合 ISA 总线标准的接插件可以方便地插入，以便对微型计算机系统进行功能扩展。

表 5-3　8 位 ISA 总线引脚信号定义

元件面			焊接面		
引脚号	信号名称	功能说明	引脚号	信号名称	功能说明
A_1	$\overline{\text{I/OCHCK}}$	I/O 校验	B_1	GND	地线
A_2	D_7	双向数据信号	B_2	RESET DRV	复位驱动信号
A_3	D_6		B_3	+5V	电源线
A_4	D_5		B_4	IRQ_2	中断请求信号
A_5	D_4		B_5	−5V	电源线
A_6	D_3		B_6	DRQ_2	DMA 通道 2 请求信号
A_7	D_2		B_7	−12V	电源线
A_8	D_1		B_8	$\overline{\text{CARDSLCTD}}$	插件板选中信号
A_9	D_0		B_9	+12V	电源线
A_{10}	I/O CHRDY	I/O 就绪信号	B_{10}	GND	地线
A_{11}	AEN	地址允许信号	B_{11}	$\overline{\text{SMEMW}}$	存储器写信号
A_{12}	A_{19}	双向地址信号	B_{12}	$\overline{\text{SMEMR}}$	存储器读信号
A_{13}	A_{18}		B_{13}	$\overline{\text{IOW}}$	I/O 接口写信号
A_{14}	A_{17}		B_{14}	$\overline{\text{IOR}}$	I/O 接口读信号
A_{15}	A_{16}		B_{15}	$\overline{\text{DACK}_3}$	DMA 通道 3 响应信号
A_{16}	A_{15}		B_{16}	DRQ_3	DMA 通道 3 请求信号
A_{17}	A_{14}		B_{17}	$\overline{\text{DACK}_1}$	DMA 通道 1 响应信号

续上表

元件面			焊接面		
引脚号	信号名称	功能说明	引脚号	信号名称	功能说明
A_{18}	A_{13}	双向地址信号	B_{18}	DRQ_1	DMA 通道 1 请求信号
A_{19}	A_{12}		B_{19}	$\overline{DACK_0}$	DMA 通道 0 响应信号
A_{20}	A_{11}		B_{20}	CLK	系统时钟信号
A_{21}	A_{10}		B_{21}	IRQ_7	中断请求输入信号
A_{22}	A_9		B_{22}	IRQ_6	
A_{23}	A_8		B_{23}	IRQ_5	
A_{24}	A_7		B_{24}	IRQ_4	
A_{25}	A_6		B_{25}	IRQ_3	
A_{26}	A_5		B_{26}	$\overline{DACK_2}$	DMA 通道 2 响应信号
A_{27}	A_4		B_{27}	T/C	计数结束信号
A_{28}	A_3		B_{28}	ALE	地址锁存允许信号
A_{29}	A_2		B_{29}	+5V	电源线
A_{30}	A_1		B_{30}	OSC	晶体振荡脉冲信号
A_{31}	A_0		B_{31}	GND	地线

8 位 ISA 总线引脚信号具有 20 条地址线、8 条数据线、若干控制信号线、电源、接地等接口信号线，下面分别进行说明。

（1）地址线 A_{19}～A_0（20 条）

地址总线为双向传输，用来指出内存地址或 I/O 接口地址。在系统总线周期中由 CPU 驱动，在 DMA 周期中由 DMA 控制器驱动，用地址允许信号 AEN 确定。存储器寻址时，20 条地址线可访问 1MB 的存储空间，进行 I/O 端口寻址时，用 16 条地址线 A_{15}～A_0 可访问 64K 个端口地址，此时 A_{19}～A_{16} 无效。

（2）数据线 D_7～D_0（8 条）

双向传输，用于在 CPU、存储器及 I/O 端口之间传输数据信息及指令操作码，采用相应的控制线进行数据选通。

（3）控制线（21 条）

- AEN：地址允许信号，输出，高电平有效，由 DMA 控制器 8237A 发出。AEN=1 时表示切断 CPU 的控制，由 DMA 控制器行使总线控制权；AEN=0 时表示正在进行 CPU 总线周期控制，由 CPU 行使总线控制权。
- ALE：地址锁存允许信号，输出，高电平有效，由总线控制器 8288 提供，以便把地址和数据分离。该信号可将地址/状态总线上送来的数据作为地址码进行锁存，进行 DMA 操作时 ALE 为低电平。
- $\overline{MEMR}$：存储器读信号，输出，低电平有效，用于请求从存储器读取数据。该信号

由总线控制器 8288 或 DMA 控制器 8237A 驱动。

- $\overline{\text{MEMW}}$：存储器写信号，输出，低电平有效。将来自数据总线的数据写入存储器，信号由总线控制器驱动。
- $\overline{\text{IOR}}$：I/O 端口的读信号，输出，低电平有效。指明当前的总线周期是一个 I/O 端口读周期，地址总线上的地址是一个 I/O 端口地址，被寻址端口的数据送上数据总线由 CPU 读取。此信号由总线控制器 8288 产生，当 DMA 操作时由 DMA 控制器 8237A 产生。
- $\overline{\text{IOW}}$：I/O 端口的写信号，输出，低电平有效。该信号可由 CPU 或 DMA 控制器提供，由总线控制器驱动后送至总线。其作用是把数据总线上的数据写入所选中的 I/O 端口中。
- IRQ_7～IRQ_2：6 级中断请求信号，输入，高电平有效。I/O 设备发出，通知 CPU 要求中断服务，由 8259A 接收，按优先级进行排队，优先级最高者将被响应，其中 IRQ_2 优先级最高，依次降低，IRQ_7 优先级最低。
- DRQ_3～DRQ_1：3 条 DMA 请求信号，输入，高电平有效。是 I/O 端口用来申请 DMA 周期的，这 3 个信号由申请 DMA 服务的 I/O 设备发到 DMA 控制器 8237。其优先权由高到低依次为 DRQ_1、DRQ_2、DRQ_3。
- $\overline{DACK_3}$～$\overline{DACK_0}$：4 条 DMA 响应信号，低电平有效。由 DMA 控制器送往 I/O 外设接口，用来响应外设的 DMA 请求或者实现对动态 RAM 的刷新。
- T/C：计数结束信号，高电平有效。由 DMA 控制器发出，表示 DMA 的某一通道到达计数终点，该信号用来结束数据块的传送。
- RESET DRV：复位驱动信号，高电平有效。在加电或单击复位按钮时，对接到总线上的电路和接口设备进行复位。

（4）状态线（2 条）

- $\overline{\text{I/OCHCK}}$：I/O 通道奇偶校验输入信号，低电平有效。由插入扩展槽的存储器卡或 I/O 卡发出，用来向 CPU 提供关于 I/O 通道上的设备或存储器的奇偶校验信息。当其为低电平时，表明奇偶校验有错，会对微处理器产生不可屏蔽中断（NMI）。
- I/O CHRDY：I/O 通道准备就绪信号，高电平有效。由扩展槽中的存储器卡或 I/O 卡发出。该信号主要用来解决慢速的外设与快速 CPU 或 DMA 控制器之间的矛盾。

（5）辅助线、电源和地线（11 条）

- OSC：晶体振荡脉冲信号，振荡周期为 70ns，主振频率为 14.318MHz。
- CLK：系统时钟信号，由 OSC 三分频得到，周期为 210ns，频率为 4.77MHz，用于总线周期同步。
- $\overline{\text{CARDSLCTD}}$：插件板选中信号，只用于 PC/XT 主板上第 8 个扩展槽中的插件板。该信号向 CPU 表明插件板已被选中，可以进行读取数据的操作。
- 电源线：有±5V、±12V 电源，其中+5V 电源线 2 条，其余电源线各 1 条。
- 地线 GND：有 3 条地线。

2. 16 位 ISA 总线

PC/AT 总线在 PC/XT 62 引脚总线的基础上增加了一个 36 引脚的插槽，形成前 62 引脚

和后 36 引脚的两个插座，构成 16 位 ISA 总线。可以利用前 62 引脚的插座插入与 PC/XT 总线兼容的 8 位接口电路卡，也可以利用整个插座插入 16 位接口电路卡。

16 位 ISA 总线的前 62 引脚的信号分布及其功能与 8 位 ISA 总线基本相同，仅有两处做了改动：一是原 B_{19} 引脚为 $\overline{DACK_0}$，现改为 $\overline{REFRESH}$，这是由于 PC/AT 机的 DRAM 刷新不再通过 DMA 传输完成，故可以直接由系统板上刷新电路产生 $\overline{REFRESH}$ 信号；二是原 B_8 引脚为 $\overline{CARDSLCTD}$，现改为 $\overline{OWS}$（零等待状态）信号，表示接口电路卡上的设备不需处理器插入任何附加等待状态，即可完成当前总线周期。

16 位 ISA 总线中新增加的 36 引脚插槽信号扩展了 8 位数据线、7 位地址线、存储器和 I/O 设备的读/写控制线、中断和 DMA 控制线、电源和地线等。

新插槽中的引脚信号分为 C（元件面）和 D（焊接面）两列，其信号名称和功能如表 5-4 所示。

表 5-4 16 位 ISA 总线新增加的 36 条引脚信号

元件面			焊接面		
引脚号	信号名称	功能说明	引脚号	信号名称	功能说明
C_1	$\overline{SBHE}$	高字节允许	D_1	$\overline{MEMCS_{16}}$	存储器 16 位片选
C_2	LA_{23}	高位地址	D_2	$\overline{IOCS_{16}}$	接口 16 位片选
C_3	LA_{22}		D_3	IRQ_{10}	中断请求信号
C_4	LA_{21}		D_4	IRQ_{11}	
C_5	LA_{20}		D_5	IRQ_{12}	
C_6	LA_{19}		D_6	IRQ_{14}	
C_7	LA_{18}		D_7	IRQ_{15}	
C_8	$LA1_7$		D_8	$\overline{BACK_0}$	DMA 请求与响应信号
C_9	$\overline{MEMR}$	存储器读信号	D_9	DRQ_0	
C_{10}	$\overline{MEMW}$	存储器写信号	D_{10}	$\overline{BACK_5}$	
C_{11}	SD_8	数据总线高字节信号	D_{11}	DRQ_5	
C_{12}	SD_9		D_{12}	$\overline{BACK_6}$	
C_{13}	SD_{10}		D_{13}	DRQ_6	
C_{14}	SD_{11}		D_{14}	$\overline{BACK_7}$	
C_{15}	SD_{12}		D_{15}	DRQ_7	
C_{16}	SD_{13}		D_{16}	+5V	+5V 电源信号
C_{17}	SD_{14}		D_{17}	$\overline{MASTER}$	主控信号
C_{18}	SD_{15}		D_{18}	GND	接地

新增加的 36 条引脚信号功能分析如下。

（1）LA_{23}～LA_{17} 地址线（7 条）：这是为了提高速度新增加不采用锁存的 7 条高位地址线，其中 4 条高位地址线 LA_{23}～LA_{20} 使原来的 1MB 寻址范围扩大到 16MB。

（2）SD_8～SD_{15}数据线（8 条）：这是新增加的高 8 位双向数据线。

（3）$\overline{SBEH}$ 数据总线高字节允许信号：该信号与其他地址信号一起，实现对高字节、低字节或一个字的操作。

（4）IRQ_{10}～IRQ_{15}中断请求信号：新增的中断请求输入信号。其中 IRQ_{13} 指定给数据协处理器使用。由于 PC/AT 总线上增加了外部中断的数量，在主板上是由两块 8259 级联实现中断优先级控制。优先级别低的一块 8259 的中断请求接到主中断控制器的 IRQ_2 上面，这样 PC/XT 总线定义的 IRQ_2 引脚在 PC/AT 总线中就变成了 IRQ_9。另外，IRQ_8 接定时器 8254 用于产生定时中断。

（5）DMA 传送控制信号线：为实现 DMA 传送，在 PC/AT 机的主板上采用两块 DMA 控制器级联使用，其中主控级的 DRQ_0 接从属级的请求信号 HRQ，形成了 DRQ_0～DRQ_7 中间没有 DRQ_4 的 7 级 DMA 优先级安排。同时，在 PC/AT 机中不再采用 DMA 实现动态存储器刷新，故总线上的设备均可使用这 7 级 DMA 传送。除原 PC/XT 机总线上的 DMA 请求信号以外，其余的 DRQ_0、DRQ_5～DRQ_7 均定义在引脚号为 36 的插槽上，与此相对应的 DMA 控制器提供的响应信号 $\overline{DACK_0}$、$\overline{DACK_5}$～$\overline{DACK_7}$ 也定义在该插槽上。

$\overline{MASTER}$ 是增加的主控信号，利用该信号可以使总线插件板上设备变为总线主控器，用来控制总线上的各种操作。

在总线插件板上的 CPU 或 DMA 控制器可以将 DRQ 送往 DMA 通道。在接收到响应信号 $\overline{DACK}$ 后，总线上的主控器可以使 $\overline{MASTER}$ 成为低电平，并且在等待一个系统时钟周期后开始驱动地址和数据总线。在发出读/写命令之前，必须等待两个系统时钟周期。总线上的主控器占用总线的时间不要超过 15μs，以免影响动态存储器的刷新。

（6）读/写信号控制线：在 PC/AT 总线上定义了新的 $\overline{MEMR}$ 和 $\overline{MEMW}$，它们与前面 8 位 ISA 总线上的 $\overline{SMEMR}$ 和 $\overline{SMEMW}$ 不同的是 8 位 ISA 总线上的信号只有在存储器的寻址范围小于 1MB 时才有效，而新定义的信号在整个 16MB 范围内均有效。

- $\overline{MEMCS_{16}}$ 是存储器的 16 位片选信号。如果总线上的某一存储器卡要传送 16 位数据，则必须产生一个有效的低电平信号 $\overline{MEMCS_{16}}$，该信号加到系统板上，通知主板实现 16 位数据传送。此信号由 LA_{23}～LA_{17} 高位地址译码产生，利用三态门或集电极开路门进行驱动。
- $\overline{IOCS_{16}}$ 为接口的 16 位片选信号。它由接口地址译码信号产生，低电平有效，用来通知主板进行 16 位接口数据传送。

3. ISA 总线的体系结构

在利用 ISA 总线构成的微机系统中，当内存速度较快时，通常采用将内存移出 ISA 总线并转移到自己的专用总线——内存总线上的体系结构，如图 5-5 所示。

图 5-5 中微型计算机系统内部采用高速总线，DRAM 通过内存总线与 CPU 进行高速信息交换。中断通常由一对 8259A 可编程中断控制器处理，两个 8259A 采取级联方式连接，ISA 总线以扩展插槽形式对外开放，磁盘控制器、显示卡、声卡、打印机等接口卡均可插在 8MHz、8/16 位 ISA 总线插槽上，以实现 ISA 支持的各种外设与 CPU 的通信。

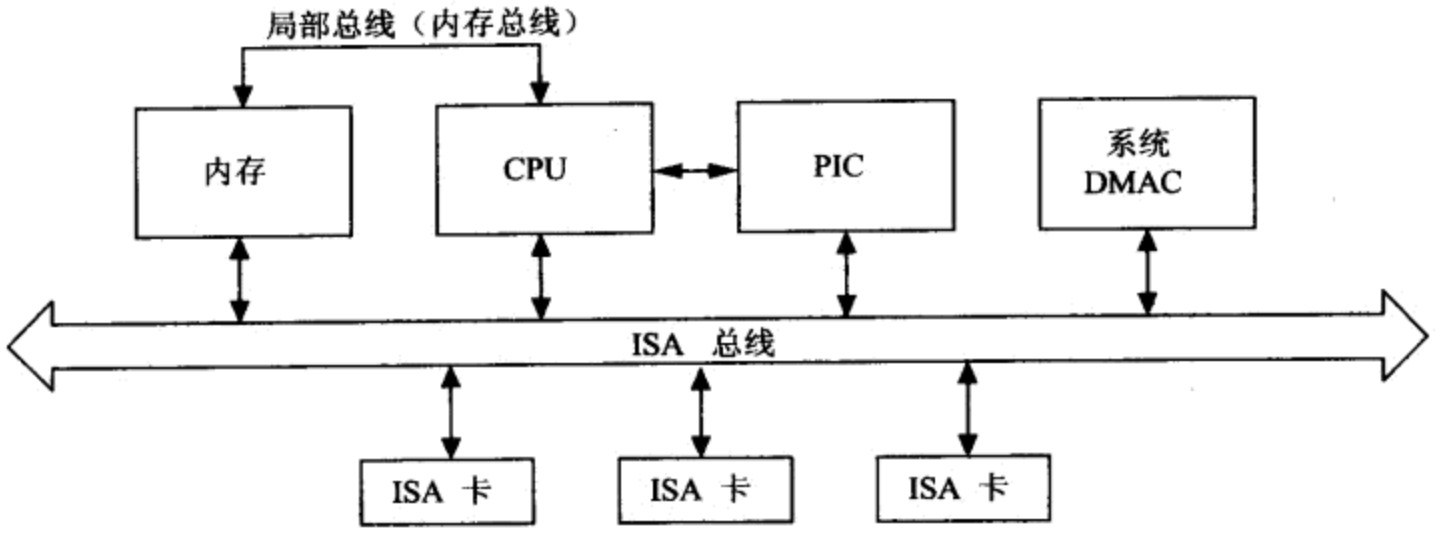


图 5-5 ISA 体系结构

5.2.3 EISA 总线

EISA（Extended Industry Standard Architecture，扩展的工业标准体系结构）总线是扩展的 ISA 总线。随着计算机速度的加快和数据总线的加宽，需要一种可以传送 32 位数据的总线。1989 年，以 Compaq 公司为代表的 9 家公司，在 ISA 总线的基础上，联合推出了 EISA 总线。在当时，尽管数据总线宽度被增加到 32 位，但 EISA 总线的时钟速度仍然维持在 8MHz，这导致了这种互联结构标准逐渐地被减少使用。但是，更新以后的 VESA 局部总线和 PCI 总线均工作在更高的速度之下，它们现在可以在 33MHz 的频率下工作。

1. EISA 总线的特点

EISA 总线具备以下特点。

（1）用于 32 位微型计算机中，支持 32 位的地址总线寻址，可寻址 4GB 的存储空间，也支持 64KB 的 I/O 端口寻址。

（2）具有 32 位数据线，大大提高了数据传输能力，保证了系统性能的提高，使最大数据传输速率达 33 MB/s。

（3）支持多处理器结构，支持多主控总线设备，具有较强的 I/O 扩展能力和负载能力。

（4）具有自动配置功能，可以根据配置文件自动地初始化，配置系统板和多扩展卡。

（5）扩展了 DMA 的范围和传输速度，支持 7 个 DMA 通道，DMA 数据传输既可在 ISA 方式下进行，也可在 EISA 方式下进行。

（6）采用同步数据传送协议，可支持常规的一次传送，也可支持突发方式即高速分组传送。

2. EISA 总线的主要信号及其功能

EISA 插槽与 ISA 插卡和 EISA 插卡均兼容，采用了双层结构，上面一层包含了 ISA 的全部信号，信号的排列、引脚间的距离、信号的定义规定与 ISA 完全一致；下层包含全部新增加的 EISA 信号。

EISA 总线的主要信号及其功能如下。

（1）地址锁存允许信号 BALE：输出，高电平有效，表示有效地址信号出现在 I/O 通道。

（2）总线时钟信号 BCLK：输出，系统总线时钟为 8.33MHz。

（3）字节允许信号 $\overline{BE_0} \sim \overline{BE_3}$：输出，低电平有效，确定 32 位数据总线传送时的字节。

（4）命令信号 $\overline{CMD}$：输出，低电平有效，表示总线控制器与时钟的重新同步。

（5）数据线 D_{16}～D_{31}：输入/输出，用来传送 32 位数据总线的高 16 位。

（6）数据线 D_{32}～D_{36}：输入/输出，用来传送 64 位数据总线的高 32 位。

（7）执行 16/32 位操作 $\overline{EX_{16}}$、$\overline{EX_{32}}$：输入，低电平有效，表示 16/32 位数据总线操作。

（8）准备就绪信号 EXRDY：输入，高电平有效，表示系统准备就绪。

（9）地址线 LA_2～LA_{16}，LA_{24}～LA_{31}：输出，用于快速 EISA 的总线周期操作。

（10）锁定信号 $\overline{LOCK}$：输出，低电平有效，封锁对存储器的访问。

（11）存储器 I/O 信号 M/$\overline{IO}$：输出，高电平表示访问存储器，低电平表示访问 I/O。

（12）主控设备确认信号 $\overline{MACK}$：输入/输出，低电平有效，表示超出总线使用时限。

（13）主控设备请求信号 $\overline{MREQ}$：输入/输出，低电平有效，表示请求对系统总线的控制。

（14）主控设备成组信号 $\overline{MSBURST}$：输入/输出，低电平有效，执行主控设备成组传送周期。

（15）从属设备成组信号 $\overline{SLBURST}$：输入，低电平有效，执行从属设备成组传送周期。

（16）开始信号 $\overline{START}$：输出，低电平有效，表明 EISA 总线上一个周期的开始。

（17）读/写信号 W/$\overline{R}$：输出，高电平表示执行写操作，低电平表示执行读操作。

EISA 总线最常见的应用是作为磁盘控制器和视频图形适配器。

5.3　局部总线

5.3.1　VESA 总线

VESA（Video Electronics Standards Association，视频电子标准协会）总线是一种 32 位接口的局部总线，通常称为 VL 总线。VESA 局部总线引脚定义如表 5-5 所示。

表 5-5　VESA 局部总线引脚定义

引脚号	焊接面	元件面	引脚号	焊接面	元件面
1	D_0	D_1	30	A_{17}	A_{16}
2	D_2	D_3	31	A_{15}	A_{14}
3	D_4	GND	32	+5V	A_{12}
4	D_6	D_5	33	A_{13}	A_{10}
5	D_8	D_7	34	A_{11}	A_8
6	GND	D_9	35	A_9	GND
7	D_{10}	D_{11}	36	A_7	A_6
8	D_{12}	D_{13}	37	A_5	A_4
9	+5V	D_{15}	38	GND	$\overline{WBACK}$
10	D_{14}	GND	39	A_3	BE_0
11	D_{16}	D_{17}	40	A_2	+5V
12	D_{18}	+5V	41	NC	$\overline{BE_1}$

续上表

引脚号	焊接面	元件面	引脚号	焊接面	元件面
13	D_{20}	D_{19}	42	$\overline{RESET}$	$\overline{BE_2}$
14	GND	D_{21}	43	$D/\overline{C}$	GND
15	D_{22}	D_{23}	44	$M/\overline{IO}$	$\overline{BE_3}$
16	D_{24}	D_{25}	45	$W/\overline{R}$	$\overline{AD_5}$
17	D_{26}	GND	46	—	—
18	D_{28}	D_{27}	47	—	—
19	D_{30}	D_{29}	48	$\overline{RDYRTM}$	$\overline{LRDY}$
20	+5V	D_{31}	49	GND	$\overline{LDEV}$
21	A_{31}	A_{30}	50	IRQ_9	LREQ
22	GND	A_{28}	51	$\overline{BRDY}$	GND
23	A_{29}	A_{26}	52	BLAST	$\overline{LGNT}$
24	A_{27}	GND	53	ID_0	+5V
25	A_{25}	A_{24}	54	ID_1	ID_2
26	A_{23}	A_{22}	55	GND	ID_3
27	A_{21}	+5V	56	LCLK	ID_4
28	A_{19}	A_{20}	57	+5V	$\overline{LKEN}$
29	GND	A_{18}	58	$\overline{LBS_{16}}$	$\overline{LEADS}$

VESA 局部总线工作频率可以达到 33MHz，常用在需要高速数据传输的系统，如用于视频和磁盘到基于 80486 的 PC 的接口等。VESA 局部总线接口卡如图 5-6 所示。与 EISA 总线一样，VESA 局部总线也是 ISA 总线的扩展，不同之处在于 VESA 局部总线没有在 16 位 ISA 总线连接器上增加任何器件，而是在 16 位 ISA 总线连接器的后面增加了第 3 个连接器，即 VESA 连接器。

VESA 局部总线上的连线与 EISA 总线卡非常相似，VESA 局部总线还包括一个 32 位地址和数据总线，用于将存储器和 I/O 设备连接到微处理器上。

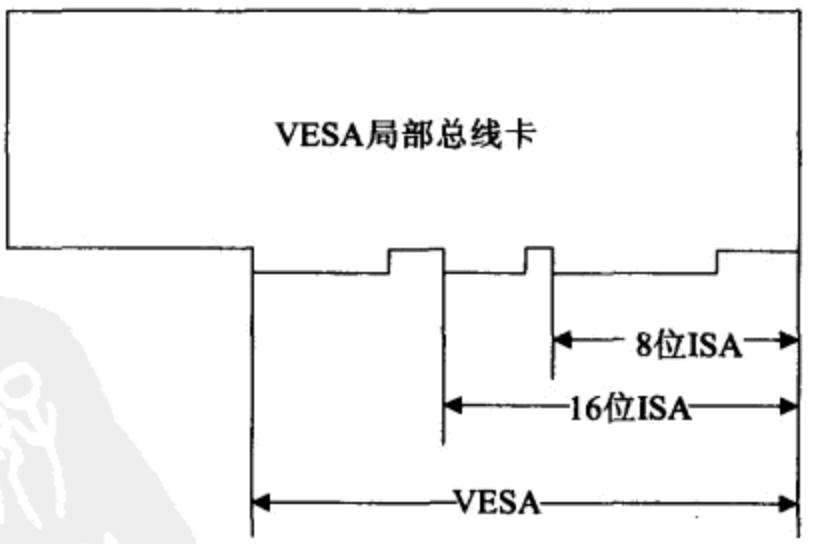


图 5-6 VESA 局部总线连接器和接口卡

5.3.2　PCI 总线

PCI（Peripheral Component Interconnect，外部组件互联）总线是目前最常用的系统总线。该总线是厂家自发制定的一种企业联盟标准，是专门为 Pentium 系列芯片设计的。采用面向 PCI 标准的芯片可以大大降低构造系统的成本，对计算机总线的发展起到了积极的推动作用。

PCI V2.0 版本支持 32/64 位数据总线，总线时钟为 25～33MHz，数据传输率达 132～264MB/s。1995 年推出的 PCI V2.1 版本支持 64 位数据总线，总线速度为 66MHz，最大数据传输率达 528MB/s。这个速度是最初的 IBM PC 总线的 100 倍，是最快的 ISA 总线的 40 倍。PCI 总线的优良性能使它成为当前 Pentium 系列芯片的最佳选择。

1. PCI 总线的特点

- 采用数据线和地址线复用结构，减少了总线引脚数，节约了线路空间，降低了设计成本。其目标设备可用 47 引脚，总线主控设备可采用 49 引脚。
- 提供两种工作信号环境，即 5V 和 3.3V，并可在两种环境中根据需要进行转换，扩大了它的适用范围。
- 对 32 位与 64 位总线的使用是透明的，它允许 32 位与 64 位器件相互协作。
- 允许 PCI 局部总线扩展卡和元件进行自动配置，提供了即插即用的能力。
- PCI 总线独立于处理器，它的工作频率与 CPU 时钟无关，可以支持多机系统。
- PCI 总线具有良好的兼容性，可支持 ISA、EISA、MCA、SCSI、IDE 等多种总线，同时还预留了发展空间。

2. PCI 总线信号的定义

PCI 总线规定了两种 PCI 扩展卡及连接器：一种称为长卡，另一种称为短卡。长卡提供 64 位接口，插槽 A、B 两边共定义了 188 个引脚；短卡提供 32 位接口，插槽 A、B 两边共定义了 124 个引脚。除去电源、地、未定义引脚之外，其余信号按功能分类列于图 5-7 中。

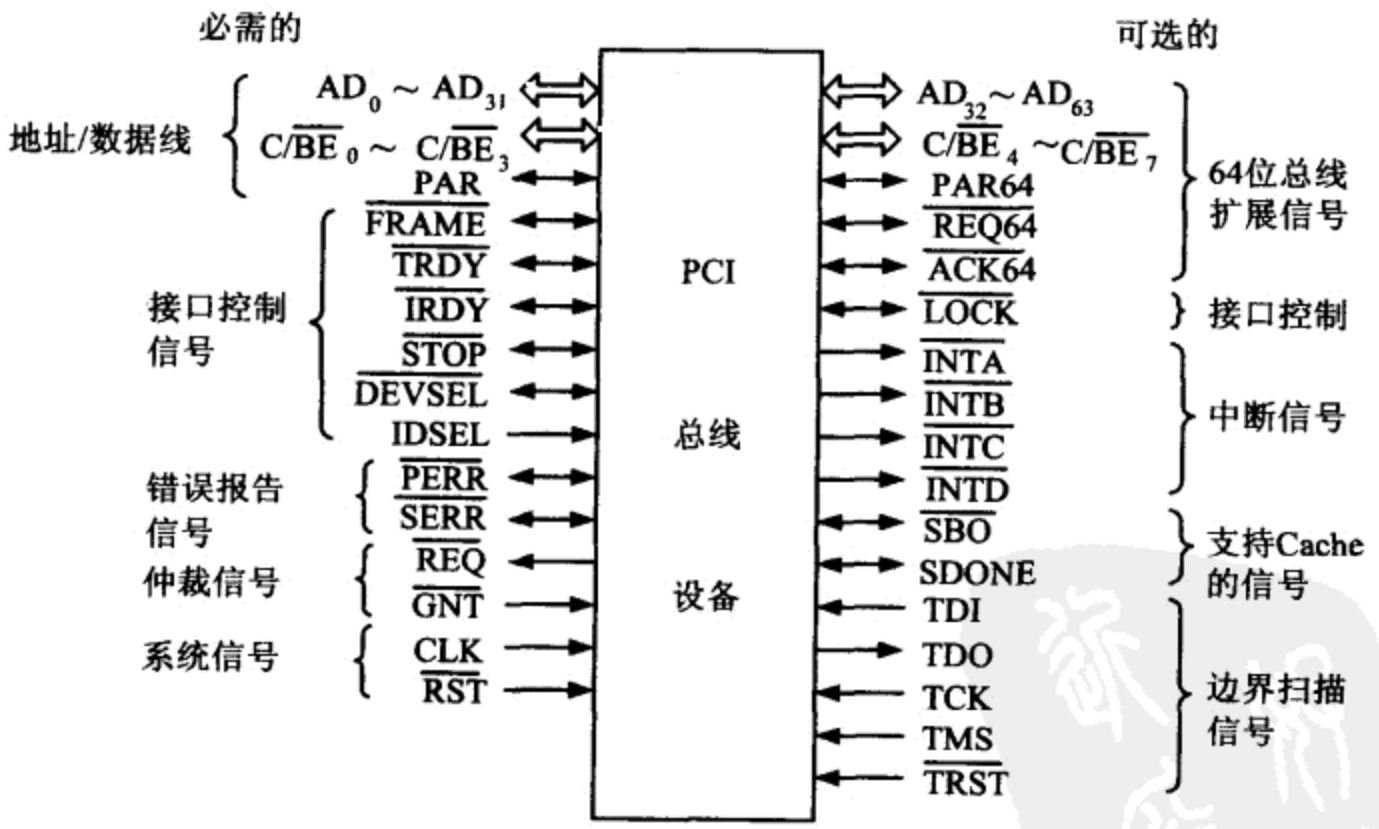


图 5-7　PCI 总线引脚信号

PCI 总线的引脚信号名称和功能解释如下。

- AD_0～AD_{63}：双向三态信号，为地址与数据多路复用信号线。
- $\overline{C/BE_0}$～$\overline{C/BE_7}$：双向三态信号，为总线命令和字节允许多路复用信号线。
- $\overline{FRAME}$：双向三态信号，低电平有效，为帧周期信号。由当前主设备驱动，表示一次访问的开始和持续时间。
- $\overline{IRDY}$：双向三态信号，低电平有效，为主设备准备好信号。该信号有效表示发起本次传输的设备能够完成一个数据周期。
- $\overline{TRDY}$：双向三态信号，低电平有效，为从设备准备好信号。该信号有效表示从设备已做好完成当前数据传输的准备。
- $\overline{STOP}$：双向三态信号，低电平有效，为停止数据传输信号。该信号有效表示从设备要求主设备中止当前的数据传送。
- $\overline{LOCK}$：双向三态信号，低电平有效，为锁定信号。该信号有效表示驱动它的设备所进行的操作可能需要多个传输才能完成。
- IDSEL：输入信号，为初始化设备选择信号。在参数配置读/写期间，用做片选信号。
- $\overline{DEVSEL}$：双向三态信号，低电平有效，为设备选择信号。该信号有效表示驱动它的设备已成为当前访问的从设备。
- $\overline{REQ}$：低电平有效的三态信号，为总线占用请求信号。该信号有效表示驱动它的设备要求使用总线。
- $\overline{GNT}$：低电平有效的三态信号，为总线占用允许信号。该信号有效表示要求使用总线的请求已被获准。
- $\overline{PERR}$：双向三态信号，低电平有效，为数据奇偶校验错误报告信号。
- $\overline{SERR}$：低电平有效的漏极开路信号，为系统错误报告信号。
- $\overline{INTA}$、$\overline{INTB}$、$\overline{INTC}$、$\overline{INTD}$：低电平有效的漏极开路信号，用来实现中断请求。其中，后 3 个只能用于多功能设备。
- $\overline{SBO}$：低电平有效的输入/输出信号，为试探返回信号。
- SDONE：高电平有效的输入/输出信号，为监听完成信号。
- $\overline{REQ64}$：双向三态信号，低电平有效，为 64 位传输请求信号。表示当前主设备要求采用 64 位数据传输。
- $\overline{ACK64}$：双向三态信号，低电平有效，为 64 位传输响应信号。表示从设备将按 64 位传输数据。
- PAR64：高电平有效的双向三态信号，为奇偶双字节校验信号。
- $\overline{RST}$：低电平有效的输入信号，为复位信号。
- CLK：输入信号，为系统时钟信号。

3. PCI 总线的系统结构

PCI 局部总线与 Pentium 机内部总线组合构成了多总线系统结构，典型的 PCI 系统如图 5-8 所示。

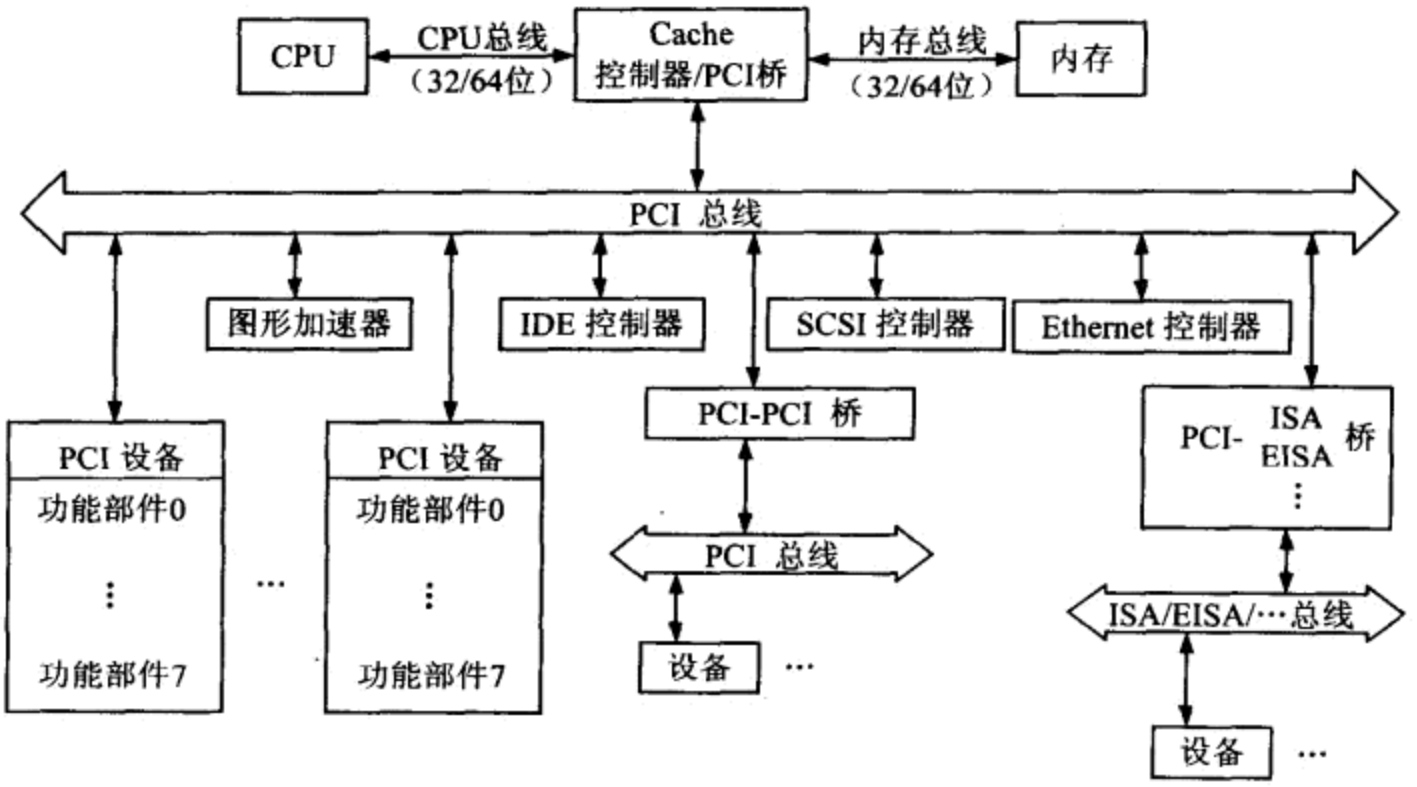


图 5-8　PCI 总线系统结构

PCI 总线允许在一个总线中插入 32 个物理部件，每一个物理部件可以含有最多 8 个不同的功能部件。除去用于生成广播消息的一个功能部件地址外，在一条 PCI 总线上最多可有 255 个可寻址功能部件。

PCI 系统中处理器与 RAM 位于主机总线上，具有 64 位数据通道和更宽及更高的运行速度。指令和数据在 CPU 和 RAM 之间快速流动，然后数据被交给 PCI 总线。PCI 负责将数据交给 PCI 扩展卡或设备。如果需要也可将数据导向 ISA、EISA、MCA 等总线或控制器。

驱动 PCI 总线的全部控制由 PCI 桥实现。PCI 桥实际是总线控制器，实现主机总线与 PCI 总线的适配偶合，它在与主机总线接口中引入了 FIFO 缓冲器，使 PCI 总线上的部件可与 CPU 并发工作。

PCI 桥的主要功能如下。

- 提供了一个低延迟的访问通路，从而使处理器能够直接访问通过它映射于存储器空间或 I/O 空间的 PCI 设备。
- 提供了能使 PCI 主设备直接访问主存储器的高速通路。
- 提供了数据缓冲功能，可以使 CPU 与 PCI 总线上的设备并行工作而不必相互等待。
- 可以使 PCI 总线的操作与 CPU 总线分开，以免相互影响，实现了 PCI 总线的全部驱动控制。

扩展总线桥电路的设置目的是为了能在 PCI 总线上接出一条标准 I/O 扩展总线，如 ISA、EISA、MCA 总线等，从而可以继续使用现有的 I/O 设备，以增加 PCI 总线的兼容性和选择范围。PCI 桥可以利用许多厂家开发的 PCI 芯片组实现。通过选择适当的 PCI 桥构成所需的系统，是构成 PCI 系统的一条捷径。

5.3.3　AGP 总线

为了解决高速视频或高品质画面的显示，Intel 公司在 1997 年推出了高速图形接口的局

部总线标准——AGP 总线。AGP 总线是对 PCI 总线的扩展和增强，但 AGP 接口只能为图形设备独占，不具有一般总线的共享特性。采用 AGP 接口允许显示数据直接取自系统主存储器，而无需先预取至视频存储器中。

AGP 总线的主要特点如下。

（1）具有双重驱动技术，允许在一个总线周期内传输两次数据。

（2）在总线上可实现地址/数据多路复用，把 32 位的数据总线给图形加速器使用。

（3）通过内存请求流水线技术对各种内存请求进行排队来减少延迟，一个典型的排队可处理 12 个以上的请求，大大加快了数据传输的速度。

（4）把图形接口绕行到 AGP 通道上，解决了 PCI 带宽问题，使 PCI 有更多的能力负责其他数据传输。

5.4 其他总线介绍

5.4.1 USB 总线

1. USB 总线的特点

USB（Universal Serial Bus）的全称是通用串行总线，它是一种支持即插即用的新型串行接口。USB 要比标准串行接口快得多，其数据传输率可达 4Mb/s～12Mb/s。USB 除了具有较高的数据传输率外，它还可以为外设提供支持。

USB 总线具有以下特点。

（1）为所有的 USB 外设提供了单一的、易于操作的连接类型，简化了用户判断哪个插头对应哪个插槽的任务。

（2）排除了对鼠标、调制解调器、键盘和打印机不同接口的需求，采用四线电缆，两根作为数据传输线，其余两根用来为设备提供电源，从而减少了硬件设计的复杂性。

（3）支持热插拔，即在不关机的情况下可安全地插上和断开 USB 设备。热插拔能力体现了 USB 的安全、可靠和智能。

（4）支持 PNP（Plag and Play）即插即用。当插入 USB 设备的时候，主计算机设备检测该外设并通过加载相关的驱动程序对该设备进行配置。

（5）在设备供电方面提供了灵活性。USB 直接连接的设备可以通过 USB 电缆供电，USB 传输线中的两条电源线可以提供 5V 电源供 USB 设备使用。

（6）USB 传输线能够提供 100mA 的电流，而带电源的 USB HUB 使得每个接口可以提供 500 mA 的电流。

（7）USB V1.1 规范提供全速 12Mb/s 的模式和低速 1.5Mb/s 的模式，USB V2.0 规范提供高达 480Mb/s 的数据传输速率，可以适应各种不同类型的外设。

（8）针对突然发生的非连续传输设备，如音频和视频设备，USB 在满足带宽的情况下才进行该类型的数据传输。

（9）为了适应各种不同类型外围设备的要求，USB 提供了 4 种不同的数据传输类型。

（10）USB 使得多个外围设备可以跟主机通信，最多支持 127 个设备。由于计算机的

USB 接口有限，必须使用 USB HUB 增加分支，根据规范 USB HUB 最多可提供 7 个分支。

2. 数据传输类型

根据 USB 设备自身的使用特点和系统资源的不同要求，在 USB 规范中规定了 4 种不同的数据传输方式。

（1）控制（Control）传输方式：双向传输，传输的不是数据而是控制信号，主要被 USB 系统软件用来进行查询、配置和给 USB 设备发送通用的命令。该方式用在主计算机和 USB 外设之间的端点（End Point）之间的传输，数据量较小且实效性要求不高。

（2）同步（Isochronous）传输方式：提供确定的带宽和时间间隔。用来连接需要连续传输的外围设备，对数据的正确性要求不高，但对时间较为敏感。例如，对执行即时通话的网络电话，使用同步传输方式是很好的选择。

（3）中断（Interrupt）传输方式：用于定时查询设备是否有中断数据要传输。典型应用在少量、分散、不可预测数据的传输方式中，键盘、操作杆和鼠标就属于这一类型。在 USB v1.0 规范中中断方式传输是单向的，并且对于主机来说只有输入方式，但在 USB v2.0 规范中，既有输入方式又有输出方式。

（4）批量（Bulk）传输方式：应用在大量传输和接收数据上，同时没有带宽和时间间隔的要求，保证传输数据正确无误，但对数据的实效性要求不高。适合于传输非常慢和大量被延迟的数据，在传输中的优先级很低，打印机、数码相机和扫描仪就属于这种类型。

3. USB 总线的拓扑结构

USB 设备和 USB 主机通过 USB 总线相连。USB 的物理连接是一个星型结构，HUB（集线器）位于每个星形结构的中心，每一段都是主机和某个集成器，或某一功能设备之间的一个点到点的连接，也可以是一个集线器与另一个集线器或功能模块之间的点到点的连接。

总线拓扑结构如图 5-9 所示。

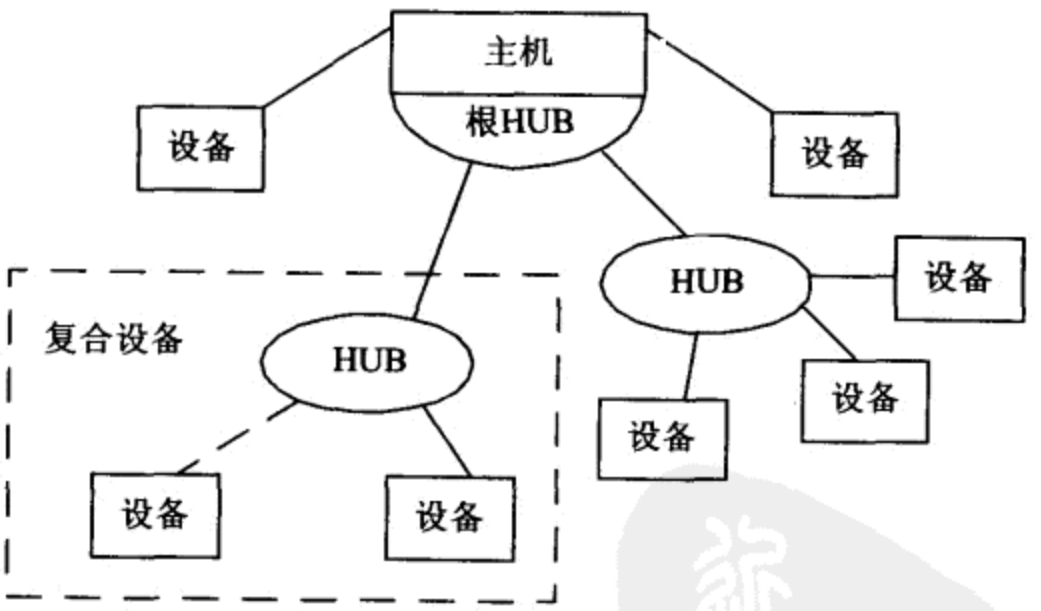


图 5-9　USB 的拓扑结构

（1）USB 主机

整个 USB 系统中只允许有一个主机。主机系统的 USB 接口称为 USB 主控制器。这里 USB 主控制器可以是硬件、固件或软件的联合体。而根集线器是集成在主机系统中的，它可以提供一个或更多的接入端口。

（2）USB 设备

USB 设备是 USB 协议的具体实现，主要包括集线器和功能部件。

- 集线器：提供用以访问 USB 总线的更多的接入点。
- 功能部件：向系统提供特定的功能，如 ISDN 连接设备、鼠标、显示器等。

4. USB 系统的构成

USB 规范将 USB 分为 5 个部分，即：控制器、控制器驱动程序、USB 芯片驱动程序、USB 设备及针对不同 USB 设备的驱动程序。各部分的主要功能如下。

（1）控制器：负责执行由控制器驱动程序发出的命令。

（2）控制器驱动程序：在控制器与 USB 设备之间建立通信信道。

（3）USB 芯片驱动程序：提供对 USB 的支持。

（4）USB 设备：包括与 PC 相连的 USB HUB 及设备。HUB 带有连接其他外围设备的 USB 端口，设备是连接在计算机上用来完成特定功能并符合 USB 规范的具体设备，如鼠标和键盘等。

（5）USB 设备驱动程序：是用来驱动 USB 设备的程序。通常由操作系统或 USB 设备制造商提供。

5. USB 总线的电气特性和机械特性

（1）电气特性

USB 总线通过一条四芯电缆传送电源和数据，电缆以点到点方式在设备之间连接。USB 接口的 4 条连接线分别是 V_{BUS}、GND、D_+和 D_-。

V_{BUS} 和 GND 用来向设备提供电源。在源端，V_{BUS} 通常为+5V。USB 主机和 USB 设备中通常包含电源管理部件。

D_+和 D_-是发送和接收数据的半双工差分信号线，时钟信号也被编码在这对数据线中传输。每个分组中都包含同步字段，以便接收端能够同步于比特时钟。

（2）机械特性

USB 连接器分为 A 系列和 B 系列两种：A 系列用于和主机连接，B 系列用于和 USB 设备的连接。这两种连接器有不同的结构，不会造成误连接。

USB 连接器的排列如表 5-6 所示。

表 5-6 USB 连接器引脚排列

端 口 号	信 号	典型电缆颜色	端 口 号	信 号	典型电缆颜色
1	V_{BUS}	红色	4	GND	黑色
2	D_-	白色	外皮	屏蔽	管线
3	D_+	绿色			

6. 总线协议

USB 总线由主机控制器控制所有的数据传输。大多数传输包含 3 个 USB 分组。

（1）主机控制器首先发出一个“令牌分组”，指明传输的类型和方向、USB 设备的地址及终点编号。USB 设备对相应的地址字段进行译码，选中被寻址的设备。

（2）如果本次传输的源端能够提供数据，那么它将发出数据分组；否则，它将发出一

个指示分组，指明它没有数据可以传输。

（3）一般情况下，目的端将回送一个握手分组指明本次传输是否成功。

在主机和设备之间的数据传输关系被称为管道（Pipe），每个管道有一组相应的数据带宽、传输服务类型及设备特性等参数。每台 USB 设备可以有多个管道，各管道中的数据传输相互独立。USB 包含流（Stream）和消息（Message）两种管道，前者是没有格式的，而后者按照 USB 定义的数据格式传输。

7. USB 设备的接入和应用

（1）操作系统对 USB 的支持

支持 USB 的操作系统应满足以下 3 个要求。

- 当一个设备连接到 USB 上或从 USB 中撤除，能自动检测出来。
- 与新连接的设备通信，可找到如何与它们通信的方法。
- 提供软件驱动与计算机的 USB 硬件以及访问 USB 的外设的应用程序通信。

（2）主机对 USB 的支持

使用 USB 设备必须激活主机板 BIOS 中的 USB 功能，以 Windows XP 操作系统为例，在 Windows XP 下先打开“设备管理器”，具体操作为：在桌面“我的电脑”图标上单击鼠标右键，在弹出的菜单中选择“属性”命令，在“系统属性”对话框的硬件选项卡中单击“设备管理器”按钮，在弹出的窗口中选择“通用串行总线控制器”选项，可看到主机系统中存在两类 USB 设备：一类是“USB Universal Host Controller”，另一类是“USB Root Hub”。

（3）USB 设备的热插拔

USB 总线协议支持热插拔功能，在 Windows 运行过程中可以接入任何符合 USB 规范的 USB 设备。当接入一个 USB 设备后，操作系统会自动检测到该硬件设备，Windows 会显示以下信息：“New Hardware Found”或“Unknow Device”。如果设备首次接入这个系统，则 Windows 还需要定位驱动。

（4）USB 设备的应用

USB 已经在 PC 的多种外设上得到应用，包括扫描仪、数码相机、数码摄像机、音频系统、显示器、输入设备等。

使用 USB 扫描仪时，用户只需放好要扫描的图文，按一下扫描仪的按钮，屏幕上会自动弹出扫描仪驱动软件和图像处理软件，并实时监视扫描的过程。USB 数码相机、摄像机利用 USB 的高速数据传输能力，使大容量的图像文件传输在短时间内即可完成。

对于便携式微型计算机来说，使用 USB 接口的意义更加重大，通用的 USB 接口不仅使便携式微型计算机对外的连接变得方便，还可以使生产厂商不再需要为不同配件在主板上安置不同的接口，促进更高主频的处理器可以迅速应用在移动计算机中。

5.4.2　IEEE1394

IEEEl394 是一种高性能的串行总线。应用范围主要是那些带宽要求超过 100kb/s 的硬盘和视频外设。利用同样的 4 条信号线，IEEE1394 既可以同步传输，也可以支持异步传输。这 4 根信号线分为差模时钟信号线对和差模数据线对。IEEE1394 规范得到了很好的定义，而且基于IEEE规范的产品也出现在了市场上。目前，IEEE1394解决方案的价位被认为可以同SCSI

磁盘接口相竞争，但它不适用于一般的桌面连接。

1. IEEE1394 的特点及拓扑结构

IEEEl394 是一种新型的高速串行总线，它具有以下显著的特点。

（1）高传输速率：IEEEl394 的传输速率可达到 100Mb/s、200Mb/s、400Mb/s，能很好地满足实时图像数据传输。

（2）采用同步传输和异步传输两种数据传输模式：同步传输模式具有固定的带宽、比特间隔及起始时间，数据传输是在通信双方事先建立好的专有带宽上进行，该方式很适合传送语音及视频信号，可出色地完成对外设进行实时高速数据采集的任务；异步传输是在总线处于空闲时才得以实施，接收方通过向发送方返回确认应答包来保证数据传输的可靠性。

（3）可以实现即插即用并支持热插拔：用户可通过菊花链、树型、星型及聚合等拓扑结构灵活连接各类设备。热插拔技术极大地简化了主机与外设的连接与初始化操作，免去了 I/O 卡跳线等诸多硬件设置，只需接好连线，各设备节点自动进行总线初始化及识别，之后就可进行高速数据采集和设备测试，取消连接也同样方便。Microsoft 从 Windows 2000 开始已经嵌入了对 IEEE1394 的操作系统级的支持，从而确保了 IEEE1394 各项出类拔萃的特性在桌面系统上的可靠运用。

上述特点使它可广泛地应用于多媒体声卡、图像和视频产品、打印机、扫描仪的图像处理等方面，尤其是磁盘阵列、数字照相机，显示器和数字录像机等。

IEEE1394 系统结构如图 5-10 所示。

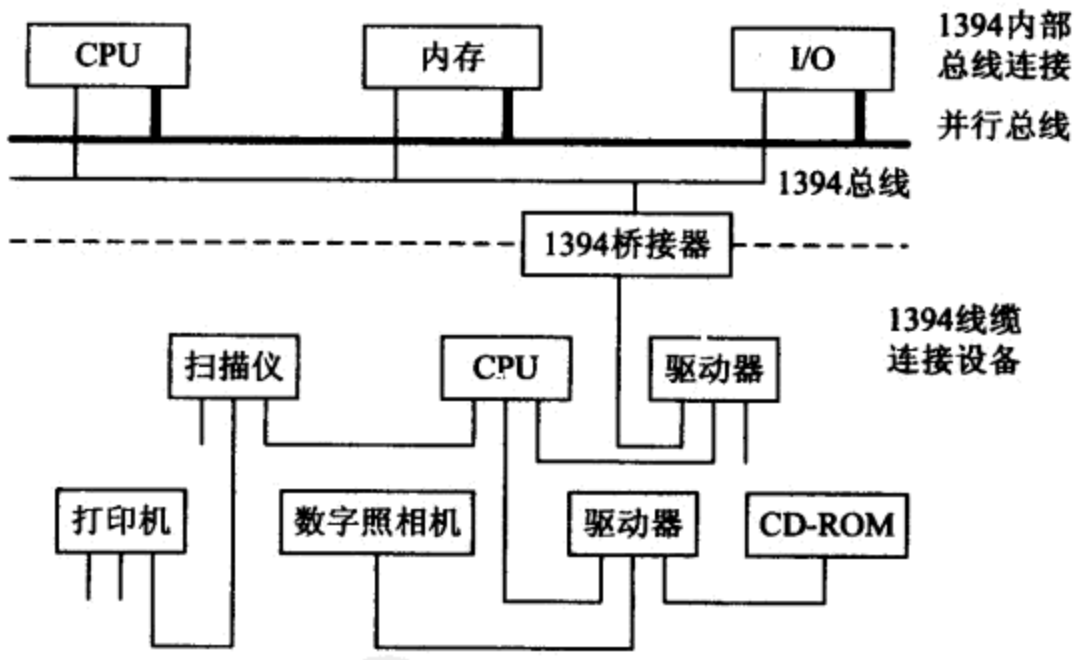


图 5-10　IEEE1394 系统结构

2. IEEE1394 的寻址

用 IEEE 1394 连接起来的设备采用一种内存编址方法，各设备就像内存中的存储单元一样。设备地址 64 位宽，占用 10 位作为网络 ID 号，6 位用做节点号，48 位用做内部编址。可得到总共 64 个节点，每个节点上有 1 023 个网络 ID 号，每个 ID 号又具有 231TM 的内存编址。

内存编址显然优于通道编址，它可以把设备资源当作寄存器或内存，因而可以进行处理器到内存的直接传输。每一个总线段称做一个节点，可对节点分别编址、复位和校验，许多

节点在物理上形成一个模块，多个端口又可以集中在一个节点上。

IEEE1394 标准是一种基于数据传输包的协议标准，它既可以用于内部总线传输，又可以用于设备间的线缆连接。计算机的基本功能单元（如 CPU，RAM 等）和外围设备都可以用它来并行连接。

3. IEEE1394 协议

IEEE1394 协议是一种基于数据包的数据传输协议，该协议中实现了 OSI 七层协议的三层：物理层、链路层和传输层。串行总线的管理层将三个层次连接起来，如图 5-11 所示。各层次的功能如下。

（1）传输层：对异步传输协议的读/写和锁定提供支持，写命令从发送端读出数据到接收端，读命令则向发送端返回数据，锁定命令综合了写和读的功能，它在发送和接收端间建立一条通道，并完成接收端应完成的动作。

（2）链路层：为两种类型的包数据提供包传送功能：异步传输和等时传输。异步传输是一种传统的传输方式，而等时传输则按预定的速率提供稳定的数据通道，这对时间要求严格的多媒体数据的及时传送非常重要。

（3）物理层：将链路层的逻辑信号根据不同的串行总线介质转换成相应的电信号，同时用来确保一次只有一个节点可发送数据。物理层也为串行总线定义了机械接口特性，实际上，物理层在两种环境下有所不同：其一是指电缆环境下的物理层，其二是指底板环境下的物理层。

串行总线管理单元按时间仲裁最优化的形式实现对串行总线的配置，保证总线上所有的设备供电充足，确定循环传送的主设备，赋予等时传送时设备的 ID 通道号及简单的错误信息。

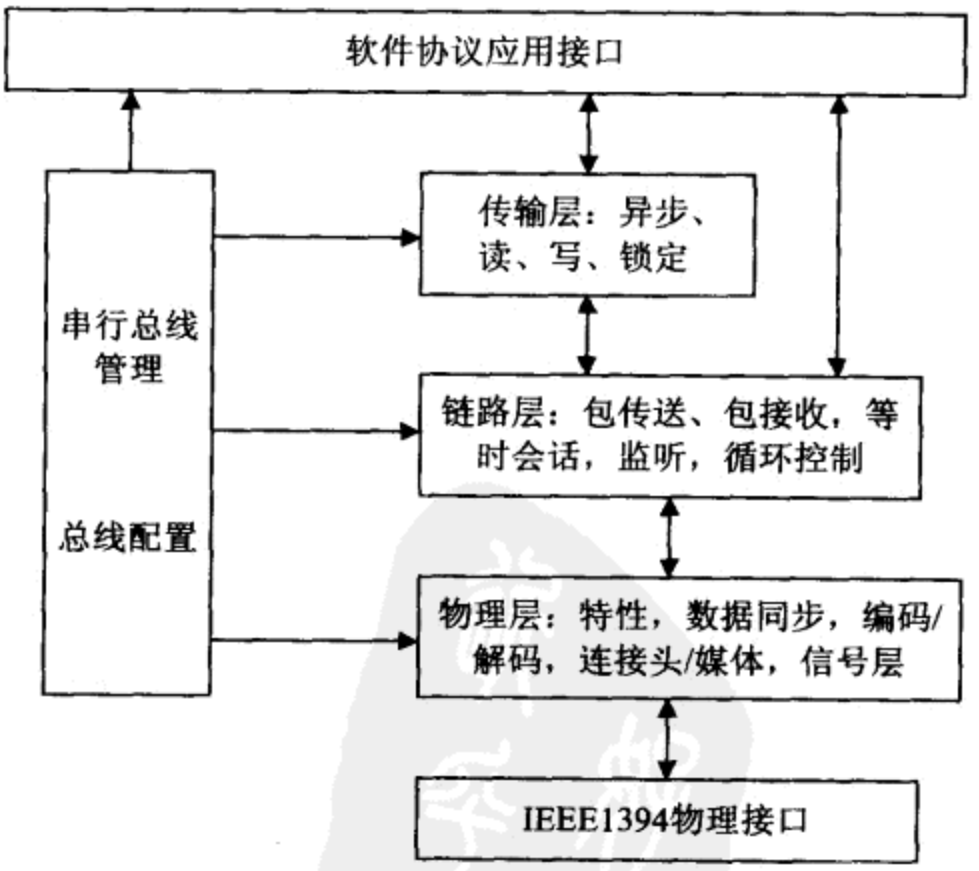


图 5-11　IEEE1394 串行总线协议图

5.4.3 I^2C 总线

1. I^2C 总线简介

I^2C 总线（Inter IC Bus）是由 Philips 公司推出的一种芯片间的串行通信总线，广泛应用于单片机系统中。

在单片机应用系统中推广 I^2C 总线后大大改变了单片机应用系统结构性能，对单片机的应用开发带来如下好处。

（1）可最大限度地简化结构。二线制的 I^2C 串行总线使得各电路单元之间只需最简单的连接，而且总线接口都已集成在器件中，不需另加总线接口电路。电路的简化省去了电路板上的大量走线，减少电路板面积，提高可靠性，降低成本。

（2）可实现电路系统的模块化、标准化设计。在 I^2C 总线上各单元电路除了个别中断引线外，相互之间没有其他连线，用户常用的单元电路基本上与系统电路无关，故极易形成用户自己的标准化、模块化设计。

（3）标准 I^2C 总线模块的组合开发方式大大缩短了新品种的开发周期。

（4）I^2C 总线各节点具有独立的电气特性，各节点单元电路能在相互不受影响及系统供电情况下接入或撤除。

（5）I^2C 总线系统的构成具有极大的灵活性。系统改型设计，或对已加工好的电路板需扩展功能时，对原有设计及电路板系统影响最小。

（6）I^2C 总线系统可方便地对某一节点电路进行故障诊断与跟踪，有极好的可维护性。

目前 Philips 及 I^2C 总线器件，除带有 I^2C 总线的单片机、常用的通用外围器件外，在家电产品、电信、电视、音像产品中已发展了成套 I^2C 总线器件，在这些部门中 I^2C 总线系统已得到了广泛的应用。

2. I^2C 总线特点

I^2C 总线的串行数据传送与一般 UART 的串行数据传送无论从接口电气特性、传送状态管理及程序编制特点都有很大不同，I^2C 总线主要具有以下特性。

（1）二线传输。I^2C 总线上所有的节点，如主器件（单片机、微处理器）、外围器件、接口模块等都连到同名端的双向数据线 SDA 和时钟线 SCL 上。

（2）系统中有多个主器件时，这些器件都可做总线的主控制器（无中心主机），I^2C 总线工作时任何一个主器件都可成为主控制器，多机竞争时的时钟同步与总线仲裁都由硬件与标准软件模块自动完成，无需用户介入。

（3）I^2C 总线传输时，采用状态码的管理方法。对应于总线数据传输时的任何一种状态，在状态寄存器中会出现相应的状态码，并且会自动进入相应的状态处理程序中进行自动处理，无须用户介入，用户只需将 Philips 公司提供的标准状态处理程序装入程序存储器一定的空间即可。

（4）系统中所有外围器件及模块，采用器件地址和引脚地址的编址方法。系统中主控制器对任何节点的寻址采用纯软件寻址方法，避免了片选线的线连接方法。系统中若有地址编码冲突，可通过改变地址引脚的电平设置来解决。

（5）所有带 I^2C 接口的外围器件都具有应答功能。片内有多个单元地址时，数据读、写

时都有地址自动加 1 功能。这样，在 I^2C 总线对某一器件读/写多个字节时很容易实现自动操作，即准备好读、写入口条件后，只需启动 I^2C 总线就可自动完成 N 个字节的读、写操作。

（6）I^2C 总线电气接口为开漏晶体管组成，开路输出没有连到电源的钳位二极管，而连到 I^2C 总线的每个器件上，其自身的电源可以独立成线上各个节点，可在系统带电情况下接入或撤出。

3. I^2C 总线原理

（1）数据传输方式

I^2C 总线上的器件之间通过串行数据线 SDA 和串行时钟线 SCL 相连接并传送信息。每一个器件由唯一的地址连接到总线上，可以根据地址来识别器件。发送器和接收器在进行数据传送时可以作为主器件，也可以作为从器件。主器件是用于启动总线上传送数据并产生时钟以开放传送的器件，此时，任何被寻址的器件均被认为是从器件。总线上主和从、发送和接收的关系不是永久的，仅取决于此时数据传送的方向。I^2C 总线上的控制完全由竞争的主器件送出的地址和数据决定，在总线上，既没有中心机也没有优先机。

送到 SDA 线上的每个字节必须为 8 位，每次传送的字节数不限，但每个字节后面必须跟一个响应位。标准模式下总线速率为 100kb/s，快速模式下总线速率为 400kb/s，高速模式下的总线速率可达到 3.4Mb/s。数据传送时先传最高位，如果接收器件不能接收下一个字节（如正在处理一个内部中断，在这个中断处理完成之前不能接收 I^2C 总线上的数据字节），则可以使时钟保持低电平，迫使主器件处于中断等待状态。当从器件准备好接收下一个数据字节时，则释放 SCL 线后继续传送。数据传送过程中确认数据是必须的，认可位对应于主器件的一个时钟，在此时钟内发送器件释放 SDA 线，而接收器件必须将 SDA 线拉成低电平，使 SDA 在该时钟的高电平期间为稳定的低电平。通常被寻址的接收器件必须在收到每个字节后做出响应，若从器件正在处理一个实时事件不能接收而不对地址认可时，从器件必须使 SDA 保持高电平，此时，主器件产生一个结束信号使传送异常结束。

图 5-12 说明了一个完整的数据在 I^2C 总线上的传送过程。

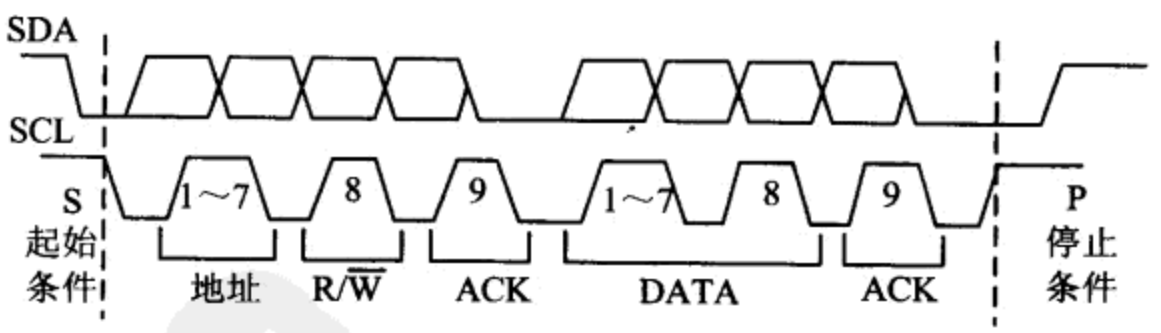


图 5-12　I^2C 总线上数据传送示意图

（2）I^2C 总线的寻址约定

为了消除 I^2C 总线系统中主控器与被控器的地址选择线，最大限度地简化总线连接线，I^2C 总线采用了独特的寻址约定，规定了起始信号后的第一个字节为寻址字节，用来寻址被控器件，并规定数据的传送方向。

在 I^2C 总线标准约定中，寻址字节由被控器的 7 位地址位（它占据了 D_7～D_1 位）和 1 位方向位（为 D_0 位）组成。方向位为 0 时表示主控器将数据写入被控器，为 1 时则表示主控器

从被控器读取数据。

主控器发送起始信号后，立即发送寻址字节，这时，总线上的所有器件都将寻址字节中的 7 位地址与自己器件地址相比较。如果两者相同，则该器件认为被主控器寻址，并根据读、写位确定是被控发送器还是被控接收器。

I^2C 总线系统中，主器件（单片机）作为被控器时，其 7 位从地址在 I^2C 总线地址寄存器中约定为纯软件地址。而非单片机类型的外围器件地址完全由器件类型与引脚电平给定，即器件的 7 位地址由器件编号地址（高 4 位 D_6～D_3）和引脚地址（低 3 位 D_2～D_0）组成，如 8 位 I/O 扩展器件 PCF8574 其器件编号地址为 0111，引脚地址为 A_2、A_1、A_0。如果在 I^2C 总线系统中 NF8574 地址引脚 A_2、A_1、A_0 皆接地，则该器件的寻址字节为 SLAW=70H，SLAR=7lH。I^2C 总线上同一编号地址器件最大允许接入数量取决于可利用的地址引脚数。

在 I^2C 总线系统中，带有 I^2C 总线的单片机，其 I^2C 总线输入/输出口的电气结构、相关的特殊功能寄存器（SFR）设置及所提供的标准程序模块，为用户掌握 PC 总线的系统设计和应用软件的编制带来极大方便。

本章小结

微型计算机系统采用总线结构，总线是系统的重要组成部分，它传递着 CPU 和其他部件之间的各类信息，以实现数据传输，使微型计算机系统具有组态灵活、易于扩展等优点。总线性能的好坏直接影响到微型计算机系统的整体工作性能。目前，应用广泛的微型计算机总线都实现了标准化，便于在连接各个部件时遵守共同的总线规范。在应用时只需根据总线标准的要求来实现和完成接口的功能，形成了一种通用的总线接口技术。

微型计算机系统中的总线可以分为芯片总线、系统总线、局部总线、外部设备总线等类别。芯片总线常用于 CPU、存储器、I/O 接口等芯片之间的信息传送，有地址总线、数据总线和控制总线。系统总线是微型计算机系统内连接各插件板的总线，用于模板之间的连接，主要有 8/16 位 ISA 和 32 位的 EISA 总线。局部总线是一种专门提供给高速 I/O 设备的总线，它具有较高的时钟频率和传输率，常用的有 VESA、PCI、AGP 总线等。外部设备总线主要用于微机系统之间或微机与外部设备、仪器仪表之间的通信，这种总线的数据传输可以是并行的，也可以是串行的，数据传输速率低于系统内部的总线。

本章分析了常用的标准总线，阐述了各类总线的特点和功能。在学习过程中，要理解总线的基本概念，熟悉微型计算机总线的组成结构，注意常用系统总线和局部总线的内部结构及引脚特性，在各种不同的应用场合中合理地选择和使用总线，为微型计算机的开发和应用奠定坚实的基础。

思考与练习题

1. 什么叫总线？总线如何进行分类？各类总线的特点和应用场合是什么？
2. 在微型机系统中采用标准总线的好处有哪些？
3. STD 总线的主要特点是什么，该总线如何实现总线程优先级控制？

4. 分析 IBM PC AT 总线与 XT 总线的特点，两者有哪些不同？
5. PCI 总线有哪些主要特点，PCI 总线结构与 ISA 总线结构有什么地方不同？
6. EISA 总线与 ISA 总线的主要区别是什么？
7. VESA 局部总线与 EISA 总线有何区别？
8. 简述 PCI 总线的系统结构，分析其引脚功能和特点。
9. 什么是 AGP 总线？它有哪些主要特点，应用在什么场合？
10. USB 接口有什么特点？USB 的数据传送有哪几种方式？
11. IEEE1394 号 USB 两种串行总线各有什么区别？
12. 简述 USB 总线的特点和应用场合。
13. 简述 I^2C 总线的特点和工作原理。
14. 讨论在开发和使用微型计算机系统时应怎样合理地选择总线，需要注意的地方有哪些。

第 6 章 存储器系统

本章主要介绍半导体存储器的基础知识，包括微机系统中常用存储器的分类、性能指标及层次结构；半导体存储器的基本结构、工作原理以及与 CPU 的连接等内容；最后介绍了微机系统中的辅助存储器和新型存储器技术。

通过本章的学习，读者应该熟悉存储器的分类和层次结构，掌握常用的 RAM 和 ROM 基本结构、原理和特点；灵活运用存储器与 CPU 进行连接和扩展；了解新型存储器技术的特性，并侧重其应用。

6.1 存储器概述

存储器是计算机中用来存储信息的部件，它能把计算机要执行的程序、数据处理的中间结果和最终结果存储在计算机中，使计算机能自动连续地工作。因此，存储器是微机系统不可缺少的组成部分，是计算机中各种信息的存储和交流中心。

能够存储一位二进制信息的最小物理基体叫做存储基元（Cell），若干个存储基元（一般为 8 个）可组成一个存储单元，许多存储单元可构成一个存储体（或称存储矩阵），存储体与存储器控制电路相配合就可构成存储器。

在运行程序时，CPU 自动连续地从存储器中取出指令并执行指令规定的操作，计算机每完成一条指令，至少要执行一次访问存储器的操作，并把处理结果存储在存储器中。

6.1.1 存储器的分类

1. 按存储介质分类

存储介质是指存储二进制信息的物理载体，这种载体具有表现两种相反物理状态的能力。目前使用的存储介质主要有半导体器件、磁性材料和光学材料。

（1）用半导体器件做成的存储器称为半导体存储器。从制造工艺的角度又把半导体存储器分为双极型和 MOS 型等。

（2）用磁性材料做成的存储器称为磁表面存储器，如磁盘存储器和磁带存储器。

（3）用光学材料做成的存储器称为光表面存储器，如光盘存储器。

2. 按读/写功能分类

存储器按读/写功能分为只读存储器和随机存取存储器。

（1）只读存储器（Read Only Memory，ROM）：ROM 所存储的内容是固定不变的，即只能读出不能写入，一般用来存放微机的系统程序，如微机的系统管理程序、监控程序等。

（2）随机存取存储器（Random Access Memory，RAM）：RAM 是既能读出又能写入的存储器，主要用来存放各种输入、输出数据及中间结果，并可与外存储器交换信息和做堆栈用。

3. 按在微机系统中的作用分类

根据存储器在微机系统中所起的作用，存储器可分为主存储器（又称内存储器，简称内

存）、辅助存储器（又称外存储器，简称外存）和高速缓冲存储器（Cache）等。

（1）主存储器在主机内部，用来存放当前正在运行的程序和数据。主存储器与 CPU 及各种接口电路一起构成微机主机，CPU 通过指令可以直接访问主存储器。其特点是速度较快，但容量较小，价格也较高。

（2）辅助存储器属于计算机的外部设备，通常用来存储 CPU 当前操作暂时用不到的程序或数据。其特点是存储容量极大，价格便宜，它所存储的大量信息在断电后也不会丢失，但速度较慢。辅助存储器主要有磁带、磁盘（硬盘和软盘）和光盘等。现代微机通常使用软盘、硬盘和光盘作为辅助存储器，其中大容量硬盘（40GB 以上）和 CD-ROM（只读型光盘）更是现代微机必不可少的外部存储器，用来存放系统程序和大型数据文件及数据库。

（3）高速缓冲存储器是计算机系统中的一个高速小容量存储器，位于 CPU 和内存之间。在现代微机中，为了提高计算机的处理速度，利用高速缓存来暂存 CPU 正在使用的指令和数据，这样可以加快信息传递的速度。目前，高速缓存主要由高速静态 RAM 组成。

6.1.2　存储器的性能指标

1. 存储容量

存储器可以存储的二进制信息总量称为存储容量。它是存储器的一个重要指标，容量越大，意味着所能存储的二进制信息越多。

存储器由许多存储单元组成，每个存储单元可存放 8 位二进制信息，其位数称为存储单元的长度。8 位二进制数称为一个字节，用 B（Byte）表示。

存储容量一般可表示为：存储容量=存储器单元数×每单元二进制位数

如一个存储器有 4 096 个单元，每个单元可存放 8 位二进制信息，则存储容量为 4 096×8，也就是 4096B。

存储容量通常以字节为单位来表示，对于大容量存储器还可以用千字节（KB）、兆字节（MB）、吉字节（GB）、太字节（TB）等表示。

其换算关系为：$1KB=2^{10}B=1\ 024B$

$1MB=2^{20}B=1\ 024KB$

$1GB=2^{30}B=1\ 024MB$

$1TB=2^{40}B=1\ 024GB$

2. 存取速度

存储器的存取速度取决于存储介质的两种物理状态的变换速度，它可以用存取时间和存取周期来衡量。

（1）存取时间：是指启动一次存储器操作到完成该操作所用的时间。具体来说，是从存储器接收到寻址地址开始到取出或存入数据为止所需要的时间。通常手册上给出这个参数的上限值，称为最大存取时间。

（2）存取周期：是指连续两次独立的存储器操作之间的最小时间间隔。通常存取周期略大于存取时间，其差别与存储器的物理实现细节有关。

存取速度的度量单位通常采用 ns。目前高速存储器的存取速度已小于 20ns。存取时间越小，则存取速度越快。

3. 价格

存储器的价格常用每位的价格来衡量。如果存储器容量为 S，总价格为 C，则每位价格为 P=S/C。它不仅包含了存储元件本身的价格，也包括为该存储器操作服务的外围电路的价格。一般来说，主存储器的价格较高，辅助存储器的价格则低得多。

存储器总价格与存储容量成正比，与存取速度成反比。速度较快的存储器，其价格也较高，容量也不可能太大。因此，容量、速度、价格 3 个指标之间是相互制约的。

4. 可靠性

可靠性是存储器对电磁场及温度等变化的抗干扰性。半导体存储器由于采用大规模集成电路结构，因此可靠性高，平均无故障时间为几千小时以上。

微型计算机要正确运行，必须要求存储器系统具有很高的可靠性。

5. 功耗

功耗指每个存储单元所消耗的功率，单位为 μW/单元，也有用每块芯片总功率来表示功率，单位为 mW/芯片。

衡量存储器性能的其他指标还有体积、重量、品质等，用户在设计和选用存储器时还要综合考虑这些因素，要根据实际需要全面衡量，尽可能满足主要需求并兼顾其他，尽量提高性能价格比。

6.1.3 存储系统的层次结构

微机系统对存储器的基本要求是容量大、速度快和成本低，但要想在一个存储器中同时兼顾这 3 个方面是很困难的。为了解决存储器的容量、速度、价格三者之间的矛盾，人们除了不断研制新的存储器件，改进存储性能外，还从存储系统结构上研究更加合理的结构模式，形成存储系统的多级层次结构。

存储系统的层次结构就是把不同存储容量、存取速度和价格的存储器按层次结构组成多层存储器，并通过管理软件和辅助硬件有机组合成统一的整体，使所存放的程序和数据按层次分布在各种存储器中。目前，在计算机系统中通常采用三级层次结构来构成存储系统，主要由高速缓冲存储器 Cache、主存储器和辅助存储器组成，如图 6-1 所示。

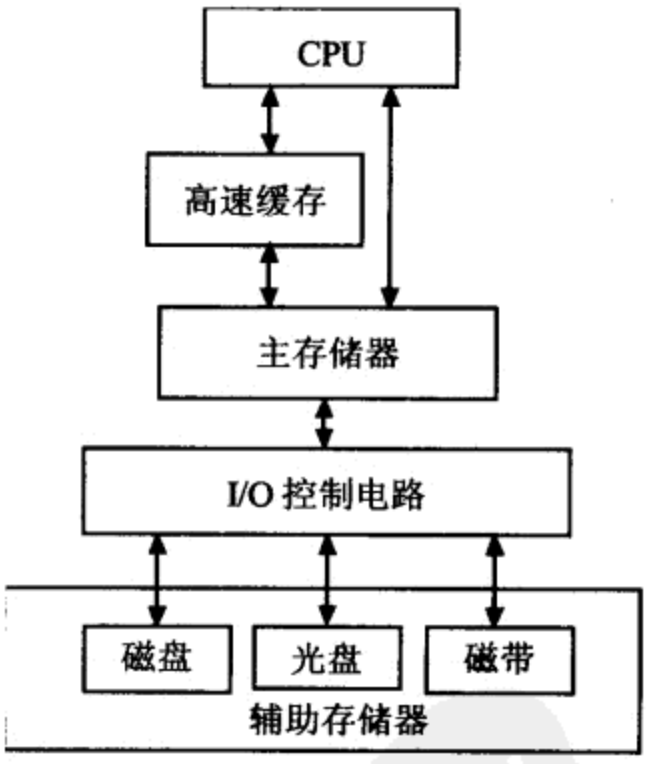


图 6-1 存储系统的层次结构

对于容量要求很大的存储系统，仅仅采用单一结构的存储器是行不通的。它至少需要两种存储器：主存储器和辅助存储器。使用速度较快、容量不大的磁芯或半导体存储器作为主存储器，而用容量极大、价格较便宜的磁带、磁盘作为辅助存储器。把 CPU 当前正在运行的程序和数据放在主存中，把暂时不用的程序和数据放在辅存中。

在程序执行过程中，不断把位于辅存中即将处理的信息调入主存，处理完毕的信息不断

调出主存。这个工作由计算机自动完成，即采用虚拟存储器技术。在这种系统中，程序员面对的是一个既有主存速度，又有辅存容量的存储器整体，编程时可直接使用辅存容量的地址空间，而不必考虑主存的容量限制。虚拟存储技术较好地解决了存储系统的容量问题。

由于 CPU 处理指令和数据的速度比从主存储器读取指令和数据的速度快(约有一个数量级的差距)，因此主存储器读取速度成为系统“瓶颈”。为了解决存储速度问题，在存储系统结构上主要采用了下列措施。

(1) 在 CPU 内部设置多个通用寄存器，以便存放中间数据，减少访问主存的次数，从而减少主存对 CPU 速度的影响。但通用寄存器数目不宜太多，否则将增加编程负担。

(2) 采用多存储模块交叉存取。可以把主存分成多个模块，按顺序把信息交叉地存放到各个模块中。CPU 顺序访问主存时，连续的信息可以从不同的存储模块中同时读取，从而提高了读取信息的平均速度。近年来出现的双端口存储器，可以从两个端口并行地访问存储器所存储的内容，有效地提高了存储器的访问速度。

(3) 采用高速缓冲存储器 (Cache)。Cache 的存取速度与 CPU 工作速度相当，容量较小，位于 CPU 和主存之间。它所存放的内容是当前机器运行中最活跃的一部分信息，是主存中部分信息的副本。程序运行时，计算机自动把要执行的程序和数据从主存调入 Cache 中，这样 CPU 只要访问 Cache 就可以取得所需信息，只有当 CPU 所需的信息不在 Cache 时才去访问主存。不断用新的信息更新 Cache 的内容，就可使 CPU 的大部分信息访问操作在 Cache 中进行，以减少对慢速主存的访问次数。

图 6-1 所示的存储系统多级层次结构由上向下分为 3 级，其容量逐渐增大，速度逐级降低，成本则逐次减少。整个结构又可以看成两个层次：它们分别是主存—辅存层次和 Cache—主存层次。这个层次系统中的每一种存储器都不再是孤立的存储器，而是一个有机的整体。它们在辅助硬件和操作系统的管理下，可把主存—辅存层次作为一个存储整体，形成的可寻址存储空间比主存储器空间大得多。由于辅存容量大，价格低，使得存储系统的整体平均价格降低。由于 Cache 的存取速度可以和 CPU 的工作速度相媲美，故 Cache—主存层次可以缩小主存和 CPU 之间的速度差距，从整体上提高存储器系统的存取速度。尽管 Cache 成本高，但由于容量较小，故不会使存储系统的整体价格增加很多。

综上所述，一个较大的存储系统是由各种不同类型的存储设备构成，这样的系统是一个具有多级层次结构的存储系统。该系统既有与 CPU 相近的速度，又有极大的容量，而价格又是较低的。采用多级层次结构的存储器系统可以有效解决存储器的速度、容量和价格之间的矛盾。

6.2　半导体存储器

大规模集成电路技术的发展使得半导体存储器的价格大大降低。现代微机的主存储器已普遍采用半导体存储器。其特点是容量大、存取速度快、体积小、功耗低、集成度高、价格便宜。

半导体存储器按存取方式可分为两大类：随机存取存储器(RAM)和只读存储器(ROM)。其中 RAM 按采用器件可分为双极型存储器和 MOS 型存储器，而 MOS 型存储器按存储原理

又可分为静态存储器（SRAM）和动态存储器（DRAM）；ROM 按存储原理可分为掩膜 ROM、可编程 PROM、光可擦除 EPROM、电可擦除 E^2PROM 和闪速存储器等，如图 6-2 所示。

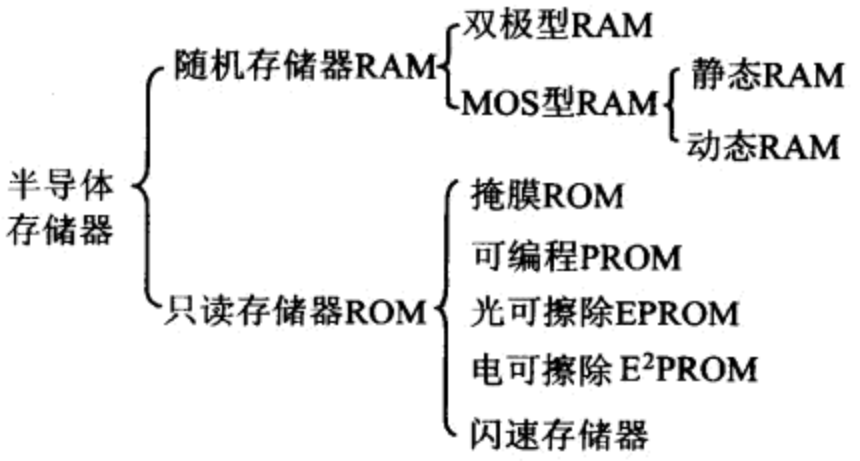


图 6-2　半导体存储器的分类

6.2.1　半导体存储器的基本结构

半导体存储器一般由地址译码器、存储矩阵、控制逻辑和输入/输出控制电路等部分组成，其结构框图如图 6-3 所示。

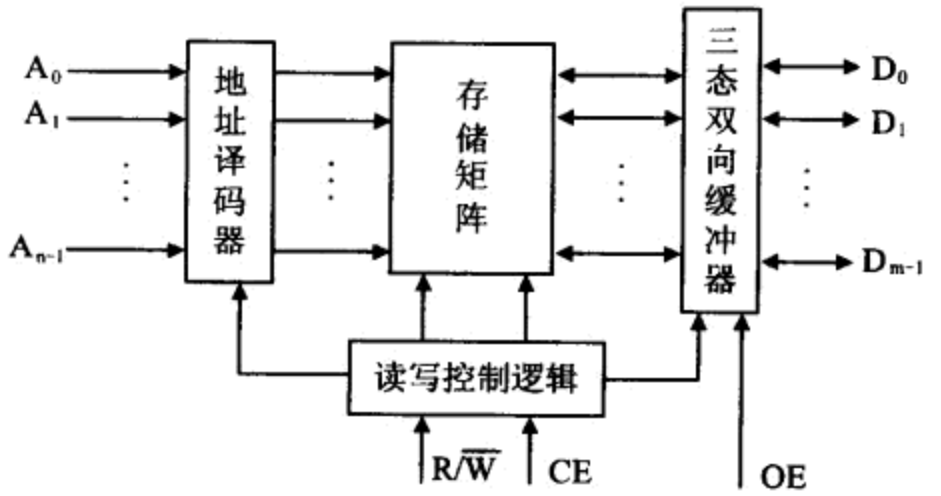


图 6-3　半导体存储器的结构框图

1. 地址译码器

该电路功能为接收 CPU 发出的地址信号，产生地址译码信号，以便选中存储矩阵中的某个存储单元。存储矩阵中基本存储电路的编址方式有两种：单译码与双译码。

单译码方式适用于小容量的字结构存储器，存储器中的存储单元呈线性排列，例如在图 6-4 中（a）图所示的单译码结构图中，当地址信号线 A_5～A_0 输入为 000101 时便选择了第 5 个存储单元。

双译码方式适用于容量较大的存储器。将地址线分为列线和行线两组分别译码，当输入一个地址编码时，例如在图 6-4 中的（b）图所示的双译码结构图中，当 A_5～A_0 输入为 001010 时，行译码产生为 2，列译码产生为 1，则选中存储单元为第 2 行第 1 列的存储单元。

2. 存储矩阵

存储体是能够存储二进制信息的基本存储单元的集合。为了便于信息的读/写，这些基本的存储单元配置成一定的阵列并进行编址，所以也称存储体为存储矩阵。

存储体中每个具有唯一地址的单元可存储一位或多位二进制数据。所以芯片的存储容量可以理解为芯片的存储单元数与每个单元存储位数之积。例如，SRAM 芯片 2114 有 10 根地址线和 4 根数据线，即存储单元数为 2^{10}，每个单元中的存储位数为 4，所以 2114 的存储容量为 $2^{10}\times4$ 位。

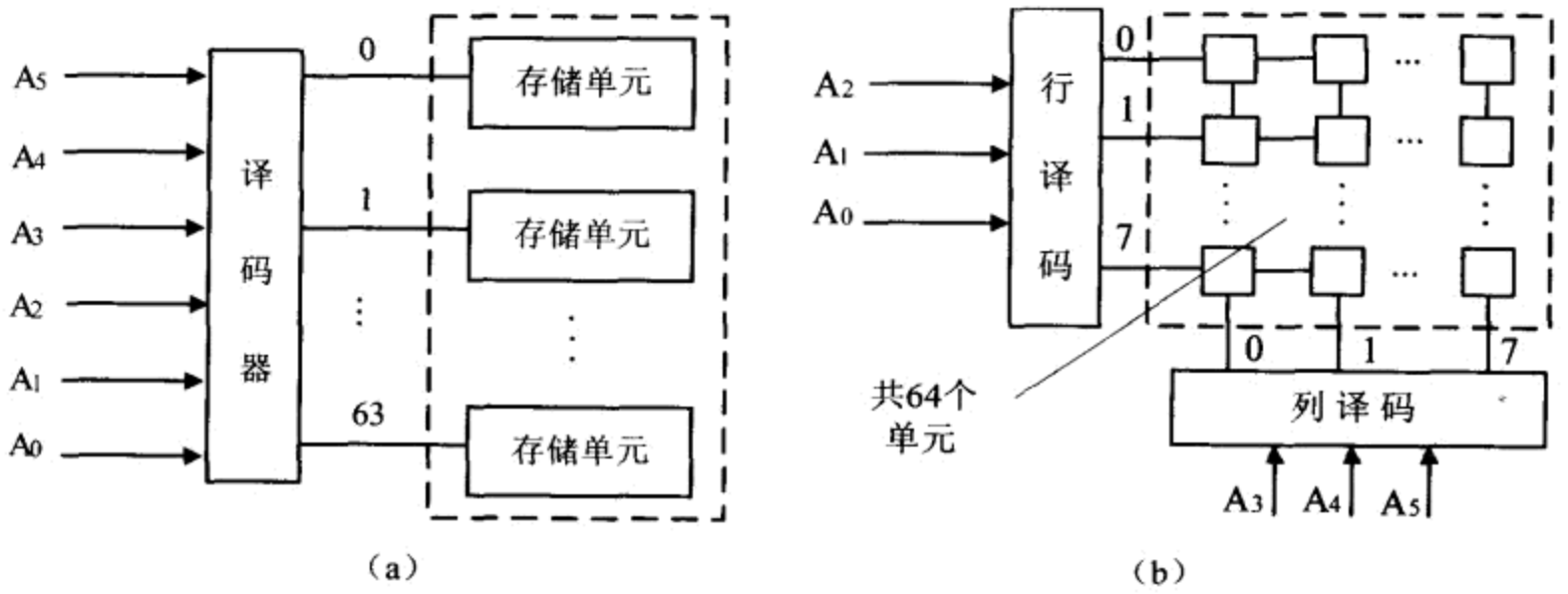


图 6-4　地址译码电路

3. 读/写控制逻辑

存储芯片的片选信号一般用 $\overline{CS}$ 来表示。有效时可对存储芯片进行读/写操作，无效时芯片脱离总线。存储芯片的片选端一般与系统高位地址相连。

存储芯片的读/写控制，以 RAM 芯片的情况最为典型。该类芯片具有两个控制端，一般用 OE（输出允许）和 $\overline{WE}$（写允许）来表示。当芯片被选中时，OE 被用来控制读的操作。有效时，允许芯片将寻址单元内的数据输出。该控制端一般与系统的读控制线 MEMR（或 RD）相连。$\overline{WE}$ 被用来控制写的操作。有效时，引脚上的数据将被允许进入芯片，写入被寻址的单元。该控制端一般与系统的写控制线 MEMW（或 WR）相连。

4. 输入/输出控制电路

半导体存储器的数据输入/输出控制电路多为三态双向缓冲器结构，以便使系统中各存储器芯片的数据输入/输出端能方便地挂接到系统数据总线上。当对存储器芯片进行写入操作时，片选信号及写信号有效，数据从系统总线经三态双向缓冲器传送至存储器中相应的基本存储单元。当存储芯片进行读出操作时，片选信号输出有效，写开放信号无效（读/写控制信号为读状态），数据从存储矩阵中相应基本存储单元中读出，经三态双向缓冲器传送至系统总线。

6.2.2　随机存取存储器（RAM）

所谓随机存取是指通过指令可以随机地对每个存储单元进行访问，访问时间都是相同的，与存储单元的地址无关。通常随机存取存储器中每个存储单元的内容，可根据程序要求随时读/写，所以实际上称它为读/写存储器更确切，但习惯上都把它叫做随机存取存储器。

随机存取存储器根据存储原理又可分为静态 RAM 和动态 RAM。静态 RAM 存放的信息在不停电的情况下能长时间保留，状态稳定。只要不掉电，保存信息就不会丢失。动态 RAM 电路简单，集成度高，但其中保存的内容即使在不掉电的情况下，隔一定时间之后也会自动

消失。因此要定时进行刷新，这就是“动态”名字的由来。

1. 静态 RAM（SRAM）

（1）基本存储电路

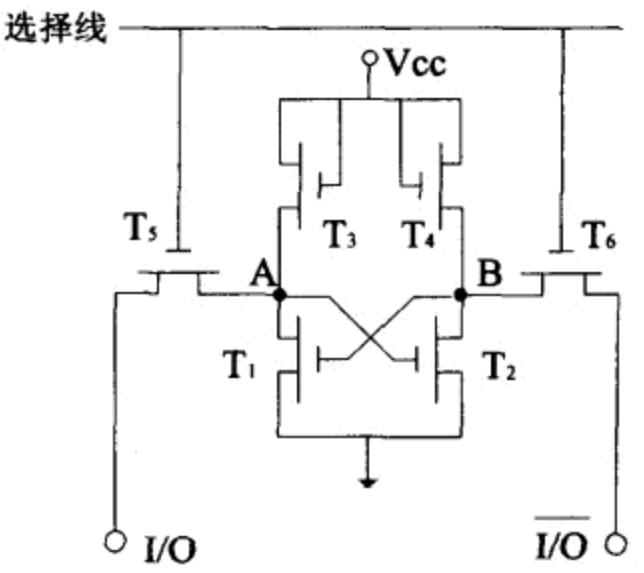


图 6-5 六管静态 RAM 存储电路

该电路通常是由 6 个 MOS 管组成的双稳态触发器电路，如图 6-5 所示。

电路中 T_1、T_2 为工作管，T_3、T_4 分别为 T_1、T_2 的负载管，由 T_1、T_2、T_3、T_4 构成的双稳态触发器，可以存储一位二进制信息 0 或 1。当 T_1 截止时，A 点为高电平，即 A=1，它使 T_2 导通，于是 B=0，而 B=0 又保证了 T_1 可靠截止，这是一个稳定状态。反之，当 T_1 导通 T_2 截止时，B 点为高，即 B=1，A=0，这也是一种稳定状态，也就是说，该电路具有两个稳定状态。因此，可用 T_1 管的两种状态来表示“1”或“0”。设 T_1 截止 T_2 导通的状态为“1”状态，T_1 导通 T_2 截止的状态称为“0”状态。显然，仅能保持这两个稳定状态还不够，还要对状态进行控制，于是就加上了 T_5、T_6 管。T_5、T_6 管作为两个控制门，起两个开关的作用。

存储电路的工作过程是：当存储基元被选中时，选择线（也叫行线）为高电平，门控管 T_5、T_6 导通，触发器与 I/O 线（位线）接通，即 A 点与 I/O 线接通，B 点与 $\overline{I/O}$ 接通。

① 写入时：写入数据信号从 I/O 线和 $\overline{I/O}$ 线进入。若要写入“1”，则使 I/O 线为 1（高电平），$\overline{I/O}$ 为 0（低电平），它们通过 T_5、T_6 管与 A、B 点相连，即 A=1，B=0，从而使 T_1 截止，T_2 导通。而当写入信号和地址译码信号消失后，T_5、T_6 截止，该状态仍能保持。若要写入“0”，使 I/O 线为 0，$\overline{I/O}$ 为 1，则使 T_1 导通，T_2 截止。只要不断电，这个状态也会一直保持下去，除非再重新写入一个新的数据。

② 读出时：当要对写入内容进行读出时也要先通过地址译码使选择线（行线）为高，于是 T_5、T_6 导通，A 点的状态被送到 I/O 线上，B 点的状态被送到 $\overline{I/O}$ 线上，这样，就读取了原来存储的信息。信息读出以后，原来存储内容仍然保持不变，所以，这种读出是一种非破坏性读出。

SRAM 的主要优点是工作稳定，不需外加刷新电路，从而简化了外部电路设计。但由于 SRAM 的基本存储电路中所含晶体管较多，故集成度较低。另外，由 T_1、T_2 管组成的双稳态触发器总有一个管子处于导通状态，所以会持续地消耗功率，从而使 SRAM 的功耗较大。

（2）静态 RAM 的结构

利用基本存储电路排成阵列，再加上地址译码电路和读/写控制电路就可以构成随机存取存储器。为了选中某一个单元，往往利用矩阵式排列的地址译码电路。例如 1KB（1 024 个单元）的内存，需 10 根地址线，其中 5 根用于行译码，另 5 根用于列译码，译码后在芯片内部排列成 32 条行选择线和 32 条列选择线，这样可选中 1 024 个存储单元中的任何一个，而每一个单元的基本存储电路个数与数据线位数相同。

静态 RAM 内部由很多如上所述的基本存储电路组成。下面以 4 行 4 列基本存储电路构成 16×1 静态 RAM 为例来说明静态 RAM 的结构。

图 6-6 所示为有 16 个存储单元、每个存储单元仅有 1 个二进制位的存储器。它的组成为：①16 个基本存储电路。②行、列地址译码电路。③4 套列开关管。④读/写控制电路。

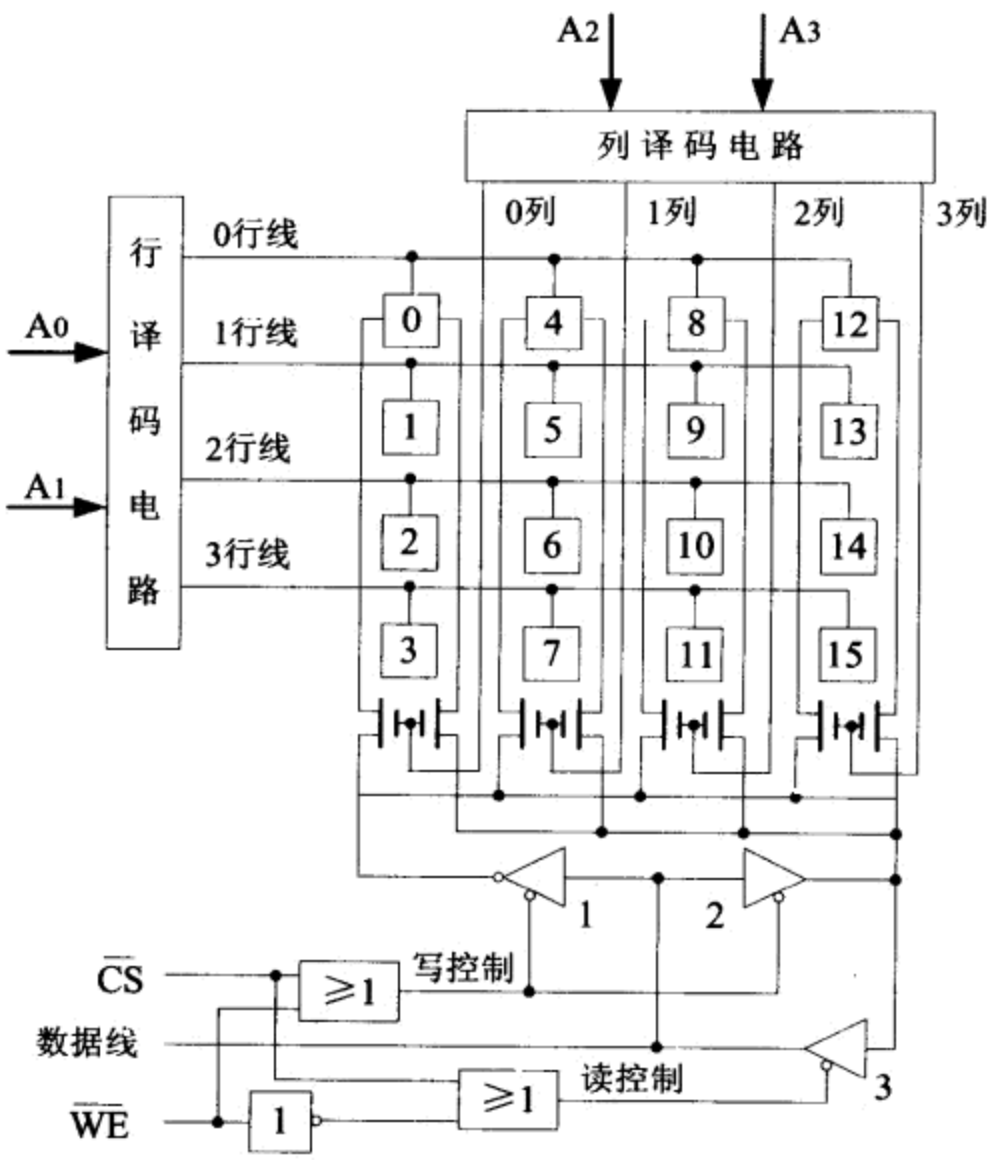


图 6-6　16×1 静态 RAM 原理图

该存储器的控制信号主要有两个：一个是片选信号 $\overline{CS}$，低电平有效。$\overline{CS}$ 有效时，该存储芯片被选中，这时才能进行读/写操作；另一个是写允许信号 $\overline{WE}$，规定低电平时存储器进行写操作，高电平时存储器进行读操作。

当给定地址码以后，例如 $A_3A_2A_1A_0$=0000，则 A_1A_0 经行地址译码使 0 行线为高电平，A_3A_2 经列地址译码电路使 0 列线为高电平，于是 0 号基本存储电路被选中，在 $\overline{CS}$ 有效的情况下，再根据给定的读/写控制电路，对 0 号基本存储电路进行相应的读出或写入操作。同理，当地址码为 $A_3A_2A_1A_0$=1000 时，则 8 号基本存储电路被选中。总之给定一个地址码，就唯一地选中一个存储单元。

常用的典型 SRAM 芯片有 6116、6264、62256、628128 等。Intel 6116 的引脚及功能框图如图 6-7 所示。6116 芯片的容量为 2K×8 位，有 2 048 个存储单元，需 11 根地址线，7 根用于行地址译码输入，4 根用于列译码地址输入，每条列线控制 8 位，从而形成了 128×128 的存储阵列。

6116 的控制线有 3 条：片选信号 $\overline{CS}$、输出允许 $\overline{OE}$ 和读/写控制 $\overline{WE}$。

Intel 6116 存储器芯片的工作过程如下。

读出时，地址输入线 A_{10}～A_0 输入的地址信号送到行、列地址译码器，经译码后选中一个存

储单元（其中有 8 个存储位），由$\overline{CS}$、$\overline{OE}$、$\overline{WE}$构成读出逻辑（$\overline{CS}=\overline{OE}=0$，$\overline{WE}=1$），打开右面的 8 个三态门，被选中单元的 8 位数据经 I/O 电路和三态门送到 D_7～D_0 输出。

写入时，选中某一存储单元的方法和读出相同。不过这时$\overline{CS}=\overline{WE}=0$，$\overline{OE}=1$，打开左边的三态门，从 D_7～D_0 输入的数据经三态门和输入数据控制电路送到 I/O 电路，从而写到存储单元中。

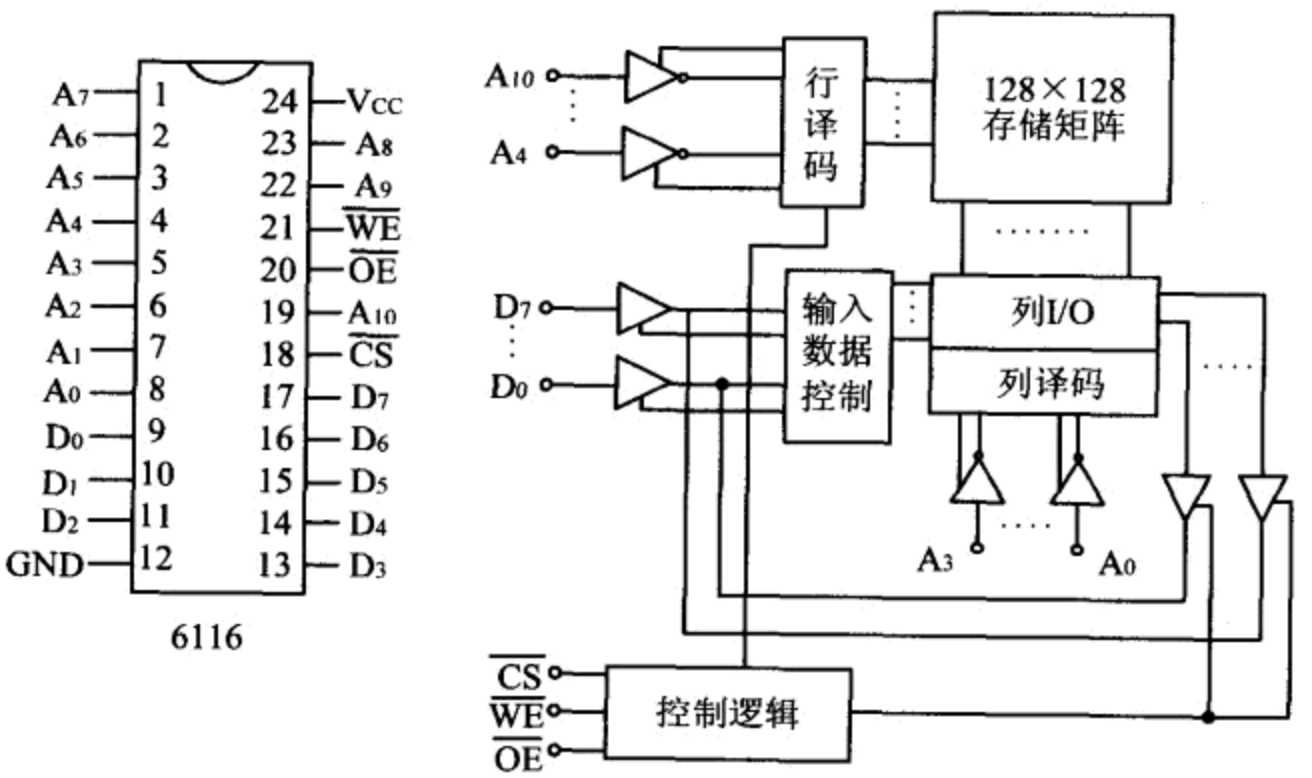


图 6-7　Intel 6116 的引脚及功能框图

没有读/写操作时，$\overline{CS}=1$，即片选信号处于无效状态。输入/输出三态门呈高阻状态，存储器芯片与系统总线“脱离”。

其他静态 RAM 的结构与 6116 相似，只是地址线不同而已。它们与同样容量的 EPROM 芯片引脚相互兼容，从而使接口电路的连线更为方便。

6.2.3　动态 RAM（DRAM）

1. 基本存储电路

DRAM 存储信息的基本电路可以采用单管电路、三管电路和四管电路。单管电路由于集成度高，功耗小，被越来越多地应用在 DRAM 中。基本存储电路如图 6-8 所示，由一只 MOS 管和一个与源极相连的电容 C 组成。由图可见，DRAM 是依靠与 MOS 管源极相连的电容 C，存储电荷或不存储电荷这两个状态来记忆信息“1”和“0”的。当电容 C 存储电荷时，为逻辑“1”，不存储电荷时，为逻辑“0”。

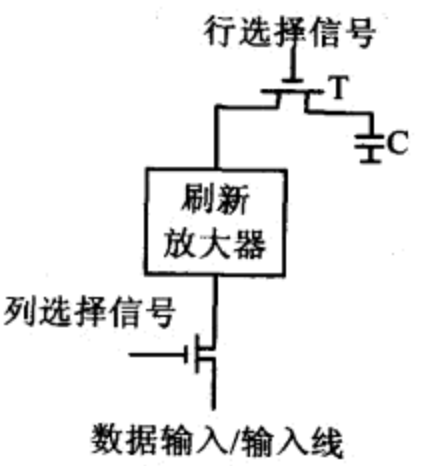


图 6-8　单管动态基本存储电路

没有读/写操作时，行选线处于低电平，MOS 管 T 截止，电容 C 与外电路断开，不能进行充、放电，将保持原状态不变。

在进行读操作时，根据行地址译码，使某一条行选择线为高电平，于是使本行上所有的基本存储电路中的管子 T 导通，使连在每一列上的刷新放大器读取对应的存储电容上的电压值。刷新放大器将此电压值转换为对应的逻辑电平“0”或“1”，又重写到存储电容上，而列地址译码产生列选择信号，所选中那一列的基本存储电路受到驱动，从而可读取信息。

在进行写操作时，行选择信号为“1”时，选中了该行，电容上的信息送到刷新放大器上，刷新放大器又对这些电容立即进行写操作。由于刷新时列选择信号总为“0”，因此电容上信息不可能被送到数据总线上。

2. DRAM 的刷新

DRAM 是利用电容存储电荷的原理来保存信息的，但由于任何电容都存在漏电现象。因此，一段时间后会由于电容的漏电现象导致电荷流失，使保存信息丢失。解决的办法是“刷新”，即每隔一定时间（一般为 2ms）就必须对动态 RAM 进行读出和再写入操作，使原来处于逻辑电平“1”的电容上所释放的电荷又得到补充，而原来处于电平“0”的电容仍保持“0”，这个过程叫动态 RAM 的刷新。

尽管进行一次读/写操作也可认为是对选中行进行刷新操作，但由于读/写操作的随机性，并不能保证在 2ms 内对 DRAM 的所有行都能遍访一次，所以需要专门安排存储器刷新周期，以便系统地完成对 DRAM 的刷新。

专门安排的存储器刷新操作不同于存储器读/写操作，主要表现在以下几点。

- 刷新地址通常由刷新地址计数器产生，而不是由地址总线提供。
- 由于 DRAM 的基本存储电路可按行同时刷新，所以刷新只需要行地址，不需要列地址。
- 刷新操作时，存储器芯片的数据线呈高阻状态，即片内数据线与外部数据线完全隔离。

DRAM 的缺点是需要刷新逻辑电路，而且刷新操作时不能进行正常读/写操作。但 DRAM 与 SRAM 相比具有集成度高、功耗低、价格便宜等优点，所以在大容量存储器中普遍采用。

3. DRAM 举例

典型的 DRAM 是 Intel 2164A，该芯片的容量为 64K×1 位，即片内有 65 536 个存储单元，每个单元只有 1 位数据，用 8 片 2164A 才能构成 64KB 的存储器。若想在芯片内寻址 64K 单元，必须用 16 条地址线。为减少地址线引脚数目，DRAM 地址线采用行地址线和列地址线分时工作，这样 DRAM 对外部只需引出 8 条地址线即可。

Intel 2164A 的内部结构示意图如图 6-9 所示。

图中 64K 存储体由 4 个 128×128 的存储矩阵组成，每个 128×128 的存储矩阵由 7 条行地址线和 7 条列地址线进行选择，在芯片内部经地址译码后可分别选择 128 行和 128 列。

Intel 2164A 芯片内部有地址锁存器，利用多路开关由行地址选通信号 $\overline{RAS}$，把先送来的 8 位地址送至行地址锁存器；由随后出现的列地址选通信号 $\overline{CAS}$ 把后送来的 8 位地址送至列地址锁存器。这 8 条地址线也用于刷新（刷新时地址计数，实现逐行刷新，2ms 内全部刷新一次）。

锁存在行地址锁存器中的 7 位行地址 $RA_6 \sim RA_0$ 同时加到 4 个存储矩阵上，在每个存储矩阵中都选中一行，则共有 512 个存储电路可被选中，它们存放的信息被选通至 512 个读出放大器，经过鉴别后锁存或重写。

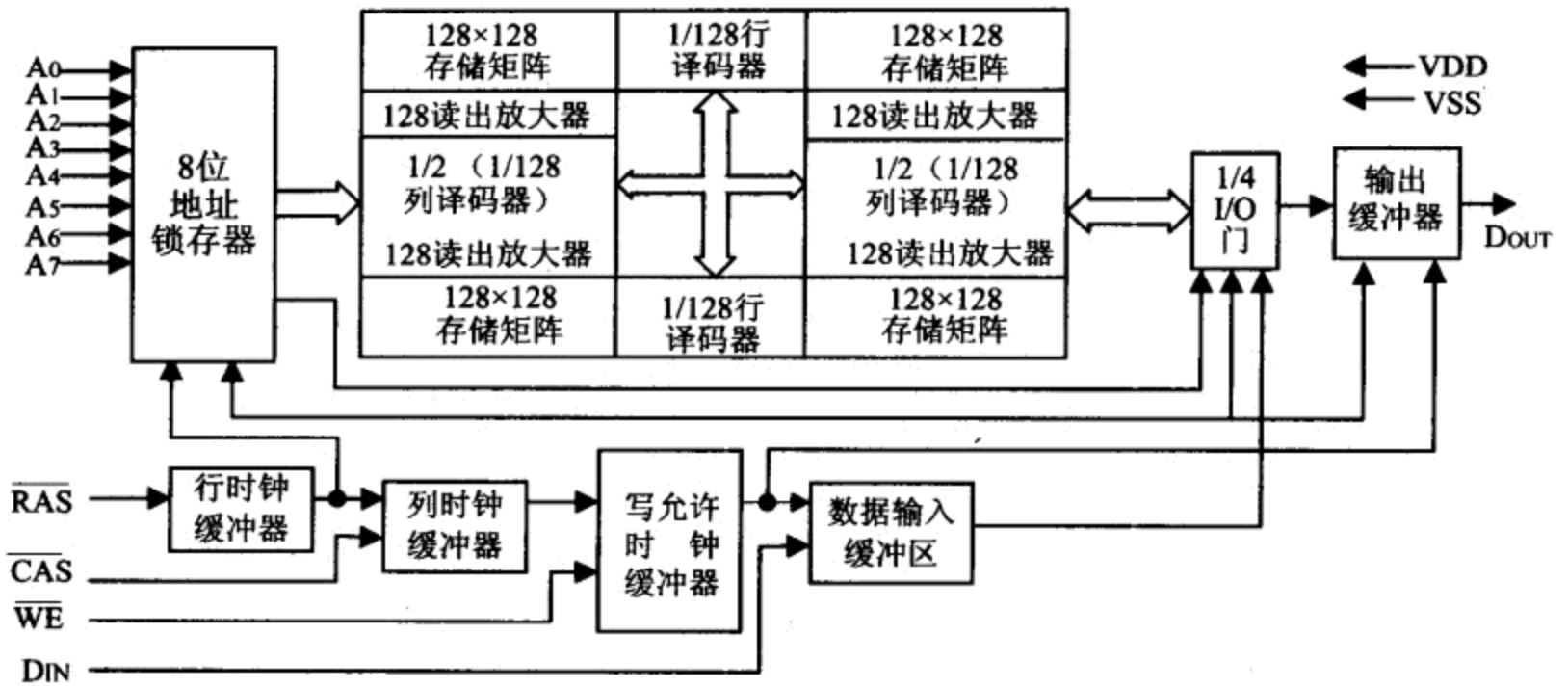


图 6-9 Intel 2164A 的内部结构示意图

锁存在列地址锁存器中的 7 位列地址 CA_6～CA_0（相当于地址总线的 A_{14}～A_8），在每个存储矩阵中选中一列，然后经过 4 选 1 的 I/O 门控电路（由 RA_7、CA_7 控制）选中一个单元，可对该单元进行读/写。

Intel 2164A 数据的读出和写入是分开的，由 $\overline{WE}$ 信号控制读/写。当 $\overline{WE}$ 为高电平时读出，即所选中单元的内容经过三态输出缓冲器在 D_{OUT} 引脚读出。而当 $\overline{WE}$ 为低电平时实现写入，D_{IN} 引脚上的信号经输入三态缓冲器对选中单元进行写入。2164A 没有片选信号，实际上用行选通 $\overline{RAS}$、列选通 $\overline{CAS}$ 信号作为片选信号。

6.3 只读存储器（ROM）

只读存储器（ROM）是一种工作时只能读出，不能写入信息的存储器。在使用 ROM 时，其内部信息是不能被改变的，故一般只能存放固定程序，如监控程序、BIOS 程序等。ROM 的特点是非易失性，即它所存储的信息一经写入，就可以长久保存，不受电源断电的影响，即使掉电后存储信息仍不会改变，十分可靠。

按存储单元的结构和生产工艺的不同，ROM 可分成以下几种：掩膜 ROM、可编程 PROM、光可擦除 EPROM、电可擦除 E^2PROM 和闪速存储器。

6.3.1 掩膜 ROM

掩膜 ROM 中存储的信息是在制造过程中写入的。生产厂家在制造这种存储器时，采用光刻掩膜技术，将程序置入其中。掩膜 ROM 制成后，存储的信息就不能再改写了，用户在使用时，不能写入，只能进行读出操作。

图 6-10 为一个简单的 4×4 位 MOS 管掩膜 ROM，采用单译码结构，两位地址线 A_1、A_0 译码后可译出 4 种状态，输出 4 条行选择线，可分别选中 4 个单元，每个单元有 4 个二进制位，有 4 条列选择线输出。这里列选择线也称位线，若在位线上加上读出控制逻辑，4 条位线就可连到外部数据线上。

如图所示，在行和列的交叉点上，有的跨接管子，有的则没有跨接。这是工厂根据用户提供的程序对芯片图形（掩膜）进行二次光刻形成的，是由存储单元的内容所决定的。如果某位存储的信息为 0，则在该位制作一个跨接管；如果某位存储的信息为 1，则该位不制作跨接管。

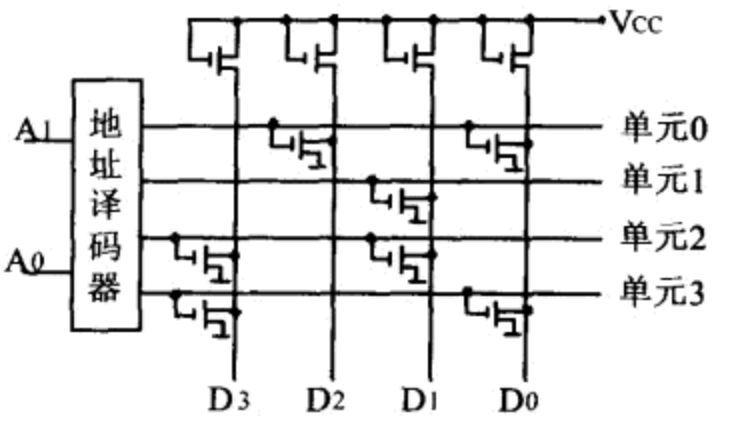


图 6-10　掩膜 ROM 示意图

若地址线 A_1A_0=00，则选中 0 号单元，即字线 0 为高电平，若有管子与其相连（如位线 2 和位线 0），其相应的 MOS 管导通，位线输出为 0，而位线 1 和位线 3 没有管子与字线相连，则输出为 1，故 $D_3D_2D_1D_0$=1010。

掩膜 ROM 制作完毕后，用户不能更改所存信息。至于存储矩阵的内部结构，除上面介绍的行列结构外，还有类似于 RAM 中的双译码结构或复合译码结构等。

6.3.2　可编程 PROM

为了便于用户根据自己的需要来决定 ROM 中所存储的信息，出现了可以编程的只读存储器 PROM。这种 ROM 一般由晶体管阵列组成，由用户在使用前一次性写入信息，写入后只能读出，不能修改。

PROM 常采用二极管或三极管作为基本存储电路，如图 6-11 所示。晶体管的集电极接 V_{CC}，基极连接行线（字选线），发射极通过一个熔丝与列线（位线）相连。

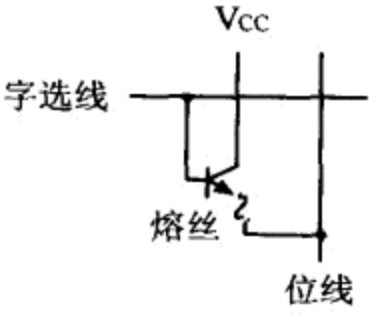


图 6-11　熔丝式 PROM 存储电路

在读操作时，选中行线为高电平。若熔丝完好，可在列线得到输出电流，表示该位存“1”；若熔丝已断开，则在该位得不到输出电流，表示该位存“0”。

PROM 在出厂时，晶体管阵列的熔丝均为完好状态。当用户写入信息时，可在 V_{CC} 端加上高于正常工作电平的“写入电平”，通过编程地址使选中的行线为该电平。若某位写“0”，写入逻辑使相应位线呈低电平，较大的电流使该位熔丝烧断，即存入“0”；若某位写“1”，相应位线呈高电平，使熔丝保持原状，即存入“1”。显然，熔丝一旦烧断，就不能再复原。因此，用户对这种 PROM 只能进行一次编程。

可编程 PROM 的电路和工艺比掩膜 ROM 复杂，又具有可编程逻辑，所以价格较贵。一般在非批量使用时，用 PROM 合算；在批量使用时，则用掩膜 ROM 比较适宜。

6.3.3　光可擦除 EPROM

在某些应用中，程序需要经常修改，可编程 PROM 由于其信息只能写入一次而受到限制，因此能够重复使用的光可擦除 EPROM 被广泛应用。这种存储器利用编程器写入后，信息可长久保持。当其内容需要变更时，可利用擦除器将其所存储信息擦除，使各单元内容复原为 FFH，再根据需要利用 EPROM 编程器编程，因此这种芯片可反复使用。

EPROM 芯片上方有一个石英玻璃接口，只要将此芯片放入一个靠近紫外线灯管的小盒中，一般照射 20 分钟，读出各单元的内容均为 FFH，则说明该 EPROM 已擦除。擦除后的 EPROM 可再写入新的信息。

光可擦除 EPROM 在出厂时，未经过编程，位线上总是“1”，即存储内容为 FFH。编程时用电信号将有关的位由“1”改写为“0”即可实现编程。

光可擦除 EPROM 的优点是一块芯片可多次重复使用，但无论读出或写入都需要专门的设备，这在实际使用时会带来很大的不便，而且在编程时多数情况下需要以字节为单位的擦写，但 EPROM 的只能实现整体擦除，即使仅有几个字节的内容需要改写，也必须把全部信息擦除掉重新写入，这必然会导致芯片的寿命受影响。

6.3.4 电可擦除 E^2PROM

电可擦除 E^2PROM（Electrically Erasable Programmable ROM）是近年来被广泛应用的一种可用电擦除的可编程只读存储器，其特点是能在微机系统中进行在线读/写，即用加电的方式来实现芯片擦除和重新写入，在断电情况下芯片所保存的数据信息也不会丢失，它既能像 RAM 那样随机地进行改写，又能像 ROM 那样在掉电的情况下非易失地保存数据，故可作为系统中可靠保存数据的存储器。

通用 E^2PROM 与 EPROM 相比，具有价格低、擦除简单等优点，擦写次数可达 10 万次以上，一般具有字节擦除和整体擦除两种方式，完成一个字节的擦写大约需要 10ms。其数据擦除仅用+5V 电源即可，写入数据时不需要高电压，也无需编程脉冲配合，使用非常方便。但由于每个存储单元有两只晶体管，因此开发大容量 E^2PROM 是非常困难的。

6.3.5 闪速存储器

近年来一种被称为闪速存储器（Flash Memory）的新型半导体存储器正受到用户广泛的欢迎，其特点是既具有 RAM 的易读易写、体积小、集成度高、速度快等优点，又有 ROM 断电后信息不丢失等优点，是一种很有前途的半导体存储器。之所以称为“闪速”，是因为它能快速地同时擦除所有单元。

闪速存储器是在 EPROM 和 E^2PROM 的制造技术基础上发展起来的，并在 EPROM 沟道氧化物处理工艺中，特别实施了电擦除和编程次数能力的设计。因此它既有 EPROM 价格便宜、集成度高的优点，又有 E^2PROM 的电可擦除性、可重写性，且具有可靠的非易失性，即使在供电电源关闭后仍能保持片内信息。对于需要实施代码或数据更新的嵌入性应用是一种理想的存储器，在固有性能和成本方面有较明显的优势。与 EPROM 只能通过紫外线照射擦除的特点不同，闪速存储器可实现大规模电擦除，而且它的擦除、重写速度较快，一块 1MB 的闪速存储器芯片，其擦除和重写一遍的时间小于 5s，比一般标准的 E^2PROM 要快得多，但它只能整片擦除，不能像 E^2PROM 那样逐个字节进行擦除、重写。

闪速存储器可重复使用。目前，商品化的闪速存储器已可以做到擦写几十万次以上，读取时间小于 90ns，在文件需要经常更新的可重复编程应用中是很重要的。闪速存储器的典型代表芯片是 28F256A（32K×8 位）。

闪速存储器从 1987 年问世，历经 10 余年时间，存储容量已从 256KB 发展到 1GB，提

高了 4 096 倍。制造工艺水平从最初的 2μm 发展到 0.18μm，主流闪速存储器厂家甚至采用 0.15μm 的制造工艺。借助于先进工艺的优势，Flash Memory 的容量可以更大。同时芯片的封装尺寸更小，从最初的 DIP 封装，到 PSOP、SSOP、TSOP 封装，再到 BGA 封装，Flash Memory 已经变得非常纤细小巧。先进的工艺技术也决定了存储器的低电压的特性，从最初 12V 的编程电压，逐步下降到现在的 1.8V 单电压供电，进一步降低了功耗。

闪速存储器展示了一种全新的个人计算机存储器技术，作为一种高密度、非易失的存储器，其独特的性能使其广泛地运用于各个领域，包括 PC 及外设、嵌入式系统、电信交换机、蜂窝电话、网络互联设备、仪器仪表和汽车器件，同时还包括新兴的语音、图像、数据存储类产品，如数码相机、数码摄像机和个人数字助理（PDA）。尤其是笔记本电脑和掌上型袖珍电脑更是大量采用闪速存储器做成的存储卡来取代磁盘，或以低成本和高可靠性替代静态 RAM 和动态 RAM，因为它不像静态 RAM 需要备用电池支持来确保数据保留，也不像动态 RAM 需要磁盘作为后备存储器，这样就可以节约电能，减轻重量，降低成本。

总之，闪速存储器是一种低成本、高可靠性的可读/写非易失性存储器。它的出现带来了固态大容量存储器的革命。

6.4　存储器的扩展与寻址

存储器是微机的基本组成部分之一，计算机对存储器不仅有存取速度的要求，而且有存储容量和存储器字长的要求。现在各厂家为用户提供了许多不同容量、不同速度、不同功能、不同字长的存储器芯片，常用的半导体存储器芯片有多字一位存储芯片和多字多位存储芯片。

由于单个存储芯片的存储容量是有限的，并且具有不同的结构，仅靠单片存储芯片并不能满足计算机对存储器的要求，因此要组成一个大容量定字长的存储器模块，通常需要几片或几十片存储器，采用一定的连接方式进行扩充，需要在字向或位向两个方面进行扩展，需要考虑地址线、数据线和控制线的连接等。下面以 2114（1K×4 位）芯片为例，说明如何形成一定容量的存储器。

6.4.1　位扩展

系统要求组成 1K×8 位的 RAM。因为 2114 只有 4 位数据线，不能满足存储器位数的要求，所以要进行位扩展。扩展示意图如图 6-12 所示。

每片 2114 是 1K×4 位，10 条地址线正好满足整个存储体存储单元数量的要求。由于每一片上的数据为 4 位，所以需要两个芯片才能形成 8 位数据。其中一片与数据总线低 4 位相连，另一片接到数据总线高 4 位。并且这两个芯片的片选端 $\overline{CS}$ 应该接在一起。

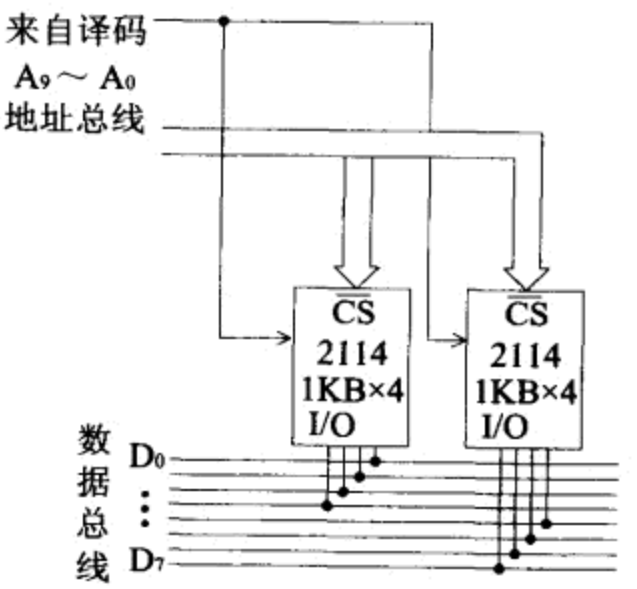


图 6-12　用 2114 组成 1K×8 位 RAM

6.4.2 字位扩展

系统要求组成 2K×8 位的 RAM，而每片 2114 是 1K×4 位，字、位都不能满足存储器的要求，所以要进行字位扩展。扩展示意图如图 6-13 所示。

把两个芯片分为一组，形成 1K×8 位的存储容量，即进行位扩展，若要组成 2K×8 位的存储容量，还要进行字扩展，则需要两组，共 4 片 2114。由于寻址 2KB 容量的 RAM 需要 11 根地址线，除了各组芯片的 10 根地址线直接与地址总线的 A_9～A_0 相连，用做组内寻址外，还需要一根地址线 A_{10} 作为组间寻址。同组芯片的 $\overline{CS}$ 端并接后，分别与地址总线的 A_{10} 或 $\overline{A_{10}}$ 相连。显然，组 1 是存储体的前 1K 单元，组 2 是存储体的后 1K 单元。

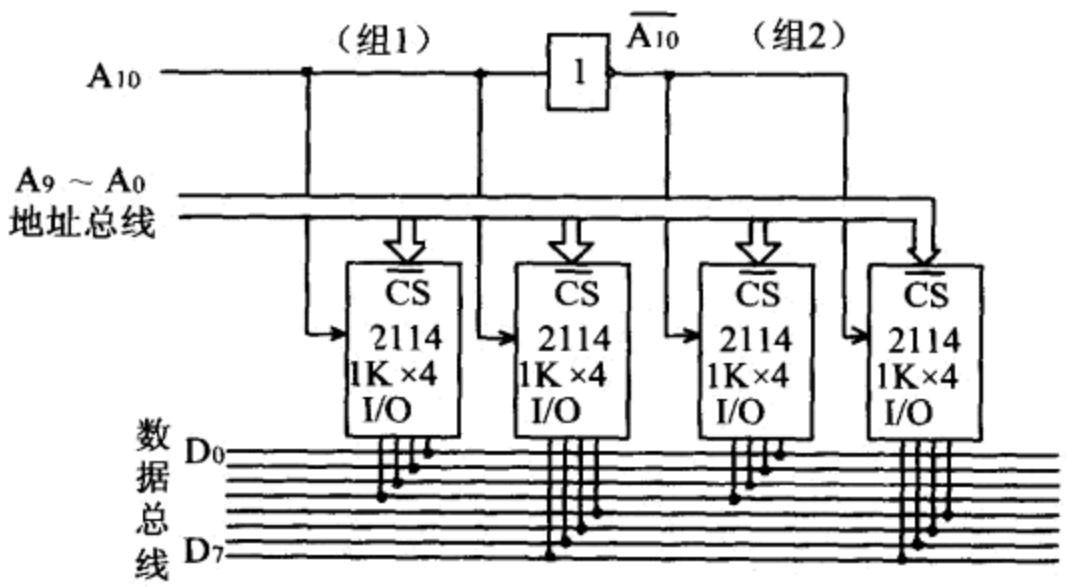


图 6-13 用 2114 组成 2K×8 位 RAM

6.4.3 存储器的寻址

为了保证 CPU 能正确寻址存储器中的所有存储单元，应解决好存储器的地址译码问题。通常将 CPU 的高位地址线用做片间寻址，低位地址线用做片内寻址。

通过地址译码实现片选的方法通常有 3 种，即线选法、全译码片选法和局部译码片选法。

1. 线选法

在简单的微机系统中，由于存储容量不大，存储器芯片数也不多，这时可用单根地址线作片选信号，每个存储芯片或每个 I/O 端口只用一根地址线选通。

线选法的优点是连接简单，无需专门的译码电路；缺点是地址不连续，CPU 寻址能力的利用率太低，会造成大量的地址空间浪费，并且会与其他芯片出现地址重叠，使一个地址码可能选中两个以上的存储单元。

2. 全译码片选法

这种方法除了将低位地址总线直接连至各芯片的地址线外，余下的高位地址总线全部参加译码，译码输出作为各芯片的片选信号。显然，这种方法可以提供对全部存储空间的寻址能力。即使不需要全部存储空间，也可采用全译码法，多余的译码输出留作将来扩展使用。

3. 局部译码片选法

如果不要求提供 CPU 可直接寻址的全部存储单元，而用线选法地址线又不够用时，虽然可用全译码法，但为了简化地址译码逻辑，可采用局部译码法，又称混合译码法。

该方法只对部分高位地址总线进行译码，以产生片选信号，剩余高位线或空着，或直接用做其他存储芯片的片选控制信号。所以，它是介于全译码法和线选法之间的一种选址方法。例如，某存储体只需 16KB 存储容量，若采用 2KB 存储芯片构成，则共需要 8 片。这时可采用局部译码法，即用 A_{15}～A_{13} 作译码，通过 3-8 译码器译码输出作为 8 个存储芯片的片选信号，A_{11}、A_{12} 空着，A_{10}～A_0 作为存储芯片的片内地址线。

6.5　存储器与微处理器的连接

6.5.1　连接时应注意的问题

在微型计算机中，CPU 对存储器进行读/写操作，首先要由地址总线给出地址信号，然后发出读/写控制信号，最后才能在数据总线上进行数据的读/写操作。所以，CPU 在与存储器连接时，地址总线、数据总线和控制总线都要连接。连接时应注意以下问题。

1. CPU 总线的负载能力

CPU 在设计时一般输出线的负载能力为一个 TTL，现在的存储器多为 MOS 管设计，直流负载较小，一般能负载 20 个。故在小型系统中，CPU 总线的负载能力可以直接驱动存储器系统，CPU 可直接与存储器相连。在较大的系统中，当 CPU 和大容量的标准 ROM、RAM 一起使用或扩展成一个多插件系统时，总线上挂接的器件超过上述负载就必须在总线上增加缓冲器或总线驱动器，来增加 CPU 总线的驱动能力，再与存储器相连。地址总线只需接入单向的驱动器，如 74LS244，数据总线需要接入双向驱动器，如 74LS245。

2. 存储器与 CPU 之间的速度匹配

CPU 的取指周期和对存储器的读/写操作都有固定的时序，由此决定了对存储器存取速度的要求。具体地说，CPU 对存储器进行读操作时，CPU 发出地址和读命令后，存储器必须在限定时间内给出有效数据。而当 CPU 对存储器进行写操作时，存储器必须在写脉冲规定的时间内将数据写入指定存储单元，否则就无法保证迅速准确地传送数据。因此，当存储器速度跟不上 CPU 时序时，则系统应考虑插入等待周期 T_W，以解决存储器与 CPU 之间的速度匹配问题。如 8086 的主频采用 5MHz，1 个时钟周期为 200ns，每个时钟周期称为 1 个 T 状态。CPU 和存储器交换数据或从存储器取出指令必须执行 1 个总线周期，而最小总线周期由 4 个 T 状态组成。如果存储器速度较慢，CPU 就会根据存储器送来的“未准备好”信号（Ready 信号无效），在 T_3 状态后插入等待状态 T_W，从而延长了总线周期。

3. 存储器的组织、地址分配和译码

在进行存储器与 CPU 连接前，首先要确定内存容量的大小和选择存储器芯片的大小。在组成存储器系统时，往往要选择若干存储器芯片才能达到内存容量的要求。这些存储器芯片如何同 CPU 有效地连接并能有效寻址，就存在一个存储器的地址分配问题。此外，内存又分为 ROM 区和 RAM 区，而 RAM 区又分为系统区和用户区，在进行地址分配时，一定要将 ROM 和 RAM 分区域安排。

存储系统的容量和存储器芯片确定之后，就要将所选择芯片与确定的地址空间联系起来，即将芯片中的存储单元与实际地址一一对应，这样才能通过寻址对存储单元进行读/写操

作。CPU 的地址输出线是有限的，不可能寻址到每一个存储单元，需要地址译码器按一定的规则译码成某些芯片的片选信号和地址输入信号，被选中的芯片就是 CPU 要寻址的芯片。

6.5.2 典型微处理器与存储器的连接

1. 8086 微处理器与只读存储器 ROM 的连接

ROM、PROM 或 EPROM 芯片可以和 8086 系统总线连接，但是要注意像 2716、2732、2764 一类的 EPROM 芯片，它们是以字节宽度输出的。因此，要用两片这样的存储芯片，才能存储 8086 的 16 位指令字。

图 6-14 是两片 2732 EPROM 和 8086 系统总线的连接图。该存储器子系统提供了 4K 字（8KB）的程序存储器（即存放指令代码的只读存储器）。

其中上面一片2732代表了高8位存储体，下面一片 2732 则代表了低 8 位存储体。为了寻址 4K 字的存储单元共需 12 条地址线（A_{12}～A_1）。两片 2732 EPROM 在总线上是并行寻址的。其余的 8086 高位地址线（A_{19}～A_{13}）用来译码产生芯片选择信号 $\overline{CS}$。两片 2732 的 $\overline{CE}$ 端连接到同一个芯片选择信号。

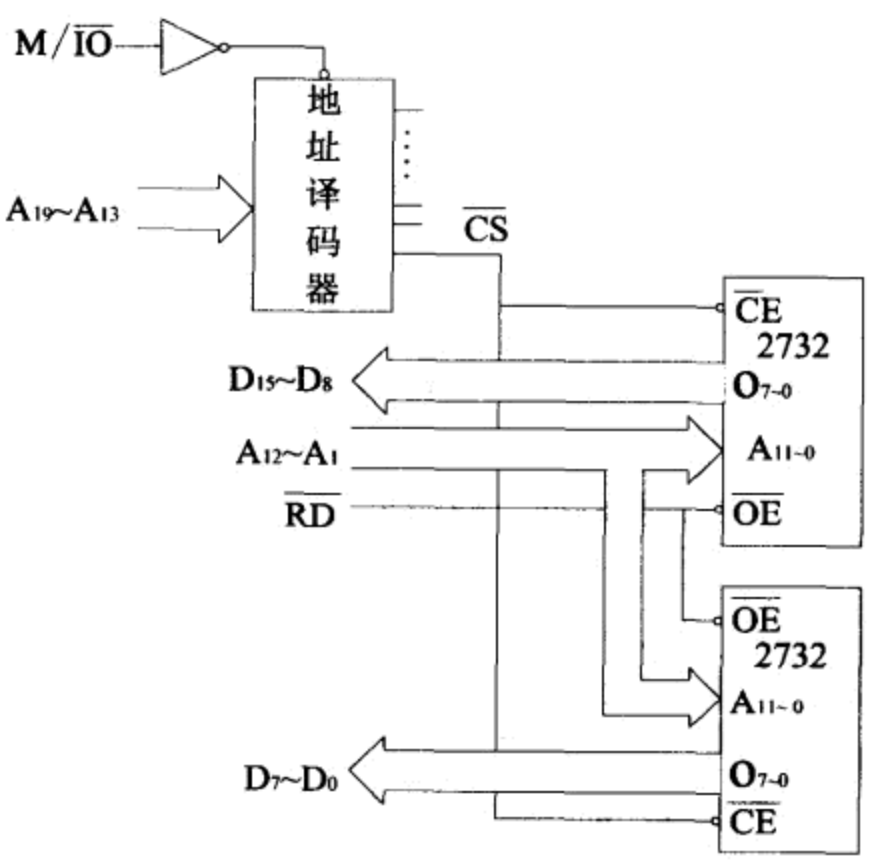


图 6-14　4K 字程序存储器

地址译码在 CPU 与存储器的连接中是必须认真加以考虑的一个问题，特别是当实际使用的存储空间远小于 8086 系统所能提供的寻址空间时。在图 6-14 中，地址线 A_{12}～A_1 已作为 4K 字 ROM 的片内寻址，其余的 7 根地址线（A_{19}～A_{13}）经译码器可输出 128 个片选信号线。若这 128 个片选信号线全部用上，则可寻址 128×8KB（即 1MB）的存储器，这种地址译码方式称为“全译码方式”。全译码方式显然要求用译码器，不浪费存储器地址空间，各芯片之间的地址可以连续且不致因考虑不周而使地址重复。当译码地址未用满时，可留作系统扩展。另外注意 M/$\overline{IO}$ 信号线在这里的作用，它可以确保只有当 CPU 要求与存储器交换数据时才会选中存储器系统。当 8086 的存储器系统较小时，可以采用地址的局部译码方式。局部译码方式只使用有限的高位地址线进行译码。显然，局部译码选择方式可以节省译码器，但比全译码选择方式的地址空间要小。

2. 8086 与 SRAM 的连接

当系统的存储器容量小于 16K 字时适宜采用 SRAM 芯片。图 6-15 给出了一个 2KB 的读/写存储器系统。存储器芯片选用 2142 SRAM，工作在 8086 最小模式系统中。由于 2142SRAM 是以 1K×4 位组织的，所以必须用 4 片 2142 连接成 2KB 的数据存储器。图 6-15 实际上给出了存储器位扩展的基本方法。位扩展的连接方式是将多片存储芯片的地址、片选、读/写端相并联，数据端单独引出。

如果要进行字向扩展则应该把存储芯片的地址线、数据线、读/写控制线并联，而由片选信号区分各片地址，故片选端单独引出。例如要在图 6-15 的基础上进行字扩展，则新加入的奇、偶存储体的地址线、数据线和读/写控制线应该同系统中已经存在的其他存储体的相应线并联，但片选端应该连接到地址译码器的其他输出端上。

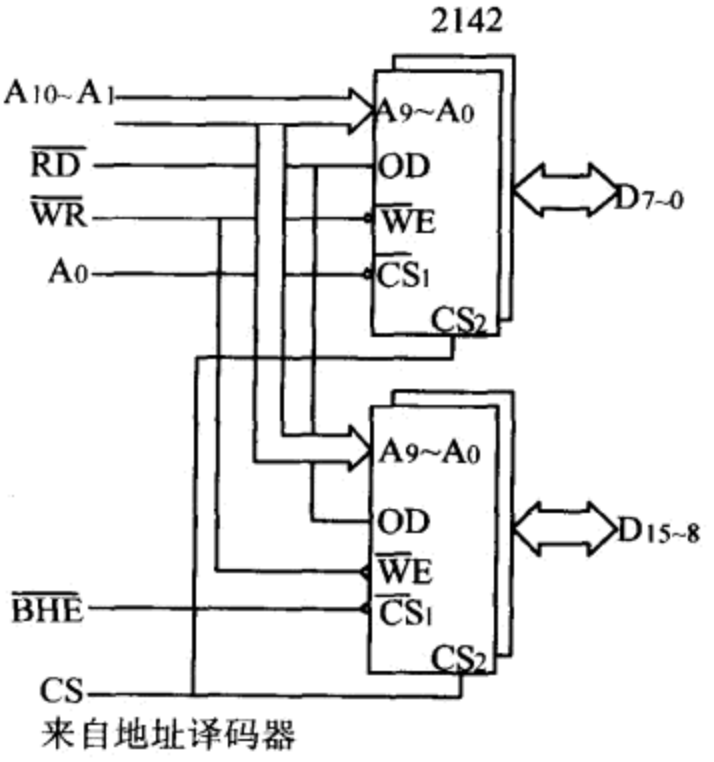


图 6-15　最小模式静态 RAM 存储器

通常，存储器容量的扩展都将位向扩展和字向扩展的方法结合起来。

图 6-15 中经过位扩展的存储器系统每两片 2142 组成一个 1K×8 位的存储体。上面两片用做低 8 位 RAM 存储体，它们的 I/O 引线和数据总线 D_7～D_0 相连，代表了偶地址字节数据；下面两片 2142 用做高 8 位存储体，其 I/O 引线和数据总线 D_{15}～D_8 相连，代表了奇地址字节数据。

地址总线 A_{10}～A_1 并行地连接到 4 片 2142 的地址输入引线，用来实现片内寻址。高位地址 A_{19}～A_{11} 和 M/$\overline{IO}$（高电平）经译码器译码产生 CS_2 的片选信号。要实现对这 2KB 存储单元的访问，高位地址部分必须引起 CS_2 片选信号为逻辑高电平，即 CS_2 必须为高电平。

$\overline{WR}$ 信号（低电平）用来通知存储芯片在写总线周期时数据总线上的数据已经有效。$\overline{WR}$ 并行地加到 4 片 2142 的 $\overline{WE}$ 输入引线上，使 2142 从总线上取数据并写入所选中的存储单元。

8086 输出的 $\overline{RD}$ 信号送到 2142 的 OD 输入线上。OD 是禁止输出的缩写。高电平 OD 禁止 2142 输出数据，低电平 OD 则允许 2142 输出数据到总线上。$\overline{RD}$ 信号为低电平时，允许 2142 在读总线周期时向数据总线输出数据。

A_0 和 $\overline{BHE}$ 信号的不同组合可以得知是同时选中奇、偶地址体还是其中之一，从而实现在数据总线传送一个字或一个字节的目的。

8086 最大模式系统中的存储器系统类似于上面描述的最小模式系统。关键的区别是在最大模式系统中，用 8288 总线控制器代替 8086 产生存储器读/写等控制信号。此外，系统中用 8286 总线收发器来缓冲数据总线。8288 的 DT/$\overline{R}$ 和 DEN 输出信号分别用做 8286 的 T 和 $\overline{OE}$ 信号，用来在读/写总线周期时选择收发器的传送方向和允许数据传送。

3. 8086 微处理器与动态 RAM（DRAM）的连接

如果要求 8086 微型计算机系统的存储容量大于 16K 字时，存储器系统通常使用 DRAM 芯片。8086 微处理器与 DRAM 的连接比较复杂，此处不再讨论。

6.6　辅助存储器

辅助存储器用来存放当前暂时不用的程序或数据，需要时再成批地调入主存。它属于外部设备，因此又称其为外存储器。

常用的辅助存储器目前主要有磁表面存储器和光存储器两类，如软盘、硬盘和光盘存储器等。其中磁盘存储器是计算机系统中最主要的外存设备，特点是存储容量大、易于脱机保存、价格低。

6.6.1 软盘存储器及其接口

1. 概述

软盘是在聚酯薄膜圆形基片上涂一层磁性材料而形成的。以其体积小、价格低、结构简单、易于维护、携带方便和对环境要求不高等优点而得到广泛的应用。

现代微机大多配置 3.5 英寸双面高密度的软盘，该软盘封装在硬壳塑料壳内。顶部的弹片可以向旁边推动，推开后露出黑色的盘片，此时可以进行读/写操作。保护套边缘上有一个滑块，是对磁盘进行写保护用的。当滑块将写保护口封住时，可以进行写操作；否则盘片处于写保护状态，只能读不能写。

2. 主要技术指标

（1）磁道：为了便于对磁盘进行信息存取，把磁盘上的记录面分成许多以盘片中心为圆心的同心圆，每个圆称为一个“磁道（Track）”。

（2）扇区：以磁道作为磁盘的信息存取单位仍然太大，还要把磁道再划分成许多小的存储区，每个存储区称为“扇区（Sector）”。每个磁道上的扇区数目一般是相同的。一张软盘在没使用前，必须进行格式化，即对磁盘按标准格式来划分扇区，而且每个扇区要按记录格式要求填写地址信息，以及定义其所能容纳的字节数。

（3）存储容量：存储容量是指每张软盘所能记录的二进制代码的总和。格式化后的软盘容量计算公式为：容量=记录面数×（磁道数/面）×（扇区数/道）×（字节数/扇区）

如常用的双面 3.5 英寸软盘，每面有 80 个磁道，每道有 18 个扇区，每个扇区有 512 个字节，则其容量为：2×80×18×512B=1.44MB。

3. 软盘驱动器

软盘驱动器（FDD）主要完成软盘的读/写工作，由软盘驱动机构和读/写控制电路组成。

（1）软盘驱动机构

软盘驱动机构可分为如下几部分：

盘片定位机构：由盘片定位槽、浮动夹紧装置及门锁扣等部件组成。磁盘插入时通过盘片定位槽插入驱动器，并定位于主轴中心。门锁关闭后，通过机械联动装置把盘片压在主轴驱动轮上。

软盘驱动装置：软盘驱动是通过同步电机经皮带传动装置拖动主轴皮带轮，使软盘随着主轴皮带轮而转动。

控制磁头寻道定位部件：软盘磁头平时是抬起的，不与磁盘接触，当接到软盘控制器发出的方向和步进信号后，便启动磁头步进马达，驱动磁头小车使磁头做径向步进移动，以寻找磁头读/写的位置。在读/写过程中，磁头加载机构使磁头与盘面接触。

状态检测部件：软盘有零磁道、索引孔、写保护 3 个传感器发出的 3 个检测信号，传送给软盘控制器，以正确完成读、写、寻道等操作。

（2）读/写控制电路

软盘驱动器读/写数据是通过读/写控制电路来完成的，该电路又可分为如下几部分。

读出放大电路：软盘记录数据是通过写电流磁化后，在磁层上保留了剩磁状态作为数据被记录下来。当读数据时，磁头相对读/写介质是运动的，读出信号具有幅度小、差别大、干扰大等特点。因此，需用读出放大电路对读出信号进行放大、滤波、鉴别及整形。

写电路：写入的数据经接收器整形后，由触发器将数据变成极性相反的两个信号，分别由两个写驱动在两个线圈产生相反电流。在恒流电源作用下，使线圈电流恒定，从而保证写数据磁场的稳定。磁头的读/写电路是共用一个线圈，但写线圈是从线圈中间抽头。因此，写线圈电感量小，故缩短了写电流的时间；而读线圈匝数较多，加大了感应电动势。

抹电路：在写磁盘时，尽管磁头接触了盘片，但是在写数据时磁道边沿仍会产生一些杂散磁场，使磁道相互影响。因此，在写入数据时，在写磁头后紧跟着抹磁头，用于清除写后在磁道两边的毛刺，避免读出时产生道间干扰。

4. 软盘控制器

软盘上的信息是按位存储的，而主机的数据处理都是以字节为单位，因此在对磁盘进行读/写操作时必须进行串—并转换或并—串转换，而且软盘驱动器是很精密的外部设备，它对各种控制信号要求很严格，不能由 CPU 直接驱动这些信号。这就要求在处理器和软盘驱动器之间有一个专用部件，CPU 通过该部件控制软盘驱动器，完成对数据的读/写。这些专用部件称为软盘驱动器适配器，其核心部件就是软盘控制器（FDC），软盘适配器的示意框图如图 6-16 中虚线所示。

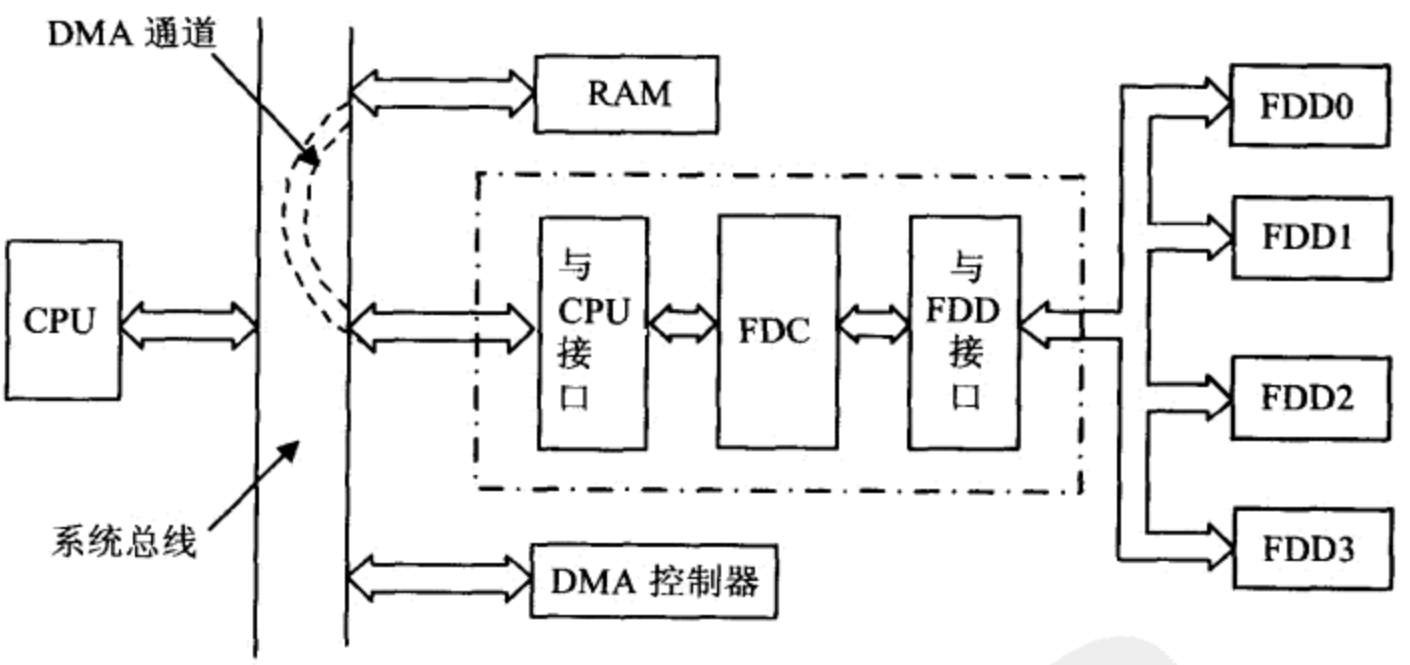


图 6-16　软盘适配器示意框图

软盘控制器的功能是解释来自主机的命令，并向软盘驱动器发出各种控制信号，同时还要检测驱动器的状态，按规定的数据格式向驱动器读/写数据等。

利用专用软盘控制器芯片（如 FD 1771、FD 1791、μPD 765 等）可以很方便地构成软盘控制器，其组成示意图如图 6-17 所示。

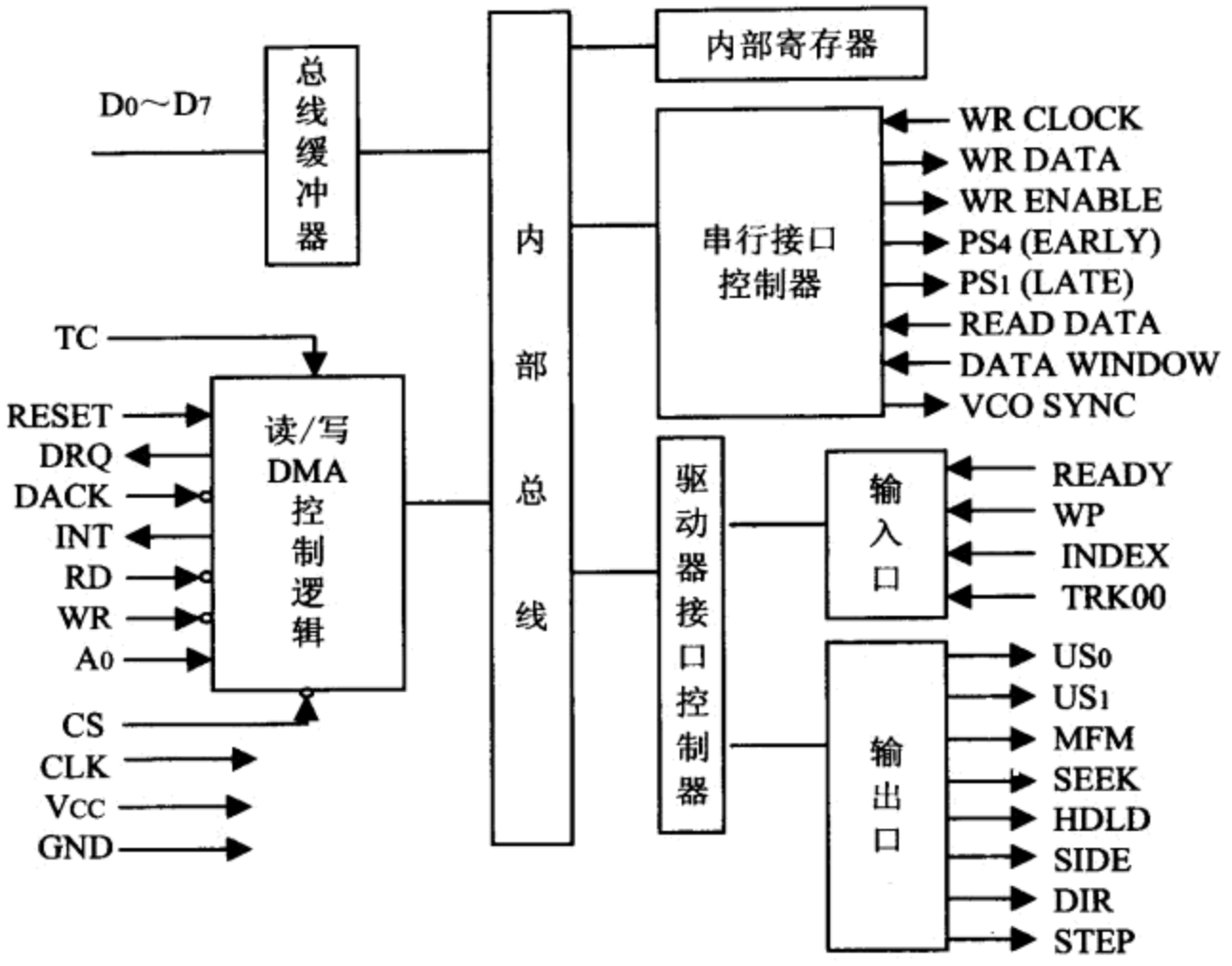


图 6-17　软盘控制器组成示意图

软盘控制器各部分的功能如下。

（1）数据总线缓冲器：用于缓冲主机送来的并行数据。缓冲器中的数据再通过内部总线与寄存器中的信息进行传送。

（2）读/写 DMA 控制逻辑：主要功能是进行读/写和 DMA 控制。采用 DMA 方式传送数据时，此部分可产生数据请求（DRQ）信号，借助 DMA 控制芯片向 CPU 申请总线控制。CPU 响应后，让出总线控制权，接着转入 DMA 数据传送。

（3）串行接口控制器：主要用来控制读/写的各种信号。当采用双密记录方式写入数据时，引入补偿电路；读出时，引入锁相电路，分离出数据。

（4）驱动器接口控制器：用来控制输入/输出的各种信号。

（5）内部寄存器：用来存放软盘控制器芯片的状态、数据、命令和参数。

6.6.2　硬盘存储器及其接口

硬盘是磁表面存储器，以厚度为 1~2mm 的非磁性的铝合金材料或玻璃基片作为盘基，在表面涂抹一层磁性材料作为记录介质。磁层可以采用电涂工艺制成，此时磁粉呈不连续的颗粒存在，也可以用电镀、化学镀或溅射等方法制成。盘面上由外向里有许多同心圆构成相互分离的磁道，通过磁化磁道可以存储信息。盘片以 3 600 转/分钟或 7 200 转/分钟旋转，通过悬浮在盘片上的磁头进行读/写操作。

1. 硬盘分类

硬盘根据磁头和盘片的不同结构和功能，可分为固定磁头磁盘机、活动磁头固定盘片磁盘机和活动磁头可换盘片磁盘机等。

（1）固定磁头磁盘机盘片的每一条磁道上方安装一个磁头，以完成对该磁道的读/写操作。这种磁盘机结构简单，可靠性高，由于几乎不需要寻道时间，故存取速度极快。但所需磁头数量多，由于磁头机械尺寸的限制使磁道密度受到影响，因此总存储容量不大，适合于快速存取的专用机械系统。

（2）活动磁头固定盘片磁盘机的每个盘面上只安装一个或两个磁头，存取数据时磁头可沿盘面径向移动到各个磁道完成读/写操作，磁头与盘面不接触且随气流浮动，称为浮动磁头。这种磁盘机可大大减少磁头数量，有效地提高磁道密度，存储容量明显增大。新型的固定盘式磁盘机一般采用温彻斯特技术，称为温彻斯特磁盘，简称为“温盘机”。这种磁盘机的盘片组也是由一个盘片或多个盘片组成，固定在主轴上，不可拆卸。

（3）活动磁头可换盘片磁盘机不但磁头可移动，而且盘片也由一片或多片磁盘构成盘盒或盘组形式，用户可方便地将它们从磁盘机上卸下或装上。一旦盘片上装满了信息，可卸下供长期保存，使脱机容量不受限制，成为名副其实的“海量存储器”。这种磁盘可以在兼容的磁盘存储器间交换数据，具有脱机保存、存储容量大的优点。为了达到可靠交换数据的目的，磁盘的道密度要适当降低，从而使可换磁盘记录密度的提高受到限制。

根据可换磁盘存储器每轴盘片数目的不同，通常把每轴装有 1～4 的存储器称为盒式磁盘存储器；6 片以上的称为盘组，常用的有 6 片、11 片、12 片等，由同心轴带动旋转。通常盘片组的最上面和最下面不作为记录介质使用，仅作为保护面。每个盘面上由外向里分成许多同心圆，即“磁道”，最外边的是 0 号磁道。每个磁道再分成同样大小的段，一个盘面上各个磁道的同一段构成一个“扇区”。由于各个磁道的半径不同，因此各磁道的存储密度也不一样。不同的盘面上的同一磁道构成一个圆柱面，可连续存放多于一个磁道的信息，即当一次写入信息超过一个磁道时，可继续写入同一柱面上另一盘面的同一个磁道上。

2. 温彻斯特技术

温彻斯特磁盘是由 IBM 公司在美国加州坎贝尔市温彻斯特大街的研究所研制成功的，于 1973 年首先应用于 IBM 3340 硬磁盘存储器中，故称做“温彻斯特技术”。

该技术是将硬盘盘片、读/写磁头、小车、导轨、主轴及控制电路等组装在一起，制成一个密封式不可拆卸的整体的技术。它具有防尘性能好、工作可靠，对使用环境要求不高的突出优点，是磁盘技术向高密度、大容量、高可靠性发展的产物。目前大容量温盘的存储容量已达到几千兆字节，已经成为硬磁盘存储器的主要产品。

温彻斯特技术的主要特点如下。

（1）密封的头—盘组合体（HAD 组合件）：由于盘片和磁头的关系固定，每一磁头读出的就是它自己写入的数据，这样就免去了换盘时的磁头调整问题，也使磁道允许公差要求放宽，提高了道密度。再加上密封结构使得硬盘的防尘效果好，也提高了可靠性。

（2）轻浮力的接触/浮动式磁头：在启动和停机时磁头接触盘面，盘片开始转动后，高速旋转使盘片上形成一个气垫，磁头在盘面上平稳浮起，浮动高度为 0.5μm。这样运转时的不接触，使磁头和盘片的寿命都大大提高，而启停时的接触又省去了磁头进出盘面的精密控制机构。当然，启停次数是有限制的，一般在 5 000~20 000 次。另外，多数硬盘机还在盘面上设立启停区，不让磁头在启停期间接触盘面的数据区。

（3）盘片表面有润滑剂：为在接触式启停过程中减少盘面被磁头擦伤的机会，在盘片

表面涂有薄薄一层润滑剂。

以上特点给磁盘机带来的好处是容量更大，存取速度更快，可靠性更高，寿命更长，制造成本更低。

3. 硬盘驱动器

硬盘驱动器（HDD）又称磁盘机，是独立于主机之外的一个完整装置，用来完成对硬盘的读/写工作。

下面以活动磁头可换盘片磁盘机为例介绍硬盘驱动器的基本结构及其工作原理。其基本结构由 5 部分组成、分别是主轴系统、数据转换系统、磁头驱动和定位系统、空气净化系统和接口电路。

（1）主轴系统：包括主轴电机、主轴部件、盘片和控制电路等。主轴系统的作用主要是安装并固定盘片和盘盒，驱动它们以额定转速稳定旋转。

（2）数据转换系统：包括磁头、磁头选择电路、读/写电路、索引、区标电路等。其作用是接收主机通过接口送来的数据并写入到盘片上，或从盘片上读出信息并送到接口电路。

（3）磁头驱动和定位系统：包括磁头驱动和磁头定位两部分。磁头驱动由音圈电机（或步进电机）和运载部件（也称磁头小车）组成。在磁盘存取数据时，磁头小车的运动驱动磁头进入指定磁道的中心位置并精确地跟踪该磁道。

（4）空气净化系统：包括风机、空气过滤器、印刷电机及其控制电路。其作用是防尘和冷却，往盆腔内送入干净的、冷却的空气，并清洁盘面。

（5）接口电路：主要是完成硬盘驱动器和硬盘控制器之间的数据传输，由接收门电路和发送门电路组成。

硬盘驱动器的工作过程是：在写入时，由控制器送来要写入的数据，通过接口送到写入电路，磁头选择电路选择要写入的磁头，磁头驱动和定位系统把该磁头定位在要写入的磁道的位置上，然后数据就可以写入到选定的盘面、磁道和扇区上；读出时，由磁头选择电路选定磁头，磁头驱动和定位系统使之定位在要读出的磁道位置上，然后由该磁头读出相应扇区的信息，通过读电路将读出信息进行放大、滤波、鉴零、整形后，再送到接口电路。

4. 硬盘控制器

硬盘控制器是主机与硬盘驱动器之间的接口，作用是接受主机发送的命令和数据，并转换成驱动器的控制命令和驱动器可以接受的数据格式，以控制驱动器的读/写操作。

一般来说，一个硬盘控制器组成如图 6-18 所示。

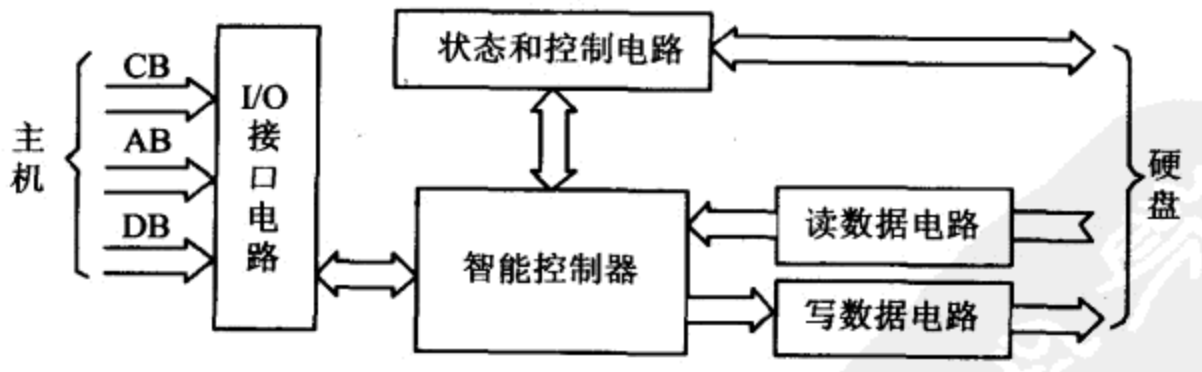


图 6-18　硬盘控制器组成示意图

（1）I/O 接口电路：主要用来和主机连接，实现控制器和主机之间的信息传送，其中包

括主机对控制器的寄存器读/写，对 ROM 芯片的直接寻址等。

（2）智能控制器：包括一个 CPU 芯片、一个 DMA 芯片和一个专用的硬盘控制器芯片。智能控制器主要用来实现对温盘的智能控制和信号的传送、处理等。

（3）状态和控制电路：状态和控制电路是智能控制器的外围电路。智能控制器通过它们监测温盘的状态并发出控制信号。

（4）读/写控制电路：读/写电路可以控制主机对温盘的数据读/写操作。

5. 硬盘驱动器接口

目前使用较多的硬盘驱动器接口有 IDE、EIDE 和 SCSI 等。

（1）IDE 接口：IDE（Integrated Drive Electronics）集成驱动器电子部件，它的最大特点是把控制器集成到驱动器内，因此在硬盘适配器中不再有独立的控制器部分。其好处把控制器和驱动器电路集成在一起，可以消除驱动器和控制器之间的数据丢失问题，使数据传输十分可靠。这就可以使得每磁道的扇区数提高到 30 以上，从而增大可访问容量。

在 IDE 接口中，除了对 AT 总线上的信号做必要的控制之外，基本上是原封不动地送往硬盘驱动器。因此，IDE 实际上是系统级的接口，故有时也称 IDE 接口为 ATA 接口（AT 嵌入式接口）。

由于把控制器集成到驱动器内部，硬盘适配器已变得十分简单，因此在新一代的微机系统中常把适配电路集成到系统板上，并留有专门的 IDE 连接器插口。IDE 接口最初只可以连接两个容量不超过 528MB 的硬盘驱动器，且成本低廉，因此在 386、486 微机系统中非常流行。但大多数 IDE 接口不支持 DMA 数据传送，只能使用标准的 PC I/O 端口指令来传送所有的命令、状态和数据。几乎所有的 586 主板上都集成了两个 40 针的双排针 IDE 接口插座，分别标注为 IDE1 和 IDE2。

（2）EIDE 标准：IDE 标准只能管理不超过 528MB 的硬盘存储容量，这显然不能满足日益增强的存储容量需求。近年来，Western Digital 又开发了 EIDE 标准，即增强型 IDE 标准，以取代原有的 IDE 标准。

EIDE 接口较 IDE 接口有了很大改进，是目前最流行的接口。在 Pentium 系列计算机主板上就配有 EIDE 接口，用户不必再买单独的适配卡。EIDE 有以下显著优点。

- 允许更大存储容量：EIDE 标准支持的每个硬盘最高容量可达 8.4GB。
- 允许连接更多的外设：EIDE 接口通常提供两个插座，分别为主插座和辅插座，每个插座又可以连接主、从两个设备，故 EIDE 标准允许一个系统连接 4 个 EIDE 设备。主插座通常与高速局部总线相连，供硬盘使用。而辅插座则与 ISA 总线相连，供 CD-ROM 驱动器使用。
- 支持多种外设：EIDE 支持符合 ATAPI 标准的 CD-ROM 驱动器和磁带驱动器。
- 具有更高的数据传输速率：IDE 驱动器的最大突发数据传输率只有 3Mb/s，而 EIDE 支持的数据传输速率可达 11.1Mb/s。

（3）SCSI 接口：SCSI（Small Computer System Interface）是 Novell 公司生产的小型计算机系统接口。SCSI 接口可提供大量、快速的数据传输，尤其比 IDE 接口支持更多数量和更多类型的外围设备，使其能广泛应用于工作站、高档微机系统中。SCSI 接口不仅适用于硬盘，而且也适用于扫描仪、CD-ROM 驱动器及打印机等设备的连接。SCSI 接口的特点如下。

- 可同时连接 7 个外设。
- 可以 8 位、16 位或 32 位的形式进行数据传输。
- 可以连接的硬盘驱动器容量可达 100GB，数据传输率达 10Mb/s～160Mb/s。
- 成本较 IDE 和 EIDE 接口高很多。若用 SCSI 接口的设备必须和 SCSI 接口卡配合使用，SCSI 接口卡也比 IDE 和 EIDE 接口贵很多。
- SCSI 接口是智能化的，可以彼此通信而不增加 CPU 的负担。在 IDE 和 EIDE 设备之间传输数据时，CPU 必须介入，而 SCSI 设备在数据传输过程中起主动作用，并能在 SCSI 总线内部具体执行，直至完成再通知 CPU。

6.6.3 光盘存储器及其接口

1. 概述

伴随多媒体技术的发展，音频、视频、动画、图像等媒体元素越来越多地应用在计算机中，这些元素的数据量一般都很大，以它们为基础的多媒体软件也向大型化不断发展，这时以软盘为载体的移动存储已越来越不能满足用户的需要，这就产生了光盘。

光盘是指利用激光能量集中的特点，以光学方式进行信息读/写的圆盘。应用激光在某种介质上写入信息，然后再利用激光读出信息的技术称为光存储技术。如果光存储使用的介质是磁性材料，即利用激光在磁记录介质上存储信息，就称为磁光存储。70 年代研究出半导体激光器，解决了光源问题，又解决了读出微小信息位的光学伺服系统等关键技术问题，使光存储技术达到了实用化水平。

光盘片一般采用丙烯树脂，在盘片上溅射碲合金薄膜或涂上其他物质。利用激光能量可以高度集中的特点，在记录信息时，使用功率较强的激光光源，聚焦成小于 1μm 的光点照射到介质表面上，根据写入的信息来调制光点的强弱，使介质表面的微小区域温度升高，产生微小凹凸或其他几何变形，即改变表面的光反射性质。在从光盘中读出信息时，利用光盘驱动器中功率较小的激光光源照射，根据反射光强弱的变化经信号处理即可读出数据。因此，一般的光盘在写入信息时是一次性的，永久保存在盘片上。

光盘具有大容量、高速度、耐用的特点，与硬盘相比，光盘更容易携带；与软盘相比，光盘的存储容量更大，一片 5.25 英寸的光盘能容纳 450 张 3.5 英寸的软盘的信息，而读/写速度是软盘的 10 倍以上。

2. 光盘存储器的种类

目前常用的光盘从读/写方式上讲，主要有以下几种。

（1）只读型光盘（CD-ROM）

CD-ROM 是一次成型的产品，由一种称为“母盘”的原始盘压制而成，一张母盘可以压制数千张光盘。这种光盘盘片上的信息是由生产厂家一次制成，光盘出厂后用户只能读取其上的数据或程序，而不能写入修改。

CD-ROM 在娱乐界主要用于电视唱片和数字音频唱片，可以获得高质量的图像和高保真度的音乐。在计算机领域里主要用于检索文献数据库或其他数据库，也可用于计算机辅助教学等。光盘的存储容量一般为 650MB～760MB，可存放约 70 分钟的动态声像。

（2）只写一次型光盘（WORM）

WORM 是用专门的制作设备——光盘刻录机刻制而成。这种光盘可由用户写入信息，写入后可以多次读出，但只能写一次，而且信息写入后不能修故。因此被称为“写一次型”（简称 WORM）。主要用于计算机系统中的文件存档或写入信息不需修改的情况。

（3）可擦写型光盘

可擦写型光盘与 CD-ROM、WORM 的最本质区别是光盘可以像磁盘一样重复读/写，是很有发展前途的光盘存储器。对于存储在光盘上的信息可以根据操作者的需要而自由更改、读出、复制、删除等。MO 光盘、PD 光盘、CD-RW 光盘均属于此类，当然，这种光盘和光驱的价格也比较昂贵。

3. 光盘驱动器

（1）光盘驱动器的读/写原理

光盘存储器是利用激光束在记录表面上存储信息的。在光盘存储器中，信息的写入和读出都是由能够将激光束汇聚成直径为 μm 级的圆型光斑的光学头（又称光头或激光头）来完成，根据激光束及反射光的强弱不同，可以完成信息的读/写。它的读/写装置与光盘片的距离可比磁存储器磁头与盘片的距离大一些，是非接触型读/写性质的存储器。

光盘的读/写原理有形变、相变和磁光存储等。

- 形变：对于只读型和只写一次型光盘，写入时，将激光束聚焦成直径为小于 1μm 的微小光点，以其热作用融化盘表面上的光存储介质薄膜，在薄膜上形成凹坑。有凹坑的位置表示记录了“1”，没有凹坑的位置表示“0”。

读出时，在读出光束的照射下，在有凹坑处和无凹坑处反射的光的强度是不同的。利用这种差别，可以读出二进制信息。由于读出光束的功率只有写入光束功率的 1/10，因此不会融出新的凹坑。

- 相变：有些光存储介质在激光照射下，晶体结构会发生变化。利用介质处于晶态和非晶态区域内反射特性不同而记录和读取信息的技术，称之为“相变可重写技术”。
- 磁光（MO）存储：利用激光在磁性薄膜上产生热磁效应来记录信息，称为磁光存储，应用于可擦写光盘上。根据磁记录原理可以知道，在一定温度下，如果在记录介质的表面上加一个强度低于该介质矫顽力的磁场，则不会发生磁通翻转，也就不能记录信息。但介质的矫顽力可随温度而变，假如能够设法控制温度，降低介质的矫顽力，使其低于外加磁场强度，则将发生磁通翻转。磁光存储就是根据这一原理存储信息的，它利用激光照射磁性薄膜，被照射处温度上升，矫顽力下降，在外加磁场的作用下发生磁通翻转，使该处的磁化方向与外加磁场一致。通常把磁记录材料受热而磁性发生变化的现象称为“热磁效应”。

（2）光盘驱动器的基本组成

光盘驱动器以光盘为载体读取信息，光盘数据存放在连续的螺旋型轨道上。当激光束扫描光盘的轨道时，利用光学反射原理，通过相应的传感器将光学信号转换成数字信息，再由主机将光盘内容读出。

一般情况下，光盘驱动器应能完成光盘旋转电机、光学头径向寻址电机和光学头自动聚焦、自动跟踪的伺服控制，以及完成写入数据的编码、读出数据的解码、检错纠错和时序控

制等的通道控制。根据上述要求，光盘驱动器一般由光学头、主轴电机、步进电机、光驱伺服定位系统、微控制器组成，如图 6-19 所示。

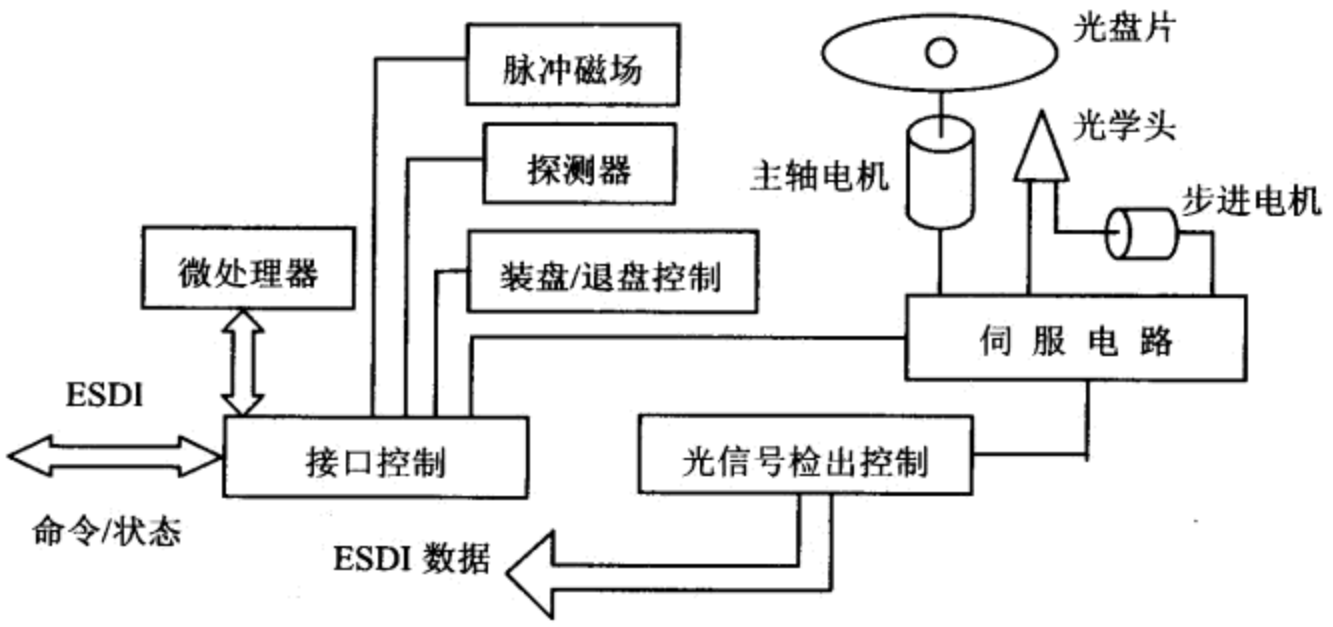


图 6-19　光盘驱动器结构框图

（1）光学头：由发射激光的激光二极管和聚焦透镜组成。

（2）主轴电机：是带动光盘盘片旋转的执行机构。

（3）步进电机：受控于伺服系统，执行寻找数据道命令。

（4）光驱伺服定位系统：由伺服电路和光信号检出电路组成。伺服电路由聚焦电路、寻道跟踪伺服驱动电路、直流电机伺服驱动电路组成。光信号检出电路包括对光学信号的锁相、滤波、整形、放大、数据编码、缓冲等。

（5）微控制器：由微处理器、接口控制电路组成，保证寻道过程中各信号的精确度。要完成从当前道到目的道的道差，需要分别计算加速、减速、匀速的道差和道数。在开始寻道时，根据分组的道数预置加、减、匀速度的初值，对寻道过程中的误差进行计数，将光点走过的距离信号及当前采用的加、减、匀速度的大小与速度预定值进行比较，并决定下一步到达的校正速度，直到到达目的道并转换为道跟踪伺服控制信号，整个寻道过程才结束。

光盘驱动器由于具有容量大、密度高、介质寿命长、能够进行非接触读/写和高速随机存取等一系列优点而得到迅速发展。

6.7　新型存储器技术

现代微机中 CPU 的速度提高非常快，导致快速 CPU 常常要等待慢速的主存，而且程序规模越来越大，只依靠主存已远远不能满足需要。为解决 CPU 与主存之间的速度匹配和存储容量问题，人们采用了许多新型的存储器技术来弥补两者在速度和容量方面的差距，其中常用的有多体交叉存储器、高速缓冲存储器（Cache）、虚拟存储器等。

6.7.1　多体交叉存储器

多体交叉存储器是从改进主存的结构和工作方式入手，设法提高其吞吐率，使主存速度

与 CPU 速度相匹配。

目前大多数主存都采用 MOS 存储器，其取数时间约为 70～300ns，比 CPU 速度要低一个数量级。当主存以单一存取控制方式工作时，CPU 每次访问主存，只能读出或写入一个信息，在访问过程中，CPU 大部分时间处于等待状态，效率很低。

多体交叉存储器的设计思想是在物理上将主存分成多个模块，每一个模块都包括一个存储体、地址缓冲寄存器和数据缓冲寄存器等，即它们都是一个完整的存储器。因此，CPU 就能同时访问各个存储模块，任何时候都允许对多个模块并行地进行读/写操作，从而提高整个存储系统的平均访问速度。

多体交叉存储器实际上是把整个主存地址空间划分为多个同样大小的地址分空间。为了提高主存的数据传输速率，可以采用交叉编址的方案，即利用主存地址的低 K 位来选择模块（可确定 2^K 个模块），高 m 位用来指定模块中的存储单元，这样连续的几个地址就位于相邻的几个模块中，而不是在同一个模块中，故称为“多体交叉编址”。于是 CPU 要访问主存的几个连续地址时，可使这几个模块同时工作，使整个主存的平均利用率得到提高。

6.7.2　高速缓冲存储器（Cache）

在多级存储系统中，为了解决 CPU 和主存之间的速度匹配问题，除了采用多体交叉存储等技术外，更有效的一种方法是在 CPU 与存储容量较大但操作速度较慢的主存之间，设置一个容量小但存取速度比主存快数倍的存储器，称为高速缓冲存储器（Cache）。

Cache 可以提高 CPU 访问存储器时的存取速度，减少处理器的等待时间，使程序员能使用一个速度与 CPU 相当而容量与主存相当的存储器。所以，这种方法对提高整个处理器的性能将起到非常重要的作用，比使全部主存都达到与 CPU 同样的速度要经济得多。

Cache 的全部功能由硬件实现，并且对程序员来说是“透明”的，Cache 的存在使得程序员面对一个既有 Cache 速度，又有主存容量的存储系统。CPU 不仅和 Cache 相连，而且和主存之间也要保持通路。

1. Cache 的工作原理

Cache 的工作原理是基于程序访问的局部性原理。对大量典型程序运行情况的分析结果表明，在一个较短的时间间隔内，由程序产生的地址往往集中在存储器逻辑地址空间的很小范围内。指令地址的分布本来就是连续的，再加上循环程序段和子程序段要重复执行多次。因此，对这些地址的访问就自然地具有时间上集中分布的倾向。数据分布的这种集中倾向不如指令明显，但对数组的存储和访问以及工作单元的选择都可以使存储器地址相对集中。这种对局部范围的存储器地址频繁访问，而对此范围以外的地址则访问甚少的现象，就称为程序访问的局部性原理。

根据该原理可以在主存和 CPU 通用寄存器之间设置一个高速但容量相对较小的存储器，把正在执行的指令地址附近的一小部分指令或数据，即当前最活跃的程序或数据从主存成批调入 Cache，供 CPU 在一段时间内随时使用，就一定能大大减少 CPU 访问主存的次数，从而加速程序的运行。

现代微机系统正是依据此原理不断地将与当前指令集相关联的一个不太大的后继指令集从内存读到 Cache，然后再高速传送给 CPU，从而达到速度匹配。CPU 对存储器进行数据

请求时，通常先访问 Cache。如果数据在 Cache 中，则 CPU 对 Cache 进行读/写操作，称为“命中”。由于局部性原理不能保证所请求的数据都在 Cache 中，所以便存在一个“命中率”，即 CPU 在任一时刻从 Cache 中可靠获取数据的几率。命中率越高，从 Cache 中正确获取数据的可靠性就越大。

当要访问的字在 Cache 中时，若是读操作，则 CPU 可以直接从 Cache 中读取数据，不涉及主存。若是写操作，则需改变 Cache 和主存中相应两个单元的内容。这时有两种处理办法，一种是 Cache 单元和主存中相应单元同时被修改，称为“直通存储法”。另一种方法是只修改 Cache 单元的内容，同时用一个标志位作为标志，当有标志位的信息块从 Cache 中移去时再修改相应的主存单元，把修改信息一次写回主存，称为“写回发”。显然直通存储法比较简单，但对于需要多次修改的单元来说，可能导致不必要的主存复写工作。

若数据不在 Cache 中，则 CPU 直接对主存进行操作。若是读操作，则把主存中相应的信息块送到 Cache 中，在送字块到 Cache 的同时就把所需的字送 CPU，不必等待整个块都装入 Cache。这种方法称为“直通取数”。若是写操作，则将信息直接写入内存。一般情况下，此时主存中的相应块并不调入缓存，因为一个写操作所涉及的往往是程序中某个数据区的一个单元，其访问的局部性并不明显。

主存和 Cache 的存储区均划分成块（Block），每块由多个信息字组成，两者之间以块为单位交换信息。Cache 中应能容纳多个信息块。Cache 的内容是在读/写过程中逐步调入的，是主存中部分内容的副本。信息块调往 Cache 时的存放地址与它在主存时的内存不可能一致，两种地址之间有一定的对应关系，这种对应关系通常称为地址映像函数。将主存地址变换成 Cache 地址的地址变换过程，一般是通过用硬件组成的地址变换机构按照所采用的地址映像函数来自动完成的。

一般来说，Cache 的存储容量比主存的容量小得多，但不能太小，太小会使命中率太低；也没有必要过大，过大不仅会增加成本，而且当容量超过一定值后，命中率随容量的增加将不会有明显的增长。只要 Cache 的空间与主存空间在一定范围内保持适当比例的映射关系，Cache 的命中率还是相当高的。一般规定 Cache 与内存的空间比为 4:1000，即 128KB Cache 可映射 32MB 内存；256KB Cache 可映射 64MB 内存。在这种情况下，命中率都在 90%以上。至于没有命中的数据，CPU 只好直接从内存获取。获取的同时，也把它拷进 Cache，以备下次访问。

2. Cache 的基本结构

Cache 通常由相连存储器实现。相连存储器的每一个存储块都具有额外的存储信息，称为标签（Tag）。当访问相连存储器时，将地址和每一个标签同时进行比较，从而对标签相同的存储块进行访问。Cache 的 3 种基本结构如下。

（1）全相连 Cache

在全相连 Cache 中，存储的块与块之间，以及存储顺序或保存的存储器地址之间没有直接的关系。程序可以访问很多的子程序、堆栈和段，而它们是位于主存储器的不同部位上。因此，Cache 保存着很多互不相关的数据块，Cache 必须对每个块和块自身的地址加以存储。当请求数据时，Cache 控制器要把请求地址同所有地址加以比较，进行确认。这种 Cache 结构的主要优点是，它能够在给定的时间内去存储主存器中的不同的块，命中率高；缺点是每

一次请求数据同 Cache 中的地址进行比较需要相当的时间，速度较慢。

（2）直接映像 Cache

直接映像 Cache 不同于全相连 Cache，地址仅需比较一次。在直接映像 Cache 中，由于每个主存储器的块在 Cache 中仅存在一个位置，因而把地址的比较次数减少为一次。其做法是，为 Cache 中的每个块位置分配一个索引字段，用 Tag 字段区分存放在 Cache 位置上的不同的块。单路直接映像把主存储器分成若干页，主存储器的每一页与 Cache 存储器的大小相同，匹配的主存储器的偏移量可以直接映像为 Cache 偏移量。Cache 的 Tag 存储器（偏移量）保存着主存储器的页地址（页号）。以上可以看出，直接映像 Cache 优于全相连 Cache，能进行快速查找；其缺点是当主存储器的组之间做频繁调用时，Cache 控制器必须做多次转换。

（3）组相连 Cache

组相连 Cache 是介于全相连 Cache 和直接映像 Cache 之间的一种结构。这种类型的 Cache 使用了几组直接映像的块，对于某一个给定的索引号，可以允许有几个块位置，因而可以增加命中率和系统效率。

3. 替换算法

替换算法发生在有冲突发生时，即新的主存页需要调入 Cache，而 Cache 已没有空闲空间可用时。这时替换机构应根据某种算法指出应移去的块，再把新块调入。替换机构是根据替换算法设计的。替换算法很多，一个好的算法首先要看访问 Cache 的命中率如何，其次要看是否容易实现。替换算法的目标是使 Cache 获得最高的命中率，就是让 Cache 中总是保持着使用频率高的数据，从而使 CPU 访问 Cache 的成功率最高。替换算法主要有 4 个。

（1）随机替换算法：这是一种最简单的替换算法。当需要找替换的信息块时，它是用随机数发生器产生一个随机数，它就是被替换的块号。由于这种算法完全不反映程序的局部性特点，没有考虑信息块的历史情况和使用情况，故其命中率很低，已无人使用。

（2）先进先出算法（FIFO）：这种算法是把最早进入 Cache 的信息块给替换掉。为了实现这种算法，需要在地址变换表中设置一个历史位，每当有一个新块调入 Cache 时，就将已进入 Cache 的所有信息块的历史位加 1。于是当需要进行替换时，只要挑选历史位中数值最大的信息块作为被替换块即可。这种算法在一定程度上能反映程序的局部性特点，比随机算法好，但由于这种方法只考虑了历史情况，并没有反映出信息的使用情况，所以其命中率也并不高。原因很简单，最先进来的信息块，或许就是经常要用的块。

（3）近期最少使用算法（LRU）：这种算法是把最近使用最少的信息块替换掉，这就要求随时记录 Cache 中各信息块的使用情况。为了反映每个信息块的使用情况，要为每个信息块设置一个计数器，以便确定哪个信息块是近期最少使用的。

LRU 算法也可用堆栈来实现，所以又被称做堆栈型算法。当所设堆栈已满，又有一个信息块要求调入 Cache 时，首先检查堆栈中是否已经有这一块。如果有，则将这一信息块从堆栈中取出压入堆栈的栈顶；如果没有，则将该信息块直接压入栈项。于是原来在栈底上的信息块就成为被替换的信息块而被压出了堆栈，这样就可以保证任何时候栈顶上的信息块总是刚被访问过的块，而栈底上的块总是最近没有被访问过的块。这种算法能较好地反映程序的局部性特征，可以获得较高的命中率。但由于近期使用少的，未必是将来使用最少的，所以这种算法的命中率比 FIFO 有所提高，但并非最理想。

（4）优化替换算法：这是一种理想算法，实现起来难度较大。因此，只作为衡量其他算法的标准，这种算法需要程序运行两次，第一次分析地址流，第二次才真正运行程序。

6.7.3 虚拟存储器

1. 虚拟存储器的概念

虚拟存储器（Virtual Memory）是以存储器访问的局部性为基础，建立在主存—辅存物理体系结构上的存储管理技术。随着计算机系统规模的扩大和复杂程度的增加，为了使用户尽量扩大可使用的存储空间，并能对其自动管理和调度，使在用户心中好像只有一个大容量、高速度、使用方便的存储器，而没有主存、辅存之分，便产生了虚拟存储器的概念。

由主存和辅存构成的两级存储层次就是虚拟存储器的基本结构，但是必须说明，在虚拟存储器中不仅有主存和辅存两级存储器，而且还必须有辅助软硬件来对主存与辅存之间的数据交换实现控制功能。任何时候只把程序的活跃部分存放在主存储器中，其他的大量非活跃部分的程序都存放在辅助存储器或其他外部设备中。在程序执行过程中，程序的活跃部分和非活跃部分是动态地改变的，什么时间主存与辅存之间需要交换信息，并不是由用户自己来安排，而是完全在辅助硬件的控制之下进行。因此，可以这样说，只有具备了辅助软硬件的主辅存两级存储层次才构成一个虚拟存储系统。

在没有虚拟存储器的系统中，主存储器的容量限制了在机内可运行程序的大小。如何能在计算机内运行比主存储器容量大、甚至大得多的程序，便成为计算机工作者们追求的目标。虚拟存储器便是实现利用小容量主存运行大规模程序的有效办法之一。

虚拟存储器指的是为了扩大容量，把辅存当作主存使用，它将主存和辅存的地址空间统一编址，形成一个庞大的存储空间。程序运行时，用户可以访问辅存中的信息，可以使用与访问主存同样的寻址方式，所需要的程序和数据由辅助软件和硬件自动调入主存，这个扩大了的存储空间，就称为虚拟存储器。之所以叫做“虚拟”，是因为这样的主存并不是真实存在的。

在一个虚拟存储系统中，展现在 CPU 面前的存储器容量并不是实存容量加上辅存容量，而是一个比实存大得多的虚存空间，它与实存和辅存空间的容量无关，取决于机器所能提供的虚存地址码的长度。例如，某计算机系统中，存储器按字节编址，可提供的虚存地址码长 48 位，能提供的虚存空间为 2^{48}≈256TB，这是任何计算机系统中主存储器所不可能达到的容量，在该机内运行的程序可达到 256TB。

CPU 以逻辑地址访问主存，由辅助硬件和软件确定逻辑地址和物理地址的对应关系，判断这个逻辑地址指示的存储单元内容是否已装入主存。如果在主存，CPU 就直接执行该部分程序或数据；如果不在主存，系统存储管理软件和辅助硬件就会把访问单元所在的程序块从辅存调入主存，并把逻辑地址转换成实地址。

主存—辅存层次的虚拟存储和 Cache—主存层次有很多相似之处，但虚拟存储器和 Cache 仍有很重要的区别。

（1）Cache 用于弥补主存与 CPU 的速度差距，而虚拟存储器是用来弥补主存和辅存之间的容量差距。

（2）Cache 每次传送的信息块是定长的，只有几十字节，而虚拟存储器信息块划分方案

有很多，有分页、分段等，长度可以很大，达几百或几千字节。

（3）CPU 可以直接访问 Cache，而 CPU 不能直接访问辅存。

（4）Cache 存取信息的过程、地址变换和替换算法等全部由辅助硬件实现，并对程序员是透明的；而虚拟存储器是由辅助软件（操作系统的存储管理软件）和硬件相结合来进行信息块的划分和程序的调度。

2. 页式虚拟存储器

以页为信息传送单位的虚拟存储器叫做“页式虚拟存储器”。在页式虚拟存储器中，将虚拟空间和主存空间机械地分成大小固定的页。页的大小通常随机器而异，一般为 512B 或几 KB 不等，通常都取 2 的整数次幂个字节。这种划分是一种逻辑划分，可以由管理软件任意指定。因此，页的起始地址都落在低位地址为零的地址上。

虚存空间中所划分的页被称为“虚页”，而主存空间中所划分的页被称为“实页”。页面由 0 开始顺序编号，分别称为虚页号和实页号。因此，虚拟地址由两部分组成：页号（页表索引）和页内地址（偏移地址），即高位字段为虚页号，低位字段为页内地址。物理地址也分成实页号和页内地址两部分，页内地址的长度（位数）由页面大小决定，实页号的长度取决于主存的容量。因为虚存和实存的页面大小一致，所以信息由虚拟页向实际存储器中的物理页调入时，以页为单位，页边界对齐，页内地址无需修改就可以直接使用。因此，虚—实地址的转换主要是虚页号向实页号的转换，这个转换关系由页表给出。为实现地址变换，通常要建立一张虚地址页号与实地址页号的对照表，记录程序的虚页面调入主存时被安排在主存中的位置，这张表叫“页表”，它是存储管理软件根据主存运行情况自动建立的。一般情况下，因为虚存空间比实存空间大得多，故虚页号要比实页号的长度长。

虚拟存储模式中，每一个程序都有一个自己独立的虚存空间。虚存空间是对每个程序而言的，也就是说，同一个系统运行不同的程序时会有不同的虚存空间。在程序运行时，存储管理软件要根据主存的运行情况，为每一个程序建立一张独立的页表，存放在主存的特定区域，而页表本身就是一个页。页表信息字按虚页号顺序排列，而在每个虚页中，都有一个描述页表状况的信息字，称为“页表信息字”，其中存放该虚页对应到实存中的页号以及一些其他信息。

当一个虚页号向实页号转换时，首先应找到该程序的页表区的首地址，然后按虚页号顺序找到该页的页表信息字。由于页表被保存在内存的特定区域中，程序投入运行时，便由存储管理软件把这个程序的页表区首地址送到页表的基址寄存器中。

3. 段式虚拟存储器

现在程序编制大都采用模块化程序设计，将一个复杂的程序按其逻辑功能分解成一系列相互关联但功能独立的简单模块。段式虚拟存储器就是以程序的逻辑结构所自然形成的段作为主存分配的单位，来进行存储器管理的一种虚拟存储器。其中每个段的长度可不同，可以独立编址。有的段甚至可以事先不确定大小，而在执行时动态地决定。程序运行时，以段为单位整段从辅存调入主存，一段占用一个连续的存储空间，CPU 访问时仍需进行虚—实地址的转换。它与页式虚拟存储器技术十分相似，每个程序都有一个段表，存放程序段装入主存的状态信息，主要有段号、段起点、装入位置、段长度等。程序运行时，要先根据段表确定所访问的虚段是否已经存在于主存中。如果没有，则先将其调入主存；如果已在主存中，则

进行虚—实转换，确定其在主存中的位置。

段式虚拟存储器配合了模块化程序设计，使各段之间相互独立，互不干扰。程序按逻辑功能分段，各有段名，便于程序段公用和按段调用，可提高命中率。缺点是由于各段长度不等，虚段调往主存时，主存分配困难。

4. 段页式虚拟存储器

页式虚拟存储器和段式虚拟存储器各有优缺点。页式虚拟存储器每页长度固定且可顺序编号，页表设置很方便。虚页调入主存时，主存空间分配简单，开销小，页面长度较小，主存空间可以得到充分地利用，因而得到广泛应用。缺点是程序不可能正好是页面的整数倍，最后一页的零头无法利用而浪费，同时机械分页无法照顾程序内部的逻辑结构，几乎不可能出现一页正好是一个逻辑上独立的程序段，指令或数据跨页的状况会增加查页表的次数和页面失效的可能性。

段式虚拟存储器因段与程序功能相对应，模块可独立编址，便于多人同时编制、修改、调试，段长度不固定，增删时对其他段不会产生影响。各段有段名，便于公用。按段调度可提高命中率。不足之处是各段长度不等，调入主存时主存空间分配工作比较复杂，段与段之间的内存空间常常不好利用而造成浪费。

为充分发挥页式和段式虚拟存储器各自的优点，可把两者结合起来形成段页式虚拟存储器，将存储空间仍按程序的逻辑模块分段，以保证每个模块的独立性和便于用户公用。每段又划分为若干个页，页面大小与实存页面相同。虚地址的格式包括段号页号、页号和页内地址 3 部分。实地址则只有页号和页内地址。虚存与实存之间信息调度以页为基本单位。每个程序有一张段表，每段对应一张页表。CPU 访问时，由段表指出每段对应的页表的起始地址，而每一段的页表可指出该段的虚页在实存空间的存放位置（实页号），最后与页内地址拼接即可确定 CPU 要访问的信息的实存地址。这是一种较好的虚拟存储器管理方式。

本章小结

存储器系统是微型计算机的重要组成部分，主要用来存储指令和各种数据。存储器有各种不同的分类方法，要重点掌握主存、辅存和高速缓存的分类方法，掌握存储器系统的层次结构。随机存取存储器 RAM 和只读存储器 ROM 要熟悉其基本结构和工作原理，并掌握存储器容量的形成与寻址方法，以及存储器与微处理器的连接技术。

本章还介绍了微型计算机常用的辅助存储器（包括软盘、硬盘和光盘存储器）的基本结构、工作原理及接口，以及闪速存储器、高速缓冲存储器、虚拟存储器等新型存储器技术，这些技术都是现代微型计算机的重要技术，读者一定要深刻理解。

思考与练习题

1. 什么是计算机的内存和外存？它们有什么区别？
2. 简述存储器系统的层次结构，并说明为什么会出现这种结构？
3. 存储器的基本结构是怎样的？

4. 静态存储器和动态存储器的最大区别是什么，它们各有什么优缺点？
5. 常用的存储器地址译码方式有哪几种，各自的特点是什么？
6. 某 SRAM 的一单元中存放有一个数据（如 3FH），CPU 将其取走后，该单元的内容是什么？
7. 已知某微机系统的 RAM 容量为 4K×8 位，首地址为 4800H，求其最后一个单元的地址。
8. ROM、PROM、EPROM、E^2PROM 各有什么特点？各用于何种场合？
9. 设有一个具有 14 位地址和 8 位数据的存储器，问：
   （1）该存储器能存储多少字节的信息？
   （2）如果存储器由 8K×4 位 RAM 芯片组成，需要多少片？
   （3）需要地址多少位做芯片选择？
10. 用 16K×1 位的 DRAM 芯片组成 64K×8 位的存储器，要求画出该存储器组成的逻辑框图。
11. 半导体存储器在与微处理器连接时应注意哪些问题？
12. 硬盘驱动器有哪几种接口标准，它们各有什么优缺点？
13. 计算机中为什么要采用高速缓冲存储器 Cache？
14. 简述虚拟存储器的概念。

第 7 章 输入/输出接口技术

输入/输出接口是计算机外部设备与系统通信的控制部件，位于总线和外部设备之间，起到信息的转换和数据传递的作用。本章主要讲解现代微机系统中的输入/输出（I/O）接口技术，包括输入/输出接口的结构和功能，CPU 与接口传递的信息，I/O 端口的编址方式和输入/输出控制方式等主要内容。

通过本章的学习，读者应该理解输入/输出接口的基本概念和功能，掌握输入/输出的控制方式，理解 CPU 与输入/输出接口之间传递的信息类型，熟悉 CPU 与外部设备之间数据传送方式的基本原理和特点，为后面可编程接口的应用奠定基础。

7.1 输入/输出接口技术概述

输入/输出（I/O）设备在计算机系统中占有重要作用，是实现人机交互和各种形式的输入/输出不可缺少的部分。计算机要处理的程序和数据都要通过输入设备进行输入，产生的结果需要通过输出设备进行输出。常用的输入设备有鼠标、键盘、扫描仪等，常用的输出设备有打印机、显示器、绘图仪等。通常将输入/输出设备统称外部设备（或 I/O 设备），简称外设。

外设种类繁多，这些外设在信息格式、工作原理、驱动方式及工作速度等方面存在很大差别，处理数据的速度与 CPU 的速度也不匹配，因此各种外设要与 CPU 连接，需要中间电路——I/O 接口来解决这一系列的问题。

7.1.1 输入/输出接口的结构与功能

1. 输入/输出接口的结构

所谓接口是指 CPU 和存储器、外部设备或者两种外部设备之间，或者两种机器之间通过系统总线进行连接的逻辑部件（或称电路），它是 CPU 与外界进行信息交换的中转站。源程序和原始数据通过接口从输入设备（如键盘）送入；运算结果通过接口通过输出设备（如 CRT 显示器、打印机）输出；设备的控制命令（如步进电机的启/停命令）通过接口发送出去；现场信息（如温度值、转速值）通过接口取进来。

I/O 接口中有为传送数据设置的数据寄存器；有为控制接口电路的工作方式设置的控制寄存器；有为反映外设或接口电路工作状态设置的状态寄存器；还有供 CPU 与外设间用中断方式传送信息所需要的逻辑电路，如中断允许寄存器、中断屏蔽寄存器等，这些寄存器使得 CPU 能及时了解外设的工作状态，实现 CPU 与外设间正确的数据传送。I/O 接口的基本结构示意图如图 7-1 所示。

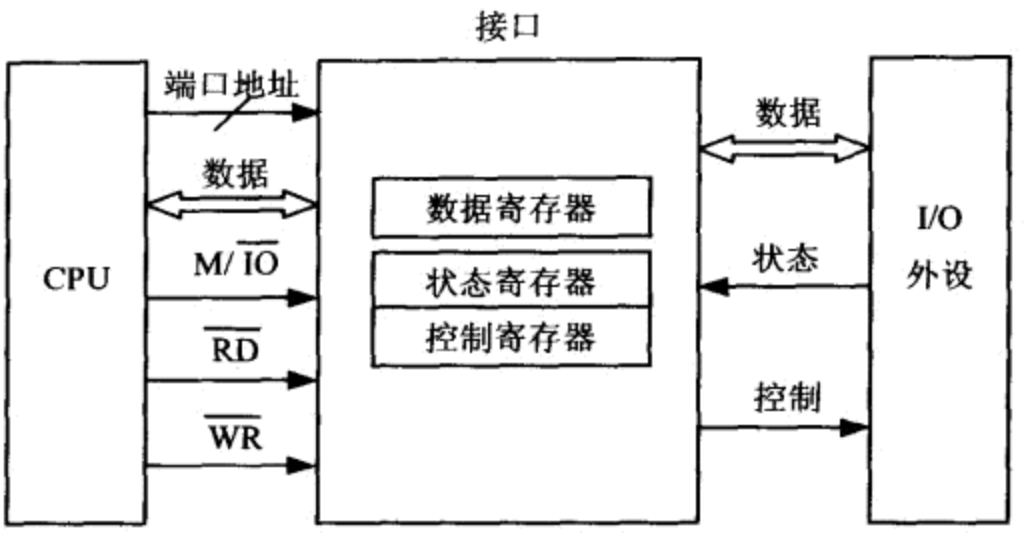


图 7-1　I/O 接口的基本结构示意图

（1）数据寄存器：起数据缓冲作用。输入时，保存外设向 CPU 发送的数据（称为数据输入寄存器）；输出时，保存 CPU 向外设发送的数据（称为数据输出寄存器）；有些数据寄存器具有输入和输出两种功能，由读/写控制决定输入还是输出。

（2）状态寄存器：反映外设工作状态。

（3）控制寄存器：确定接口电路的工作方式，选择数据传送方向（输入或输出）及交换信息方式（查询或中断方式）。

（4）命令译码、端口地址译码及控制电路：负责选择端口，对 CPU 送来的命令进行译码，能用中断方式传送信息。

接口电路的功能越强，内部寄存器的种类和数量就越多，电路结构就越复杂，使用接口时要发送的控制命令就越多，程序也就越复杂。

2. 输入/输出接口的功能

输入/输出接口电路在 CPU 和外设之间，CPU 与外设之间的数据交换必须通过接口来完成。I/O 接口一般应具有以下基本功能。

（1）数据的寄存和缓冲

外设工作速度与 CPU 相比相差甚远，为充分发挥 CPU 的工作效率，通过在接口电路中设置数据寄存器或用 RAM 芯片组成数据缓冲区，可以在一定程度上缓解外设与 CPU 处理速度上的差异，也为主机和外设间进行批量数据交换创造了条件。

（2）信号电平转换

CPU 使用的都是 TTL 电平或 CMOS 电平，而外设大都是复杂的机电设备，其往往不能用 TTL 电平或 CMOS 电平驱动，有些复杂的机电设备必须有自己的电源和信号电平。这就需要用 I/O 接口进行信号的电平转换。

（3）信息格式转换

由于外设传送的信息可以是模拟量、数字量或开关量，而计算机只能处理数字信号，要交流的信息可能格式完全不同。因此，I/O 接口必须通过模/数转换（A/D）或数/模转换（D/A）将信息变换成适合对方的形式。

即使大多数外设使用的都是数字量，但与计算机通信时，仍然会存在数据格式的转换问题。因为外设使用的数据与主机系统总线上传送的数据在数据位数、格式等方面往往也存在

很大差异。例如主机系统总线上传送的是 8 位、16 位或 32 位并行数据，而外设采用的却是串行数据传送方式，这就要求 I/O 接口进行并—串转换或者串—并转换。

（4）设备选择

系统中所有外设都通过 I/O 端口接在系统总线上，而在同一时间里只能允许一台外设与 CPU 交换信息，这就要借助于接口的地址译码来选定外设。只有被选定的外设才能与 CPU 进行数据交换，而未被选中的外设 I/O 接口呈现为高阻状态，与总线隔离。

（5）对外设的控制与检测

接口接收 CPU 送来的命令字或控制信号、定时信号，实施对外设的控制和管理，外设的工作状态或应答信号也及时返回给 CPU，以“握手联络”方式来保证 CPU 和外设输入/输出操作的同步。如 I/O 接口提供“READY”、“BUSY”等状态，反映 CPU 何时能与外设进行数据传送。

（6）产生中断请求及 DMA 请求

为了满足实时性和外设与主机并行工作的要求，需要采用中断传送的方式，为了提高传送速率有时又采用 DMA 传送方式，这就要求接口有产生中断请求和 DMA 请求的能力，并能管理中断和 DMA。

（7）可编程功能

接口电路可以简单到只由一块中小规模或大规模集成电路芯片组成，也可能复杂到不亚于一块系统主板，其类型多种多样，但其核心部分往往是一块或数块大规模集成电路芯片，这类芯片称为接口芯片。现在大多数接口芯片都是可编程的，这样在不改变硬件的情况下，只需修改程序就可以改变接口的工作方式，大大增加了接口的灵活性和可扩充性，使接口向智能化方向发展。

7.1.2 CPU 与 I/O 接口之间传递的信息类型

CPU 与 I/O 设备之间要传送的信息，通常包括数据信息、状态信息和控制信息。

1. 数据信息

CPU 与外设交换的基本信息是数据，数据信息通常为 8 位或 16 位，大致可分为以下 3 种基本类型。

（1）数字量：数字量可以是以二进制形式表示的数据，或是以 ASCII 码表示的数据及字符，通常以字节为单位。如由键盘、光电输入机、读卡机等输入的信息以及从 CPU 到打印机、穿孔机输出的信息等。

（2）模拟量：当计算机系统用于实时控制时，输入的信息往往是一些现场连续变化的物理量，如温度、湿度、位移、压力、流量等。这些物理量由传感器转换成电量，经过放大电路成为需要的模拟电压或电流，这些模拟量通过模/数转换器（A/D）转换为数字量送往计算机；计算机输出的数字量要经过数/模转换器（D/A）转换成模拟量，输出给控制执行装置。

（3）开关量：开关量通常可表示两个状态，如开关的闭合或断开、电机的运转或停止等，可用一位二进制数“0”或“1”表示。

2. 状态信息

状态信息反映当前外设接口或外设的状态，CPU 可根据这些状态信息决定对外设进行操

作或控制。CPU 用程序查询 I/O 设备的状态时，实际就是将状态位信息读至 CPU 进行分析。对于输入设备来说，常用“设备准备好”（READY）信号来表明待输入的设备是否准备就绪；对于输出设备来说，则用输出设备缓冲区是否“空”（EMPTY）来表示可以输出新的数据，或用“忙”（BUSY）信号表示输出设备是否处于空闲状态。如为空闲状态，CPU 可输出给它新的数据；否则 CPU 要等待外设工作结束，才能输出新的数据。

不同外设，其状态信息的多少和种类也不同，设备越复杂状态信息就越多。

3. 控制信息

控制信息是 CPU 通过接口发送给外设的，用于控制外设的启动或停止，设置外设的工作方式等。

状态信息和控制信息与数据信息是不同性质的信息，必须分别传送。为了使它们相互区别开来，它们必须有自己的端口。数据输入/输出需要一个端口；外设状态需要一个端口，以便 CPU 对其进行读入；控制信息往往也需要一个端口输出，以便 CPU 对外设进行控制。所以，一个外设往往有几个端口，各端口有自己的地址，CPU 寻址的是具体的端口地址，而不是笼统的外设。通常一个外设的数据端口是 8 位的，而状态信息与控制信息只有 1～2 位，因此，不同外设的状态信息或控制信息可共用一个端口。

从广义上讲，控制信息、状态信息以及数据信息都是数据信息，都是通过数据总线进行传送，但 3 种信息在 I/O 接口中占用的寄存器不同：存放数据信息的是数据寄存器（数据端口）；存放状态信息的是状态寄存器（状态端口）；控制信息则存放在控制寄存器（控制端口）中。

7.1.3　I/O 端口的编址方式

接口中数据寄存器、控制寄存器、状态寄存器都有自己的地址，称为端口地址，便于 CPU 选择端口，交换各种信息。

一个接口可以有多个端口，如命令端口、状态端口和数据端口等，分别对应于控制寄存器、状态寄存器和数据缓冲寄存器等。CPU 通过这些端口向接口电路中的寄存器发送命令、读取状态和传送数据。

在接口电路中，一个端口可以对应一个或多个寄存器，此时由内部控制逻辑根据程序指定的 I/O 端口地址和数据标志位选择相应寄存器进行读/写操作。也就是说，访问端口就是访问接口电路中的寄存器。这样，I/O 操作实质上转化为对 I/O 端口的操作，即 CPU 所访问的是与 I/O 设备相关的端口，而不是 I/O 设备本身。对 I/O 端口的访问，取决于 I/O 端口的编址方式。I/O 端口地址常用的编址方式有两种：统一编址和独立编址。

1. 统一编址

统一编址也称为存储器映射编址方式，是把每一个端口视为一个存储单元，I/O 端口与存储单元在同一个地址空间中进行编址，如图 7-2 所示。

I/O 端口地址与存储单元地址形式完全相同，CPU 访问端口就如同访问存储器，只是地址编号不同而已，所有访问内存的指令都适用于 I/O 端口。端口地址被映像到存储空间作为存储空间的一部分，这种编址方式既具有一定优点也具有一定缺点。

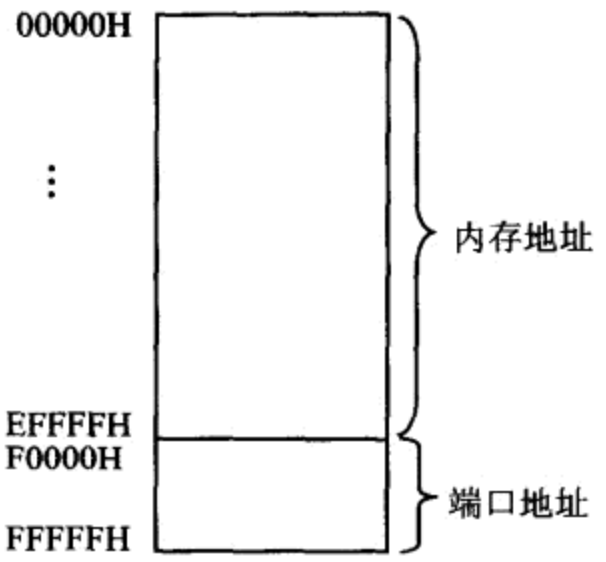


图 7-2 I/O 端口与内存单元统一编址

统一编址的优点是：CPU 对所有外设的操作与对存储器的操作完全相同，可使用全部的存储器操作指令，而不必使用专用的 I/O 指令，这可大大增强系统的 I/O 功能，使访问外设端口的操作方便、灵活；端口有较大的编址空间。统一编址的缺点是：I/O 端口地址占用了存储器的一部分地址空间，使可用的主存空间减少；寻址速度比专用的 I/O 指令慢，端口指令较长，执行速度较慢；难以区分访问主存还是访问外设，造成程序阅读困难；地址译码电路复杂。

2. 独立编址

这种编址方式是将 I/O 端口与存储器分别单独编址，两者的地址空间是互相独立、互不影响的。例如，8086 系统的内存地址范围为 00000H~FFFFFH，而外设端口的地址范围为 0000H~FFFFH，CPU 在访问内存和外设时，需要提供不同的控制信号来区分当前要进行操作的对象是内存还是外设。采用这种编址方式，CPU 访问 I/O 端口必须采用专用的 I/O 指令，所以也叫专用 I/O 指令方式。

独立编址的优点是：I/O 端口不占用内存单元地址，节省内存空间；由于系统需要的 I/O 端口寄存器一般比存储器单元要少得多，故 I/O 地址线较少，因此 I/O 端口地址译码较简单，寻址速度快。缺点是：专用 I/O 指令类型少，远不如存储器访问指令丰富，使程序设计灵活性较差，且使用 I/O 指令一般只能在累加器和 I/O 端口交换信息，处理能力不如统一编址方式强。

7.2 I/O 数据传送方式

随着计算机技术的飞速发展，计算机系统中输入/输出设备的种类越来越多，速度差异越来越大，对这些设备的控制也变得越来越复杂，在计算机 CPU 与外设进行数据（数据信息、状态信息和控制信息）传送时必须采用多种输入/输出控制方式，才能满足各类外设的要求，才能保证高效、可靠地数据传送。在现代微型计算机中，采用的 I/O 数据传送方式主要有无条件传送方式、查询传送方式、中断传送方式和 DMA 传送方式。

7.2.1 无条件传送方式

无条件传送方式也称为同步传送方式，主要用于对简单外设进行操作，或者外设的定时

是固定的还是已知的场合。对于这类外设，在任何时刻均已准备好数据或处于接收数据状态，因此程序可以不必检查外设的状态，就可以进行输入/输出操作。当 I/O 指令执行后，数据传送便立即进行。这是一种最简单的传送方式，所需要的硬件和软件都较少。

图 7-3 是无条件传送输入方式的典型例子，要将几个按键开关的状态输入 CPU 时，将这些开关连接到一个三态缓冲器，缓冲器的输出端接到 CPU 的数据总线，构成一个最简单的输入端口。CPU 可随时执行输入/输出指令，它使 M/$\overline{IO}$、$\overline{RD}$ 和片选信号 $\overline{CS}$ 同时变成低电平，它们相与后打开缓冲器的三态门，使各开关的当前状态以二进制的形式出现在数据总线上，然后读入 CPU，检查这个字节各位的内容，便能了解各开关的当前状态。在其他时刻，三态门呈高阻态，将开关和数据总线隔离。

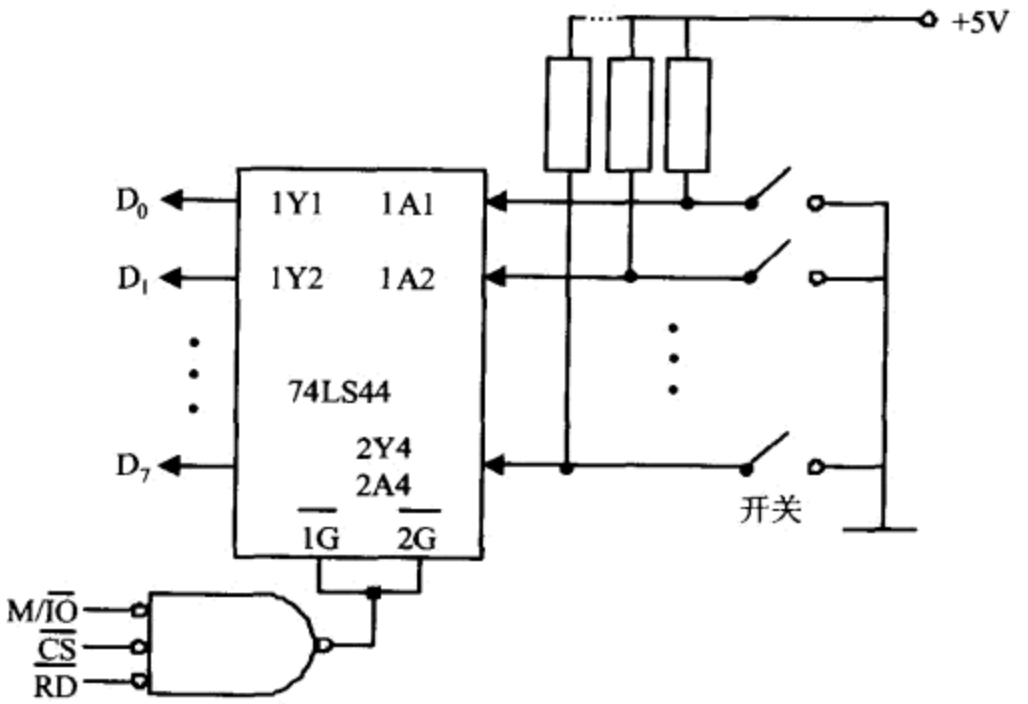


图 7-3　简单输入端口

无条件传送输出方式的典型示例如图 7-4 所示，用程序来控制 LED 显示器的亮灭。通常用一个由锁存器构成的输出端口来把 LED 接到计算机的数据总线上，并串接一个限流电阻。图中各 LED 的阴极接地，成为共阴连接。这样，需要点亮某个 LED 时，只要用输出指令向此端口输出一个字节，该字节中相应于点亮的 LED 的位是 1，其余各位为 0。显然，在这个例子中，LED 总是处于可用状态，随时都可以向这个端口输出数据，控制各 LED 的亮灭。

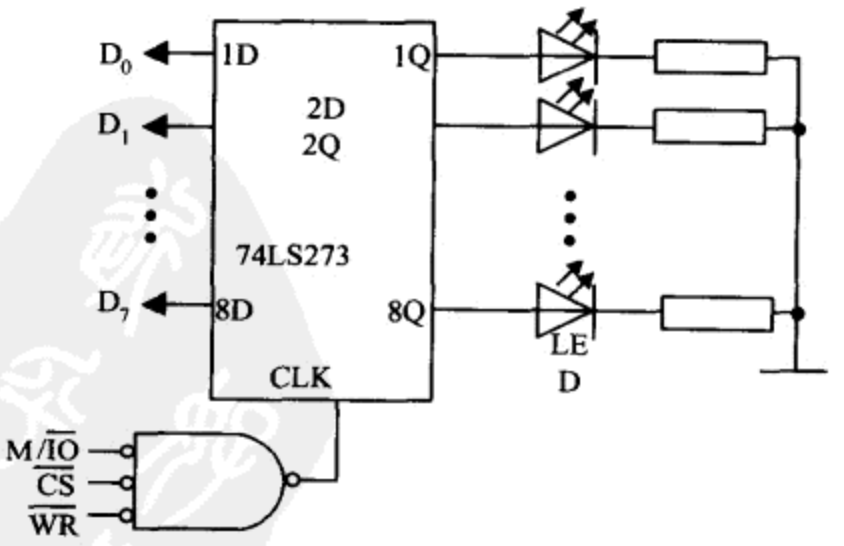


图 7-4　简单输出端口

无条件传送方式虽然软、硬件实现简单，但具有一定的局限性，且 CPU 与外设工作不同步时，传输数据不可靠，故仅适用于慢速设备。

7.2.2 查询传送方式

查询传送方式也称为条件传送方式。一般情况下，当 CPU 用输入/输出指令与外设交换数据时，很难保证输入设备总是准备好了的数据，或者输出设备已经处在可以接收数据的状态。为此，在开始传送前，必须先确认外设已做好准备，才能进行传送。

采用这种方式传送数据前，CPU 要先执行一条输入指令，从外设的状态口读取它的当前状态。如果外设未准备好数据或处于忙状态，则程序要反复执行读状态指令，不断检测外设状态。对于输入而言，当外设准备好时，则使 I/O 接口的状态端口的“准备好”标志有效，CPU 可以接收数据；对于输出而言，当外设取走数据后，则 I/O 接口将状态端口对应的“忙”标志清除，表明外设处于空闲状态，外设可以接收下一个数据。

1. 查询式输入

查询式输入程序流程如图 7-5 所示。程序首先利用 IN 指令读入 I/O 端口的状态，CPU 查询输入设备是否已准备好数据。若未准备好（READY=0），则 CPU 循环读状态端口，直到准备好（READY=1）后才退出循环，用 IN 指令读输入设备的数据(读输入数据端口)，完成一次数据输入。

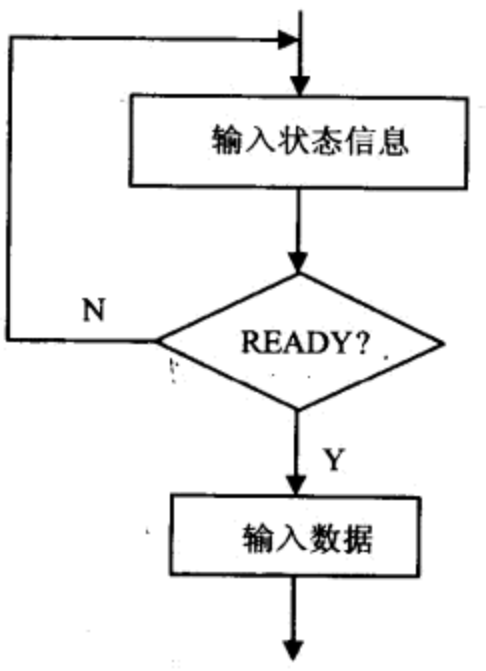


图 7-5 查询式输入程序流程

查询式输入的接口电路如图 7-6 所示。当输入装置的数据准备好以后，发出一个选通信号。该信号一方面把数据送入锁存器，另一方面使 D 触发器置“1”，即把 Ready 信号设为真，并将此信号送至状态口的输入端。锁存器输出端连接数据口的输入端，数据口的输出端接系统数据总线。设状态端口的最高位 D_7 连接 Ready 信号，CPU 先读状态口，查 Ready 信号是否为高电平（准备好）。若为高电平就输入数据，同时使 D 触发器清 0，使 Ready 信号为假；若未准备好，则 CPU 循环等待。

【例 7.1】从外设端口输入并存入内存一组数据，设 SR 为状态寄存器，其第 7 位为 READY 位，DR 为数据寄存器，查询式输入部分的程序如下：

```
WAITING: IN  AL,SR          ;输入状态信息
         TEST  AL,80H       ;检查 READY 是否为高电平
         JE  WAITING        ;未准备好,循环检测
         IN  AL,DR          ;准备好,读入数据
         MOV  [BX],AL       ;写入内存
         INC  BX
         LOOP WAIT
```

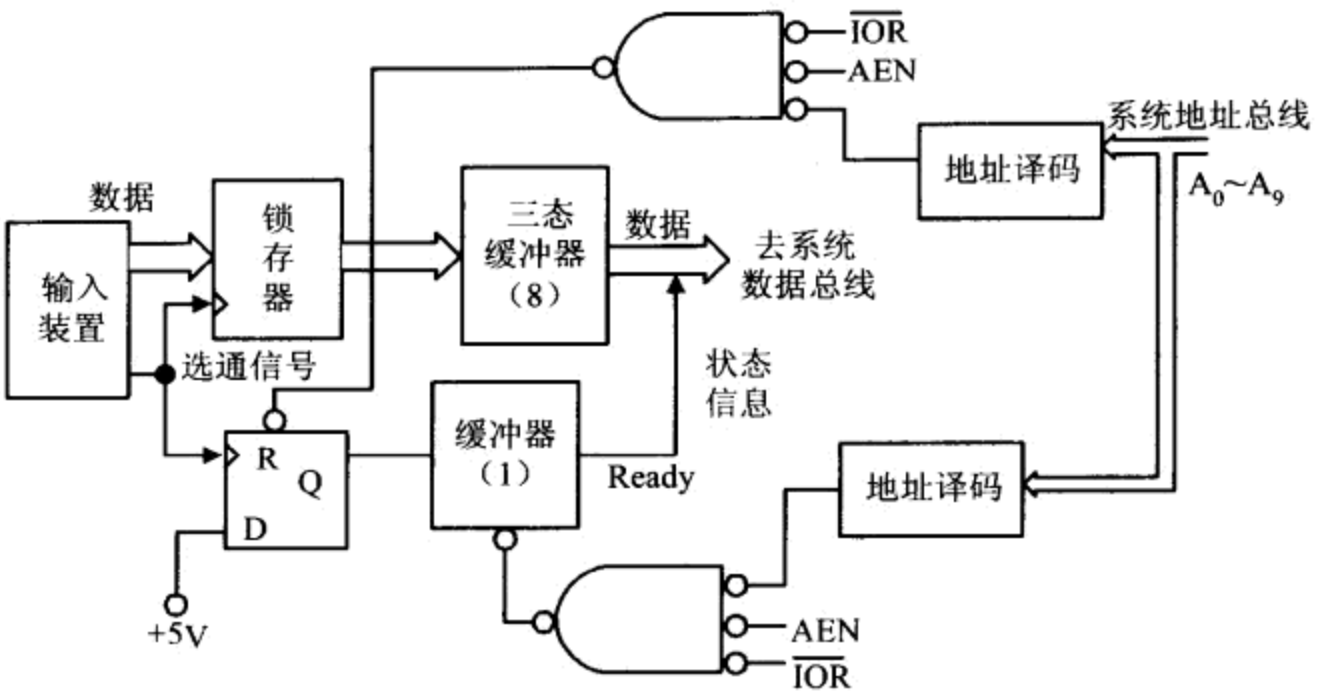


图 7-6　查询式输入接口电路

2．查询式输出

查询式输出程序流程如图 7-7 所示。若 CPU 需要向输出设备输出数据时，首先查询输出设备是否空闲。程序将要输出的数据准备好，然后用 IN 指令读取 I/O 端口的状态，查询输出设备是否空闲。如果忙（BUSY=1），CPU 就循环等待，始终判断状态端口状态，直到输出设备空闲（BUSY=0），CPU 执行输出指令，将待输出的数据通过 OUT 指令送到输出设备，完成一次数据输出。

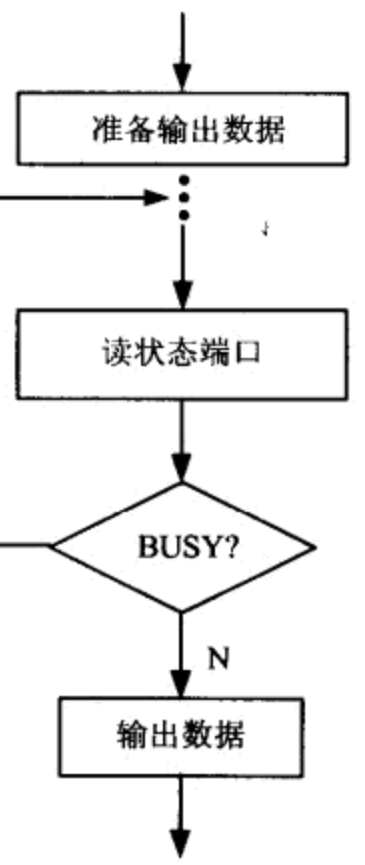


图 7-7　查询式输出程序流程

查询式输出的接口电路如图 7-8 所示。由图中可以看出，电路包含两个端口：状态口和数据输出口。状态口由一个 D 触发器和一个三态门构成，而数据输出口只含一个 8 位数据锁存器。输出设备把数据缓冲区中的数据输出以后，发一个 $\overline{ACK}$ 信号，使 D 触发器清 0，即 BUSY=0。CPU 读入该状态信息后知道数据缓冲区已“空”，于是执行输出命令。在 $M/\overline{IO}$、$\overline{WR}$ 及地址译码器输出信号三者相与后，发出选通信号，将数据锁存到锁存器中，同时使 D 触发器置“1”。它一方面通知外设数据已准备好，可以执行输出操作；另一方面在输出装置尚未完成输出以前，一直维持 BUSY=1，阻止 CPU 输出新的数据。

【例 7.2】从外设端口输出内存中一组数据，设 SR 为状态寄存器，其第 7 位为 BUSY 位，DR 为数据寄存器，查询式输出部分的程序为：

```
WAITING: IN   AL,SR                ;输入状态信息
         TEST  AL,80H              ;检查 BUSY 位
         JNE   WAITING             ;BUSY=1,则等待循环;否则准备输出数据
         MOV   AL,[BX]             ;从缓冲区取数据
         OUT   DR,AL               ;输出数据
         INC   BX
         LOOP  WAIT
```

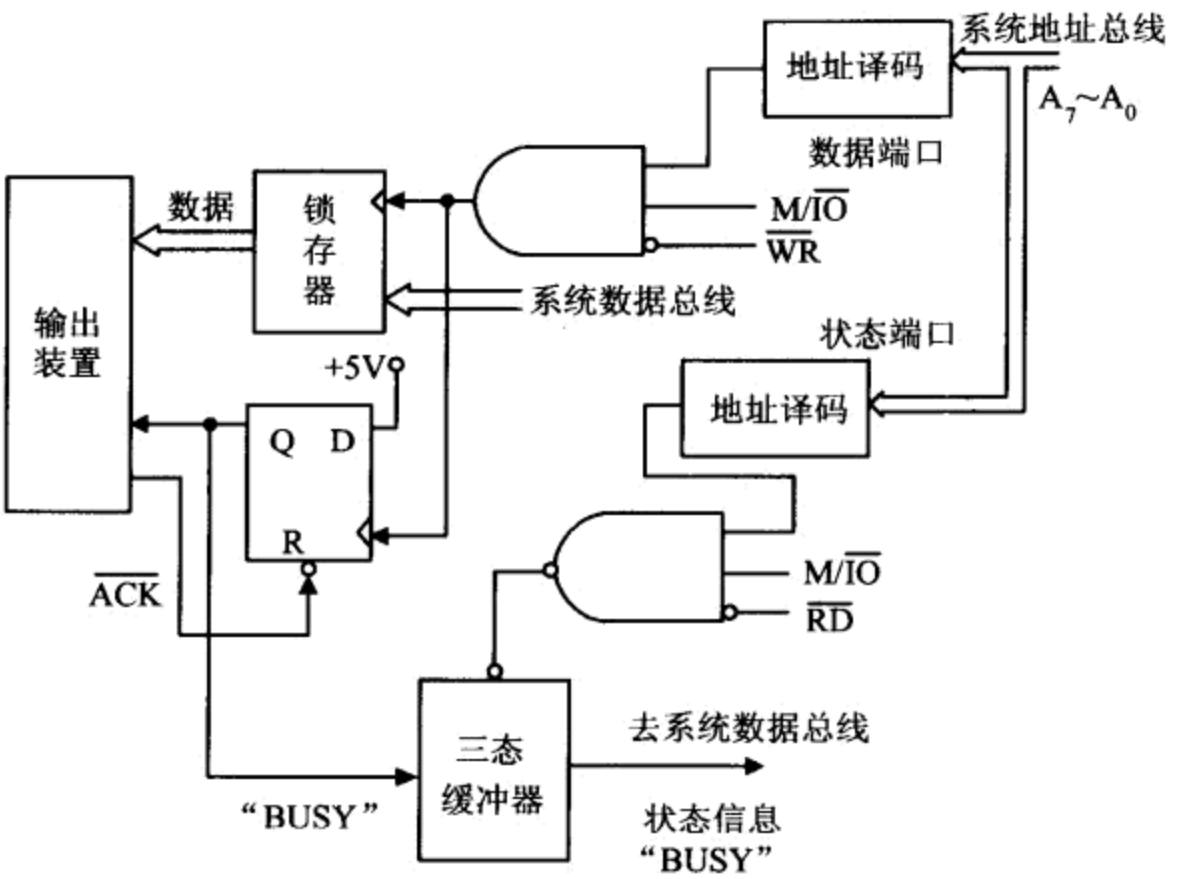


图 7-8 查询式输出接口电路

7.2.3 中断传送方式

CPU 采用查询传送方式与外设交换数据时，要不断读取状态信息，检查输入设备是否已准备好数据，输出设备是否忙碌或输出缓冲器是否已空。若外设没有准备就绪，CPU 就必须反复查询，进入循环等待状态，使 CPU 的效率降低。为提高 CPU 的利用率和进行实时数据处理，CPU 常采用中断方式与外设交换数据。

中断是指 CPU 在执行程序过程中，被内部或外部的事件打断，转去执行一段事先安排好的中断服务程序；服务结束后，又返回到原来的断点，继续执行原来的程序。中断传送方式就是当 CPU 与外设交换数据时，无需连续不断地查询外设的状态，而是在需要时，由外设主动地向 CPU 提出请求，请求 CPU 为其服务。在输入时，当输入设备准备好数据后，就向 CPU 提出中断请求，CPU 接到该请求后，暂停当前程序的执行，转去执行相应的中断服务程序，用输入指令进行一次数据输入，然后再返回到原来被中断的程序继续执行；在输出时，当输出端口的数据缓冲器已空时，外设向 CPU 发出中断请求，CPU 接到该请求后，暂停当前程序的执行，转到相应的中断服务程序，用输出指令向外设进行一次数据输出。输出操作完成之后，CPU 返回去执行原来被中断的程序。

以打印机为例，其工作过程如下：

（1）首先 CPU 启动打印机设备工作，然后就去继续自己的工作。

（2）当打印机准备好或已完成一个字符输出时，把设备置为就绪状态。

（3）I/O 接口在设备就绪时向 CPU 发出中断请求，要求服务。

（4）CPU 接到中断请求信号，暂停当前工作，响应中断，转入中断服务程序。服务程序实现发送下一个字符到 I/O 接口并选通到打印机。

（5）CPU 从中断服务程序返回，继续自己的工作。

（6）重复（2）~（5）步，直至整个文件输出结束后关闭打印机。

中断传送方式接口电路如图 7-9 所示。

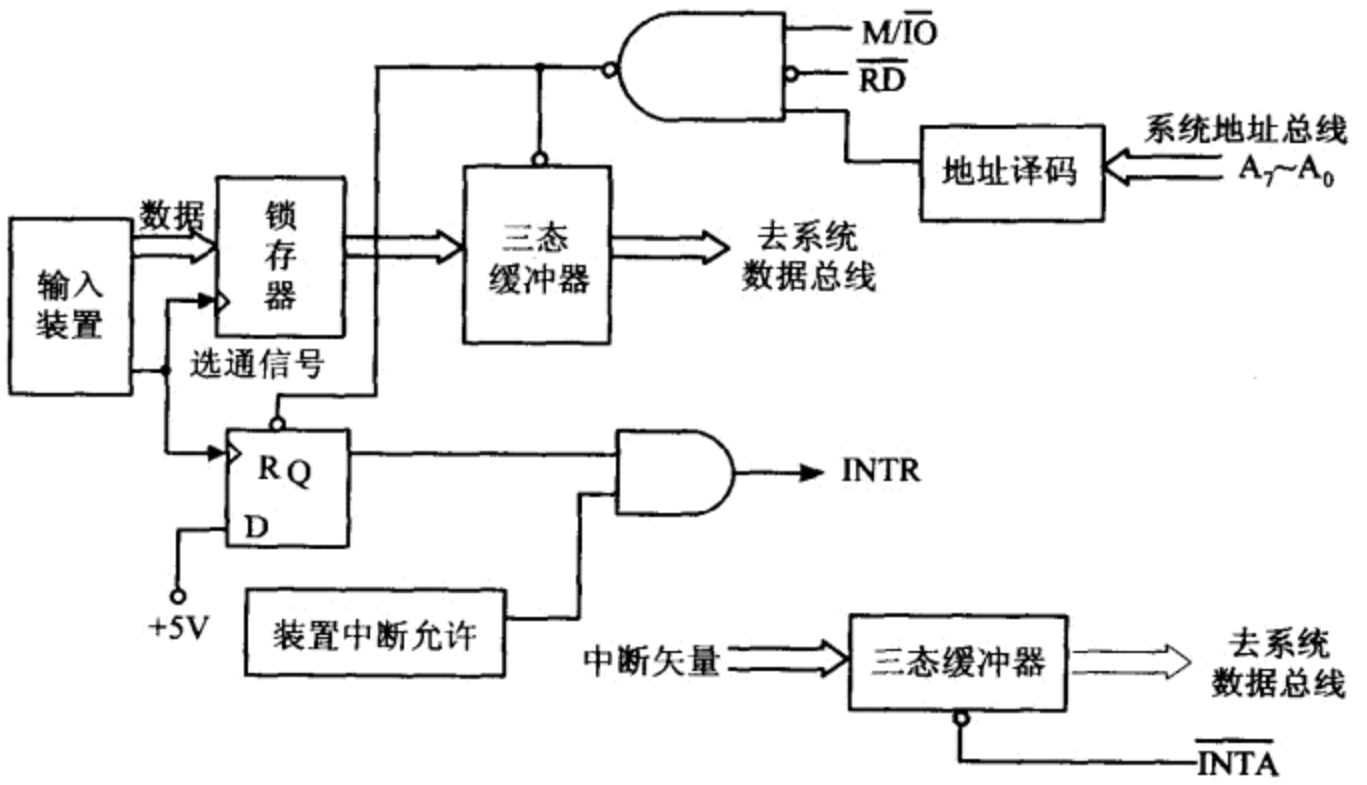


图 7-9　中断传送方式的接口电路

以输入过程为例，当输入设备输入数据后，发出选通信号，把数据锁存在锁存器中，D 触发器置 1，发出中断请求，若中断允许，CPU 响应中断，在现行指令执行完成后，暂停当前程序的执行，发出中断响应信号，然后外设将一中断向量放在数据总线上，CPU 转入中断服务程序，进行数据输入，同时为阻止其他中断的发生，将中断请求标志位清 0。中断程序执行完后，CPU 返回原程序继续执行。

7.2.4　DMA 传送方式

利用中断传送方式进行数据传送，可以实现外设和 CPU 并行工作，大大提高 CPU 的利用率，但在中断传送方式下，仍必须通过 CPU 执行程序来完成数据传送。每进行一次数据传送，CPU 都要执行一次中断服务程序。这时，CPU 要保护和恢复断点，通常还要执行一系列保护和恢复寄存器的指令，即保护现场，以便完成中断处理后能正确返回主程序。显然，这些操作与数据传送没有直接关系，但会花费掉 CPU 的不少时间。所以，在这段时间内执行部件和总线接口部件就不能并行工作，这也会造成数据传输效率的降低。为此提出了在外设和内存之间直接传送数据的方式——DMA（Direct Memory Access）传送方式，即直接存储器存取方式。

DMA 传送方式是在内存与外设间开辟专用的数据通道，这个数据通道在特殊的硬件电路——DMA 控制器的控制下，直接进行数据传送而不必通过 CPU。该方式要利用系统的数据总线、地址总线和控制总线来传送数据，当外设需要利用 DMA 方式进行数据传送时，DMA 控制器首先要向 CPU 发出总线请求信号 HOLD，要求 CPU 让出对总线的控制权。当 CPU 发出总线响应信号 HLDA，CPU 进入“保持”状态，暂停工作。DMA 控制器取代 CPU，临时接管总线，控制外设和存储器之间直接进行高速的数据传送，而不需要 CPU 进行干预。

DMA 控制器能给出访问内存所需要的地址信息，并能自动修改地址指针，也能设定和修改传送的字节数，还能向存储器和外设发出相应的读/写控制信号。在 DMA 传送结束后，它能释放总线，把总线控制权还给 CPU。可见，采用 DMA 方式传送数据时，不再需要 CPU 管理，也就不需要进行保护和恢复断点及现场之类的额外操作，一旦进入 DMA 操作，就可直接在 DMA 控制器的控制下快速完成一批数据的交换任务，减少了 CPU 管理 I/O 数据传送的负担，大大加速了数据传送过程，数据传送的速度基本上取决于外设和存储器的存取速度。

CPU 在每一个非锁定时钟周期（Lock 为高电平）结束后，都要检测一下 HOLD 引脚，看是否有 DMA 请求信号。若有，则暂时中止正在执行的程序，进入上述的 DMA 周期，系统总线由 DMA 控制器接管。故 DMA 控制器必须具备以下功能：

- 能向 CPU 发出要求控制总线的 DMA 请求信号 HRQ；
- 当收到 CPU 发出的 HLDA 信号后能接管总线，进入 DMA 模式；
- 能发出地址信息对存储器寻址并能修改地址指针；
- 能发出存储器和外设的读、写控制信号；
- 决定传送的字节数，并能判断 DMA 传送是否结束；
- 接受外设的 DMA 请求信号和向外设发 DMA 响应信号；
- 能发出 DMA 结束信号，使 CPU 恢复正常。

DMA 传送方式的工作过程如下。

（1）开始数据传送前，CPU 利用指令预置 DMA 控制器和设备地址，并启动设备；预置主存单元起始地址，指定与设备交换数据的主存单元；预置交换数据的字节数；预置读/写控制方式。

（2）CPU 继续执行主程序，与外设并行工作。当外设准备好时，就向 CPU 发出 DMA 请求信号 HRQ，要求进行 DMA 传送。

（3）CPU 响应 DMA 请求，发出应答信号 HLDA，交出总线控制权，系统转入 DMA 工作方式。

（4）DMA 发出主存单元地址及读/写控制命令，与主存交换数据。DMA 控制器与主存每交换一个字节的数据，地址寄存器自动加 1，字节计数寄存器自动减 1。

（5）DMA 控制器占据一个总线周期，交换一个数据后交出总线控制权，并检查字节计数寄存器的内容是否为 0。如果不为 0，则继续进行数据传送。当 DMA 控制器取得数据后，再次向 CPU 发出 DMA 请求。如果字节计数寄存器的内容为 0，则表明这次数据传送的任务已经完成，DMA 控制器向 CPU 发出中断请求，结束 DMA 传送，把总线控制权交还 CPU。

综上所述，DMA 传送方式实际上是把外设与内存交换信息的操作与控制交给了 DMA 控制器，简化了 CPU 对数据交换的控制，但这种方式显然电路结构复杂，硬件开销大。

本章小结

本章主要介绍了输入/输出接口的概念、基本功能和 I/O 接口传送数据的类型以及 4 种常用的输入/输出控制方式。

接口是指 CPU 和存储器、外部设备或者两种外部设备之间，或者两种机器之间通过系统

总线进行连接的逻辑部件，它是 CPU 与外界进行信息交换的中转站。接口技术是采用硬件与软件相结合的方法来研究微处理器如何与外部设备进行最佳匹配，以实现 CPU 与外界高效、可靠的信息交换的一门技术。

CPU 与外设之间的数据传输必须采用多种控制方式，才能满足各类外设的要求。在微型计算机中，可采用的输入/输出控制方式主要有程序传送方式（包括无条件传送方式和查询传送方式）、中断传送方式、DMA 传送方式和 I/O 处理机方式。

程序传送方式进行数据交换时通过软件来实现，方法比较灵活，但 CPU 要不断地执行指令并等待外设准备就绪，使 CPU 的工作效率降低。中断传送方式可以提高 CPU 的利用效率，使 CPU 与外设实现并行工作。对于需要高速、频繁地进行外设与内存间大批量数据交换时，采用 DMA 传送方式和 I/O 处理机方式会得到更好的效果。

在实际应用中，要根据系统的条件和需求合理地加以选择。

思考与练习题

1. 什么是接口，为什么计算机内一定要配置接口？
2. 微机的接口一般应具备哪些功能？
3. 什么是端口，I/O 端口的编址方式有哪几种？各有何特点，各适用于何种场合？
4. CPU 与输入/输出设备之间传送的信息有哪几类，相应的端口称为什么端口？
5. CPU 和外设之间的数据传送方式有哪几种，无条件传送方式通常用在哪些场合？
6. 相对于条件传送方式，中断方式有什么优点？和 DMA 方式比较，中断传送方式又有什么不足之处？
7. 采用无条件输入方式与外设接口时，接口电路应如何设计？
8. 说明查询式输入和输出接口电路的工作原理。
9. 简述在微机系统中，DMA 控制器从外设提出请求到外设直接将数据传送到存储器的工作过程。
10. 在一个系统中，采用何种方式进行数据传送的依据是什么？
11. 在一个系统中，有一个 I/O 数据输入端口，如 CPU 采用 MOV 指令从该端口读入数据，说明该系统采用的是哪种 I/O 端口编址方式？

第 8 章 可编程 DMA 控制器 8237A

本章主要讲解可编程控制器 8237A 的内部结构，8237A 内部寄存器的功能及格式以及 8237A 的编程和应用等主要内容。

通过本章的学习，读者应该理解和掌握可编程控制器 8237A 的内部结构、工作原理与实际编程应用。

8.1 可编程 DMA 控制器 8237A 概述

Intel 8237A 是一种有 40 个引脚的高性能可编程 DMA 控制器，采用主频 5MHz 的 8237A 传送速度可达到 1.6MB/s。8237A 的主要功能如下。

（1）在一个 8237A 芯片中有 4 个独立的 DMA 通道，每个通道均可独立地传送数据，可控制 4 个 I/O 外设进行 DMA 传送。

（2）每个通道的 DMA 请求都可以分别允许和禁止。每个通道的 DMA 请求有不同的优先权，优先权可以是固定的，也可以是循环的。

（3）每个通道均有 64KB 的寻址和计数能力，即一次 DMA 传送的数据最大长度可达 64KB。

（4）可以在存储器与外设间进行数据传送，也可以在存储器的两个区域之间进行传送。

（5）8237A 有 4 种 DMA 传送方式，分别为单字节传送、数据块传送、请求传送和级联传送方式。

（6）8237A 芯片有一条结束处理的输入信号 $\overline{\text{EOP}}$，允许外界用此输入端结束 DMA 传送或重新初始化。

（7）8237A 可以级联、扩展更多的通道。

（8）40 脚双列直插式，+5V 工作电源。

8237A 有两种不同的工作状态，分别为从态方式和主态方式。

在 DMA 控制器未取得总线控制权时必须由 CPU 对 DMA 控制器进行编程，以确定通道的选择、数据传送的方式和类型、内存单元起始地址、地址是递增还是递减及要传送的总字节数等，CPU 也可以读取 DMA 控制总线的状态。这时，CPU 处于主控状态，而 DMA 控制器就和一般的 I/O 芯片一样，是系统总线的从设备，这种工作方式称为从态方式。

当 DMA 控制器取得总线控制权后，系统就完全在它的控制下，使 I/O 设备和存储之间或存储器与存储器之间进行直接的数据传送，这种工作方式称为主态方式。

8.2　8237A 的内部结构及引脚

8.2.1　8237A 的内部结构

8237A 的内部结构如图 8-1 所示，主要由 5 个部分组成。

（1）时序与控制逻辑：8237A 处于从态时，该部分电路接收系统送来的时钟、复位、片选和读/写控制等信号，完成相应的控制操作；处于主态时则向系统发出相应的控制信号。

（2）优先级编码电路：该部分电路根据 CPU 对 8237A 初始化时送来的命令，对同时提出 DMA 请求的多个通道进行排队判优，决定哪一个通道的优先级最高。对优先级的管理有两种方式：固定优先级和循环优先级。无论采用哪种优先级管理，一旦某个优先级高的设备在服务时，其他通道的请求均被禁止，直到该通道的服务结束时为止。

（3）数据和地址缓冲器组：8237A 的 A_7~A_4、A_3~A_0 为地址线；DB_7~DB_0 在从态时传输数据信息，主态时传送地址信息。这些数据引线、地址引线都与三态缓冲器相连，因而可以接管或释放总线。

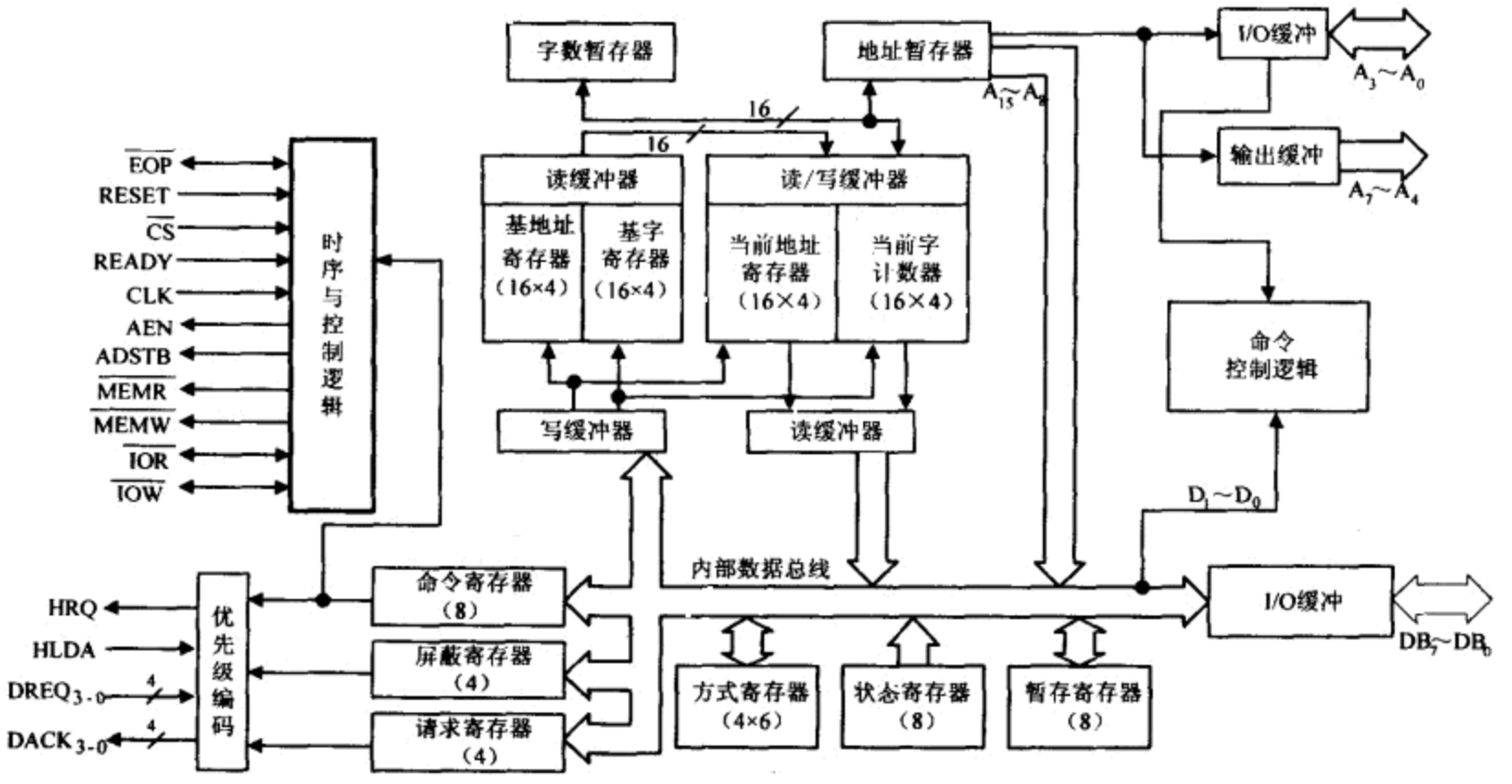


图 8-1　8237A 的内部结构图

（4）命令控制逻辑：该部分电路在从态时，接收 CPU 送来的寄存器选择信号（A_3~A_0），选择 8237A 内部相应的寄存器；主态时，对方式字的最低两位（D_1D_0）进行译码，以确定 DMA 的操作类型。A_3~A_0 与 $\overline{IOR}$、$\overline{IOW}$ 配合可组成各种操作命令。

（5）内部寄存器：8237A 内部的其余部分主要为寄存器。每个通道都有一个 16 位的基址寄存器、基字计数器、当前地址寄存器和当前字计数器。8237A 有 4 个 DMA 通道，因此上述这几种寄存器在片内各有 4 个。片内还各有一个命令寄存器、屏蔽寄存器、请求寄存器、状态寄存器和暂存寄存器。上述这些寄存器均是可编程寄存器。另外还有字数暂存器和地址暂存器等不可编程的寄存器。

8.2.2 8237A 的引脚

8237A 采用双列直插式，有 40 个引脚，其引脚排列如图 8-2 所示，下面分别介绍各引脚的功能。

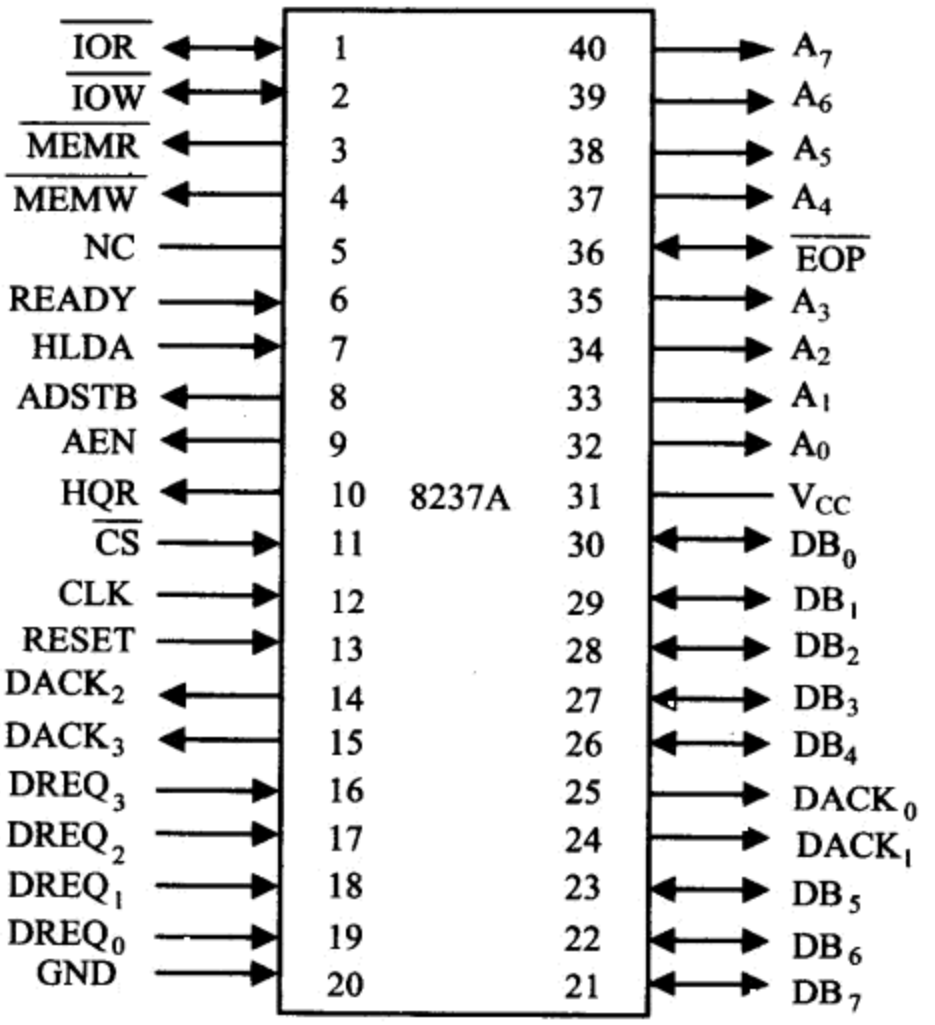


图 8-2 8237A 的引脚

（1）CLK：时钟信号，输入。该信号用来控制 8237A 的内部操作和数据传送速率。8237A 的时钟频率为 4MHz，8237A-5 的时钟频率可达到 5MHz。8237A-5 DMA 控制器是 8237A 的改进型产品，工作速度较高，但工作原理和使用方法与 8237A 完全一样。

（2）$\overline{CS}$：片选信号，输入，低电平有效。在从态工作方式下，$\overline{CS}$有效时选中 8237A，这时 DMA 控制器作为一个 I/O 设备，可以通过数据总线与 CPU 通信。

（3）RESET：复位信号，输入，高电平有效。该信号用来复位 8237A 芯片。8237A 复位时，屏蔽寄存器被置 1，其他寄存器均清 0，复位后 8237A 工作在空闲周期。

（4）READY：准备就绪信号，输入，高电平有效。当参与 DMA 传送的设备中有慢速 I/O 设备或存储器时，可能要求延长读/写操作周期，这时可使 READY 变成低电平，使 8237A 可在 DMA 周期中插入等待周期 T_W。当存储器或外设准备就绪时，READY 端变成高电平。

（5）ADSTB：地址选通信号，输出，高电平有效。该信号有效时，DMA 控制器把当前地址寄存器中的高 8 位地址（通过 DB_0～DB_7）锁存到外部锁存器中。

（6）AEN：地址允许信号，输出，高电平有效。该信号使地址锁存器中锁存的高 8 位地址送到地址总线上，与芯片直接输出的低 8 位地址一起构成 16 位内存偏移地址。AEN 信号也使与 CPU 相连的地址锁存器无效。这样就保证了地址总线上的信号来自 DMA 控制器，

而不是来自 CPU。

（7）$\overline{\text{MEMR}}$：存储器读信号，三态，输出，低电平有效。主态时，可与$\overline{\text{IOW}}$配合把数据从存储器读出到外设中，也可用于控制器内存间数据传送，使数据从源地址单元中读出。从态时信号无效。

（8）$\overline{\text{MEMW}}$：存储器写信号，三态，输出，低电平有效。主态时，可与$\overline{\text{IOR}}$配合把数据从外设写入存储器，也可用于内存间数据传送的场合，控制把数据写入目的单元。同样，从态时该信号无效。

（9）$\overline{\text{IOR}}$：输入/输出设备读信号，双向，三态，低电平有效。从态时，它作为输入控制信号，送入 8237A，当它有效时，CPU 读取 8237A 内部寄存器的值。主态时，它作为输出控制信号，与$\overline{\text{MEMW}}$相配合，控制数据由外设传送到存储器中。

（10）$\overline{\text{IOW}}$：输入/输出设备写信号，双向，三态，低电平有效。从态时，它是输入控制信号，当它有效时，CPU 向 DMA 控制器的内部寄存器中写入信息，对 8237A 进行初始化编程。在主态时，它作为输出控制信号，与$\overline{\text{MEMR}}$相配合，把数据从存储器传送到外设。

（11）$\overline{\text{EOP}}$：传输过程结束信号，双向，低电平有效。在 DMA 传送时，当 DMA 控制器的任一通道中的当前字计数器减为 0，再由 0 减为 FFFFH 而终止计数时，会在$\overline{\text{EOP}}$引脚上输出一个有效的低电平信号，作为 DMA 传输过程结束信号。8237A 也允许从外部输入一个低电平信号到$\overline{\text{EOP}}$引脚上来终止 DMA 传送。不论是内部计数结束引起终止 DMA 过程，还是外部终止 DMA 过程，都会使请求存储器的相应位复位。如对 8237A 方式的寄存器编程，如果将通道设置成自动预置状态的话，那么该通道完成一次 DMA 传送，出现$\overline{\text{EOP}}$信号后，又能自动恢复有关寄存器的初值，继续执行另一次 DMA 传送。

（12）DREQ_3～DREQ_0：通道 3～0 的 DMA 请求信号，输入。当外设请求 DMA 服务时，就向 8237A 的 DREQ 引脚送出一个有效的电平信号，有效电平的极性由源序确定。在固定优先权情况下，DREQ_0的优先级最高，DREQ_3的优先级最低，在循环优先权时，优先级可以改变。

（13）DACK_3～DACK_0：通道 3~0 的 DMA 响应信号，输出。其有效电平的极性由编程确定。当 8237A 收到 CPU 的 DMA 相应信号 HLDA，开始 DMA 传送后，相应通道的 DACK（DMA Acknowledge）有效，将该信号输出到外部，通知外部电路现已进入 DMA 周期。

（14）HRQ：总线请求信号，输出，高电平有效。该信号送到 CPU 的 HOLD 端，是向 CPU 申请获得总线控制权的 DMA 请求信号。8237A 任一个未被屏蔽的通道有 DMA 请求（DREQ 有效）时，都可使 8237A 的 HQR 端输出有效的高电平。

（15）HLDA：总线响应信号，输入，高电平有效。其与 CPU 的 HLDA 端相连。当 CPU 收到 HRQ 信号后，至少必须经过一个时钟周期后，使 HLDA 变高，表示 CPU 已把总线的控制权交给 8237A 了，8237A 收到 HLDA 信号后，就开始进行 DMA 传送。

（16）A_3～A_0：最低 4 位地址线，三态，双向。在从态时，它们是输入信号，用来寻址 DMA 控制器的内部寄存器，使 CPU 对各寄存器进行读/写操作，即对 8237A 进行编程。在主态时，输出的是要访问内存的最低 4 位地址。

（17）A_7～A_4：4 位地址线，三态，输出。A_7～A_4 始终工作于输出状态或浮空状态。在主态时输出 4 位地址信息 A_7~A_4。

（18）DB_7～DB_0：8 位数据线，三态，输入/输出。DB_7～DB_0 被连到系统数据总线上。从态时，CPU 可用 I/O 读命令从数据总线上读取 8237A 的地址寄存器、状态寄存器、暂存寄存器和字计数器的内容，CPU 还可以通过这些数据线用 I/O 写命令对各个寄存器进行编程。在主态时，高 8 位地址信号 A_{15}～A_8 经 8 位 I/O 缓冲器从 DB_7～DB_0 引脚输出，并由 ADSTB 信号将 DB_7～DB_0 输出的信号锁存到外部的高 8 位地址锁存器中，它们与 A_7～A_0 输出的低 8 位地址线一起构成 16 位地址。当 8237A 工作于存储器到存储器的传送方式时，先把从源存储器中读出来的数据，经这些引线送到 8237A 的暂存寄存器中，再经这些引线将暂存器中的数据写到目的存储单元中。

8.3 8237A 工作方式

8237A 可编程 DMA 控制器共有 4 种工作方式，它的每个通道均可以采用其中一种进行工作。选择工作方式可以通过对模式寄存器的第 6、7 位进行设置来实现。

8.3.1 单字节传送方式

在这种工作方式下，每进行一次 DMA 操作，只传送一个字节的数据。8237A 每完成一个字节的传送，计数器便自动减 1，地址寄存器的值加 1 或减 1。接着，8237A 释放系统总线，把控制权交还给 CPU，这样 CPU 至少可以得到一个总线周期。但是 8237A 在释放总线后，会立即对 DREQ 端进行测试，一旦 DREQ 有效，则 8237A 会立即发送总线请求，在获得总线控制权后，又成为总线主模块而进行 DMA 传送。

当内部字节计数器由 0 减为 FFFFH 时，产生传输过程结束信号$\overline{\text{EOP}}$，表示 DMA 传送过程结束。

这种方式的特点是一次 DMA 传送只传送一个字节的数据，占用一个总线周期，然后释放系统总线。因此，这种方式又被称为总线周期窃取方式，每次总是窃取一个总线周期，完成一个字节的传送之后立即归还总线。

8.3.2 块传送方式

在这种工作方式下，一旦开始传送，就会一个字节一个字节的进行下去，直到把整个数据块全部传送完毕，才交出系统总线控制权。若需要提前结束其传输过程，可由外部输入一个有效的$\overline{\text{EOP}}$信号来强制 8237A 退出。

当外设准备好后，向 8237A 发出 DREQ 信号，8237A 收到该信号后向 CPU 发出 HRQ 信号请求占用总线，CPU 同意 HRQ 请求，则向 8237A 发回 HLDA 信号，然后 8237A 向外设发出 DACK 信号，开始 DMA 传送，直到整个数据块传送完毕。

在每次 DREQ 有效后，若 CPU 响应其请求让出总线控制权给 8237A，8237A 进行 DMA 服务时，就会连续传送数据，只有当字节计数器计数由 0 减为 FFFFH 时或外部送来有效的$\overline{\text{EOP}}$信号时，才将总线控制权交给 CPU，从而结束 DMA 服务。

这种方式的特点是数据传输效率高，DREQ 有效电平只要保持到 DACK 有效，就能传送完整批数据，但整个数据块传送期间，CPU 失去总线控制权，因而别的 DMA 请求也被禁止。

8.3.3　请求传送方式

这种工作方式与块传送方式类似，不同点在于每传送一个字节后，8237A 都对 DREQ 端进行测试，询问其是否有效。如果检测到 DREQ 端变为无效电平，则立刻“挂起”，停止 DMA 传送，但并不释放系统总线，测试过程仍然进行。当检测到 DREQ 端变为有效电平时，就在原来的基础上继续进行传送。由于请求传送方式在传送完一个字节的数据之后就询问 DREQ 信号是否有效，故又称询问传送方式。

这种方式的特点是 DREQ 信号一直有效时，则连续传送数据，只有当字节计数器由 1 减为 FFFFH，或外部送来有效的$\overline{\text{EOP}}$信号，或 DREQ 变为无效时才结束 DMA 传送过程。

8.3.4　级联传送方式

级联传送方式是为扩展 DMA 通道采用的一种方式。在这种方式下，可以把一片 8237A（称为主片）和几片 8237A（称为从片）进行级联，以便扩充 DMA 通道。

图 8-3 所示为二级 8237A 级联时的情况。如图所示，每一片从 8237A 的 HRQ 和 HLDA 信号接到主 8237A 的 DREQ 和 DACK 端上，而主 8237A 的 HRQ 和 HLDA 信号则接到 CPU 的 HOLD 和 HLDA 端上。在这种情况下，各个从片的每一个通道均可进行 DMA 传输，而主片主要用来传送从片的 DMA 请求信号，还管理各从片的优先级。当 CPU 响应从片的 DREQ 请求，并输出 DACK 信号作为响应时，主片输出的信号除 HRQ 外都被禁止。在 DMA 操作期间，它不输出任何地址和控制信号，避免与从片中正在运行的输出信号发生冲突。

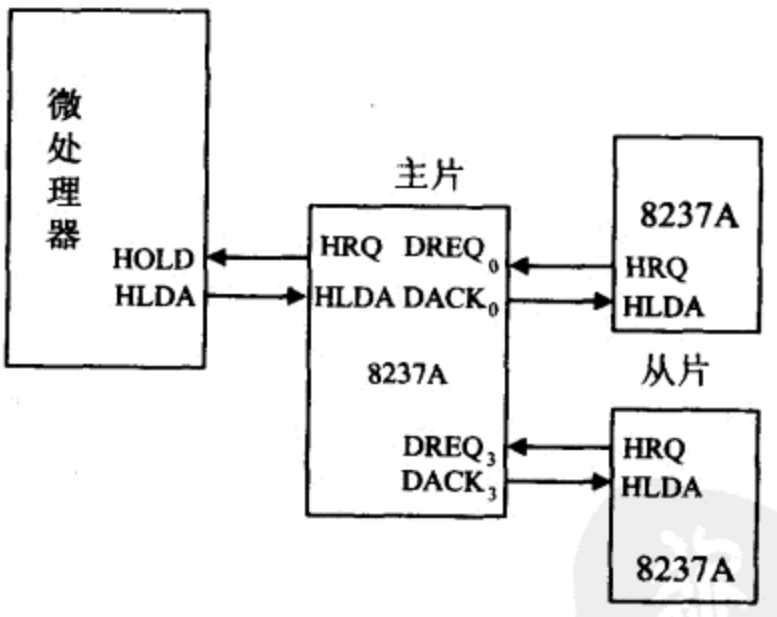


图 8-3　二级 8237A 级联

采用级联传送方式时，1 块主片最多允许与 4 块从片相连，若用 5 片 8237A 芯片构成两级 DMA 系统，可得到 16 个 DMA 通道。主片通过编程设置为级联传送方式，从片不用设置为级联传送方式，但要设置其他 3 种方式之一。

8.4 8237A 内部寄存器功能及格式

8237A 的内部可编程寄存器主要有 10 个，如表 8-1 所示。

表 8-1 8237A 的内部寄存器

名　称	位　数	数　量	功　能
当前地址寄存器	16	4（每通道一个）	保存在 DMA 传送期间的地址值，可读/写
当前字节计数寄存器	16	4（每通道一个）	寄存当前字节数，初始值比实际值少 1，可读/写
基地址寄存器	16	4（每通道一个）	寄存当前地址寄存器的初始值，只能写
基字节数寄存器	16	4（每通道一个）	保存相应通道当前字（节）数的计数器的初值
工作方式寄存器	8	4（每通道一个）	寄存相应通道的方式控制字，由编程写入
命令寄存器	8	1（4 个通道共用一个）	寄存 CPU 发送的控制命令
状态寄存器	8	1（4 个通道共用一个）	存放 8237A 各通道的现行状态
请求寄存器	4	1（每通道 1 位）	寄存各通道的 DMA 请求信号
屏蔽寄存器	4	1（每通道 1 位）	用于选择允许或禁止各通道的 DMA 请求信号
暂存寄存器	8	1（每通道 1 位）	暂存传输数据，仅用于存储器到存储器的传输

在 8237A 中，不同类型的寄存器起着不同的控制作用。在 DMA 操作前必须对各寄存器写入一定的内容，以得到所要求的功能，即对 DMA 控制器编程。编程最关键的是要弄清每个寄存器各位的含义，下面就介绍一下主要的寄存器。

8.4.1 当前地址寄存器

每个通道都有一个 16 位的当前地址寄存器，用于存放 DMA 传送的存储器地址值。每传送一个数据，地址值自动增 1 或减 1，以指向下一个存储单元。在编程状态下，CPU 可用输出指令对该寄存器写入初值，也可由输入指令读出该寄存器中的值，但每次只能读/写 8 位数据，故对该寄存器的读/写操作要分两次进行。若工作方式寄存器编程为自动预置操作，则当 DMA 传送结束，产生$\overline{\text{EOP}}$信号后，8237A 自动将基地址的值装入该寄存器中。

8.4.2 当前字节计数寄存器

每个通道都有一个 16 位长的当前字节计数寄存器。它保存当前 DMA 传送的字节数。实际传送的字节数比编程写入的字节数大 1（例如，编程的初始值为 10，将导致传送 11 个字节），每次传送以后，字节计数器减 1。当其内容从 0 减 1 而到达 FFFFH 时，将产生终止计数 TC 脉冲输出。CPU 访问它是以连续两字节对其读出或写入。在自动预置方式时，当$\overline{\text{EOP}}$有效后，被重新预置成初始值；如果在非自动预置方式，这个计数器在终止计数之后将为 FFFFH。

8.4.3 基地址寄存器

每个通道都有一个 16 位的基地址寄存器，它用来存放对应通道当前地址寄存器的初值，该值是在 CPU 对 DMA 控制器进行编程时，与当前地址寄存器的值一起被写入的，即两个寄

存器有相同的写入端口地址，编程时写入相同的内容。但基地址寄存器的内容不能被 CPU 读出，也不能被修改。设置该寄存器的主要目的在于，当执行自动预置操作时，当前地址寄存器能恢复到初始值。

8.4.4　基字节数寄存器

每个通道都有一个 16 位的基字节数寄存器，用于存放对应通道当前字节计数器的初值，该值也是在 CPU 对 8237A 进行初始化编程时与当前字节计数器一起被写入的，且两者具有相同的写入端口，写入相同的内容。该寄存器的内容不会自动修改，也不能被 CPU 读出，它主要用于自动预置操作时使当前字节计数器恢复初值。

注意：以上寄存器都是 16 位的，利用 8 位数据线如何读/写呢？因为 8237A 内部有一个高/低触发器，它控制读/写 16 位寄存器的高字节或低字节。触发器为 0，则操作的是低字节；为 1，则操作的是高字节。软、硬件复位之后，此触发器被清为 0。每当 16 位通道寄存器进行一次操作（读/写 8 位），此触发器便自动改变状态。因此，对 16 位寄存器的读/写可分两次连续进行，不必清除这个触发器。

8.4.5　命令寄存器

它是一个 8 位寄存器，用来控制 8237A 的操作。编程时，由 CPU 对其写入命令字，而由复位信号（RESET）和软件清除命令清除它。命令寄存器的格式如图 8-4 所示。

（1）D_0 位：当 D_0=1 时，允许进行存储器至存储器之间的传送；当 D_0=0 时，禁止该操作。这种传送方式能以最短的时间成组地将数据从存储器的一个区域传送到另一个区域。此时先由通道 0 采用软件请求的方法来启动 DMA 服务，规定通道 0 用于从源地址读入数据，然后将读入的数据字节存放在暂存器中，由通道 1 把暂存器的数据字节写到目的地址存储单元。通道 0 的当前地址寄存器用于存放源地址，通道 1 的当前地址寄存器和当前字节计数器提供目的地址和进行计数。

（2）D_1 位：用于执行存储器到存储器传送操作时，决定是否允许通道 0 的地址保持不变。当 D_1=1 时，可以使通道 0 在整个传送过程中保持同一地址，这样可把这个地址单元中的数（也即同一个数据）写到一组存储单元中去。D_1=0 时禁止这种操作。当然，当 D_0=0 时，不允许在存储器间直接进行数据传送，这种方法也就无效。

（3）D_2 位：用来表示允许还是禁止 8237A 工作，D_2=0 时允许工作，否则禁止工作。

（4）D_3，D_5 位：与时序有关的控制位。

（5）D_4 位：用来设定通道优先权结构。D_4 位为 0 时，为固定优先权，这时规定通道 0 的优先级最高，通道 1 次之，通道 3 最低。D_4=1 时为循环优先权，它使刚服务过的通道 i（i 表示通道号）的优先权变成最低，而让通道 i +1 的优先权变为最高，当 i +1= 4 时使通道号为 0。例如，若某次传输前优先权从高到低的次序为 2、3、0、1，那么在通道 2 进行一次传输后，优先级次序变成 3、0、1、2，通道 3 完成传输后优先级次序成为 0、1、2、3 等。随着 DMA 操作的不断进行，优先权也不断循环变化，这样可防止某一通道长时间占用总线。但要注意，任意通道进入 DMA 服务后，其他通道均不能去打断它。

（6）D_6位：D_6=0 则 DREQ 为高电平有效；D_6=1 为低电平有效。

（7）D_7位：D_7=0 则 DACK 为低电平有效；D_7=1 为高电平有效。

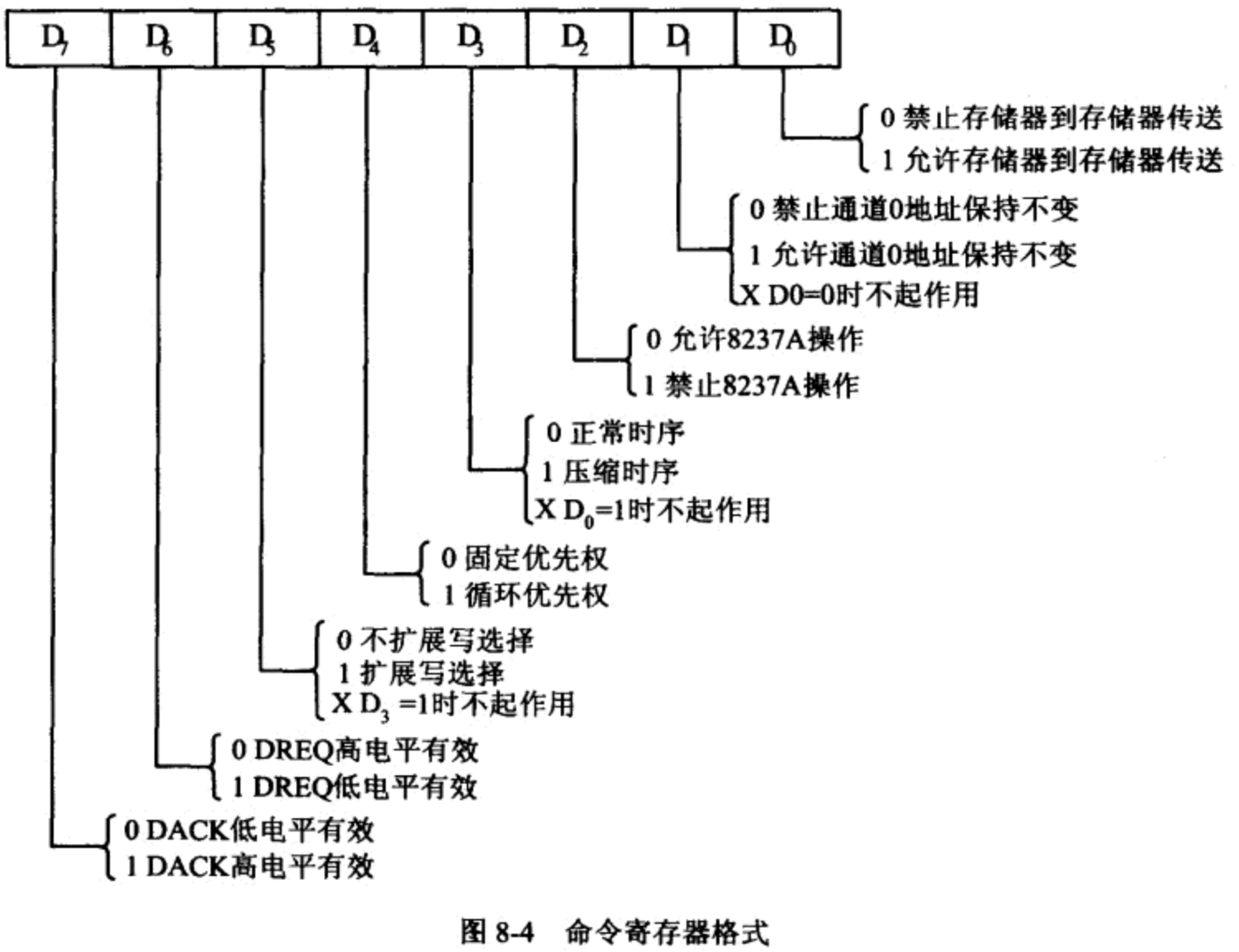


图 8-4　命令寄存器格式

8.4.6　工作方式寄存器

工作方式寄存器又称方式控制寄存器。每个通道都有一个 8 位的工作方式寄存器，它用于指定 DMA 的操作类型、传送方式，是否自动预置和传送一字节数据后地址是按增 1 还是减 1 修改，其格式如图 8-5 所示。

CPU 在编程初始化时写入工作方式寄存器的控制字。其低两位 D_0、D_1 指定写入的通道号，即根据 D_0、D_1 两位的编码，确定此命令字写入的通道。而 D_2～D_7 是通道相应的工作方式设定位。原则上，4 个通道要写入 4 个工作方式控制字。

（1）D_3、D_2 位：当 D_7、D_6 位不同时为 1 时，D_3、D_2 位用来设定通道的 DMA 的传送类型：读传送、写传送和校验传送。

- 读传送。当 D_3D_2=10 时，将数据从存储器读出，再写入 I/O 设备，即数据从存储器传送到 I/O 接口，这时 8237A 要发出对存储器的读信号$\overline{\text{MEMR}}$和对 I/O 接口的写信号$\overline{\text{IOW}}$。
- 写传送。当 D_3D_2=01 时，将数据从 I/O 设备读出，再写入存储器，即数据从 I/O 接口传送到存储器，这时 8237A 要发出对 I/O 接口的读信号$\overline{\text{IOR}}$和对存储器的写信号$\overline{\text{MEMW}}$。

- 校验传送。当 D_3D_2=00 时为校验传送。这种操作实际上不进行数据传送，主要用来对写传送或读传送功能进行校验。由 8237A 产生地址信息，并影响$\overline{EOP}$等，但不发出存储器和外部设备的读/写控制信号，这就阻止数据的传送。但是 8237A 仍将保持着它对系统总线的控制权，I/O 设备可以使用这些响应信号，在 I/O 设备内部对一个指定数据块的每一个字节进行存取，以便进行校验。设定校验方式时，要设定命令寄存器为禁止存储器至存储器的 DMA 操作方式。

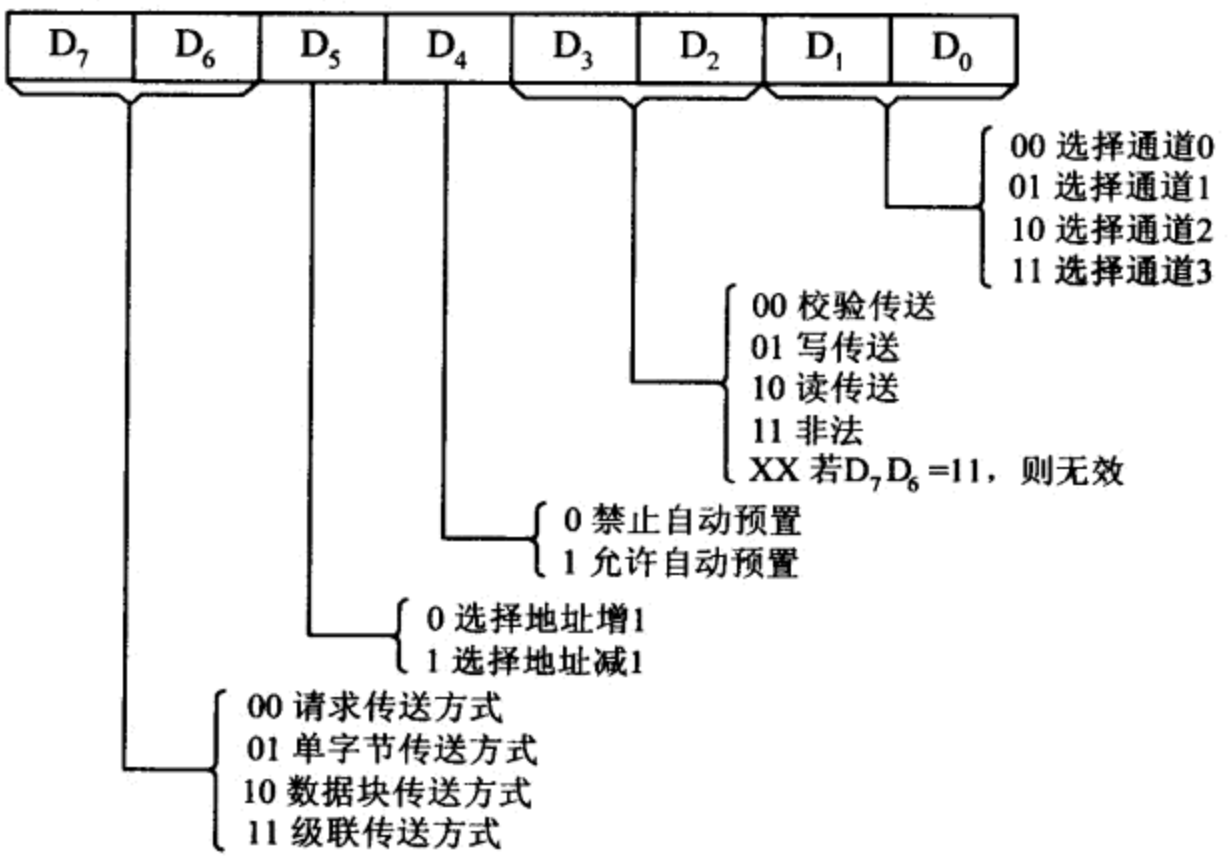


图 8-5　工作寄存器格式

（2）D_4 位：设定通道是否进行自动预置。当 D_4=1 时，在接收到$\overline{EOP}$信号后，该通道自动将基地址寄存器内容装入当前地址寄存器，将基字节计数器内容装入当前字节计数器，而不必通过 CPU 对 8237A 进行初始化，就能执行另一次 DMA 服务。

（3）D_5 位：决定每传送 1 个字节后，存储器地址是加 1 还是减 1。

（4）D_7，D_6 位：决定该通道 DMA 传送的工作方式。

8.4.7　请求寄存器

当外部有 DMA 请求信号 DREQ 或软件产生一个 DMA 请求时，选中通道的请求位置 1。请求寄存器用于由软件来启动 DMA 请求的设备。存储器到存储器传送，必须利用软件产生 DMA 请求。这种软件请求 DMA 传送操作必须是成组传送方式，在传送结束后，$\overline{EOP}$信号变为有效，该通道对应的请求标志位被清 0，因此，每执行一次软件请求 DMA 传送，都要对请求寄存器编程一次，RESET 信号清除所有通道的请求寄存器。软件请求位是不可屏蔽的。可以用请求控制字对各通道的请求标志进行置位和复位。该寄存器只能写，不能读。对某个通道的请求标志进行置位和复位的命令格式如图 8-6 所示。

8237A 接收到命令请求时，按 D_1D_0 确定的通道，对该通道的请求标志执行 D_2 规定的操作。D_2=1，将请求标志位置 1。D_2=0，将请求标志位清 0。例如，若用软件请求通道 0 进行

DMA 传送，则向请求寄存器写入 04H 控制字。

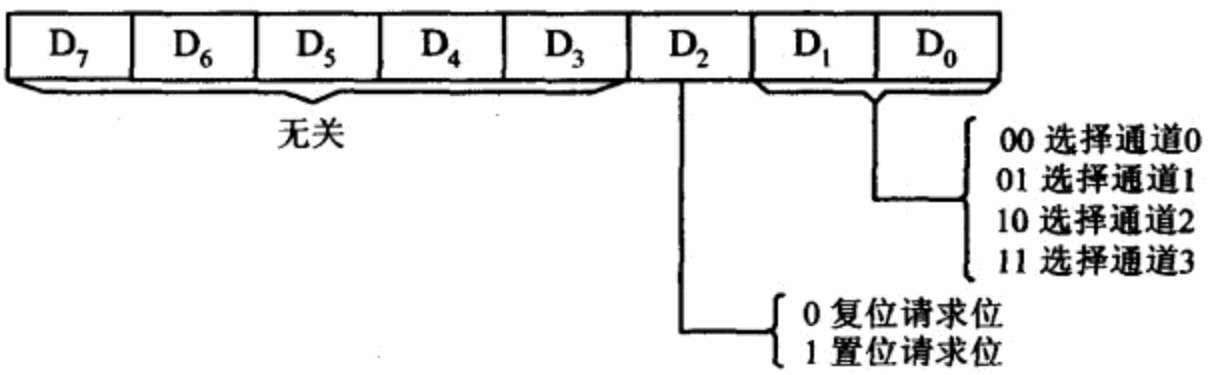


图 8-6　请求寄存器格式

8.4.8　屏蔽寄存器

屏蔽寄存器用于选择是否禁止各通道接收 DMA 请求信号 DREQ。当某通道的屏蔽位为 1 时，表示屏蔽相应通道，并禁止该通道 DMA 操作。RESET 信号使所有通道的屏蔽标志位都置 1，这时禁止所有通道产生 DMA 请求，直到用一条清除屏蔽寄存器的命令使之复位后才允许接收 DMA 请求。对 8237A 允许写入两种屏蔽字，使各屏蔽位置位或复位。两种屏蔽字需写入不同的端口地址中。

（1）通道屏蔽字。可用指令对屏蔽寄存器写入通道屏蔽字来对单个屏蔽位进行操作，使之置位或复位。通道屏蔽字的格式与通道请求字的格式相类似，如图 8-7 所示。在 X86 系统中其端口地址为 000AH。

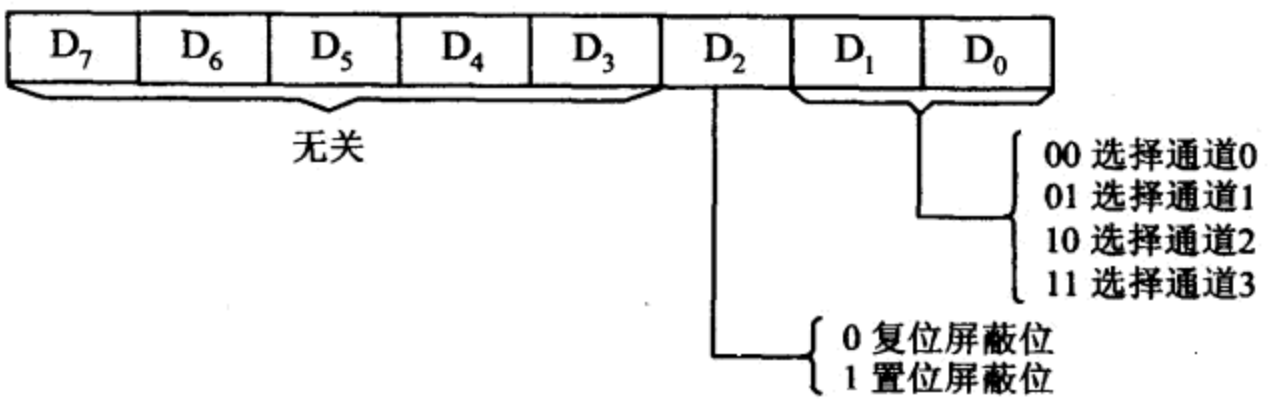


图 8-7　通道屏蔽字格式

（2）主屏蔽字。8237A 还允许使用主屏蔽命令来设置通道的屏蔽触发器，主屏蔽字格式如图 8-8 所示。

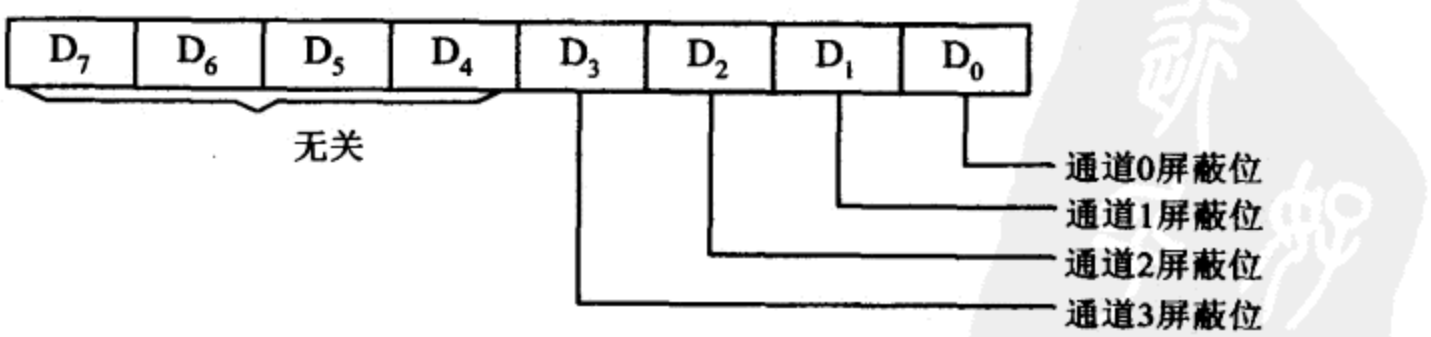


图 8-8　主屏蔽字格式

D_3～D_0 位对应通道 3～0 的屏蔽位，各个屏蔽位中，0 表示清除屏蔽位，1 表示置位屏蔽位，这样利用一条主屏蔽字命令就可一次完成对 4 个通道的屏蔽位的设置。在 X86 系统中其端口地址为 000FH。

在需要同时清除 4 个通道的屏蔽位时，还可用软件命令实现。

8.4.9　状态寄存器

8237A 中有一个可由 CPU 读取的状态寄存器。它的低 4 位反映读命令这个瞬间每个通道是否产生 TC（为 1，表示该通道传送结束），高 4 位反映每个通道的 DMA 请求情况（为 1，表示该通道有请求）。状态寄存器的格式如图 8-9 所示。

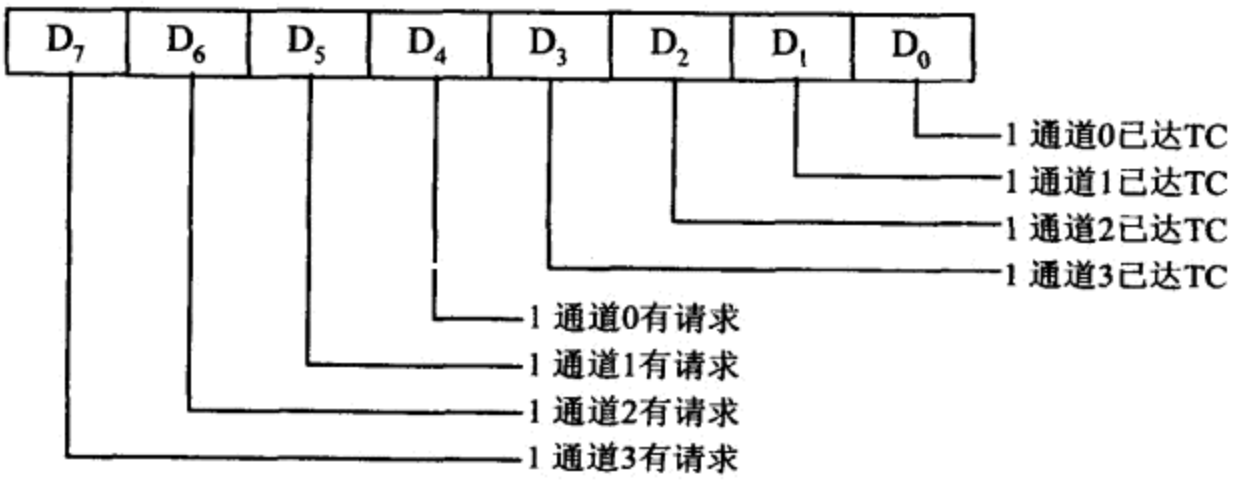


图 8-9　状态寄存器格式

8.4.10　暂存寄存器

暂存寄存器是一个 8 位的寄存器，在存储器至存储器传送期间，用来暂存从源地址单元读出的数据。当数据传送完成时，所传送的最后一个字节数据可以由 CPU 读出。用 RESET 信号可以清除此暂存器。

8.4.11　软件命令

8237A 设置了 3 条软件命令，它们是：主清除命令、清除字节指示器命令和清除屏蔽寄存器命令。这些软件命令只要对某个适当地址进行写入操作就会自动执行清除命令。

（1）主清除命令。该命令与硬件的 RESET 具有相同的功能。它使控制寄存器、状态寄存器、各通道的请求寄存器、暂存寄存器和字节指示器清 0；使屏蔽寄存器的各位置 1，使 8237A 进入空闲周期，以便进行编程。此命令寄存器地址的低 4 位为 0DH。

（2）清除字节指示器命令。字节指示器又称为先/后触发器或字节地址指示触发器。因为 8237A 各通道的地址和字节寄存器是 16 位的，而 8237A 的数据总线为 8 位，所以 CPU 访问这些寄存器时，要用连续两个字节进行。当字节指示器为 0 时，CPU 访问这些 16 位寄存器的低字节；当字节指示器为 1 时，CPU 访问这些 16 位寄存器的高字节。为了按正确顺序访问 16 位寄存器的高字节和低字节，CPU 首先使用清除字节指示器命令来清除字节指示器，使 CPU 第一次访问 16 位寄存器的低字节。第一次访问后，字节指示器自动置 1，这样使 CPU 第二次访问 16 位寄存器的高字节。而后，字节指示器又自动恢复为 0。此命令寄存器地址的

低 4 位为 0CH。

（3）清除屏蔽寄存器命令。这条命令清除 4 个通道的全部屏蔽位，使各通道均能接受 DMA 请求。此命令寄存器地址的低 4 位为 0EH。

8.5 8237A 的编程及应用

在进行 DMA 传送之前，CPU 要对 8237A 进行初始化编程。对 8237A 的编程实际上就是利用 OUT 指令对相应通道或寄存器写入命令或数据，使 DMA 控制器处于选定的工作方式从而进行指定的操作。

1. 8237A 主要寄存器的端口地址分配

每片 8237A 有 4 个地址选择线 A_3~A_0，故占用 16 个连续的端口地址，地址的高 4 位 0。为了便于编程，这里给出各个寄存器对应的端口地址，如表 8-2 所示。

表 8-2 8237A 寄存器的寻址

A3	A2	A1	A0	通道号	读操作（/IOR）	写操作（/IOW）
0	0	0	0	0	读当前地址寄存器	写当前（基）地址寄存器
0	0	0	1		读当前字节数寄存器	写当前（基）字节数寄存器
0	0	1	0	1	读当前地址寄存器	写当前（基）地址寄存器
0	0	1	1		读当前字节数寄存器	写当前（基）字节数寄存器
0	1	0	0	2	读当前地址寄存器	写当前（基）地址寄存器
0	1	0	1		读当前字节数寄存器	写当前（基）字节数寄存器
0	1	1	0	3	读当前地址寄存器	写当前（基）地址寄存器
0	1	1	1		读当前字节数寄存器	写当前（基）字节数寄存器
1	0	0	0	公共	读状态寄存器	写命令寄存器
1	0	0	1		-	写请求寄存器
1	0	1	0		-	写屏蔽寄存器某一位
1	0	1	1		-	写模式控制寄存器
1	1	0	0		-	清除字节指示器
1	1	0	1		读暂存寄存器	主清除
1	1	1	0		-	清除屏蔽寄存器
1	1	1	1		-	写屏蔽寄存器所有位

2. 8237A 编程的一般步骤

对 8237A 进行初始化编程的步骤如下。

（1）输出主清除命令，使 8237A 处于复位状态，以接收新的命令。

（2）写入工作方式寄存器，以确定 8237A 工作方式和传送类型。

（3）写入命令寄存器，以控制 8237A 的工作。

（4）根据所选通道，输入相应通道当前地址寄存器和基地址寄存器的初始值。

（5）输入当前字节计数器和基字节寄存器的初始值。

（6）写入屏蔽寄存器。

（7）写入请求寄存器，便可由软件 DMA 传送；否则，经过（1）～（6）步编程后，由通道 DREQ 启动 DMA 传送过程。

【例 8.1】在某一个系统中，用一片 8237A 设计了 DMA 传输电路，8237A 的基地址为 00H。要求利用它的通道 0，从外设（如磁盘）输入一个 1KB 的数据块，传送到内存中 6000H 开始的区域中，每传送一个字节，地址增 1，采用数据块连续传送方式，禁止自动预置，外设的 DMA 请求信号 DREQ 和响应信号 DACK 均为高电平有效。初始化 8237A 的程序如下：

```
DMA   EQU   00H                    ;8237A 的基地址为 00H
;输出主清除命令
OUT   DMA+0DH,AL                   ;发总清除命令
;将基地址 6000H 写入通道 0 基地址和当前地址寄存器,分两次进行
MOV   AX,6000H                     ;基地址和当前地址寄存器
OUT   DMA+00H,AL                   ;先写入低 8 位地址
MOV   AL,AH
OUT   DMA+00H,AL                   ;后写入高 8 位地址
;把要传送的总字节数 1K=400H 减 1 后,送到基字计数器和当前字计数器
MOV   AX,0400H                     ;总字节数
DEC   AX                           ;总字节数减 1
OUT   DMA+01H,AL                   ;先写入字节数的低 8 位
MOV   AL,AH
OUT   DMA+01H,AL                   ;后写入字节数的高 8 位
;写入方式字:数据块传送,地址增量,禁止自动预置,写传送,选择通道
MOV   AL,10000100B                 ;方式字
OUT   DMA+0BH,AL                   ;写入方式字
;写入屏蔽字: 通道 0 屏蔽位清 0
MOV   AL,00H                       ;屏蔽字
OUT   DMA+0AH,AL                   ;写入 8237A
;写入命令字:DACK 和 DREQ 为高电平,固定优先级,非存储器间传送
MOV   AL,10000000B                 ;命令字
OUT   DMA+08H, AL                  ;写入 8237A
;写入请求字:通道 0 产生请求
MOV   AL,04H                       ;请求字
OUT   DMA+09H,AL                   ;将请求字写入 8237A,用软件启动 8237A
```

3. 8237A 的应用

在 PC/XT 机中，用一片 8237A-5 构成 DMA 控制电路形成 4 个 DMA 通道，提供数据宽度为 8 位的 DMA 传输。使用固定优先级，所以通道 0 的优先级最高，通道 3 最低。这 4 个 DMA 通道的功能分配如下：

通道 0　　用于动态 RAM 的刷新

通道 1　　为用户保留

通道 2　　用于软盘 DMA 传送

通道 3　　用于硬盘 DMA 传送

虽然，8237A 既能提供外设和存储器之间的 DMA 传输，也能进行存储器和存储器之间的 DMA 传输，但在 PC/XT 机的 BIOS 初始化系统时，将 8237A 的存储器和存储器间传送方式禁止掉了，因此只用它实现外设和内存间的高速数据交换。下面就以磁盘 DMA 传输为例，

对其初始化程序进行介绍。

在 PC/XT 机进行软盘或硬盘 DMA 传输时，先要对 8237A-5 进行编程。下面是 ROM BIOS 中的一段程序，名为 DMA_SETUP，位于首地址为 FEEC8H 的内存中。它被读软盘、写软盘和软盘校验程序等调用，用来向 8237A-5 输入所要读/写数据的 20 位首地址和字计数器初始值。调用前要求的入口参数是：

AL=DMA 方式字（读= 46H，写= 4AH，校验= 42H）。

DH=要传送的扇区个数。每个扇区的字节数基数为 128，实际由磁盘驱动器的规格而定，可能是 128，256，512 或 1024 个，也就是说可能是 128 的 1，2，4，8 倍，表示成 2 的 N 次方，N 分别为 0、1、2、3。N 存放在磁盘基值区 DISK_BASE 的第 03 号单元中，可由它和 DH 中的扇区数计算出实际要传送的总字节数。

```
ES: BX=所读/写数据的内存首地址（段地址: 偏移量）
     DMA_SETUP  PROC  NEAR
     PUSH  CX                ;保护 CX 的值
     CLI                     ;关中断
     OUT   DMA+0CH,AL        ;清除先/后触发器
     PUSH  AX                ;延时,满足 8237A I/O 定时要求
     POP   AX
     OUT   DMA+0BH,AL        ;将 AL 中的方式字写入方式寄存器
     ;计算 20 位物理地址,结果的最高 4 位放在 CH 中,低 16 位放入 AX
     MOV   AX,ES             ;AX←段基地址
     MOV   CL,4              ;循环次数
     ROL   AX,CL             ;AX 循环左移 4 位
     MOV   CH,AL             ;AX 的低 8 位暂存入 CH
     AND   AL,0F0H           ;AX 的最低 4 位清零
     ADD   AX,BX             ;加上偏移量,形成 16 位地址（A15~A0）
     JNC   RLL               ;无进位,转移
     INC   CH                ;有进位,最高 4 位加 1。至此形成了 20 位物理地址,低 16 位在
                             ;AX 中,最高 4 位在 CH 的低 4 位中
RLL: PUSH    AX             ;保存 16 位起始地址
     ;将 16 位地址送到通道 2 基址和当前地址寄存器中
     OUT   DMA+4,AL          ;先写低字节
     MOV   AL,AH
     OUT   DMA+4,AL          ;后写高字节
     ;将最高 4 位地址预置到页面寄存器中
     MOV   AL,CH             ;最高 4 位地址
     AND   AL,0FH            ;截低 4 位
     OUT   81H,AL            ;置入页面寄存器(通道 2,软盘 DMA 传送)
     MOV   AH,DH             ;取传输的扇区数送 AH
     SUB   AL,AL             ;AL 清零后,使 AX= 256×扇区数
     SHR   AX,1              ;将 AX 内容除以 2 后,AX= 128×扇区数
     ;调用取软盘参数子程序，要求入口参数: BX= 字节索引值(03 号单元)×2
     PUSH  AX                ;保存 AX 的值
     MOV   BX,6              ;BX= 字节索引值×2
     CALL  GET_PARM          ;调用取参数子程序
     ;调用后结果,AH= N,N 可以是 0、1、2 或 3,它表示所选软盘每扇区的字节数: 0 或
     ;128,1 为 256,2 为 512,3 为 1024
     MOV   CL,AH             ;基数值 N 由 AH 送 CL
     POP   AX                ;弹出"128×扇区数"送 AX
```

```
SHL    AX,CL              ;左移后,AX←AX×N
DEC    AX                 ;传输总字节数减 1
PUSH   AX                 ;保存此值（即基字计数值）
;将减 1 后的字节数送通道 2 基字和当前字计数器
OUT    DMA+5,AL           ;先送高字节
MOV    AL,AH
OUT    DMA+5,AL           ;后送高字节
STI                       ;开中断
POP    CX                 ;恢复计数值（基字计数值）
POP    AX                 ;恢复低 16 位起始地址
;将基地址和基字计数器值相加,判别是否有进位,若有,表示超出 64K,出错
;因为 DMA 传送过程中,页寄存器的值是不变的,8237A 只允许在 64K 范围内变化
ADD    AX,CX              ;相加,根据结果建立进位位 CF
POP    CX                 ;恢复进入子程序时保护的 CX 值
MOV    AL,02H
OUT    DMA+0AH, AL        ;清除通道 2（软盘 DMA 传送）的屏蔽位
RET                       ;返回,若 CF=1,则 64K 溢出,应减少传送的扇区数,字组
                          ;的传送要分块进行
DMA_SETUP  ENDP
```

本章小结

本章主要介绍可编程控制器 8237A 的内部结构及其引脚，内部寄存器的功能及其格式，8237A 的编程及其应用等。

DMA 控制器 8237A 有总线主模块和总线从模块两种不同的工作状态。在总线主模块下 DMA 控制器可以直接控制系统总线，在总线从模块下和其他接口一样，接受 CPU 对它的读/写操作。

8237A DMA 控制器含有 4 个独立的 DMA 通道，可以用来实现内存到接口、接口到内存及内存到内存之间的高速数据传送。

思考与练习题

1. 8237A DMA 控制器有哪两种工作状态，其工作特点如何？
2. 8237A DMA 控制器的当前地址寄存器、当前字计数寄存器和基字寄存器各保存什么值？
3. 8237A 进行 DMA 数据传送时有哪几种传送方式？
4. 8237A 可执行哪几条软件命令？
5. 若 8237A 的端口基地址为 000H，要求通道 0 和通道 1 工作在单字节读传输，地址减 1 变化，无自动预置功能。通道 2 和通道 3 工作在数据块传输方式，地址加 1 变化，有自动预置功能。8237A 的 DACK 为高电平有效，DREQ 为低电平有效，用固定优先级方式启动 8237A 工作，试编写 8237A 的初始化程序。

第 9 章 中断技术

现代微型计算机系统的中断系统功能强弱已成为评价系统整体性能的一项重要指标。本章对中断的基本知识、8086 中断系统和可编程中断控制器 8259A 进行了讨论。主要介绍中断技术概述、8086 的中断类型、中断优先权及其管理、中断矢量、中断处理过程、中断嵌套等，通过可编程中断控制器 8259A 介绍中断技术的应用。

通过本章的学习，读者应该熟悉中断的基本概念、特点、中断处理过程等基础知识，掌握可编程中断控制器 8259A 的结构、原理、工作方式及编程应用，为今后中断技术的应用打下基础。

9.1 中断技术概述

中断是现代微型计算机系统中用以提高计算机工作效率的一种重要技术。最初，它只是作为计算机与外设交换信息的一种同步控制方式而提出来的，但随着计算机技术的发展，特别是 CPU 速度的迅速提高，对计算机内部机制的要求也就越来越高。如希望计算机能随时发现各种错误，当出现各种意想不到的事件时，能及时妥善地处理。于是，中断的概念延伸了，除了传统的外部事件（硬件）引起中断外，又产生了 CPU 内部软件中断的概念。

所谓“中断”是指 CPU 在正常执行程序的过程中，由于内部/外部事件或由程序的预先安排，引起 CPU 暂时中断当前程序的运行而转去执行为内部/外部事件或预先安排的事件服务的子程序，待中断服务子程序执行完毕后，CPU 再返回到暂停处（断点）继续执行原来的程序，这一过程称为中断，其示意图如图 9-1 所示。对于何时申请中断 CPU 并不知道，因此中断具有随机性。

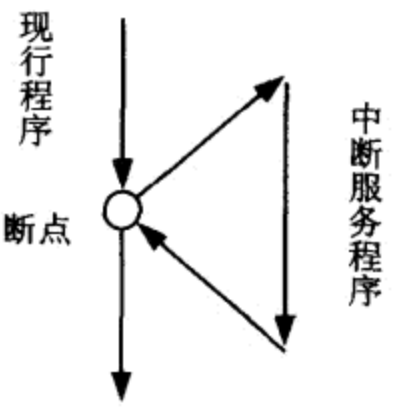


图 9-1 中断过程示意图

可以将“中断”想象成门铃，只有当门铃响时，用户才会放下（停止）手头的事情，走去开门；如果门铃没有响，用户就可以继续做其他事情，而不必定时去查看是否有客人到了门前，因为用户不可能知道何时会有随机来访的客人。

在计算机系统中，中断的例子很多。用户使用键盘时，每击一键都发出一个中断信号，告诉 CPU 有“键盘输入”事件发生，要求 CPU 读入该键的键值。打印机打印字符时，当打印完一个字符，它要发出“打印完成”的中断信号，告诉 CPU 一个字符已打印完毕，要求送来下一个字符。当磁盘驱动器准备好把一个扇区的数据传送到主存时，它会发出“数据传送准备好”的中断信号，告诉 CPU，要求 CPU 处理。串行通信中，当串行线路上已到达了一个字符时，就会引起“接收数据准备好”的中断，要求及时读入这个字符。数据采集系统中，ADC 转换启动后，就开始转换，一旦转换结束，发出“转换完毕”的中断信号，要求 CPU 读取数据。

利用中断可以大大提高 CPU 的工作效率。实现中断功能的控制逻辑称为中断机构或中断系统。

9.1.1　中断技术的优点

在现代微机系统中采用中断技术后，可以带来以下优点。

（1）并行操作。由于 CPU 的运行速度很快，而多数外部设备的工作速度较慢，在 CPU 和外设传送数据时，CPU 要花费大量的时间等待与外设交换数据，致使 CPU 的利用率降低。为了使 CPU 与外设能以并行的方式进行工作，可通过中断技术来实现。

在 CPU 执行程序过程中，如需要和某个外设传送数据，则 CPU 先启动外设工作，即执行一条启动该设备工作的命令，然后继续执行现行程序，同时外设也在工作。当外设为传送数据作好准备后，向 CPU 发出中断请求信号，请求 CPU 暂时停止正在执行的程序。CPU 响应后，终止执行现行程序（一般称为主程序），转去执行中断服务程序，以完成数据的输入或输出操作。当中断服务程序执行完毕，CPU 恢复执行主程序。

（2）实时处理。微型计算机的重要应用领域之一是用来进行实时信息的采集、处理和控制。所谓“实时”是指计算机能够对现场采集到的信息及时作出分析和处理，以便对被控制的对象立即作出响应，使被控制的对象保持在最佳工作状态。

在实时系统中，系统要求计算机随时对其进行处理，系统的响应速度是一个重要指标。利用中断技术可以及时处理随机输入到微型计算机的各种参数和信息，使微型计算机具备实时处理与控制的能力。如果没有中断技术，对随机输入信息的实时处理是很难实现的。因此，在实时控制系统中，中断技术得到了广泛应用。

（3）故障处理。计算机在运行过程中，往往会随机出现一些无法预料到的故障情况，如电源和硬件故障，数据存储和运算错误等。有了中断系统，CPU 可以根据故障源发出的中断请求，立即去执行相应的故障处理程序，自行处理故障而不必停机，因此提高了微型计算机工作的可靠性，降低了损失。

（4）分时操作。采用中断技术后，CPU 可以连接多个外设同时工作，还可以设置多个外设的中断优先级，根据提出中断请求外设的优先级别的高低，分时执行各自的中断服务程序。由此可见，中断方式不仅可以实现 CPU 和外设之间的并行工作，而且 CPU 可命令多个外设同时工作，这大大提高了 CPU 的利用率，也加快了输入/输出的速度。

在现代微型计算机系统中，随着计算机软、硬件技术的发展，中断技术也得到了不断地发展和完善，中断系统功能的强弱已成为评价计算机系统整体性能的一项重要指标。

9.1.2　中断源

能引起中断的外部设备或内部原因称为中断源。不同的计算机系统，中断源的设置有所不同，通常按照 CPU 与中断源的位置关系可分为内部中断和外部中断。内部中断是 CPU 在处理某些特殊事件时所引起或通过内部逻辑电路自己去调用的中断。外部中断是由于外部设备要求数据输入/输出操作时请求 CPU 为之服务的一种中断。

1. 中断源种类

通常中断源有以下几种。

（1）设备中断：如键盘、打印机等设备的数据传送请求等。

（2）指令中断：为了方便用户使用系统资源或调试软件而设置的中断指令，由程序预先安排的中断指令（INT N）引起，如调用 I/O 设备的 BIOS 及 DOS 系统功能的中断指令和设置断点中断等。

（3）故障中断：计算机内部设有故障自动检测装置，如电源掉电、奇偶校验错或协处理器中断请求等意外事件，都能使 CPU 中断，进行相应的中断处理。

（4）实时时钟中断：在自动控制系统中，常遇到定时检测与时间控制，这时可采用外部时钟电路进行定时。CPU 可发出命令启动时钟电路开始计时，待定时时间到，时钟电路就会向 CPU 发出中断申请，由 CPU 进行处理。

（5）CPU 内部运算产生的某些错误所引起的中断：如除法出错、运算溢出、程序调试中设置断点等。

2. 中断源识别

在实际的微机系统中，由于中断申请是随机的，有时会出现多个中断源，但 CPU 每次只能响应一个中断请求。因此，当外设与 CPU 以中断方式进行信息交换时，CPU 必须要从多个外设中判别出正在申请中断的设备，找到相应的中断服务程序的入口地址，才能转去为其服务。识别中断源通常有两种方法：查询中断和矢量中断。

（1）查询中断

查询中断时采用软件或硬件（串行顺序链电路）查询技术来确定发出中断请求的中断源。当 CPU 接收到中断请求信号后，通过执行一段查询程序，逐个询问外设，从多个可能的外设中查找申请中断的那个外设。

（2）矢量中断

矢量中断又称向量中断，是识别中断源最快的一种方法。矢量中断是由中断向量来指示中断服务程序的入口地址，每个中断源都预先指定一个矢量标志，外设在提出中断请求时要提供该中断矢量标志。CPU 响应某个中断源的中断请求时，控制逻辑就将该中断源的矢量标志送入 CPU，CPU 根据矢量标志自动指向相应的中断服务程序的入口地址，转入中断服务程序。8086 就是以矢量中断的方法实现中断源识别的。矢量中断又称为中断类型码。

矢量中断主要是靠硬件实现的，在微机系统中所用的可编程中断控制器都能提供中断矢量标志。矢量中断是以硬件开销换取中断响应速度，而查询中断是以额外软件为代价来节省硬件的，有时候混合使用这两种方法会取得更好的中断源识别效果。

9.1.3 中断处理过程

一个微机系统的中断处理过程大致可分为中断请求、中断响应、中断服务和中断返回 4 个过程，如图 9-2 所示。这些步骤有的是通过硬件电路完成的，有的是由程序员编写程序来实现的。

1. 中断请求

当中断源需要 CPU 为其服务时，可以向 CPU 发出中断请求。中断请求可以是由中断指令或是某些特定条件产生，也可以是通过 CPU 引脚向 CPU 发出中断请求信号而产生。但一般应满足以下两个条件：中断源已处于准备就绪状态；系统允许该中断源发出中断请求，即

该中断源未被屏蔽。

由于外设的中断请求是随机发生的，而大多数 CPU 都是在现行指令周期结束时，才检测有无中断请求信号，因此系统中必须设置一个中断请求触发器，把随机输入的中断请求信号锁存起来，并保持到 CPU 响应这个中断请求后，才能被清除。当中断源有中断请求时，将中断请求触发器相应位置 1。

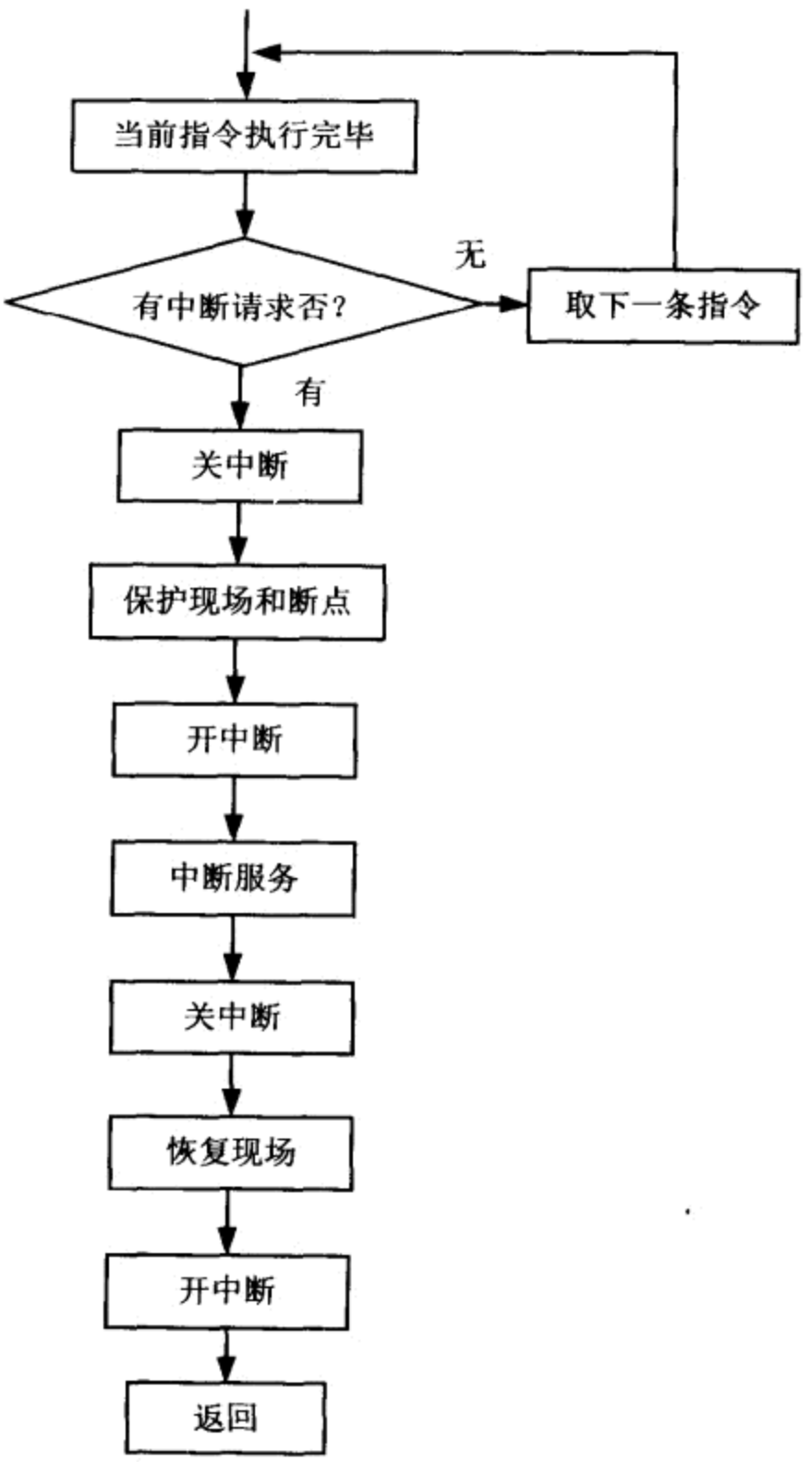


图 9-2　微机系统的中断处理过程

实际系统中往往有多个中断源。为了增加控制的灵活性，常要求在每一个外设的接口电路中设置一个中断屏蔽触发器。若中断屏蔽触发器为“0”表示允许该中断源向 CPU 发出中断“信号”，CPU 响应此中断后将中断请求触发器清 0；若中断屏蔽触发器为“1”表示禁止该中断源向 CPU 发出中断“信号”，称该中断请求被屏蔽。

2. 中断响应

CPU 没有接到中断请求信号时一直执行原来的主程序。当 CPU 接收到外设的中断请求信号时，能否立即为其服务呢？这就要看中断的类型。若为非屏蔽中断请求，则 CPU 执行完现行指令后，就立即响应中断。若为可屏蔽中断请求，能否响应中断，还取决于 CPU 内部的中断允许触发器的状态。只有当其为“1”（允许中断），CPU 才能响应可屏蔽中断：若其为“0”（禁止中断），即使有可屏蔽中断请求，CPU 也不响应。

CPU 要响应可屏蔽中断请求，必须满足 3 个条件：① 无总线请求；② CPU 允许中断；③ CPU 执行完现行指令。

可用开中断指令 STI 和关中断指令 CLI 来设置中断允许触发器的状态。当 CPU 复位时，中断允许触发器为“0”，即关中断。所以为了响应可屏蔽中断请求，必须用 STI 指令来开中断。非屏蔽中断不受中断允许触发器的影响。

当 CPU 响应中断进入中断响应周期时，自动完成以下操作。

（1）关中断。CPU 响应中断，对外发出中断响应信号 INTA 的同时，内部自动由硬件实现关中断，以禁止接受其他的可屏蔽中断请求。

（2）保护断点。将正在执行的程序地址（称为断点）的 CS、IP 值压入堆栈保护，以保证中断返回时能正确回到原来的断点处。

（3）保护现场。把标志寄存器 FLAGS 的内容压入堆栈，其余寄存器的保护由中断服务程序来实现。

（4）中断服务程序段地址送入 CS，偏移地址送入 IP。不同类型的 CPU 形成中断服务程序入口地址方式不同。小系统中的中断源较少，可采用固定入口地址的方法；当中断源较多时，通常采用向量表的方法，8086 微处理器采用后一种方法。

3. 中断服务

中断服务是指 CPU 执行中断服务程序，一般有如下操作。

（1）保护有关寄存器内容。CPU 响应中断时自动完成 CS、IP 寄存器及标志寄存器 Flags 的保护，但主程序中使用的其他寄存器要由用户根据情况而定，可以用入栈指令 PUSH 将有关寄存器的内容送入堆栈保护。

（2）开中断。CPU 接收并响应一个中断请求后自动关闭中断，其好处是不允许其他可屏蔽中断来打扰。但在某些情况下为了能实现中断嵌套，必须在中断服务程序中开中断。

中断嵌套是当 CPU 响应某一中断请求并为其服务时，若有优先权更高的中断源发出中断请求，则要求 CPU 能打断正在执行的中断服务程序，响应更高级别的中断请求。在高级别中断请求处理完后，再返回被打断的中断服务程序继续执行，即实现中断嵌套。CPU 允许高优先级中断请求可以打断低优先级中断服务，使 CPU 对于急需处理的事件立即作出响应。

（3）中断服务。CPU 通过执行中断服务程序，完成对中断情况的处理。例如：传送数据、处理掉电故障、各种错误处理等，这是中断服务程序的核心。

4. 中断返回

中断返回是由执行中断返回指令 IRET 来完成的。通常情况下，中断服务程序的最后一条指令是 IRET。当 CPU 执行该指令时，自动把断点地址从堆栈中弹出到 CS 和 IP 中，原来的标志寄存器内容弹回 FLAGS。这样被中断的程序就可以从断点处继续执行。

在中断返回时要进行以下操作。

（1）关中断。若中断服务程序设置了开中断，则此时应关中断，以保证下一步恢复现场的操作不被打扰。

（2）恢复现场：在返回主程序之前要将用户保护的寄存器内容从堆栈中弹出，以便能正确执行主程序。恢复现场按后进先出的原则，通过执行 POP 指令将送入堆栈保护的寄存器弹出，注意 PUSH 指令和 POP 指令应成对使用，且弹出时的顺序与入栈的顺序正好相反。

（3）开中断：中断服务程序的倒数第二条指令往往是开中断指令，此处的开中断与前面的关中断相对应，目的是使 CPU 能继续接收中断请求。

9.1.4　中断优先权

实际系统中有可能出现两个或两个以上的中断源同时提出中断请求的情况。在多个中断源同时请求中断时，CPU 必须首先确定为哪一个中断源服务，要能辨别优先权最高的中断源并响应之。当 CPU 在处理中断时，也要能响应更高级别的中断申请，而屏蔽掉同级或较低级的中断申请，这就是中断优先权问题。

在中断系统中，CPU 一般要根据各中断请求的轻重缓急分别处理，即给每个中断源确定一个中断优先级别——中断优先权，系统能够自动地对它们进行排队判优，保证首先处理优先级别高的中断请求，待级别高的中断请求处理完毕后，再响应级别较低的中断请求。

通常有两种方法解决中断优先权的识别问题。

1．用软件查询方法确定中断优先权

采用软件查询中断方式时，中断优先权由查询顺序决定，最先查询的中断源具有最高的优先权。使用这种方法也需要配以一定的硬件，需要设置一个中断请求信号的锁存接口，将各申请中断的请求信号锁存下来以便查询。该方法首先要把外设的中断请求触发器组合起来作为一个端口，供 CPU 查询，同时把这些中断请求信号相“或”后，作为 INTR 信号，这样任一外设有请求都可向 CPU 送 INTR 信号。CPU 响应中断后，读入中断寄存器的内容并逐位检测它们的状态，若有中断请求（相应位=1）就转到相应的服务程序入口，检测的顺序就是优先级的顺序。软件查询方法的接口电路如图 9-3 所示。

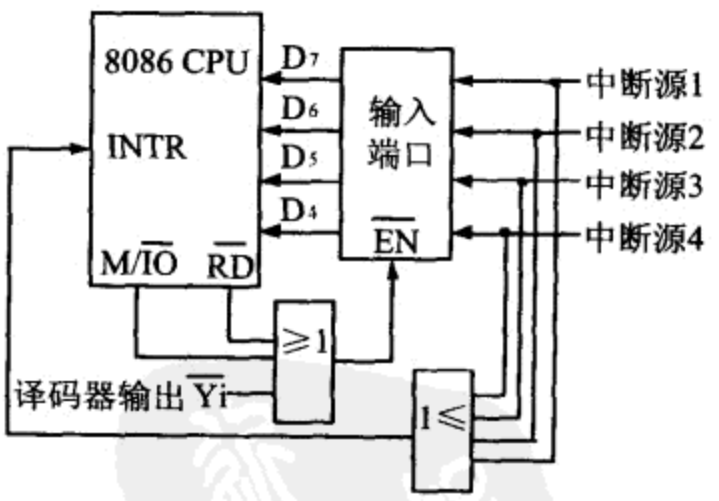


图 9-3　软件查询方法的接口电路

软件查询方法的优点是电路比较简单。软件查询的顺序就是中断优先权的顺序，不需要专门的优先权排队电路，可以直接修改软件查询顺序来修改中断优先权，不必更改硬件。缺

点是当中断源个数较多时，由逐位检测查询到转入相应的中断服务程序所耗费的时间较长，中断响应速度慢，服务效率低。

2. 硬件优先权排队电路

硬件优先权排队电路形式众多，有采用编码器组成的，有采用链式电路的。

硬件链式优先权排队电路又称为菊花环式优先权排队电路，它是利用外设连接在排队电路的物理位置来决定其中断优先权的，排在最前面的优先权最高，排在最后面的优先权最低，电路如图 9-4 所示。

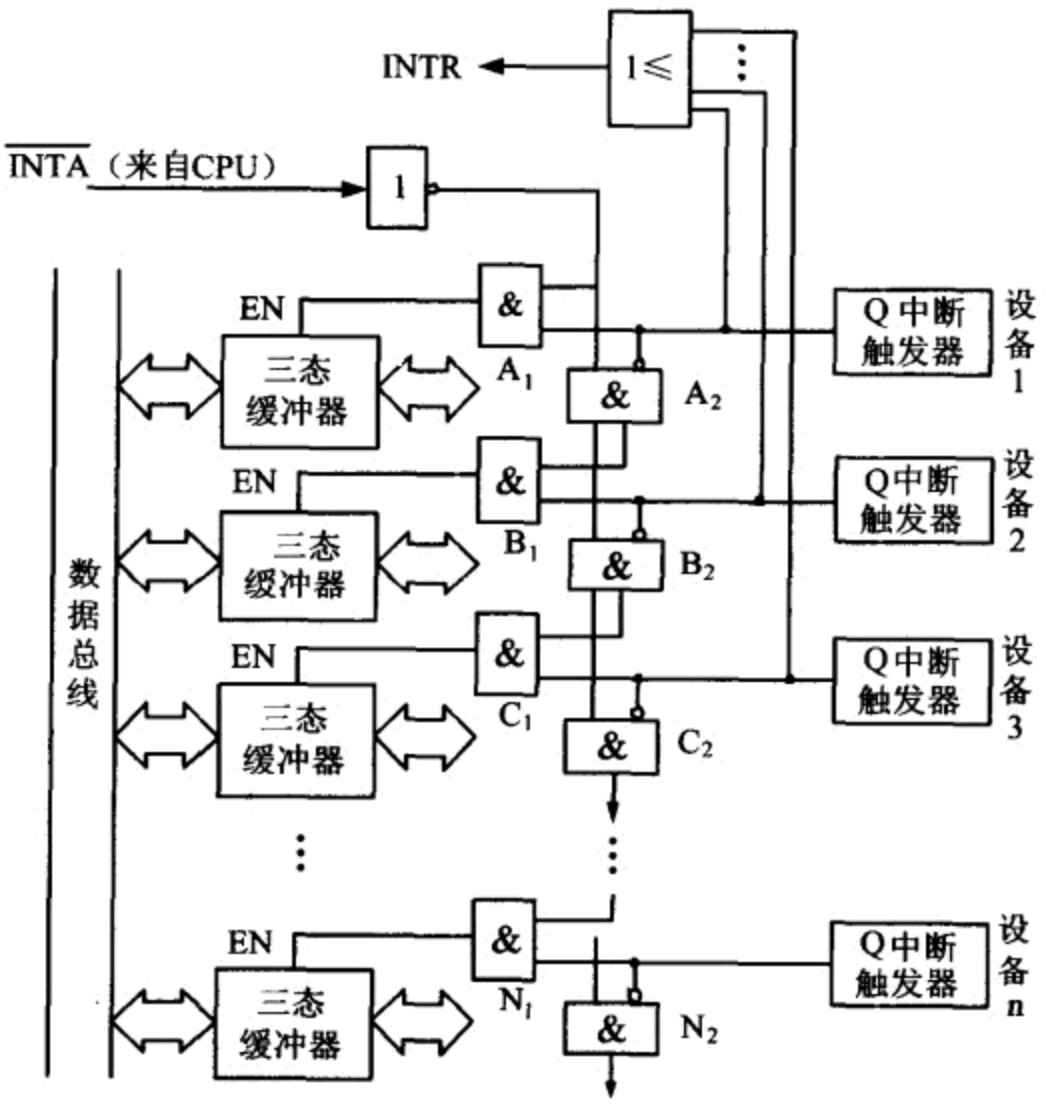


图 9-4 链式优先权排队电路

当多个外设有中断请求时，则由中断请求信号或电路产生的 INTR 信号，送至 CPU。CPU 在现行指令执行完后响应中断，发出中断响应信号 $\overline{\text{INTA}}$（低电平有效）。CPU 究竟响应哪个中断呢?

当中断请求得到响应时，中断响应信号 $\overline{\text{INTA}}$ 就传送到优先权最高的设备 1，并按串行方式往下传送。当设备 1 有中断请求时，则它的中断触发器 Q_1 输出为高，于是与门 A_1 输出为高，设备 1 的数据允许线 EN 变为有效，从而允许设备 1 使用数据总线，将其中断类型码经数据总线送入 CPU，由它控制中断矢量 1 的发出。CPU 收到中断矢量 1 后转至设备 1 的中断服务程序入口。同时 A_2 经反相为低电平，则中断响应信号在门 A_2 处被封锁，使 B_1、B_2、C_1、C_2、……所有下面各级输出全为低电平，中断响应信号 $\overline{\text{INTA}}$ 不再下传，使后继设备得不到 CPU 的中断响应信号，$\overline{\text{INTA}}$ 即屏蔽了所有的低级中断。

若设备 1 没有中断请求，则中断响应输出 Q_1 为低电平，即 $Q_1=0$，此时 A_2 输出为高电平，

中断响应信号可以通过 A_2 门传给下一设备 2。若此时 Q_2=l，则 B_1 输出为高。控制转向中断服务程序 2，B_2 输出为低，屏蔽以下各级。若 Q_2=0，则中断响应信号传至中断设备 3，其余各级类推。

综上所述，在链式优先权排队电路中，若上一级的中断响应传递出信号为 0，则屏蔽了本级和所有的低级中断；若上一级的中断响应传递输出信号为 1，在本级有中断请求时，则转去执行本级的中断服务程序，且使本级传递至下级的中断响应输出为 0，屏蔽所有低级中断；若本级没有中断请求，则允许下一级中断。故在链式电路中，排在最前面的中断源优先权最高。

9.2　8086 的中断结构

8086 有一个简单而灵活的中断系统，每个中断都有一个类型码，供 CPU 进行识别。8086 可以处理多达 256 种不同类型的中断。中断可以由外部设备启动，也可以由软件中断指令启动，在某些情况下，也可由 CPU 自身启动。

9.2.1　中断类型

8086 采用了矢量型的中断结构，共有 256 个中断矢量号，又称中断类型号。这种中断结构既简单又灵活，而且响应速度快。

8086 的中断源可分为两大类：一类是内部中断，另一类是外部中断，如图 9-5 所示。其中虚线下面是内部中断，虚线上面是外部中断。

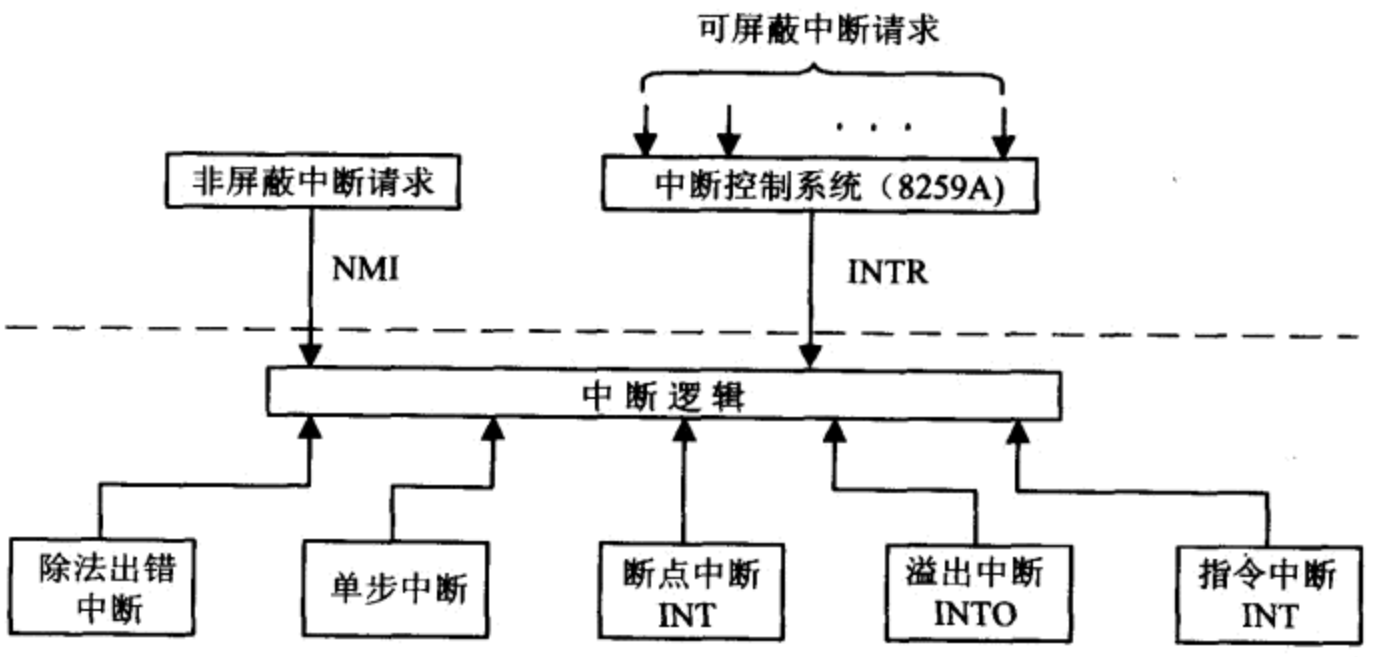


图 9-5　8086 的中断源

1. 内部中断

内部中断也称软件中断，是由处理器检测到异常情况或执行软件中断指令所引起的一种中断。通常有除法出错中断、单步中断、断点中断、INTO 溢出中断、INT N 中断等。

（1）除法出错中断

在执行除法指令 DIV 或 IDIV 时，若发现除数为 0 或商超过了目的寄存器所能表达的范

围，则 CPU 会立即产生一个中断类型码为 0 的内部中断。

（2）单步中断（类型 1）

当标志寄存器中的 TF 位为 1 时，CPU 就处于单步工作方式。这时 CPU 在每条指令执行完后自动产生中断类型码为 1 的内部中断，把标志寄存器的内容和断点压入堆栈保存起来，然后将 TF 和 IF 清零。这样，当 CPU 进入单步中断服务程序时，它就不处于单步工作方式，而以正常的方式工作，即单步中断服务程序仍连续执行。当单步中断过程结束时，由中断返回指令 IRET 从堆栈中将原来保存的标志内容弹出堆栈，恢复到标志寄存器中，此时 TF=1，使 CPU 返回单步工作方式，当下一条指令执行后又产生新的单步中断。

单步中断是一种有用的调试工具，它成为能逐条指令地观察系统操作的一个“窗口”。例如单步中断过程可以在每执行一条指令后打印或显示寄存器内容、指令指针的值以及关键的存储变量等，这样就能详细跟踪一个程序的具体执行过程，确定问题所在。

（3）断点中断（类型 3）

8086 微处理器的指令系统中有一条设置程序断点的单字节中断指令 INT 3。执行完这条指令就立即产生一个中断类型码为 3 的内部中断，主要用于程序调试。程序员可用它在程序中设置一个程序断点，程序执行到断点处 CPU 就会转去执行断点中断服务程序，以进行某些检查和处理。例如 DEBUG 调试程序的 G 命令允许设置多达 10 个程序断点，并对断点处的指令执行结果进行显示，供用户调试程序时检查。

（4）INTO 溢出中断（类型 4）

若上一条指令执行的结果使溢出标志位 OF=1，则执行中断指令 INTO 时，将引起中断类型码为 4 的内部中断。若 OF=0 时，溢出中断指令 INTO 不起作用，程序执行下一条指令。与除法出错中断不同，溢出状态不会自动产生中断请求，OF=1 仅是一个必要条件。两个条件中任何一个条件不具备，溢出中断则不发生。

用户在编程时，若要对某些运算操作加以监控时，应在这些操作指令之后加一条 INTO 指令，并设计相应的运算溢出中断服务程序，以便在算术运算过程中出现溢出错误时，能及时进行处理。

（5）INT N 指令

在 8086 指令系统中有一类中断指令 INT N，其中 N 为中断类型码（范围为 0～255）。CPU 执行一条这种指令，就会发生一次中断。在 IBM PC/XT 的操作系统中，用不同类型码代表一些标准的中断服务程序，用户程序可以用 INT N 指令方便地调用。

内部中断的特点如下。

- 中断矢量号是由 CPU 自动提供的，不需要执行中断响应总线周期去读取矢量号。
- 除单步中断外，所有内部中断都无法禁止，即都不能通过执行 CLI 指令使 IF 位清零来禁止对它们的响应。
- 除单步中断外，任何内部中断的优先权都比外部中断高。

2. 外部中断

外部中断也称硬件中断，是由 CPU 的外部中断请求信号触发的一种中断，分为不可屏蔽中断 NMI 和可屏蔽中断 INTR。

（1）不可屏蔽中断请求 NMI

由 NMI 引脚引入的是不可屏蔽中断请求，它不受中断允许标志位 IF 的影响。当 CPU 在 NMI 引脚上采样到一个由低到高的跳变信号时，就自动进入 NMI 中断服务程序。即使在关中断（IF=0）的情况下，CPU 也能在当前指令执行完毕后就响应 NMI 上的中断请求。

不可屏蔽中断请求 NMI 的中断类型码为 2，CPU 接收到 NMI 提供的中断请求以后，将自动按中断类型码 2，转入相应的 NMI 中断服务程序。

通常非屏蔽中断请求 NMI 用于通知 CPU 发生了紧急事件，如电源掉电、存储器读/写出错等。

（2）可屏蔽中断 INTR

由 INTR 引脚引入的是可屏蔽中断请求，高电平有效。若 CPU 发现有可屏蔽中断请求，将根据 IF 的状态决定是否响应。如果 IF=0，CPU 处于关中断状态，屏蔽 INTR 线上的中断，CPU 不理会该中断请求而继续执行下一条指令；如果 IF=1，CPU 处于开中断状态，允许 INTR 线上的中断，CPU 执行完现行指令后转入中断响应周期。

此时，CPU 要执行两个操作：首先发出 $\overline{\text{INTA}}$ 信号，通知外设中断请求已被接受，要求外设将相应的中断类型码送入数据总线的低 8 位（D_7～D_0）；然后 CPU 采样数据总线，获取中断类型码，转入相应的中断服务程序。

IF 的状态可以用 STI 指令使其置 1，也可以用 CLI 指令使其置 0。因此，对可屏蔽中断请求是否响应，可由软件来控制。当系统复位后，或当 8086 响应中断请求后，都会使 IF=0。此时要允许 INTR 中断请求，必须先用 STI 指令使 IF 位置 1，才能响应 INTR 的中断请求。

由于 8086 微处理器芯片仅有一条 INTR 线，直接由 INTR 引脚引入的中断只有一个，为增加其处理外部中断的能力，Intel 公司设计了专用的可编程中断控制器 8259A，用来管理多个外部中断。8259A 的 8 级中断请求输入端 IR_0～IR_7 依次接收需要请求中断的外部设备。这些外部设备请求中断时，请求信号输入 8259A 的 IR 端，由 8259A 根据优先权和屏蔽状态决定是否发出 INT 信号到 CPU 的 INTR 端。

3. 中断处理顺序

中断处理顺序即按中断优先权从高到低的排队顺序对中断源进行响应，8086 系统的中断处理次序如下。

（1）除法错误中断、溢出中断、INT N 指令中断、断点中断。

（2）非屏蔽中断 NMI。

（3）可屏蔽中断 INTR。

（4）单步中断。

9.2.2　中断向量表

中断向量是中断服务程序的入口地址，每一个中断服务程序都有一个唯一确定的入口地址。把系统中的所有中断向量集中起来放到存储器的某一区域内，这个存放中断向量的存储区就叫中断向量表（或中断矢量表）。

8086 的中断系统可处理 256 种不同的中断，对应的中断类型码为 0～255，每个中断类型码与一个中断服务程序相对应，每个中断服务程序都有自己的入口地址即中断向量。该入

口地址包括中断服务程序的段基址（CS）和偏移地址（IP），各占 2 个字节单元，每个中断向量需占用 4 个字节单元，256 个中断向量共需占用 1 024 个（即 1KB）字节单元。为了寻址的方便，8086 系统中在存储器的最低端地址从 0000H～03FFH 共 lKB 单元作为中断向量的存储区，即中断向量表，如图 9-6 所示，每个入口地址所占的 4 个字节单元中，都是两个高字节单元存放段基址 CS，两个低字节单元存放偏移地址 IP。

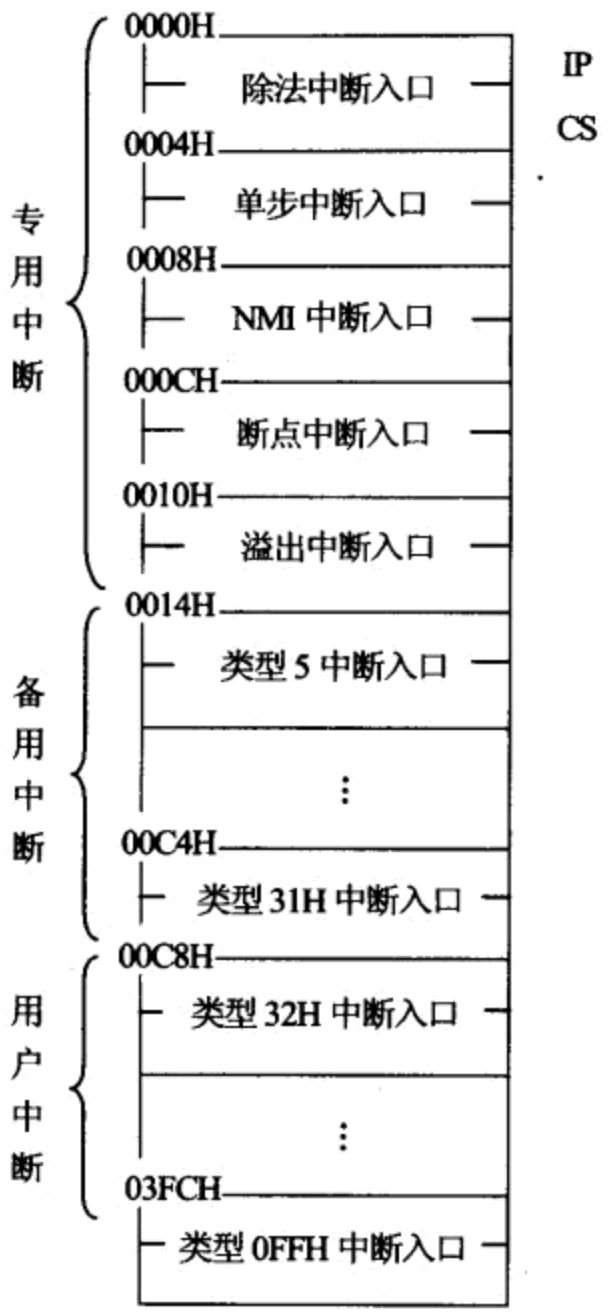


图 9-6　8086 的中断向量表

由图可见，中断向量表分为 3 部分：专用中断、备用中断和用户中断。

（1）专用中断：类型 0～类型 4，共有 5 种类型。专用中断的中断服务程序的入口地址由系统负责装入，用户不能随意修改。

（2）备用中断：类型 5～类型 3FH，这是 Intel 公司为软、硬件开发保留的中断类型，一般不允许用户改作其他用途。例如类型 10H～1FH 为 ROM—BIOS 中断，类型 21H 为 DOS 功能调用。

（3）用户中断：类型 40H～类型 FFH，为用户可用中断，其中断服务程序的入口地址由用户程序负责装入。这些中断可由用户定义为软件中断，由 INT N 指令引入，也可以通过 INTR 引脚直接引入或通过可编程中断控制器 8259A 引入的可屏蔽中断。

当 CPU 响应中断请求，调用中断类型号为 n 的中断服务程序时，首先将中断类型号 n 乘以 4，得到中断向量的存放地址（中断向量指针）4n，然后将中断向量表中 4n 和 4n+1 两个单元中的内容（中断服务程序入口地址的偏移地址）装入 IP 寄存器，即：IP←（4n,4n+1），再把向量表中两个高字节单元，即（4n+2,4n+3）中的内容（中断入口的段基址）装入代码段寄存器 CS 中，即：CS←（4n+2,4n+3）。这样就把 CPU 引导到类型 N 的中断服务程序的起始地址，开始中断处理过程。

例如：键盘中断的中断矢量号为 09H，所对应的中断服务程序入口地址为 0BA9：0125H。该入口地址位于中断矢量表 0000：0024H（等于 09H×4）开始的 4 个单元中，这 4 个单元所存储的内容如下：

0024H	25
0025H	01
0026H	A9
0027H	0B

当发生键盘中断时，CPU 会自动将其中断矢量号 09H 乘以 4，得到中断矢量表的地址指针 0024H，把由该地址开始的两个单元内容 0125H 送入 IP 寄存器，再把两个高字节单元内

容 0BA9H 送入 CS 寄存器，CPU 就会根据 CS、IP 寄存器的内容自动转到键盘中断的中断服务程序去进行中断处理。

需要说明的是，中断向量表虽然设置在 RAM 的低位存储区（第 0 段中的 0H～03FFH 单元）内。但它并非常驻内存，而是每次开机上电后，在系统正式工作之前都必须对其进行初始化，即由程序将相应的中断服务程序的入口地址装入指定的中断向量表区中。PC 启动过程中，首先由 ROM BIOS 自测试代码对 ROM BIOS 控制的中断向量进行初始化装入。对 8086 系统，仅仅装入 00H～lFH 共 32 个中断向量，对于 80286 以上的 CPU，系统装入 0H～77H 共 120 个中断向量。若用户自行开发的应用程序采用 INT N 形式调用，则要自己将中断服务程序入口地址装入中断向量表中所选定的单元中。

9.3　可编程中断控制器 8259A 及其应用

8259A 可编程中断控制器能够管理输入到 CPU 的中断请求，实现中断优先权判别，提供中断矢量（即中断类型号）和屏蔽中断等功能。它能直接管理 8 级中断，如果采用级联方式，则不用附加外部电路就能管理 64 级中断输入。它使用单+5V 电源供电，具有多种工作方式，能适应各种系统要求。优先级方式在执行主程序的任何时间都可以动态改变。PC/XT 系统板上使用了一片 8259A，AT 机上用了两片级联的 8259A。

8259A 可编程中断控制器的主要功能如下。

- 单片 8259A 仅具有 8 级中断优先权控制，通过级联方式可扩展到 64 级中断优先权控制。
- 每一级中断都可以通过程序来单独屏蔽或允许。
- 在中断响应周期，8259A 可提供相应的中断类型码。
- 8259A 有多种中断管理方式，可通过编程来进行选择。

9.3.1　8259A 的内部结构及引脚

1. 内部结构

8259A 的内部结构示意图如图 9-7 所示。

由图可见，8259A 主要由以下几部分组成：

（1）中断请求寄存器（IRR）：是一个具有锁存功能的 8 位寄存器，用来存放外部输入的中断请求信号 IR_0～IR_7。每一位对应一个外部中断请求信号 IR_i，当某个 IR 端有中断请求，IRR 中的相应位置 1，其内容可用操作命令字 OCW3 读出。当中断请求被响应时，IRR 的相应位就复位。

（2）中断屏蔽寄存器（IMR）：是对 8 级中断请求分别独立地加以禁止和允许的寄存器。若某位置 1，则与之对应的中断请求被禁止。屏蔽优先权高的中断请求不影响优先权较低的中断请求线。

（3）中断服务寄存器（ISR）：用来存放所有正在进行服务的中断请求（包括尚未服务完而中途被优先权更高的中断所打断的中断请求）。若某位置 1，表示正在为相应的中断源服务。

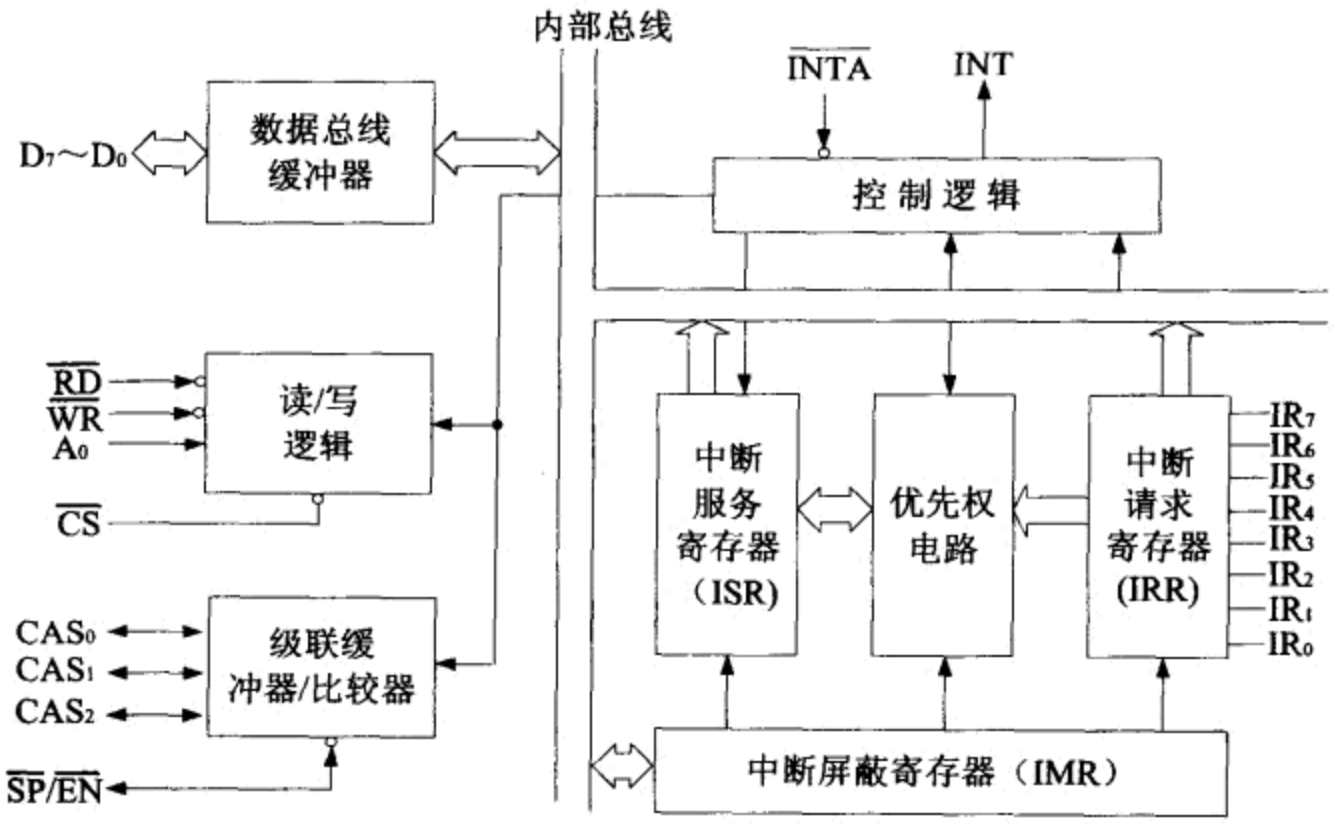


图 9-7　8259A 的内部结构示意图

（4）数据总线缓冲器：是 8259A 与系统数据总线的接口，为双向三态缓冲器。CPU 向 8259A 发送的数据、命令、控制字，以及 8259A 向 CPU 输入的数据、状态信息都要经过数据总线缓冲器。

（5）读/写控制逻辑：接收来自 CPU 的读/写命令，配合 $\overline{CS}$ 端的片选信号和 A_0 端的地址输入信号，完成规定的操作。当 CPU 对 8259A 进行写操作时，它控制将写入的数据送相应的命令寄存器中（包括初始化命令字和操作命令字），当 CPU 对 8259A 进行读操作时，它控制将相应的寄存器的内容（IRR，ISR，IMR）输出到数据总线上。

（6）优先权电路：用来识别各中断请求信号的优先级别。多个中断请求信号同时产生时，由优先权电路判定哪个中断请求具有最高优先权。该电路对保存在 IRR 中的各个中断请求，判定其中优先权最高的，使 ISR 中相应位置 1，表示正在为之服务。若有中断嵌套，则将后来的中断请求与 ISR 中正在被服务的优先级相比较，以决定是否向 CPU 发出中断请求。

（7）控制逻辑：该部件按照编程设置的工作方式管理 8259A 的全部工作。在 IRR 中有未被屏蔽的中断请求位时，控制逻辑输出高电平的 INT 信号，向 CPU 申请中断。在中断响应期间，它允许 ISR 的相应位置 1，并发出相应的中断类型号，通过数据总线缓冲器输出到系统总线上。在中断服务结束时，它按照编程规定的方式对 ISR 进行处理。

（8）级联缓冲器/比较器：该部件在级联方式的主从结构中，用来存放和比较系统中各 8259A 的从设备标志（ID）。与此相关的是三条级联线 CAS_0～CAS_2 和从片编程/允许缓冲器 $\overline{SP}$ / $\overline{EN}$ 线。其中 CAS_0～CAS_2 是 8259A 相互间连接用的专用总线，用来构成 8259A 的主-从式级联控制结构，编程可以使从 8259A 的从设备标志保存在级联缓冲器中。

1 片 8259A 只能接收 8 级中断，当超过 8 级时，可用多片 8259A 级联使用，此时可用 1 片 8259A 做主片，1～8 片 8259A 作从片，构成主从控制结构。此时主 8259A 的 $\overline{SP}$ 端为 1（高电平），从 8259A 的 $\overline{SP}$ 端为 0（低电平），且从 8259A 的 INT 输出接到主 8259A 的中断请求

信号线 IR 上，因此最多可组合成 64 级中断优先级控制。

级联时，主片和从片的 CAS_0～CAS_2 并接在一起作为级联总线。在中断响应过程中，主片的 CAS_0～CAS_2 是输出信号，从片的 CAS_0～CAS_2 是输入信号。在第一个 $\overline{INTA}$ 脉冲结束时，主片把被响应的中断请求的从片编码送入 CAS_0～CAS_2 总线。从片接收后，将主片送来的编码与自己的编码相比较。若相同，表明本从片被选中，则在第二个 $\overline{INTA}$ 脉冲期间把相应的中断类型码送至数据总线，传送给 CPU。

图 9-8 所示为 3 片 8259A 级联的连接图。主片与从片的数据总线、$\overline{WR}$、$\overline{RD}$ 及 A_0 线分别接到系统总线，片选信号 $\overline{CS}$ 分别接到对应的端口地址译码电路。

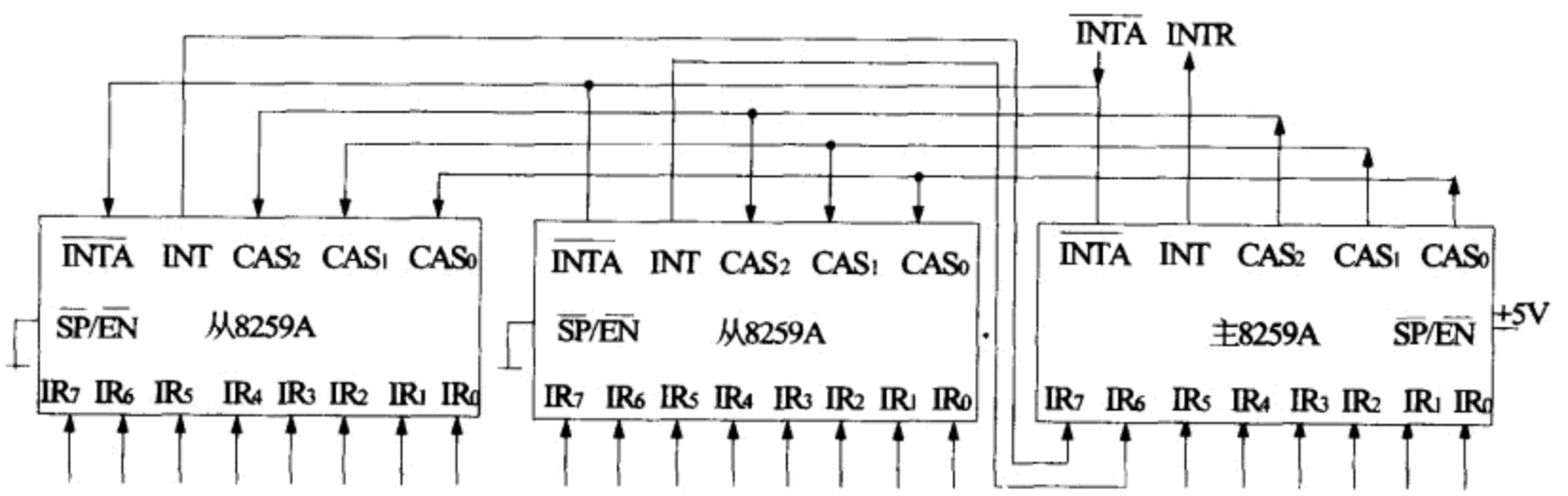


图 9-8　8259A 级联的连接图

2. 引脚功能

8259A 引脚如图 9-9 所示，各引脚功能说明如下。

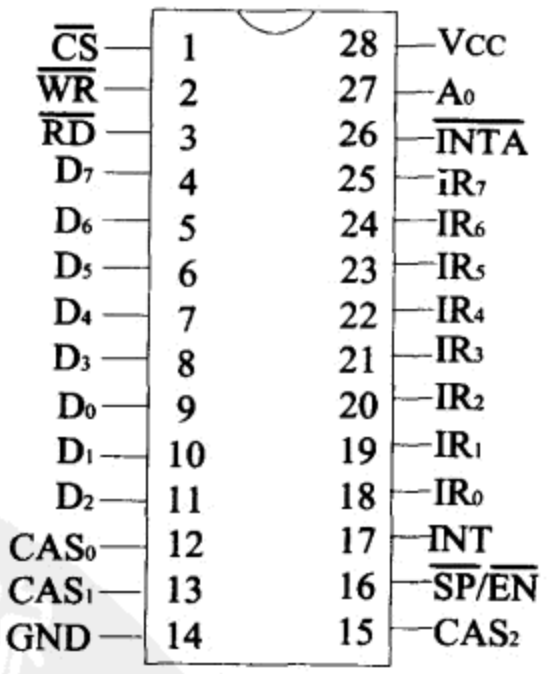


图 9-9　8259A 引脚图

（1）$\overline{CS}$：片选输入信号，低电平有效。$\overline{CS}$ 为低电平时，CPU 可以通过数据总线对 8259A 进行读/写操作。当进入中断响应时，这个引脚的状态与进行的中断处理无关。

（2）$\overline{WR}$：写控制信号，低电平有效。该信号有效时，CPU 可向 8259A 写入命令控制字。

（3）$\overline{RD}$：读控制信号，低电平有效。该信号有效时，8259A 将状态信息送至数据总线

供 CPU 检测。

（4）D_0～D_7：双向三态数据线。直接与系统数据总线相接，用来传送控制、状态和中断类型码等信息。

（5）CAS_0～CAS_2：级联信号线。

（6）IR_0～IR_7：外设中断请求信号线。

（7）$\overline{SP}/\overline{EN}$：从片编程/允许缓冲器，双向，低电平有效。这条信号线有下面两种功能。当工作在缓冲方式时，它是输出信号，用作允许缓冲器接收和发送的控制信号（$\overline{EN}$）；当工作在非缓冲器方式时，它是输入信号，用来指明该 8259A 是作为主片工作（$\overline{SP}/\overline{EN}=1$），还是作为从片工作（$\overline{SP}/\overline{EN}=0$）。

（8）INT：中断请求输出信号，接 CPU 的中断输入端 INTR。

（9）$\overline{INTA}$：中断响应输入信号，接收 CPU 送来的中断响应信号。

（10）A_0：地址选择信号。与 $\overline{CS}$、$\overline{WR}$、$\overline{RD}$ 一起用来对寄存器进行选择，表示正在访问 8259A 的哪个端口，通常接地址总线的 A_0。

9.3.2 8259A 的中断管理方式

8259A 具有非常灵活的中断管理方式，可满足用户的各种不同要求。中断优先级的管理是中断管理的核心问题，8259A 对中断的管理可分为对优先权的管理和对中断结束的管理。

1. 中断优先权管理

8259A 对中断优先权的管理可分为 4 种情况：一般完全嵌套方式、自动循环方式、中断屏蔽方式和特殊完全嵌套方式。

（1）一般完全嵌套方式

在此方式下，8259A 的中断请求输入端引入的中断具有固定的优先权排队顺序，IR_0 为最高优先级，IR_1 为次高优先级……依此类推，IR_7 为最低优先级。同时，在某个级别的中断请求正在被服务期间，8259A 将禁止同级或较低级的中断请求，但允许高优先级的中断打断低优先级的服务，实现中断嵌套。

（2）自动循环方式

在一般完全嵌套方式下，中断请求 IR_0～IR_7 的优先级别是固定不变的，从 IR0 引入的中断总是具有最高优先级。在某些情况下需要改变这种优先级别，这时可采用自动循环方式。在这种方式下，从 IR_0～IR_7 引入的中断轮流具有最高优先权，当任何一级中断被处理完后，它的优先级别就被改变为最低，而最高优先级分配给该中断的下一级中断。例如：初始优先级队列为 IR_0、IR_1…IR_7，如果这时 IR_4 有请求，响应 IR_4 后优先级队列变为 IR_5、IR_6、IR_7、IR_0、…、IR_4。

（3）中断屏蔽方式

用中断屏蔽方式管理优先权有两种方法：① 普通屏蔽方式。这种方式是在 IMR 中，将某一位或几位置 1，来屏蔽掉相应级别的中断请求。例如，CPU 在执行某级的中断服务中，为了禁止比它级别高的中断进入，可在中断服务程序中将 IMR 中的相应位置“1”而加以屏蔽。② 特殊屏蔽方式。8259A 工作在特殊屏蔽方式时，所有未被屏蔽的优先级中断请求（较高的和较低的）均可在某个中断过程中被响应，即低优先级别的中断可以进入正在服务的高

优先级别中，这种方式可以在中断服务程序执行期间动态地改变系统的优先结构。

（4）特殊完全嵌套方式

特殊完全嵌套方式与一般完全嵌套方式基本相同，唯一的区别是在特殊完全嵌套方式下，当处理某一级中断时，如果有同级的中断请求，也会给予响应，从而实现对同级中断请求的特殊嵌套。而在一般完全嵌套方式中，只有更高级的中断请求到来时，才可以嵌套中断；对同级中断请求，则不会予以响应。

特殊完全嵌套方式用在 8259A 有级联的情况。任何一个从 8259A 接收到的中断请求，如果经该 8259A 判定为当前最高优先级，则通过 INT 端向主 8259A 相应的 IR 端提出中断请求。如果这时主 8259A 中 ISR 的相应位已置 1，则说明该从 8259A 的其他输入端已提出过中断请求，且正在服务，从 8259A 的优先权电路判别到刚申请的中断优先级最高，故应停止现行中断服务转去为刚申请的中断服务。如果主 8259A 工作在一般完全嵌套方式，主 8259A 将把同一从 8259A 的不同级别中断请求认为是同级的，而不予以响应。为了实现真正的完全嵌套，就要求主 8259A 工作在特殊完全嵌套方式，此时 8259A 仅不响应那些比置 1 的 ISR 位更低级别的中断请求，而响应同级或更高级的中断。

8259A 在级联情况下，按特殊完全嵌套方式管理优先级。显然，接在主片 IR_3 上的从片比接在 IR_4 上的从片具有高的优先级，而主片上 IR_0、IR_1、IR_2 上的中断比接在主片 IR_3 上的从片优先级更高。

2. 中断结束的管理

当 8259A 响应某一级中断而为其服务时，中断服务寄存器 ISR 的相应位置 1，当有更高级的中断请求进入时，ISR 的相应位又要置 1。因此，中断服务寄存器 ISR 中可有多位同时置 1。在中断服务结束时，ISR 的相应位应清零，以便再次接收同级别的中断。中断结束的管理就是用不同的方式使 ISR 的相应位清零，并确定随后的优先权排队顺序。

8259A 中断结束的管理可分为以下 3 种情况。

（1）一般完全嵌套时的中断结束管理

中断嵌套过程中 ISR 的内容在不断变化。例如：8259A 正在为 IR_7 的中断请求服务，则 ISR 中的 D_7 位置 1，此时有更高级别 IR_4 请求服务，若准许为其服务，则 ISR 中的 D_4 位又置 1。中断服务结束时也应按从高到低的顺序结束，即先结束 IR_4，后结束 IR_7。每个中断结束使 ISR 中的相应位清零，待全部嵌套中断均结束，则 ISR 中每一位均为 0。

8259A 在一般完全嵌套方式下可采用 3 种中断结束方式。

- 一般 EOI 方式：任何一级中断服务程序结束时给 8259A 发送一个 EOI 命令，8259A 将 ISR 中级别最高的中断位置零。该方式只有在当前结束的中断总是尚未处理完的级别最高的中断时才能使用。如果在中断服务中修改过中断级别，则不能采用这种方式。
- 指定 EOI 方式：在一般 EOI 方式基础上，当中断服务程序结束给 8259A 发出 EOI 命令的同时，将当前结束的中断级别也传送给 8259A，这就是特殊 EOI 方式。该方式下 8259A 将 ISR 中指定级别的相应位置零，适合于在任何情况下使用。
- 自动 EOI 方式：若 8259A 处于自动 EOI 方式，则在第二个中断响应信号 $\overline{\text{INTA}}$ 结束时，8259A 自动将 ISR 相应位置零。中断服务程序结束时，不需要向 8259A 送 EOI 命令，这是一种最简单的结束方式。但是存在一个明显的缺点，任何一级中断在执行

中断服务程序期间，在 8259A 中没有留下任何标志，如果在此过程中出现了新的中断请求，则只要 CPU 允许中断，不管出现的中断级别如何，都将打断正在执行的中断服务程序而被优先执行，这显然是不合理的。因此，只有在一些以预定速率发生中断，且不会发生同级中断互相打断或低级中断打断高级中断的情况下，才使用自动 EOI 方式。

（2）自动循环方式时的中断结束管理

这种情况下也可采用 3 种中断结束方式，和前面的一般完全嵌套方式的情况相同，分为不指定的 EOI 方式、指定的 EOI 方式和自动 EOI 方式，但结束后的优先权处理有所不同。

- 不指定的 EOI 方式：任何一级中断服务程序结束时给 8259A 发一个不指定的 EOI 命令，8259A 将 ISR 中级别最高的置 1 位清零，同时赋给它最低优先级，将最高优先级赋给比它低一级的中断请求，其他中断请求的优先级别按自动循环方式类推。
- 指定的 EOI 方式：主要用在既有自动循环优先权管理方式又有嵌套的情况下。在不指定 EOI 方式下，结束的总是当前级别最高的中断，若在中断服务程序中安排一条优先权置位指令，使得优先权次序发生了变化，就必须采用指定 EOI 方式，在向 8259A 发中断结束命令的同时，将其中断优先级别也传送给 8259A。这样，8259A 可根据用户要求将 ISR 中指定级别的相应位置零，同时赋给它最低优先级，其余中断源的优先级别按自动循环方式类推。
- 自动 EOI 方式： 8259A 在第二个中断响应信号 $\overline{\text{INTA}}$ 结束时，自动将 ISR 中的相应位置零，并赋给它最低优先级，其余中断源的优先级别按自动循环方式类推。

（3）特殊完全嵌套时的中断结束管理

这种情况是因为 8259A 有级联，因而 CPU 应发出两个 EOI 命令，一个送给主 8259A，用来将主 8259A 的 ISR 相应位清零；另一个送给从 8259A，用来将从 8259A 中的 ISR 相应位清零。需要强调的是，在从片中断服务程序结束处，要检测刚服务完的中断是否为该从片唯一的中断。通常，首先向该从片发一个不指定的 EOI 命令，然后读取 ISR 的内容，检查是否全为 0。若全为 0，表示该从片已无低一级的中断服务，则可向主片发一个不指定的 EOI 命令，结束主 8259A 的此 8259A 从片的中断，否则不向主片发中断结束命令。

3. 连接系统总线的方式

（1）缓冲方式

在多片 8259A 级联的大系统中，8259A 通过总线驱动器与系统数据总线相连，这就是缓冲方式。该方式下有一个对总线驱动器的启动问题。为此，将 8259A 的 $\overline{\text{SP}}/\overline{\text{EN}}$ 端和总线驱动器的允许端相连。8259A 工作在缓冲方式时，会在输出状态字或中断类型码的同时，从 $\overline{\text{SP}}/\overline{\text{EN}}$ 端输出一个低电平，此低电平正好可作为总线驱动器的启动信号。

（2）非缓冲方式

非缓冲方式是相对于缓冲方式而言的。当系统中只有单片 8259A 时，一般要将它直接与数据总线相连；在一些不太大的系统中，即使有几片 8259A 工作在级联方式，只要片数不多，也可以将 8259A 直接与数据总线相连。上述两种情况下，8259A 就工作在非缓冲方式下。此方式下，8259A $\overline{\text{SP}}/\overline{\text{EN}}$ 端作为输入端。当系统中只有单片 8259A 时，其 $\overline{\text{SP}}/\overline{\text{EN}}$ 端必须接高电平；当有多片 8259A 时，主片的 $\overline{\text{SP}}/\overline{\text{EN}}$ 端接高电平，从片的 $\overline{\text{SP}}/\overline{\text{EN}}$ 端接低电平。

4. 8259A 的中断响应过程

8259A 应用于 8086 微处理器系统中其中断响应过程如下。

（1）当中断请求线（IR_0～IR_7）上有一条或若干条变为高电平时，则使中断请求寄存器 IRR 的相应位置位。

（2）当 IRR 的某一位被置 1 后，就会与 IMR 中相应的屏蔽位进行比较，若该屏蔽位为 1，则封锁该中断请求；若该屏蔽位为 0，则中断请求被发送给优先权电路。

（3）优先权电路接收到中断请求后，分析它们的优先权，把当前优先权最高的中断请求信号由 INT 引脚输出，送到 CPU 的 INTR 端。

（4）若 CPU 处于开中断状态，则在当前指令执行完后，发出 $\overline{\text{INTA}}$ 中断响应信号。

（5）8259A 接收到第一个 $\overline{\text{INTA}}$ 信号，把允许中断的最高优先级请求位放入 ISR，并清除 IRR 中的相应位。

（6）CPU 发出第二个 $\overline{\text{INTA}}$，在该脉冲期间，8259A 发出中断类型号。

（7）若 8259A 处于自动中断结束方式，则第二个 $\overline{\text{INTA}}$ 结束时，相应的 ISR 位被清零。在其他方式中，ISR 相应位要由中断服务程序结束时发出的 EOI 命令来复位。

若 8259A 为级联方式的主从结构，并且某从片 8259A 的中断请求优先级最高，则在第一个 $\overline{\text{INTA}}$ 脉冲结束时，主 8259A 把这个从设备标志 ID 送到级联线上。各个从 8259A 把这个标志与自己级联缓冲器中保存的标志相比较，在第二个 $\overline{\text{INTA}}$ 期间，被选中的从 8259 的中断类型号送到数据总线上。

（8）CPU 收到中断类型号，将它乘以 4 得到中断矢量表的地址，然后转至中断服务程序。

9.3.3　8259A 的编程

8259A 是可编程的中断控制器，它的操作是用软件通过命令字进行控制的。8259A 的编程包括两类：一类是初始化编程，8259A 在进入操作前必须由初始化命令字 ICW_1～ICW_4 使它处于初始状态。对 8259A 的初始化编程是微机上电初始化时由基本输入/输出系统 BIOS 完成的，用户一般不应改变。另一类是操作方式编程，规定 8259A 的工作方式。操作命令字 OCW 可在 8259A 初始化以后的任何时间写入。基本输入/输出系统 BIOS 是系统板上固化在 EPROM 芯片上的程序，是低层的输入/输出系统管理模块，负责微机上电测试、引导 DOS、管理键盘输入、屏幕上文本和图形显示及磁盘输入/输出等工作。该程序依赖具体设备，当外部设备改变时，这部分程序也相应要改变。

1. 初始化编程

8259A 有 4 个初始化命令字 ICW_1～ICW_4，在 8259A 开始工作之前，必须用初始化命令字建立 8259A 操作的初始状态，它们必须按照一定的顺序输入，如图 9-10 所示。ICW_1 和 ICW_2 是必须输入的，ICW_3、ICW_4 是否要输入则由 ICW_1 的相应位决定。当 SNGL=0 时，需要 ICW_3 分别对主片和从片编程，其格式是不同的。当 ICW_1 的 IC_4=1 时，需要输入 ICW_4。对 8086 系统总是要输入 ICW_4。

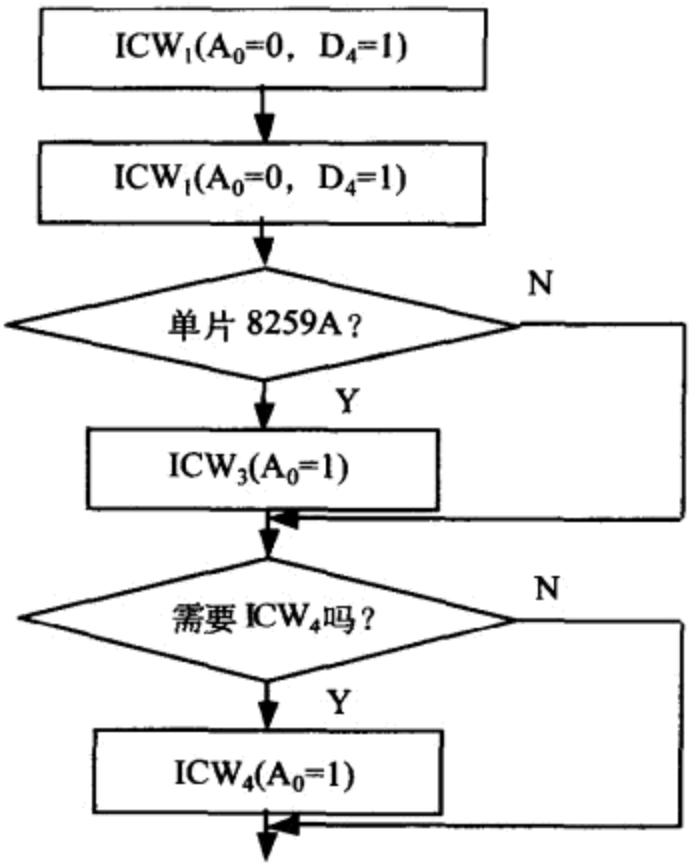


图 9-10　8259A 初始化流程图

初始化命令字完成以下任务：① 设定中断请求信号的有效方式，是高电平有效还是上升沿有效。② 工作于单片方式还是工作于级联方式。③ 若为级联工作方式，则规定主 8259A 中每个 IR 端是否带从 8259A，从 8259A 则要规定由主 8259A 的哪个 IR 端引入。④ 设置中断类型码。⑤ 完成中断管理方式的设定。

一旦初始化完成以后，若要改变某一个初始化命令字，则必须重新再进行初始化编程，不能只写入单独的一个初始化命令字。

（1）ICW_1——芯片控制初始化命令字

ICW_1 的格式如下所示，其特征是 A_0=0，并且控制字的 D_4=l。当 CPU 向 8259A 写命令字，只要地址线 A_0=0，数据线的 D_4=l 时，就被 8259A 译码为 ICW_1，启动 8259A 的初始化过程：清除 IMR，把最低优先权分配给 IR_7，把最高优先权分配给 IR_0，将从设备标志 ID 置成 7，清除特殊屏蔽方式以及设置读 IRR 方式。

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	A_7	A_6	A_5	1	LTIM	ADI	SNGL	IC_4

D_4=1：D_4=1 和 A_0=0 是 ICW_1 的标志。初始化命令字设置完后，当 A_0=0 时，D_4=0 表示操作控制字 OCW_2 或 OCW_3。

A_7～A_5：在 8088/8086 系统中不用，可全写 0。

LTIM：中断输入信号 IR 的输入触发方式。LTIM=0 为上升沿有效的边沿触发，IR 输入信号由低到高的跳变被识别且送入 IRR；LTIM=1 为高电平有效的电平触发方式，IR 输入为高电平即进入 IRR，不进行边沿检测。在两种触发方式下，IR 输入的高电平在置位 IRR 相应位后仍要保持，直到中断被响应为止。再次响应前应撤销。

ADI：8088/8086 系统中不用，可为 0。

SNGL：单片/级联方式指示。SNGL=0 为级联方式，这时在 ICW_1、ICW_2 之后要跟 ICW_3，以设置级联方式工作状态。SNGL=1 表示为单片方式，初始化过程中不需要 ICW_3。

IC_4：指示初始化过程中有无 ICW_4。IC_4=0 表示不需要写 ICW_4；IC_4=1，表示要写 ICW_4。8088/8086 系统中 IC_4 必须置 1，即需要写 ICW_4。

（2）ICW_2——中断类型码初始化命令字

ICW_2 的格式如下：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
1	T_7	T_6	T_5	T_4	T_3			

ICW_2 规定中断类型号字节。编程时规定高 5 位 T_7～T_3，低 3 位由 IR 的编码插入，输入时地址线 A_0=1。在 PC/XT 机中 T_7～T_3=00001，所以 IR_0～IR_7 对应的中断矢量是 08H～0FH。

（3）ICW_3——主/从片初始化命令字

当 ICW_1 中的 SNGL 位为 0 时工作于级联方式，才需要写 ICW_3 设置 8259A 的状态，主片的 8259A 格式与从片的不同，从片的 INT 接主片的 IR_i，如下所示。

对于主片，ICW3 格式为：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
1	IR_7	IR_6	IR_5	IR_4	IR_3	IR_2	IR_1	IR_0

D_7～D_0 对应于 IR_7～IR_0 引脚上的连接情况。当某一引脚上接有从片，则对应位为 1，否则为 0。若仅有主片的 IR_6 和 IR_2 上接有从片，则 ICW_3 应是 44H。ICW_3 应该送到 8259A 的奇地址，即 A_0=1。

对于从片，ICW_3 格式如下：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
1	0	0	0	0	0	ID_2	ID_1	ID_0

其中，D_7～D_3 不用。D_2～D_0 的取值取决于从片的 INT 脚连到主片的哪个中断请求输入端。例如，若从片 INT 接在主片的 IR_6 上，则 D_2～D_0 为 110，从片的 ICW_3 也送奇地址。主、从片的地址是不相同的，都占两个地址。

主从片的 CAS_0～CAS_2 同名脚连接在一起。主片的 CAS_0～CAS_2 作为输出。从片的 CAS_0～CAS_2 则作为输入。当第一个 $\overline{INTA}$ 到达时，主片经优先权处理后将最高优先级的从片的标识码送到 CAS_0～CAS_2 上。从片收到标识码，与自己的 ICW_3 规定的标识码比较，如果相等，则在第二个 $\overline{INTA}$ 到来时，将自己的优先权最高的中断类型号送到数据总线。

（4）ICW_4——方式控制初始化命令

ICW_4 是在 ICW_1 的 IC_4=1 时才使用，其格式如下：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
1	0	0	0	SFNM	BUF	M/S	AEOI	μPM

μPM：指定 CPU 类型，μPM=0 表示 8259A 工作于 8080/8085 系统中，μPM=1 表示 8259A 工作于 8088/8086 系统中。

AEOI：指定是否为自动中断结束方式。AEOI=1 时，为自动中断结束方式。指定这种方式时，在第二个 $\overline{INTA}$ 信号的后沿，8259A 自动使中断源在 ISR 中的相应位复位。当 AEOI=0 时，为非自动中断结束方式。这时必须在中断服务程序结束前，由 CPU 向 8259A 发出 EOI 命令，使 ISR 中最高优先权的位复位。

M/S：M/S=1 为主片，M/S=0 为从片。它与 BUF 配合使用，当 BUF=1 时，M/S 决定是主片还是从片；若 BUF=0，则 M/S 不起作用。

BUF：指示 8259A 是否工作在缓冲方式，由此决定 $\overline{SP}$ / $\overline{EN}$ 的功能。BUF=1 时，8259A 工作于缓冲方式，$\overline{SP}$ / $\overline{EN}$ 用做允许缓冲器接收/发送的输出控制信号 $\overline{EN}$。BUF=0 时，8259A 工作于非缓冲方式，$\overline{SP}$ / $\overline{EN}$ 用做主片/从片的输入控制信号 $\overline{SP}$。

当 8259A 工作于一个大系统时，系统数据总线要用总线缓冲器驱动，这时就要对总线缓冲器的工作进行控制。8259A 工作于缓冲方式时（BUF=1），输出信号 $\overline{EN}$ 用来允许数据收发器工作。当 $\overline{EN}$ 为低电平时，8259A 通过数据收发器与数据总线接通，进行数据传送。

当 8259A 工作于非缓冲方式（BUF=0）时，$\overline{SP}$ / $\overline{EN}$ 用做输入控制信号 $\overline{SP}$，从而决定本器件作为主片还是从片工作。$\overline{SP}$ =1 时，8259A 为主片；$\overline{SP}$ =0 时，8259A 为从片。

SFNM：用来决定 8259A 在级联时是否工作于特殊完全嵌套方式。如果对主片设置 SFNM=1，就为特殊完全嵌套方式，SFNM=0，表示 8259A 工作于一般完全嵌套方式。

ICW_1～ICW_4 写入 8259A 后，IR_0 被指定为最高优先级，IR_7 最低，优先级别固定不变，8259A 的 ISR 和 IMR 被清零，处于普通屏蔽方式，对 A_0=0 的端口进行读操作时，读取的是 IRR 的状态。

8259A 在任何情况下从 A_0=0 的端口接收到一个 D_4 位为 1 的命令就是 ICW_1，后面紧跟的就是 ICW_2～ICW_4，8259A 接收完 ICW_1～ICW_4 后，就处于就绪状态，可接收来自 IR 端的中断请求。但是在 8259A 工作期间，根据需要可随时利用操作命令字对 8259A 进行动态控制，以选择或改变初始化后设定的工作方式。

2. 操作控制字的编程

用初始化命令字初始化后，8259A 就进入工作状态，准备好接收 IR 输入的中断请求信号。在 8259A 工作期间，可通过操作控制字 OCW 来使它按不同的方式操作。操作控制字从 OCW_1～OCW_3，可以独立使用。

（1）OCW_1——中断屏蔽操作命令字

OCW_1 格式如下：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
1	M_7	M_6	M_5	M_4	M_3	M_2	M_1	M_0

OCW_1 必须送到奇地址，即 A_0=1。它用于设置 8259A 的屏蔽操作。M_7～M_0 对应着 IR_7～IR_0，M_i=1 就屏蔽对应的 IR_i 输入，禁止它产生中断输出信号 INT；M_i=0 则允许对应 IR_i 输入信号产生 INT 输出，向 CPU 请求服务。中断屏蔽寄存器 IMR 的内容，CPU 可以通过读 8259A 的奇地址得到。

（2）OCW_2——控制中断结束和优先权循环的操作命令字

OCW_2 的格式如下：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	R	SL	EOI	0	0	L_2	L_1	L_0

D_4D_3=00 是 OCW_2 的标志。OCW_2 必须写入偶地址，故 A_0=0。

R：优先权循环位。R=1 为循环优先权，R=0 为固定优先权。

SL：选择指定的 IR 级别位。SL=1 时，操作在 L_2～L_0 指定的 IR 编码级别上执行；SL=0 时，L_2～L_0 无效。

EOI：中断结束命令位。在初始化 ICW_4 中定义为非自动中断结束方式（AEOI=0）时，就需要 OCW_2 来控制结束。EOI=1 表示中断结束命令。它使 ISR 中最高优先权的位复位；EOI=0 则不起作用。EOI 命令常用在中断服务程序中，中断返回指令前。

这 3 个控制位的组合格式所形成的命令和方式如下：

R	SL	EOI	功　能
0	0	1	一般 EOI 命令
0	1	1	特殊 EOI 命令
1	0	1	循环优先权的一般 EOI 命令
1	0	0	设置优先级自动循环方式的命令
0	0	0	结束优先级自动循环方式的命令
1	1	1	自动循环的特殊 EOI 命令
1	1	0	设置特殊优先权循环方式命令
0	1	0	无效

【例 9.1】若对 IR_3 中断源采用指定中断结束方式，则需编写如下程序：

```
MOV   AL,01100011B
OUT   20H,AL
```

若对 IR_3 中断源采用指定中断结束方式，且 IR_3 中断优先权为最低，则编写如下程序：

```
MOV   AL,11100011B
OUT   20H,AL
```

（3）OCW_3

OCW_3 主要控制 8259A 的中断屏蔽、查询和读寄存器等的状态。OCW_3 格式如下：

A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	×	ESMM	SMM	0	1	P	RR	RIS

ESMM：允许或禁止 SMM 位起作用的控制位。ESMM=1，允许 SMM 起作用；ESMM=0，禁止 SMM 位起作用。

SMM：设置特殊屏蔽方式选择位。当 ESMM=1 和 SMM=1 时，选择特殊屏蔽方式；当 ESMM=1 和 SMM=0 时，消除特殊屏蔽方式，恢复一般屏蔽方式。当 ESMM=0 时，该位不起作用。

P：查询命令位。P=1 时是查询命令，P=0 时不是查询命令。OCW_3 设置查询方式后，随后送到 8259A 的 $\overline{RD}$ 端的读脉冲作为中断响应信号，读出最高优先权的中断请求 IR 级别码。

RR：读寄存器命令位。RR=1 时允许读 IRR 或 ISR；RR=0 时禁止读这两个寄存器。

RIS：读 IRR 或 ISR 选择位。如果 RR=1 和 RIS=1，则允许通过读偶地址读取中断服务寄

存器 ISR 内容；如果 RR=1 和 RIS=0，则允许读中断请求寄存器 IRR。当 RR=0 时，则 RIS 无效。

P、RR、RIS 组合后的操作如下表 9-1 所示。

表 9-1 P、RR、RIS 组合后的操作

P	RR	RIS	操　　作
0	0	×	无操作
0	1	0	下一个读指令读取 IRR 内容
0	1	1	下一个读指令读取 ISR 内容
1	×	×	下一个读指令读中断状态（查询命令）

3. 8259A 编程举例

【例 9.2】在 IBM PC/XT 机中，只有一片 8259A 作中断管理用。它设计为主片结构，端口地址为 20H 和 21H，不需要 ICW3。下面是 IBM PC 的 ROM BIOS 对 8259A 的编程字。其中 ICW1 是通过 20H 端口写入的，ICW2 和 ICW4 是通过端口 21H 写入的，8259A 内部将两个字节分别写到不同的寄存器中。

对 8259A 的初始化程序如下：

```
MOV  AL,13H          ;ICW1,设置 ICW4,单片结构,边沿触发
OUT  20H,AL
MOV  AL,08H          ;ICW2,设置中断类型号,起始中断号为 08H
OUT  21H,AL
MOV  AL,0DH          ;ICW4,数据总线缓冲方式,非自动中断结束,主片
OUT  21H,AL
```

【例 9.3】在 IBM PC/AT 机中，硬件中断管理由主、从两片 8259A 构成，共 15 级向量中断。主 8259A 的端口地址为 20H 和 21H，从 8259A 端口地址为 0A0H 和 0A1H。主片和从片的中断请求信号均采用边沿触发，一般完全嵌套方式，优先权排列顺序为 IRQ0、IRQ1、IRQ8～IRQ15、IRQ3～IRQ7。从片的中断请求 INT 输出与主片的中断请求输入端 IR2 相连，如图 9-11 所示。其中 IRQ0～IRQ7 对应的中断类型号为 08H～0FH，IRQ8～IRQ15 对应的中断类型号为 70H～77H。对主 8259A 和从 8259A 的初始化程序如下：

```
;初始化主 8259A
MOV  AL,11H          ;ICW1,边沿触发,设置 ICW4,级联方式
OUT  20H,AL
MOV  AL,08H          ;ICW2,设置中断类型号,起始中断号为 08H
OUT  21H,AL
MOV  AL,04H          ;ICW3,表示从 8259A 的 INT 端接到主 8259A 的 IR2
OUT  21H,AL
MOV  AL,01H          ;ICW4,非缓冲方式,非自动中断结束,一般完全嵌套方式
OUT  21H,AL
```

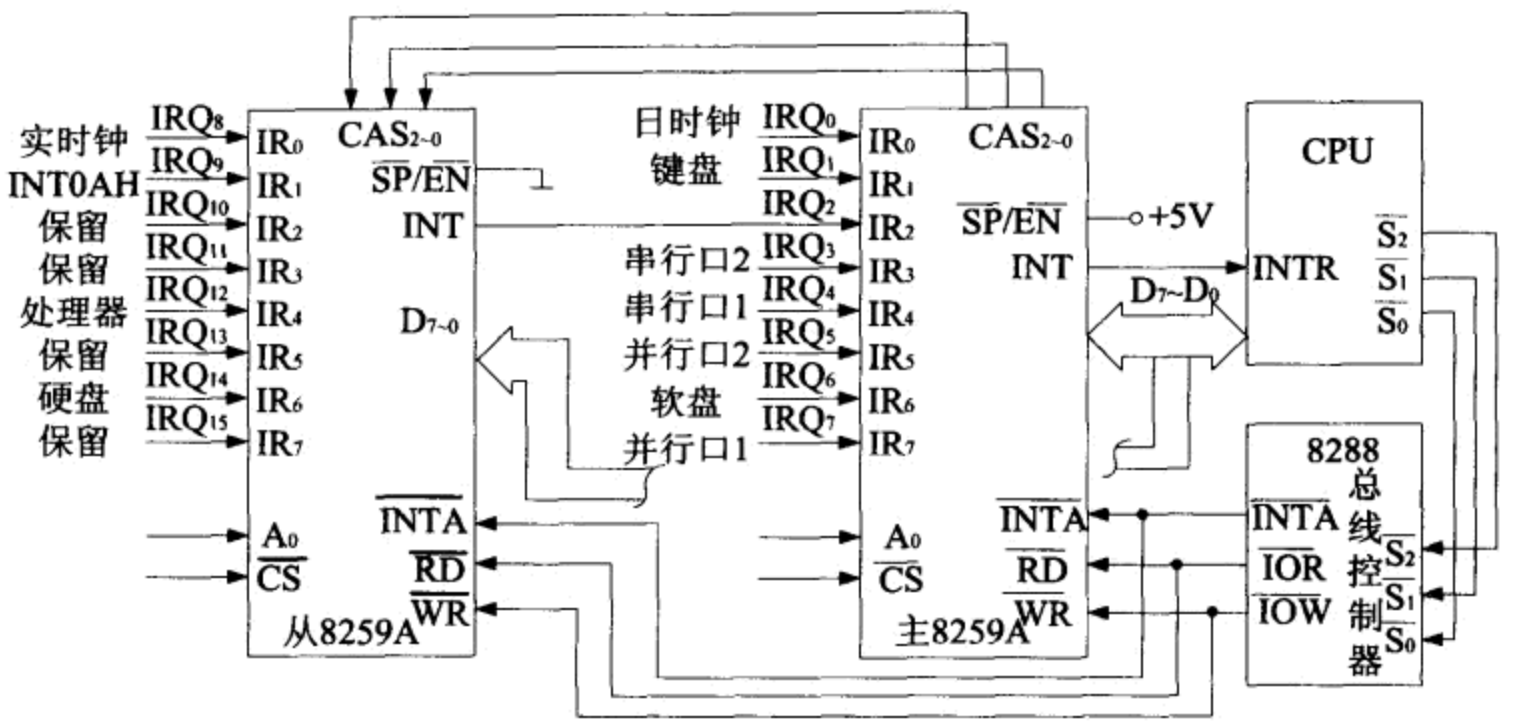


图 9-11　PC/AT 机中两片 8259A 硬件连接示意图

```
;初始化从 8259A
MOV  AL,11H          ;ICW1,设置 ICW4,多片级联,边沿触发
OUT  0A0H,AL
MOV  AL,70H          ;ICW2,设置从 8259A 的中断类型号,起始中断号为 70H
OUT  0A1H,AL
MOV  AL,02H          ;ICW3,设置从 8259A 的地址,接主片的 IR2
OUT  0A1H,AL
MOV  AL,01H          ;ICW4,非缓冲方式,非自动中断结束,一般完全嵌套方式
OUT  0A1H,AL
```

【例 9.4】OCW 的编程：对 OCW 的编程是由用户程序进行的，分别讨论如下。

（1）OCW_1：这是一个中断屏蔽寄存器操作控制字，直接写入中断屏蔽寄存器 IMR。由于地址用 A_0=1 寻址，故在 IBM PC 和 XT 机中，该控制字的写入是由端口 21H 写操作完成的。在 IBM PC/AT 机中，主片用端口 21H 写入，从片用端口 0A1H 写入，在中断服务程序内部和其他子程序中，均可使用该命令。例如对主片屏蔽 IR_4～IR_7，对从片屏蔽 IR_{15} 和 IR_8，则应该向主片 21H 端口写入 11110000=0FOH，向从片 0A1H 端口写入 10000001=81H，程序如下：

```
MOV  AL,0F0H
OUT  21H,AL
MOV  AL,81H
OUT  0A1H,AL
```

（2）OCW_2：在 IBM PC 机中，OCW_2 主要用来结束中断。

ISR 中保存着当前正在服务的中断请求，如果该位不能复位为 0，则后续的同级和较低优先级的中断请求将不会被响应。在 PC 中，ICW_4 设置的中断结束方式是非自动中断结束，它必须在中断服务程序结束前，由 OCW_2 命令字来清除 ISR 中的最高优先权位。在 OCW_2 中，一般的 EOI 命令代码是 00100000=20H。又因为 OCW_2 写入要求地址线 A_0=0，因此是通过端

口 20H 写入的，所以中断结束命令由下列程序完成：

```
MOV   AL,20H
OUT   20h,al
```

对 AT 机，先通过端口 0A0H 向从片写结束命令，再通过端口 20H 向主片写结束命令，程序为：

```
MOV   AL,20H
OUT   20h,AL
OUT   0A0H,AL
```

（3）OCW_3：OCW_3 也是通过端口 20H 和 0A0H 写入的，不同之处在于 OCW_2 中 D_4D_3=00，而在 OCW_3 中 D_4D_3=01。8259A 通过 D_4D_3 判断是 OCW_2 还是 OCW_3，然后写入各自的寄存器。

本章小结

中断技术是微型计算机系统进行信息交换的一种重要技术，本章对中断技术的基本知识进行了全面介绍。“中断”是指 CPU 在正常执行程序时暂时终止执行现行程序，转去执行中断服务程序，待该服务程序执行完毕，又能自动返回到被中断的程序继续执行。这样就可以实现 CPU 与外设的同步操作和故障的实时处理，提高系统的工作速度。

8086 的中断可分为外部中断（又称硬件中断）和内部中断（又称软件中断），其中外部中断有不可屏蔽中断 NMI 和可屏蔽中断 INTR 两种，内部中断又有除法出错中断、INTO 溢出中断、INT N 中断、断点中断和单步中断等。不同的中断类型具有不同的中断类型码，可以采用软件查询、硬件优先权排队电路等方法来确定中断优先权。8086 采用中断矢量表来管理 256 种中断。

8086 微处理器的中断处理过程可分为中断请求、中断响应、中断处理和中断返回。

可编程中断控制器 8259A 能够直接管理 8 级中断，实现中断优先权判别，提供中断矢量（即中断类型号）和屏蔽中断等功能。如果采用级联方式，不需附加外部电路就能管理 64 级中断输入，具有多种工作方式，本章最后介绍了 8259A 的初始化编程和操作控制字的编程。

思考与练习题

1. 什么是中断？什么是中断源，常见的中断源有哪几类？
2. 在微型计算机系统中采用中断技术有哪些好处？
3. 为什么在主程序和中断服务程序中都要安排开中断指令？如果开中断指令安排在中断服务程序的末尾，会产生什么后果？
4. 确定中断的优先权有哪两种方法，各有什么优缺点？IBM PC 系列微机中采用的是什么方法？
5. 8086 的中断分哪两大类？各自的特点是什么？什么是中断矢量，什么是中断矢量表？8086 总共有多少级中断？
6. 什么是非屏蔽中断，什么是可屏蔽中断？它们得到 CPU 响应的条件是什么？

7. 在编写程序时，为什么通常总要用 STI 和 CLI 中断指令来设置中断允许标志？8259A 的中断屏蔽寄存器 IMR 和中断允许标志 IF 有什么区别？
8. 8259A 有几种结束中断处理的方式，各自应用在什么场合？在非自动中断结束方式中，如果没有在中断处理程序结束前发送中断结束命令，会出现什么问题？
9. 如何识别 4 条 ICW 命令和 3 条 OCW 命令？
10. 设 8259A 的操作命令字 OCW_2 中，EOI=0，R=1，SL=1，$L_2L_1L_0$=011，试指出 8259A 的优先权排队顺序。
11. 在两片 8259A 级联的中断系统中，主片的 IR_6 接从片的中断请求输出，请写出初始化主片、从片时，相应的 ICW_3 的格式。
12. 某外部可屏蔽中断的类型码为 08H，它的中断服务程序的入口地址为 0020:0040H。请编程将该中断服务程序的入口地址填入中断矢量表中。

第 10 章 可编程并行接口芯片 8255A

本章主要讲解并行传输的基本概念和并行接口技术，介绍微型计算机系统中可编程并行接口芯片的 8255A 的基本结构和编程方法，主要包括 8255A 的 3 种工作方式及各自特点，控制字寄存器的应用及编程设计方法。

通过本章学习，读者应该掌握并行输入/输出接口技术的概念和功能，熟悉 8255A 的内部结构及引脚，理解并掌握 8255A 的工作方式及其编程应用。

10.1 并行接口概述

微型计算机和外界的通信分为并行和串行两种方式，并行传输方式是将数据的各位同时传送，串行传输方式是将数据按一位一位顺序传送。因此，接口也分并行接口和串行接口，并行接口由于传送速率高而获得广泛的应用。

10.1.1 并行接口的分类

并行接口根据其传送的信息位和握手联络线的多少可以分为两类。

第 1 类是按照接口实现并行传送信息的位数或按数据通道的宽度来分。数据通道宽度可以有 4 位、8 位、16 位甚至更宽。比较常见的数据通道宽度是 8 位，因为大多数计算机外设例如打印机等最初都是为 8 位机设计的，并且传送最常用的 ASCII 字符码也至少需要 7 位接口。

第 2 类是按照在数据线上传送信息所用的握手联络（Handshake）线（也称应答线）的多少来分。握手联络线是指在接口和外设间传送数据所用的状态控制信息线，包括无握手联络线（零握手联络线）、一条握手联络线、二条握手联络线和三条握手联络线几类。

握手联络线是并行数据线以外的信息线，是为保证接口和外设间高效可靠传送数据而增加的状态控制信息线，且这种信息线间有一定的应答关系。是否需要握手联络线和需要几条握手联络线决定于外设的特性和并行通信协议的要求。

有些简单的外设如继电器、指示灯及发光二极管等，它们的工作状态变化极缓慢，仅需要供给一个固定的电压，这类外设用 I/O 端口读/写操作指令即可正确地传送数据，CPU 无需考虑外设的状态，彼此之间没有必要进行通信联系，直接进行输入或输出即可。可编程并行接口 8255A 的方式 0 即为无握手联系线的输入或输出。

为了提高信息的传送速度和可靠性，一般需要除数据线以外的握手联络线。单线握手联络线是外部设备送给 CPU 的状态信号，在数据输入过程中握手联络线是外设通知 CPU 数据准备好的状态信号，当 CPU 检测到该信号有效后便可使用 IN 指令进行数据输入；在数据输出过程中，握手联络线是外设通知已准备好接收数据的状态信号，当 CPU 检测到该信号有效后便可使用 OUT 指令进行数据输出。

双线握手联络是真正的握手联络控制，即在原来的单线握手联络的基础上再增加一条状

态控制线，如在输入过程中外设通知 CPU 数据准备好可以取走数据，当 CPU 读入数据后再向外设发回一个应答信号，通知外设当前数据已经取走，可以进行下一轮的输入；输出过程中输出缓冲器满，CPU 向外设发出一个启动信号，以通知外设读取端口数据。当外设读取端口数据后，外设向 CPU 发出一个外部响应的应答信号，CPU 接到这个信号后可以进行下一轮的输出。

10.1.2　并行接口的特点

并行接口主要有以下几方面的特点。

（1）并行接口是在多根数据线上，数据以字节或字为单位与输入/输出设备或控制对象传送信息的。如打印机接口、A/D、D/A 转换器接口、IEEE-488 接口、开关量接口、控制设备接口等。与此相对的串行接口是在一根数据线上以 1 位数据位为单位与 I/O 设备或通信设备传送信息。如 CRT 显示器、调制解调器接口等。

实际应用中，凡是在 CPU 与外设之间同时需要两位以上信息传送时，就要采用并行接口。并行接口适用于近距离、高速度的场合。

（2）并行传送的信息不要求固定的格式，这与串行传送的信息有数据格式的要求不同。

（3）从并行接口的电路结构来看，并行接口有简单硬件连线接口和可编程接口之分。硬件连线接口的工作方式及功能用硬件连线来设定，用软件编程的方法不能加以改变；如果接口的工作方式及功能可以用软件编程的方法加以改变就叫可编程接口。本章主要对可编程接口进行讨论。

10.2　并行接口芯片 8255A

8255A 是一种通用可编程并行 I/O 接口电路(NMOS)芯片，又称可编程外围接口(Programmable Peripheral Interface，PPI)，它是 Intel 公司为 8085、80X86 系列微处理器配套的接口芯片，也可以和其他微处理器系统相配。80X86 系统中常采用 8255A 作为键盘、扬声器、打印机等外设的接口电路芯片。

以后推出的 82C55A 芯片是工业标推的 CHMOS I/O 接口芯片，其功能内部结构与 8255A、8255A-5 一样，管脚引脚功能全兼容。

通用可编程并行接口芯片 8255A 可为外设提供 3 个 8 位 I/O 端口，即 A 口、B 口、C 口，每个端口又可分两组编程，能工作于 3 种操作模式，还具有直接的位控制能力，允许采用同步、异步和中断方式传送 I/O 数据。

10.2.1　内部结构及引脚

1．8255A 内部结构

8255A 内部结构如图 10-1 所示，主要包括数据输入/输出端口（即 A 口、B 口和 C 口）、A 组控制器和 B 组控制器、数据缓冲器及读/写控制逻辑等。

（1）三并行输入/输出端口（端口 A、端口 B、端口 C）

8255A 有 A、B、C 三个并行输入/输出端口（简称为 A 口、B 口、C 口），其功能全部由

程序设定，但每个端口都有自己的功能特点。A 口、B 口通常作为独立的 I/O 端口使用，C 口也可以作为一般的 I/O 端口使用。但当 A 口、B 口作为应答式的 I/O 口使用时，C 口分别用来为 A 口、B 口提供应答控制线。此时 C 口分为 A 组 C 口（或称上 C 口）、B 组 C 口（或称下 C 口），规定分别用来作为 A 口和 B 口的应答控制线使用。各端口的功能如表 10-1 所示。

各端口功能概括如下。

A 口：由一个 8 位数据输出缓冲/锁存器和一个 8 位数据输入缓冲/锁存器组成。

B 口：由一个 8 位数据输入缓冲器和一个 8 位数据输出锁存缓冲器组成。由于输入无锁存，因此数据输入时不锁存，输出时锁存。

C 口：由一个 8 位数据输入缓冲器和一个 8 位数据输出锁存缓冲器组成。数据输入时不锁存，输出时锁存。

通常 A 口和 B 口工作为方式 1 和方式 2 时，C 口作为控制状态信息口，在“方式控制字”的控制下，与 A 口和 B 口配合使用，作为控制联络信号的输入/输出端。

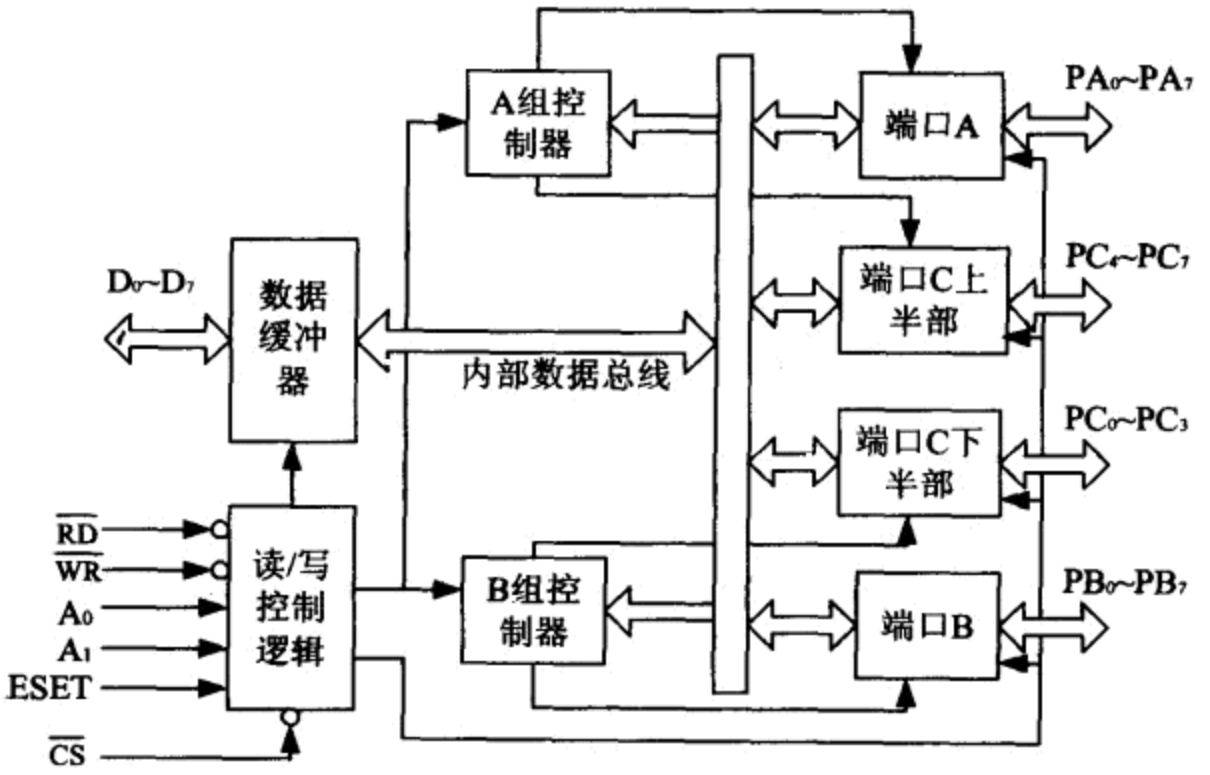


图 10-1　8255A 的内部结构

表 10-1　A、B、C 各端口的功能

工作方式	A 口	B 口	C 口
0	基本输入/输出端口 输入不锁存，输出锁存	同 A 口	同 A 口
1	应答式输入/输出端口 输出/输出均可锁存	同 A 口	上 C 口作为应答式 A 口的应答线；下 C 口作为应答式 B 口的应答线
2	应答式双向输入/输出端口，均可锁存	同 A 口	用做 A 口的应答控制线

（2）读/写控制逻辑

读/写控制逻辑的功能是用于管理数据、控制字或状态字的传送。它接收来自 CPU 的地

址信息和一些控制信号，然后向 A 组、B 组控制电路发送命令，控制端口数据的传送方向。其控制信号主要有：$\overline{CS}$（片选信号）、$\overline{RD}$（读有效信号）、$\overline{WR}$（写有效信号）、RESET（复位信号）、A_1 和 A_0（8255A 内部端口寻址地址线）。各信号的功能将在后面介绍。

（3）A 组控制器和 B 组控制器

由控制字寄存器和控制逻辑组成。控制字寄存器接收计算机送来的控制字，设定 8255A 的工作模式，控制逻辑用于对 8255A 工作模式的控制。A 组控制字寄存器控制 A 口和 C 口上半部（PC_7～PC_4），B 组控制器控制 B 口和 C 口下半部（PC_3～PC_0）。

（4）数据缓冲器

双向 8 位缓冲器，用于传送计算机和 8255A 间的控制字、状态字和数据字。

2．8255A 引脚功能

8255A 有 40 条引脚，如图 10-2 所示，引脚功能如下。

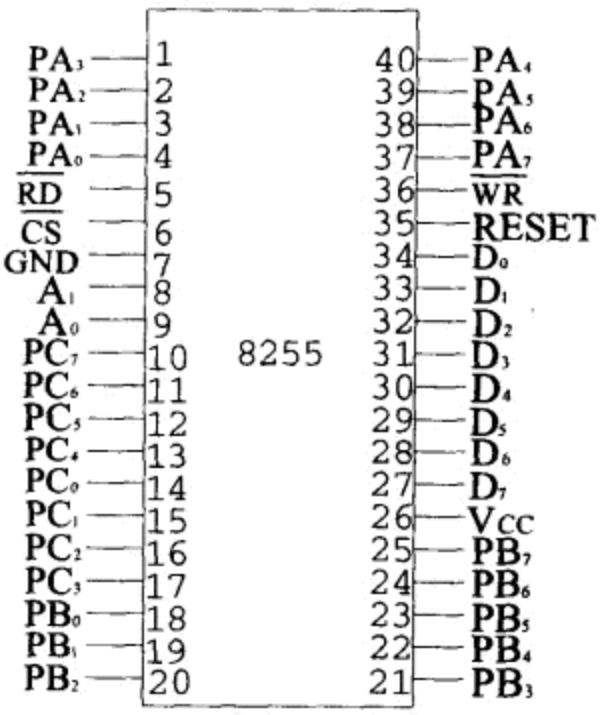


图 10-2　8255A 引脚图

D_7～D_0：数据总线，用于传送计算机和 8255A 间的数据、命令和状态字。

RESET：复位线，高电平有效。若 RESET 为高电平，则复位芯片，控制寄存器被清除，A、B、C 三端口均被置为输入方式。

$\overline{CS}$：片选线，低电平有效。

$\overline{RD}$：读命令线，低电平有效。若为高电平（$\overline{WR}$ 为低电平），则本片 8255A 处于写状态；若为低电平（$\overline{WR}$ 为高电平），则所选 8255A 处于读状态。

$\overline{WR}$：写命令线，低电平有效。

A_0、A_1：地址线，用于选择 A 口、B 口、C 口。与片选信号 $\overline{CS}$ 一起构成 8255A 的各口的地址。当对 8255A 的各口及控制逻辑进行操作时，必须给定相应的地址。详见表 10-2 所示。

表 10-2　8255A 工作方式表

$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	A_1	A_0	工作方式
0	1	0	0	0	A 口数据送到数据总线
0	1	0	0	1	B 口数据送到数据总线
0	1	0	1	0	C 口数据送到数据总线
1	0	0	0	0	总线数据送 A 口
1	0	0	0	1	总线数据送 B 口
1	0	0	1	0	总线数据送 C 口
1	0	0	1	1	总线数据送控制字寄存器
1	1	0	×	×	数据总线三态
0	1	0	1	1	非法
×	×	1	×	×	数据总线三态

PA_7～PA_0：A 端口输入/输出线，双向 I/O 总线。可以设定为输入或输出方式，也可设定为输入/输出双向方式，由控制字决定。

PB_7～PB_0：B 端口输入/输出线，双向 I/O 总线。可以设定为输入或输出方式，由控制字决定。

PC_7～PC_0：C 端口输入/输出线，双向 I/O 总线。可以设定为输入或输出方式，也可设定为控制/状态方式，由控制字决定。

RESET：复位信号，高电平有效。它清除 8255A 所有控制寄存器内存，并将各端口都置成输入方式。

V_{CC}：电源，+5V。

GND：地线。

10.2.2 8255A 的工作模式

1．8255A 的控制字和状态字

8255A 通过控制字可设定为 3 种工作模式。控制字有两个，一个是工作方式控制字，用于 8255A 的初始化；另一个是 C 口位控制字，用于 C 口的位操作。这两个控制字使用同一口地址，由最高位 D_7 区分，若 D_7 为 1，此控制字为 8255A 的工作方式控制字；若 D_7 为 0，此控制字为 8255A 的 C 口的位控制字。

（1）工作方式控制字

8255A 的工作方式控制字用来设定 8255A 三个端口的工作方式及输入/输出状态。控制字的位定义如图 10-3 所示。

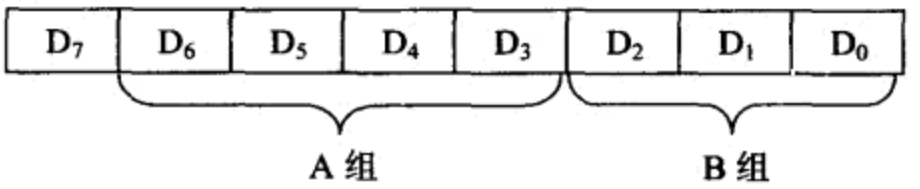


图 10-3　控制字的位定义

- D_7：控制字标志位。

D_7 为 1，则本控制字为方式控制字；D_7 为 0，则本控制字为 C 口控制字。

- D_6～D_3：A 组控制位。

➢ D_5、D_6：A 组方式选择位，D_6D_5 为 00 时，A 组设定为方式 0；D_6D_5 为 01，A 组设定为方式 1；若 D_6D_5=10 或 D_6D_5=11 时，A 组设定为方式 2。

➢ D_4：A 口输入/输出控制位，D_4 为 0，则 PA_7～PA_0 用于输出数据；D_4 为 1，则 PA_7～PA_0 用于输入数据。

➢ D_3：C 口高四位输入/输出控制位；D_3 为 0，则 PC_7～PC_4 为输出数据方式；D_3 为 1，则 PC_7～PC_4 为输入方式。

- D_2～D_0：B 组控制位。

➢ D_2：方式选择位，D_2 为 0 时 B 组设定为方式 0；D_2 为 1 时 B 组设定为方式 1。

➢ D_1：B 口输入/输出控制位，D_1 为 0，则 PB_7～PB_0 用于输出数据；D_1 为 1，则 PB_7～PB_0 用于输入数据。

➢ D_0：C 口低 4 位输入/输出控制位，D_0 为 1，则 PC_3～PC_0 用于输出数据；D_0 为 1，则 PC_3～PC_0 用于输入数据。

【例 10.1】若将 8255A 编程为 A 口方式 0 输入，B 口方式 1 输出，C 口的上半部分（PC_7～PC_4）输出，下半部分（PC_3～PC_0）输入，则工作方式控制字应设定为 10010101，即 95H。

设 8255A 控制寄存器的地址号为 48H，则其初始化程序如下：

```
MOV  AL,95H              ;控制字送 AL
OUT  40H,AL              ;送至 8255 的控制寄存器中
```

（2）C 口位控制字

对 C 口某位的置位/复位控制字，主要用于指定 C 口某位输出高电平还是低电平，作为输出的控制信号，如用于控制开关的通（置 1）/断（置 0），继电器的吸合/释放，电机的启/停控制等。控制字的 D_0 位用于区分是置 1 还是清 0 操作，但究竟对 C 口的哪一位按位操作，则由控制字中的 D_1、D_2、D_3 位决定。

控制字格式如图 10-4 所示。

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
		无效位			位选择		控制位

图 10-4　控制字格式

- D_7：对 C 口某位的置位/复位控制字的特征位，0 有效。
- D_3～D_0：用于控制 PC_7～PC_0 中某一位置位和清 0。

$D_3D_2D_1$=000　PC_0 置位或清 0
=001　PC_1 置位或清 0
=010　PC_2 置位或清 0
=011　PC_3 置位或清 0
=100　PC_4 置位或清 0
=101　PC_5 置位或清 0
=110　PC_6 置位或清 0
=111　PC_7 置位或清 0

- D_0：置位/复位的控制位。

当 D_0 为 0 时，控制 C 口的某位清 0；当 D_0 为 1 时，控制 C 口的某位置位。

【例 10.2】 需对 C 口 P_7 位实现置 1 操作时，此时控制字为 00001111B（0FH）。假设 8255A 控制寄存器的端口地址为 40H，故对 P_7 位置 1 操作的程序为：

```
MOV  AL,00001111B        ;控制字内容送 AL 中
OUT  40H,AL              ;控制字送 8255A 控制寄存器中
```

（3）8255A 状态字

8255A 设定为方式 1 和方式 2 时，读 C 口便可读得相应状态字，可以了解 8255A 的工作状态。

当 8255A 的 A 口、B 口工作在方式 1，则输入时的状态字格式如图 10-5 所示。在这个状态字中，INTEA 和 INTEB 分别为 A 组和 B 组的中断允许触发器状态，其余各位为相应引脚上的电平信号。

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
I/O	I/O	$\overline{IBFA}$	INTEA	INTR	INTEB	$\overline{IBFB}$	INTR
		A 口				B 口	

图 10-5　输入时的状态字格式

当 8255A 的 A 口、B 口工作在方式 l，则为输出时的状态字格式如图 10-6 所示。

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
$\overline{OBFA}$	INTEA	I/O		INTR	INTEB	$\overline{OBFB}$	INTR
		A 口				B 口	

图 10-6　方式 1 下输出时的状态字格式

8255A 在方式 2 下的状态字格式如图 10-7 所示。在这个状态字中，INTEl、INTE2 和 INTR 为 8255A 的允许中断触发器状态。其中 INTEl 和 INTE2 由 C 口的置复位控制字决定，其余各位为同名引脚上的电平信号。D_2～D_0 由 B 组工作方式决定。

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
$\overline{OBFA}$	INTE	$\overline{IBFA}$	INTEA	INTR			
		A 口				B 口	

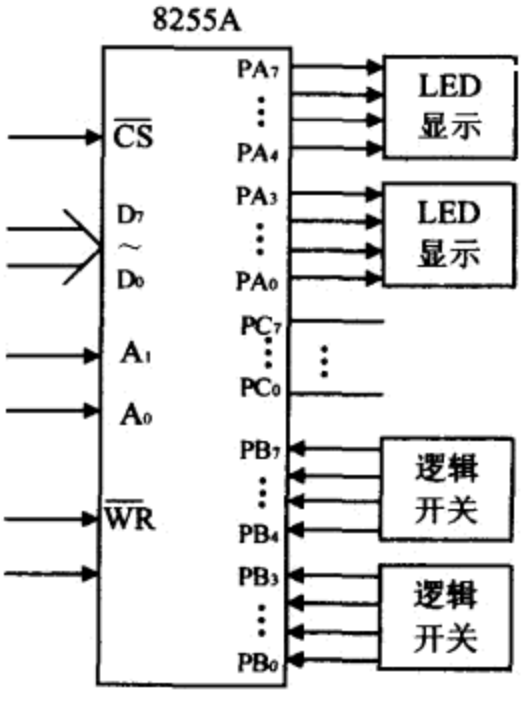
图 10-7　方式 2 下的状态字格式

2．8255A 的工作方式

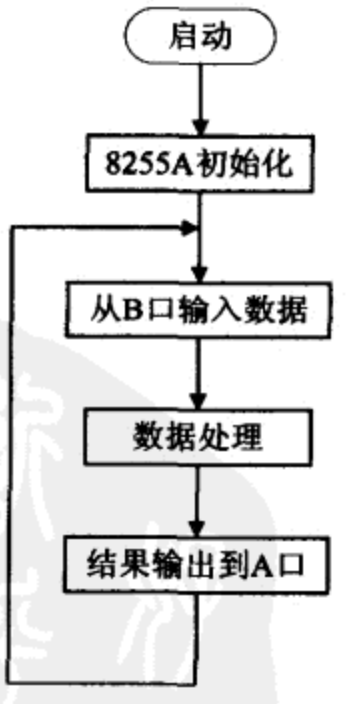
可通过软件设定 8255A 的工作方式，即方式 0、方式 1 和方式 2。

（1）方式 0

方式 0 又称为基本的输入/输出工作方式。A 口、B 口、C 口均可以工作在方式 0，这是一种基本的输入/输出工作方式：当外设始终处于传送数据的一切准备工作就绪这样一种状态时，则无需用专用的应答联络信号，故 CPU 可以通过 8255A 随时与外设间输入/输出数据。典型的例子是以开关或计数器状态作为输出信号，以发光二极管（LED）作为显示输出。如图 10-8 所示。



（a）逻辑开关、发关二级管显示接口



（b）操作流程图

图 10-8　8255A 方式 0 应用示意图

方式 0 的基本特点如下。

① A 口、C 口的高 4 位、B 口以及 C 口的低 4 位可分别定义为输入或输出，各端口互相独立，故共有 16 种不同的组合。例如可定义 A 口和 C 口高 4 值为输入口，B 口和 C 口低 4 值为输出口。或 A 口为输入，B 口、C 高 4 位、C 口低 4 位为输出等等。

② 定义为输出的口均有锁存数据的能力，而定义为输入的口则无锁存能力。

③ 在方式 0 下，C 口有按位进行置位和复位的能力。

方式 0 最适合用于无条件传送方式，由于传送数据的双方互相了解对方，所以既不需要发控制信号给对方，也不需要查询对方状态，故 CPU 只需直接执行输入/输出指令便可将数据读入或写出。

方式 0 也能用于查询工作方式，由于没有规定固定的应答信号，这时常将 C 口的高 4 位（或低 4 位）定义为输入口，用来接收外设的状态信号。而将 C 口的另外 4 位定义为输出口，输出控制信息。此时的 A、B 口可用来传送数据。

【例 10.3】设 8255A 的控制字寄存器地址为 9BH，则令 A 口和 C 口高 4 位工作在方式 0 输出方式，以及 B 口和 C 口低 4 位工作于方式 0 输入方式，指令序列如下：

```
MOV     AL,83H          ;方式控制字 83H 送 AL
MOV     DX,9BH
OUT     DX,AL           ;83H 送控制字寄存器
```

（2）方式 1

方式 1 是选通输入/输出工作方式，在这种工作方式下，选通信号与输入/输出数据一起传送，由选通信号对数据进行选通，其基本特点如下：

① 三个端口分为两组，即 A 组和 B 组。

② A 组包括 8 位数据端口 A 和 PC7 一 PC3 五位控制状态端口，B 组为 8 位数据端口 B 和 PC2 一 PC0 三位状态控制端口。

③ 每一个 8 位数据端口均可设置为输入或输出方式，且两种工作方式均可锁存。

④ 控制状态口除了指示两组数据口的状态及选通信号外，还可用做 I/O 口，如 PC7 和 PC6，用位控方式传送。

8255A 工作在方式 1 时，A 口和 B 口皆可独立地设置成这种工作方式。输入/输出有着各自规定的联络信号和中断信号，为方便起见，下面分别以 A 口、B 口均作为输入或均作为输出来加以说明。

① 方式 1 下 A 口、B 口均为输入

A 口定义为方式 1 输入时，以 C 口的 PC_5、PC_4、PC_3 为选通联络线，B 口定义为方式 1 输入时，PC_2、PC_1 和 PC_0 为选通联络线。当 A 口工作于方式 1 输入状态时如图 10-9 所示。

C 口的相应联络线定义如下。

$\overline{STB}$（PC_4、PC_2）——选通输入，低电平有效。由外设输入数据，并将数据送到输入锁存器。其中 PC_4 对应 A 口，PC_2 对应 B 口。

IBF（PC_5、PC_1）——输入缓冲器满，高电平有效。当它为 1 时，说明 CPU 还未读取上次输入的数据，通知外设不应送新数据。当它为 0 时，通知外设可送新数据。其中 PC_5 对应 A 口，PC_1 对应 B 口。

INTR（PC_3、PC_0）——中断请求，高电平有效。当中断允许位 INTR 置 1 时，若输入缓冲器满，则产生一个“高”有效的中断请求 INTR 至 CPU，对外设送来的新数据以中断方式输入。其中 PC_3 对应 A 口，PC_0 对应 B 口。

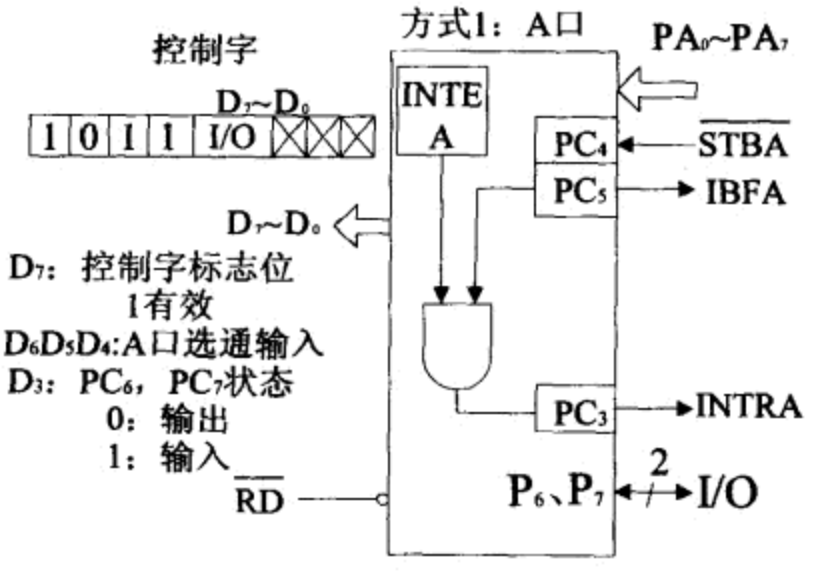


图 10-9　8255 的 A 口方式 1 输入方式下状态

② 方式 1 下 A 口、B 口均为输出

A 口定义为方式 1 输出时，以 C 口的 PC_7、PC_6、PC_3 为选通联络线，B 口定义为方式 1 输出时，PC_2、PC_1 和 PC_0 为选通联络线。当 A 口工作在方式 1 输出状态时如图 10-10 所示。

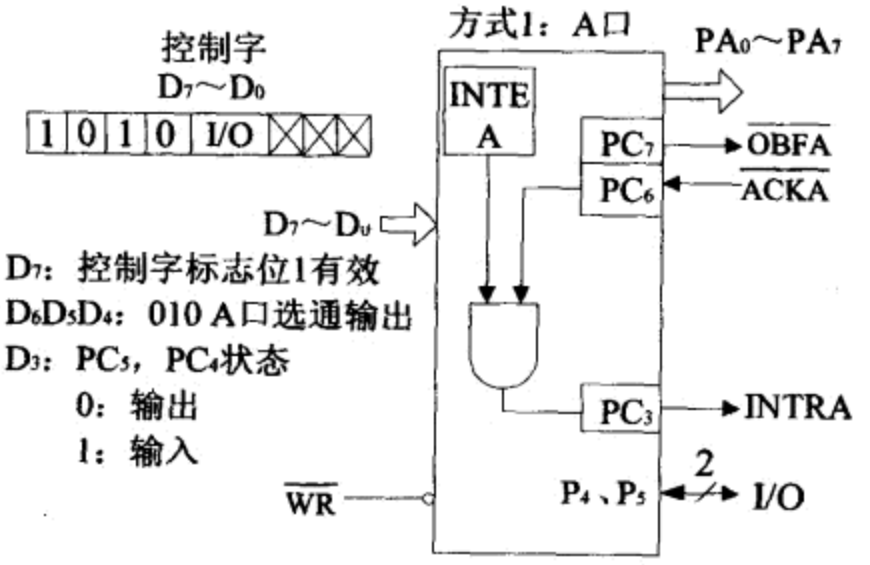


图 10-10　8255 的 A 口方式 1 输出方式下状态

C 口的各位定义如下。

$\overline{OBF}$（PC_7、PC_1）——输出缓冲器满信号，低电平有效。当数据写入该口的数据寄存器时，即启动该信号，以通知外设读取端口数据。其中 PC_7 对应 A 口，PC_1 对应 B 口。

$\overline{ACK}$（PC_6、PC_2）——外部响应输入信号，低电平有效。当外设读取端口数据后，回发一“低”有效信号作为回答。其中 PC_6 对应 A 口，PC_2 对应 B 口。

INTR（PC_3、PC_0）——中断请求信号，高电平有效。当中断允许位 INTR 置 1 时，若输出缓冲器空（$\overline{OBF}$=1），则产生一个“高”有效的中断请求 INTR 至 CPU，于是可在其中断处理程序中向该口输出新的数据。其中 PC_3 对应 A 口，PC_0 对应 B 口。

（3）方式 2

8255A 的另一种工作方式为带选通双向总线 I/O 方式，所以方式 2 又称为双向传输方式。且只有 A 口可以工作在这一方式下，A 口为输入/输出数据端，输入、输出均可锁存，既可以发送数据，也可以接收数据，握手联络信号和 A 口在方式 1 下的输入或输出时的握手联络信号分别相对应，输入/输出时的中断请求都共用 PC_3。这是一个“或”逻辑，即 PC_6 置 1 时，输出缓冲器为“空”可引起中断，PC_4 置 1 时输入缓冲器“满”也能引起中断。

当 A 口工作于方式 2 时，B 口可工作在方式 0 或者方式 1；C 口的高 5 位为 A 口的握手联络信号，低 3 位可用于 B 口在方式 1 时的握手联络信号。由 8255A 的控制字选择。

方式 2 的特点为：

① 工作方式 2 只适用于 A 口，B 口仍按方式 0 或方式 1 工作。

② A 口可工作于双向方式，C 口的 PC_7～PC_3 位作为 A 口的控制状态信号端口，PC_2～PC_0 用于 B 组。

③ A 口的输入/输出均有锁存功能。在方式 2 工作状态下、A 口既可工作于查询方式，又可工作于中断方式。

A 口工作于方式 2 状态如图 10-11 所示。

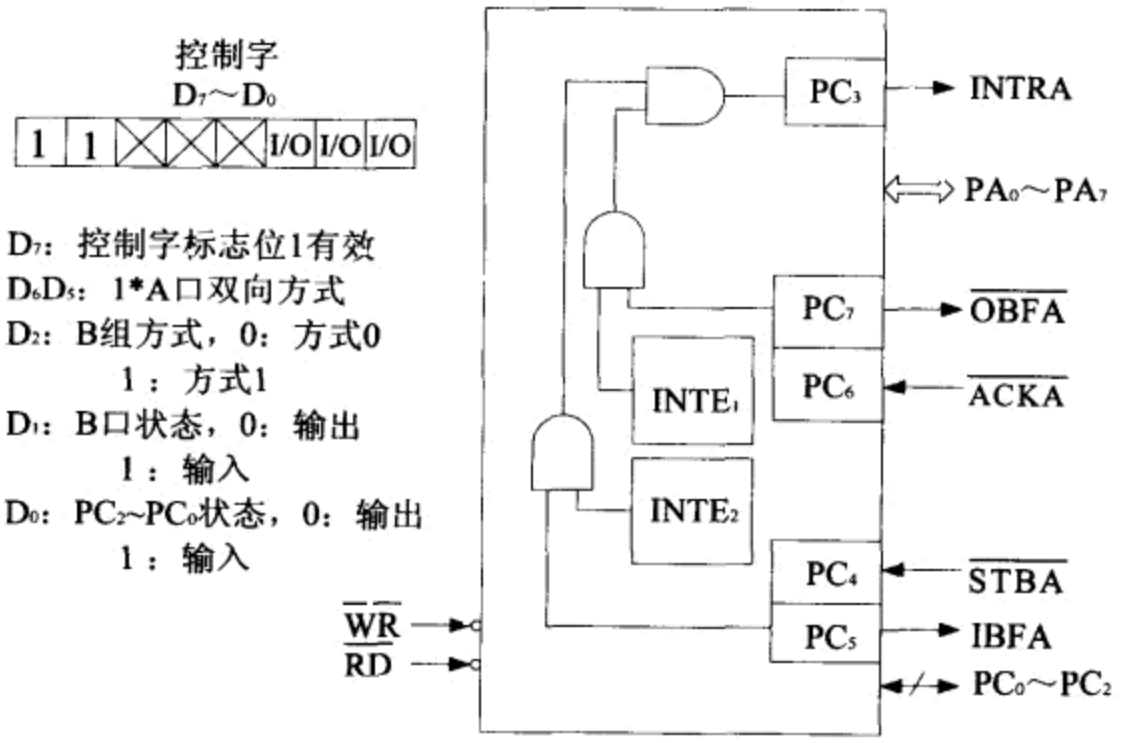


图 10-11　8255A 方式 2 工作状态

其中：

INTRA（PC_3）——中断请求信号，高电平有效。

$\overline{OBFA}$（PC_7）——输出缓冲器满信号，低电平有效。

$\overline{ACKA}$（PC_6）——外部响应信号，低电平有效。

$INTE_1$：输出缓冲器的中断允许触发器，由 PC_6 置位/复位控制。

$\overline{STBA}$（PC_4）——选通输入信号，低电平有效。

IBFA（PC_5）——输入缓冲器满信号，高电平有效。

$INTE_2$：输入缓冲器的中断允许触发器，由 PC_4 置位/复位控制。

10.2.3 8255A 的编程及应用

8255A 是计算机外围接口芯片中典型的一种，主要用于接口扩展、外设扩展应用等。对 8255A 编程，首先应对 8255A 进行初始化，即向 8255A 写入控制字，规定 8255A 的工作方式，A 口、B 口、C 口的工作方式等。然后，如果需要中断，则用控制字将中断允许标志置位。再以后就可以按相应的要求向 8255A 送入数据或从 8255A 读出数据。

【例 10.4】要求 8255A 工作在方式 0，A 口、B 口输入，C 口输出。

其硬件电路如图 10-12 所示，片选端 $\overline{CS}$ 接译码电路输出（译码端由地址线 A_7、A_6、A_5 译码输出），按要求 8255A 的控制字为 92H（D_7～D_0 对应的数据为 10010010）。

其工作程序如下：

```
PORTK       EQU     1FH       ;8255A 控制口地址
PORTA       EQU     1CH       ;8255A 的 A 口地址
PORTK       EQU     1DH       ;8255A 的 B 口地址
PORTK       EQU     1EH       ;8255A 的 C 口地址
;初始化 8255A
MOV     AL,92H                ;控制字方式 0,A、B 输入,C 输出
MOV     DX,PORTK              ;控制寄存器地址
OUT     DX,AL                 ;控制字送控制寄存器
;A 口、B 口、C 口读/写
MOX     DX,PORTA              ;A 口地址
IN      AL,DX                 ;从 A 口读数据
MOX     DX,PORTB              ;B 口地址
IN      AL,DX                 ;从 B 口读数据
MOX     DX,PORTC              ;C 口地址
MOV     AL,DATA
OUT     DX,AL                 ;向 C 口输出数据 DATA
```

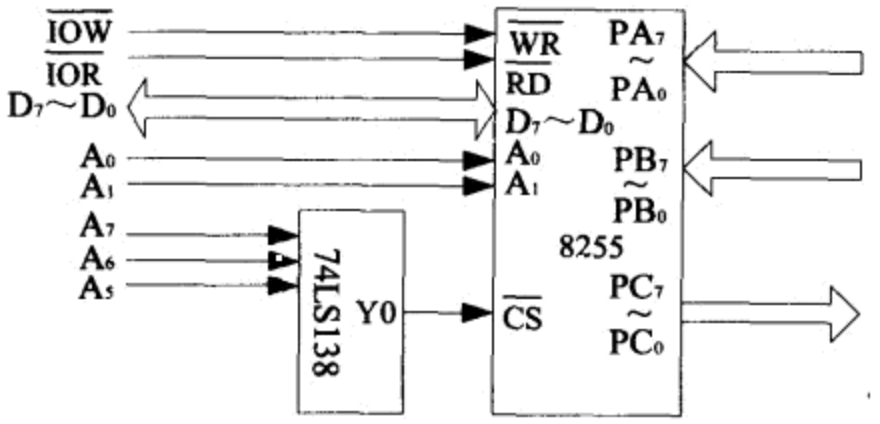


图 10-12　8255A 的基本应用

【例 10.5】要求 8255A 工作方式 1，A 口输入，B 口输出，PC_6、PC_7 输出，禁止 A 口中断。

按要求 8255A 控制字为 0B7H，程序如下：

```
PORTK       EQU     1FH       ;8255A 控制口地址
PORTA       EQU     1CH       ;8255A 的 A 口地址
PORTK       EQU     1DH       ;8255A 的 B 口地址
PORTK       EQU     1EH       ;8255A 的 C 口地址
;初始化 8255A
MOV     AL, 0B7H              ;控制字方式 1,A 输入,B 输出
MOV     DX, PORTK             ;控制寄存器地址
```

```
OUT     DX, AL                  ;控制字送控制寄存器
MOV     AL, 09H
OUT     DX, AL
MOV     AL, 04H
OUT     DX, AL
```

【例 10.6】利用 8255A 实现打印机接口，打印 BL 中的数据。

打印机工作时序如图 10-13 所示。

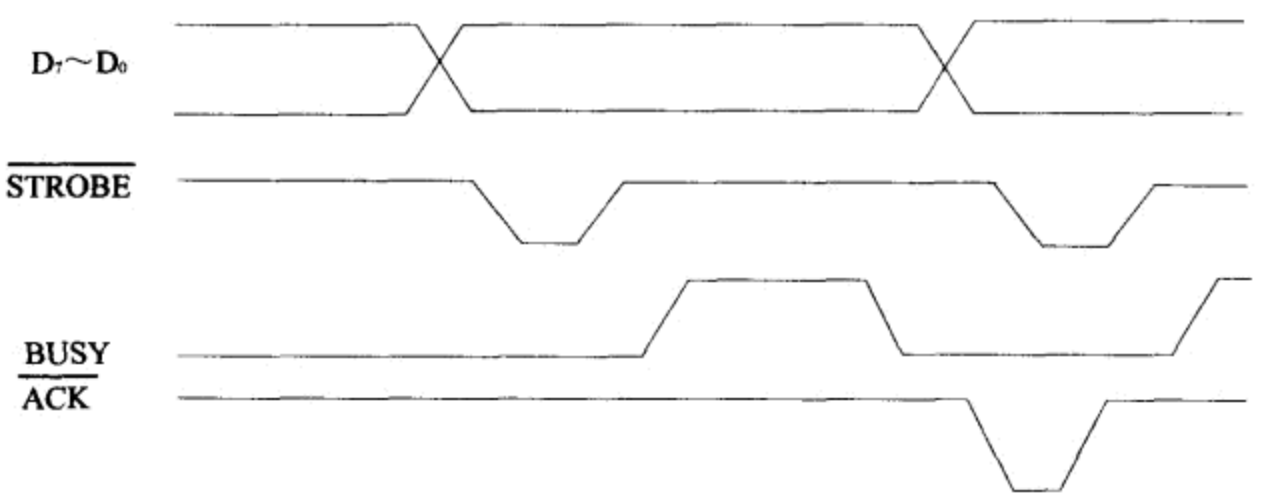


图 10-13　打印机工作时序

数据接口将数据传送到打印机的数据端口，利用一个负脉冲启动锁存，然后由打印机处理。同时打印机送出高电平信号 BUSY，表示打印机忙于打印。一旦 BUSY 变为低电平，表示打印结束，即可接收下一数据。利用 8255A 工作方式 0 实现上述打印控制。

硬件连接如图 10-14 所示，8255A 的地址线 A_1、A_0 与计算机的 A_1、A_0 连接，片选信号由 74LS138 译码产生。根据连线可知 8255A 的 A 口、B 口、C 口及控制口的地址为：1CH、1DH、1EH、1FH。

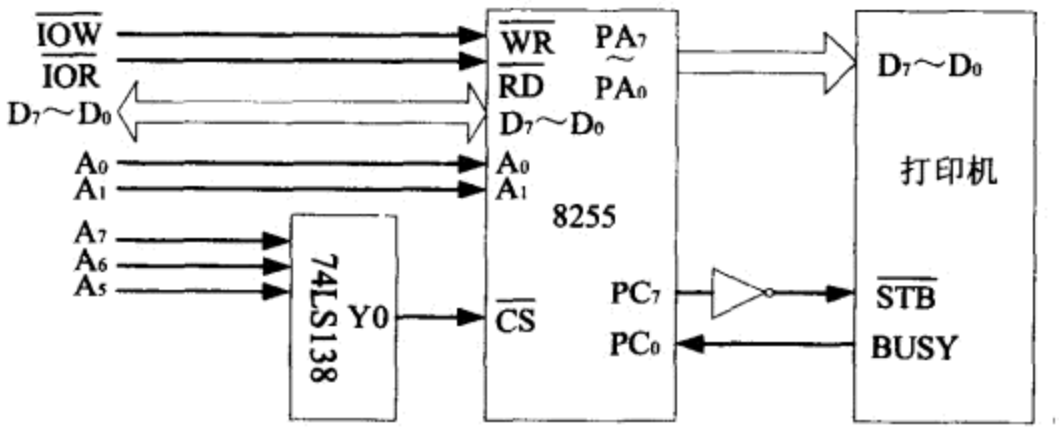


图 10-14　8255 与打印机接口电路

编制程序如下：

```
            MOV     AL,81H          ;控制字，方式 0,A、B、C(高 4 位)口输出
                                    ;C 口低 4 位输入
            OUT     1FH,AL          ;送控制字
WAITING:    IN      AL,1EH          ;读 C 口
            TEST    AL,01H          ;测试 PC0 是否为 1
            JNZ     WAITING         ;是 1,打印机忙,等待
            MOV     AL,BL           ;是 0,送数据至打印机
            OUT     1CH,AL
            MOV     AL,0FH
```

```
        OUT     1EH,AL
        MOV     AL,0F0H
        OUT     1EH,AL
```

【例 10.7】在某一系统中有 8 个开关 K_7～K_0，要求不断检测它们的通断状态，并随时在发光二极管 LED_7～LED_0 上显示出来。开关断开，相应的 LED 点亮；开关闭合，LED 熄灭。要求在 8086 微机系统中选用 8255A 和 74LS138 译码器等芯片完成上述功能。

分析：开关和 LED 显示管都是简单的外部设备，不需用应答和选通信号，因此可用 8255A 的 A 口和 B 口分别工作在方式 0 的输入和输出状态，用 A 口读入开关状态，用 B 口将开关状态在 LED 显示管上显示出来，8255A 的 $\overline{RD}$、$\overline{WR}$ 和 RESET 引脚分别与 CPU 的相应输出相连，8255A 的数据线 D_7～D_0 与 8086 微处理器的低 8 位数据线 D_7～D_0 相连，74LS138 译码器的 Y_4 与 8255A 的片选信号 $\overline{CS}$ 相连，A_7 连 G_1，M/IO 连 G_{2A}，A_6、A_5 接与非门输入端，与非门输出端与 G_{2B} 相连。具体情况如图 10-15 所示。

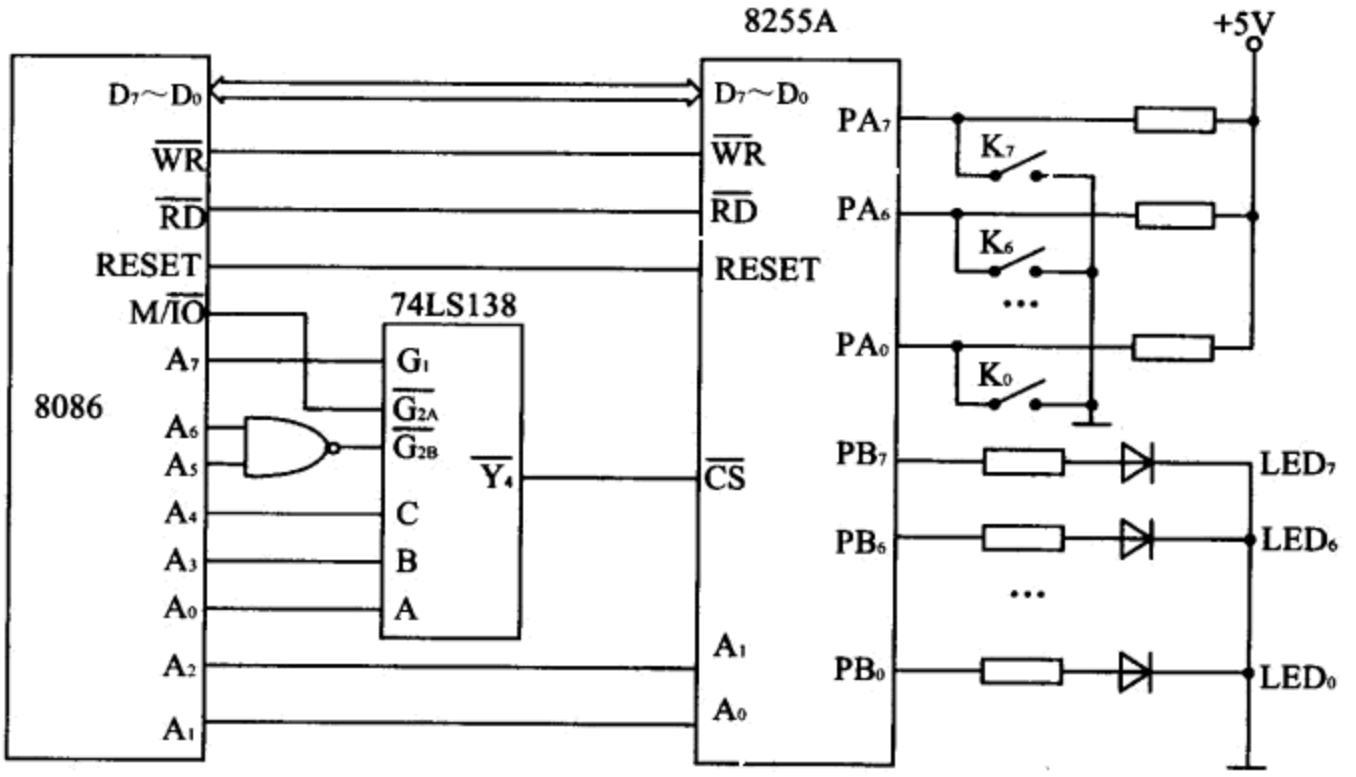


图 10-15　例 10.7 硬件连接示意图

分析图 10-10 可以得到，当 $A_7A_6A_5$=111，$A_4A_3A_0$=100 时，$\overline{Y4}$=0 选中 8255A，这样，4 个端口地址分别为 F0H、F2H、F4H 和 F6H，对应 8255A 的 A 口、B 口、C 口和控制字寄存器。

编程时首先要确定方式选择控制字。由于 A 口工作于方式 0 输入，B 口工作于方式 0 输出，C 口未用，控制字中与 C 口对应的控制位可以设置为 0，这样写入控制寄存器的控制字的内容应为 10010000。完成初始化后，即可将 A 口的开关状态读入寄存器 A 加。若开关合上，AL 中的相应位为 0，断开则为 1。当把 AL 中的内容从 B 口输出时，相应的位上的 LED 熄灭，表示对应的开关是合上的；否则 LED 点亮，指示开关断开。具体程序如下。

```
        MOV   DX,0F6H              ;控制字寄存器地址送 DX
        MOV   AL,10010000B         ;控制字内容送 AL 中
        OUT   DX,AL                ;控制字输出
TEST_IT:MOV   DX,0F0H              ;指向 A 口
        IN    AL,DX                ;从 A 口读入开关状态
```

```
MOV  DX,0F2H              ;指向 B 口
OUT  DX,AL                ;B 口控制 LED,显示开关状态
JMP  TEST_IT              ;循环检测
```

【例 10.8】在甲乙两台微机之间并行传送 1KB 的数据。甲机发送在数据段中从 DAT 单元开始的 1KB 数据，乙机接收后存放在数据段的 DAT 开始的 1KB 单元中。甲机一侧的 8255A 的 A 口采用方式 1 工作，乙机一侧的 8255A 的 A 端口采用方式 0 工作。两机的 CPU 与接口之间都采用查询方式交换数据。两台微机的 8255A 端口的地址都为 300H～303H。

本例题的分析过程如下。

（1）硬件连接

根据上述要求，甲机 8255A 是方式 1 发送，因此 PA 口规定为输出，发送数据，而 PC_7 和 PC_6 引脚分别固定作联络信号线 $\overline{OBF}$ 和 $\overline{ACK}$。乙机 8255A 是方式 0 接收，故把 PA 口定义为输入，接收数据，而选用引脚 PC_4 和 PC_0 作为联络信号。

接口电路连接图如图 10-16 所示。

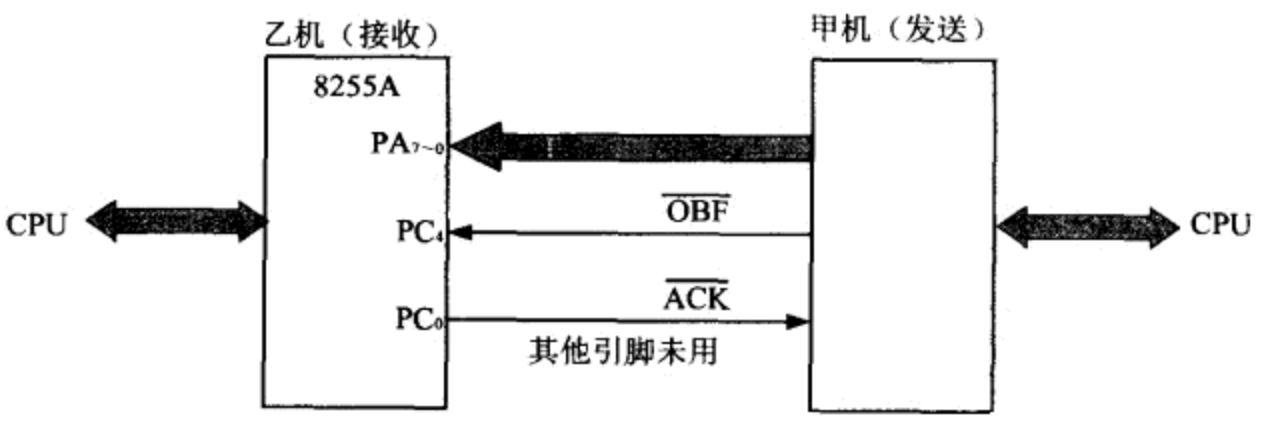


图 10-16　双机并行传送硬件连接示意图

（2）程序设计

甲机发送程序设计思路如下。

① 初始化 PA 口为方式 1 输出。

② 做好循环准备工作，即 BX 指向数据区 DAT 单元，发送字节数送 CX 寄存器。

③ 向 A 口发送一个字节，并修改内存地址和计数器的值。

④ 检测 PC_3 是否有效（即为高电平 1），不为 1 继续检测。

⑤ PC_3=1 向 A 口发送下一个字节。

⑥ 修改地址指针和计数器值。

⑦ 计数器为 0 退出，否则转到④继续。

乙机接收程序设计思路如下。

① 初始化 PA 口为方式 0 输入，$\overline{ACK}$=1（PC_0=1）。

② 做好循环准备工作，即 BX 指向数据区 DAT 单元，发送字节数送 CX 寄存器。

③ 检测 $\overline{OBF}$=0?(PC_4=0？），不相等继续检测。

④ PC_4=0，从 A 口接收数据送到内存中。

⑤ 发应答信号 $\overline{ACK}$（PC_0=0→1）。

⑥ 修改地址指针和计数器值。

⑦ 计数器为 0 退出，否则转到③继续。

（3）软件编程

甲机发送程序：

```
DATA    SEGMENT
  DAT   DB 41H,43H,12H,0AH…          ;定义 1K 个字节数据
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE, DS:DATA
START:  MOV  AX,DATA
        MOV  DS,AX                   ;段寄存器初始化
        MOV  DX,303H                 ;8255A 控制寄存器端口地址送 DX
        MOV  AL,10100000B            ;控制字内容送 AL
        OUT  DX,AL                   ;写控制字寄存器
        MOV  AL,0DH                  ;将 PC6 置 1
        OUT  DX,AL
        MOV  BX,OFFSET DAT           ;BX 指向数据的首单元
        MOV  CX,1024                 ;传输的字节数送 CX
        MOV  DX,300H                 ;向 A 口写第一个数，产生第一个 OBF 信号
        MOV  AL,[BX]                 ;送给对方，以便获取对方的 ACK 信号
        OUT  DX,AL
        INC  BX                      ;指针调整
        DEC  CX                      ;计数器减 1
LP:     MOV  DX,302H                 ;C 口地址送 DX
        IN   AL,DX                   ;读 C 口内容
        AND  AL,08H                  ;检测 INTRa=1?
        JZ   LP                      ;不为 1，说明数据未取走，继续等待
        MOV  DX,300H                 ;否则发送下一个数据
        MOV  AL,[BX]
        OUT  DX,AL
        INC  BX                      ;调整地址指针
        DEC  CX                      ;计数器减 1
        JNZ  LP                      ;计数器不为 0，继续发送下一个字节数据
        MOV  AH,4CH
        INT  21H                     ;否则退出程序
CODE    ENDS
        END  START
```

乙机接收程序如下：

```
DATA    SEGMENT
  DAT   DB 1024 DUP(?)               ;定义 1K 个字节单元，作为输入缓冲区
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE, DS:DATA
START:  MOV  AX,DATA
        MOV  DS,AX                   ;段寄存器初始化
        MOV  DX,303H                 ;8255A 控制寄存器端口地址送 DX
        MOV  AL,10011000B            ;控制字内容送 AL
        OUT  DX,AL                   ;写控制字寄存器
        MOV  AL,01H                  ;置 ACK=1 (PC0=1)
        OUT  DX,AL
        MOV  BX,OFFSET DAT           ;BX 指向数据的首单元
        MOV  CX,1024                 ;传输的字节数送 CX
LP:     MOV  DX,302H                 ;C 口地址送 DX
        IN   AL,DX                   ;读 C 口内容
        TEST AL,10H                  ;检测甲机的 OBF=0?(PC4=0)
        JNZ  LP                      ;不为 0，说明甲机没有数据发来，继续等待
```

```
        MOV  DX,300H                ;有
        IN   AL,DX                  ;从 A 口读入数据
        MOV  [BX],AL                ;存入内存中
        MOV  DX,303H                ;产生 ACK 信号，发回给甲机
        MOV  AL,00H                 ;使 PC0 置 0,接在甲机的 ACK
        OUT  DX,AL
        NOP
        NOP
        NOP
        NOP
        MOV  AL,01H                 ;PC0 置 1，使 ACK 端出现负脉冲
        OUT  DX, AL
        INC  BX                     ;调整地址指针
        DEC  CX                     ;计数器减 1
        JNZ  LP                     ;计数器不为 0 继续接收下一个字节数据
        MOV  AH,4CH
        INT  21H                    ;否则退出程序
CODE    ENDS
        END  START
```

本章小结

本章主要介绍了可编程接口芯片 8255A 的内部结构、引脚功能、初始化编程和主要应用。

8255A 是可编程的并行接口芯片，可以为计算机外设提供 3 个 8 位并行接口，即端口 A、端口 B 和端口 C。可以工作在 3 种工作方式下，即方式 0（基本的输入/输出工作方式）、方式 1（选通的输入/输出方式）和方式 2（双向选通输入/输出方式）。

8255A 各数据端口的工作方式由方式选择控制字进行设置。对 8255A 进行初始化编程时，通过向控制字寄存器写入方式选择控制字，可以让 3 个数据端口按照用户所需要的方式进行工作。

思考与练习题

1. 可编程并行接口芯片 8255A 有哪些主要特征?
2. 可编程并行接口芯片 8255A 有哪几种工作方式，每种工作方式有何特点？
3. 某 8255A 的端口地址范围为 03F8H～03FBH，A 组和 B 组均工作在方式 0，A 口作为数据输出端口，C 口低 4 位作为状态信号输入口，其他端口未用。试画出该片 8255A 与系统的连接图，并编写初始化程序。
4. 用 8255A 作为接口芯片，编写满足下述要求的 3 段初始化程序：

   （1）将 A 组和 B 组置成方式 0，A 口和 C 口作为输入口，B 口作为输出口。

   （2）将 A 组置成方式 2，B 组置成方式 1，B 口作为输出口。

   （3）将 A 组置成方式 1 且 A 口作为输入，PC_6 和 PC_7 作为输出，B 组置成方式 1 且 B 口作为输入口。
5. 将例 10.8 的双机并行传送改为接收和发送方均工作在方式 1，且利用中断程序来实现，试分析工作原理并编制相应程序。

第 11 章 可编程串行接口芯片 8251A

本章主要讲解串行传输的基本概念和串行接口技术，介绍微型计算机系统中可编程串行接口芯片 8251A 的基本结构和编程方法，主要包括 8251A 的 3 种工作方式及特点，控制字寄存器的应用及编程设计方法，并介绍串行异步通信接口 RS-232 的特点与应用，介绍 BIOS 串行异步通信接口的功能调用。

读者通过本章的学习，应该理解串行传输和串行接口的基本概念，熟悉串行接口芯片 8251A 的编程结构与使用方法，掌握串行异步通信接口 RS-232 的功能与应用。

11.1 串行传输的基本概念

11.1.1 串行通信的基本概念

1. 串行通信

串行通信是指两个功能模块只通过一条或两条数据线进行数据交换。发送方需要将数据分解成二进制位，一位一位地分时经过单条数据线传送。接收方需要一位一位地从单条数据线上接收数据，并且将它们重新组装成一个数据。串行通信数据线路少，在远距离传送时比并行通信造价低。但是一个数据只有经过若干次以后才可以传送完，速度较慢。

串行通信具有以下特点。

（1）由于数据在计算机内部传输和处理都采用并行方式，所以在进行串行传输之前必须将并行数据转换成串行数据流；在接收端又要将收到的串行数据流转换成并行数据。这种转换通常是以字节为单位进行的。

（2）由于传输过程中只使用一个信道，所以传输的二进制位流中必须包含数据流和控制流。控制流用于接收端控制数据的组装，识别数据的真伪。

（3）为了能够区分所传输的信息流中的数据流和控制流，必须在通信的双方约定串行通信的数据格式。这种约定就是串行通信协议。

（4）由于在串行通信中对信号的定义往往与常用 TTL 信号不同，因此串行通信中需要进行逻辑关系和逻辑电平的转换。

（5）串行通信中的物理传输手段各不相同。如可以用电缆方式，也可以用无线方式。

（6）串行通信用于计算机与其周边设备间的信息交换时，连接线路简单。

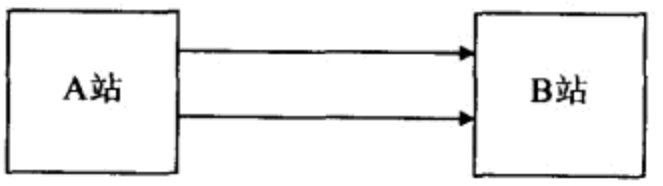
2. 半双工和全双工

串行通信是一位接一位地顺序通过一条信号线进行传输的方式。它的通路可以只有一条，此时发送和接收信息不能同时进行，只能采用分时使用线路的方法，如果 A 发送信息，B 只能接收；而当 B 发送信息时，A 只能接收。这种串行通信的工作方式称为半双工通信方式，如图 11-1（a）所示。

如果两个通信站之间有两条通路，则发送和接收信息就可同时进行。如 A 发送信息，B

接收，B 同时也能用另一条通路发送信息而由 A 接收。这种方式称为全双工通信方式，如图 11-1（b）所示。

（a）半双工通信



（b）全双工通信

图 11-1　通信方式示意图

3. 数据传输率

数据传输率是指每秒钟传送的二进制位数，通常称为波特率（Band Rate）。国际上规定了标准波特率系列，最常用的标准是：110 波特、300 波特、600 波特、1 200 波特、1 800 波特、2 400 波持、4 800 波特、9 600 波特和 19 200 波特。

例如，在某个异步串行通信系统中，数据传送速率为 960 字符/秒，每个字符包括一个起始位、8 个数据位和一个停止位，则波特率为 10×960=9 600(位/秒)=9 600(波特)。

4. 发送/接收时钟

在异步串行通信中，发送端需要用一定频率的时钟来决定发送每一位数据所占的时间长度（称为位宽度），接收端也要用一定频率的时钟来测定每一位输入数据的位宽度。发送端使用的用于决定数据位宽度的时钟称为发送时钟，接收端使用的用于测定每一位输入数据位宽度的时钟称为接收时钟。由于发送/接收时钟决定了每一份数据的位宽度，所以发送/接收时钟频率的高低决定串行通信双方发送核收字符数据的速度。

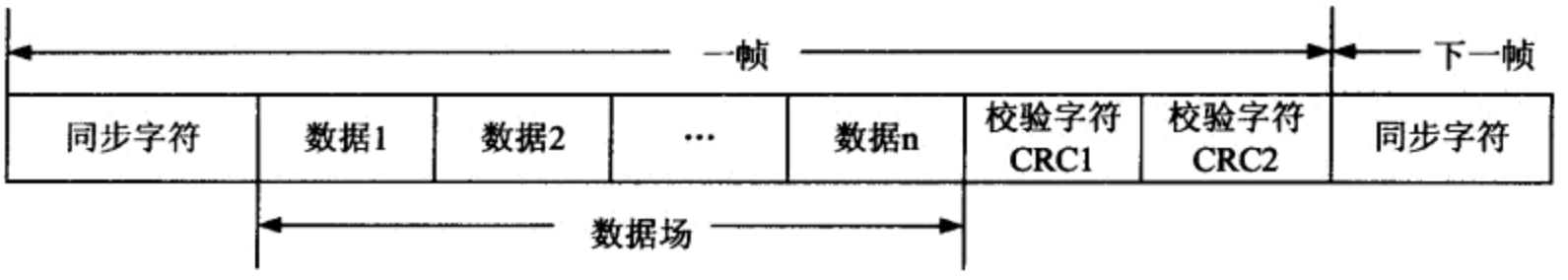
在进行串行通信时，根据传送的波特率来确定发送时钟和接收时钟的频率。在异步传送中每发送一位数据的时间长度由发送时钟决定，每接收一位数据的时间长度由接收时钟决定，它们和波特率之间有如下关系：时钟频率=n×波特率。

式中的 n 叫做波特率系数或波特率因子，它的取值可以为 1、16、32 或 64。

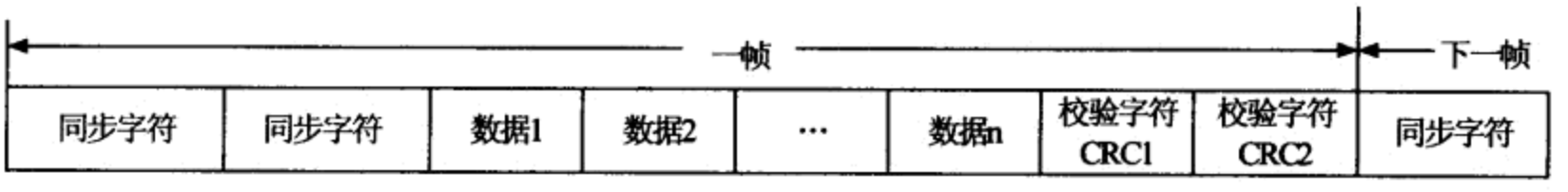
5. 串行通信的方式

串行通信按通信约定的格式分为两种：同步通信和异步通信。

（1）同步通信：同步通信是指在约定的数据通信速率下，发送方和接收方的时钟信号频率和相位始终保持一致（同步），保证通信双方在发送数据和接收数据时具有完全一致的定时关系。在有效数据传送之前首先发送一串特殊的字符进行标识或联络，这串字符称为同步字符或标识符。在传送过程中，发送端和接收端的每一位数据均保持同步。传送的信息组亦称为信息帧，信息帧的位数几乎不受限制，通常可以是几个到几千个字节，甚至更多。同步通信采用的同步字符个数不同，存在着不同的格式结构，具有一个同步字符的数据格式称为单同步数据格式，有两个同步字符的数据格式称为双同步数据格式，如图 11-2 所示。



（a）单同步通信格式



（b）双同步通信格式

图 11-2　同步通信格式

同步通信要求在传输线路上始终保持连续的字符位流，若计算机没有数据传输，则线路上要用专用的“空闲”字符或同步字符填充。

（2）异步通信：异步通信是指通信中两个字符之间的时间间隔是不固定的，而在一个字符内各位的时间间隔是固定的。异步通信规定字符由起始位（Start Bit）、数据位（Data Bit）、奇偶校验位（Parity）和停止位（Stop Bit）等组成。起始位表示一个字符的开始，接收方可以用起始位使自己的接收时钟与数据同步。停止位则表示一个字符的结束。这种用起始位开始，停止位结束所构成的一串信息称为一帧（Frame）。异步通信的传送格式如图 11-3 所示。

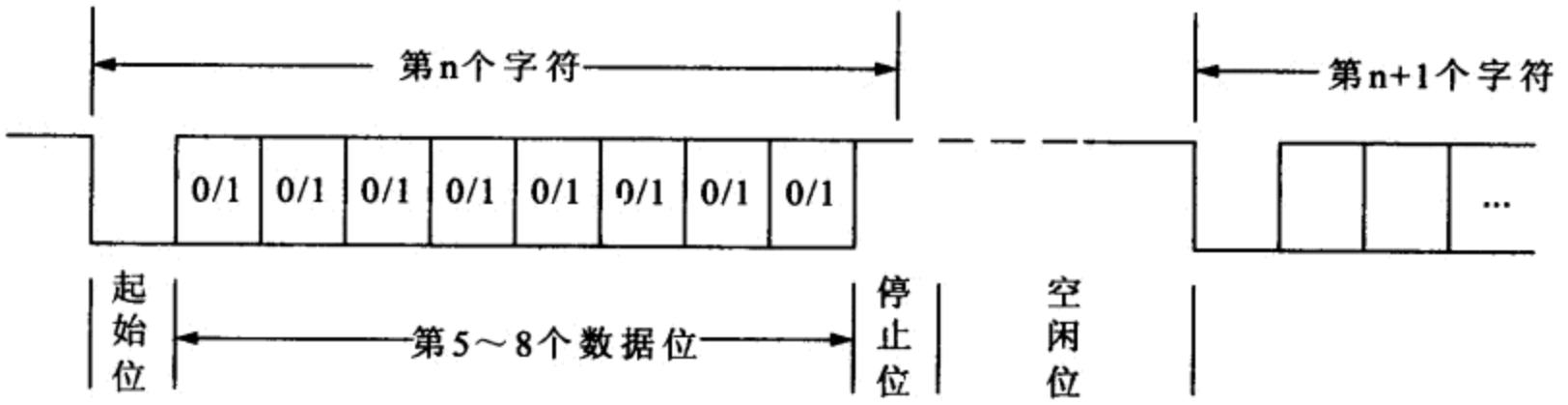


图 11-3　异步通信格式

异步通信在传送一个字符时，由一位低电平的起始位开始，接着传送数据位，数据位的位数为 5～8 位。在传送时，按低位在前，高位在后的顺序传送。奇偶校验位用于检验数据传送的正确性，也可以没有，或由程序来指定。最后传送的是高电平的停止位，停止位可以是 1 位、1.5 位或 2 位，两个字符之间的空闲位要由高电平 1 来填充。

11.1.2　信号的调制与解调

当串行传输距离较远时，就需要引入通信设备，在通信线路上采用调制解调技术。发送方使用调制器（Modulator），要把传送的数字信号调制转换为适合在线路上传输的音频模拟信号；接收方则使用解调器（Demodulator）从线路上测出这个模拟信号，并还原成数字信号。图 11-4 所示是使用调制解调技术后的通信机理。

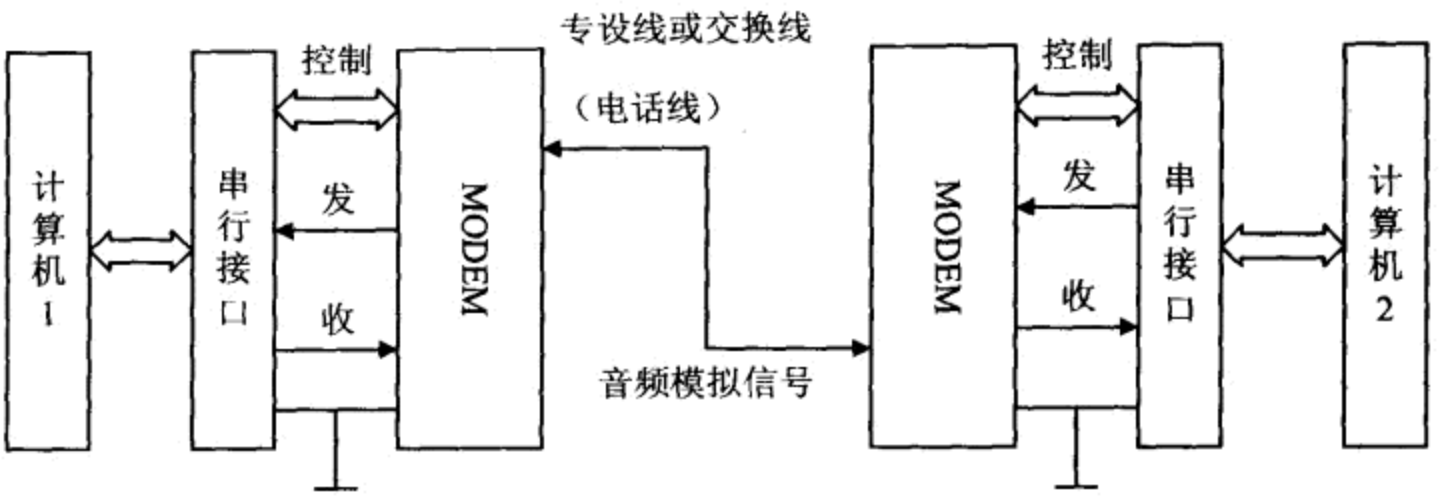


图 11-4　远程通信示意图

图中 MODEM 为调制解调器，兼具有发送方的调制和接收方的解调功能。对于“双工”方式，通信的任何一方都需要这两种功能，所以实际中常将调制和解调的功能集成在一起，构成完整的调制解调器供用户选用。

11.2　可编程串行接口芯片 8251A

11.2.1　8251A 的基本性能

8251A 是高性能串行通信接口芯片，它既是一种通用的异步收发器（UART），也是一种通用的同步收发器（USRT），能管理信号变化范围很大的串行数据通信，而且可以直接与多种微型计算机接口，其基本性能有以下几点。

（1）通信初始化程序可以工作在同步通信或异步通信方式下。同步方式下，波特率为 0～64kb/s；异步方式下，波特率为 0～19.2kb/s。

（2）同步方式时，可设定为内同步或外同步两种做法，同步字符允许采用单同步字符和双同步字符，由用户选定。数据位可在 5～8 位之间进行选择。

（3）异步方式时，数据位仍可在 5～8 位范围内选用，用 1 位作为奇偶校验位或不设置奇偶位。此外，8251A 在异步方式下能自动为每个数据增加 1 位启动位及 1 位、1.5 位或 2 位停止位（由初始化程序选择）。

（4）8251A 具有奇偶校验、帧校验和溢出校验 3 种字符数据的校验方式，校验位的插入、检查和出错标志的建立均由芯片自动完成。

（5）8251A 能与 MODEM 直接相连，接收和发送的数据均可存放在各自的缓冲器中，以便实现全双工通信。

11.2.2　8251A 的基本结构

可编程串行接口芯片有多种型号，常用的有 Intel 公司生产的 8251A，Motorola 公司生产的 6850、6952、8654，ZILOG 公司生产的 SIO 及 TNS 公司生产的 8250 等。这些芯片结构和工作原理大同小异，不必一一介绍。下面以 8251A 为例介绍可编程串行通信接口的基本工作原理、编程结构、编程方法及应用实例。

1. 8251A 的内部结构

8251A 的内部结构如图 11-5 所示，它具有双缓冲结构的接收器（接收缓冲寄存器、接收移位寄存器）和发送器（发送保持寄存器、发送移位寄存器），波特率发生器为发送器和接收器提供所需的同步控制时钟信号；还有实现与调制解调器连接的调制解调器控制逻辑，以及实现中断控制和优先权判断的中断控制逻辑。反映给用户的是各种寄存器，这将在后面介绍。另外还有与 CPU 连接必不可少的数据缓冲器和选择控制逻辑。

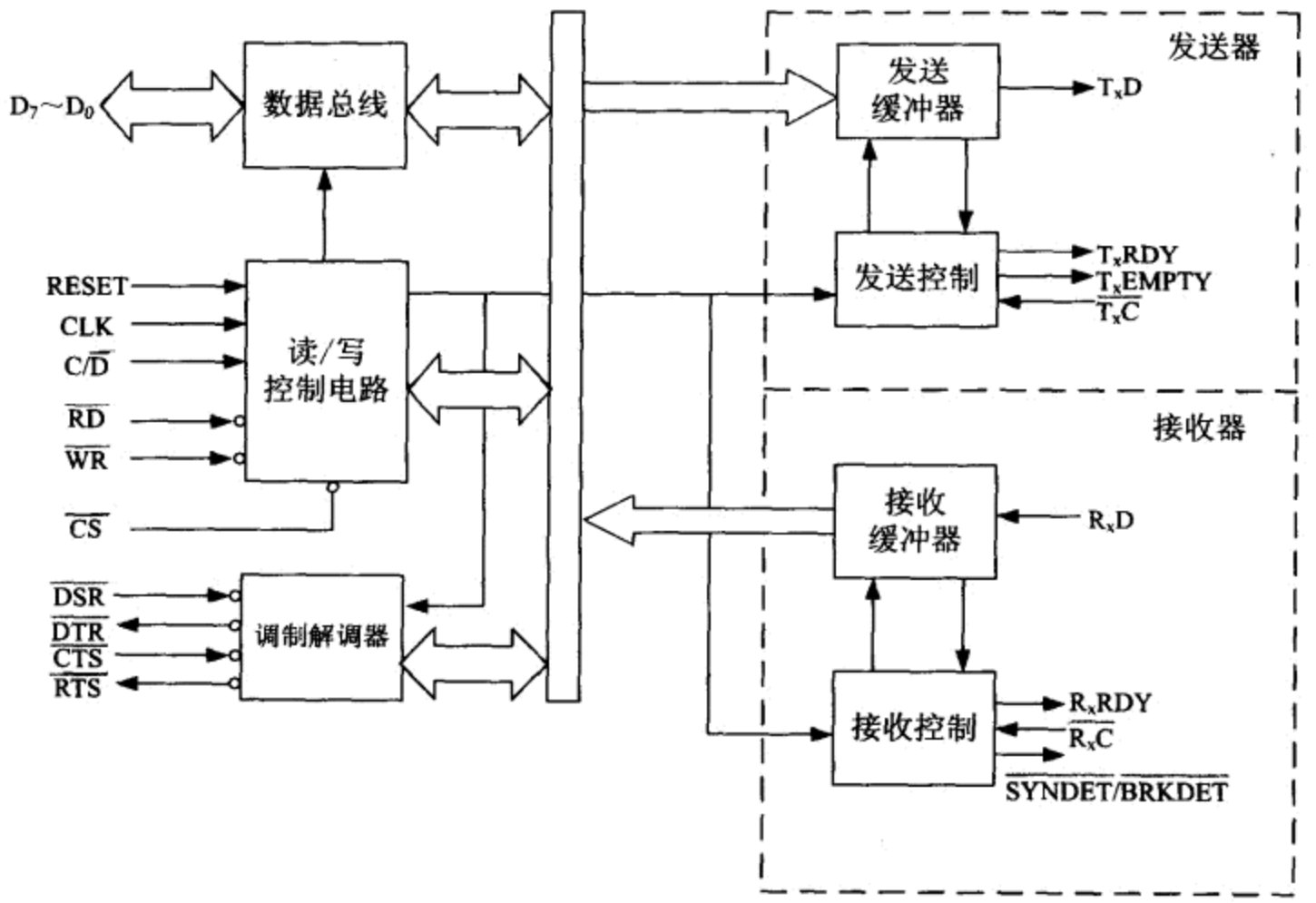


图 11-5　8251A 内部功能结构图

（1）数据总线缓冲器

数据总线缓冲器是 CPU 与 8251A 之间的数据接口，它包含 3 个 8 位缓冲寄存器，其中两个寄存器分别用来存放 CPU 从 8251A 读取的状态信息或数据，一个寄存器存放 CPU 向 8251A 写入的控制字或数据。数据总线缓冲器将 8251A 的 8 条数据线 D_7~D_0 和 CPU 的系统数据总线相连。

（2）发送器

发送器由发送缓冲器和发送控制电路两部分组成。CPU 需要发送的数据经数据发送缓冲器并行锁入发送缓冲器中。如果采用异步方式，则由发送控制电路在其首尾加上起始位和停止位，然后从起始位开始，经移位寄存器从数据输出线 T_xD 逐位串行输出，其发送速率取决于 T_xC 端上收到的发送的时钟频率。如果采用同步方式，则在发送数据之前，发送器将自动送出一个或两个同步字符，然后才逐位串行输出数据。当发送器做好发送数据准备时，由发送控制电路向 CPU 发出 T_xRDY 有效信号，CPU 可立即向 8251A 并行输出数据。如果 8251A 和 CPU 之间采用中断方式交换信息，则 T_xRDY 信号可作为向 CPU 发出的中断请求信号。待发送器中的 8 位数据串行发送完毕，由发送控制电路向 CPU 发出 T_xEMPY 有

效信号，表示发送器中移位寄存器已空。因此，发送数据缓冲器和发送移位寄存器构成发送器的双缓冲结构。

（3）接收器

接收器由接收缓冲器和控制电路组成。从外部通过数据接收端 R_xD 接收的串行数据逐位进入接收移位寄存器中。如果是异步方式，则应识别并删除起始位和停止位；如果是同步方式，则要检测到同步字符，确认已达到同步；接收器才可以开始接收串行数据，待一组数据接收完毕，可将移位寄存器中的数据并行写入接收数据缓冲器中，同时输出 R_xRDY 有效信号，表示接收器中已准备好数据，等待向 CPU 传送。其接收数据的速率取决于 R_xC 端输入的接收时钟频率。

和接收器有关的是读/写控制电路和调制解调控制电路。读/写控制电路用来接收 CPU 送来的一系列控制信号，以实现对 8251A 的读/写功能；调制解调控制电路是 8251A 将数据输出端的数字信号转换成模拟信号，或将数据接收端的模拟信号解调成数字信号的接口电路。8251A 要与调制解调器相连，它提供的接口信号一部分为与 CPU 接口的信号，另一部分为与外设或调制器的接口信号。

2. 8251A 的引脚功能

8251A 的引脚排列如图 11-6 所示，各类引脚功能分析如下。

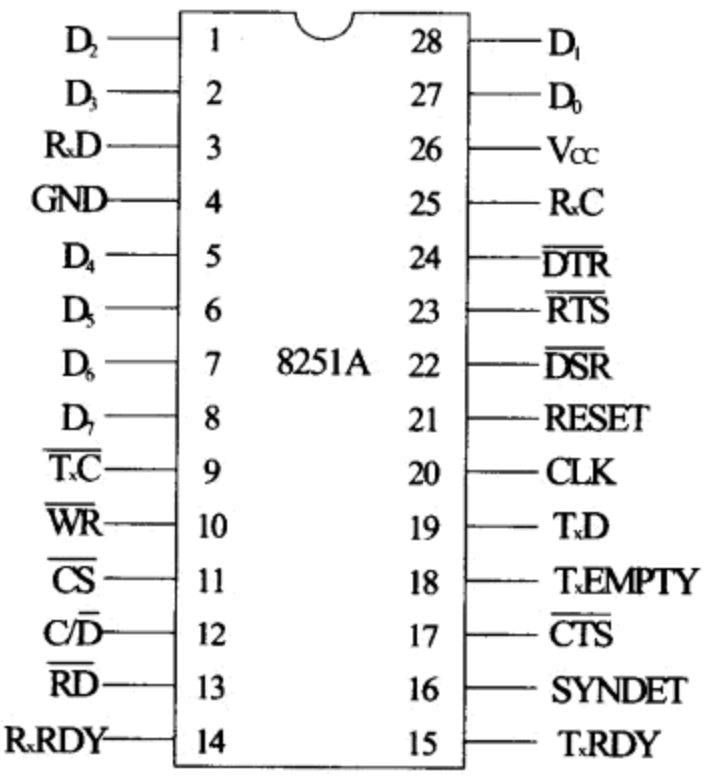


图 11-6　8251A 引脚排列图

（1）数据线、时钟信号线

- DB_7～DB_0：三态双向数据总线，可连到 CPU 数据总线。CPU 与 8251A 之间的命令信息、数据以及状态信息都是通过这组数据总线传送的。
- CLK：输入产生 8251A 的内部时序。CLK 的频率在同步方式工作时必须大于接收器和发送器输入时钟频率的 30 倍；在异步方式工作时必须大于输入时钟的 4.5 倍。另外，规定 CLK 的周期要在 0.42～1.35μs 的范围内。

（2）读/写控制逻辑

- $\overline{CS}$：片选信号，低电平有效。由 CPU 的 IO/$\overline{M}$ 及地址信号经译码后供给。
- C/$\overline{D}$：控制/数据端。为高电平时 CPU 从数据总线读入的是状态信息；为低电平时 CPU 读入的是数据。同样，C/$\overline{D}$ 端为高电平时 CPU 写入的是命令；C/$\overline{D}$ 为低电平时 CPU 输出数据。C/$\overline{D}$ 与 CPU 的一条地址线相连。
- RESET：芯片复位信号。为高电平时 8251A 各寄存器处于复位状态。收、发线路上均处于空闲状态。通常该信号与系统的复位线相连。
- $\overline{RD}$：CPU 读 8251A 的控制信号，低电平有效，与 CPU 的 $\overline{RD}$ 端相连。
- $\overline{WR}$：CPU 向 8251A 写数据的控制信号，低电平有效，与 CPU 的 $\overline{WR}$ 端相连。

$\overline{CS}$、C/$\overline{D}$、$\overline{RD}$和$\overline{WR}$信号相互配合可以决定CPU与8251A间的各种操作，如表11-1所示。

表11-1 8251A读/写功能表

$\overline{CS}$	C/$\overline{D}$	$\overline{RD}$	$\overline{WR}$	功 能
0	0	0	1	CPU从8251A读数据
0	1	0	1	CPU从8251A读状态
0	0	1	0	CPU向8251A读数据
0	1	1	0	CPU向8251A读命令
1	×	×	×	数据总线浮空

（3）与发送器相关的信号线

- T_xRDY：发送器准备好信号。T_xRDY=1发送缓冲器空；T_xRDY=0发送缓冲器满。当T_xRDY=1、T_xEN=1、CTS=0时，8251A已做好发送准备，CPU可以向8251A传输下一个数据。用查询方式时，CPU可从状态寄存器的D_0位检测这个信号，判断发送缓冲器所处状态。用中断方式时，此信号作为中断请求信号。
- T_xE：发送器空信号。有效时表示发送器中的并行到串行转换器空。在同步方式工作时，若CPU来不及输出一个新的字符，则它变高，同时发送器在输出线上插入同步字符，以填补传送空隙。
- T_xD：数据发送端，输出串行数据送往外部设备。
- $\overline{T_xC}$：发送时钟信号，外部输入。对于同步方式，$\overline{T_xC}$的时钟频率应等于发送数据的波特率。对于异步方式，由软件定义的发送时钟可以是发送波特率的1倍、16倍或64倍。

（4）与接收器相关的信号线

- R_xRDY：接收器准备好信号。R_xRDY=1表示接收缓冲器量装有输入的数据，通知CPU取走数据。若用查询方式，可从状态寄存器D_1位检测该信号。若用中断方式，可用该信号作为中断申请信号，通知CPU输入数据。R_xRDY=0表示输入缓冲器空。
- SYNDET/BRKDET：双功能检测信号，高电平有效。对于同步方式，SYNDET是同步检测信号。该信号既可工作在输入状态，也可工作在输出状态。同步工作时，该信号为输出信号。SYNDET=1表示8251A已经监测到所要求的同步字符。若为双同步，此信号在传送第二个同步字符的最后一位的中间变高，表明已经达到同步。外同步工作时，该信号为输入信号。当从SYNDET端输入一个高电平信号，接收控制电路会立即脱离对同步字符的搜索过程，开始接收数据。对于异步方式BRKDET为间断检出信号，用来表示R_xD端处于工作状态还是接收到断缺字符。BRKDET=1表示接收到对方发来的间断码。
- R_xD：数据接收端，接收由外设输入的串行数据。
- R_xC：接收时钟信号，输入。在同步方式时。R_xC等于波特率；在异步方式时，可以是波特率的1倍、16倍或64倍。

（5）与调制解调器有关的引脚

- $\overline{DTR}$：数据终端准备好信号，向调制解调器输出的低电平有效信号。CPU 准备好接收数据，$\overline{DTR}$ 有效，可由控制字中的 DTR 位置 l 输出该有效信号。
- $\overline{DSR}$：数据装置准备好信号，由调制解调器输出的低电平有效信号。当调制解调器已做好发送数据准备时，就发出 DSR 信号。CPU 可用 IN 指令读入 8251A 的状态寄存器，检测 $\overline{DSR}$ 位，当 $\overline{DSR}$ 位为 1 时，表示 $\overline{DSR}$ 信号有效。该信号实际上是对 $\overline{DSR}$ 信号的回答，通常用于接收数据。
- $\overline{RTS}$：请求发送信号，向调制解调器输出的低电平有效信号。CPU 准备好发送数据，由软件定义，使控制字中的 RTS 位置 1，输出 $\overline{RTS}$ 低电平有效信号。
- $\overline{CTS}$：准许发送信号，由调制解器输出的低电平有效信号是对 $\overline{RTS}$ 的回答信号。将控制字中的 T_xEN 位置 l，$\overline{CTS}$ 为低电平有效，发送器可串行发送数据。如果在数据发送过程中使 $\overline{CTS}$ 无效，或控制字中 T_xEN 位置 0，发送器将正在发送的字符结束后停止继续发送。

11.2.3　8251A 的编程控制

8251A 是可编程串行接口，在使用之前必须由程序对其工作状态进行设定（称为初始化），其中包括同步方式还是异步方式、传输波特率、字符代码位数、校验方式、停止位位数等。8251A 内部既有数据寄存器，还有控制字寄存器和状态寄存器。控制字寄存器用于 8251A 的方式控制和命令控制，状态寄存器则存放 8251A 的状态信息。

1．方式控制字

方式控制字用来确定 8251A 的通信方式（同步/异步）、校验方式（奇/偶校验、不校验）、数据位数（5、6、7 或 8 位）及波特率参数等。方式控制字的格式如图 11-7 所示。

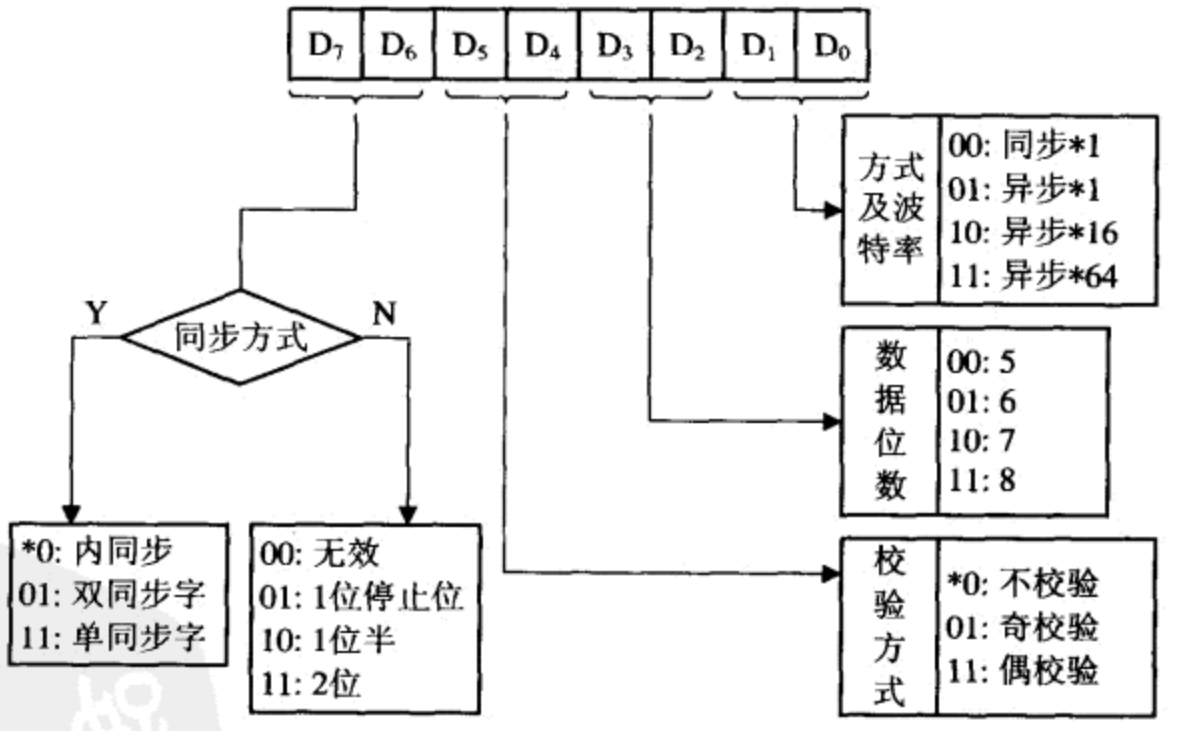


图 11-7　8251A 方式控制字的格式

2．命令控制字

命令控制字用于控制 8251A 的工作，使 8251A 处于规定的状态以准备发送或接收数据，应在写入方式控制字后写入，命令控制字的格式如图 11-8 所示。

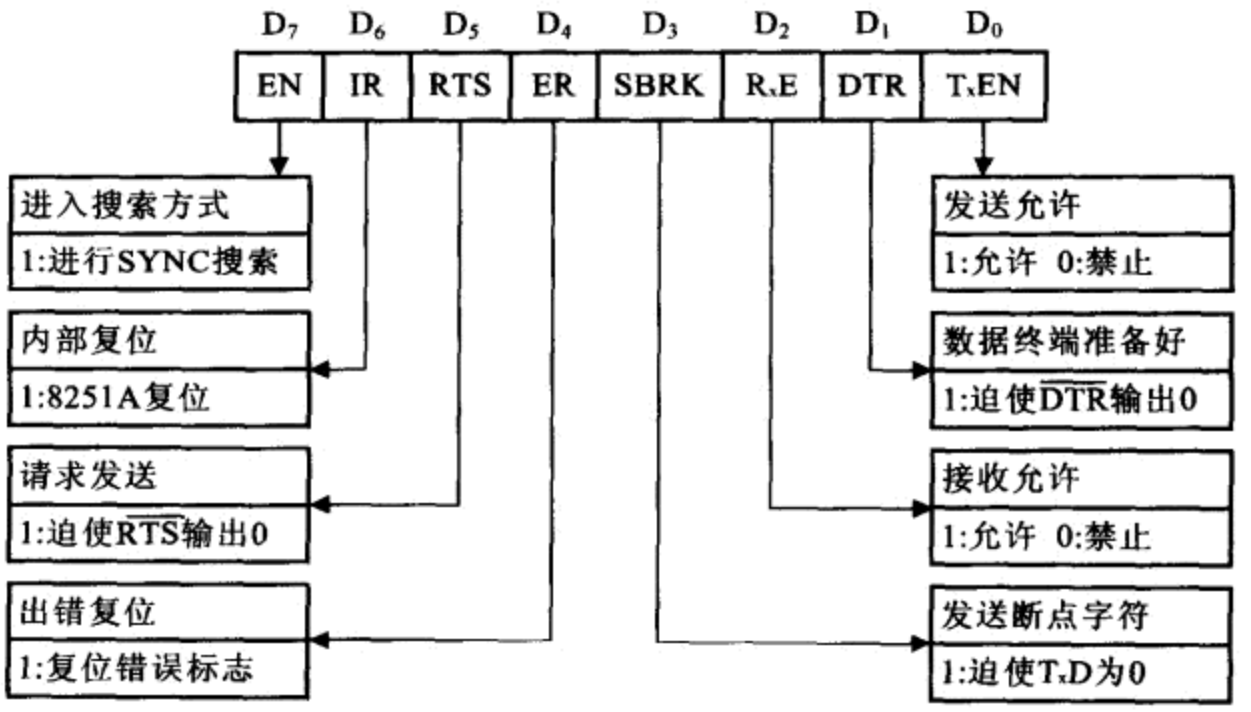


图 11-8 8251A 命令控制字的格式

方式控制字和命令控制字本身无特征标志，也没有独立的端口地址，8251A 是根据写入先后次序来区分这两者的：先写入者为方式控制字，后写入者为命令控制字。所以 CPU 在对 8251A 初始化编程时必须按一定的先后顺序写入方式控制字和命令控制字。

3. 状态控制字

CPU 通过输入指令读取状态控制字，了解 8251A 传送数据时所处的状态，作出是否发出命令，是否继续下一个数据传送的决定。状态字存放在状态寄存器中，CPU 只能读状态寄存器，而不能对它写入内容，状态字各位的意义如图 11-9 所示。

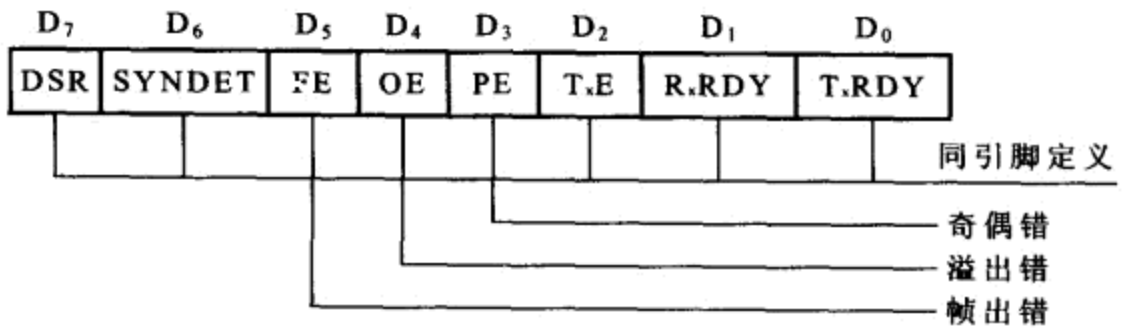


图 11-9 8251A 状态字的格式

帧出错 FE 标志只用于异步方式，当在任何一个字符的末尾没有检测到有效的停止位时，该标志位置 1，此标志位由命令指令字中的 ER 位清除；溢出错 OE 标志位由命令指令字中的 ER 位清除；奇偶校验错 PE 是当检测到奇偶错误时该标志位置 1，同样由命令指令中的 ER 位清除。这 3 种出错并不禁止 8251A 的工作。

11.2.4 8251A 的初始化和编程应用

1. 8251A 的初始化

传送数据前要对 8251A 进行初始化才能确定发送方与接收方的通信格式和通信的时序，从而保证准确无误地传送数据。由于 3 个控制字没有特征位，且工作方式控制字和操作命令控制字放入同一个端口，因而要求按一定顺序写入控制字，不能颠倒。正确顺序如

图 11-10 所示。

需要指出的是，工作方式控制字必须跟在复位命令之后。复位命令可用硬件的方法从 RESET 引脚输入复位信号，也可通过软件方法发送复位命令。这样 8251A 才可以重新设置工作方式控制字，改变工作方式完成其他传送任务。

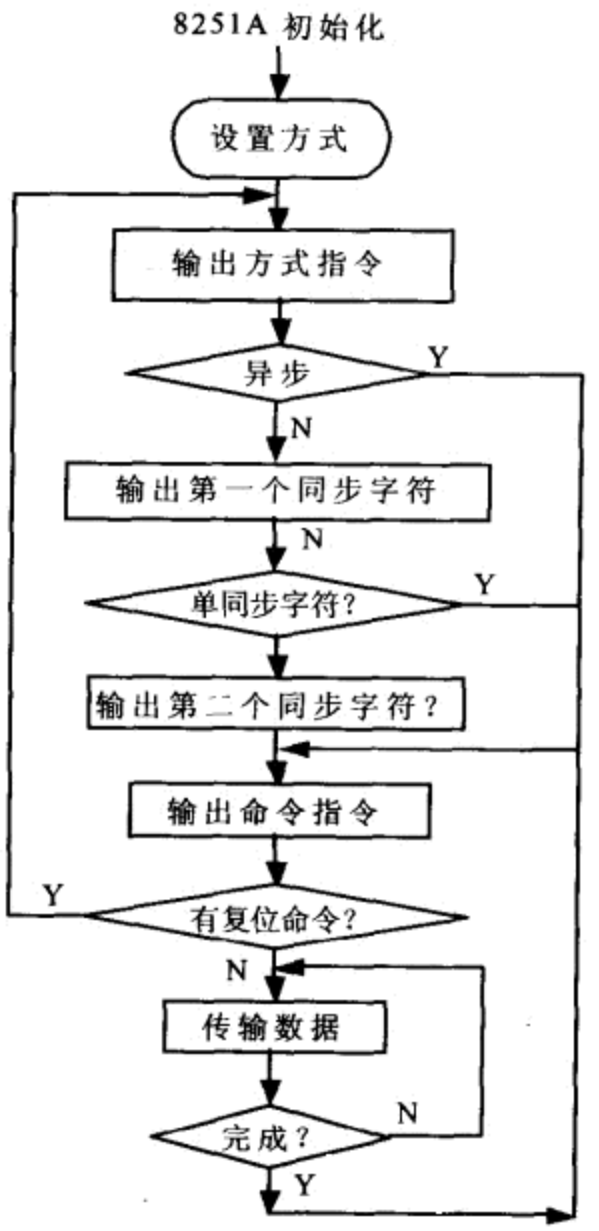


图 11-10　8251A 的初始化流程图

【例 11.1】编制一个通过 8251A 查询方式接收数据的程序。要求 8251A 定义为异步传送方式，波特率系数为 64，采用偶校验，1 位停止位，7 位数据位。

设 8251A 的数据端口地址为 04A0H，控制/状态寄存器端口地址为 04A2H。

程序段设计如下：

```
    MOV  DX,04A2H
    MOV  AL,7BH          ;写工作方式控制字
    OUT  DX,AL
    MOV  AL,14H          ;写操作命令控制字
    OUT  DX,AL
LP: IN   AL,DX           ;读状态控制字
    AND  AL,02H          ;检查 RxRDY 是否为 1
    JZ   LP
    MOV  DX,04A0H
    IN   AL,DX
```

2. 8251A 和 CPU 的通信方式

8251A 和 CPU 的通信方式可以采用查询方式和中断方式实现，这两种方式的应用分析如下。

（1）查询方式：该方式的应用特点是发送数据的程序在初始化程序之后。

【例 11.2】采用查询方式发送数据，假定要发送的字节数据放在 TABLE 开始的数据区，且要发送的字节数据放在 BX 中，数据端口地址为 04A0H，控制/状态寄存器端口地址为 04A2H。发送数据的程序段设计如下。

```
START:  MOV  DX,04A2H
        LEA  SI,TABLE
WAIT:   IN   AL,DX
        TEST AL,04AH                ;检查发送寄存器是否空
        JZ   WAIT                   ;若为空,则继续等待
        PUSH DX
        MOV  DX,04AOH
        LODSB
        OUT  DX,AL                  ;否则发送一个字节
        POP  DX
        DEC  BX
        MOV  DX,04A2H
        JNZ  WAIT
```

同样，在初始化程序后可以用查询方式类实现接收数据。

【例 11.3】下面是一个接收数据程序，假设接收后的数据送入 DATA 开始的数据存储区中。8251A 各寄存器地址安排同上。

```
RECV:   MOV  SI,OFFSET DATA
        MOV  DX,04A2H
WAIT:   IN   AL,DX                  ;读入状态寄存器
        TEST AL,38H                 ;检查是否有任何错误产生
        JNZ  ERROR                  ;有,转出错处理
        TEST AL,01H                 ;否则检查数据是否准备好
        JZ   WAIT                   ;未准备好,继续等待检测
        MOV  DX,04AOH
        IN   AL,DX                  ;否则接收一个字节
        AND  AL,7FH                 ;保留低 7 位
        MOV  [SI],AL                ;送数据缓冲区
        INC  SI
        MOV  DX,04A2H
        JMP  WAIT
```

（2）中断方式：利用中断方式可实现 8251A 和 CPU 的串行通信。

【例 11.4】设系统以查询方式发送数据，以中断方式接收数据。波特率系数为 16，1 位停止位，7 位数据位，奇校验。程序段设计如下。

```
MOV     DX,04A2H
MOV     AL,01011010B            ;写工作方式控制字
OUT     DX,AL
MOV     AL,14H                  ;写操作命令控制字
OUT     DX,AL
```

当完成对 8251A 的初始化后，接收端便可进行其他工作，接收到一个字符后便自动执行

中断服务程序。

【例 11.5】中断服务程序设计如下：

```
RECIVE: PUSH AX
        PUSH BX
        PUSH DX
        PUSH DS
        MOV  DX,04A2H
        IN   AL,DX
        MOV  AH,AL                    ;保存接收状态
        MOV  DX,04AOH
        IN   AL,DX                    ;读入接收到的数据
        AND  AL,7FH
        TEST AH,38H                   ;检查有无错误产生
        JZ   SAVAD
        MOV  AL,'?'                   ;出错的数据用'?'代替
SAVAD:  MOV  DX,SEG BUFFER
        MOV  DS,DX
        MOV  BX,OFFSET BUFFER
        MOV  [BX],AL                  ;存储数据
        MOV  AL,20H
        OUT  20H,AL                   ;将 EOI 命令发给中断控制器 8259
        POP  DS
        POP  DX
        POP  BX
        POP  AX
        STI
        IRET
```

当 8251A 的接收数据寄存器满而产生中断时，此中断请求经过中断控制器 8259A 送给 CPU。CPU 响应中断后，转向上述中断服务程序。该中断服务程序首先进行现场保护，接着接收状态寄存器中的内容和数据，并检查有无错误。若有错则进行错误处理，无错将接收到的数据送到数据区中，然后恢复断点，开中断并返回。注意在中断服务程序结束前，必须给 8259A 一个中断结束命令 EOI，使 8259A 能将中断服务寄存器的状态复位，使系统又能处理其他低级别的中断。

3. 8251A 编程应用举例

【例 11.6】通过串行总线将两台计算机的串行接口连接，两台 PC（设编号为 A 和 B）可实现互送数据。这里采用查询方式进行通信。其中 A 机器首先向 B 机器发送数据，B 机器接收数据并显示；接着 B 机器向 A 机器发送数据，A 机器接收并显示。

A 机器程序设计如下：

```
DATA    SEGMENT
DA      DB '123456789A'
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,ES:DATA,DS:DATA
        MOV     AX,DATA
        MOV     DS,AX
        MOV     ES,AX                 ;程序初始化
CF:     MOV     DX,3FBH
```

```
        MOV     AL,80H                  ;将 AL 置 1
        OUT     DX,AL
        MOV     DX,3F8H
        MOV     AL,0CH                  ;送除数低 8 位
        OUT     DX,AL
        MOV     DX,3F9H
        MOV     AL,0                    ;送除数高 8 位
        OUT     DX,AL
        MOV     DX,3FBH
        MOV     AL,03H
        OUT     DX,AL                   ;初始化通信控制字寄存器
        MOV     BX,OFFSET DA            ;BX 指向发送数据区首单元
        MOV     CX,0AH                  ;发送次数送 CX
S:      MOV     DX,3FDH
        IN      AL,DX                   ;读入状态寄存器值
        TEST    AL,01                   ;接收数据缓冲寄存器是否准备好
        JZ      S                       ;未准备好,转 S 继续等待
        MOV     DX,3F8H
        IN      AL,DX                   ;准备好读入一个字符数据并显示
        MOV     DL,AL
        MOV     AH,02
        INT     21H
F:      MOV     DX,3FDH
        IN      AL,DX                   ;读入状态寄存器值
        TEST    AL,20H                  ;发送数据寄存器是否为空
        JZ      F                       ;非空,转 F 继续等待
        MOV     DX,3F8H
        MOV     AL,[BX]                 ;否则发送一个字符数据
OUT     DX,AL
        LOOP    S
        MOV     AH,4CH
        INT     21H
        CODE    ENDS
        END
```

B 机器的程序结构和 A 机器相同，只要将发送和接收两个程序段对调即可，此处略。

11.3 PC 串行异步通信接口

11.3.1 串行异步通信的基本概念

1. 概述

使用最早、应用最广泛的一种串行异步通信总线是 RS-232C，它是美国电子工业协会（Electronic Industries Association，EIA）1962 年公布的，1969 年最后一次修订而成，其中 RS 是 Recommended Standard 的缩写，表示推荐的意思，232 是该标准的标识，C 表示最后一次修订。RS-232C 主要用来定义计算机系统的一些数据终端设备（DTE），如计算机和数据终端，与数据通信设备（DCE），如调制解调器二者之间接口的电气特性。实际应用中 CRT、键盘、扫描仪等与 CPU 的通信大都采用 RS-232C 总线。

PC 系列为了能实现串行异步通信功能而设置一个符合 RS-232C 接口标准的串行接口，例如图 11-11 所示的 INS 8250。在不同机型上，串行接口的硬件定位是有区别的。

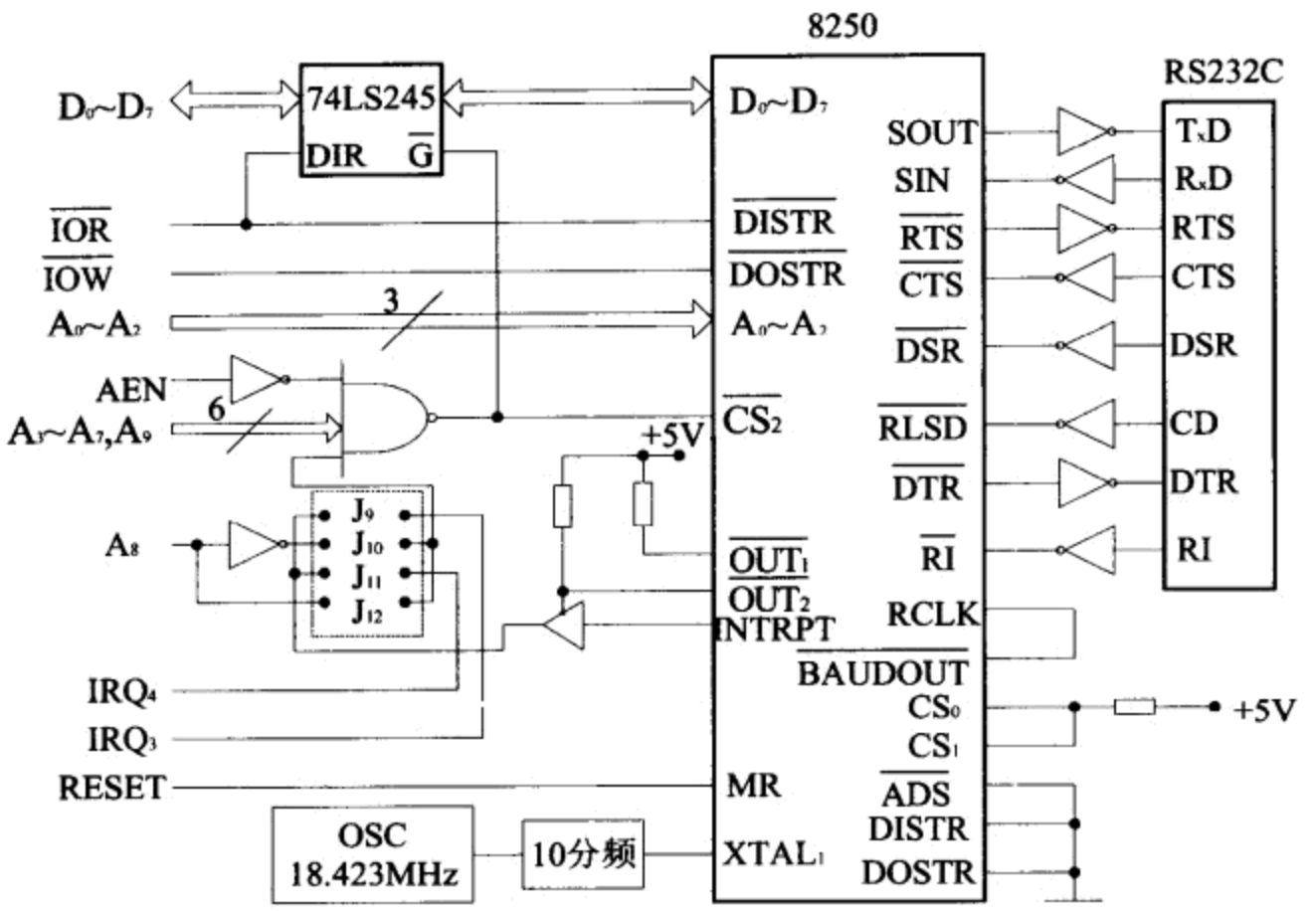
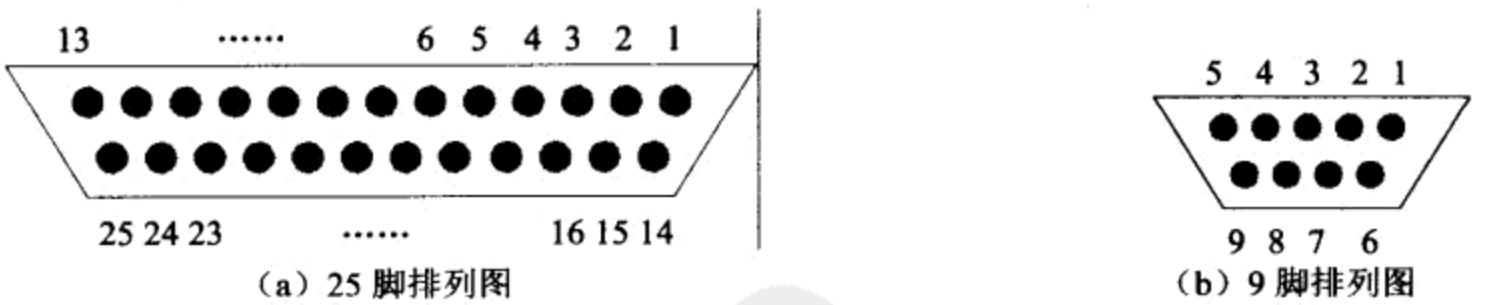


图 11-11　串行异步适配器结构图

2. RS-232 串行通信总线标准

RS-232 是一种串行通信总线标准，当初制订此标准的目的是为了使不同厂家生产的设备能达到插接的“兼容性”。这个标准仅保证硬件兼容而没有软件兼容。此外，用它进行数据传输时，由于线路的损耗和噪声干扰，传输距离一般都不超过 15m。通常两计算机的近距离通信可以通过 RS-232C 接口连接起来。

25 脚、9 脚 RS-232 标准接口引脚排列如图 11-12 所示，各引脚功能如表 11-2 所示。



（a）25 脚排列图　　（b）9 脚排列图

图 11-12　RS-232 引脚排列图（插座）

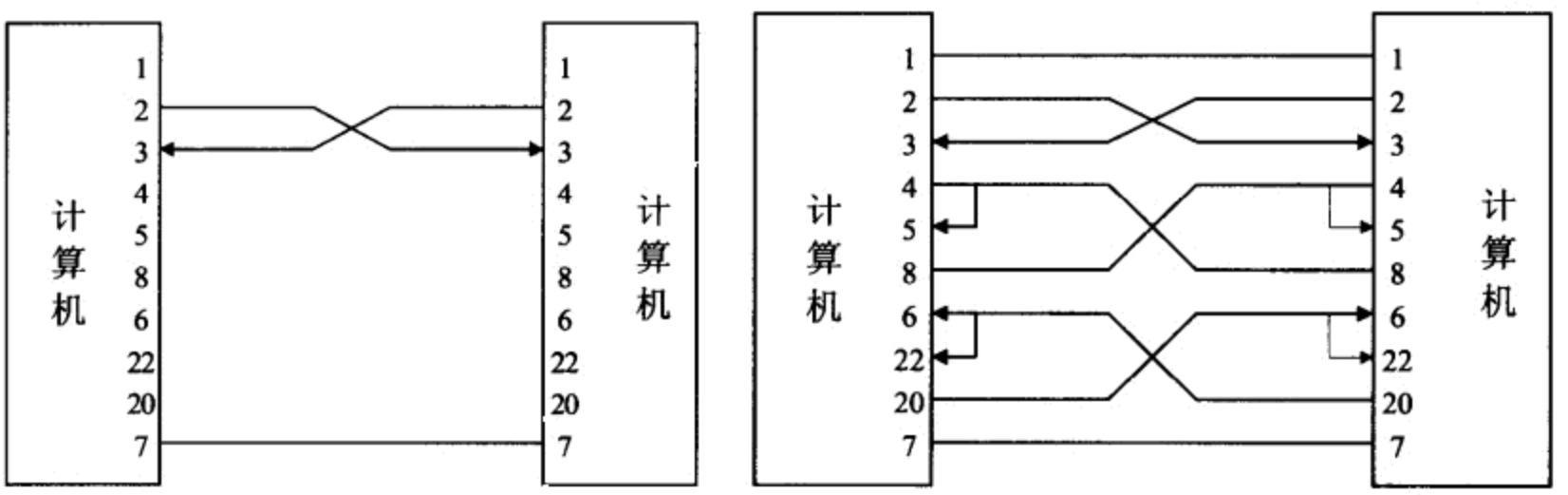
表 11-2　RS232 引脚功能

25 脚引脚号（9 脚）	符　　号	方　　向	功　　能
2（3）	T_XD	输出	发送数据
3（2）	R_XD	输入	接收数据
4（7）	RTS	输出	请求发送
5（8）	CTS	输入	清除发送

续上表

25 脚引脚号（9 脚）	符　　号	方　　向	功　　能
6（6）	DSR	输入	数据通信设备准备好
7（5）	GND		信号地
8（1）	DCD	输入	数据载波检测
20（4）	DTR	输出	数据终端准备好
22（9）	RI	输入	振铃指示

通过 PC 的串行接口，可以连接串行传输数据的外围设备，如调制解调器、鼠标等。同样串口还可以连接两台计算机，两台计算机的连接方法如图 11-13 所示。



（a）两台计算机直接接口　　（b）两台计算机简化连接

图 11-13　通过 RS-232C 两台计算机间的连接方法

11.3.2　BIOS 串行异步通信接口的功能调用

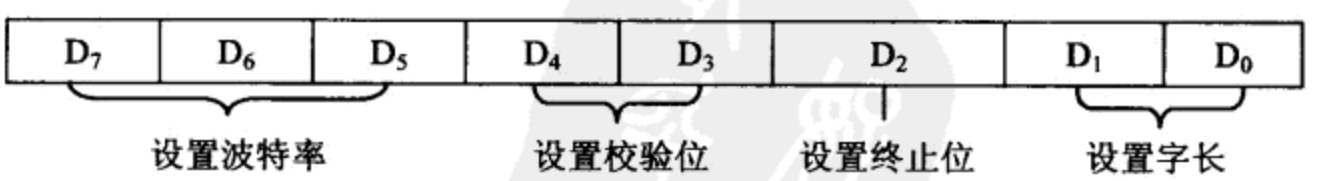
1. 功能调用原理

IBM PC 的兼容机有比较灵活的串行口 I/O 方法，即通过 INT 14H 调用 ROM BIOS 串行通信口例行程序。串行异步通信接口功能调用的具体情况分析如下。

（1）初始化串行通信口（AH=0）

入口参数：AL=初始化参数，DX=通信口号；出口参数：AH=通信口状态，AL=调制解调器状态。

初始化参数据可以用来设置串行口的波特率、奇偶性、字长和终止位。8 位数据具体含义如下：



- 位 7、6、5 用于设置波特率，3 位的代码组合功能分别为：000=110 波特；001=150 波特；010=300 波特；011=600 波特；100=1 200 波特；101=2 400 波特；110=4 800 波特；111=9 600 波特。
- 位 4、3 用于设置校验位，2 位的代码组合功能分别为：01=奇校验，11=偶校验，×0=无校验。
- 位 2 用于设置终止位，：0=1 位；1=2 位。
- 位 1、0 用于设置字长，2 位的代码组合功能分别为：10=7 位；11=8 位。

例如，指令序列：

```
MOV   AH，0
MOV   AL，0A3H
MOV   DX，0
INT    14H
```

完成的功能是将 0 号通信口的波特率设置为 2 400 波特，字长为 8 位，1 位终止位，无奇偶校验位。

（2）向串行通信口写字符（AH=1）

入口参数：AL=所写字符，DX=通信口号。出口参数：写字符成功 AH=0，AL=字符；写字符失败$(AH)_7$=1，$(AH)_{0\sim6}$=通信口状态。

（3）从串行通信口读字符（AH=2）

入口参数：DX=通信口号。出口参数：读字符成功$(AH)_7$=0，AL=字符；读字符失败$(AH)_7$=1，$(AH)_{0\sim6}$=通信口状态。

（4）取串行通信口状态（AH=3）

入口参数：DX=通信口号；出口参数：AH=通信口状态，AL=调制解调器状态。

该功能用来读串行口的当前状态，调用时只需给出被查询的串行通信口是 COM1 或是 COM2，调用返回时其状态信息放入 AX 中，（AL）=MODEM 状态，（AH）=线路状态。

AL 中 MODEM 状态位的含义如下：

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
载波检测	振铃指示	数据设备就绪	清除发送	DCD 状态变化	RI 由接通到断开	DSR 状态变化	CTS 状态变化

AH 中线路状态位的含义如下：

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
超时错	发送移位寄存器空	发送缓冲寄存器空	间断条件	帧格式错	奇偶校验错	重叠错	接收数据准备好

2. 功能调用的应用实例设计

通过前面的介绍，已经知道串行通信的编程方式有 I/O 指令方式、DOS 功能调用方式和 BIOS 中断调用方式，在这里选择 BIOS 中断调用方式来进行实例设计。编程的关键是确定串行通信的基本方式。

【例 11.7】在下面的程序中设置串行通信的传输参数为：1 200 波特、7 个数据位，1 个

奇偶校验位，2 个停止位。两台计算机可互发数据，即在程序开始首先检测是否有数据要接收，若没有则检测是否有键按下，若有数据就发送，否则重新检测。

程序设计如下：

```
DATA  SEGMENT
SHOWMESS DB   '                欢迎使用本程序!                    ',0DH,0AH
         DB   '        在这里您可以实现两台电脑之间的通信!        ',0DH,0AH
         DB   '*******************************************',0DH,0AH
         DB   '*                  帮助文件                *',0DH,0AH
         DB   '*          按下 "Enter" 键可以回车          *',0DH,0AH
         DB   '*          按下  "ESC"  键可以退出本程序    *',0DH,0AH
         DB   '*          ----------VAR 1.0----------     *',0AH,0AH
         DB   '*******************************************',0DH,0AH
         DB   '            现在您可以进入本程序了          ',0DH,0AH,0AH
         DB   '            Are You Ready?      $'
DATA  ENDS
CODE  SEGMENT
      ASSUME  CS:CODE,DS:DATA
START:MOV    AX,DATA
      MOV    DS,AX                  ;数据段基值装入(通过 AX)
      LEA    DX,SHOWMESS            ;字符串有效地址装入 DX 寄存器
      MOV    AH,09H
      INT    21H                    ;DOS 显示字符串功能调用
      ;初始化串口 COM2
      ;串行口初始化为 1200 波特、数据位
      MOV    AH,0
      MOV    DX,1
      MOV    AL,8EH
      INT    14H                    ;初始化 COM2 口
FORE: MOV    AH,03H
      MOV    DX,1
      INT    14H                    ;读串口 2 状态字
      TEST   AH,01H                 ;数据是否准备好
      JNZ    RECE                   ;准备好转接收程序
      TEST   AH,20H                 ;发送移位寄存是否为空
      JZ     FORE                   ;不为空转 FORE 继续检测
      MOV    AH,1
      INT    16H                    ;否则利用 BIOS 键盘中断调用,输入字符
      JZ     FORE                   ;没有按键,继续
      MOV    AH,0
      INT    16H                    ;BIOS 键盘输入
      CMP    AL,1BH
      JZ     QUIT                   ;是否是 ESC 键,"是"则退出
      MOV    AH,1
      MOV    DX,1
      INT    14H                    ;否则发送字符
      CMP    AL,0DH
```

```
        JNZ     RECE
        MOV     AH,02H
        MOV     DL,0AH
        INT     21H             ;若发送的为回车符,显示换行
        MOV     DL,0DH
        INT     21H
RECE:   MOV     AH,3
        MOV     DX,1
        INT     14H             ;读串口 2 状态字
        TEST    AH,01H          ;数据是否准备好
        JZ      FORE            ;数据未准备好转 FORE 继续检测
        MOV     AH,2
        MOV     DX,1
        INT     14H             ;否则读入字符
        MOV     DL,AL
        AND     DL,7FH
        MOV     AH,02H          ;屏蔽校验位
        INT     21H             ;DOS 中断显示字符
        JMP     FORE            ;接收方发送字符
QUIT:   MOV     AH,4CH          ;退出程序,返回 DOS
        INT     21H
CODE    ENDS                    ;代码段结束
        END     START           ;程序结束
```

本章小结

可编程串行接口芯片 8251A 能够为 CPU 提供并/串行转换功能，同时为外设提供串/并行转换功能。8251A 的内部有可编程寄存器，要采用片选信号、读/写控制信号进行译码。

8251A 进行初始化时要设置传输波特率、停止位位数、校验位、数据位位数以及是否允许中断等，8251 和 CPU 通信的方式主要有查询方式和中断方式。

RS-232C 是美国电子工业协会（Electronic Industries Association，EIA）异步串行通信总线标准。RS-232C 主要用来定义计算机系统的数据终端设备、数据通信设备等，CRT、键盘、扫描仪等与 CPU 的通信大都采用 RS-232C 总线。

思考与练习题

1. 在串行通信中有哪几种数据传送模式，各有什么特点？
2. 串行通信按信号格式分为哪两种，这两种格式有何不同？
3. 试说明 8251A 的工作方式控制字、操作命令控制字和状态控制字各位的含义及它们之间的关系。在对 8251A 进行初始化编程时，应按什么顺序向它的控制口写入控制字？
4. 若 8251A 以 9 600 波特的速率发送数据，波特率因子为 16，发送时钟 $\overline{T.C}$ 频率为多少？
5. 8251A 的 SYNDET/BRKDET 引脚有哪些功能？
6. 什么叫波特率？什么叫波特率因子？常用的波特率有哪些？

7. 某系统中使可编程串行接口芯片 8251A 工作在异步方式，7 位数据位，偶校验，2 位停止位，分频系数为 96，允许发送也允许接收，若已知其控制口地址为 3FBHH，试编写初始化程序。
8. 设 8251A 的控制口和状态口地址为 03FBH，数据输入/输出口地址为 03F8H，输入 100 个字符，并将字符放在 BUFFER 所指的内存缓冲区中。请写出这段的程序。
9. 简述异步串行的概念，说明 RS-232C 的工作原理和应用。
10. BIOS 串行异步通信接口的功能调用主要有哪些？

第 12 章 可编程定时器/计数器接口芯片 8253

本章主要讲解定时器/计数器的基本原理及可编程定时器/计数器芯片 8253 的基本结构和编程方法，主要包括 8253 的 6 种工作方式及各自的特点，8253 的编程方法，定时器/计数器的综合应用。

通过本章的学习，读者应该理解定时器/计数器的基本概念，掌握定时器/计数器芯片 8253 的基本结构、工作方式、编程结构和使用方法。

12.1 定时器/计数器的工作原理

12.1.1 微机系统中的定时

定时的本质就是计数，只不过这里的“数”的单位是时间单位。如果将计时单位累加起来，就可以获得一段时间。例如，以秒为单位来计数，计满 60 秒为 1 分，计满 60 分为 1 小时，计满 24 小时即为 1 天。

微型计算机经常要用来对外部事件进行定时控制或记录外部事件产生次数，即定时或计数控制。如函数发生器、计算机中的系统日历时钟、DRAM 的定时刷新、定时采样和控制系统等都要用到定时信号。

微机系统中的定时可以分为内部定时和外部定时两类。内部定时是计算机本身运行的时间基准或时序关系，计算机每个操作都是按照严格的时间节拍执行的。计算机内部定时由 CPU 硬件结构决定，是固定的时序关系，无法更改；外部定时是外部设备实现某功能时本身所需要的一种时序关系。由于外设和被控对象的任务不同，功能各异，无一定模式，因此往往需要用户根据 I/O 设备的要求进行安排。当然用户在考虑外设和 CPU 的连接时，不能脱离计算机的定时要求，即应以计算机的时序关系为依据，来设计外部定时机构，以满足计算机的时序要求，这叫做时序配合。

12.1.2 外部定时法

为了获得所需的定时，要求有准确而稳定的时间基准。产生这种时间基准通常采用两种方法：软件定时方法和硬件定时方法。

1. 软件定时方法

软件定时方法是编写一段具有循环功能的程序，在循环体内执行每一条指令都有固定的时钟周期数，只要统计出循环体内所有指令一共花费的时钟周期数，再乘以一个时钟周期的时间，便得出执行一次循环所需的总时间。显然，设计不同的循环次数，便可得到不同的延迟时间。这种定时方法很准确，但要占用 CPU 的工作时间去延时等待，一般用在延时时间不长，且使用次数不多的场合。如果要统计外部事件发生的次数，那么用一个寄存器记录事件发生的个数就可以了，程序很简单，但也要用等待的方法来等待事件的发生。

2. 硬件定时方法

硬件定时可分两种情况：一种是不可编程硬件定时，由定时器件来实现，如 555 定时器。也可利用计数器直接对系统时钟脉冲计数。这种方式不占用 CPU 的时间，而且很容易产生确定宽度的单脉冲或固定频率的连续脉冲，一旦硬件电路确定后，定时特性不易改变。

另外一种就是利用专用的硬件定时器/计数器，在简单软件的控制下产生准确的延时时间。其基本原理是通过软件确定定时器/计数器的工作方式，设置计数初值并启动计数器工作，当计数到给定值时，便自动产生定时信号。这种方法的成本不高，程序上也很简单，且大大提高了 CPU 的效率，既适合长时间、多次重复的定时，也可用于延时时间较短的场合，因此得到了广泛的应用。

12.1.3 可编程定时器/计数器的工作原理

定时器/计数器在计数方法上分为加法计数器和减法计数器。加法计数器是每有一个脉冲就加 1，当加到预先设定的计数值时，产生一个定时信号。减法计数器是在送入计数初值后，每送来一个脉冲，计数器就减 1，减到 0 时产生一个定时信号输出。如果用做定时器，在计数到满值或至 0 后，重置初始值自动开始新的计数过程，从而获得连续的脉冲输出。

可编程定时器 8253 是一个减法计数器，它是 Intel 公司专门为 X86 系列 CPU 配置的外围接口芯片。本章将由 8253 的内部结构和外部引脚特性出发介绍 8253 的内部特性和外部特性，以及 8253 与系统的连接和编程方法。

12.2 可编程定时器/计数器 8253

12.2.1 8253 的特点

Intel 系列可编程定时器/计数器 8253 有几种芯片型号，外型引脚及功能都是兼容的，只是工作的最高计数速率有所差异，例如 8253（2.6MHz），8253-5（5MHz），8254（8MHz），8254-5（5MHz），8254-2（10MHz）。

可编程计数器/定时器 8253 是用软、硬技术相结合的方法实现定时和计数控制，其特点主要有以下几个方面。

（1）有 3 个独立的 16 位计数器，每个计数器均以减法计数。

（2）每个计数器都可按二进制计数或十进制（BCD 码）计数。

（3）每个计数器都可由程序设置 6 种工作方式。

（4）每个计数器计数速度可达 2.6MHz。

（5）所有 I/O 都可与 TTL 兼容。

12.2.2 8253 内部结构和引脚功能

1. 内部结构

8253 内部包含 3 个 16 位计数器，每个计数器可按二进制或十进制计数，有 6 种工作方式，可通过编程选择。8253 采用单一+5V 电源供电，NMOS 工艺制造，24 引脚 DIP 封装，

其内部结构如图 12-1 所示。

（1）数据总线缓冲器：数据总线缓冲器是 8 位、双向、三态的缓冲器，通过 8 根数据线 D_7～D_0，接收 CPU 向控制寄存器写入的控制字，或 CPU 向计数器写入的计数初值。也可把计数器的当前计数值读入 CPU 中。

（2）读/写控制逻辑电路：读/写控制逻辑电路从系统总线接收输入信号，经过译码产生对 8253 各部分的控制。和读/写控制逻辑相关的引脚为地址线 A_1、A_0，片选线 CS、读有效信号线 $\overline{RD}$ 和写有效信号 WR。

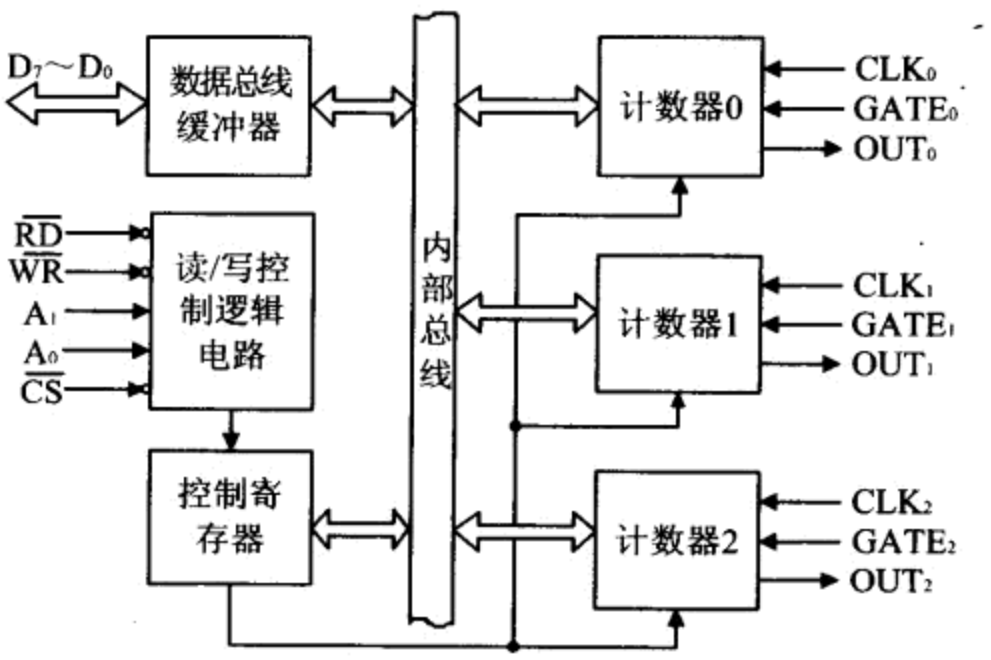


图 12-1　8253 的内部结构

（3）控制字寄存器：用户对 8253 的控制是通过控制字寄存器实现的。它接收来自 CPU 的方式控制字，控制相应计数器的工作方式，这一项工作要在 8253 的初始化过程中完成。它只能写入不能读出。

（4）计数通道：8253 有 3 个相互独立的同样的计数电路，分别称做计数器 0、计数器 1 和计数器 2。每个计数器包含 1 个 8 位的控制寄存器，它存放计数器的工作模式控制字；1 个 16 位的初值寄存器 CR，8253 工作之前要对它设置初值；1 个 16 位计数执行单元 CE，它接收计数初值寄存器 CR 送来的内容，并对该内容执行减 1 操作；1 个 16 位输出锁存器 OL，它锁存 CE 的内容，使 CPU 能从输出锁存器内读出一个稳定的计数值。计数器的功能结构如图 12-2 所示。

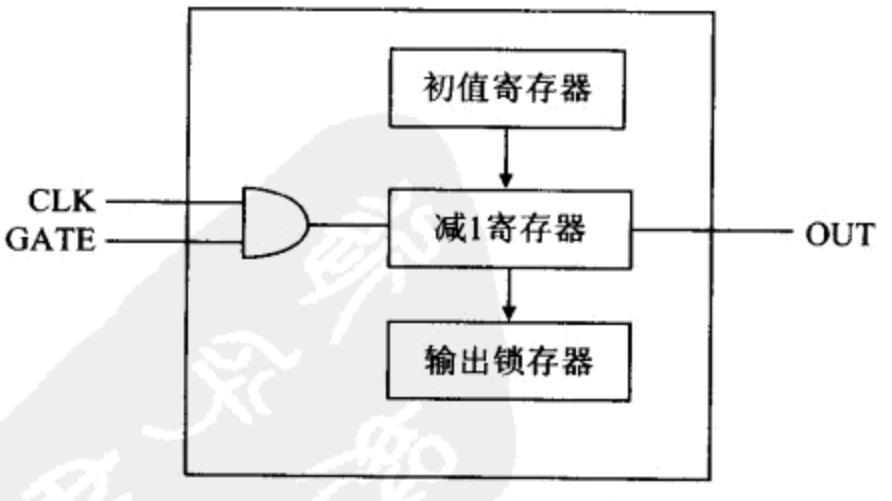


图 12-2　计数器的内部结构

2. 8253 的引脚功能

8253 采用双列直插封装，24 条引脚，各引脚排列如图 12-3 所示。引脚功能分析如下：

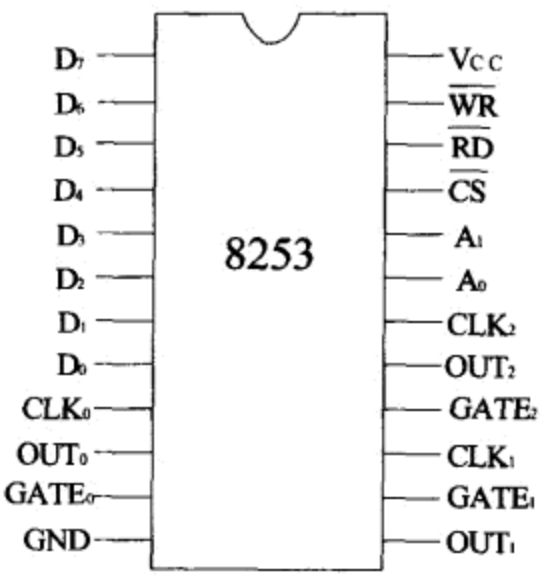


图 12-3　8253 的引脚排列图

- D_7～D_0：8 位双向数据线。传送数据、控制字和计数器的计数初值。D_7～D_0 和数据总线的低 8 位相连。
- $\overline{CS}$：片选信号，输入，低电平有效。由系统送来的高位 I/O 地址译码产生，有效时定时芯片被选中。
- $\overline{RD}$：读有效信号，输入，低电平有效。有效时表示 CPU 要对此芯片进行读操作。该信号和系统总线中的 $\overline{RD}$（最小方式）或 $\overline{IOR}$（最大方式）相连。
- $\overline{WR}$：写有效信号，输入，低电平有效。有效时表示 CPU 要对此芯片进行写操作。该信号和系统总线中的 $\overline{WR}$（最小方式）或 $\overline{IOW}$（最大方式）相连。
- A_1、A_0：地址信号线。高位地址信号经译码产生 CS 片选择信号，决定了 8253 芯片所具有的地址范围。A_0 和 A_1 地址信号经片内译码产生 4 个有效地址，分别对应芯片内部的 3 个独立的计数器和一个控制寄存器。

片选信号、读/写有效信号和地址信号共同控制对 8253 内部的寄存器的读、写操作。具体内容如表 12-1 所示。

表 12-1　8253 内部端口地址和操作

$\overline{CS}$	A_1	A_0	$\overline{RD}$	$\overline{WR}$	功　能
0	0	0	1	0	写计数器 0
0	0	1	1	0	写计数器 1
0	1	0	1	0	写计数器 2
0	1	1	1	0	写方式控制字
0	0	0	0	1	读计数器 0
0	0	1	0	1	读计数器 1
0	1	0	0	1	读计数器 2
0	1	1	0	1	无效

- CLK_0～CLK_2：计数器的时钟信号输入端。8253 各计数器对该脉冲信号进行计数，其大小直接影响计数的速率。CLK 信号是计数器工作的计时标准，因此要求其频率很精确。
- $GATE_0$～$GATE_2$：门控信号，用于控制计数器的启动和停止。多数情况下，GATE=1 时允许计数，GATE=0 时停止计数。但有的方式下用 GATE 的上升沿启动计数，启动后则 GATE 的状态不再影响计数过程。具体控制作用在工作方式一节中详细介绍。
- OUT_0～OUT_2：计数器输出信号。当计数结束时，会在 OUT 端产生输出信号，不同的方式，会有不同的波形输出。

12.2.3　8253 的初始化

1. 写控制字

8253 在工作之前，必须对它进行编程，以确定每个计数器的工作方式和对计数器赋计数初值。CPU 通过写控制字指令，将每个计数通道分别初始化，使之工作在某种工作方式之下。

对 8253 芯片的初始化编程包括写入控制字和写入计数值两方面，任一通道的控制字要从 8253 的控制口地址写入，控制哪个通道由控制字的 D_7D_6 位来决定。计数初始值经由各通道的端口地址写入，可以采用二进制或十进制来计数。8253 的控制字格式如图 12-4 所示。

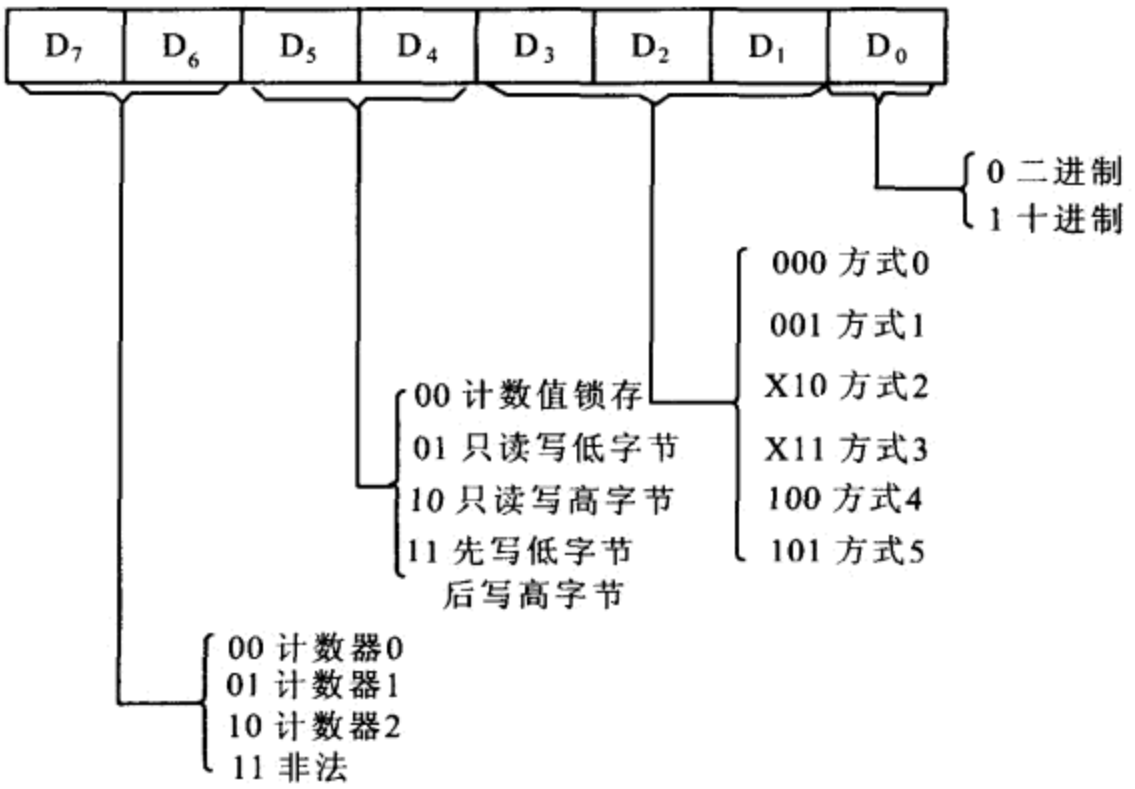


图 12-4　8253 的控制字格式

2. 写计数初值

8253 编程时首先向控制字寄存器写入控制字，以选择计数器的工作方式，然后对相应的计数器输入计数值。在计数值送到计数寄存器后，需经一个时钟周期才能把此计数值送到递减计数器。

当控制字 D_0=0 时，即二进制计数，初值可在 0～FFFFH 之间选择；当 D_0=1 时，则为十进制计数，其值可在 0～9 999 十进制数之间选择。但无论何种计数方式，当计数初值为 0000H 时，计数器的计数值最大。即二进制计数时，计数值最小为 1，最大为 65 536；十进制计数时，计数值最小为 1，最大为 10 000。

计数初值的选择和定时长短以及时钟频率有关。

【例 12.1】在某系统中 8253 的 CLK_0～CLK_2 的时钟频率为 2MHz，端口地址为 200H～203H。要求计数器 0 工作在方式 0 下，十进制计数，定时 100μs 后产生中断请求；计数器 1 工作在方式 3 下，二进制计数，用于产生周期为 10μs 的对称方波；计数器 2 工作在方式 2 下，二进制计数，每隔 1ms 产生一个负脉冲。

根据上述要求和已知条件，可知 CLK 的时钟周期为 0.5μs，则可计算出计数器 0 的初始值为 100μs/0.5μs=200，计数器 1 的计数初始值为 10μs/0.5μs=20，计数器 2 的计数初始值为 1ms/0.5μs=2000。

初始化程序如下：

```
MOV  DX,203H                    ;控制字地址送 DX
MOV  AL,00110001B               ;控制字内容送 AL
OUT  DX,AL                      ;写控制字
MOV  AX,0200H                   ;计数初始值送 AX
MOV  DX,200H                    ;计数器 0 地址 200H 送 DX
OUT  DX,AL                      ;写低 8 位
MOV  AH,AL
OUT  DX,AL                      ;写高 8 位
MOV  DX,203H                    ;控制字地址送 DX
MOV  AL,01010110B               ;控制字内容送 AL
OUT  DX,AL                      ;写控制字
MOV  DX,201H                    ;计数器 1 地址 201H 送 DX
MOV  AL,20                      ;计数初始值送 AX
OUT  DX,AL                      ;写低 8 位
MOV  DX,203H                    ;控制字地址送 DX
MOV  AL,10110100B               ;控制字内容送 AL
OUT  DX,AL                      ;写控制字
MOV  AX,2000                    ;计数初始值送 AX
MOV  DX,202H                    ;计数器 2 地址 202H 送 DX
OUT  DX,AL                      ;写低 8 位
MOV  AL,AH
OUT  DX,AL                      ;写高 8 位
```

若计数器 2 工作在十进制计数方式，则应把 2000H 送 AX 中。计数器 2 的初始化程序段可改为下列指令序列：

```
MOV  DX,203H
MOV  AL,10100101B
OUT  DX,AL
MOV  DX,202H
MOV  AL,20H
OUT  DX,AL
```

3. 8253 的读操作

所谓读操作是指读出某计数器的计数值至 CPU 中，有以下两种读数方法。

（1）直接读操作：由于 8253 处于计数状态时，输出锁存器（OL）的内容是随着减 1 计数器（CE）内容而变化的，故读 CE 的内容就是读 OL 的值。当采用这种读操作时，应暂停计数过程，这可用门控制信号 GATE 暂停计数或采用外部逻辑电路暂停时钟 CLK 的输入，以保证读出数据的稳定性。计数器停止计数后，再根据控制字中 D_4D_3 位的 RL1 和 RL0 状态，直接用一条或两条输入（IN）指令读出 OL（即在 CE）中的当前计数值。

（2）锁存后读计数值：该方法允许在计数过程中读出计数值，同时不影响 CE 的计数操作。为了实现该方式，首先需要向 8253 计数器发出一个锁存命令字，当 8253 计数器接收到这个锁存命令后，输出锁存器 OL 中的计数器就被锁存，不再随 CE 计数器变化而变化了。

【例 12.2】设 8253 的端口地址为 40H～43H，要求检查计数器 2 中的值是否为 1000H，若不是，则继续等待，否则顺序执行程序。程序段设计如下：

```
LP: MOV  AL,10000100B           ;控制字送 AL 中
    OUT  43H,AL                 ;写控制字
    IN   AL,42H                 ;读计数器 2 低 8 位当前值
```

```
MOV  AH,AL          ;暂存到 AH 中
IN   AL,42H         ;读计数器 2 高 8 位当前值到 AL
XCHG AL,AH          ;16 位计数值送至 AX 中
CMP  AX,1000H       ;比较
JNZ  LP             ;不等转 LP 继续
...                 ;否则向下执行
```

12.2.4　8253 的工作方式

8253 有 6 种不同的工作方式，在不同的工作方式下，计数过程的启动方式、OUT 端的输出波形都不一样。自动重复功能和 GATE 的控制作用及写入新的计数初值对计数器的工作过程产生的影响不一样。下面借助波形来分别说明这 6 种工作方式的计数过程。

1. 方式 0——计数结束，产生中断

当一个计数通道通过控制字设置为方式 0 后，其输出端信号 OUT 随即变为低电平。当用 OUT 指令将计数初值装入该计数器后，计数器开始计数，输出仍为低电平。以后 CLK 引脚上每输入一个时钟信号（下降沿），计数器中的内容减 1。计数值减为 0 时计数结束，输出变为高电平，并且一直保持到该通道重新装入计数值或重新设置工作方式为止。方式 0 的时序图如图 12-5（a）所示。

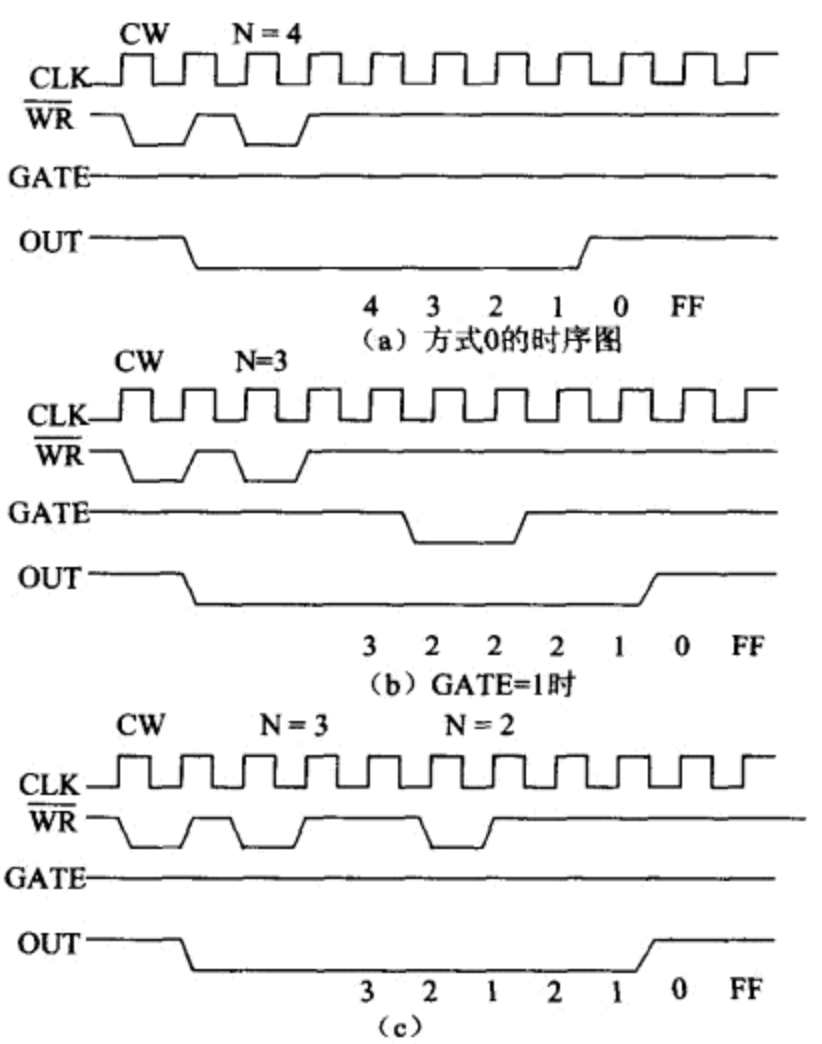


图 12-5　方式 0 波形图

方式 0 有以下特点：

（1）计数过程可由 GATE 信号控制。GATE=0 时停止计数操作；GATE=1 时计数器继续计数。GATE 信号电平的变化不影响 OUT 端的输出，如图 12-5（b）所示。

（2）设置初始值时 GATE=0，下一个 CLK 脉冲也将使初始值从 CR 装入 CE，但不启动

计数工作，GATE=1 时开始计数，计数完毕，OUT 端输出变为高电平。

（3）计数通道如果置入的初始值为 N，则从置入时算起要经过 N+1 个 CLK 脉冲，计数值才减为 0，OUT 端输出才变为高电平。这是因为 CPU 不能将计数初始值直接写到计数执行部件 CE，而是先写到计数初始值寄存器 CR，然后再经过一个 CLK 脉冲转到 CE 中。

（4）如果在计数期间重新给计数器装入新值，若是 16 位，则会在写入第一个字节时停止现行计数过程，在写入第二个字节时开始新的计数过程；若是 8 位，在写入新的计数值后，计数器将按新的数值重新计数。从计数开始输出 OUT 变为低电平，一直保持到计数结束。写入新值后 OUT 波形图如图 12-5（c）所示。

2. 方式 1——可重复触发的单稳态触发器

当一个计数通道通过控制字设置为方式 1 后，其输出端信号 OUT 随即变为高电平。用 OUT 指令将计数初值装入该计数器，再经过一个 CLK 时间计数初始值送计数执行部件。CPU 写完计数值后计数器并不开始计数，直到外部门控脉冲 GATE 上升沿到来后，下一个 CLK 脉冲的下降沿开始计数，同时 OUT 输出变低。在整个计数过程中，OUT 端都维持为低，直到计数到 0，输出才变为高电平。因此，输出为一个单稳脉冲。若外部再次触发启动，则可以再产生一个单稳脉冲，如图 12-6（a）所示。

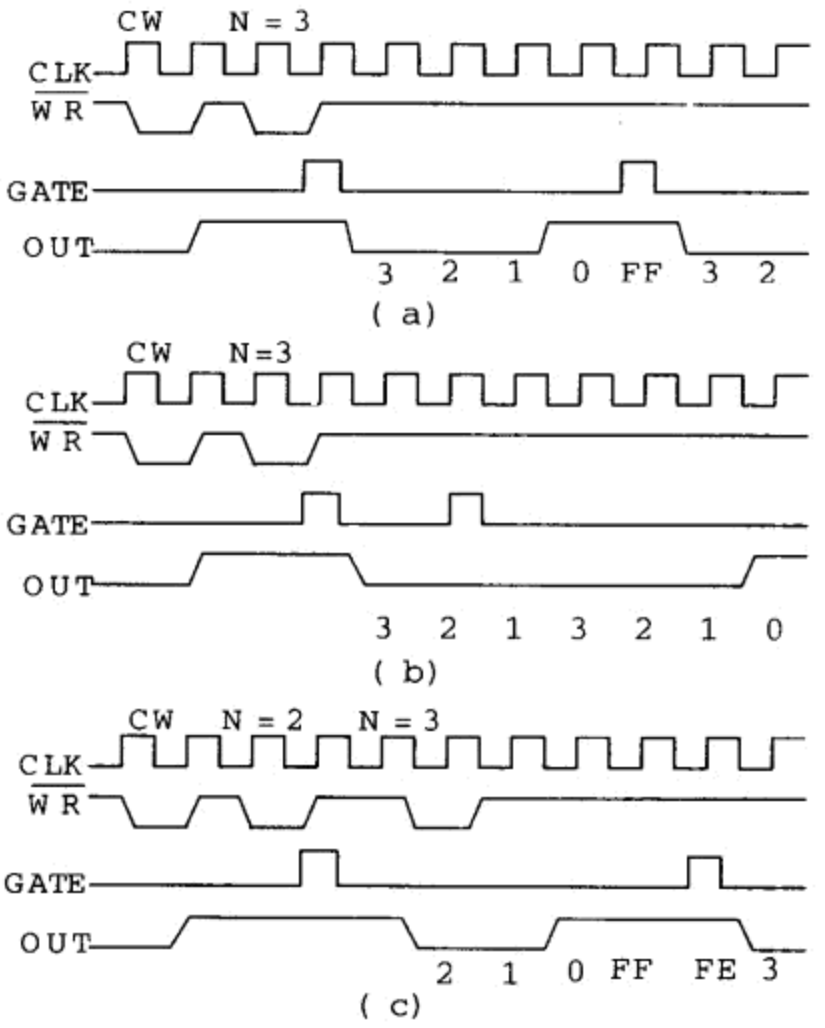


图 12-6 方式 1 波形图

方式 1 有以下特点：

（1）计数过程受门控信号 GATE 的影响。一方面是计数结束后若再来一个门控信号上升沿，则在下一个时钟周期的下降沿又从初值开始计数，而且不需要重新写入计数初值，即门控脉冲可重新触发计数，同时 OUT 端输出从高电平变为低电平，直到计数结束，再恢复

到高电平；另一方面是在计数进行中若来一个门控信号的上升沿，也要在下一个时钟下降沿终止原来的计数过程，从初值起重新计数。在这个过程中，OUT 输出保持低电平不变，直到计数执行单元内容减为 0 时，OUT 输出才恢复为高电平。这样，使 OUT 输出低电平持续时间加长，即输出单次脉冲的宽度加宽。GATE 对 OUT 输出波形的影响如图 12-6（b）所示。

（2）在计数过程中如果写入新的初值不会影响计数过程，只有在下一个门控信号到来后一个时钟下降沿，才终止原来的计数过程，而按新值开始计数。OUT 输出的变化是高电平持续到开始计数前，低电平持续到计数过程结束，如图 12-6（c）所示。

3. 方式 2——分频器

在方式 2 下，CPU 输出控制字后，OUT 端输出将变高。在写入计数值后，若门控信号 GATE=1，计数器执行部件将自动对输入时钟 CLK 计数。在计数过程中输出始终保持为高，直到计数器减为 1 时，输出变低。经一个 CLK 周期，输出恢复为高电平，且计数器开始重新计数。方式 2 波形图如图 12-7（a）所示。

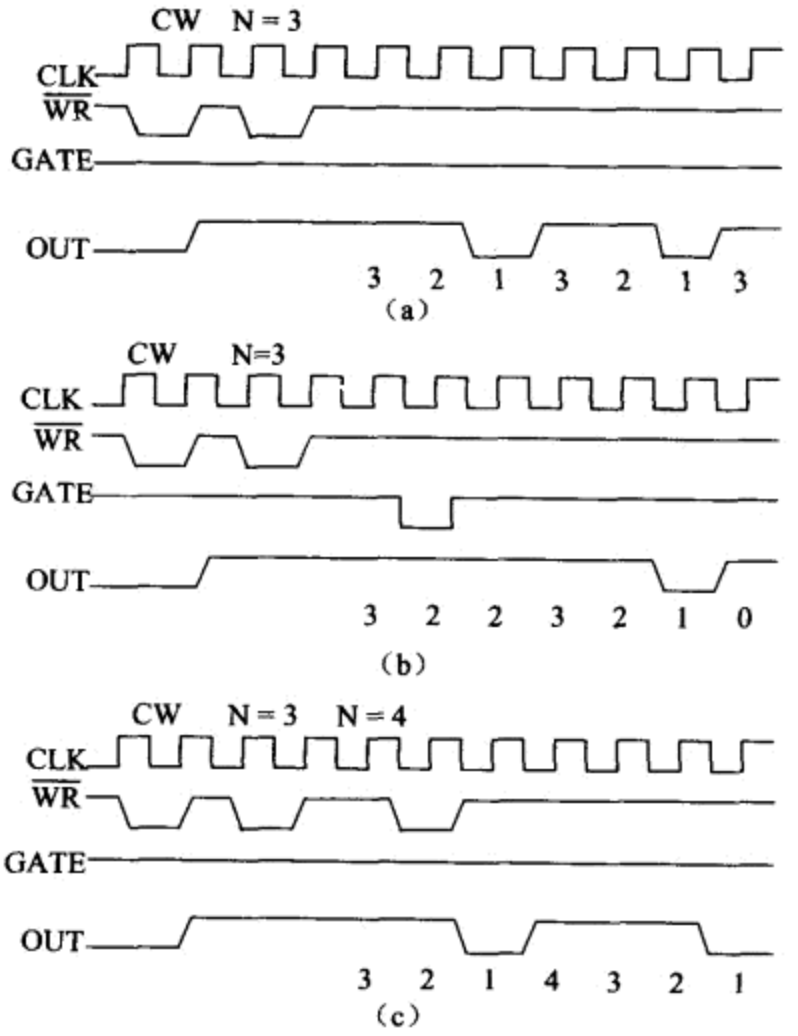


图 12-7　方式 2 波形图

方式 2 有以下特点：

（1）计数器能够连续工作。如果计数值为 N，每输入 N 个 CLK 脉冲 OUT 输出一个负脉冲。该方式很像一个频率发生器或分频器。

（2）计数过程中可改变初值。一种是 GATE 一直维持高电平，新的初值不影响当前的计数过程，但在计数结束后下一个计数周期将按新的初值计数；另一种是写入新的初值后遇到 GATE 的上升沿，则结束现行计数过程，从下一个时钟下降沿开始按新的初始值进行计数。第三种情况是计数值未减到 0 又重新按新的初值进行计数，OUT 一直维持高电平，可随时通过重新送计数值来改变输出脉冲的频率。GATE 的变化对 OUT 的影响如图 12-7（b）所示。

（3）计数过程中如果 GATE 变为低电平将停止计数，输出继续保持高电平。GATE 变高电平后计数器将重新装入预置的计数值，并开始计数。这样，GATE 能用硬件对计数器进行同步。写入新值对 OUT 输出端的影响如图 12-7（c）所示。

4. 方式 3——方波发生器

方式 3 与方式 2 的工作类似，输出均为周期性的，但方式 3 的输出为方波或者为基本对称的矩形方波。CPU 向 8253 写入控制字后 OUT 输出变为高电平。在写完计数初始值后，计数器自动开始对输入时钟 CLK 计数，这时 OUT 输出将保持高电平。在计数初值减到一半时，计数器将 OUT 端输出变为低电平。直到减到 1 后，重新装入计数初始值，OUT 端变高，开始新一轮计数。

方式 3 有以下特点：

（1）计数初始值有奇偶之分，计数处理上有区别。写入控制字后，在时钟上升沿 OUT 变为高电平。当初始值为偶数时，经过一个时钟周期，计数初值被送入计数执行单元，下一个时钟下降沿开始减 1 计数。减到 N/2 时 OUT 变为低电平，计数器继续减 1 计数，减到 0 时 OUT 又变成高电平，计数器重新从初值开始计数。只要 GATK 为 1，此过程一直重复。输出端可得到一方波信号，OUT 端波形如图 12-8 中的（a）图所示；当初始值为奇数时，在 GATE 一直为高电平情况下，OUT 输出波形为连续的近似方波，高电平持续时间为(N+1)/2 个脉冲，低电平持续时间为(N−1)/2 个脉冲，OUT 端波形如图 12-8 中的（b）图所示。

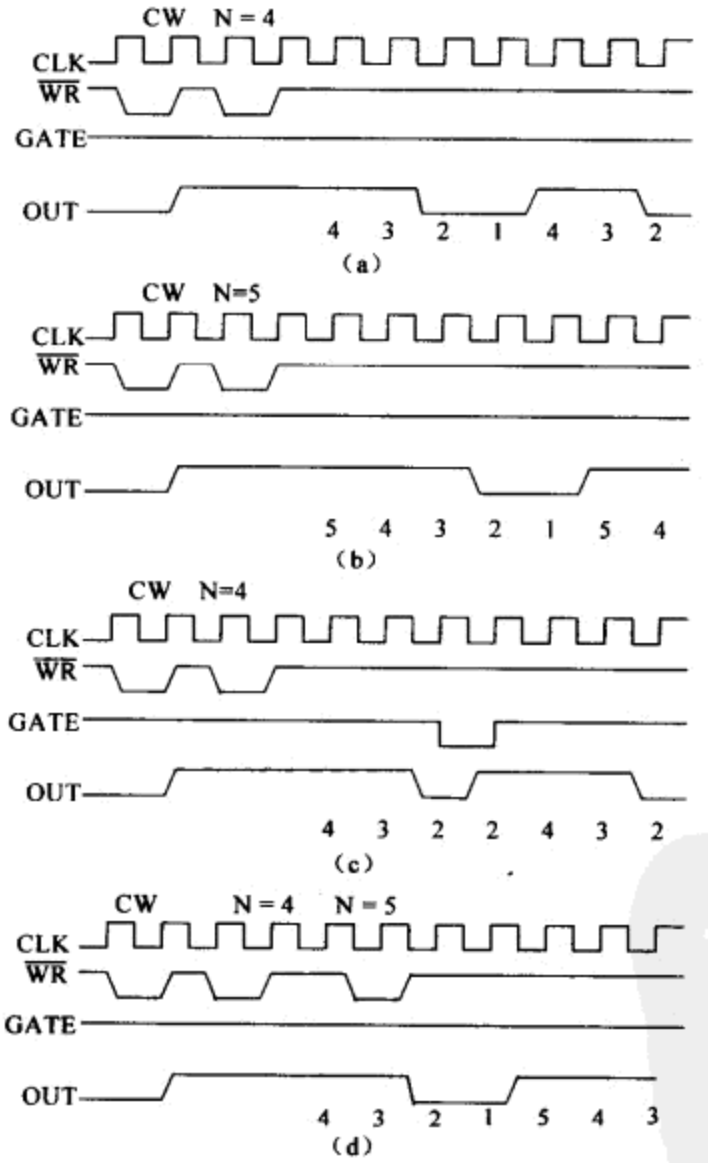


图 12-8　方式 3 波形图

（2）计数过程受 GATE 影响。写入控制字和计数初始值后，若 GATE=1 允许计数，GATE=0 禁止计数。在计数执行过程中，GATE 变为低电平时，若 OUT 为低电平，则 GATE 从低电平变为高电平；若已是高电平则保持不变，且计数器停止计数。当 GATE 恢复高电平后的第一个 CLK 的下降沿，计数初值重新装入 CE，并从初始值重新开始计数。GATE 对 OUT 端的影响如图 12-8 中的（c）图所示。

（3）GATE=1 时计数过程中，新值写入并不影响现行计数过程，只是在下一个计数过程中按新值进行计数。还有一种是在计数过程中 GATE 端出现一个脉冲信号，停止现行计数过程，在 GATE 上升后的第一个时钟周期的下降沿将按新的初始值开始计数。写入新值对波形的影响如图 12-8 中的（d）图所示。

5. 方式 4——软件触发的选通信号发生器

方式 4 下写入控制字后，在时钟上升沿 OUT 变成高电平，将计数初值写入初值寄存器中，经过一个时钟周期计数初值被送入计数执行单元 CE，下一个时钟下降沿开始减 1 计数，减到 0 时输出端变低，持续一个时钟周期，然后自动恢复成高电平。下一次启动计数时，必须重新写入计数值。由于每进行一次计数过程必须重新写入初值一次，不能自动循环，所以称方式 4 为软件触发。又由于 OUT 低电平持续时间为一个脉冲周期，常用此负脉冲作为选通信号，所以又称为软件触发选通方式。方式 4 波形图如图 12-9(a) 和 12-9（b）所示。

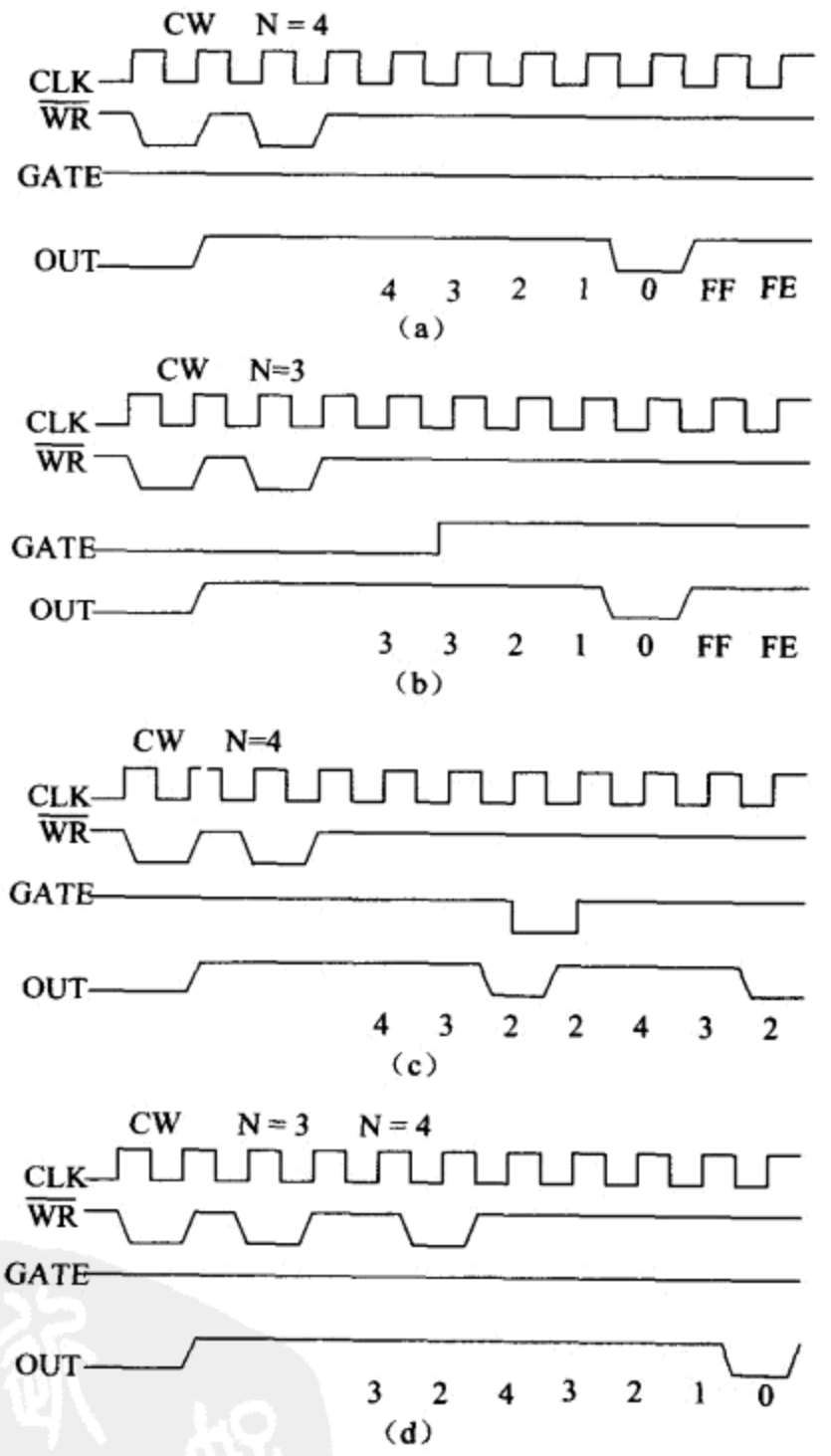


图 12-9　方式 4 波形图

方式 4 有以下特点：

（1）GATE 的变化对计数器的工作有直接影响。当 GATE=1 时计数器正常计数，当 GATE=0 时计数器停止计数，但 OUT 端还保持高电平输出。需要注意的是当 GATE=0 时停止计数，GATE=1 时并不是恢复计数，而是重新从初值开始计数。只有计数器减为 0 时，才使 OUT 产生电平的变化。GATE 对 OUT 输出波形的影响如图 12-9（c）所示。

（2）方式 4 允许改变计数值。如果在计数过程中写入新值，则分两种情况：GATE=1 时写入新的初始值，则计数器立即终止现行的计数过程，并从下一个时钟周期开始按新的计数值重新计数；若在写入新值时 GATE=0，停止计数，在门控信号 GATE 上升后的第一个时钟周期的下降沿，按新初值开始计数。OUT 波形情况如图 12-9（d）所示。

6. 方式 5——硬件触发的选通信号发生器

方式 5 为硬件触发的选通方式，完全由 GATE 端引入的触发信号控制定时和计数。CPU 写入控制字后，OUT 变成高电平，写入计数初值后，计数器并不开始计数。当 GATE 的上升沿到来后，在下一个时钟下降沿出现的时刻，将计数初始值写入 CE，然后开始对 CLK 脉冲进行减 1 计数。计数器减到 0 时 OUT 变为低电平。持续一个时钟周期又变为高电平，并一直保持高电平，直至下一个 GATE 的上升沿的到来。所以采用方式 5 可循环计数，并且计数初值可自动重装。计数过程的进行是靠门控信号触发的，因此称方式 5 为硬件触发。OUT 输出低电平持续时间仅一个时钟周期，可作为选通信号。方式 5 波形图如图 12-10（a）所示。

方式 5 有以下特点：

（1）GATE 对计数过程有影响。该信号重复使用时对当前输出状态没有影响。但如果在计数过程中又来一个 GATE 上升沿，则立即终止现行的计数过程，在下一个时钟周期的下降沿，又从初始值开始计数。如果计数过程结束后出现一个 GATE 的上升沿，计数器也会在下一个时钟周期下降沿从初值开始减 1 计数，不用重新写入初值。所以在方式 5 中，计数过程是由 GATE 的上升沿触发的。GATE 的变化对 OUT 端输出的影响如图 12-10（b）所示。

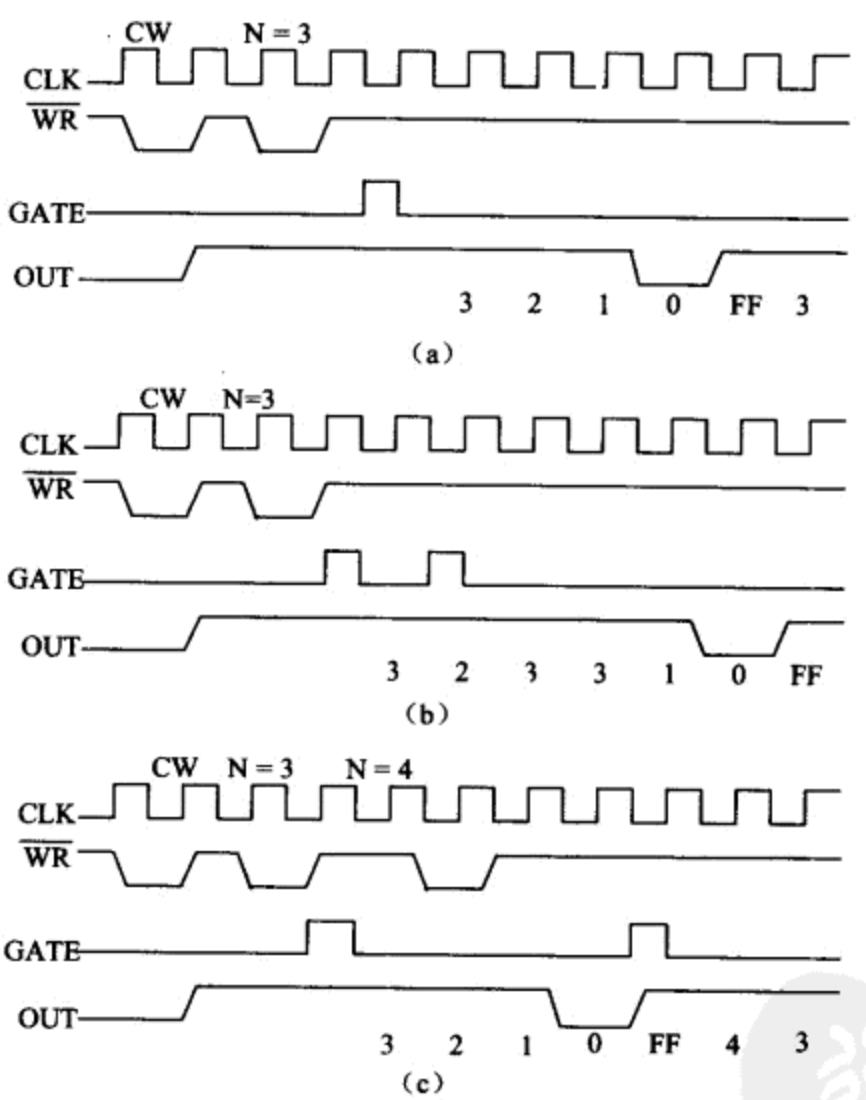


图 12-10　方式 5 波形图

（2）在减 1 计数期间置入新的初始值，当前计数过程不受影响，只是在下一个 GATE 的上升沿到来后，新的初始值装入计数执行单元 CE，按新值开始计数。如果新的初始值写入后，当前计数过程未结束就出现了 GATE 的上升沿，则在 GATE 上升沿出现后的第一个 CLK 时钟脉冲信号将使新的初始值装入 CE，开始新的计数，OUT 端波形如图 12-10（c）所示。

7. 8253 工作方式小结

8253 芯片的每个计数通道都有 6 种工作方式可供选择。区分这 6 种工作方式的主要标志有下面 3 点。① OUT 端的输出波形不同。② 计数过程的启动方式不同。③ 计数过程中门控信号 GATE 对计数操作产生的影响不同。

8253 的 6 种工作方式功能、输出波形特点、触发性质等内容比较如表 12-2 所示。GATE 信号功能如表 12-3 所示。工作方式与装入新初值的关系如表 12-4 所示。

表 12-2　8253 的 6 种工作方式比较

工作方式	功　　能	输出波形	触发性质
方式 0	计数结束中断	写入初值后，OUT 端变低，经过 N+1 个 CLK 后，OUT 变高	软件触发的单次负脉冲
方式 1	可编程单稳态触发器	输出宽度为 N 个时钟周期的负脉冲	硬件触发的单次负脉冲
方式 2	频率发生器	输出宽度为 1 个时钟周期的负脉冲	自动触发连续的脉冲波
方式 3	方波发生器	N 为偶数时占空比为 1/2；N 为奇数时输出 (N+1)/2 个正脉冲，(N−1)/2 个负脉冲	自动触发连续的方波
方式 4	软件触发选通	写入初值后，经过 N 个时钟周期，OUT 端变低 1 个时钟周期	软件触发单次单拍负脉冲
方式 5	硬件触发选通	门控触发后，经过 N 个时钟周期，OUT 端变低 1 个时钟周期	硬件触发单次单拍负脉冲

表 12-3　GATE 信号功能表

GATE / 工作方式	低电平或变到低电平	上 升 沿	高 电 平
方式 0	禁止计数	不影响	允许计数
方式 1	不影响	启动计数	不影响
方式 2	禁止计数并置 OUT 为高电平	初始化计数	允许计数
方式 3	禁止计数并置 OUT 为高电平	初始化计数	允许计数
方式 4	禁止计数	不影响	允许计数
方式 5	不影响	启动计数	不影响

表 12-4　工作方式与装入新初值的关系

工作方式	装入新的初值后计数过程的变化
方式 0	立即有效
方式 1	门控触发后有效
方式 2	下一个计数周期或门控触发后有效
方式 3	下一个计数周期或门控触发后有效
方式 4	立即有效
方式 5	门控触发后有效

12.2.5 8253 的应用

1. 8253 的初始化

对 8253 的初始化也称为对 8253 的编程。完成初始化后 8253 即开始自动按设置好的工作方式工作。初始化程序包括两部分，一是写各计数器的控制字，二是设置计数初始值。

【例 12.3】假设 8253 的计数器 0 工作在方式 5，按二进制计数，计数初始值为 100；计数器 1 工作在方式 1 下，BCD 码计数，计数初始值为 4000；计数器 2 工作在方式 2 下，按二进制计数，计数初始值为 600。8253 占用的端口地址为 200H 到 203H。以上情况的初始化程序如下：

```
MOV  DX,203H                ;控制寄存器地址送 DX
MOV  AL,00011010B           ;计数器 0,写低字节,方式 5,二进制计数
OUT  DX,AL                  ;写控制字寄存器
MOV  DX,200H                ;计数器 0 的地址送 DX
MOV  AL,100                 ;计数初始值为 100
OUT  DX,AL                  ;写入计数初始值
MOV  DX,203H                ;控制寄存器地址送 DX
MOV  AL,01100011B           ;计数器 1,写高字节,方式 1,十进制计数
OUT  DX,AL                  ;写控制字寄存器
MOV  DX,201H                ;计数器 1 的地址送 DX
MOV  AL,40H                 ;计数初始值为 4000H,只写高 8 位即可
OUT  DX,AL                  ;写入计数初始值
MOV  DX,203H                ;控制寄存器地址送 DX
MOV  AL,10110100B           ;计数器 2,16 位初始值,方式 1,二进制计数
OUT  DX,AL                  ;写控制字寄存器
MOV  DX,202H                ;计数器 2 的地址送 DX
MOV  AX,600                 ;计数初始值为 600
OUT  DX,AL                  ;先写低 8 位
MOV  AL,AH
OUT  DX,AL                  ;再写高 8 位
```

2. 8253 与系统的连接

IBM PC/XT 机使用一片 Intel 8253 作为定时/计数芯片。3 个计数通道分别用于日时钟计时、DRAM 刷新定时和控制扬声器发声。8253 和系统的连接如图 12-11 所示。IBM PC/AT 使用与 8253 兼容的 Intel 8254，AT 机的连接使用也与 XT 机一样。

从连接图可以看出，8253 共占用 4 个端口地址，地址范围由高位地址信号决定，高位地址的译码输出接到片选端 $\overline{CS}$，A_0 和 A_1 分别接到系统总线的 A_0 和 A_1 地址信号线上，用来寻址芯片内部的 3 个计数器及控制字寄存器。系统总线的 $\overline{IOW}$ 和 $\overline{IOR}$ 分别接入 8253 的 $\overline{WR}$ 和 $\overline{RD}$，数据总线的低 8 位连接至 8253 的数据线 D_7～D_0。PC 中只有 A_9～A_0 参加 I/O 地址空间的译码，所以 PC 中可寻址的 I/O 端口的个数最多为 512 个。下面给出 XT 机中系统板上端口片选生成逻辑，如图 12-12 所示。

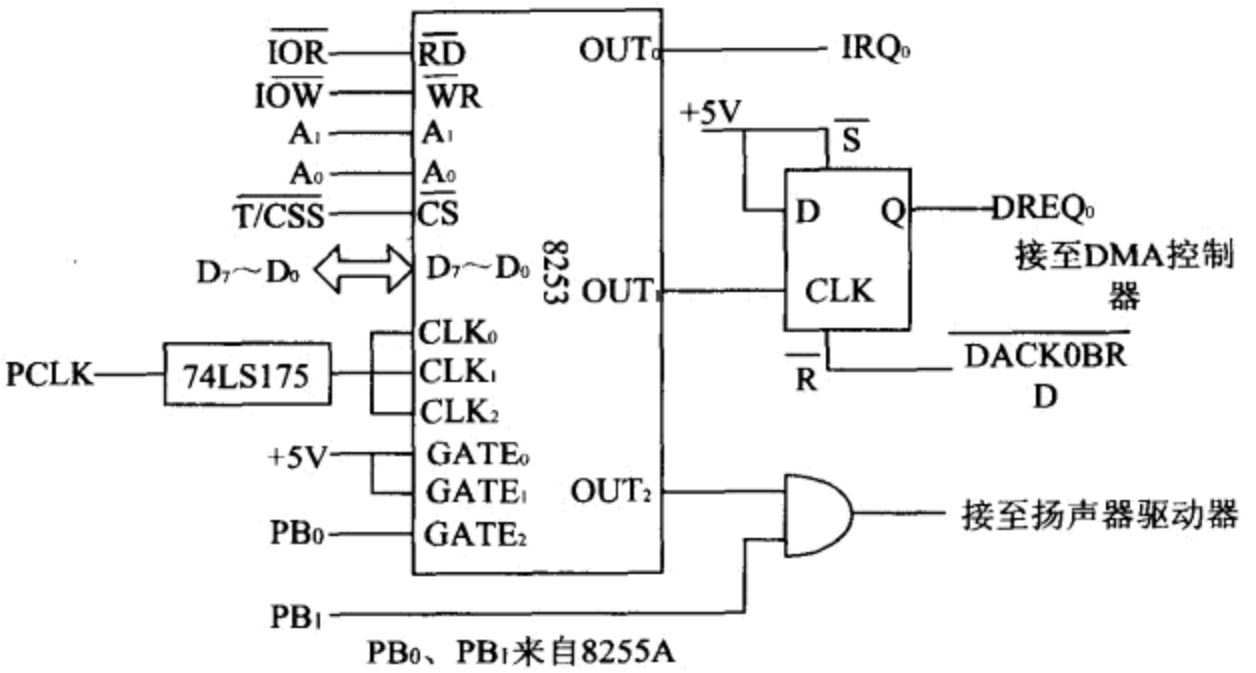


图 12-11　IBM PC/XT 机中 8253 和 CPU 连接图

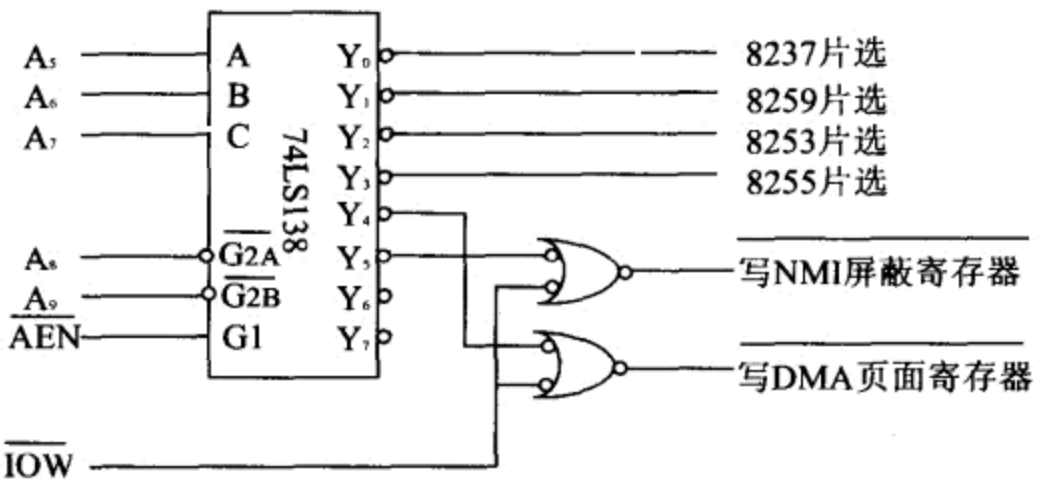


图 12-12　系统板上片选生成逻辑

通过图 12-12 可知，8253 的片逻辑为：$A_9 \cdot A_8 \cdot A_7 \cdot A_6 \cdot A_5$，接入 8253 内部的地址为 A_1、A_0，地址信号 A_4、A_3、A_2 未参加译码。因此出现了影像地址，即 8253 的地址为 040H～043H，同时 044H～05FH 也不能再分配给其他端口。

PCLK 是来自时钟发生器 8284A 的输出时钟，经 74LS175 二分频后作为 8253 三个计数器的时钟输入。三个计数器的具体情况如下：

（1）计数器 0 工作在方式 3 下，$GATE_0$ 固定为高电平，作为中断请求接至 8259A 中断控制器的 IRQ_0，此中断用于报时时钟和磁盘驱动器的马达定时。

（2）计数器 1 工作在方式 2 下，$GATE_1$ 固定为高电平，OUT1 的输出经 74LS74 后作为 8237ADMA 控制器通道 0 的 DMA 服务请求 $DREQ_0$，用于定时刷新动态存储器 DRAM。

（3）计数器 2 工作在方式 3 下，1kHz 的方波输出，滤掉高频分量后送至扬声器，它的门控制信号是 8255A 的 PB_0，OUT_2 输出经与门控制，控制信号为 8255AA 的 PB_1，可用 PB_1、PB_0 同时为高的时间来控制发音时间。

3. 8253 应用实例

【例 12.4】在 8086 系统中，8253 的各端口地址为 81H、83H、85H 和 87H。现提供时钟频率为 2MHz，要求用 8253 来控制一个 LED 发光二极管的点亮和熄灭，点亮 10s 后再让它熄灭 10s，并重复上述过程。

分析：因为计数频率为 2MHz，计数器的最大计数值为 65 536，所以最大的定时时间为 0.5μs×65 536=32.768ms，达不到 20s 的要求，因此需用两个计数器级联来解决问题。

将 2MHz 的时钟信号直接加在 CLK_0 输入端，并让计数器 0 工作在方式 2，选择计数初始值为 5000，则从 OUT_0 端可得到频率为 2MHz/5000=400HZ 的脉冲，周期为 0.25ms。再将该信号连到 CLK_1 输入端，并使计数器 1 工作在方式 3 下，为了使 OUT_1 输出周期为 20s（频率为 1/20=0.05Hz）的方波，应取时间常数 N1=400Hz/0.05=8000。硬件连接图如图 12-13 所示。

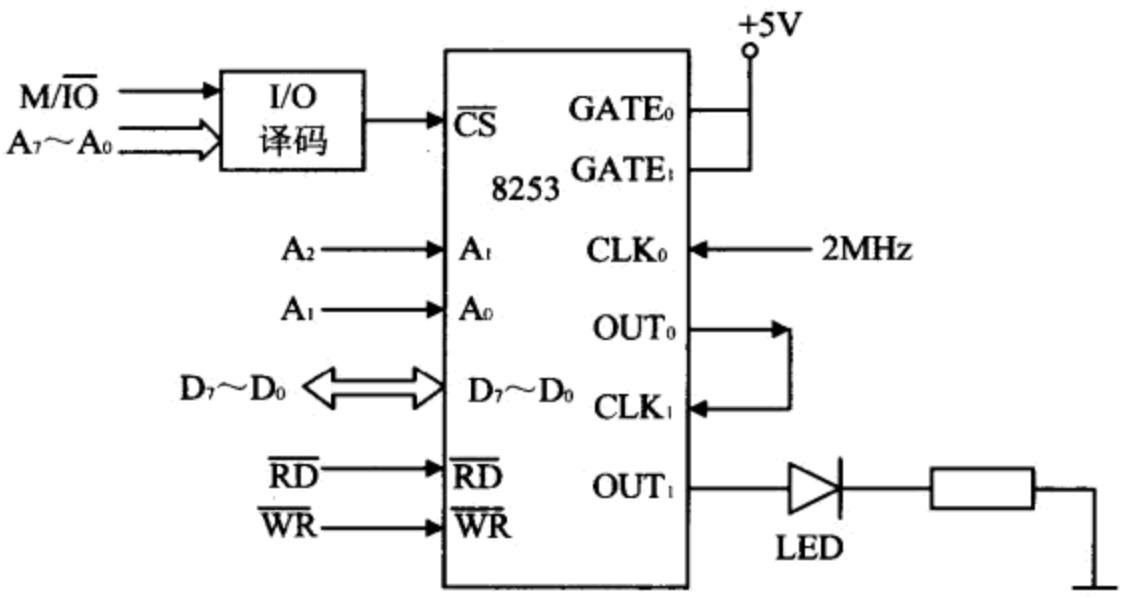


图 12-13　例 12.4 连接图

初始化程序如下：

```
MOV  AL,00110101B        ;计数器 0 控制字内容送 AL,先低后高,方式 2,BCD 计数
OUT  87H,AL              ;写计数器 0 控制字
MOV  AL,00H              ;计数器初始值低 8 位
OUT  81H,AL
MOV  AL,50H              ;计数器初始值高 8 位
OUT  81H,AL
                         ;
MOV  AL,01110111B        ;计数器 1 控制字内容送 AL,先低后高,方式 3,BCD 计数
OUT  87H,AL
MOV  AL,00H              ;计数器初始值低 8 位
OUT  83H,AL
MOV  AL,80H              ;计数器初始值高 8 位
OUT  83H,AL
```

本章小结

本章主要介绍了定时器/计数器接口芯片 8253 的内部结构、引脚功能、初始化编程和主要应用。

8253 有定时和计数功能，内部包含 3 个 16 位计数器，每个计数器可按二进制或十进制计数，有 6 种工作方式，可通过编程选择。在不同的工作方式下，计数过程的启动方式、OUT

端的输出波形都不一样。自动重复功能和 GATE 的控制作用以及写入新的计数初值对计数器的工作过程产生的影响是不一样的。

思考与练习题

1. 试说明 8253 的 6 种工作方式，其时钟信号 CLK 和门控信号 GATE 分别起什么作用？
2. 8253 的最高工作频率是多少?8254 与 8253 的主要区别是什么？
3. 对 8253 进行初始化编程分哪几步进行?
4. 设 8254 芯片的计数器 0、计数器 1 和控制口地址分别为 04B0H、04B2H、04B6H。定义计数器 0 工作在方式 2，CLK_0 为 5MHz，要求输出 OUT_0 为 1kHz 方波；定义计数器 1 用 OUT_0 作计数脉冲，计数值为 1000，计数器减到 0 时向 CPU 发出中断请求，CPU 响应这一中断请求后继续写入计数值 1000，开始重新计数，保持每一秒钟向 CPU 发出一次中断请求。试编写出对 8253 的初始化程序，并画出硬件连接图。
5. 将 8253 定时器 0 设为方式 3（方波发生器），定时器 1 设为方式 2（分频器）。要求定时器 0 的输出脉冲作为定时器 1 的时钟输入，CLK_0 连接总线时钟 2MHz，定时器 1 输出 OUT_1 约为 40Hz，试编一段程序。

第 13 章 人机交互设备及接口

本章主要讲解微机系统中常用的人机交互设备的基本工作原理，其中包括：键盘的分类、工作原理、按键识别方法以及键盘中断调用编程方法；显示器的分类及 CRT 显示器工作原理，显示器中断调用的编程方法；打印机的基本结构、工作原理，与 CPU 的连接方法；简单介绍鼠标的基本原理、分类和与 CPU 的连接方法；简单介绍其他外部设备的原理与应用情况。

通过本章的学习，读者应该理解键盘、显示器、打印机、鼠标、扫描仪、数码相机和触摸屏等人机交互设备的工作原理及工作过程，掌握相应的编程方法及应用技术。

13.1 人机交互设备概述

人机交互设备是指人和计算机之间建立联系、交流信息的输入/输出设备。这些输入/输出设备直接与人的运动器官（如手、口）或感觉器官（如眼、耳）有关。通过它们可以把要执行的命令和数据传送给计算机，同时又从计算机获得易于理解的信息。常见的人机交互设备有键盘、显示器（LCD 显示器和 CRT 显示器等）、打印机、鼠标等。这些设备的输入/输出是以计算机为中心，信息以二进制、十六进制码或 ASCII 码形式进行传送。该形式计算机容易接受和便于处理，但人接收和处理起来却不方便，必须通过大脑的思维、判断和转换。

由于计算机的普遍应用和智能控制技术的发展，出现了智能型人机交互设备。这类设备是以人为中心，以人的感觉器官易于接受而且可识别的图像、语音、文字形式来传输信息。这些信息必须通过适当的处理和变换，成为计算机所能接受的二进制等数字形式，才能送入计算机。这类智能人机交互设备有文字图像扫描输入机、语音输入机和图像显示器、语音合成器等。

本章讨论几种常见的人机交互设备，以及它们与系统接口的硬、软件设计思想和方法。

13.2 键盘与鼠标

13.2.1 键盘及接口电路

1. 键盘的分类

（1）根据按键本身的结构方式分类

- 机械触点式：每个按键的下部有两个触点，把机械上的通断转换成电气上的逻辑关系，按键被按下后两触点才接通。这种键盘手感差，易磨损，故障率较高，寿命短。
- 电容式按键：按键通过改变电容器电极之间的距离来使电容发生变化。每个按键由活动极、驱动极与检测极组成两个串联的电容器。有键按下时上下两极片靠近，极板间距离缩短，来自振荡器的脉冲信号被电容耦合后输出；反之无信号输出。该类键盘手感好，寿命长，灵敏度和稳定性也都比较好。目前使用的计算机键盘大多为电容式无

触点键盘。

（2）按照控制形态分类

- 编码键盘：带有相应的硬件电路，由专用控制器对键盘进行扫描，按键按下时系统可自动检测并能提供按键对应的键值，并将数据保持到新键按下为止。该键盘接口简单，使用方便，但硬件电路复杂，价格较贵。
- 非编码键盘：主要采用软件来识别键的按下和释放，并给出相应的编码。该类键盘可靠性高，扩充和更改方便灵活，是目前的主流键盘。

（3）按照键盘的用途分类

- 通用键盘：指通用计算机都配置的键盘。如 PC 系列微机键盘有 101 个按键或 104 个按键，包括字符数字键、扩展功能键、组合键等。键盘由单片机及相关的输入/输出电路构成，单片机负责识别键的按下和释放，产生相应的扫描码并送到键盘的接口。
- 专用键盘：常用于单片机、单板机以及微机控制系统的小键盘，一般具有数字键、字符键和若干功能键。该类键盘的控制电路相对较简单，使用者可根据需要来设计或定义各功能键的功能。

2. 键盘的工作原理

非编码键盘有线性键盘和矩阵键盘。线性键盘是指其中每个按键均有一条输入线送到计算机接口，若有 N 个键则需 N 条输入线，如图 13-1 所示。该结构适合于按键不多的场合。

矩阵键盘是指按键按行（M 行）和列（N 列）排列，可排列 M×N 个按键，送往计算机的输入线为 M+N 条，如图 13-2 所示。该结构适合于按键较多的场合。

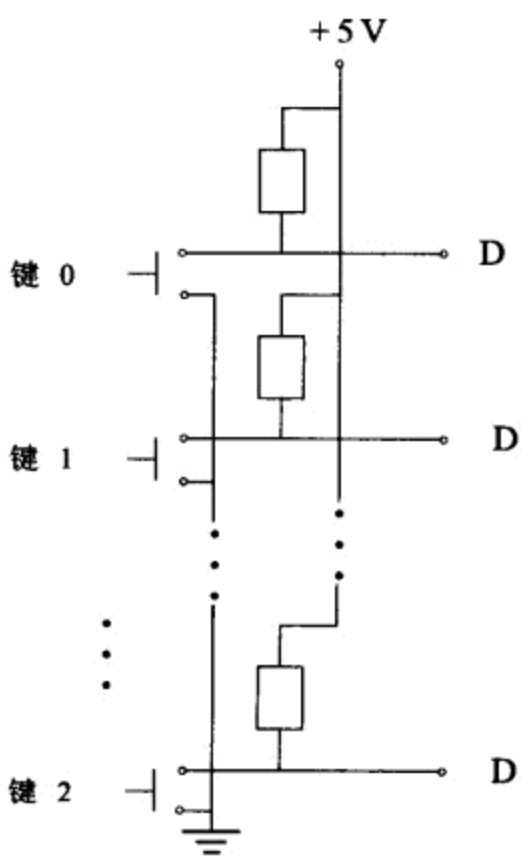


图 13-1　简单线性键盘

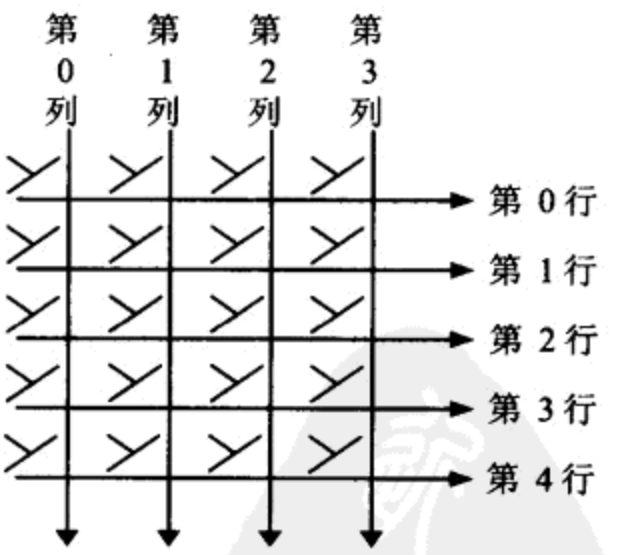


图 13-2　矩阵键盘

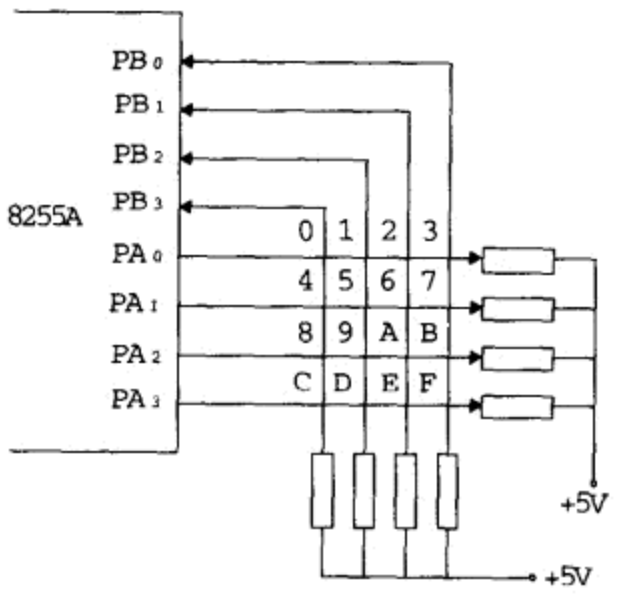
非编码键盘中为了检测哪个键被按下常采用硬件方法或软、硬件结合的方法。其目的是识别键盘矩阵中的被按键，清除按键时产生的抖动干扰，防止键盘操作的串键错误，产生被按键相应的编码等。

常用的按键识别方法有行扫描法、行反转法和行、列扫描法等 3 种。

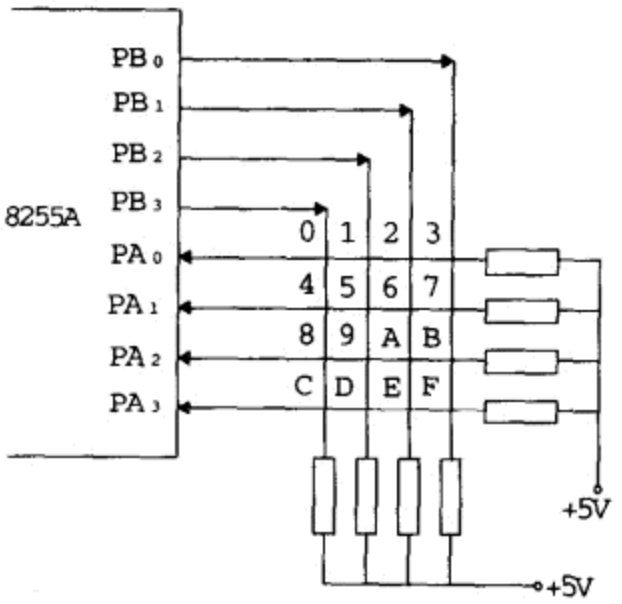
（1）行扫描法：先进行全扫描判断是否有键被按下，将所有行线置低电平，然后扫描全部列线，如果扫描的列值全是高电平，说明没有任何一个键被按下；如果读入的列值不是全“1”，说明有键按下，再用逐行扫描的方法确定哪一个键被按下：先扫描第一行，置该行为低电平，其他行为高电平，然后检查列线，如果某条列线为低电平，说明第一行与该列相交的位置上的按键被按下；如果所有列线全是高电平，说明第一行没有键被按下，接着扫描第二行，以此类推，直至找到被按下的键。

（2）行反转法：又称线反转法，利用可编程并行接口（如 8255A）来实现。将行线接到并行口先工作在输出方式，将列线接到另一个并行口先工作在输入方式。编程使 CPU 通过输出端口向各行线全部送低电平，然后读入列线的值。如果有某一个键被按下，则必有一条列线为低电平。然后通过编程对两个并行端口进行方式设置，并将刚才读到的列线值通过所连接的并行口再输出到列线，然后读取行线的值，那么闭合键所对应的行线必为低电平，这样当一个键被按下时，就可以读到一对唯一的列值和行值。

行反转法示意如图 13-3 所示，如果标号为 5 的按键被按下，则第一次向行线输出低电平后读出的列值为 1011，第二次向列线输出刚才得到的 1011 后，会从行线上读得 1101，于是行值和列值合并在一起得到一个数值 11011011，即 DBH，这个值对应 5 号键且是唯一的。



（a）行线输出



（b）列线输出

图 13-3　行反转法示意图

【例 13.1】分析用行反转扫描法对 4×4 矩阵键盘进行扫描识别的程序。设 8255A 的 A 口、B 口、C 口和控制口地址分别为 80H、81H、82H 和 83H。程序设计如下：

```
START:  MOV  AL,  82H
        OUT  83H, AL          ;初始化 8255,方式 0,A 口输出,B 口输入
        MOV  AL,  0
        OUT  80H, AL          ;A 口输出 0,即行线全为 0
WAIT:   IN   AL,  81H         ;读入 B 口的列线内容
        AND  AL,  0FH         ;屏幕无关位
        CMP  AL,  0FH         ;是否有键闭合
        JZ   WAIT             ;没有,等待键入
        MOV  BL,  AL          ;列线存入 BL 中
        CALL DELAY            ;延时消除抖动
```

```
        MOV  AL,  90H
        OUT  83H, AL                ;8255 初始化,A 口输入,B 口输出
        MOV  AL,  BL
        OUT  81H, AL                ;读入列线的值再输出
        IN   AL,  80H               ;读 A 口行线值
        MOV  AH,  AL                ;行线值存入 AH
        MOV  AL,  BL
        LEA  SI,  TABLE             ;取键码表首地址
        MOV  CX,  16
LOOP1:  CMP  AX,[SI]
        JZ   KEY                    ;转 KEY 进行键盘处理
        INC  SI
        INC  SI
        LOOP LOOP1
        JMP  START
TABLE:  DW   0FEFEH,0FEFD
```

（3）行、列扫描法：是通过计数译码使各行依次输出低电平。在扫描每一行时读列线，若读得的结果为全“1”，说明没有键按下，或者未扫描到闭合键；若某一列为低电平，说明有键按下，而且行号和列号已经确定。然后，用同样的方法依次向列线扫描输出，读行线。如果两次所得到的行号和列号仍相同，则键码确定无疑，即得到了闭合键的行列扫描码。

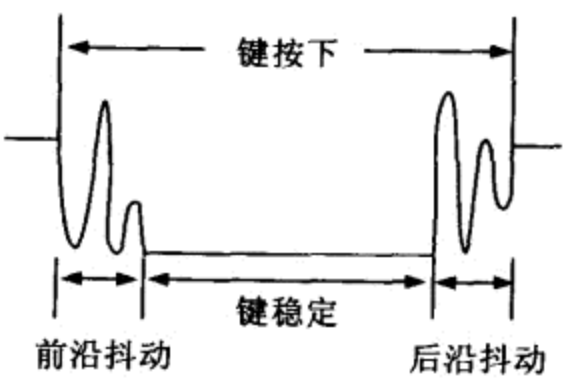
目前在 PC 系列微机中使用的键盘多数是采用行列扫描法。

3. 抖动和重键问题

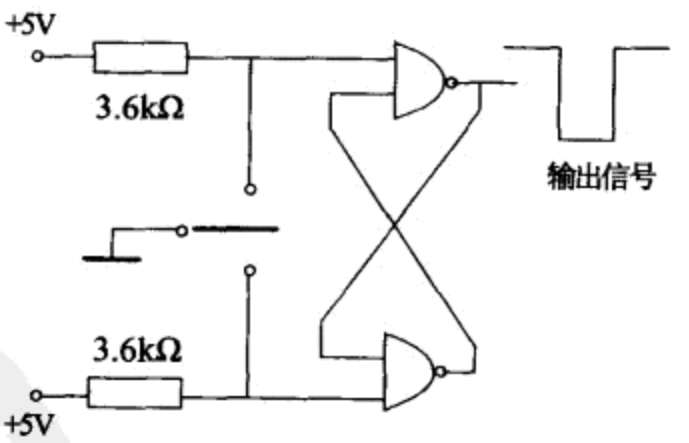
键盘设计时除了对键码的识别外，还有两个问题需要解决：抖动和重键。

用手按下一个键时往往会出现按键在闭合位置和断开位置之间跳几下后才稳定到闭合状态，在释放一个键时也会出现类似的情况，这就是抖动。抖动的持续时间随操作员而异，不过通常总是不大于 10ms。

利用硬件电路很容易消除抖动，如图 13-4 所示。在键数很多的情况下用软件方法也很实用，采用延时来等待抖动消失，然后再读入键值。



（a）键开关抖动图



（b）硬件消除抖动示意图

图 13-4　键盘硬件消除抖动示意图

重键是指两个或多个键同时闭合。对重键的处理简单情况下可以不予识别，即认为重键是一次错误的按键。通常情况是只承认先识别出来的键，对此时同时按下的其他键均不做识别，直到所有键都释放以后才读入下一个键。这是前面键盘扫描程序使用的方法，称为“连

锁法”。还有一种“巡回法”其基本思想是：等被识别的键释放以后，再对其他闭合键做识别，而不必等待全部键释放，巡回法比较适合于快速键入的操作。当然，对于某些特定的重键也可以认为是正常的组合键，只要将它们都识别出来即可。

13.2.2 PC 键盘接口及其应用

1. IBM PC 键盘接口

键盘接口的主要功能是接收键盘送来的扫描码，输出缓冲区满产生键盘中断，接收并执行系统命令，对键盘进行初始化、测试、复位等操作。

目前 PC 机常用的键盘接口有标准接口，PS/2 接口和 USB 接口 3 种。

（1）标准接口：为圆形，一般用于早期的 AT 主板上，所以也叫做 AT 接口。

（2）PS/2 接口：为具有 6 针的圆形插座。

（3）USB 接口：USB 设备具有即插即用，支持热插拔等优点，键盘也采用了该接口。

PC 键盘的接口安装在主板上，键盘和主机通过插头电缆相连。键盘触点电路按 16 行×8 列的矩阵来排列，用单片机 Intel 8048（或 8049）来完成键盘开关矩阵的扫描、键盘扫描码的读取和发送等功能。

2. 微机与键盘的接口

微机键盘是一个智能键盘，由单片机对整个键盘上的字符键、功能键、控制键和组合键进行管理。键入一个键时，键盘上的处理器首先向主机产生硬件中断请求（IRQ_1），然后将该键的扫描码以串行的方式传送给主机；主机在 IRQ_1 硬件中断的作用下，调用 INT 09H 中断把键盘送来的扫描码读入，并转化为 ASCII 码存入键盘缓冲区。

（1）键盘的工作原理：系统加电后固化在单片机中的键盘扫描程序用行列扫描法周而复始地扫描键盘矩阵，如果有闭合键则根据其位置获得对应的扫描码，当闭合键松开时也对应一个扫描码，前者称为闭合扫描码，后者称为断开扫描码，两者合起来称为键盘扫描码。键盘内部有一个 16 字节的 FIFO 队列缓冲器，用于存放按键扫描码。

（2）键盘接口的功能：微机系统按中断方式支持键盘随机地输入键符，每按一次键，键盘就会向系统发送一个扫描码，主机接口通过中断请求 CPU 处理，只有处理完一个键码才能再输入下一个键码。键盘接口的功能是：接收来自键盘的按键扫描码数据；对接收的数据进行奇偶校验；控制和检测传送数据的时间；对接收的数据进行串/并行转换；将按键的行列位置扫描码转换为系统扫描码；接收并执行系统命令；向系统发键盘中断，请求主机进行键盘代码处理。

目前的主板已经内置 USB 接口，USB 接口键盘通过 USB 接口和主机相连。USB 接口的 2#脚和 4#脚用于双向传送数据。该接口具有热插拔功能，最快传输速率可达 12Mb/s，且允许一次连接多达 127 个外设，打破了各种接口连接的限制，成为目前的流行趋势。

（3）键盘中断处理程序：与之有关的一个是 09H 中断程序，另一个是 16H 中断程序。当键盘接口控制器将数据送入输出缓冲器后，引发硬件中断请求 IRQ_1，系统调用 INT 09H 中断程序进行键盘代码处理，读取来自键盘的扫描码或命令，对命令代码进行处理，将扫描码转换为两字节的 ASCII 码或扩充码，送入到设置在内存 BIOS 数据区的一个 32 字节的键盘缓冲区中。键盘缓冲区的循环队列形式由软件实现，进队列由中断 09H 处理程序完成，出队列

由 INT 16H 程序来完成。为此，循环队列需要设置头、尾两个指针。头指针总是指向缓冲区最早送入的键码位置，尾指针总是指向压入的键码的下一个位置。

3. 键盘中断调用

读取键盘缓冲区中的内容可通过 BIOS 或 DOS 中断功能调用来实现。

（1）采用 BIOS 中断调用的方法：BIOS 键盘中断（INT 16H）提供了基本的键盘操作，它的中断处理程序包括了 3 个不同的功能，分别根据 AH 寄存器中的内容来进行选择。其对应关系如表 13-1 所示。

表 13-1　BIOS 键盘中断（INT 16H）

功 能 号	执行操作	特　　点
AH=00H	从键盘读入一个字符	AL 中的内容为字符码，AH 中的内容为扫描码
AH=01H	读键盘缓冲区的字符	ZF=0 时：AL 中的内容为字符码，AH 中的内容为扫描码； ZF=1 时：缓冲区空
AH=02H	读键盘状态字节	AL 中的内容为键盘状态字节

【例 13.2】下面给出一个利用键盘 I/O 功能的程序例子。用 INT 16H（AH=0）调用实现键盘输入字符。

```
DATA    SEGMENT
        BUFF    DB 100 DUP(?)
        MESS    DB 'NO CHARACTER!',0DH,0AH,'$'
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA
START:  MOV     AX,DATA
        MOV     DS,AX
        MOV     CX,100
        MOV     BX,OFFSET BUFF          ;设内存缓冲区首址
LOP1:   MOV     AH,1
        PUSH    CX
        MOV     CX,0
        MOV     DX,0
        INT     1AH                     ;设置时间计数器值为 0
LOP2:   MOV     AH,0
        INT     1AH;                    ;读时间计数值
        CMP     DL,100
        JNZ     LOP2                    ;定时时间未到,等待
        MOV     AH,1
        INT     16H                     ;判有无键入字符
        JZ      DONE                    ;无键输入,则结束
        MOV     AH,0
        INT     16H                     ;有键输入,则读出键的 ASCII 码
        MOV     [BX],AL                 ;存入内存缓冲区
        INC     BX
        POP     CX
        LOOP    LOP1                    ;100 个未输完,转 LOP1
        JMP     EN
DONE:   MOV     DX,OFFSET MESS
```

```
        MOV     AH, 09H
        INT     21H                     ;显示提示信息
EN:     MOV     AH,4CH
        INT     21H
        CODE    ENDS
        END     START
```

（2）采用 DOS 功能调用的方法：DOS 功能调用是通过 INT 21H 实现的，和键盘有关的功能调用如表 13-2 所示。

表 13-2　DOS INT 21H 功能调用

功 能 号	执行操作	特　　点
AH=01H	键盘输入并回显	输入字符送 AL 保存
AH=06H	直接控制台输入/输出字符	此调用的功能是从键盘输入或输出一个字符到屏幕
AH=07H	直接控制台输入，无回显	此调用同 1 号功能调用相似，不同的是输入不回显并且不检查快捷键【Ctrl+Break】
AH=08H	键盘输入，无回显	此调用同 1 号功能调用相似，不同的是输入的字符不回显
AH=0AH	字符串输入到缓冲区	缓冲区首址送 DS：DX

【例 13.3】利用 09H 和 0AH 系统功能调用，实现人机对话功能。程序设计如下：

```
DATA    SEGMENT
        MESS    DB  'WHAT IS  YOUR NAME?',0AH,0DH,'$'
        IN_BUF  DB  81
                DB  ?
                DB  81  DUP(?)
DATA    ENDS
STACK   SEGMENT
STA     DB   100  DUP(?)
TOP     EQU  $-STA
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA,SS:STACK
START:  MOV   AX,DATA
        MOV   DS,AX
        MOV   AX,STACK
        MOV   SS,AX
        MOV   SP,TOP
DISP:   MOV   DX,OFFSET MESS
        MOV   AH,09H
        INT   21H
KEYI:   MOV   DX,OFFSET IN_BUF
        MOV   AH,0AH
        INT   21H
        MOV   DL,0AH
        MOV   AH,02H
        INT   21H
        MOV   DL,0DH
        MOV   AH,02H
        INT   21H
```

```
DISPO:  LEA   SI,IN_BUF
        INC   SI
        MOV   AL,[SI]
        CBW
        INC   SI
        ADD   SI,AX
        MOV   BYTE  PTR [SI],'$'
        MOV   DX,OFFSET IN_BUF+2
        MOV   AH,09H
        INT   21H
        MOV   AH,4CH
        INT   21H
CODE    ENDS
        END   START
```

13.2.3　鼠标及接口电路

1. 鼠标的工作原理

鼠标是一种快速定位器，可方便地定位光标在屏幕上的任何位置。鼠标在平面上移动时随着方向和速度的变化会产生两个在高低电平之间不断变化的脉冲信号，CPU 接收这两个脉冲信号并对其计数。根据脉冲信号的个数，CPU 控制屏幕上的鼠标指针在横（X）轴、纵（Y）轴两个方向上移动的距离。

根据结构的不同，鼠标一般分为光机式和光电式（也称为机械式和光学式）鼠标。

（1）光机式鼠标：在基座底部凹处安有一个实心的橡胶球，内部有两个互相垂直的滚轴紧靠在橡胶球上。两个上滚轴的顶端各装有一个边缘开槽的光栅轮，光栅轮的两侧分别安装发光二极管和光敏三极管构成的光电检测电路。当鼠标在平面上移动，橡胶球与平面摩擦滚动时带动滚轴及其上的光栅轮旋转。因为光栅轮开槽处透光，未开槽处遮光，使得光敏三极管接收到的由发光二极管发出的光线时断时续，产生不断变化的高低电平，形成脉冲电信号。互相垂直的两个轴对应屏幕上的 X、Y 轴两个方向。脉冲信号的数量对应着位移的大小。

（2）光电式鼠标：光电式鼠标没有橡胶球和带光栅轮的滚轴，是在其基座上装有两对发光二极管和光敏接收管。两对光电检测器互相垂直，光敏三极管检测发光二极管照射到鼠标下面垫板上产生的反射光的强度变化，从而确定鼠标在 X、Y 两个方向上的位移。

2. 鼠标接口

鼠标按接口分类主要有串口鼠标、USB 鼠标及 PS/2 鼠标。

（1）串口鼠标：采用 RS-232C 标准接口进行通信。PC 上有 2 个串行通信接口 COM1 和 COM2。串口鼠标不需要专门的电源线，而是由 RS-232C 串行通信接口线路中的 RTS 提供驱动，SGND 作为地线，使用 TXD 发送数据，DTR 作为联络信号线。在串行通信鼠标控制板上配置有微处理器，其作用是判断鼠标是否启动工作，工作时组织输出 X、Y 方向串行位移数据。大多数鼠标采用 7 位数据位、1 位停止位，无奇偶校验方式，以 1 200～2 400b/s 的速率发送数据。

（2）USB 鼠标：选择 USB 接口的鼠标要考虑主机上是否具有空余的 USB 接口。

（3）PS/2 鼠标：使用专用的鼠标插座（6 芯 DIN 型头），安装灵活方便，不占串口，但

应注意使用 PS/2 鼠标时主板上要有专门支持 PS/2 鼠标接口的插座。

3. 鼠标的编程应用

Microsoft 为鼠标提供了一个软件中断指令 INT 33H，只要加载了支持该标准的鼠标驱动程序，在应用程序中就可直接调用鼠标器进行操作。INT 33H 有多种功能，可通过在 AX 中设置功能号来选择。INT 33H 功能调用如表 13-3 所示。

表 13-3 INT 33H 功能表

功　能	入口参数	出口参数	参数描述
AX=0H；初始化鼠标		AX、BX	AX=-1，已安装鼠标；AX=0，未安装鼠标；BX=按键数目
AX=01H；显示鼠标			
AX=02H；关闭光标			
AX=03H；读取按键状态及光标位置		BX(CX,DX)	B0，B1，B2 表示左，右，中按键，B1=1 表示某键按下，光标位置(X,Y)
AX=04H；设置光标位置	(CX,DX)		光标位置(X,Y)
AX=05H；读取按键按下信息	BX	AX、BX(CX,DX)	B0，B1，B2 表示左，右，中按键，A0，A1，A2 表示左，右，中按键状态按键按下次数、光标位置(X,Y)
AX=06H；读取按键释放信息	BX	AX、BX (CX,DX)	B0，B1，B2 表示左，右，中按键，A0，A1，A2 表示左，右，中按键状态、按键按下次数、光标位置(X,Y)
AX=07H；设光标横向移动范围	CX,DX		CX=X 最小值；DX=X 最大值
AX=08H；设光标纵向移动范围	CX,DX		CX=Y 最小值；DX=Y 最大值
AX=09H；定义图形光标形状	(BX,CX) EX:DX		光标基点 X，Y 坐标；光标图案首址（0～1FH:背景；20H～3FH:图案）
AX=0BH；读鼠标位移量	CX,DX		分别是 X 方向和 Y 方向的位移量（-32 768～32 767），单位 0.127mm

13.3 显示器及接口

显示器是计算机系统中最重要的部件之一，用来显示字符、图形和图像。它既可作为计算机内部信息的输出设备，又可与键盘配合作为输入设备。根据显示原理的不同，目前至少已有 6 种类型的显示器：阴极射线显示器（CRT）、发光二极管显示器（LED）、液晶显示器（LCD）、等离子显示器（PDP）、电子发光显示器（EL）和真空荧光显示器（VFD）。

在单片机或单板机上一般都使用 LED 七段数码管或 LCD 液晶数码管显示数据，大部分台式微机都使用 CRT 显示器，便携式微机则使用 LCD 显示器。这里主要介绍 CRT、LCD 和 LED 的工作原理。

13.3.1　CRT 显示器

1. CRT 显示器性能指标

CRT 显示器是目前台式机中最常用的显示设备，显示器的性能可通过下面列出的有关技术指标反映出来。

（1）尺寸：是指显示器屏幕的对角线的长度。如 17 英寸显示器，不但可以在文字编辑时看到整页的编辑区，减少频繁的切换，观察大幅面的工程图纸，而且提供了更好的显示特性。

（2）分辨率：是指整个屏幕每行每列的像素数，每帧画面的像素数决定了显示器画面的清晰度。分辨率可用水平显示的像素个数×水平扫描线数表示，如 1 024×768 指每帧图像由水平 1 024 个像素、水平方向 768 条扫描线组成。分辨率越高，屏幕上能显示的像素个数也就越多，图像也就越细腻。但分辨率受到点距和屏幕尺寸的限制。屏幕尺寸相同，点距越小，分辨率越高；行扫描频率越高，分辨率相应地也就越高。要注意显示器的分辨率不仅和显示器有关，而且还决定于显示器配套的显示卡。

（3）点距：指显示器荧光屏上两个相邻的相同颜色磷光点之间的对角线距离，如图 13-5 所示。显像管荧光屏的点距决定了屏幕的最高分辨率，点距越小，分辨率越高。点距越小，意味着单位显示区域内可以显示更多的像素点，显示的图像就越清晰细腻。现在多数彩色显示器的点距为 0.28mm，高档显示器的点距只有 0.26mm 或 0.25mm。当然，点距越小价格也就越高。

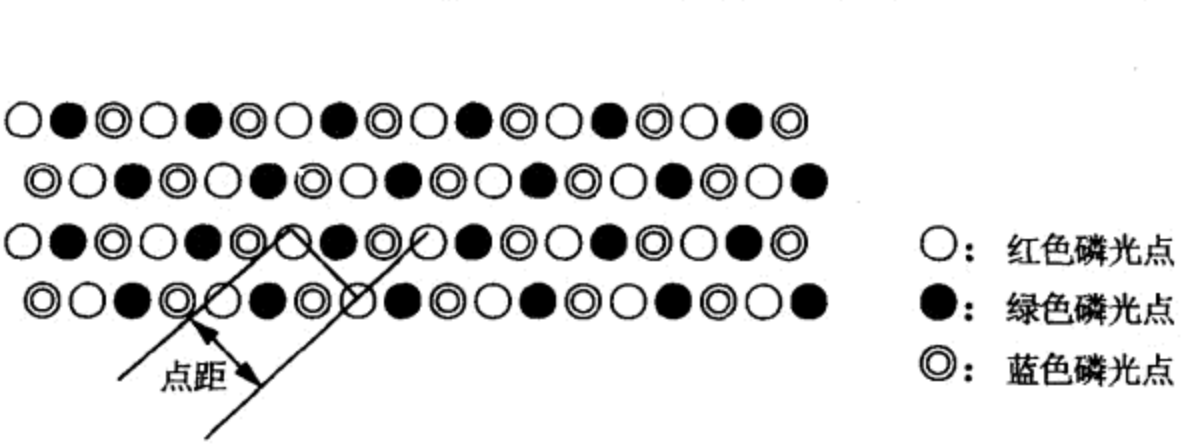
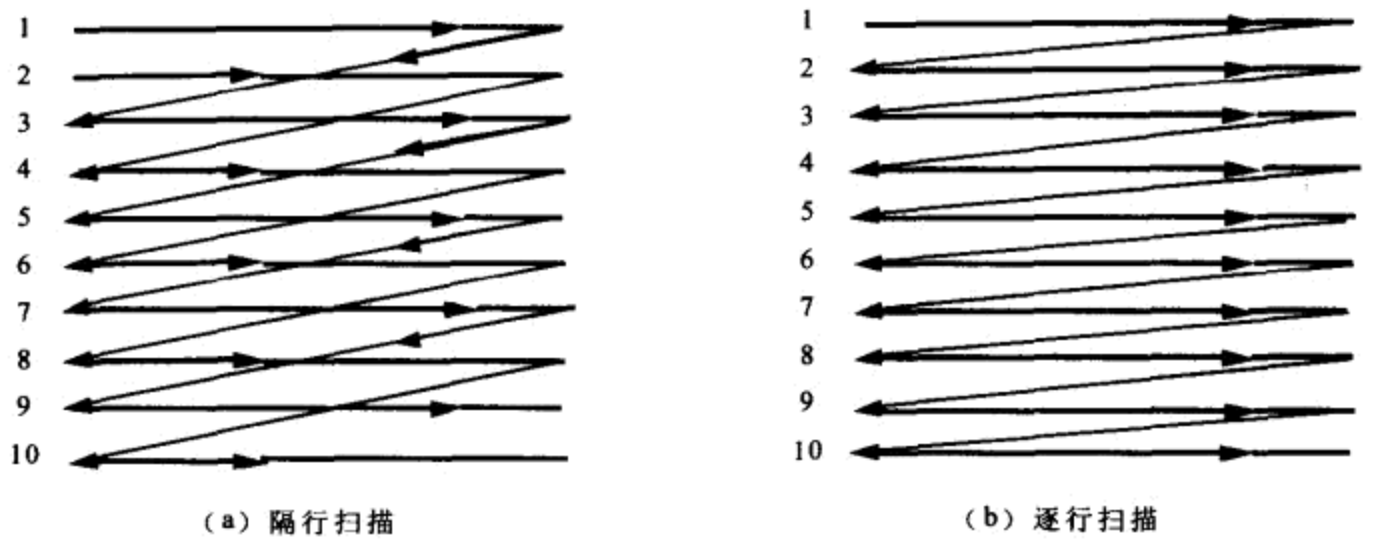


图 13-5　点距示意图

（4）垂直扫描频率：又称场频或刷新领率，指显示器在某一显示方式下所能完成的每秒从上到下刷新的次数，单位为 Hz。垂直扫描频率越高，图像越稳定，闪烁感越小。显示器使用垂直扫描频率在 60Hz～90Hz 之间，72Hz 以上的刷新频率下闪烁感明显减少，较好的彩显垂直扫描频率可达 100 Hz。

（5）水平扫描频率：又称行频，指电子束每秒在屏幕上水平扫描的次数，单位为 kHz。行频的范围越宽，可支持的分辨率就越高。如 14 英寸彩色显示器的行频通常为 30kHz～50kHz，15 英寸彩色显示器的行频范围在 30kHz～70kHz 之间。

（6）隔行扫描和逐行扫描：水平扫描有隔行扫描和逐行扫描方法。现在一般显示器都采用逐行扫描法。隔行扫描法是电子枪先扫描奇数行，后扫描偶数行，由于一帧图像分两次扫描，所以屏幕有闪烁现象。逐行扫描指逐行一次性扫描完组成一帧图像，如图 13-6 所示。逐行扫描最佳无闪烁的标准是垂直频率为 85Hz。



（a）隔行扫描　　（b）逐行扫描

图 13-6　隔行扫描和逐行扫描示意图

（7）视频带宽：视频带宽是指每秒钟扫描过的总像素数，等于水平分辨率×垂直分辨率×刷新频率。

2. CRT 显示器的基本结构

CRT 显示器主要由阴极射线管（电子枪）、视频放大电路和同步扫描电路 3 部分组成。彩色 CRT 组成结构如图 13-7 所示。

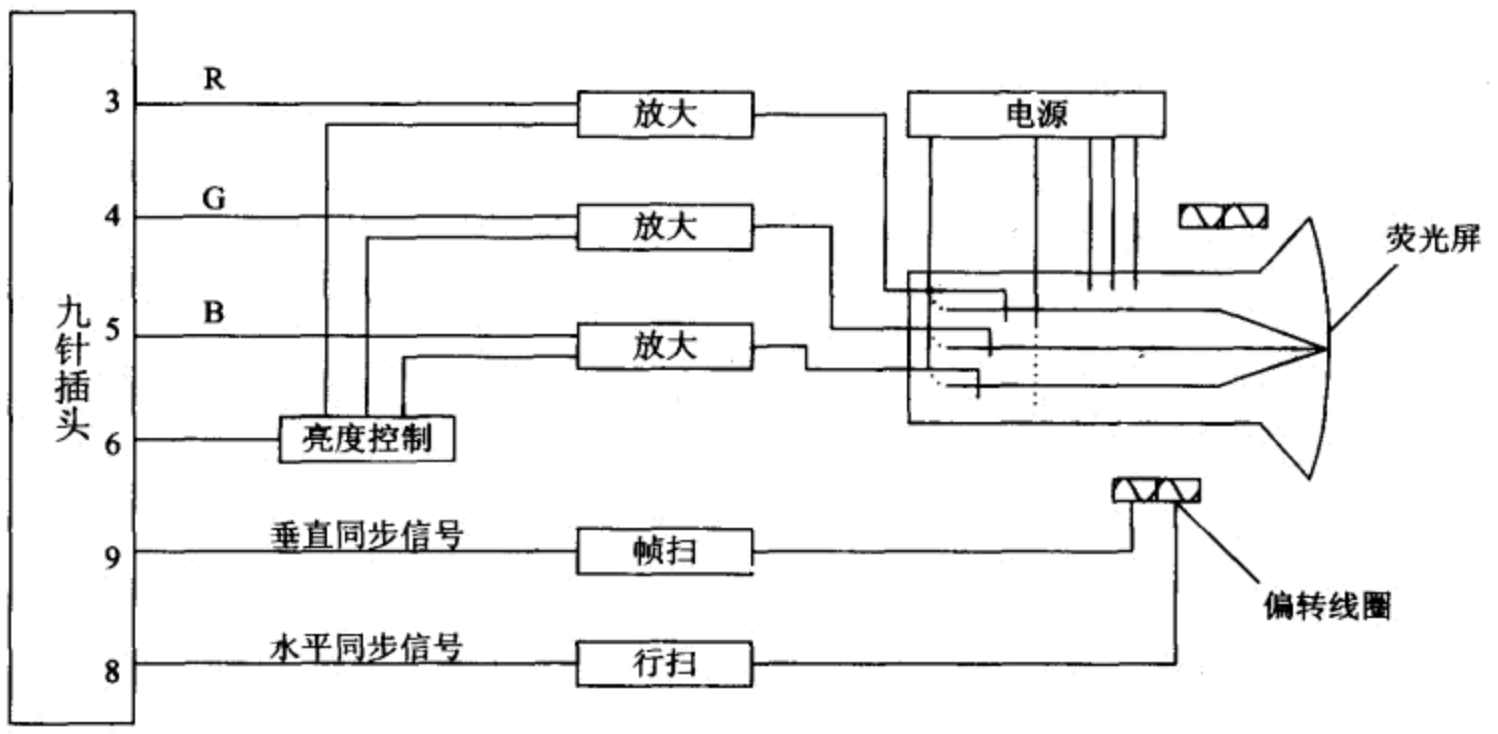


图 13-7　彩色 CRT 组成结构图

（1）阴极射线管：阴极射线管的灯丝加热后，由视频信号放大驱动电路输出的电流驱动阴极，使之发射电子束，俗称“电子枪”。彩色 CRT 由红、绿、蓝三基色的阴极发射的三色电子束，经栅极、加速极和聚焦极，并在高压极的作用下，形成具有一定能量的电子束射向荧光屏。在垂直偏转线圈和水平偏转线圈经相应扫描电流驱动产生的磁场控制下，三色电子束会被聚到荧光屏内侧金属荫罩板上的某一小孔中，并轰击荧光屏的某一位置。此时，涂有荧光粉的屏幕被激励而出现红、绿、蓝三基色之一或由三基色组成的其他各种彩色点。

荧光屏的发光亮度随加速级电压增加而增加。三基色阴极由外界 3 个相应的视频信号放大驱动电路所驱动，因此，改变三基色的组合状态可获得色彩的变化。于是，利用三基色 R、G、B 和加亮 I 的组合可得到 16 种色彩，如表 13-4 所示。

表 13-4　16 种色彩的组合选择

色　彩	R	G	B	I	色　彩	R	G	B	I
黑	0	0	0	0	灰	0	0	0	1
蓝	0	0	1	0	浅蓝	0	0	1	1
绿	0	1	0	0	浅绿	0	1	0	1
表	0	1	1	0	浅青	0	1	1	1
红	1	0	0	0	浅红	1	0	0	1
品红	1	0	1	0	浅品红	1	0	1	1
棕	1	1	0	0	黄	1	1	0	1
白	1	1	1	0	强白	1	1	1	1

（2）视频放大驱动电路：视频显示接口通过 9 针连接器的引脚 3～6 将 R、G、B、I 信号送到相应的放大驱动电路。R、G、B 放大驱动电路都是相同的，采用前置放大和末级功率推挽电路输出电流驱动阴极。此外，这 3 个驱动电路的输出还送到一个白色调整电路去平衡。因为，当 R、G、B 均为 1 时，出现的白色并非各占三分之一，而要经过白色平衡调整，使 R 占 30%、G 占 59%、B 仅占 11%。

I 信号驱动电路的输出可以控制上述 3 个前置放大级的基极驱动电流。当 I=0 时，前置放大级的驱动能力下降，使末级推挽驱动强度减弱，形成 8 种正常色彩；反之，当 I=1 时，驱动强度增强，形成另外 8 种加亮色彩。

（3）同步扫描电路：该扫描电路接收来自视频显示接口的垂直同步 VSYNC 和水平同步 HSYNC 信号，经各自的振荡电路和输出电路的控制，最终产生垂直锯齿波扫描电流 VDY 与水平锯齿波扫描电流 HDY，分别驱动相应的偏转线圈，使电子束在偏转磁场的作用下进行有规律的扫描。

3. 视频显示原理

显示器上图像的显示实际上是在光栅扫描的过程中将图像信号分解成按时间分布的视频信号去控制电子束在各条光栅位置上点的亮度和色彩。为使图像稳定且不消失，必须确保视频信号发送规律在时间上与水平和垂直同步扫描电流保持一致，同时，要把一帧图像存放在显示缓存中，以帧频的速率用缓存的内容刷新屏幕。

显示器可以实现字符和图形两种显示方式，无论哪一种方式，都要求将视频信息存储到视频显示缓冲区 VRAM 中。

（1）字符显示：采用的方法是把屏幕划分为许多小方格（字符窗口），要显示的字符就位于字符窗口中，并以 ASCII 码形式存放在 VRAM 中，每个字符占用一个单元，如果一个屏幕上字符窗口的数目为 80 列×25 行，那么这个显示缓存至少有 80×25B=2KB。显示 80 列×25 行的字符，行号为 0～24，列号为 0～79。假设显示缓存的起始地址为 A，则 I 行 J 列位置字符的代码存放在地址为 A+(I×80+J)的单元中。例如，起始地址为 C000H，则 2 行 10 列字符代码的存放位置为 C0AAH。

由于 VRAM 中存放的是字符的 ASCII 码，显示字符用的是点阵，这就需要把字符的 ASCII

码转换为点阵信息，由字符发生器来完成。如图 13-8 所示就是字符 Z 对应的 8×8 点阵示意图，黑点为显示的亮点，空白为显示的暗点，右侧和最下方为暗点，以便和上下左右的其他字符分开。

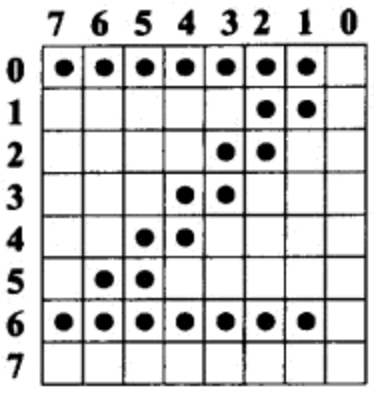


图 13-8　字符点阵示意图

一个 8×8 点阵的字模需要 8 个字节。所有可能显示的字符点阵放在一起构成字符发生器。字符发生器其实就是一个 ROM，工作时以字符编码为索引，找出相应字符点阵的编码，以并行方式输出。由于显示是以横向单向扫描为基础，所以并行输出的点阵必须按横向打点的速率变成串行方式，由寄存器完成，工作过程如图 13-9 所示。

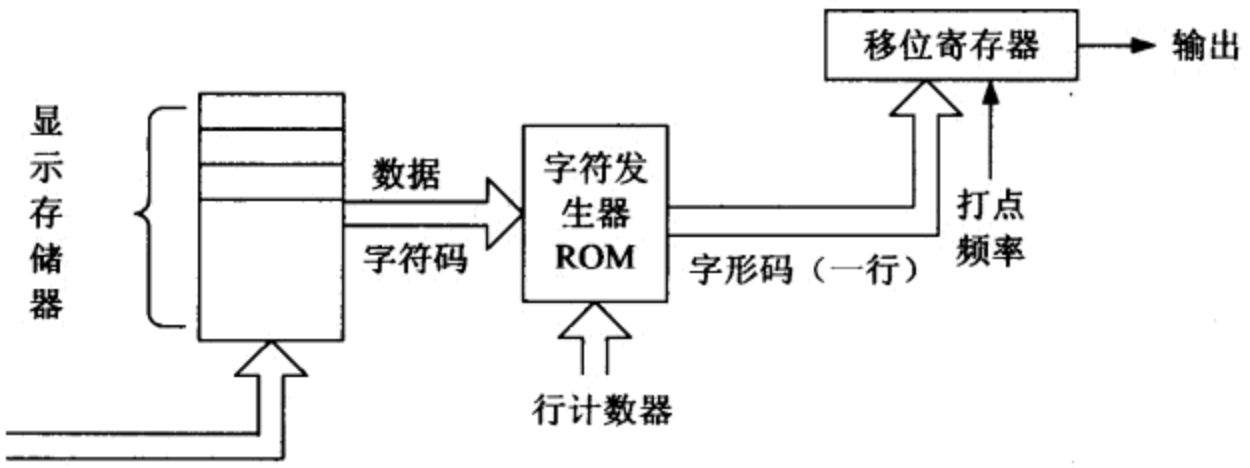


图 13-9　字符显示的工作过程

（2）图形显示及汉字显示：在光栅扫描 CRT 中，不管哪种显示都是利用某些像素亮、一些像素不亮的组合来表示不同信息的。字符显示中组合的类型十分有限。图形显示可能出现的像素组合几乎是无限的，比如，它可以画各种几何图形，可以画各种不规则的图形等。

图形显示原理的示意如图 13-10 所示。

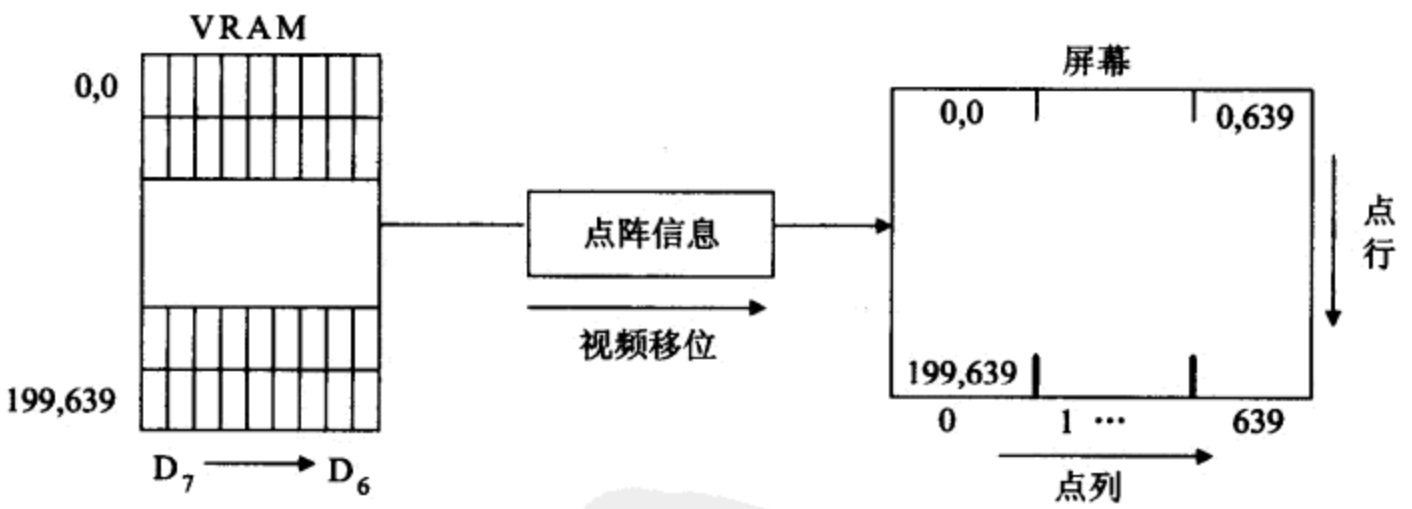


图 13-10　图形显示原理示意图

显示缓存中的每一个存储位对应 CRT 屏幕上的一个像素点，规定对应存储“1”表示该像素点亮，存储“0”表示像素点暗。按照这个规律，使显示缓存中的位与 CRT 上像素排列一致，就能控制 CRT 显示器上像素的亮与暗。当然也可以用显示缓存中的多个二进制位对应一个屏幕像素，那么多位组合所表示的状态可用来控制该像素点的亮度和颜色。

根据 CRT 像素多少和显示颜色数很容易计算出显示缓存的容量。比如，一个显示器分辨

率为 640×400，则它的显示缓存容量应为 640×400/8B=32KB。若一个显示器可显示 16 种颜色，即用 4 个二进制位来控制像素的显示，分辨率为 1 024×768，则它的显示缓存的容量为 1 024×768×4/8，约等于 400KB。

4. 视频显示标准

视频显示标准反映了各种视频显示图形卡的性能，或显示工作方式、屏幕显示规格、分辨率及显示色彩的种类。从 IBM 公司最早推出的视频显示标准 MDA 开始，陆续形成了一系列新的标准 CGA、EGA、VGA 和 TVGA 等。显然，在同样尺寸的显示器上，字符显示的列、行数越多，图形显示的分辨率越高，可显示的色彩种类越多，即表明相应的视频显示接口——图形显示卡的性能越好。

表 13-5 列出了视频显示标准。

表 13-5　视频显示一览表

方　式	模　式	字符规格	图形分辨率	颜 色 数	VRAM 起址	支持的视频标准
0,1	字符	40 列×25 行		16	B8000H	CGA/EGA/VGA/TVGA
2,3	字符	80 列×25 行		16		
4,5	图形	40 列×25 行	320×200	4		
6	图形	80 列×25 行	640×200	2		
7	字符	80 列×25 行		单色	B0000H	MDA/EGA/VGA/TVGA
13	图形	40 列×25 行	320×200	16	A0000H	EGA/VGA/TVGA
14		80 列×25 行	640×200	16		
15		80 列×25 行	640×350	单色		
16		80 列×25 行	320×350	16		
17	图形	80 列×30 行	640×480	2	A0000H	VGA/TVGA
18		80 列×30 行	640×480	16		
19		40 列×25 行	320×200	256		
50H	字符	80 列×30 行		16/256K	B8000H	TVGA
51H		80 列×43 行		16/256K		
52H		80 列×60 行		16/256K		
53H		132 列×25 行		16/256K		
54H		132 列×30 行		16/256K		
55H		132 列×43 行		16/256K		
56H		132 列×60 行		16/256K		
57H		132 列×25 行		16/256K		
58H		132 列×30 行		16/256K		
59H		132 列×43 行		16/256K		
5AH		132 列×60 行		16/256K		
5BH	图形	100 列×75 行	800×600	16	A0000H	TVGA
5CH		80 列×25 行	640×200	256		
5DH		80 列×30 行	640×480	256		
5EH		100 列×75 行	800×600	256		

续上表

方　式	模　式	字符规格	图形分辨率	颜 色 数	VRAM 起址	支持的视频标准
5FH	图形	128 列×48 行	1024×768	16		
60H		128 列×48 行	1024×768	4		
61H		96 列×64 行	800×600	16		
62H		128 列×48 行	800×600	256		

（1）MDA 标准：是单色字符显示接口。它只支持 25 行×80 列单色字符显示，不支持图形方式，仅在早期的 PC 中使用。

（2）CGA 标准：是彩色显示适配器。与 MDA 相比增加了彩色显示和图形显示两大功能，它支持字符、图形两种方式，但分辨率不高，颜色种类较少，是最早的显示卡产品，目前已不使用。

（3）EGA 标准：为增强型显示适配器，其字符、图形功能比 CGA 卡有较大提高，显示分辨率也较高，显示方式也比 CGA 卡丰富，有 11 种标准模式。

（4）VGA 标准：是视频图形阵列彩色显示接口。颜色可达 256 色。 当分辨率为 800×600 时，可显示 16 色；当分辨率为 1 024×768 时，可显示 8 色。VGA 卡兼容了上述各种显示卡的显示模式，支持更高的分辨率和更多的颜色种类。

（5）SVGA 标准：是超级 VGA。它是 VESA 所推荐的一种比 VGA 更强的显示标准。SVGA 的标准模式是 800×600，新型显示器分辨率可达 1 280×1 024、1 600×1 200 等。

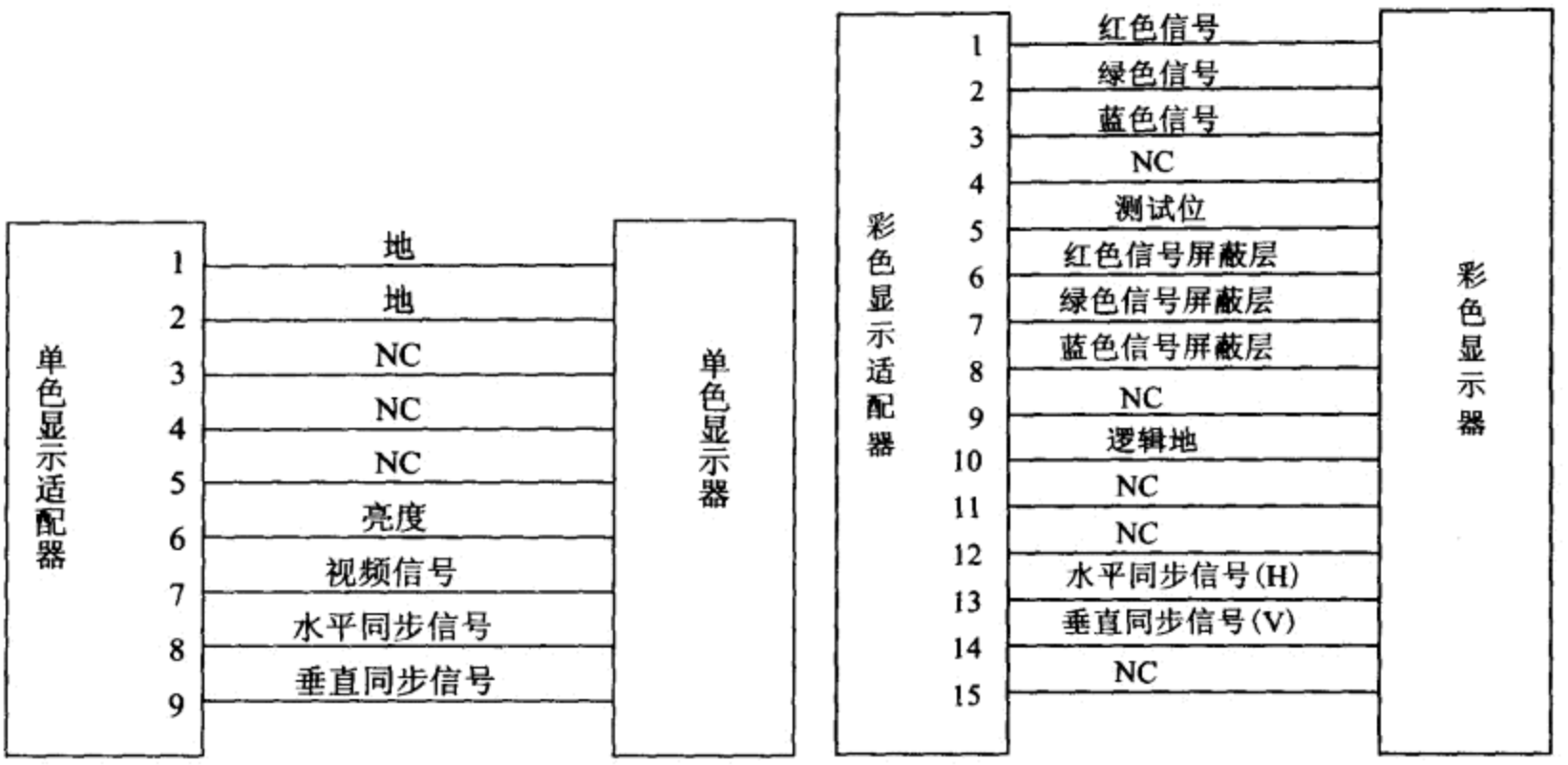
（6）TVGA 标准：是全功能视频图形阵列显示接口，它兼容 VGA 全部显示标准，并扩展了若干字符显示和图形显示的新标准，具有更高的分辨率和更多的色彩选择。当分辨率为 1 024×768 时，可显示高彩色或真彩色。

13.3.2 CRT 显示器接口电路编程方法

显示卡又称显示适配器，包括 3 个主要部分：视频控制器（Video Controller）、视频 BIOS（Video BIOS）和视频显示存储器（Video Display Area）。视频控制器是一片专用的大规模集成电路芯片，接受 CPU 指令的直接控制，产生监视器工作所需要的定时和控制信号，把显示存储器中的信息变换成显示屏面上的信息；视频 BIOS 是微机系统 ROM BIOS 的扩展部分，包含多种用户可以调用的显示子程序。需要在屏幕上显示什么实际上就是把所要显示的信息变换成相应的格式，写入显示存储器。

目前 CRT 显示器一般采用 9 芯或 15 芯 D 形插座与适配器连接，再通过适配器与主机连接。显示器与适配器之间的连接如图 13-11 所示。

显示适配器与 CRT 的接口按信号形式可分为数字信号接口和模拟信号接口。数字信号接口是指显示适配器送往 CRT 显示器的信号为数字信号，如前所述的彩色显示适配器。在模拟信号接口中，显示适配器只需 3 路信号线向 CRT 显示器传送表征显示颜色的模拟信号，幅值范围为 0～5V，不同的幅值表示不同的颜色深度。由于模拟信号的连续性，这 3 个信号可以组合出无限的颜色。目前 VGA 和 TVGA 显示系统采用的就是这种 15 针模拟信号接口。



（a）　单色显示器连接示意图　　（b）　彩色显示器连接示意图

图 13-11　显示器连接示意图

1. 单色显示适配器的 I/O 端口

IBM-PC 分配给单色显示适配器的输入/输出口地址为 3B8H、3BAH、3B4H 和 3B5H。对相应端口的操作可设定显示器工作方式以及工作参数。

控制寄存器用于控制显示器的工作方式，其端口地址为 3B8H。状态寄存器用来保存显示器的工作状态信息，其端口地址为 3BAH，CPU 可以用输入指令读取 CRT 状态字。

2. 彩色显示适配器

（1）工作方式控制：采用工作方式选择寄存器来实现。IBM-PC 的端口地址为 3D8H。在彩色显示适配器支持下，彩色图形显示器有以下两种工作方式。

- 字符方式：显示器以字符为单位在屏幕上显示字母和数字。在低分辨率显示方式下，屏幕可显示 40×25 个字符，在高分辨率显示方式下，屏幕可显示 80×25 个字符。
- 图形方式：该方式下通过控制各点的亮度或颜色显示出图形。IBM-PC 支持高分辨率和中分辨率两种图形显示方式。高分辨率方式每屏显示 640（列）×200（行）点，每点可取黑白两种颜色；中分辨率方式每屏显示 320（列）×200（行）点，每点可取两类 4 种颜色。

（2）显示缓冲区：为 16KB，IBM-PC 中，其起始地址为内存中的 B8000H。彩色字符方式下其显示缓冲区结构与单色显示缓冲区类似，字符的 ASCII 码保存于偶数地址单元，彩色属性字节保存于奇数地址单元。

（3）彩色选择：由彩色选择寄存器确定彩色图形显示器的色彩选择，IBM-PC 中其端口地址为 3D9H。

（4）色彩编码器：彩色选择寄存器的值和显示缓冲器内的彩色属性码送到色彩编码器，在色彩编码器内合成后，输出 R、G、B、I 控制信号，其中 R、G、B 信号控制彩色 CRT 的 3 个控制栅极；I 信号控制 CRT 的亮度。

3. 彩色显示适配器的 I/O 端口

彩色显示适配器包括 5 个 I/O 寄存器，它们分别是索引寄存器、数据寄存器、工作方式选择寄存器、彩色选择寄存器和状态寄存器。CPU 通过输入/输出指令访问这些寄存器，以实现对彩色图形显示器的控制。

索引寄存器和数据寄存器与单色显示适配器的作用相同，其端口地址分别为 3D4H 和 3D5H。工作方式选择寄存器和彩色选择寄存器如前所述，状态寄存器用于标志彩色图形显示器的工作情况。

PC 系列的 ROM BIOS 中有一组驱动 MDA 和 CGA 的显示 I/O 功能程序，显示器中断调用号为 10H，共有 17 个功能，方便用户编写有关显示器的接口程序，可参见附录中给出的 BIOS 中断调用。

下面用两个实例说明其应用情况。

【例 13.4】设置彩色图形方式，在屏幕中央显示一个带条纹的矩形。背景颜色设置为黄色，矩形边框设置为红色，横条颜色为绿色。

程序设计如下：

```
CODE    SEGMENT
        ASSUME  CS:CODE
START:  MOV  AH,0
        MOV  AL,4                    ;设置 320×200 彩色图形方式
        INT  10H
        MOV  AH,0BH
        MOV  BH,0                    ;设置背景颜色为黄色
        MOV  BL,0EH
        INT  10H
        MOV  DX,50
        MOV  CX,80                   ;行号送 DX,列号送 CX
        CALL LINE1                   ;调 LINE1,显示矩形左边框
        MOV  DX,50
        MOV  CX,240                  ;修改行号、列号
        CALL LINE1                   ;调 LINE1,显示矩形右边框
        MOV  DX,50
        MOV  CX,81                   ;置行号、列号
        MOV  AL,2                    ;选择颜色为红色
        CALL LINE2                   ;调 LINE2,显示矩形上边框
        MOV  DX,150
        MOV  CX,81
        CALL LINE2                   ;调 LINE2,显示矩形下边框
        MOV  DX,60
LP3:    MOV  CX,81                   ;置矩形内横线初始位置
        MOV  AL,1                    ;选择横条颜色为绿色
        CALL LINE2                   ;调 LINE2,显示绿色横线
        ADD  DX,10
        CMP  DX,150
        JB   LP3                     ;若行号小于 150,转 LP3 继续显示横线
        MOV  AH,4CH
        INT  21H                     ;否则返回 DOS
LINE1   PROC NEAR                    ;画竖线子程序
```

```
LP1:    MOV  AH,0CH              ;写点功能
        MOV  AL,2                ;选择颜色为红色
        INT  10H
        INC  DX                  ;下一点行号增 1
        CMP  DX,150
        JBE  LP1                 ;若行号小于等于 150,则转 LP1 继续显示
        RET
LINE1   ENDP
LINE2   PROC NEAR                ;画横线子程序
        MOV  AH,0CH
LP2:    INT  10H
        INC  CX                  ;下一点列号增 1
        CMP  CX,240
        JB   LP2                 ;若列号小于等于 240,则转 LP2 继续显示
        RET
LINE2   ENDP
CODE    ENDS
        END  START
```

【例 13.5】在屏幕上以红底蓝字显示“WOLRD”，然后分别以红底绿字和红底蓝字相间地显示“SCENERY”。程序段如下：

```
DATA    SEGMENT
        STR1   DB  'WORLD'
        STR2   DB  'S',42H,'C',41H,'E',42H,'N',41H
               DB  'E',42H,'R',41H,'Y',42H
        LEN    EQU $-STR2
DATA    ENDS
CODE    SEGMENT
        ASSUME   CS:CODE,DS:DATA,ES:DATA
START:  MOV  AX,DATA
        MOV  DS,AX
        MOV  ES,AX                    ;初始化
        MOV  AL,3
        MOV  AH,0                     ;设置 80×25 彩色文本方式
        INT  10H
        MOV  BP,SEG STR1
        MOV  ES,BP
        MOV  BP,OFFSET STR1           ;ES: BP 指向字符串首地址
        MOV  CX,STR2-STR1             ;串长度送 CX
        MOV  DX,0                     ;设置显示的起始位置
        MOV  BL,41H                   ;设置显示属性
        MOV  AL,1                     ;设置显示方式
        MOV  AH,13H                   ;显示字符串
        INT  10H
        MOV  AH,3                     ;读当前光标位置
        INT  10H
        MOV  BP,OFFSET STR2           ;ES: BP 指向下一个串首地址
        MOV  CX,LEN                   ;长度送 CX
        MOV  AL,3                     ;设置显示方式
        MOV  AH,13H                   ;显示字符串
        INT  10H
```

```
        MOV  AH,4CH
        INT  21H                    ;返回 DOS
CODE    ENDS
        END  START                  ;汇编结束
```

13.3.3 LED 与 LCD 显示

1. LED 显示器

LED（Light Emitting Diode）是一种重要的显示手段，可显示系统的状态、数字和字符。由于 LED 显示器的驱动电路简单，易于实现并且价格低廉，因此是比较常用和简便的显示器。

LED 是由半导体 PN 构成的固态发光器件，在正向导电时能发出可见光，常用的 LED 有红色、绿色、黄色以及蓝色。LED 的发光颜色与发光效率取决于制造材料与工艺，发光强度与其工作电流有关。发光时间常数约为 10μs～200μs，其工作寿命可长达 10 万小时以上，工作可靠性高。LED 具有类似于普通二极管的伏-安特性，在正向导电时其端电压近于恒定，通常约为 1.6～2.4V，工作电流一般为 10 mA～200mA。适合于与低电压的数字集成电路器件匹配工作。

（1）LED 显示器结构与原理

常用的七段 LED 显示器由 7 条发光线组成，按“日”字形排列，每一段都是一个发光二极管，这七段发光管可以称为 a、b、c、d、e、f、g，有的还带有小数点，如图 13-12（a）所示。

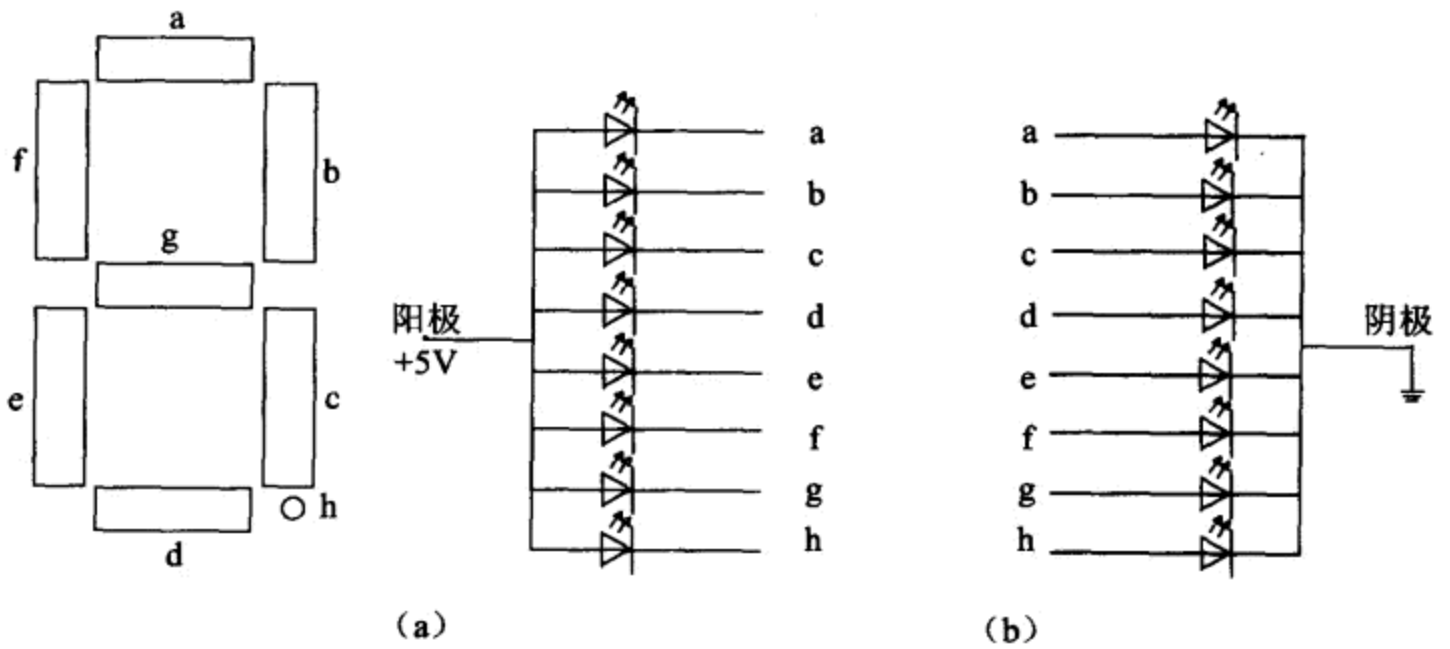


图 13-12　LED 显示块示意图

通过 7 个发光组的不同组合，可以显示 0～9 和 A～F 共 16 个字母数字。图中 h 段为小数点，若要显示小数点，则用 D_7 位控制。数字量和段的对应关系如下：

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
未用	g	f	e	d	c	b	a

表 13-6 列出了 7 段 LED 管显示 16 个十六进制数码对应的段码表。这种显示块有共阴极和共阳极两种，共阴极 LED 的发光二极管阴极共地，当某个二极管的阳极为高电平时，该发光二极管点亮，如图 13-12（a）所示；共阳极 LED 的发光二极管阳极并接，如图 13-12（b）所示。共阴极一般比共阳极亮，所以多数场合用共阴极。

表 13-6　7 段 LED 显示器字符段码表

显示字符	共阴极段码	共阳极段码	显示字符	共阴极段码	共阳极段码
0	3FH	C0H	8	7FH	80H
1	06H	F9H	9	6FH	90H
2	5BH	A4H	A	77H	88H
3	4FH	B0H	B(b)	7CH	83H
4	66H	99H	C	39H	C6H
5	6DH	92H	D(d)	5EH	A1H
6	7DH	82H	E	79H	86H
7	07H	F8H	F	71H	8EH

（2）LED 的显示方式

LED 显示器有静态和动态两种显示方式。

静态显示：该方式下，共阴极时将阴极连在一起接地，用“1”选通被显示的段；共阳极时将所有阳极连在一起接+5V 电压，用“0”选通即将显示的数码段。每个 LED 的段选线分别与一个 8 位并行相连，如图 13-13 所示。该图表示一个 4 位静态 LED 显示器电路。该电路每一位可独立显示，只要在该位的段选线保持段选码电平，就能保持相应的显示字符。由于每一位有一个 8 位输出口控制段选码，故在同一时间里每一位显示的字符可以各不相同。由于 N 位静态显示器要求有 N×8 根 I/O 口线，因此占用 I/O 资源较多。

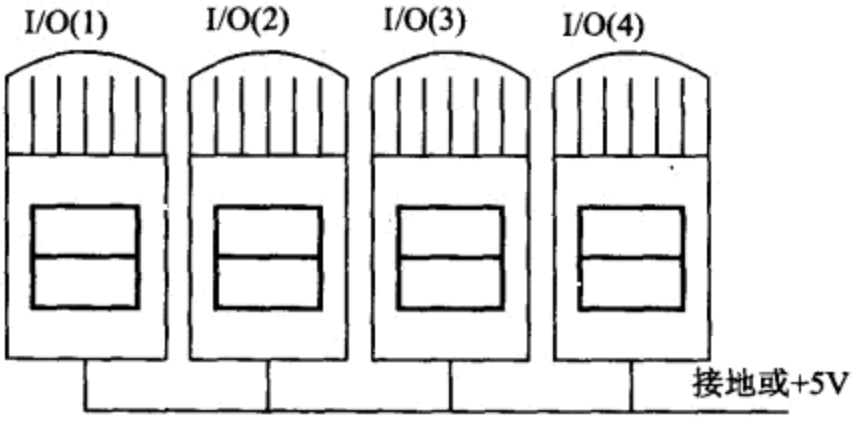


图 13-13　LED 静态显示连接图

动态显示：为了简化电路和降低成本，可将多位 LED 所有位的段选线并联在一起，由一个 8 位 I/O 端口控制，而共阴极或共阳极点分别由相应的 I/O 端口线控制。图 13-14 所示为 6 位 LED 显示连接示意图。

（3）LED 接口的编程应用

【例 13.6】某应用系统需要用 8 位 7 段 LED（共阴极）管显示 8 位的数据信息。被显示的内容由主程序存入显示缓存 DISPBUF 中，16 个数码 0～9 和 A～F 的显示码依次存放在 DISPCODE 数据区中，现要求利用 8255 完成动态显示，画出硬件连接图并给出软件程序。

分析：要完成动态显示就需要有 8 条段选线和 8 条位选线，因此需要 8255 提供两个 8 位端口，段选线提供显示码，位选线每一次选中一个 LED 显示管。具体硬件连接图如图 13-15 所示。8255 的端口地址为 200H～203H，主程序中完成 8255 初始化，将 A 口和 B 口定义为方式 0。

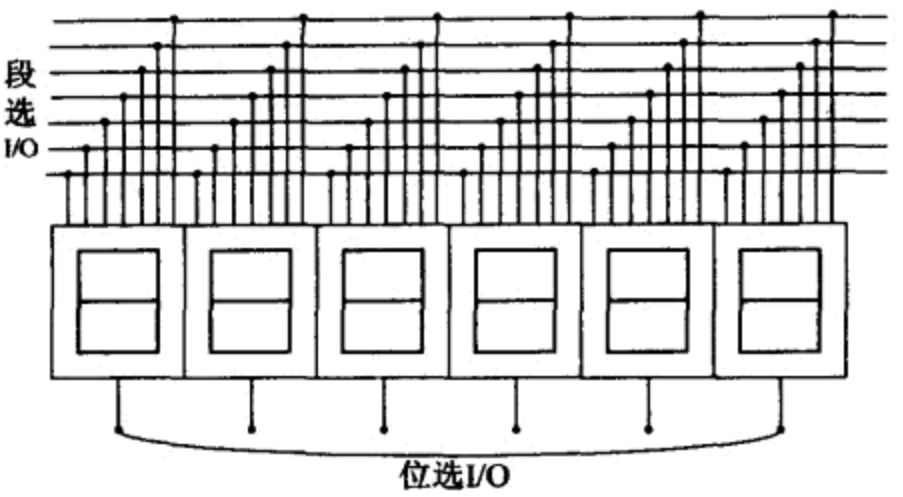


图 13-14　LED 动态显示连接示意图

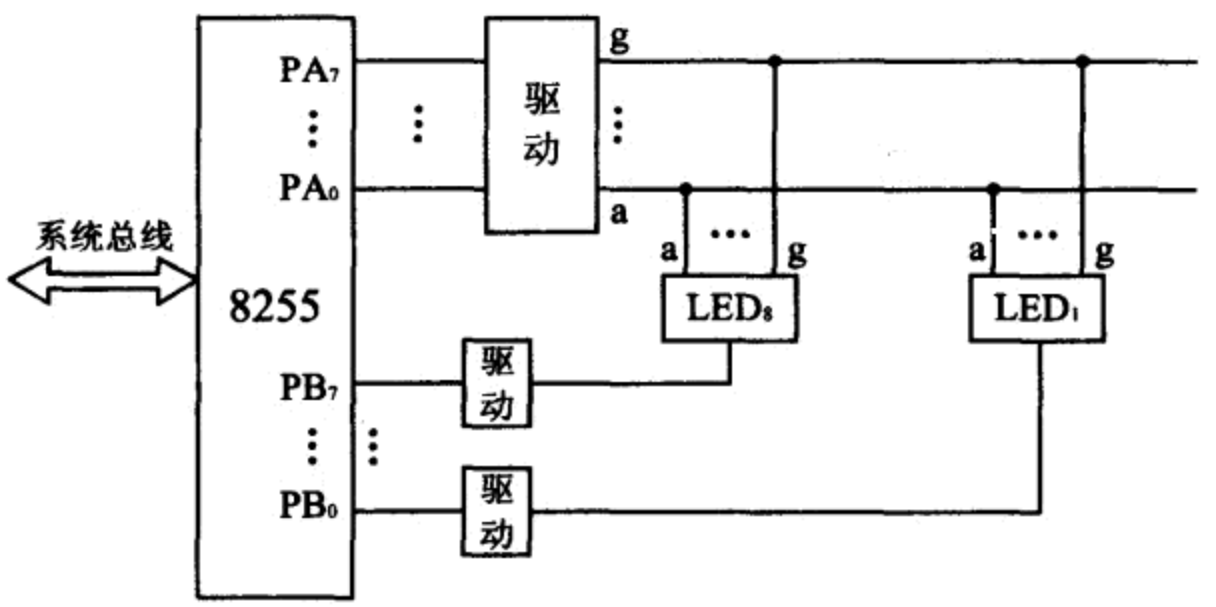


图 13-15　例 13.6 的系统硬件连接图

程序设计如下：

```
DATA    SEGMENT
        DISPCOEE    DB  40H,79H,24H,30H,19H,12H,02H
                    DB  78H,00H,18H,08H,03H,46H,21H
                    DB  06H,0EH,7DH,7FH,0CH
        DISPBUF     DB  8 DUP(?)
DATA    ENDS                          ;主程序定义显示缓冲区及字符段码表

;动态扫描
DISP    PROC   FAR
        LEA    BX,DISPCODE
        LEA    SI,DISPBUF
        MOV    CX,8
        MOV    AH,80H
LP:     MOV    AL,[SI]
        XLAT
        MOV    DX,2C0H
        OUT    DX,AL
        MOV    AL,AH
        INC    DX
        CALL   NEAR PTR DLIMS
        INC    SI
        ROR    AH,1
```

```
        LOOP   LP
        RET
DISP    ENDP
```

2. LCD 液晶显示器

LCD 液晶显示器特别是点阵式液晶已经成为现代仪器仪表用户界面的主要发展方向。其特点是体积小、外形薄、重量轻、功耗小、低发热、工作电压低（1.6V～6V）、无污染、无辐射、无静电感应，尤其是视阈宽、显示信息量大、无闪烁，并能直接与 CMOS 集成电路相匹配，同时，它是真正的“平板式”显示设备。

LCD 可以分为 4 种：无源阵列单色 LCD、无源阵列彩色 LCD、有源阵列单色 LCD、有源阵列彩色 LCD。下面简单介绍 LCD 显示器的基本技术指标和工作原理。

（1）LCD 的技术指标

速度：显示器的字符显示模式响应时间在 250～500ms 就可以满足要求。如使用鼠标驱动光标显示，在 175ms 以内才能防止拖影。对于实时视频显示则必须小于 50ms。目前，只有有源阵列 LCD 可以在 30～50ms 范围内，无源阵列 LCD 无法满足实时视频的要求。

对比度：是指开状态像素（白色）与关状态像素（黑色）亮度的比度，高对比度使文本更清楚，颜色更生动。典型的 CRT 显示器对比度是 245:1，LCD 显示器的对比度从最低 120:1 到最高 430:1。

亮度：LCD 采用背景照亮的方法，亮度取决于 LCD 的结构和背景照亮的类型。LCD 的亮度以 $1m^2$ 多少坎德拉(Candela)来计算，单位：cd/m^2。一般有源 LCD 可以达到 150～250cd/m^2 甚至更高，好的组合是高尼特率和高对比度。

视角：是指人观察显示器的范围，用垂直于显示器平面的法向平面的角度来度量。如±45° 的视角，表示人可以从法向量开始向上下左右任意方向的 0～45° 内均可观察到显示器的内容。由于 LCD 是背景照亮发光，所以其视角远小于 CRT 显示器。

（2）LCD 的显示原理

LCD 显示器的结构如图 13-16 所示。由于液晶的四壁效应，在定向膜的作用下，液晶分子在正、背玻璃电极上呈水平排列，但排列方向为正交，而玻璃间的分子呈连续扭转过度，这样的构造能使液晶对光产生旋光作用，使光偏振方向旋转 90° 。

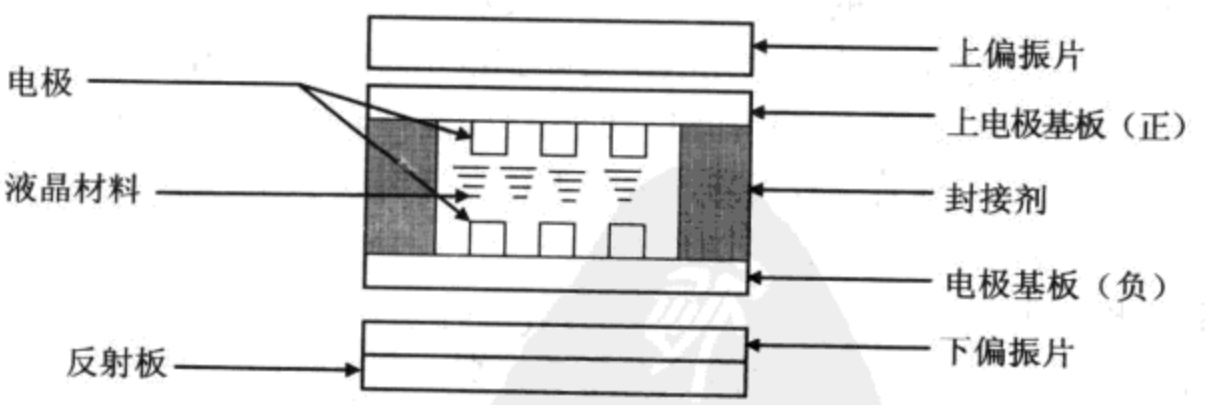


图 13-16　液晶显示器结构图

液晶显示器的工作原理为：外部光线通过上偏振片后形成偏振光，偏振方向成垂直排列。当偏振光通过液晶材料之后被旋转 90°，偏振方向成水平方向，此方向与下偏振片的偏振方向一致，因此光线能完全穿过下偏振片而达到反射极，经反射后沿原路返回，从而呈现出透

明状态。当液晶盒的上、下电极加上一定的电压后，电极部分的液晶分子转成垂直排列，从而失去旋光性。因此，从上振片入射的偏振光不被旋转，当此偏转光到达下偏振片时，因其偏振方向与下偏转片的偏振方向垂直，因而被下偏振片吸收，无法到达反射板形成反射，所以呈现出黑色。根据需要，将电极做成各种文字、数字或点阵，就可获得所需的各种显示。

LCD 显示屏是笔记本电脑的主要组件之一。随着液晶显示技术的迅速发展，各家厂商竞相推出各种专用的控制和驱动大规模集成电路 LSIC（Large Scale Integrated Circuit），使得液晶显示的控制和驱动极为方便，而且可由 CPU 直接控制，以满足用户对液晶显示的多种要求。

13.4 打印机接口

13.4.1 常用打印机及工作原理

1. 打印机概述

打印机是微型计算机系统中主要的硬拷贝输出设备，利用它可以打印数字、文字、字符和图形等。

（1）打印机的分类

按微型计算机接口方式可分为并行输出和串行输出打印机。前者通过并行接口与计算机连接，目前主要采用 CENTRONIC 标准总线，其打印数据传送速率高，但传送距离较短，一般不超过 2m；后者通过串行接口与计算机连接，目前主要采用 RS-232C 标准总线或 USB 接口连接，打印其数据传送速率低，但传送距离长，一般为 15～20m。

按打印机印字技术可分为击打式和非击打式：击打式又分为针式和字模式。针式打印机是以字符或图形点阵方式由打印头上的打印针在纸上打印小点，构成字符或图形；字模式打印机是利用刻有字符的韧带或印字轮，在运动中由打印锤隔纸打击钢带或印字轮，从而在钢带（或印字轮）与打印锤之间的纸上印出字符。非击打式常用技术有喷墨、激光、热转、喷蜡等。喷墨打印机是将墨粒子直接喷到纸上印出字符或图形；激光打印机是利用激光感光技术印出字符或图形；热转印字机是利用电流流过发热元件产生热量，使色带上的油墨溶化印出字符或图形；喷蜡印字机是将热融化的蜡喷射到纸上形成色点印出字符或图形。

按印字方式可分为行式和页式打印机：行式打印是打印针头在驱动机构的带动下从左向右往复运动，打印出一行或若干行点阵，由若干行点阵构成一行字符或图形；页式主要指非击打式打印，以页为单位印出字符或图形。

（2）打印机的主要技术指标

采用不同打印技术的打印机之间性能有很大差别，衡量打印机性能优劣的指标主要包括以下几个方面。

分辨率：用每英寸的点数（dpi）表示，决定了打印机的打印质量。要达到好的印刷质量，分辨率应在 400dpi 以上。一般针式打印机的分辨率为 180dpi，激光打印机可达 600dpi 以上。

打印速度：一般用每秒钟打印字数表示。页式打印机用每分钟打印页数表示。打印速度在不同的字体和文字下有较大差别，另外，不同的打印方式对打印速度影响较大。

行宽：也称为规格，是指每行中打印的标准字符数，可分为窄行和宽行。窄行每行打印标准字符 80 个，宽行每行可打印 120 个或 180 个标准字符。

除了以上主要技术指标之外，常考虑的还有功耗、稳定性和性能价格比等。

2. 打印机的工作原理

下面简要分析激光打印机的工作原理。激光打印机是通过激光技术和电子照相技术完成印字功能的，是一种高精度、高速度、低噪声的非击打式打印机。其工作原理如图 13-17 所示，主要由激光扫描系统、电子照相系统和控制系统 3 部分组成。

激光扫描系统的作用是使激光器产生的激光经调制后变成载有字符或图形信息的激光束，该激光束经扫描偏转装置在感光鼓上扫描，形成“静电潜像”。电子照相系统把带有“静电潜像”的感光鼓接触带有相同极性电荷的干墨粉，鼓面被激光照射的部位将吸附墨粉，来显影图像。该图像转印在纸上，经红外线热辐射定影后使墨分子渗透到纸纤维中。控制系统包括激光扫描控制、电子照相系统控制、缓冲存储器和接口控制等，其作用是完成接收和处理主机的各种命令和数据以及向主机发送状态。

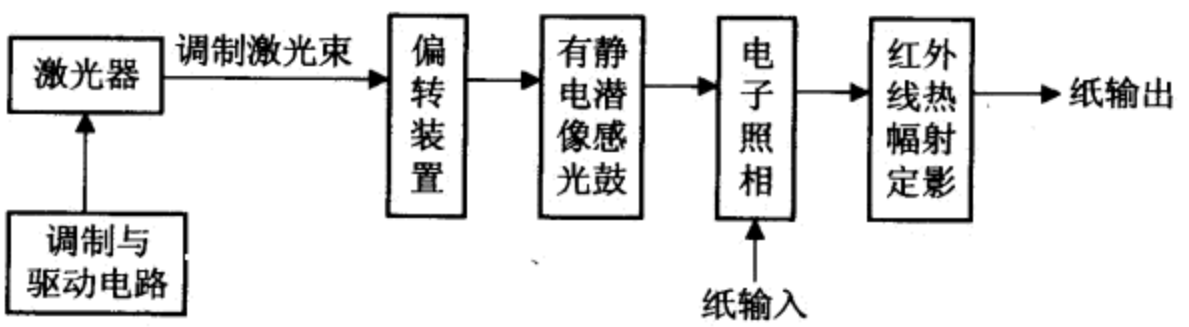


图 13-17　激光打印机工作原理

13.4.2　主机与打印机的接口

主机和打印机之间的数据传输既可用并行方式也可用串行方式。并行打印机采用 Centronics 并行接口标准，该标准定义了 36 脚插头，而 PC/XT 的并行接口采用 25 脚的插座，如图 13-18 所示。这也是目前大多数打印机采用的接口方式。打印机与主机之间通过电缆线连接，电缆线的 36 芯插座与打印机相连，25 芯与主机并行接口相连。

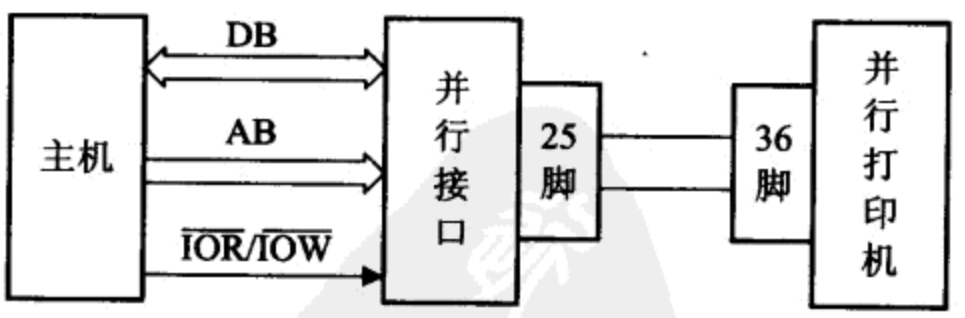


图 13-18　并行打印机信息传送示意图

采用 Centronics 并行接口标准定义的 36 脚插头座，其引脚的含义如表 13-7 所示。

表 13-7 并行打印机的接口信号

引 脚 号	引脚名称	方 向	操作功能
1	$\overline{\text{STROBE}}$	输入	用于打印机接收数据的选通信号，负脉冲有效，脉冲宽度在接收端应大于 0.5μs
2～9	$DATA_0$～$DATA_7$	输入	主机向打印机发出的 8 位并行数据信号线
10	$\overline{\text{ACK}}$	输入	打印机接收一个数据字节后就送回给主机的应答信号，负脉冲有效，表示打印机已准备好接收新数据
11	BUSY	输出	打印机回送给主机的忙信号，打印机为下列状态时为忙：接收数据；正在打印；脱机状态；打印机出错
12	PE	输出	纸用完信号，为高电平时说明打印机缺纸
13	SLCT	输出	选中信号，该信号为高电平表示处于联机选中状态
14	$\overline{\text{AUTOFEEDXT}}$	输入	自动走纸信号，有效时打印机打印后自动换行
16、17	SGND、CGND		信号地、机壳地
18	+5V	输出	电源
19～30	GND		双绞线信号返回地
31	$\overline{\text{INIT}}$	输入	初始化信号，低电平有效，打印机被复位成初始状态，打印机的数据缓冲区被清除
32	$\overline{\text{ERROR}}$	输出	出错信号，低电平表示打印机处于无纸、脱机或错误状态之一
33	GND		双绞线信号返回地
36	$\overline{\text{SLCTIN}}$	输入	选择信号，当信号为低电平时才能将数据输出到打印机

36 条信号线按功能可分为：8 条数据线、9 条控制和状态线，15 条地线、1 条+5V 电源线，其余 3 条不用。其中的 8 条数据线 $DATA_0$～$DATA_7$ 、打印机接收数据的选通信号 $\overline{\text{STROBE}}$ 、打印机回送给主机的忙信号 BUSY、打印机应答信号 $\overline{\text{ACK}}$ 以及地线是打印机和主机通信的基本信号线，它们是必不可少的，其他可视实际情况加以取舍。

13.4.3 打印机的中断调用

PC 系列的 ROM BIOS 中有一组打印机 I/O 功能程序，显示器中断调用号为 17H，共有 3 个功能，用户可利用中断调用很方便地编写有关显示器的接口程序，如表 13-8 所示。

表 13-8 打印机 BIOS 中断调用 INT 17H 功能表

AH 取值	操作功能	入口参数	出口参数
00H	打印一个字符，并回送状态字节	(AL)= 打印字符的 ASCII 码 (DX)= 打印机号	(AH)= 打印机状态字节
01H	初始化打印机，并回送状态字节	(DX)= 打印机号	(AH)= 打印机状态字节
02H	取打印机状态字节回送	(DX)= 打印机号	(AH)= 打印机状态字节

打印机的初始化指令序列如下：

```
MOV  AH,01H
MOV  DX,0
INT  17H
```

初始化操作时要发送一个换页符，所以该操作能把打印机头设置在页的顶部。大多数打印机只要一接通电源就会自动完成初始化操作。

13.5　其他外设简介

13.5.1　扫描仪原理及其应用

1. 工作原理

对大量的文档资料和一些精美的图画进行处理需要借助计算机的另一个重要的输入设备——扫描仪来完成。使用扫描仪可以将各种图像或文字输入到电脑中。

扫描仪是一种光、机、电一体化的电脑外设产品，它是将各种形式的图像信息输入到计算机中的一个重要工具，是除鼠标和键盘之外计算机使用最广泛的输入设备之一。

扫描仪的基本原理是通过传动装置驱动扫描组件（光电耦合器件——Charge Coupled Device，CCD），将各类文档、相片、幻灯片、底片等稿件经过一系列的光/电转换，最终形成计算机能识别的数字信号，再由控制扫描仪操作的扫描软件读出这些数据，并重新组成数字化的图像文件，供计算机存储、显示、修改、完善，以满足人们各种形式的需要，图 13-19 给出了图像扫描的过程示意图。

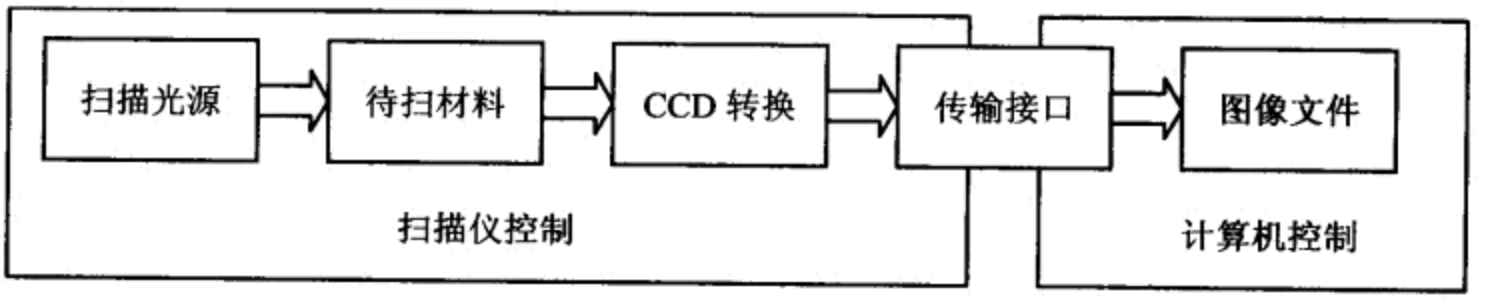


图 13-19　扫描仪扫描过程示意图

扫描仪由顶盖、玻璃平台和底座构成。玻璃平台用于放置被扫描图稿；塑料上盖内侧有一黑色（或白色）的胶垫，其作用是在顶盖放下时压紧被扫描文件，当前大多数扫描仪采用了浮动顶盖，以适应扫描不同厚度的对象。

透过扫描仪的玻璃平台，能看到安装在底座上的机械传动机构、扫描头及电路系统（电路板）。机械传动机构的功能是带动扫描头沿扫描仪纵向移动；扫描头的功能是将光信号转换为电信号；电路系统的功能是处理、传输图像。

当将被扫描图稿正面向下放置在玻璃平台上开始扫描时，机械传动机构带动扫描头沿扫描仪纵向移动，扫描头上光源发出的光线射向图稿，经图稿反射的光线（光信号）进入光电转换器被转换为电信号后，经电路系统处理后送入计算机。光电转换机构沿扫描头上横向放置，机械传动机构带动扫描头沿扫描仪纵向每移动一个单位距离，光电转换机构就采集扫描图稿上一条横线上的图形数据，当扫描头沿纵向扫过原稿以后，扫描仪就采集并传输了原稿上的全部图形信息。

目前，扫描仪作为计算机的重要输入设备，已被广泛应用于报纸、书刊、出版印刷、广告设计、工程技术、金融业务等领域之中。它以强大的功能，不仅能迅速实现大量的文字录入、计算机辅助设计、文档制作、图文数据库管理，而且能逼真实时地录入各种图像，特别是在网络和多媒体技术迅速发展的今天，扫描仪更能有效地应用于传真（配 Fax/Modem 卡）、复印（配打印机）、电子邮件等工作，依靠其他软件的支持，扫描仪还能用于制作电于相册、请柬、挂历等许多个性鲜明和充满乐趣的作品。随着计算机技术和应用的发展，扫描仪的应用领域还将得到进一步的拓展。

2. 扫描仪的主要技术指标

扫描仪的性能指标主要有分辨率、灰度级和色彩数，另外还有扫描速度、扫描幅面等。

（1）分辨率：体现了扫描仪对图像细节的表现能力，用每英寸长度上扫描图像所含有的像素点的个数来表示，即 DPI（Dot Per Inch）。分辨率又分为光学分辨率和最大分辨率，光学分辨率从根本上决定了扫描仪的档次，决定于扫描仪电荷耦合器（CCD）元件的数量和透镜的质量。

（2）灰度级：表示灰度图像的亮度层次范围，代表扫描仪扫描时由亮到暗的扫描范围大小。灰度级数越大，说明扫描生成图像的亮度范围越大；层次越丰富，扫描的效果也就越好。

（3）色彩数：表示彩色扫描仪所能产生的颜色范围，用每个像素点上颜色的数据位数（bit）表示。色彩数越多，扫描图像越逼真。比如 30bit，就是每个像素点上有 2^{30} 种颜色。

扫描速度通常用一定分辨率和图像尺寸下的扫描时间表示，扫描幅面表示可扫描图稿的最大尺寸，常见的有 A_4 和 A_3 幅面等。

13.5.2 数码相机原理及其应用

1. 数码相机的基本原理

数码相机与传统相机从外观上看区别不是很大，只不过大部分数码相机都有一个彩色液晶显示屏，两者最大的区别在于它们的内部结构和工作原理。数码相机不需要胶卷，在拍摄时图像被聚焦到 CCD 元件上，然后通过 CCD 将图像转换成许多的像素，以二进制数字方式存储于相机的存储器中。只要将存储器与电脑连接，即可在显示器上显示所拍摄的图像，并进行加工处理或打印机输出。数码相机基本结构如图 13-20 所示。

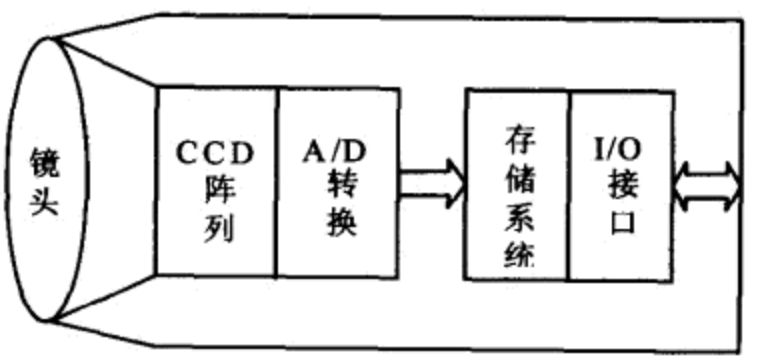


图 13-20 数码相机基本结构

数码相机主要由以下几部分组成。

（1）快门：用来控制相机的成像时间，通过调节快门的速度可以控制照片的曝光程度。

（2）镜头：是相机的光学部分，决定的相机的质量。数码相机通过 CCD 将由镜头取得

的光信号转换为数字信号，并用数字信号来记录图像。

（3）机身：用来承接数码相机的各个组成部件。

（4）取景器：用来帮助用户选取所要拍摄的景物。

（5）闪光灯：用来辅助照明，以达到所要求的拍摄效果。

2. 数码相机主要技术指标

（1）分辨率：分辨率的高低决定了数码相机拍摄的图像质量的高低。分辨率越高，图像越清晰，但生成的数据文件越大。

（2）存储媒体：数码相机存储媒体分为内置存储和可移动式存储媒体。

（3）感光度：感光度是感光材料、感光速度和感光灵敏度的度量方式，单位是 ISO。传统的相机使用的感光元件是胶卷，而数码相机使用的是 CCD 元件。

数码相机的技术指标还有很多，此处不再过多介绍。

3. 数码相机的应用特点

数码相机可将图像数字化，操作简便，特别是能与计算机直接连接，而且在计算机上利用丰富且强大的图像处理软件对图像做各种平面处理，得到更好的艺术效果，因此数码相机被广泛应用于各个领域。由于数码相机具有高速数据传输，大容量存储和快捷方便等特点，因此近几年在新闻摄影、网页制作、电子出版、广告设计等领域得到了广泛的应用。

13.5.3　触摸屏原理及其应用

触摸屏是通过触摸屏幕来进行人机交互的输入装置。在 CRT 屏幕上安装一层或多层透明感应薄膜，或在屏幕外框四周安装感应元件，再加上接口控制电路和软件之后，就可以利用手指或笔等工具，通过触摸屏幕直接向计算机输入指令或图文消息，使信息的输入变得非常方便。

触摸屏可分为红外线式、电磁感应式、电阻式、电容式及声控式等种类。作为一种定位装置，可将触摸点的坐标输入给计算机。以电阻式和红外式为例，前者是在屏幕上安装感应薄膜，触摸时，由于触摸点电阻受压发生阻值变化而感知触摸位置；后者在屏幕外框四周安装一系列 LED 元件及光敏元件，使得 CRT 表面形成一个纵横交错的光线网络。不触摸时，X、Y 方向的光线均不受阻；触摸时，则与触摸点坐标对应的 X、Y 方向上的光敏元件就会接收不到某束水平方向和某束垂直方向的光线，因此可以检测出触摸位置。电阻式的分辨率较高，价格也较贵；红外线式分辨率虽不高，但较耐用，售价低。

触摸屏由触摸检测装置，接口控制逻辑及控制软件等部分组成。接口控制器有的放在 CRT 内部，有的在 CRT 外部或插在主机箱内，通过 RS-232 串口与主机通信。其主要功能是：

（1）检测并计算触摸点的坐标，经缓冲后送给主机。

（2）接收和执行主机的命令。一般包括设定触摸模式（触入时数据有效，离开时数据无效，在行列位置信号变化的上沿或下沿报告，定时报告或连续报告坐标信息等触摸模式），设定行工作模式，设置屏幕窗口。

（3）触摸屏一般都提供一个标准程序，可交互地定义显示区的尺寸和位置，进行有效位置的校准或其他控制。

本章小结

本章介绍了常见外部设备的基本结构和工作原理、与 CPU 的连接方法、通过中断调用进行控制的编程方法等。

计算机系统中的键盘、鼠标、CRT 显示器和打印机是必备的人机交互设备，它们能够完成各种常规信息的输入和输出。键盘工作时要完成键开关状态的可靠输入、键的识别和将键值送给计算机等 3 项任务；鼠标是一种快速定位器，可方便地将光标准确定位在要指定的屏幕位置，是计算机图形界面人机交互必不可少的输入设备；显示器是计算机中用来显示各类信息以及图形和图像的输出设备，CRT 显示器一般采用 15 芯 D 形插座作为与 CPU 联系的接口电路，再通过显示卡与主机连接。

打印机也是常用的输出设备，它将计算机中的各类信息打印到纸上，可以长期保存。目前常见的打印机有针式打印机、喷墨打印机和激光打印机 3 类，其中激光打印机的打印速度可达每分钟 2 000 行，是目前打印机中速度最快的一种；扫描仪是采用光、机、电一体化的计算机外设产品，是除鼠标和键盘之外计算机使用最广泛的输入设备，它不仅能迅速实现大量的文字录入，而且能实时录入和处理各种图像信息，目前应用最多的是平台式 CCD 扫描仪。

思考与练习题

1. 非编码键盘一般需要解决几个问题？识别被按键有哪几种方法，各有什么优缺点？
2. 与 PC 键盘发生关联的是哪两类键盘程序，它们各自的特点是什么？
3. 试简单说明 CRT 显示器的工作原理。
4. 什么叫光栅扫描？在光栅扫描中，电子束受到哪些信号的控制？
5. 在字符型显示器上，如果可以显示 40×80 个字符，显示缓存容量至少为多少？
6. 一个分辨率为 1 024×768 的显示器，每个像素可以有 16 个灰度等级，那么相应的缓存容量应为多少？
7. 概述打印机的分类，用于评价打印机性能的有哪些指标？
8. 常见打印机接口有哪几种工作方式？说明并行打印机有哪些接口信号，怎样与主机进行连接，信号如何传递？
9. 简要论述数码相机和扫描仪的工作原理及应用特点。

第 14 章 D/A 及 A/D 转换器

A/D、D/A 转换器是微机测控系统中的重要组成部件，在工程实践中有广泛的应用。本章主要讲解 A/D、D/A 转换器的内部结构，转换原理及外部引脚特性和编程方法。

通过本章的学习，读者应该掌握 D/A、A/D 转换器的基本概念及编程技术，熟悉 D/A、A/D 转换器的原理及特点，掌握 D/A、A/D 转换器的选用方法。

14.1 D/A 及 A/D 转换器概述

随着数字技术的飞速发展，计算机已广泛应用于自动控制和测量领域中，在一般的控制系统中，传感器所检测的信号是模拟量，如温度、压力、流量、速度、湿度等。而计算机所能处理的是数字量，不能直接处理模拟量，因此必须把模拟量通过 A/D 转换器转换成数字量送入计算机进行处理，即模/数转换。同样，经过计算机处理后的数字量所产生的结果依然是数字量，要进行对外部设备的控制必须转换成模拟量，即通过 D/A 转换将数字量转换成模拟量，完成数/模转换的过程。D/A 转换器的作用是把数字量转换成模拟量，A/D 转换器的作用是把模拟量转换成数字量。

一般微机测控系统框图如图 14-1 所示。

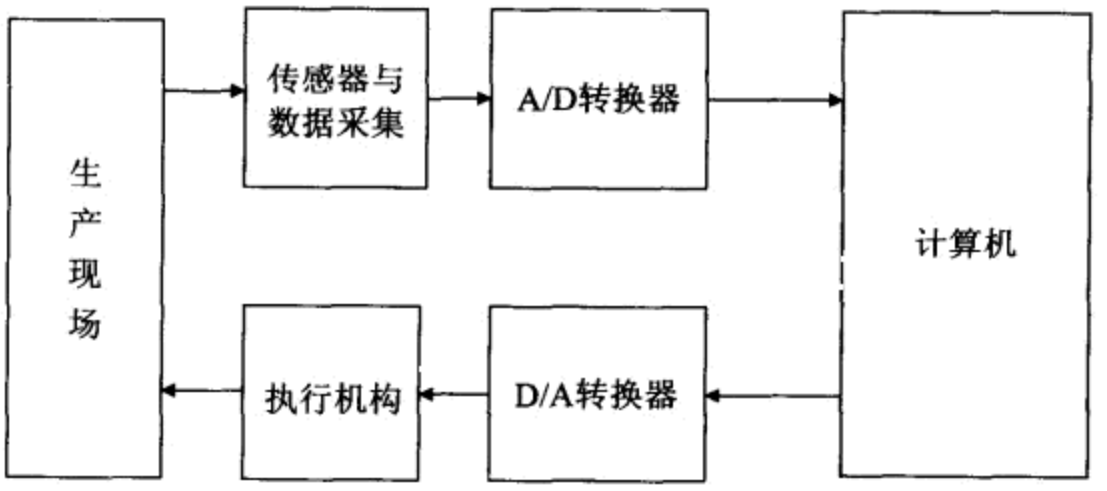


图 14-1 微机测控系统框图

A/D、D/A 转换器在微机控制系统中应用非常广泛。A/D 转换器属微机控制系统的前向通道，D/A 转换器属微机控制系统的后向通道。A/D、D/A 转换器都有串行输入（如 TLV5615 MAX525）和并行输入（如 DAC0832）两种形式，串行输入转换器体积小，与微机接口简单，速度比并行慢。

A/D、D/A 转换器的位数不同，其分辨率也不同，应用较多的有 8 位、10 位和 12 位转换器。按 D/A 转换器的输出方式有电流输出型（如 DAC0832，AD7502 等）和电压输出型（如 AD558、AD3860 等）。其中电流输出型建立时间快，通常为几十纳秒。而电压输出型则需要几百纳秒。

此外，还有许多特殊的 D/A 转换器，如 4mA～20mA D/A 转换器，多通道 D/A 转换器等，以满足各种不同系统的要求。

14.2 典型 D/A 转换器芯片

14.2.1 D/A 转换器工作原理

D/A 转换器是指将数字量转换成模拟量的集成电路，它的模拟量输出（电流或电压）与参考量（电流或电压）以及二进制数成比例。一般来说，可用下面的式子表示模拟量输出和参考量及二进制数的关系：$X = K \times V_{REF} \times B$。

其中 X 为模拟量输出，K 为比例常数，V_{REF} 为参考量（电压或电流），B 为待转换的二进制数，通常 B 的位数为 8 位、12 位等。

通过 D/A 转换器进行 D/A 转换就是按照一定的解码方式将数字量转换成模拟量。解码方式主要有以下两种解码网络方式。

1. 二进制加权电阻网络型 D/A 转换器

二进制加权电阻网络型 D/A 转换器由以下 4 部分组成。

- 电子开关 $k_1 \sim k_n$。
- 产生二进制权电流（电压）的电阻网络。
- 加法放大器。
- 参考电压。

二进制加权电阻网络型 D/A 转换器的结构原理如图 14-2 所示。

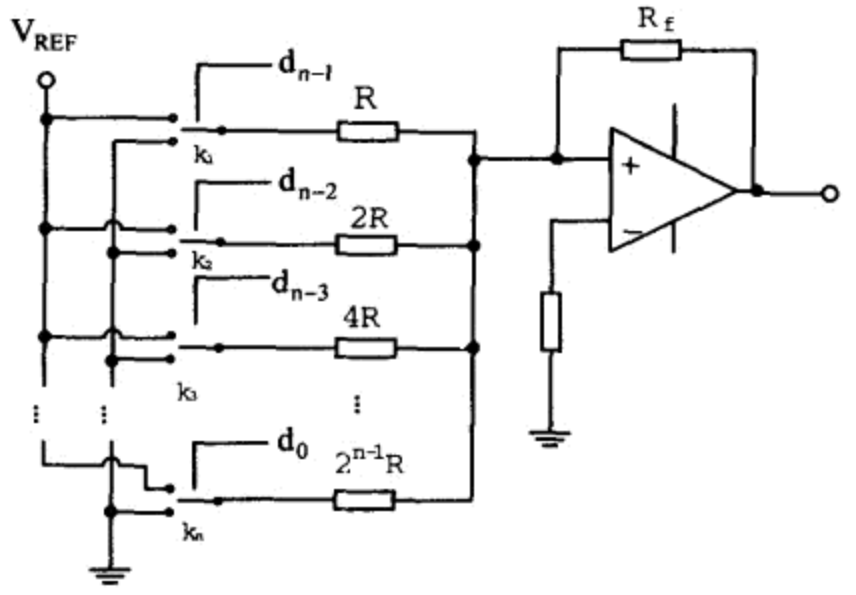


图 14-2 二进制加权电阻网络型 D/A 转换器的结构原理

工作原理是：每一位电子开关由相应位的二进制数控制，当控制某一个电子开关的二进制数（$d_0 \sim d_{n-1}$）为 1 时，相应电子开关从接地方式转到闭合方式，参考电压接入电阻网络，该位权电流流向求和点；而当控制某一位电子开关的二进制数（$d_0 \sim d_{n-1}$）为 0 时，电子开关保持接地或从闭合转到接地，该位无权电流流向求和点。由此电阻网络把参考电压转换成相应的电流，并将其求和后经放大器放大输出。

每位的权由电流表示，而权电流由权电阻限制其大小，若最高位权电阻为 R，次高位为 2R，依次为 4R、8R、……、第 n 位为 $2^{n-1}R$，这样权电流将分别是 V/R，V/2R、V/（4R）、…。当 N 位二进制数控制相应的模拟开关接向 V_{REF} 或地时，则总输出电流是：

$$I_{OUT}=V_{REF}\sum_{i=1}^{n}\frac{D_i 2^i}{2^{i-1}R}$$

其中 V_{REF} 为参考电压，$D_i 2^i$ 为第 i 位二进制数。

通过以上分析可知，由二进制加权电阻网络可以进行 D/A 转换，把二进制数表示的数字量转换成模拟量的电流输出。

但是由二进制加权电阻网络进行的 D/A 转换，如果二进制位数超过 8 位，则将造成加权电阻阻值差别很大，例如取 R=10kΩ，则第 8 位权电阻 $2^7R=2^7\times10k=1.28M\Omega$，如果选参考电压 V_{REF}=10V，则该位二进制数为 1 时权电流为 10V/1.28MΩ=7.8μA，这样小的电流变化，通常早已被噪声所淹没。若选的 R 很小，则将导致总电流太大，转换芯片的总体功耗过大，因此对于位数较多的 D/A 转换，这种方法已不适用，解决的方法是采用梯形电阻网络。

2. 梯形电阻网络

这种转换的原理如图 14-3 所示。该电阻网络中，仅有 R 和 2R 两种电阻，电子开关工作原理与二进制加权电阻网络 D/A 转换工作原理相同。D/A 转换的输出总电流为：

$$I_{OUT}=-B\times\frac{V_{REF}}{3R\times2^n}$$

其中 B 为：$B=b_{n-1}2^{n-1}+b_{n-2}2^{n-2}+\cdots+b_0 2^0$

$b_0\sim b_{n-1}$ 为 N 位二进制数。

以电压形式输出（设 R_f=3R）则：

$$V_{OUT}=-B\times\frac{V_{REF}}{2^n}$$

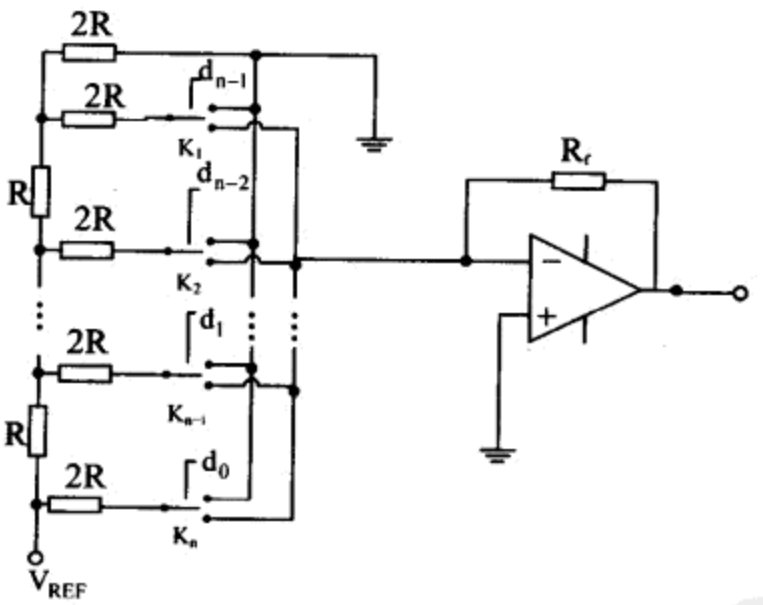


图 14-3　梯形电阻网络 D/A 转换器的结构原理

14.2.2　D/A 转换器的主要参数

D/A 的参数是描述 D/A 转换器的性能指标，主要参数介绍如下。

（1）精度：指 D/A 转换器的实际输出与理论满刻度输出之间的差异，是由 D/A 转换器的增益误差、失调误差（零点误差）、线性误差和噪声等综合引起的。精度反映了 D/A 转换的总误差。

（2）分辨率：指当输入数字量发生单位数据变化时所对应的输出模拟量的变化量。在实际应用中，分辨率通常用二进制位数来表示，例如 8 位的 DAC 能给出满量程电压的 $1/2^8$ 的分辨能力。

（3）建立时间：指当 D/A 转换器的输入数据发生变化后，输出模拟量达到稳定数值所需要的时间，也称为电流建立时间。

（4）温度系数：环境温度的变化会对 D/A 转换精度产生影响，分别用失调温度系数、增益温度系数和微分非线性温度系数来表示。这些系数的含义是当环境温度变化 1℃时该项误差的相对变化率。

（5）非线性误差：也称为线性度，是实际转换特性曲线与理想转换特性曲线之间的最大偏差。

（6）线性度：D/A 转换的实际转换特性曲线与理想曲线之间的最大偏差。

14.2.3 8 位 D/A 转换器 DAC0832 及应用

典型的 D/A 转换器有 8 位 DAC0832，12 位 DAC1208，电压输出型的 AD558 和多路输出型 AD7528 等，下面重点讨论 DAC0832 的结构特点和应用。

DAC0832 是 8 位分辨率的 D/A 转换集成芯片，其明显特点是与微机连接简单、转换控制方便、价格低廉，在微机系统中得到了广泛的应用。D/A 转换器的输出一般都要接运算放大器，微小信号经放大后才能驱动执行机构的部件。

DAC0832 的主要技术指标有：分辨率为 8 位；转换速度约为 1μs；非线性误差为 0.20%FS（FS 是 Full Scale 的缩写）；温度系数为 2×10^{-6}/℃；工作方式为双缓冲、单缓冲和直通方式；逻辑输入与 TTL 电平兼容；功耗为 20mW；单电源供电。

1. 内部结构及引脚

DAC0832 的内部结构及引脚如图 14-4 所示。

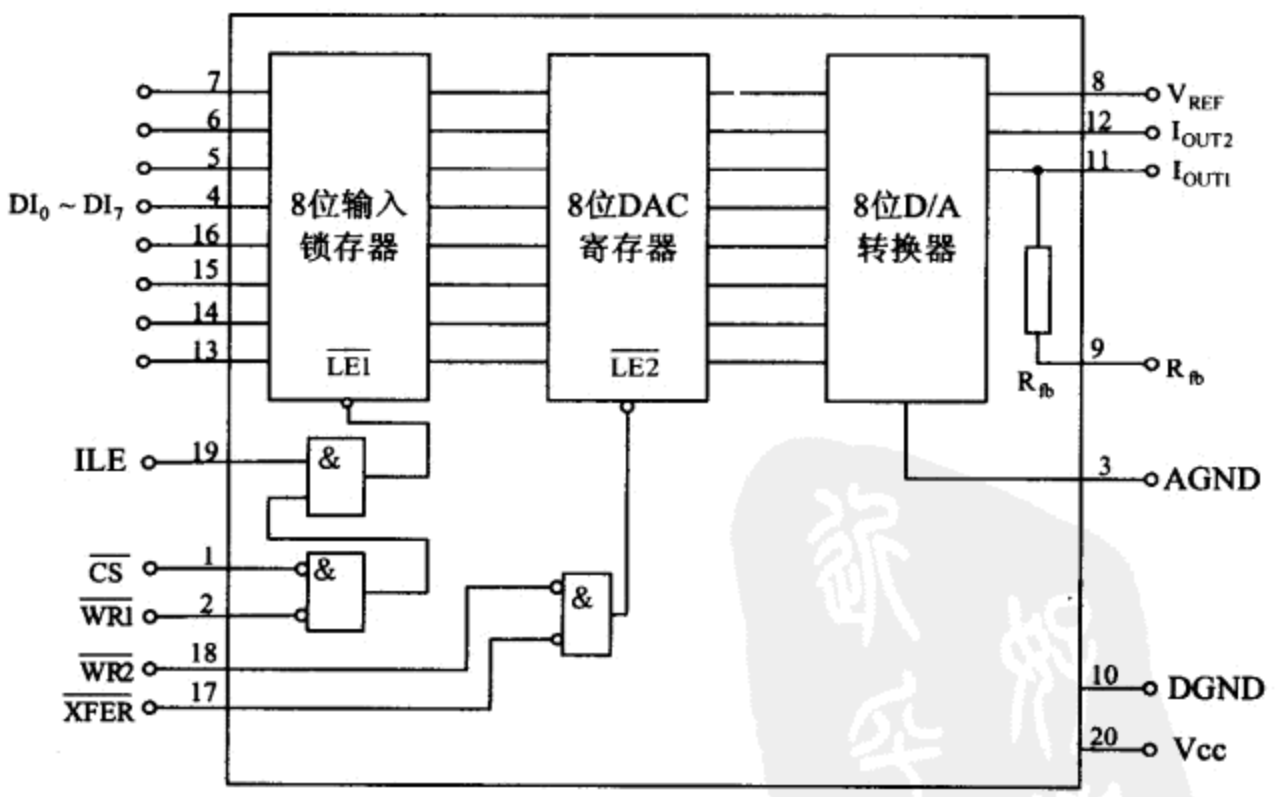


图 14-4 DAC0832 内部结构

从结构图中可看出，该芯片有两级 8 位缓冲寄存器，它的 D/A 转换器采用梯形电阻网络。

2. 引脚功能分析

DAC0832 的外部引脚如图 14-5 所示。

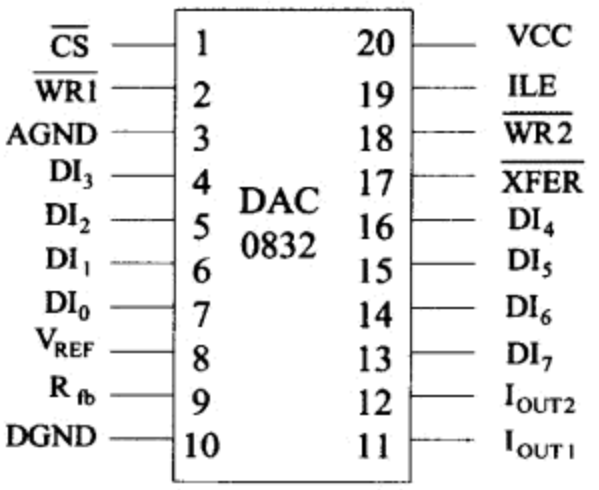


图 14-5　DAC0832 引脚

$DI_0 \sim DI_7$：数据输入端。

$\overline{CS}$：片选信号输入端，低电平有效。

ILE：允许输入锁存信号，高电平有效。

$\overline{WR1}$、$\overline{WR2}$：写信号。

$\overline{XFER}$：传送控制信号，用来控制 $\overline{WR2}$ 信号，低电平有效。

I_{OUT1}：DAC 电流输出 1 端，此电流输出端为是 1 的各位权电流汇集输出端。当 DAC 寄存器全为 1 时，此电流最大，当 DAC 寄存器全为 0 时，此电流为 0。

I_{OUT2}：DAC 电流输出 2 端，此电流输出端是 0 的各位权电流汇集输出端，当 DAC 寄存器各位全为 0 时，此电流为最大，反之等于 0。

R_{fb}：反馈电阻。

V_{REF}：参考电压输入端，可在-10V～+10V 范围内选择。

AGND、DGND：模拟地和数字地，为了防止串扰，系统的模拟地应共接于一点，系统数字地汇总于一点，然后两地再共接于一点。

VCC：电源，可在+5V～+15V 间选择。

3. DAC0832 工作方式

通过控制端 $\overline{CS}$、$\overline{ILE}$、$\overline{WR1}$、$\overline{WR2}$、$\overline{XFER}$ 的不同接线方式可以控制 DAC0832 工作在不同方式下。

（1）输出直通方式：$\overline{WR2}$、$\overline{XFER}$、$\overline{CS}$、$\overline{WR1}$ 接地，ILE 接高电平，输入数据就直接送 D/A 转换器进行电流转换。

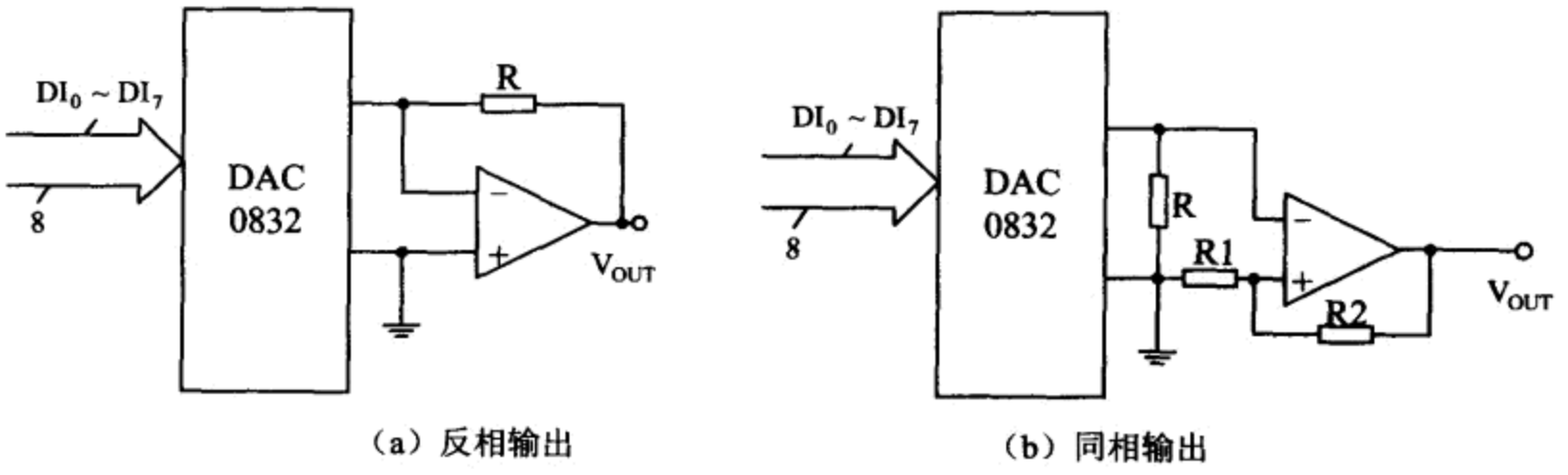
（2）缓冲寄存器工作方式：当 $\overline{WR2}$、$\overline{XFER}$ 接地，则 DAC 寄存器为不锁存状态，ILE 接高电平，$\overline{CS}$、$\overline{WR1}$ 信号有效时，输入数据就直通输入寄存器，当 $\overline{WR1}$ 信号变高电平时，数据就被锁存到输入寄存器。若 $\overline{WR1}$ 接地，ILE 接高电平，$\overline{CS}$ 为恒低时，则输入寄存器为不锁存状态，若 $\overline{XFER}$ 接地，当 $\overline{WR2}$ 信号有效时，输入数据直通到 DAC 寄存器，当 $\overline{WR2}$ 变高电平时，数据就被锁存到 DAC 寄存器。

（3）双缓冲寄存器工作方式：ILE 接高电平，当 $\overline{CS}$ 和 $\overline{WR1}$ 信号有效时，此时若 $\overline{WR1}$ 信

号由低电平变高电平，则输入数据被锁存到输入寄存器，$\overline{XFER}$ 接地，$\overline{WR2}$ 信号有效时，输入寄存器的数据直通 DAC 寄存器，当 $\overline{WR2}$ 信号变高电平时，DAC 寄存器就将直通数据进行锁存。这种工作方式，可提高采集数据速度，因双缓冲分时锁存，当 DAC 寄存器锁存的数据被转换时，输入寄存器可锁存下次待转换的数据，一旦 DAC 寄存器数据被转换结束便可送入下一个数据锁存并转换。

4. DAC0832 的输出

DAC0832 是电流形式输出，当需要电压形式输出时，必须外接运算放大器。如图 14-6 所示。



（a）反相输出　　（b）同相输出

图 14-6　DAC0832 的输出

根据输出电压的极性不同，DAC0832 又可分为单极性输出和双极性输出两种方式。

（1）单极性输出：图 14-7 是 DAC0832 的单极性输出电路。V_{REF} 可以接±5V 或±10V 参考电压，当接+5V（或-5V）时，输出电压范围是-5V～0（或 0～+5V）；当接+10V（或-10V）时，输出电压范围是-10V～0 或（0～+10V）。

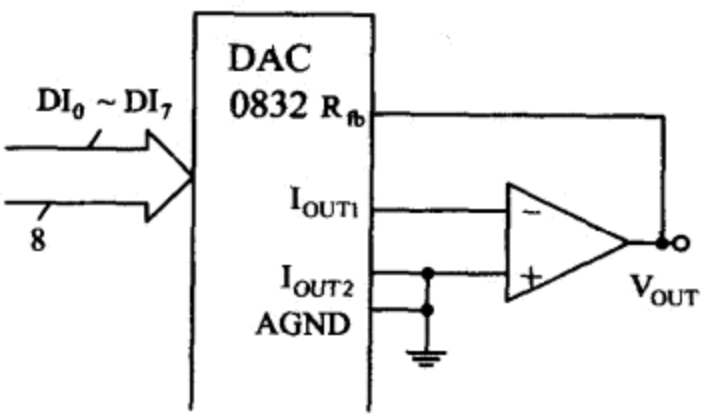


图 14-7　DAC0832 的单极性输出

若输入数字为 0～255，则输出为：

$$V_{OUT} = -\frac{V_{REF} \times B}{256}$$

其中 B 为输入的二进制数，范围 0～255。

（2）双极性输出：图 14-8 是 DAC0832 的双极性输出电路。其输出为：

$$V_{OUT} = -(128 - B) \times \frac{V_{REF}}{256}$$

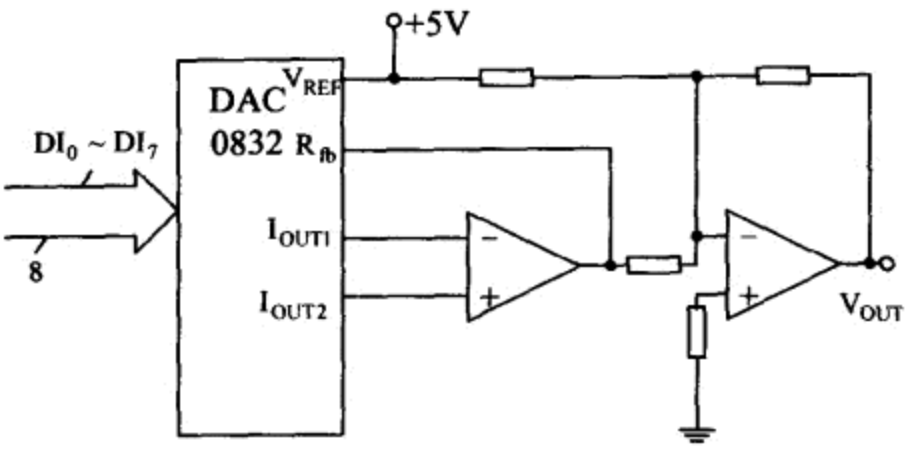


图 14-8　DAC0832 双极性输出

5. DAC0832 与微机接口电路设计

（1）D/A 转换器使用的注意事项

D/A 转换器选择要点：首先考虑 D/A 转换器的分辨率和工作温度范围是否满足系统要求，其次根据 D/A 转换芯片的结构和应用特性选择 D/A 转换器，应使接口方便，外围电路简单。

接口设计：具有三态输入数据寄存器的 D/A 芯片可直接与计算机 I/O 槽上的数据总线相接。但也要为 D/A 转换器分配一个端口地址。

参考电源的配置：若 D/A 芯片无参考电压源则需外接。参考电压必须稳定，温度漂移小。

（2）D/A 转换器的应用

使用 D/A 转换器构造一个波形发生器，如图 14-9 所示。利用 0832 将一组数据转变为 3 种波形（锯齿波、三角波、正弦波）。按 Q 键返回 DOS 操作系统。0832 片选端口地址为 228H～22FH。通过接入双运算放大器 LM358，将 D/A 转换输出的电流转换为电压输出。地址译码输出为 360H，电路输出的各种波形描述如下。

- 矩形波：持续 256 次送 0，然后 256 次送 FFH，如此重复，D/A 输出一个矩形波。
- 梯形波：持续 256 次送 0，然后逐次加 1 直到 255，然后持续 256 次，接着将 255 逐次减 1，如此重复，D/A 输出一个梯形波。
- 三角波：持续 256 次送 0，然后逐次加 1 直到 255，接着将 255 逐次减 1 到 0，如此重复，D/A 输出一个三角波。

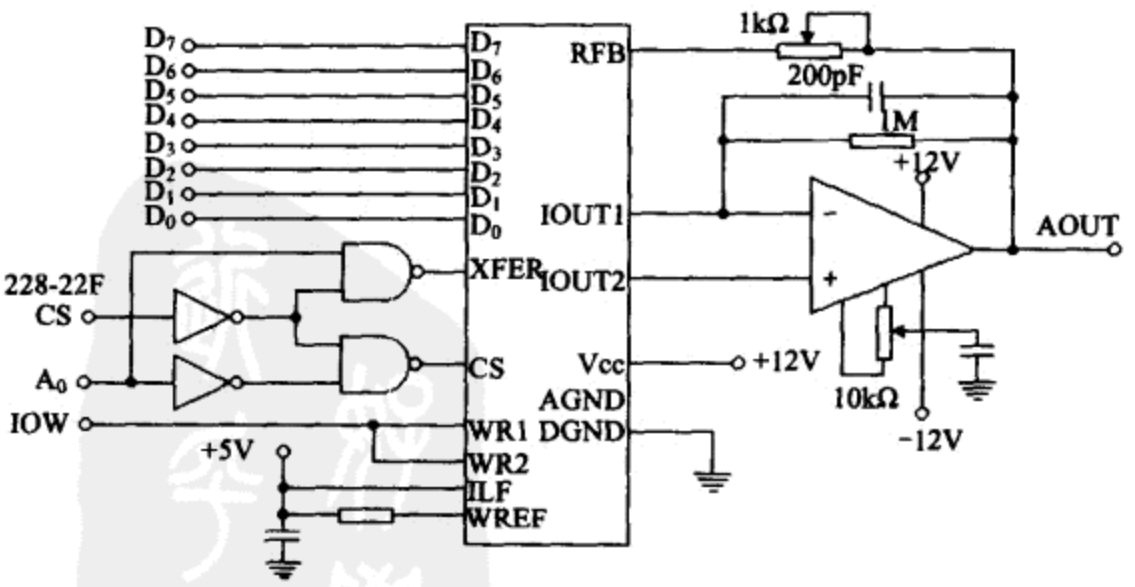


图 14-9　DAC0832 构造的波形发生器

编制程序如下：

```
CRLF    MACRO
        MOV DL,0DH
        MOV AH,02H
        INT 21H
        MOV DL,0AH
        MOV AH,02H
        INT 21H
        ENDM
DATA    SEGMENT
  MESS1     DB 'START D/A--1,END WITH Q!',0DH,0AH,'$'
  MESS2     DB 'START D/A--2,END WITH Q!',0DH,0AH,'$'
  MESS3     DB 'START D/A--3,END WITH Q!',0DH,0AH,'$'
  SINBUF    DB 128,88,53,24,6,0,6,24,53,88,128,168,203,232
            DB 250,255,250,232,203,168
DATA    ENDS
STACK   SEGMENT
        STA DB 20 DUP(?)
        TOP EQU LENGTH STA
STACK   ENDS
CODE    SEGMENT
ASSUME  CS:CODE,DS:DATA,SS:STACK,ES:DATA
START:  MOV AX,DATA
MOV DS,AX
MOV AX,STACK
MOV SS,AX
MOV AX,TOP
    MOV SP,AX
    MOV DX,OFFSET MESS1
    MOV AH,09H
    INT 21H
A1: MOV CX,0FFFFH
    MOV DX,228H
    MOV AL,00H
BBB:OUT DX,AL
    INC DX
    OUT DX,AL
    DEC DX
    ADD AL,10H
    CMP AL,00H
    JNZ BBB
    LOOP BBB
    MOV AH,01H
    INT 21H
    CMP AL,'Q'
```

```
        JNZ A1
        CRLF
        MOV DX,OFFSET MESS2
        MOV AH,09H
        INT 21H
DDD:    MOV CX,0FFFFH
        MOV DX,228H
        MOV AL,00H
EEE:    OUT DX,AL
        INC DX
        OUT DX,AL
        DEC DX
        ADD AL,10H
        CMP AL,0F0H
        JNZ EEE
FFF:    OUT DX,AL
        INC DX
        OUT DX,AL
        DEC DX
        SUB AL,10H
        CMP AL,00H
        JNZ FFF
        LOOP EEE
        MOV AH,01H
        INT 21H
        CMP AL,'Q'
        JNZ DDD
        CRLF
        MOV DX,OFFSET MESS3
        MOV AH,09H
        INT 21H
GGG:    MOV CX,0FFFFH
        MOV DX,228H
HHH:    MOV SI,OFFSET SINBUF
        MOV BL,20
III:    MOV AL,[SI]
        OUT DX,AL
        INC DX
        OUT DX,AL
        DEC DX
        INC SI
        DEC BL
        JNZ III
        LOOP HHH
```

```
                MOV AH,01H
                INT 21H
                CMP AL,'Q'
                JNZ GGG
                MOV AH,4CH
                INT 21
CODE    ENDS
        END START
```

14.3 典型 A/D 转换器芯片

14.3.1 A/D 转换器工作原理

A/D 转换器是模拟信号源与数字设备、数字计算机或其他数据系统之间联系的桥梁，它的任务是将连续变化的模拟信号转换为离散的数字信号以便于数字系统进行处理、存储、控制和显示。在工业控制系统和数据采集及许多领域中，A/D 转换部件常常是不可缺少的重要部件。

A/D 转换器形式很多。按转换器输入模拟量的极性分有单极型和双极型两种。按转换器的数字量输出分，既有并行方式又有串行方式及串/并行方式。按转换器的转换原理分有积分型、逐次逼近型和并行转换型。以下对最常用的逐次逼近型 A/D 转换器进行介绍。

逐次逼近型 A/D 转换器具有较快的转换速度，与微机接口方便是其主要优点。比较适合对快速连续变化的物理量进行跟踪采集与记录等。其电路框图如图 14-10 所示。

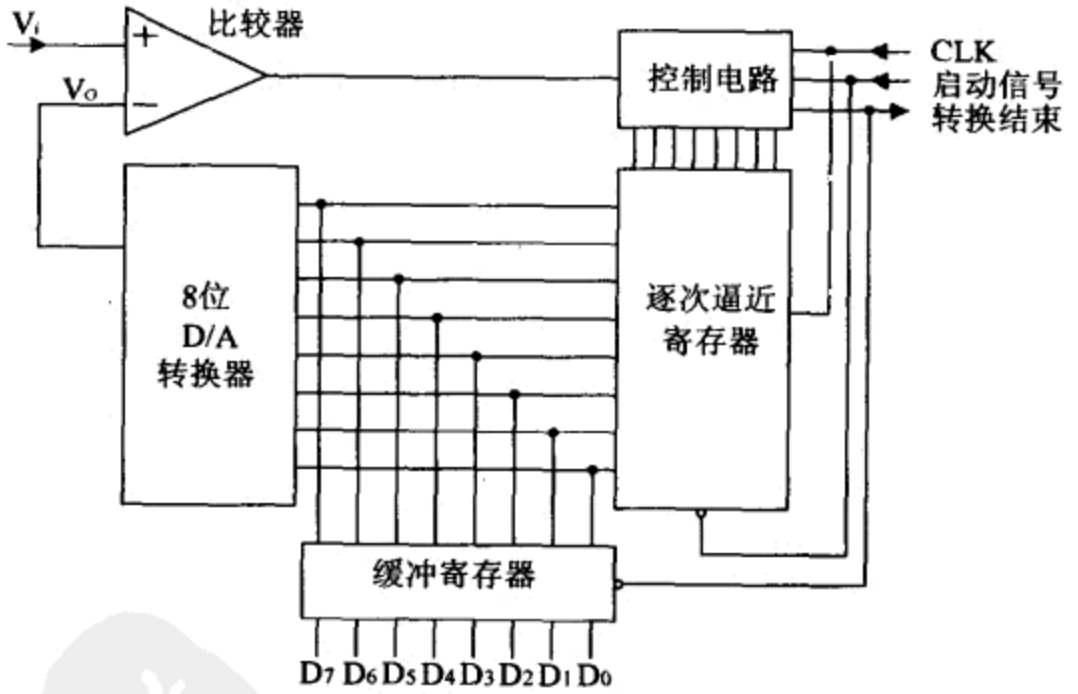


图 14-10 逐次逼近式 A/D 转换器的电路框图

组成部件主要有逐次逼近寄存器 SAR、D/A 转换器、比较器、置数选择逻辑和时钟电路等。首先置数选择逻辑给 SAR 置数，经 D 从转换器转换成模拟量并和输入的模拟信号比较，输入的模拟电压大于或等于 D/A 转换器的输出电压，比较器置 1，否则置 0。置数选择逻辑根据比较器的结果修正 SAR 的数值，使所置数据转换后得到的模拟电压逐渐逼近输入电压，经过 n 次修改后，SAR 中的数字值便是 A/D 转换的结果。

逐次逼近式 A/D 转换速度较快，在 1～100μs 之间，分辨率可达 18 位，转换时间固定，

但抗干扰能力比较差。这类芯片典型的有 ADC0809、ADCl210、AD574 等，是应用得较多的模/数转换器芯片。

14.3.2　ADC 的性能参数

ADC 的性能参数主要包括分辨率、精度、转换时间、温度系数、量程、逻辑电平、工作温度等。

（1）分辨率：反映了 A/D 转换器对输入微小变化的响应能力，通常用数字量最低位所对应的模拟输入电平值表示。

（2）精度：采用绝对误差和相对误差两种方法来表示。绝对误差等于实际转换结果与理论转换结果之差；相对误差是指整个转换范围内任一数字量所对应的模拟输入量实际值与理论值之差，用模拟电压满量程的百分比表示。表示误差的指标还可分为失调误差（零点误差）、增益误差（满量程误差）、非线性误差和微分非线性误差等。

（3）转换时间：指完成一次 A/D 转换所需要的时间。

（4）温度系数：是表示 A/D 转换器受环境温度影响的程度，采用环境温度变化 1℃所产生的相对转换误差来表示。

（5）量程：指所能转换的模拟输入电压范围，分单极性、双极性两种类型。单极性常见量程为 0～+5V，0～10V，0～20V；双极性量程通常为-5V～+5V，-10V～+10V。

（6）逻辑电平及方式：多数 A/D 转换器输出的数字信号与 TTL 电平兼容，以并行方式输出。

（7）工作温度范围：由于温度会对比较器、运算放大器、电阻网络等产生影响，A/D 转换器的工作温度范围一般为 0～70℃。

14.3.3　典型 A/D 转换器件 ADC0809

逐次逼近式 A/D 转换器 ADC0809 片内有 8 路模拟开关，可以同时连接 8 路模拟量，单极性，量程为 0～5V，片内有三态输出缓冲器，可以直接与微机总线相连接。该芯片有较高的性能价格比，适用于对精度和采样速度要求不高的场合或一般的工业控制领域。由于其价格低廉，便于与微机连接，因而应用十分广泛。

1. ADC0809 的工作原理

ADC0809 采用单一的+5V 电源供电，外接工作时钟。当时钟为 500kHz 时，转换时间大约为 128ms，工作时钟为 640kHz 时，转换时间大约为 100ms。允许模拟输入为单极性，无需零点和满刻度调节。内部有 8 个锁存器控制的模拟开关，可以编程选择 8 个通道中的一个。

ADC0809 没有片选引脚，需要外接逻辑门电路对 ADC0809 进行读/写信号的控制并与端口地址组合起来实现编址。

2. ADC0809 的主要技术指标

ADC0809 的主要技术指标有：分辨率为 8 位；非调整误差为±1 LSB；工作温度范围为-40～+85℃；低功耗电量为 20mW；单电源+5 V 供电；转换速度约 1μs，转换时间为 100μs（时钟频率为 640Hz）；具有锁存控制功能的 8 路模拟开关，能对 8 路模拟电压信号进行转换；输出电平与 TTL 电平兼容。

3. ADC0809 的结构及引脚说明

ADC0809 是 8 位模数转换芯片，其内部结构框图如图 14-11 所示，主要包括 8 位逐次比较 A/D 转换器，8 路模拟开关及地址锁存与译码。

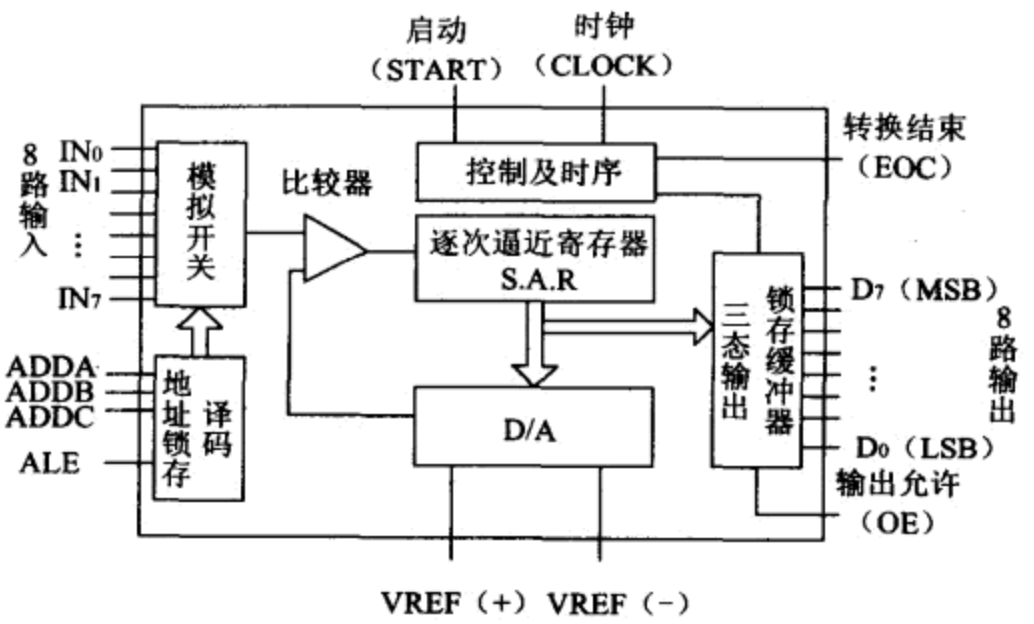


图 14-11　ADC0809 内部结构图

ADC0809 的引脚见图 14-12 所示。各信号功能分析如下：

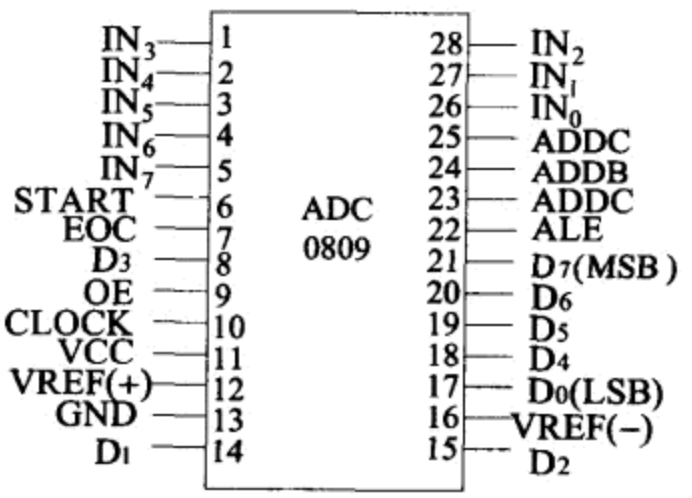


图 14-12　ADC0809 引脚图

IN_0～IN_7：8 路模拟输入，可通过 ADDA、ADDB、ADDC 三个地址译码来选通一路。

D_0～D_7：8 位数据输出端，为三态输出，可直接和微机数据线连接。

ADDA、ADDB、ADDC：模拟通道选择地址信号如表 14-1 所示。

ADDA 为低位、ADDC 为高位，如 000 则选 0 通道，001 则选 1 通道，依此类推。

VREF、+VREF：正、负参考电压输入端。

表 14-1　模拟通道选择地址

地　址			选择通道
ADDC	ADDB	ADDA	
0	0	0	IN_0
0	0	1	IN_1

续上表

地　址			选择通道
ADDC	ADDB	ADDA	
0	1	0	IN_2
0	1	1	IN_3
1	0	0	IN_4
1	0	1	IN_5
1	1	0	IN_6
1	1	1	IN_7

CLOCK：时钟输入信号，最大 640kHz。

START：A/D 转换启动信号，最小宽度为允许地址锁存信号。

ALE ：允许地址锁存信号，当此信号有效时，送入的通道选择地址便被锁存，使用时该信号可以和 START 信号连在一起，以便同时锁存通道地址并 A/D 采样开始转换。

EOC：A/D 转换结束信号，此信号常用来作为中断请求信号。

OE：允许输出信号，送出此信号时 0809 三态门与微机数据总线接通，可把转换的数据取走，若在中断方式下工作，则此信号往往是微机发出的中断请求的响应信号。

4. 工作时序

ADC0809 工作时序如图 14-13 所示。

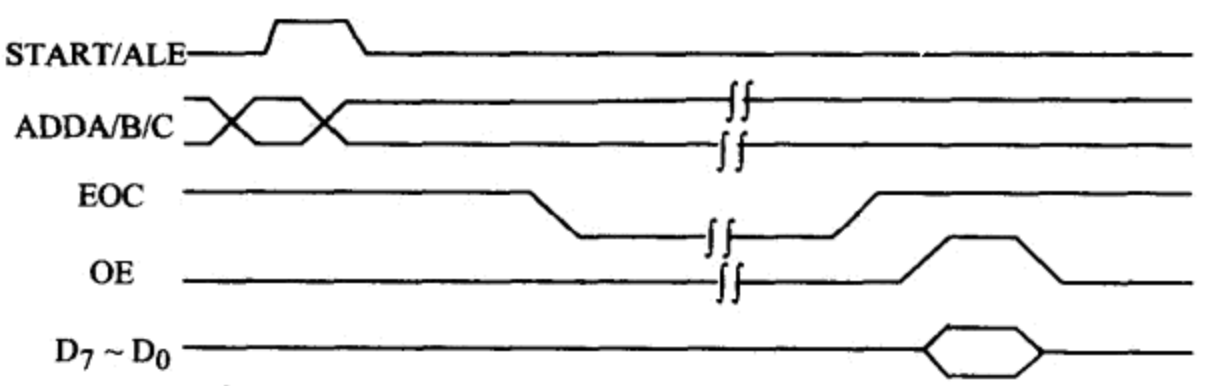


图 14-13　ADC0809 工作时序

当通道选择地址 ADDA、ADDB、ADDC 到来时，ALE 信号有效，地址便立刻被锁存，A/D 转换器将选通的输入信号进行转换，当转换结束时，ADC0809 送出 EOC 信号，微处理器接到此信号时，便立即送出允许输出信号 OE，此时 AD0809 将转换数据通过内部三态门，送到微机数据总线，由微处理机取走。

ALE 信号宽度不小于 100ns，启动信号宽度不小于 100ns，地址保持时间不应小于 25ns。

5. ADC0809 与微机接口

ADC0809 的数据输出端为 8 位三态输出，故数据线可直接与微机的数据线相接。但因为无片选信号线，因而需相关的逻辑电路相匹配。

（1）ADC0809 与 CPU 的直接连接：如图 14-14 所示。

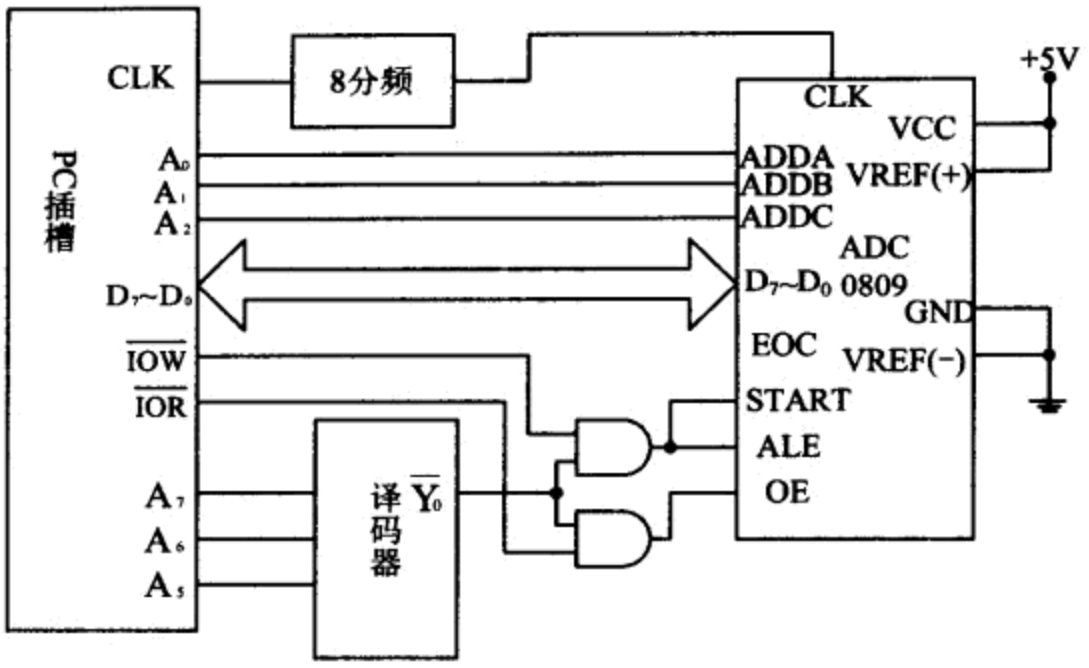


图 14-14　ADC0809 与 CPU 的连接

转换程序如下：

```
MOV  AL,07H
OUT  1FH,AL
CALL DELAY100
IN   AL,1FH
HLT
```

（2）系统通过并口与 ADC0809 的连接：系统可对 8 路模拟量分时进行数据采集，转换结果采用查询方式传送，除了一个传送转换结果的输入端口外，还需要传送 8 个模拟量的选择信号和 A/D 转换的状态信息。因此，可以采用 8255 作为 ADC0809 和 CPU 的连接接口，将 A 口设为方式 0 的输入方式，B 口的 PB_5～PB_7 输出选择 8 路模拟量的地址选通信号，PC_7 输出 ADC0809 的控制信号，PB_0 作为启动信号。硬件连接如图 14-15 所示。

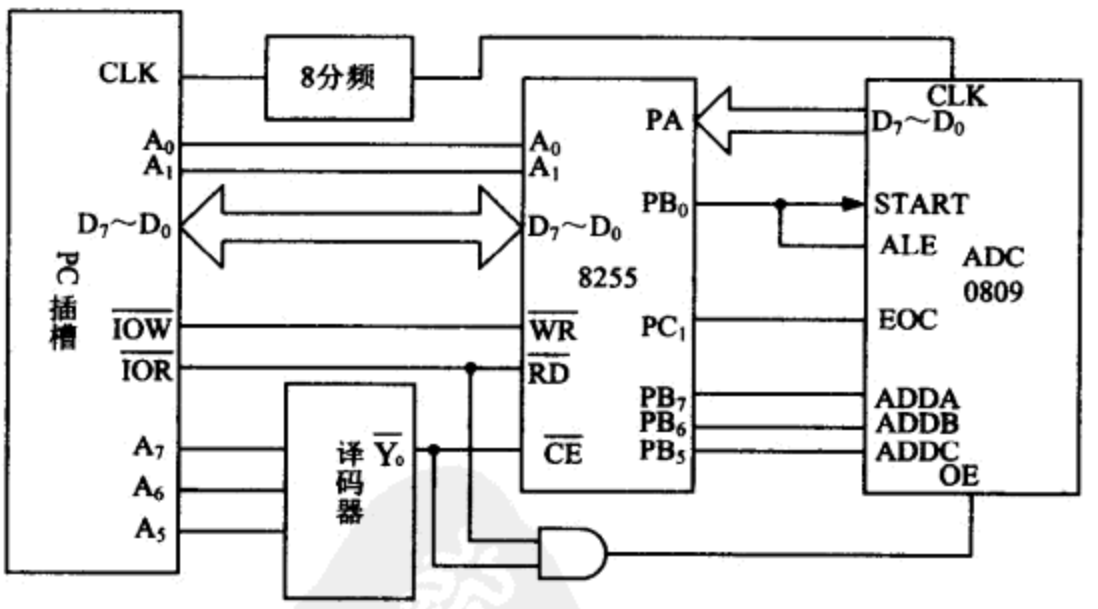


图 14-15　ADC0809 与系统连接图

由于 ADC0809 需要脉冲启动，所以通过软件编程让 PB_0 输出一个正脉冲。EOC 信号直接接 PC_7。8 位数据通过读 8255 芯片 A 口。现假设 8255 的 A 口、B 口、C 口及控制口地址分别为 1CH、1DH、1EH 和 1FH，A/D 转换结果的存储区首地址设为 40H，采样顺序从 IN_0 到 IN_7。译码器输出 Y_0 选通 8255，Y_1 输出选通 ADC0809，ADC0809 的输出为 3FH。

程序设计如下：

```
        MOV     DX,1FH          ;8255 初始化
        MOV     AL,99H
        OUT     DX,AL
        MOV     SI,40H          ;数据存储地址
        MOV     CX,08H
        MOV     BH,00H
LOOP1:  MOV     BH,08H
        MOV     AL,01H
        MOV     DX,1DH          ;启动 0809,送脉冲
        OUT     DX,AL
        MOV     AL,00H
        OUT     DX,AL
LOOP2:  IN      AL,DX
        TEST        AL,1EH      ;查寻 EOC 信号
        JZ      LOOP2
        MOV     DX,1CH          ;取数据
        MOV     AL,DX
        MOV     [SI],AL
        INC     SI              ;存储单元加 1
        INC     BH              ;循环寄存器加 1
        LOOP    LOOP1           ;8 路转换结束,没有结束循环
        HLT
```

（3）编制程序，按中断方式采样 A/D 转换数据，结果送内存 6000H 段，采样点为 300 个。将 A/D 转换结果在计算机屏幕上动态显示出来，设置计算机显示方式为 640×200 图形方式。0809 片选端口地址为 220H～227H。

其硬件电路如图 14-16 所示。

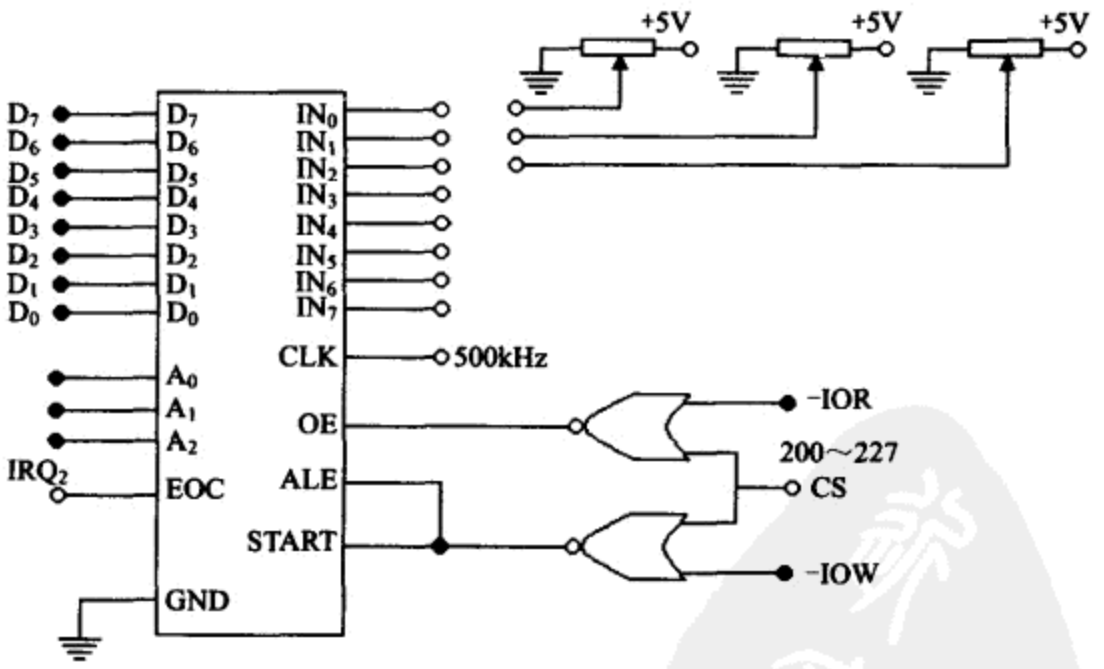


图 14-16　A/D 转换器 0809 硬件电路连接图

分频电路的控制端 A 接 8MHz 脉冲输出，B 端接+5V，分频电路 Q3（500kHz）端接 A/D 转换器的 500kHz。模拟量输入端 IN_0 接 2.2k 电位器中心抽头，电位器两端分别接+5V 和地。

0809 的 CS 端接译码电路的 200H～227H 端口地址。0809 的 EOC 端接总线的 IRQ2。

程序设计如下：

```
RLF MACRO
        MOV DL,0DH
        MOV AH,02H
        INT 21H
        MOV DL,0AH
        MOV AH,02H
        INT 21H
        ENDM
DATA    SEGMENT
        INR DB ?
        RESULT  DB ?
DATA    ENDS
STACK SEGMENT
  STA   DB 20 DUP(?)
  TOP   EQU LENGTH STA
STACK   ENDS
CODE    SEGMENT
  ASSUME CS:CODE,DS:DATA,SS:STACK,ES:DATA
START:  MOV AX,DATA
        MOV DS,AX
        MOV AX,STACK
        MOV SS,AX
        MOV AX,TOP
        MOV SP,AX
        MOV AL,0AH
        MOV AH,35H
        INT 21H
        PUSH ES
        PUSH BX
        PUSH DS
        MOV AX,SEG ADINT
        MOV DS,AX
        MOV DX,OFFSET ADINT
        MOV AL,0AH
        MOV AH,25H
        INT 21H
        POP DS
        IN  AL,21H
        MOV BP,AX
        AND AL,11111011B
        OUT 21H,AL
        MOV CX,300
CCC:    STI
        MOV DX,220H
```

```
        OUT DX,AL
        HLT
        CLI
        MOV AX,SI
        PUSH DS
        MOV BX,6000H
        MOV DS,BX
        MOV BX,CX
        DEC BX
        MOV [BX],AL
        AND AL,0F0H
        PUSH CX
        MOV CL,04H
        SHR AL,CL
        POP CX
        ADD AL,30H
        CMP AL,39H
        JBE AS1
        ADD AL,07H
AS1:    MOV DL,AL
        MOV AH,02H
        INT 21H
        MOV AL,[BX]
        AND AL,0FH
        ADD AL,30H
        CMP AL,39H
        JBE AS2
        ADD AL,07H
AS2:    MOV DL,AL
        MOV AH,02H
        INT 21H
        MOV DL,20H
        MOV AH,02H
        INT 21H
        POP DS
        LOOP CCC
        POP DX
        POP DS
        MOV AL,0AH
        MOV AH,25H
        INT 21H
        MOV AX,BP
        OUT 21H,AL
```

```
        MOV AH,4CH
        INT 21H
ADINT   PROC NEAR
        USH AX
        PUSH DX
        MOV DX,220H
        IN AL,DX
        MOV SI,AX
        MOV AL,20H
        OUT 20H,AL
        POP DX
        POP AX
        IRET
ADINT   ENDP
CODE    ENDS
        END START
```

14.3.4 A/D 转换器的选择原则

A/D 转换器的选择原则如下。

（1）根据检测通道的总误差和分辨率要求，选取 A/D 转换精度和分辨率。一些大型测控系统中不太计较成本，应选 A/D 的精度和分辨率比系统要求的精度和分辨率高一个数量级，这样系统的误差指标可全部分配给传感器和其他环节，以降低系统的设计或研制难度。在小的智能仪器中，则对系统各环节进行合理地精度分配，根据分配给 A/D 部分的指标进行选取，原则上满足系统要求即可。

（2）根据被测对象的变化率及转换精度要求确定 A/D 转换器的转换速率。为保证实时性和采样信号不失真的要求，A/D 转换时间必须满足由采样定理所确定的时间。采样定理指出：当采样器的采样频率高于或至少等于输入信号频谱中的最高频率的两倍时，信号才不会失真。因此，需估算出输入信号的最高频率，选 A/D 转换速率大于两倍的信号的最高频率。

（3）根据环境条件选择 A/D 芯片的环境参数。如工作温度、功耗等。

（4）根据接口设计是否简便及价格等选取 A/D 芯片。现在 A/D 芯片种类繁多，有些内部含基准电源，有些则需外配；有些是 TTL 电平锁存或三态输出。在价格差别不大的情况下，应选取既能保证性能，接口设计又最简单的芯片。

14.4 DAC 及 ADC 应用实例

A/D 转换接口是数据采集系统前向通道中的一个环节，D/A 转换接口是数据采集系统后向通道中的一个环节。

实例分析：用 ADC0809 及 DAC0832 构建一个通用 8 位 A/D 采集，一个 D/A 输出的采集卡。

1. 系统电路原理图

根据要求画系统电路原理图，如图 14-17 所示。

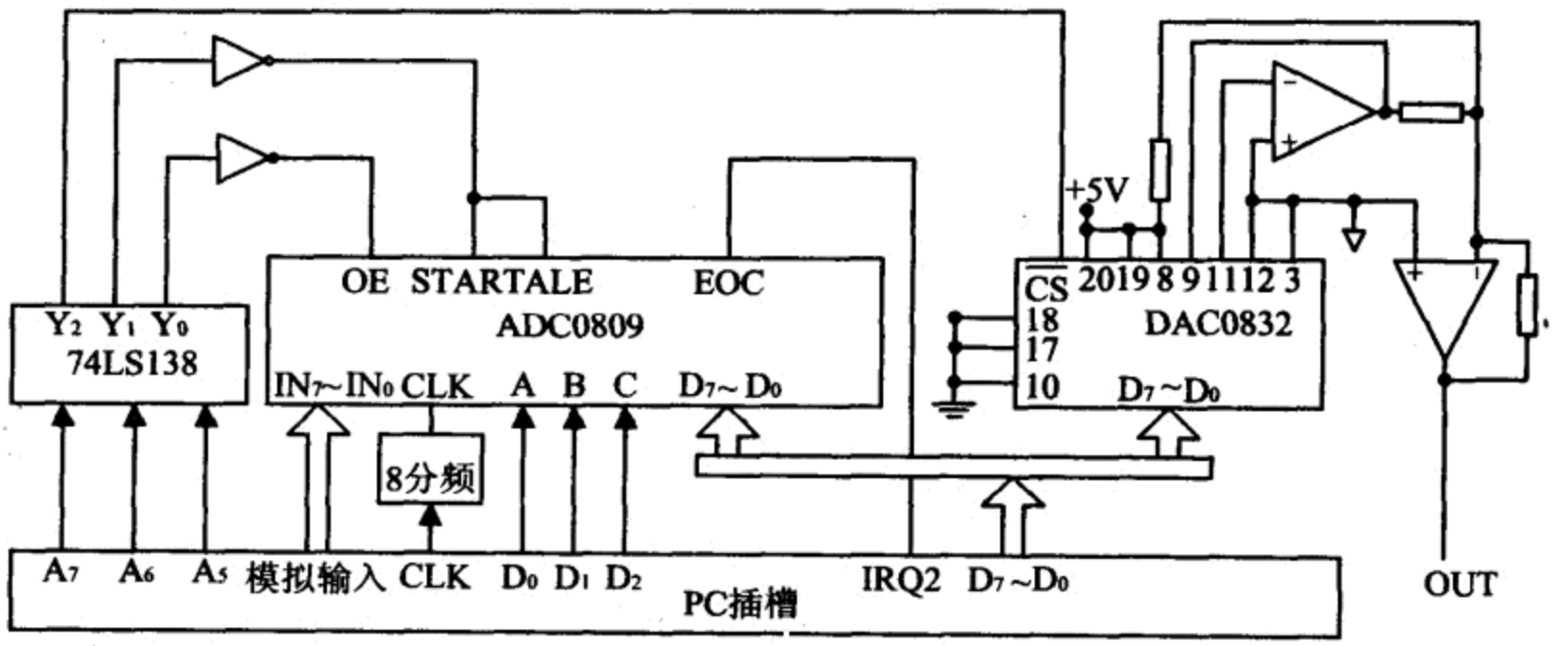


图 14-17　A/D、D/A 应用

2. 相关的地址分配

ADC0809 输出允许：1FH；ADC0809 启动：3FH；DAC0832 使能：5FH；DAC0832 输出为单极性输出，可根据要求变换为双极性输出。

编制程序如下。

A/D 采样程序：

```
        MOV  SI,40H
        MOV  CX,08H
LP:     MOV  BH,0
        MOV  AL,BH
        MOB  DX,3FH
        OUT  DX,AL              ;启动 0809,送脉冲
        MOV  AL,0FFH
        OUT  DX,AL
        CALL DELAY100           ;延时 100us
        MOV  AL,3FH
        MOV  [SI],AL
        INC  SI
        INC  BH
        LOOP LP
        HLT
```

D/A 转换程序：

```
        MOV  SI,80H             ;转换数据存储区地址送 SI
        MOV  DX,5FH             ;D/A 转换地址送 DX
        MOV  AL,[SI]
        OUT  DX,AL              ;D/A 转换
```

本章小结

本章介绍了 A/D、D/A 转换器的工作原理和典型的 A/D、D/A 转换器芯片。DAC0832 是 8 位 D/A 转换器，从输出信号来说，D/A 转换器的直接输出是电流量，若片内有输出放大器，则能输出电压量，并能实现单极性或双极性电压输出。D/A 转换器的转换速度较快，一般其电流建立时间为 1μs。ADC0809 是逐次逼近型 8 位 A/D 转换芯片，片内有 8 路模拟开关，可以同时连接 8 路模拟量，单极性，量程为 0～5V，典型的转换速度为 100μs，片内有三态输出缓冲器，可直接与 CPU 总线连接。该芯片有较高的性能价格比，适用于对精度和采样速度要求不高的场合或一般的工业控制领域。

在进行系统应用时，应正确对待转换器性能与价格的关系。在构建一个测控系统时，要正确选用合适的 A/D、D/A 转换器件。

思考与练习题

1. D/A 转换器由哪几部分组成，如何选择 D/A、A/D 转换器？
2. 用 D/A 转换器 0832 实现一阶梯波的产生？
3. 高于 8 位的 D/A 转换器如何与微机接口（举例说明）。
4. 编写 8 通道 A/D 转换器 0809 的测试程序。
5. 画出 ADC0809 直接与 CPU 扩展槽连接图并编写采样程序。

附录 A 8086 指令系统

表 A-1 指令符号说明

符　号	说　明
r8	任意一个 8 位通用寄存器 AH、AL、BH、BL、CH、CL、DH、DL
r16	任意一个 16 位通用寄存器 AX、BX、CX、DX、SI、DI、BP、SP
reg	代表 r8、r16
seg	段寄存器 CS、DS、ES、SS
m8	一个 8 位存储器操作数单元
m16	一个 16 位存储器操作数单元
mem	代表 m8、m16
i8	一个 8 位立即数
i16	一个 16 位立即数
imm	代表 i8、i16
dest	目的操作数
src	源操作数
label	标号

表 A-2 指令汇编格式

指令类型	指令汇编格式	指令功能简介
传送指令	MOV reg/mem,imm	dest←src
	MOV reg/mem/seg,reg	
	MOV reg/seg,mem	
	MOV reg/mem,seg	
交换指令	XCHG reg,reg/mem	Reg←→reg/mem
	XCHG reg/mem,reg	
转换指令	XLAT label	AL←[BX+AL]
	XLAT	
堆栈指令	PUSH rl6/m16/seg	寄存器/存储器入栈
	POP rl6/m16/seg	寄存器/存储器出栈
标志传送	CLC	CF←0
	STC	CF←1
	CMC	CF←~CF
	CLD	DF←0
	STD	DF←1
	CLI	IF←0

续上表

指令类型	指令汇编格式	指令功能简介
标志传送	STI	IF←1
	LAHF	AH←FLAG 低字节
	SAHF	FLAG 低字节←AH
	PUSHF	FLAGS 入栈
	POPF	FLAGS 出栈
地址传送	LEA r16,mem	r16←16 位有效地址
	LDS r16,mem	DS:r16←32 位远指针
	LES r16,mem	ES:r16←32 位远指针
输入	IN AL/AX,I8/DX	AL/AX←I/O 端口 I8/DX
输出	OUT I8/DX,AL/AX	I/O 端口 I8/DX←AL/AX
加法运算	ADD reg,imm/reg/mem	dest←dest+src
	ADD mem,imm/reg	
	ADC reg,imm/reg/mem	dest←dest+src+CF
	ADC mem,imm/reg	
	INC reg/mem	reg/mem←reg/mem+1
减法运算	SUB reg,imm/reg/mem	dest←dest−src
	SUB mem,imm/reg	
	SBB reg,imm/reg/mem	dest←dest−src−CF
	SBB mem,imm/reg	
	DEC reg/mem	Reg/mem←reg/mem−1
	NEG reg/mem	Reg/mem←0−reg/mem
	CMP reg,imm/reg/mem	dest−src
	CMP mem,imm/reg	
乘法运算	MUL reg/mem	无符号数值乘法
	IMUL reg/mem	有符号数值乘法
除法运算	DIV reg/mem	无符号数值除法
	IDIV reg/mem	有符号数值除法
符号扩展	CBW	把 AL 符号扩展为 AX
	CWD	把 AX 符号扩展为 DX.AX
十进制调整	DAA	将 AL 中的加和调整为压缩 BCD 码
	DAS	将 AL 中的减差调整为压缩 BCD 码
	AAA	将 AL 中的加和调整为非压缩 BCD
	AAS	将 AL 中的减差调整为非压缩 BCD
	AAM	将 AX 中的乘积调整为非压缩 BCD
	AAD	将 AX 中的非压缩 BCD 码扩展成二进制数

续上表

指令类型	指令汇编格式	指令功能简介
逻辑运算	AND reg,imm/reg/mem	dest←dest AND src
	AND mem,imm/reg	
	OR reg,imm/reg/mem	dest←dest OR src
	OR mem,imm/reg	
	XOR reg,imm/reg/mem	dest←dest XOR src
	XOR mem,imm/reg	
	TEST reg,imm/reg/mem	dest AND src
	TEST mem,imm/reg	
	NOT reg/mem	reg/mem←NOT reg/mem
移位	SAL reg/mem,1/CL	算术左移 1/CL 指定的次数
	SAR reg/mem,1/CL	算术右移 1/CL 指定的次数
	SHL reg/mem,1/CL	与 SAL 相同
	RCR reg/mem,1/CL	带进位循环右移 1/CL 指定的次数
串操作	MOVS[B/W]	串传送
	LODS[B/W]	串读取
	STOS[B/W]	串存储
	CMPS[B/W]	串比较
	SCAS[B/W]	串扫描
	REP	重复前缀
	REPZ/REPE	相等重复前缀
	REPNZ/REPNE	不等重复前缀
控制转移	JMP label	无条件直接转移
	JMP rl6/m16	无条件间接转移
	JCC label	条件转移
循环	LOOP label	CX←CX−1；若 CX≠0，循环
	LOOPZ/LOOPE label	CX←CX−1；若 CX≠0 且 ZF=1，循环
	LOOPNZ/LOOPNE label	CX←CX−1；若 CX≠0 且 ZF=0，循环
	JCXZ label	CX=0，循环
子程序	CALL label	直接调用
	CALL rl6/m16	间接调用
	RET	无参数返回
	RETil6	有参数返回
中断	INTi8	中断调用
	IRET	中断返回
	INTO	溢出中断调用

续上表

指令类型	指令汇编格式	指令功能简介
处理器控制	NOP	空操作指令
	SEG	段超越前缀
	HLT	停机指令
	LOCK	封锁前缀
	WAIT	等待指令
	ESCi8,reg/mem	交给浮点处理器的浮点指令

附录 B DEBUG 调试程序的应用

调试程序 DEBUG 是用于调试汇编语言程序的一个工具软件。可用于建立汇编语言源程序（*.ASM），并对源程序进行汇编；还可用于程序的控制执行，跟踪运行程序的踪迹，了解程序中每条指令的执行结果及每条指令执行完毕后各个寄存器的内容，以便检查和修改可执行程序；也可用于对接口操作和对磁盘进行读/写操作等。

DEBUG 命令是在命令提示符“_”下由键盘键入的。每条命令以单个字母的命令符为开头，然后是命令的操作参数，操作参数与操作参数之间，用空格或逗号隔开，操作参数与命令符之间用空格隔开，命令的结束符是回车键（ENTER）。

DEBUG 的主要命令及功能参如下表所示。

命令名	含　义	使用格式	功　能
D	显示存储单元命令	-D[address]	按指定地址范围显示存储单元内容
		-D[range]	按指定首地址显示存储单元内容
E	修改存储单元内容命令	-E address[list]	用指定内容表替代存储单元内容
		-E address	逐个单元修改存储单元内容
F	填写存储单元内容命令	-F range list	将指定内容填写到存储单元
R	检查和修改寄存器内容命令	-R	显示 CPU 内所有寄存器内容
		-R register name	显示和修改某个寄存器内容
		-RF	显示和修改标志位状态
G	运行命令	-G[=address1][address2]	按指定地址运行
T	跟踪命令	-T[=address]	逐条指令跟踪
		-T[=address][value]	多条指令跟踪
A	汇编命令	-A[address]	按指定地址开始汇编
U	反汇编命令	-U[address]	按指定地址开始反汇编
		-U[range]	按指定范围的存储单元开始反汇编
N	命名命令	-N filespecs [filespecs]	将两个文件标识符格式化
L	装入命令	-L address drive sector sector	装入磁盘上指定内容到存储器
		-L[address]	装入指定文件
W	写命令	-W address drive sector sector	把数据写入磁盘指定的扇区
		-W[address]	把数据写入指定的文件
Q	退出命令	-Q	退出 DEBUG

附录 C | DOS 系统功能调用（INT 21H）

AH	功　能	调用参数	返回参数
00	程序终止（同 INT 21H）	CS=程序段前缀 PSP	
01	键盘输入并回显		AL=输入字符
02	显示输出	DL=输出字符	
03	辅助设备（COM1）输入		AL=输入数据
04	辅助设备（COM1）输出	DL=输出字符	
05	打印机输出	DL=输出字符	
06	直接控制台 I/O	DL=FF（输入）；DL=字符（输出）	AL=输入字符
07	键盘输入（无回显）		AL=输入字符
08	键盘输入（无回显） 检测 Ctrl+Break 或 Ctrl+C		AL=输入字符
09	显示字符串	DS:DX=串地址；以‘$’结尾	
0A	键盘输入到缓冲区	DS:DX=缓冲区首址	
0B	检验键盘状态		AL=00，有输入； AL=FF，无输入
0C	清除缓冲区并请求指定的输入功能	AL=输入功能号（1，6，7，8）	
0D	磁盘复位		清除文件缓冲区
0E	指定当前默认的磁盘驱动器	DL=驱动器号 （0=A，1=B，…）	AL=系统中的驱动器数
0F	打开文件（FCB）	DS:DX=FCB 首地址	AL=00，文件找到； AL=FF，文件未找到
10	关闭文件（FCB）	DS:DX=FCB 首地址	AL=00，目录修改成功； AL=FF，未找到文件
11	查找第一个目录项（FCB）	DS:DX=FCB 首地址	AL=00，找到匹配的目录项； AL=FF，未找到目录项
12	查找下一个目录项（FCB） 使用通配符进行目录项查找	DS:DX=FCB 首地址	AL=00，找到匹配的目录项； AL=FF，未找到匹配的目录项

续上表

AH	功　能	调用参数	返回参数
13	删除文件（FCB）	DS:DX=FCB 首地址	AL=00，删除成功； AL=FF，文件未删除
14	顺序读文件（FCB）	DS:DX=FCB 首地址	AL=00，读成功； AL=01，文件结束 未读到数据； AL=02，DTA 边界 错误； AL=03，文件结束记 录不完整
15	顺序写文件（FCB）	DS:DX=FCB 首地址	AL=00，写成功； AL=01，磁盘满或 只读文件； AL=02，DTA 边界 错误
16	建文件（FCB）	DS:DX=FCB 首地址	AL=00，建立文件 成功； AL=FF，磁盘操作 有错
17	文件改名（FCB）	DS:DX=FCB 首地址	AL=00，文件被改名； AL=FF，文件未改名
19	取当前默认磁盘驱动器 0=A，1=B，2=C，…	AL=00，默认的驱动器号	
1A	设置 DTA 地址	DS:DX=DTA 地址	
1B	取默认驱动器 FAT 信息		AL=每簇的扇区数； DS:BX=指向说明的 指针； CX=物理扇区的 字节数； DX=每个磁盘簇数
1C	取指定驱动器 FAT 信息		同上
1F	取默认磁盘参数块		AL=00 无错；AL=FF 出错； DS:BX=磁盘参数块 地址

续上表

AH	功　能	调用参数	返回参数
21	随机读文件（FCB）	DS:DX=FCB 首地址	AL=00，读成功； AL=01，文件结束； AL=02，DAT 边界错误； AL=03，读部分记录
22	随机写文件（FCB）	DS:DX=FCB 首地址	AL=00，写成功； AL=01，磁盘满或是只读文件； AL=02 DAT 边界错误
23	测文件大小（FCB）	DS:DX=FCB 首地址	AL=00，成功，记录数填入 FCB； AL=FF，未找到匹配的文件
24	设置随机记录号	DS:DX=FCB 首地址	
25	设置中断向量	DS:DX=中断向量； AL=中断类型号	
26	建立程序段前缀 PSP	DX=新 PSP 段地址	
27	随机分块读（FCB）	DS:DX=FCB 首地址； CX=记录数	AL=00，读成功； AL=01，文件结束； AL=02 DTA 边界错误 AL=03 读入部分记录； CX=读取的记录数
28	随机分块写（FCB）	DS:DX=FCB 首地址； CX=记录数	AL=00，写成功； AL=01，磁盘满或只读文件；AL=02DAT 边界错误
29	分析文件名字符串（FCB）	ES:DI=FCB 首地址； DS:SI=ASCIIZ 串	AL=00，标准文件； AL=01，多义文件； AL=02 DAT 边界错
2A	取系统日期		CX=年；DH=月； DL=日；AL=星期
2B	设置系统日期	CX=年（1980~2099） DH=月（1~12）；DL=日（1~31）	AL=00 成功； AL =FF 无效

续上表

AH	功　能	调用参数	返回参数
2C	取系统时间		CH:CL=时：分； DH:DL=秒：1/100秒
2D	设置系统时间	CH:CL=时：分； DH:DL=秒：1/100秒	AL=00成功； AL=FF无效
2E	设置磁盘检验标志	AL=00关闭检验 AL=FF打开检验	
2F	取DAT地址		ES:BX=DAT首地址
30	取DOS版本号		AL=版本号；AH=发行号； BH=DOS版本标志； BL:CX=序号（24位）
31	结束并驻留	AL=返回号；DX=驻留区大小	
32	取驱动器参数块	DL=驱动器号	AL=FF，驱动器无效 DS:BX=驱动器参数块地址
33	Ctrl+Break检测	AL=00取标志状态	DL=00，关闭检测； DL=01，打开检测
35	取中断向量	AL=中断类型号	ES:BX=驱动器参数块地址
36	取空闲磁盘空间	DL=驱动器号 0=默认，1=A，2=B，…	成功：AX=每簇扇区数；BX=可用扇区数；CX=每扇区字节数；DX=磁盘总扇区数
39	建立子目录	DS:DX=ASCII Z串	AX=错误代码
3A	删除子目录	DS:DX=ASCII Z串	AX=错误代码
3B	设置目录	DS:DX=ASCII Z串	AX=错误代码
3C	建立文件	DS:DX=ASCII Z串 CX=文件属性	成功：AX=文件代号； 失败：AX=错误代码
3D	打开文件	DS:DX=ASCII Z串 AL=访问和文件的共享方式 0=读，1=写，2=读/写	成功：AX=文件代号； 失败：AX=错误代码
3E	关闭文件	BX=文件代号	失败：AX=错误代码

续上表

AH	功　能	调用参数	返回参数
3F	读文件或设备	DS:DX=ASCII Z 串 BX=文件代号 CX=读取的字节数	成功：AX=实际读入的字节数； AX=0 已到文件末尾； 失败：AX=错误代码
40	写文件或设备	DS:DX=ASCII Z 串； BX=文件代号；CX=写入的字节数	成功：AX=实际写入的字节数； 失败：AX=错误代码
41	删除文件	DS:DX=ASCII Z 串	成功：AX=00； 失败：AX=错误代码
42	移动文件指针	BX=文件代号；CX:DX=位移量； AL=移动方式	成功：DX:AX=新指针位置； 失败：AX=错误码
43	置/取文件属性	DS:DX=ASCII Z 串地址； AL=00 取属性；AL=01 置属性；CX=属性	成功：CX=文件属性； 失败：AX=错误码
44	设备驱动程序控制	BX=文件代号；AL=设备子功能代码（0~11H）； BL=驱动器代码； CX=读/写的字节数	成功：DX=设备信息； AX=传送的字节数； 失败：AX=错误码
45	复制文件代号	BX=文件代号 1	成功：AX=文件代号 2； 失败：AX：错误码
46	强行复制文件代号	BX=代号 1；CX=代号 2	失败：AX=错误码
47	取当前目录路径名	DL=驱动器号 DS:SI=ASCII Z 串地址	成功：DS：SI=当前 ASCII 码串地址；失败：AX=错误码
48	分配内存空间	BX=申请内存字节数	成功：AX 分配初始段地址； 失败：AX=错误码； BX=最大可用空间
49	释放已分配内存	ES=内存起始段地址	失败：AX=错误码
4A	修改内存分配	ES=原内存起始段地址 BX=新申请内存字节数	失败：AX=错误码； BX=最大可用空间

续上表

AH	功　能	调用参数	返回参数
4B	装入/执行程序	DS:DX=ASCII Z 串地址； ES:BX=参数区首地址； AL=00 装入并执行程序； AL=0l 装入程序但不执行	失败：AX=错误码
4C	带返回码终止	AL=返回码	
4D	取返回代码		AL=子出口代码； AH=返回代码：00=正常终止；01=用Ctrl+C终止；02=严重设备错误终止；03=功能调用31H终止
4E	查找第一个匹配文件	DS:DX=ASCII Z 串地址； CX=属性	失败：AX=错误码
4F	查找下一个匹配文件	DTA 保留 4EH 的原始信息	失败：AX=错误码
50	置 PSP 段地址	BX=新 PSP 段地址	
51	取 PSP 段地址		BX=当前运行进程的 PSP
52	取磁盘参数块		ES: BX=参数块表指针
53	把 BIOS 参数块转换为 DOS 的驱动器参数块	DS:SI=BPB 的指针 ES:BP=DPB 的指针	
54	取写盘后读盘的检验标志		AL=00 检验关闭； AL=01 检验打开
55	建立 PSP	DX=建立 PSP 的段地址	
56	文件改名	DS:DX=当前串地址； ES:DI=新串地址	失败：AX=错误码
57	置/取文件日期和时间	BX=文件代号； AL=00 读取；AL=0l 设置； (DX:CX)=日期：时间	失败：AX=错误码
58	取/置内存分配策略	AL=00 取策略代码； AL=01 设置策略代码； BX=策略代码	成功：AX=策略代码； 失败：AX=错误码
59	取扩充错误码	BX=00	AX=扩充错误码； BH=错误类型；BL=建议的操作；CH=出错设备代码

续上表

AH	功　能	调用参数	返回参数
5A	建立临时文件	CX=文件属性； DS:DX=串地址	成功：AX=文件代号； 失败：AX=错误代码
5B	建立新文件	CX=文件属性； DS:DX=ASCII Z 串地址	成功：AX=文件代号； 失败：AX=错误代码
5C	锁定文件存取	AL=00 锁定指定区域；AL=01 开锁；BX=文件代号； CX:DX=文件区域偏移值； SI:DI=文件区域的长度	失败：AX=错误代码
5D	取/置严重错误标志的地址	AL=06 取严重错误标志地址； AL=0A 置 ERROR 结构指针	DS:SI=严重错误标志的地址
60	扩展为全路径名	DS:SI=ASCII Z 串的地址； ES:DI=工作缓冲区地址	失败：AX=错误代码
62	取程序段前缀地址		BX=PSP 地址
68	刷新缓冲区数据到磁盘	AL=文件代号	失败：AX=错误代码
6C	扩充的文件打开/建立	AL=访问权限；BX=打开方式； CX=文件属性；DS:SI=ASCII Z 串地址	成功：AX=文件代号； CX：采取的动作； 失败：AX=错误代码

附录 D BIOS 功能调用

INT	AH	功　能	调用参数	返回参数
10	0	设置显示方式	如 AL= 11 640×480；黑白图形（VGA）	
			=12 640×480；16 色图形（VGA）	
			=13 320×200；256 色图形（VGA）	
10	1	置光标类型	$(CH)_{0\sim3}$=起始行；$(CL)_{0\sim3}$=结束行	
10	2	置光标位置	BH=页号；DH/DL=行/列	
10	3	读光标位置	BH=页号	CH=起始行；CL=光标行；DH/DL=行/列
10	4	读光笔位置		AX=0，未触发；AX=1，触发；CH/BX=像素行/列；DH/DL=字符行/列
10	5	置当前显示页	AL=页号	
10	6	屏幕初始化或向上滚动	AL=0，初始化窗口； AL=向上滚动行数；BH=卷入行属性； CH/CL=左上角行/列号； DH/DI=右上角行/列	
10	7	屏幕初始化或向下滚动	AL=0，初始化窗口	
			AL=向下滚动行数	
			BH=卷入行属性	
			CH/CL=左上角行/列号	
			DH/DL=右上角行/列	
10	8	读光标位置的字符和属性	BH=显示页	AH/AL=字符/属性
10	9	在光标位置显示字符和属性	BH=显示页；AL/BL=字符/属性 CX=字符重复次数	
10	A	在光标位置显示字符	BH=页；AL=字符；CX=字符重复次数	
10	B	置彩色调色板	BH=彩色调色板 ID BL=和 ID 配套使用的颜色	
10	C	写像素	AL=颜色值；BH=页号； DX/CX=像素行/列	
10	D	读像素	BH=页号；DX/CX=像素行/列	AL=像素的颜色值

续上表

INT	AH	功　能	调用参数	返回参数
10	E	显示字符（光标前移）	AL=字符；BH=页号；BL=前景色	
10	F	取当前显示方式		BH=页号；AH=字符列数；AL=显示方式
10	10	置调色板寄存器	AL=0；　BL=调色板号；　BH=颜色值	
10	11	装入字符发生器（EGA/VGA）	AL=0~4，全部或部分装入字符点阵集；AL=20~24，置图形方式显示字符集；AL=30，读当前字符集信息	ES：BP=字符集位置
10	12	返回当前适配器设置的信息（EGA/VGA）	BL=10H（子功能）	BH=0 单色方式；BH=1 彩色方式；BL=VRAM 容量；CH=特征位设置；CL=EGA 开关设置
10	13	显示字符串	ES：BP=字符串地址；AL=写方式（0~3）CX=字符串长度；DH/DI=起始行/列 BH/DI/=页号/属性	
11		取设备信息		AX=返回值；0=设备未安装；1=设备已安装
12		取内存容量		AX=字节数（KB）
13	0	磁盘复位	DL=驱动器号（00，01 为软盘，80h、…为硬盘）	失败：AH=错误码
13	1	读磁盘驱动器状态		AH=状态字节
13	2	读磁盘扇区	AL=扇区数；$(CL)_{6\sim7}(CH)_{0\sim7}$ =磁道号；$(CL)_{0\sim7}$=扇区号；DH/DL=磁头号/驱动器号；ES：BX=数据缓冲区地址	读成功：AH=0；AL=读取的扇区数；读失败：AH=错误码
13	3	写磁盘扇区	同上	写成功：AH=0；AL=写入的扇区数；写失败：AH=错误码
13	4	检验磁盘扇区	AL=扇区数；$(CL)_{6\sim7}(CH)_{0\sim7}$ =磁道号；$(CL)_{0\sim5}$ =扇区号；DH/DI=磁头号/驱动器号	成功：AH=0；AL=检验的扇区数；失败：AH=错误码
13	5	格式化盘磁道	AL=扇区数；$(CL)_{6\sim7}(CH)_{0\sim7}$ =磁道号；$(CL)_{0\sim5}$=扇区号；DH/DL=磁头号/驱动器号；ES：BX=格式化参数表指针	成功：AH=0；失败：AH=错误码
14	0	初始化串行口	AL=初始化参数；DX=串行口号	AH=通信口状态；AL=调制解调器状态

续上表

INT	AH	功　能	调用参数	返回参数
14	1	向通信口写字符	AL=字符； DX=通信口号	写成功：$(AH)_7$=0； 写失败：$(AH)_7$=1； $(AH)_{0\sim6}$=通信口状态
14	2	从通信口读字符	DX=通信口号	读成功：$(AH)_7$=0， (AL)=字符； 读失败：$(AH)_7$=1
14	3	取通信口状态	DX=通信口号	AH=通信口状态； AL=调制解调器状态
14	4	初始化扩展 COM		
14	5	扩展 COM 控制		
15	0	启动盒式磁带机		
15	1	停止盒式磁带机		
15	2	磁带分块读	ES:BX=数据传输区地址； CX=字节数	AH=状态字节；AH=00 读成功；AH=01 冗余检验错；AH=02 无数据传输；AH=04 无引导；AH=80 非法命令
15	3	磁带分块读	DS:BX=数据传输区地址；CX=字节数	AH=状态字节（同上）
16	0	从键盘读字符		AL=字符码；AH=扫描码
16	1	取键盘缓冲状态		ZF=0，AL=字符码； AH=扫描码； ZF=l，无按键等待
16	2	取键盘标志字节	AL=键盘标志字节	
17	0	打印字符回送状态字节	AL=字符	AH=打印机状态字节； DX=打印机号
17	1	初始化打印机回送状态字节	DX=打印机号	AH=打印机状态字节
17	2	取打印机状态	DX=打印机号	AH=打印机状态字节
18		ROMBASIC 语言		
19		引导装入程序		
1A	0	读时钟		CH:CL=时:分； DH:DL=秒:1/100
1A	1	置时钟	CH:CL=时:分； DH:DL=秒:l/100 秒	